

The Thiele Machine

A Computational Model with Explicit Structural Cost

Devon Thiele

Contents

I	The picture	18
1	How I got here	18
2	What structure is	22
2.1	A string, a square, and a drowning that didn't happen	22
2.2	Shapes poured into nothing	24
2.3	Prove it from what you and I both accept	25
2.4	The statue and the bronze	26
2.5	Where things are, without numbers	26
2.6	Naming a thing by saying when two are the same	26
2.7	Choose the changes, keep what survives	27
2.8	An invariance comes with a bill	28
2.9	All of mathematics as structures	28
2.10	A set with some named relations	29
2.11	The structure is in the arrows	29
2.12	Structure is the part that compresses	29
2.13	The four parts of every computer	32
2.14	Laying them all down	33
2.15	The smallest piece	35
2.16	Buckets	37
II	The axiom	39
3	A record and a toll	39
3.1	Clay, seals, and seven witnesses	39
3.2	Two columns and a stamp	40
3.3	A demon, and the step that pays	41
3.4	What they all agree on, and who they rule out	42
3.5	The smallest thing that could carry the bill	43
3.6	What this does and doesn't buy	46
4	Why the toll isn't arbitrary	47
4.1	A closed room with no spare chair	47
4.2	Now charge for forgetting, and the toll falls out	49
4.3	Door one: keep everything, and nothing ever has to merge	50
4.4	Door two: let the flag come back down	51
4.5	Door three: forget for free	53
4.6	How much: the price grows with the squeeze	54
4.7	The same bill, written in bits, and when it's zero	55
4.8	What you're entitled to, and what you're not	60

5	Learning is free, committing is not	61
5.1	A small being at a small door	61
5.2	Bennett moves the bill	62
5.3	Two different things called “narrowing”	62
5.4	The demon on a sheet of paper	63
5.5	Who holds the pen changes the bill	65
5.6	The smallest machine that teaches for free, and the one that can’t	66
5.7	What the toll prices, said at last	66
III	The logic	67
6	The counter	68
6.1	A stick, split down the middle	68
6.2	Why it has to climb	69
6.3	Conservation: the total is the charges and nothing else	70
6.4	Only one honest count	70
6.5	The toll is an old trick with a new name	72
6.6	What the counter is, said once	74
7	Windows and shadows	75
7.1	A cave, and the people who got good at it	76
7.2	The shadow of a cube	76
7.3	The whole fact, in one paragraph	77
7.4	People who found this on their own	78
7.5	The clerk with two photographs	79
7.6	The three escapes, and what each one costs	80
7.7	What this isn’t	82
8	Structural entitlement	82
8.1	The coin that moved	83
8.2	Plato nearly had it	83
8.3	The shape, said once	83
8.4	What narrowing costs: count the rivals	85
8.5	What “narrower” buys: a stronger test	86
8.6	The count, exactly	87
8.7	What you’ve got now	88
9	What insight is	88
9.1	Sultan and the sticks	89
9.2	It isn’t a feeling	89
9.3	What happens when a computation learns	90
9.4	Test, then commit, and nothing forces the second to follow the first . .	91
9.5	What the word means now	92

10	Records over any base: the latch	93
10.1	A cup and a die	93
10.2	What “honest” has to mean	93
10.3	Both rules do work: the toggle and the clock	96
10.4	Is there only one such machine? No. One structure, two free choices.	98
10.5	Bases from outside, and a relation I have to be honest about	100
10.6	Records with more than two values	102
10.7	What the base being free buys you	103
11	The diagonal	104
11.1	Five names and a table	104
11.2	The same move, again and again	105
11.3	What it doesn’t say	106
11.4	Now point it at the record	107
11.5	How to kill it	112
11.6	What this does to the book	112
12	Five small models	113
12.1	Proof-of-stake finality: the price is on going back	114
12.2	Gas: the one that prices the write	116
12.3	Attestation: hash chains, and a quote about the past	118
12.4	Certificate transparency: visible, not priced	120
12.5	Proof-carrying code: no price, no permanence	121
12.6	One example, three levels	122
12.7	The same table, five ways	123
13	The pointer conjecture	124
13.1	The claim	124
13.2	Where the shape comes from, and how much of it survived	125
13.3	What’s checked	125
13.4	What isn’t checked, and must stay that way on the page	129
13.5	How to kill it	129
13.6	The end of the logic	130
IV	The machine	130
14	The smallest honest machine	130
14.1	What the model actually asks for	131
14.2	Where the counters come from	131
14.3	The machine	132
14.4	It computes everything two counters can	136
14.5	It’s a certification system, and the flag is a latch	137
14.6	Commitments are earned	139

14.7	What a passing check means	141
14.8	Nothing gets forged	141
14.9	A full table traps	142
14.10	The window can't see the record	143
14.11	The exact price is three	144
14.12	What it doesn't do	145
14.13	What comes next	146
15	One machine, built three times	146
16	The state record	148
16.1	The twelve fields	148
16.2	What isn't in it	151
16.3	The partition graph	152
16.4	Inside memory, and sixty-four numbers	155
16.5	Where a run starts	157
16.6	The size constants	158
16.7	What the record promises, and what it doesn't	159
17	The instruction set and the cost schedule	159
17.1	Group 1: partition (4)	160
17.2	Group 2: logic and certification (3)	162
17.3	Group 3: compute and data transfer (14)	164
17.4	Group 4: memory and control (8)	165
17.5	Group 5: revelation, ports, heap, tensor (7)	166
17.6	Group 6: witness and certification (8)	167
17.7	Group 7: arrows (7)	169
17.8	The schedule, read as a whole	171
17.9	What the chip has	175
18	What a morphism is	175
18.1	Objects, arrows, and a quiet promotion	176
18.2	The two laws, and why each	176
18.3	What a coupling is	177
18.4	What Coq checks, exactly	179
18.5	Where the laws meet the stored arrows	180
18.6	Why this isn't just extra state	183
18.7	Two small facts that matter later	184
19	Claims a module can carry	184
19.1	A question with a definite answer	185
19.2	Hang it on a group	186
19.3	A label is not a check	186
19.4	Who checks the checker	189

19.5	Where, not just how much	190
20	Two states that look the same	192
20.1	The pair	192
20.2	The window is a quotient	193
20.3	The caveat	194
20.4	A step the shadow can't see	195
20.5	Three things the windows drop, each on its own	196
20.6	What the pair doesn't prove	198
20.7	The machine versus the program it runs	198
21	The receipt in four acts	201
21.1	Act 1: the attempt costs, even when it fails	201
21.2	Act 2: build the chain	202
21.3	Act 3: assert	203
21.4	Act 4: equal observations can hide different graphs	204
21.5	What the receipt is	206
22	Insight and entitlement on the machine	206
22.1	The two readings, and who writes them	207
22.2	Insight on the machine	208
22.3	Test, then commit, on the machine	211
22.4	Entitlement on the machine	212
22.5	Every sound shortcut lands here, and what that sentence is worth . . .	218
22.6	A search whose surviving candidates can be counted	218
22.7	A small two-state witness	220
22.8	What landed	223
23	The machine as witness	223
23.1	What this machine is, relative to an ordinary one (Section 2, Section 11)	223
23.2	The counter on this machine (Section 6)	225
23.3	What no window recovers (Section 7)	229
23.4	The floor, on fifty-one instructions (Section 3, Section 9)	230
23.5	The finite fragment that pays for its merges (Section 4)	231
23.6	Learning for free (Section 5)	233
23.7	Exact-price collisions (Section 7)	234
23.8	The verifier and its three escapes (Section 7)	235
23.9	Records on this machine (Section 10)	238
23.10	Pricing and verifying on this machine (Section 12)	242
23.11	The certified flag as a durable consensus (Section 13)	243
23.12	What landed	244
24	Interpreters and the guest machine	244
24.1	The diagonal, pointed at the machine	244

24.2	Why an interpreter at all	248
24.3	The first interpreter: a two-counter machine as guest	249
24.4	The second interpreter: the machine running a piece of itself	250
24.5	Rice, on the guest	252
24.6	The guest recursion theorem	254
24.7	The evaluator, without a black box	256
24.8	What this closes	259
25	The CHSH check	259
25.1	The game	259
25.2	Where it came from, and why anyone cared	260
25.3	Wrong turns to take off the table	261
25.4	The five-by-five, and what positive semidefinite means	262
25.5	What the machine actually does	263
25.6	A rational stand-in for an irrational number	266
25.7	Two more rungs, built and not bound in	267
25.8	What isn't proved	268
26	Irreversibility accounting and the mu-hierarchy	269
26.1	A bookkeeping indicator, bounded by what it indicates	269
26.2	The calibration is a unit conversion	270
26.3	The premises have to be able to hold together	271
26.4	The calorimeter protocol: a specification, not a measurement	272
26.5	What permanence adds to Landauer: nothing new, said plainly	276
26.6	The mu-hierarchy, and how small it is	276
26.7	What this section is	277
V	The hardware	277
27	From the proof to the chip	278
27.1	Hardware as rules	278
27.2	The chain, top to bottom	278
27.3	Trusted, tested, proved: the table	289
27.4	Why generated, and why it's the stronger artifact	290
27.5	Thompson's knife, again	290
27.6	The numbers, true of the last recorded run	290
27.7	What the chip is, and isn't	290
VI	Checking it, and its edge	290
28	What Coq is, and why	291
28.1	Types as claims	291
28.2	Why I used it	291

28.3	What “zero Admitted” means	292
28.4	What you’re trusting, in one place	293
28.5	The auditor on top	293
28.6	Checking the companion formalization	294
29	How to break this	294
29.1	The abstract claims (Parts II and III)	295
29.2	The machine claims (Parts IV and V)	296
29.3	What’s mine and what isn’t	299
30	The claim ledger	300
30.1	Claims about the model	300
30.2	Claims about the machine	306
30.3	What I propose and haven’t proved	317
31	Scope	318
32	Conclusion	322
A	Things I had to learn first	324
A.1	Bits, bytes, and what a CPU sees	324
A.2	The notation the formal parts use	325
A.3	Coq, in just enough detail	325
A.4	Category theory beyond Part IV	326
A.5	Bell-test vocabulary	326
A.6	Hardware and synthesis vocabulary	327
A.7	Other terms I use without rebuilding	328
A.8	Matrices	329
A.9	Borrowed ideas the book pushes on	329
A.10	Words this book builds	330
B	Every assumption in one place	332
B.1	The records that carry a rule	332
B.2	The premises the derivations take	334
B.3	The axioms of the checker itself	335
C	What’s mine, and what isn’t	336
C.1	1. The universal floor and the potential method	336
C.2	2. The toll from finiteness, permanence, and a price on merges	336
C.3	3. Learning free, committing priced	337
C.4	4. Windows that can’t price what they hide	337
C.5	5. Structural entitlement and its bound	337
C.6	6. The record axis is a latch	337
C.7	7. The diagonal, L, and the guest’s recursion theorem	338
C.8	8. The game check	338

C.9	9. The verified path to a chip	338
C.10	10. Five record disciplines, one question	339
C.11	11. The pointer conjecture	339
C.12	What that adds up to	339
D	Every result and where it is checked	339
E	The CHSH derivation, in full	377
E.1	A. Setting and notation	378
E.2	B. The classical bound	378
E.3	C. Tsirelson’s bound from algebra	380
E.4	D. The zero-marginal 5x5 moment matrix	381
E.5	E. The integer check	385
E.6	F. Level 1+AB	387
E.7	G. The elliptope completion	393
E.8	H. What is not proved	395
E.9	I. Prior work	396
F	Hardware retirement: the assumptions in full	396
F.1	Common notation and conditions	397
F.2	Single-step cases	398
F.3	Memory-reading and checking cases	399
F.4	What the catalogue establishes	400
F.5	Reading a snapshot as a kernel state	401
F.6	The full-state bridge conditions	401
G	Constructing the quoting operation	402
G.1	Numerals and syntax as data	403
G.2	Recursion without running under a lambda	403
G.3	Quoting numerals	404
G.4	Quoting terms	404
H	The two fixed interpreters, in full	405
H.1	Packing the program	405
H.2	The four-register guest	405
H.3	The two-counter guest	409
I	Sources and mathematical dependencies	411
I.1	Structure, information and knowledge	411
I.2	Cost, records and thermodynamics	412
I.3	Computation, self-reference and checking	413
I.4	Correlations and environmental records	414

Before the book: the short version

The book in one paragraph. The Thiele Machine is an abstract model of computation: a state, a step rule, a counter that only climbs, a rule that the step turning a certification reading from no to yes costs at least one unit on that counter (I call that rule the toll, and Part II builds it), and a window that says what an observer sees. Here's what's proved, in Coq (a proof assistant: software that checks every step of a proof), with every premise printed. Any system that follows the toll pays something to get from uncertified to certified. On a finite machine whose certificate is never taken back, the certifying step merges states, so the toll follows from pricing merges, with Landauer's principle (the physicist's rule that forgetting costs heat) as a named premise. A measurement that forgets nothing can be priced at zero. A window that shows a certifying step and a quiet one as the same can't price certification exactly. Over any base machine, an honest permanent record is a latch on an event (a switch the event flips once and nothing flips back). And no procedure a substrate can represent decides which programs admit a shortcut, given a recursion theorem (the fact that a program can act as though it held its own code). A substrate is the model's bare shape, a kind of state and a rule for stepping, with whatever extra pieces a given theorem asks for; here the extra pieces are programs written as states and that recursion theorem. All of that is about the model: any system with the named pieces. One build carries the model, and that build is what I mean every time I say "my machine": a fifty-one-instruction machine in Coq, a runner extracted from it as ordinary code, and hardware in Kami (a Coq library for describing circuits) refined instruction by instruction. On it, leaning on a structural fact costs at least the index bits it commits to (roughly, how many times it halves its list of possibilities), under the hypotheses of one representation theorem; the recursion theorem is proved for its twelve-instruction guest (a small fragment it runs as data), as it is for the lambda calculus L, and assumed for the full machine; and an integer check on eight counters implies that a fixed moment matrix is positive semidefinite, and so Tsirelson's bound, the ceiling quantum mechanics puts on the score of a two-player game (Part IV builds the game, the matrix and the check from zero). Not proved: a physical price for one mark, any separation of complexity classes, that certification is the event worth recording (that's a conjecture), or anything about a powered-on board, since none was.

I didn't invent a machine. I found one. That's how I see this work, and I'll be straight about the difference between that conviction and a theorem before I do anything else.

What the Thiele Machine is. An abstract model of computation. A machine the way Turing's machine is a machine: mathematics you can reason about, not a device. It isn't a CPU, an instruction set, a graph, or category theory; those are things I built with it, or things it can carry. The model is small. A state. A step rule. A counter that only climbs. A rule for which steps have to pay. And a window: what an observer is allowed to see. Like a Turing machine, it's a precise way to say what a computation is. Unlike a Turing machine, it computes nothing new. It adds what a computation

establishes, and what establishing it cost.

What this book is. An argument, made with that model, about what an account of computation should preserve. The usual account describes a computation by what it reads and writes. I think that account is a shadow, and that there's another axis, holding what the computation established and what that cost. Here's the argument in five steps, each with where it stands.

Holding a sorting costs something; standing behind one might. That's Part I: one word, structure, pushed on with you through as many of the people who wrestled it before me as I could reach. It finds three different acts that people run together. Building and storing a sorting you can read back costs work and space. Forgetting where things came from costs heat, if Landauer is right. And standing behind a sorting, vouching for it, is a third act that nobody I read had put a price on. Whether it has one is the question the rest of the book takes up.

Establishing is not free. If the step that turns uncertified into certified has to pay, every run from uncertified to certified has paid. Proved, for any system that follows that rule, whatever its states and steps. And the rule isn't arbitrary: on any system with finitely many states, a certificate nothing takes back can only be switched on by a step that merges two states. That part is pure counting, proved. If merging is charged for, the way Landauer's principle charges for it in its worst case, the rule follows. That part is proved with the charge as a named premise, its critics named, and every premise refusable with a counterexample built. Learning isn't what costs: a measurement that forgets nothing can be priced at zero by an honest price list, and an observer still learns from it. Proved. What can't be free, on those premises, is committing to what you learned in a record the machine can't take back.

Two more things belong to that step. First, which steps a price on merging has to charge: exactly the ones that merge. Every step that writes a record nothing takes back is one of them, and on a finite machine a step that switches a certificate on escapes the price only when the same step switches one off somewhere else. The tidier guess, that the priced steps are exactly the permanent records, is false, and three states show it. No theorem picks out a wider class of events every account must price. With one mark per halving, the step that writes a permanent record costs at least the base-two logarithm of how much it squeezes together (how many times it halves the number of states it can tell apart), and the same bound holds in bits of Shannon entropy, the standard measure of information. Proved, with the price as a premise. Second, where the machine I built stands, which is a Part IV fact and not a Part II one. A small finite piece of it, eight states seen through a narrow window, is such a machine, and the machine runs it paying exactly what that finite machine charges; one of those charges, two for a jump, is the program's own choice, which the machine's price list allows and doesn't force. The full machine isn't one: every step that raises its certified flag merges states and costs at least one, but an ordinary jump can merge states too and

cost nothing. So the full machine doesn't price every merge, and that's a theorem too (Section 23).

A window can hide what was paid. Two runs can look identical through the usual window and differ in what they established and paid, and nothing reading only the window gets the difference back, so no price read off the window charges the certifying step exactly. Proved, for any window with such a pair; and the machine I built has such a pair for both windows you'd point at it.

Leaning on a fact takes a paid history. A computation is entitled to lean on a structural fact when it holds the object, a checked relation, and a paid event on its record. Defined, for the model, with a counting fact beside it: narrowing a list of rivals down takes at least the yes-or-no questions that name the survivors. The cost floor that ties those questions to steps a run actually paid for is proved on my machine (Part IV), for shortcuts that hand over their receipts (the record of what they paid), under a list of hypotheses printed in full. Entitlement is a license to lean on a claim, not a proof that the claim is true.

So the axis belongs in the step. An account of computation should carry what was established and what it cost as part of how it steps, not as something bolted on after. Proposed. This is the conviction the proofs are there to test. What's proved around it: over any base machine, an honest record is a latch on an event, with two free choices, the price list and the event; no procedure a substrate can itself represent decides which programs admit a shortcut, given a recursion theorem, by the same self-referring trick, the diagonal, that put a hole in Gottlob Frege's foundations for arithmetic (Part I tells that story); and which event deserves the record is a conjecture, with its refutations listed.

Certification is one point I could pin down. It gave me a law I could state, a machine I could build and run, and proofs I could put in front of someone who didn't share my intuition. It's one witness in a larger argument about structure and observation. It isn't the argument.

How the book is laid out. Part I is the picture: how I got here, and what structure is. Part II is the axiom: the smallest thing that could carry a bill, and why its one rule isn't arbitrary. Part III is the logic: the counter, the windows, entitlement, insight, the latch, the diagonal, five real systems, and a conjecture. Parts I to III never use my machine; every claim in them is about any system with the pieces it names, and the pictures are toys. Part IV builds twice. First the smallest honest machine I could make: two counters, six instructions, a table of checked facts, a ledger and a flag. On it a raised flag provably went through a passing check, a commitment and a certification of the same claim, in that order, and it computes everything Minsky's two-counter machines can, so whether it halts is undecidable. Then the big build, one try, in three materials, as the witness for everything before it: its state and fifty-one instructions, the claims a module of it can carry, what insight and entitlement look like on it, and where each

Part III result has a machine instance. The big build prices every commitment and doesn't enforce that one was earned; the small one is where earned commitments and universality are proved. Part V is the chain from the proof to a chip, with each link labelled proved, tested, or trusted, and the sentence that no board was ever powered on. Part VI is how to check all of it and how to break it. The vocabulary I had to learn is at the back.

One more thing before the book starts, because people get it backwards. I used language models to write the code and the proofs in this project, and I trust language models about as far as I can throw them, which is exactly why every formal declaration is checked by a proof assistant that doesn't care who or what wrote it. Coq rejects a wrong proof the same whether a tenured professor or a model hands it over, and it accepted these, with none of my own axioms anywhere in them, re-checkable by the procedure in Part VI. Being self-taught and using modern tools didn't make this looser. It's the reason I made a machine check the formal corpus, and then built an auditor to read over the checker's shoulder for every way I might have cheated, on purpose or by accident (Part VI says what it looks for). It doesn't trust me either. Good.

I think there's something foundational here. The results in this book have statements, premises, and proof terms. The larger claim is a conjecture, not a theorem. I built the machine so that the work could be done in public, with something firmer than my confidence to argue over.

Let me be straight with you

What this project is. An argument about what an account of computation should preserve, made with an abstract model built to carry it. Its central proposal is that what a computation established, and what establishing it cost, belong in how it steps. The working form of that proposal is structural entitlement: a computation is entitled to lean on a structural fact when it holds the object, a checked relation, and a history that earned it. The model, its laws, and the instructions of any one build are different things, and I keep them apart. The generic flag-raising step pays to set a flag; it checks nothing. A claim that something is actually *true* needs explicit witnesses and a checker proved sound, and some results here are of that kind; the generic accounting law isn't one of them. It's a law about marks.

The proved core, in three steps. Anything that follows the toll pays something to get from uncertified to certified. A window that shows two states as the same, when they disagree on the counter or on some other question, can't answer that question on every state. Those two are about any system with the named pieces. The third is checked on my machine, in Part IV: a checker that only sees the bare window, where a named pair of the machine's states collides, can't be both sound and complete about the claim that separates them. All three are statements about named laws and named windows. An ordinary machine can keep a richer encoding, or enforce the same invariant through its own transition table. What I'm proposing is about why the extra distinctions matter.

It isn't a theorem that every Turing-equivalent system lacks them.

Substrate versus scaffolding. Two layers run through the book, and confusing them makes the theorems quietly stop meaning what they say. The *substrate* is the shape: a kind of state and a rule for stepping. Different theorems ask that shape for different extra pieces, and the pieces aren't the same. The universal floor (the "pays something" result above) asks for a certification reading and the toll on it. The counter's uniqueness (only one count fits a chosen start and price list) asks for a starting value and a schedule of increments; Part III gives the argument, a short induction that works for any system with those two pieces, and the checked theorem is stated for my machine and sits in Part IV (Section 23). The diagonal asks for a way to write programs as states and a recursion theorem for the transformations the substrate can express, and needs no toll at all. "Substrate-level" doesn't mean one fixed plug that every theorem shares. A theorem only gets to use the parts it asks for. Ask it what it needs and it'll tell you, and whatever else is lying around the workshop isn't the theorem's. So each result is stated over exactly the extension it uses. The substrate with the extension a theorem names is what I mean by *the model*, and the name Thiele Machine belongs to the model. The *scaffolding* is everything that holds the shape up so you can touch it, and when I say *my machine*, *the machine I build* or *the build*, this is what I mean: its state record (registers, memory, a program counter, a graph of groups and arrows, the certification readings, the counters), fifty-one instructions and their prices, a reference kernel (the machine's definition in Coq), the guest it runs as data, an extracted runtime, the Kami hardware, a simulator harness. The build fills in several extensions at once and borrows, for any one theorem, only the extension that theorem names. Let that line blur and a theorem stated over one extension starts reading as if it carried premises it never uses. "Substrate" never means the build, and it never means the side of a simulation doing the stepping; for that I say "the machine doing the stepping," as against "the program it runs."

So here's how the book keeps the two apart. Parts I to III are about any system with the pieces each result names, and their pictures are toys you can hold (two boxes on a sheet of paper, chairs in a room, a row of dots), not my machine. My machine first appears in Part IV. Every Part III claim that has a machine instance gets that instance in Part IV or V, with its own checked name, and Part III says so in one sentence at most. When the origin story, a few pages on, says the shape "was the substrate the entire time," it means this layer, the model's shape, and not any piece of the build. Same story with two more of its words. When it says "the machine" about the thing it named the Thiele Machine, or calls that thing a meta machine, it means the model; "my machine" is the build, and the build doesn't show up until Part IV. And when it lists what the model asks for (a state, a step, a ledger, a window), read the ledger and the window as two of the extra pieces: a result asks for them only if it uses them.

Part IV has two pieces of scaffolding, not one, and I'd rather you know that up front. Section 14 builds the smallest honest machine I could: two counters, six instructions,

a table of checked facts, a ledger and a flag. Earned commitments are proved there (a raised flag was checked, then committed, then certified, on the same edition, with nothing in between), and so is universality, through Minsky’s two-counter machines and a halting result borrowed from the Coq undecidability library. Then comes the big build of fifty-one instructions, which is what “my machine” and “the build” mean everywhere else in this book. It prices commitments without enforcing them: its check commits to nothing and its commit doesn’t look for a check (Section 22 shows where). Neither one is the model. The small one shows the model fits in six instructions; the big one shows how much a single build can carry while the toll holds.

What “new model of computation” means here. It adds something to how a machine runs: structure it keeps, and an account of what that structure cost. It doesn’t add computable functions. It’s an abstract model the way the Turing machine is; it isn’t a new primitive the way the Turing machine is. When I say “shadow” I mean the particular window I name, not every way a conventional machine could store things. Enforcing the law, storing it, and letting an observer see it are three separate questions, and the book keeps them separate.

What this project is not. Not a solution to P versus NP, and not a complexity-class separation of any other kind. Not a theory of everything. Not a claim that computation is physics, or that physics is computation. Not a physics result: the game result (the integer check and Tsirelson’s bound from the short version) is pure mathematics, and Landauer’s principle enters only as a premise with its name on it, never as something derived. Not a quantum computer or a claim about quantum behaviour. Not a verified chip; a verified model with a chain of trusted and tested links below it. Not a claim to priority on any idea in it. Not finished; the open questions are listed at the end.

What I do and don’t know about prior work. I came to this with no formal training in the fields it turned out to need, and I’ve read what I could reach, with every tool I had. The prior-work map below is orientation, not scholarship: for each piece of this book, who did the thing it leans on, and what, if anything, this development adds. I’d welcome correction on any row.

What I compare each time is what a result is *about*. Landauer’s bound is about heat, a physical cost under a thermodynamic model. No Free Lunch (an optimisation result, in the list below the table) is about averages over a whole spread of problems. The toll is a floor on one kind of step, the step that turns a certification reading on. Exact pricing adds a second rule on top: never charge more. So the comparisons are about the shape of each result. They aren’t claims about who got somewhere first, and they don’t show that certification is the only event worth pricing.

The right-hand column uses words the book only builds later (the latch, the fibre, the guest, the integer gate, the matrix test), so read it as a list of places to go, not as something you’re meant to follow yet.

Prior work	What this book leans on	What this development adds
Landauer 1961; Bennett 1973, 1982, 2003	erasure and merging cost; measurement can be free	the derivation of a certification toll from finiteness and permanence, with each premise's counterexample built
Maroney 2009; Sagawa and Ueda 2009; Sagawa 2014	thermodynamic bounds that depend on the actual spread over states; a known state can be reset for free	the certificate-specific entropy bound with its support and permanence premises out in the open; heat still needs the Landauer premise, and no new physical law is proved
Shannon 1948; Hartley 1928	entropy and log-of-count bounds	the same squeeze bound in counted states and in bits, for a permanent flip
Tarjan 1985; Lyapunov 1892; Nipkow; Charguéraud and Pottier	the potential method, already mechanized	the toll as that method with the flag as potential
Katsumata 2014; Orchard, Liepelt and Eades 2019; Danner and Licata	cost carried as a grade that composes with the computation	a run's cost as a writer cost and as a grade fixed before the run; the counter rides in the state, put there by the step rule, and whether that embeds in graded semantics or is something weaker I haven't settled
Lamport 1978; Floyd 1967; Turing 1949	one-way counters give order and termination	the receipt's three properties (it only climbs, it equals the sum of the charges, it's the only such total) stated and proved for one schedule
Moore 1956; Kalman 1960; Cousot 1977; Codd 1970 Smith 2009 (quantitative information flow)	what a window loses how much an observer learns about a secret from what it sees	the fibre fact with a price in place of a decoder; the machine's named collisions none for leakage: the collisions rule out exact decoding through named windows, which isn't zero leakage, and no leakage measure is supplied
Gettier 1963; Necula 1997; Cook 1971	entitlement is not truth; proof-carrying checks	entitlement as object, relation, paid event, with a counted bound
Dwork and Naor 1992; Back 1997 (Hashcash)	make establishing something cost, in a way anyone can check cheaply	a comparison, not an identification: proof of work checks a found value against a difficulty target, while the toll charges one mark at the step that certifies and verifies nothing about effort

Prior work	What this book leans on	What this development adds
Eilenberg and Mac Lane 1945	the category	couplings as relational content; laws proved on couplings and on the stored arrows, with the identity flag carried through composition
Eccles and Jordan 1918; Plotkin 1989; Shapiro and coauthors; Abadi and Lamport 1991	the latch; write-once and grow-only state; history variables	the latch form over any base, with its two counterexamples and converse
Cantor, Gödel, Turing, Kleene, Rice; Lawvere 1969	the diagonal	the shortcut wall on any substrate; the guest recursion theorem by construction
Bell, CHSH, Tsirelson, Wehner, NPA, Landau, Masanes, Fine; Echenim and Mhalla 2023 and Lean’s mathlib (Tsirelson’s bound already machine-checked)	the game, its walls, and the matrix test	an integer gate on counters, inside one instruction, that implies the pinned matrix is PSD, which implies the bound
Buterin and Griffith; Wood; TCG; RFC 9162; Nacula	five record disciplines	five fragments read through one question: is the write priced
Zurek 2009; Ollivier, Poulin and Zurek 2004; Blume-Kohout and Zurek 2006	pointer states and redundancy	a conjecture borrowing the shape, with the gap to the physics stated
Choi et al. 2017 (Kami); Hoe and Arvind; Nikhil	hardware as rules in Coq	per-opcode refinement, a proved serial loader and status report, a labelled chain to a bitstream
Leroy 2009; Klein et al. 2009	the discipline of saying where the proof stops	the proved/tested/trusted table
Minsky 1961, 1967; Forster, Larchey-Wendling and coauthors 2019, 2020	two-counter machines are universal, and their halting is undecidable, checked in Coq	a six-instruction machine on two counters where earned commitments, no forging and an exact minimum price of three are theorems, with halting reduced to the library’s
Forster and Smolka 2017; the Coq Library of Undecidability; Norrish 2011	L as a model of computation in Coq; counter-machine constructions	the guest’s recursion theorem, with its evaluator compiled through L and the library into guest instructions

Everything in the right column is stated with its premises in the body of the book, and

everything in the middle column is cited where it's used. The table is the short form. The long form is Appendix C, which takes each row, says what was already known with sources, and sorts what's added into new result, new proof, mechanization, implementation, synthesis, or conjecture. Most of the right column is mechanization, implementation, and synthesis.

A few more labels I went and looked up don't get a row, because nothing in the book leans on them. They're neighbours, and I list them so you can see why I don't think the toll is one of them in a hat. Each is a claim inviting correction.

- Linear logic (Girard 1987) treats an assumption as a resource that gets used up. It constrains what a program, or a proof, may consume. The toll constrains the model's step function instead. Same instinct, different layer.
- Foundational proof-carrying code (Appel 2001) checks safety proofs against the smallest trusted base it can: a logic plus the machine's semantics. Here no proof rides along with the program. The step rule writes the receipts as the run goes.
- Session types (Honda 1993) constrain how programs talk to each other so the conversation can't go off script. The toll is about what one computation has paid to establish. Different problem.
- No Free Lunch (Wolpert and Macready 1997): averaged over every problem, whatever an optimiser gains on some problems it gives back on others. That's an average over a whole problem space, and the edge is paid for elsewhere. The toll charges one step in one run, with no refund anywhere.
- Kolmogorov complexity (Kolmogorov 1965): the structure in an object is the length of the shortest program that prints it. That's a property of a finished string, with no steps and no moment of paying. The toll is a law about a step.
- Cook and Levin (Cook 1971; Levin 1973): checking can be cheap while finding looks expensive, and whether finding is really harder is the open P-versus-NP question, a gap in how difficulty grows with size. The toll puts a fixed floor under the one step that commits, however easy or hard the finding was.
- The thermodynamic cost of choosing (Rovelli 2024): settling a binary choice at temperature T gives off at least $kT \ln 2$. That's the closest thing to the toll I've found, because it charges a commitment rather than an object or an average. But it's physics, indexed to a temperature. The toll charges the commitment in a model of computation, with no temperature in it. Closest neighbour, not the same chair.
- CerCo, the Certified Complexity project of Amadio and coauthors, builds a verified C compiler that carries the object code's execution cost back up to annotations on the source. There the cost decorates the program and survives compilation. Here it lives in the model's step rule.

Part I

The picture

1 How I got here

What I am is curious, past the point of good sense.

I pick a thing up and cannot set it down until I have turned it over enough times to see how it holds together, and this time it did not let go.

I am not a programmer and I was never formally taught any of this.

I was not trying to build a formally verified CPU.

I would not have known what one was if it bit me.

I was trying to build a 3D renderer.

There was no plan behind it.

Just a pull to make something real with my own hands, and the kind of stubbornness that will not let a thing go until it stands up on its own.

I have worked on it in off hours every day since.

Once I started digging in, two things came clear fast. I knew nothing about computers, and the field was already saturated, every obvious road paved and crowded. So I needed an angle nobody else was standing on, and the angle I had was not a strategy, it was just the only way my head works. I did not know what compositional meant. I had never heard of category theory. I called the thing I was building digital DNA, because that is what it felt like in my hands, and I thought about it in the only language I had: gravity pipes and directions and vectors, things that were linear and somehow not linear at the same time, structure that flowed and joined and came apart without my owning one proper word for any of it. I built a custom rendering pipeline out of those intuitions, by feel, throwing out pieces that did not sit right even when I could not have told you the rule they were breaking. And then, the way I always do, I went looking for what I was actually doing, and found out it already had a name. An old one. A deep one. What I had been reaching for in the dark was composition. It was category theory, a whole field I had never once heard of, sitting there describing the exact shapes I had been shoving around. The moment that landed, the rendering pipeline stopped being a rendering pipeline and started turning into a categorical engine, and I started swallowing the real theory as fast as I could find the words for things I had already built.

The method underneath was crude and it held. I wrote a long structured spec in JSON (a plain-text format for describing data) as a context document, fed it to Copilot, and

had it build one Python file per concept, one at a time, each with its own tests, and I would not move to the next until I could watch the current one actually run. Category. Object. Morphism. Functor. Monoidal product. It came together in pieces and never in chapters, the way a picture resolves out of static instead of getting drawn in left to right. The Yoneda lemma (a result buried deep in category theory) was the piece that made the rest snap into place, the night I quit borrowing other people's runtimes and started writing my own around it, with Z3 (a constraint solver out of Microsoft Research) wired into the middle.

Then within weeks I found out the engine I had built to draw pictures was general enough to run a physics simulation on the exact same primitives, without my rewriting any of it. I wrote Newton's laws as a program in it, half expecting nothing, and it ran. I want to be careful about what that was and what it was not, because this is exactly the kind of thing that gets misread. It was not me deriving physics, and I am not claiming the machine explains gravity. It was the compositional structure being general enough to express a physics simulation the same way it had expressed a renderer, and that generality is what told me the structure was the point and the pixels never were. Somewhere in those weeks I crossed from debugging why one pixel would not show up to understanding that I had quietly built a categorical computing engine, and that the rendering had only ever been the first thing I aimed it at. The pixel did eventually show up, for what it is worth.

I kept going, because by then I could not not keep going. I built a small custom language for describing scenes (the name for that is a domain-specific language, a DSL), then a program that read the language and caught my mistakes in it (a linter), then a way to type commands at the whole thing from a terminal (a command-line interface, a CLI). The night the engine first ran a program declaratively, meaning I told it what should happen and let it work out the how instead of my spelling out every step, I told my wife, Julie, I thought I was onto something. She said okay.

Then I hit a wall, and it is the wall this whole document is about. Z3 could settle the small questions and fell over on the exact ones I needed answered, which left me staring down an infinite hallway of guess-and-test with no proof waiting at the end of it, ever. One night, not even working, I was watching a video about Flatland, the old story about a creature living in two dimensions who cannot so much as picture a third. And it landed on me sideways, all at once, the way the real ones always do. I said it out loud: I get it now. There is another axis. Classical computation only ever sees what it can read and write off its own surface, and there is a whole direction it is blind to, not because it is broken but because it was never built with an eye that points that way. I opened a new repo that same night and called the thing in it the Thiele Machine. The name began as a working title and became the name of the model. It carries the larger claim plainly: the machine treats structural state as part of computation.

What survived the whole trip from there to here is not the code. Not one line of it.

By the time the Coq proofs finally landed, every line of that first renderer had been thrown out and rewritten at least twice, probably more, and good riddance to all of it. What survived was a single architectural commitment I made early and never once let anyone talk me out of: structure is the state itself, the actual thing the machine runs on, not a metaphor laid over some realer state underneath. Category theory was the first language I found for that structure. It is not what the model is. The model asks for a state, a step, a ledger, and a window, and category theory appears nowhere in that list. It lives on in the build. The `Category` structure that came together in that first week and first ran morphisms correctly at two in the morning is, in different words and different syntax, the same object the partition graph is in the machine I ended up with (the graph of groups, and arrows between them, that Part IV builds). I did not write that code, not then and not now. I directed the models and refused everything they handed me until the morphisms composed the way the shape said they had to. The morphisms in that early code were objects that remembered where they came from; the morphisms in the machine I ended up with are objects that remember where they came from and that you can walk up to and challenge. The receipt grew teeth. But the shape itself never moved. The shape was the substrate the entire time, and everything else, every renderer and language and proof, was me slowly learning to see what I had already built. By the shape I mean structure held as state. Not category theory. Category theory was just the first language I had for it.

Formal verification was a graveyard of dead ends further on. The hardware came after that. Every step pulled back to show another layer underneath that I had not known was there, and I mean that as plainly as it sounds: I did not know what a Turing machine was when I started, I did not know what Coq was, I did not know most of the words in this document existed. I learned each one the same way, by walking face-first into a wall that needed it and not being let past until I had it. The needing always came first. The learning came after, dragged along behind the needing because it had no say in the matter.

Large language models were my hands the whole way through, and I want the division of labor exact, because it is the thing people most want to get wrong about work like this. The hands had nothing like this to draw on: no reference implementation, no prior project to point at. The ideas, the architecture, the direction, the demand that the thing be true: those are mine, all of them, no asterisk. The symbol-pushing, the actual Coq and the actual math, the models produced and the proof checker judged. The proofs compile or they do not. I refused every one that did not. Coq is an obedient idiot. It checks the proof matches the statement. It does not check that the statement means what you think it means. A theorem can claim “every cat that is also a prime number has whiskers” and the proof can be three characters long and Coq will nod and stamp it. So I read every statement. The proofs you can’t fake. The statements you can fake by accident, which is the more dangerous failure mode. I refused both kinds. However the symbols got onto the page, the argument on it is checkable by

reading, and it would be exactly as right or as wrong if a million monkeys had typed it. The monkeys could not have refused.

A word on what I actually do, because it is stranger than the usual split and it runs under everything after it. I do not write code or math by hand. I do not pick constants. I do not grind through derivations. I have never written a line of code on paper or in a notepad in my life. What I do is see, and direct, and refuse. When I take hold of an idea it does not stay a point. It opens up. It becomes a shape I can blow up and turn over and take apart, lay across another shape to find where the two of them touch, run forward and then back to where it started, zoom into until it is the only thing in the world and pull back until it is one dot in a field of other dots, and the whole time it stays wired to everything I have ever connected it to, the right connections and the wrong ones both, all of them live at once. I am not reading that field neutrally, the way a search would. The thing doing the looking is me, the whole of me, my gut and my mood and my stubbornness picking what lights up and which way I move through it, and that is the part you cannot lift out without turning me back into a search engine. So I see where a thing has to fit, the shape it is obligated to have, usually well before I could write it down in a single symbol. Then the model gives me the symbols, the proof checker takes them or throws them back, and I refuse anything that will not compile or does not match the shape I am holding. That refusal is the one thing in this entire document that is mine and only mine: the demand that it be true, and the throwing-out of everything that wasn't.

I am telling you all of this because it is not background color, it is the engine. I did not come up through anywhere that hands you a foundation and says trust us, this part is settled. No professors, no colleagues, no field full of authority to lean back on when a claim got heavy. From the outside that is a deficit. From the inside it is the whole reason any of this is solid: when I meet a claim I either find the proof or I do not believe it, and that goes double for my own, because my own are the ones I most want to wave through. It is how I am built. It is the only way I know to hold a thing and actually trust that I am holding it.

So everything in here gets built from the ground up, and not as a courtesy to you. I build it because I do not believe I understand a thing until I have built it with my own hands, watched it stand up on its own, and tried to knock it over. I built this document the exact way I built the machine, brick by brick, refusing each brick until it held weight. That includes the parts you already knew cold before you opened this. I did not skip them for you, and I could not skip them for me.

The vocabulary I had to go look up to do any of this sits at the back of the book, so the argument comes first. If a word stops you, it is there. If you already know this material, you never have to open it. But if you catch something in the basics that is wrong, I want to know, and I mean that as a real ask and not a polite one. Nothing in here is correct just because I wrote it. Especially the parts I wrote.

On the name. I wanted to call it the meta machine. Then Facebook bought the word “meta”: a company renaming itself after a term for self-reference and draining it of any in the same motion. So I went one level up, where this machine already lives. The most meta move available, for a machine that runs on meta-level reasoning, was to put my own name on the cover and take the word back. That’s the whole of it, a meta move on a meta machine. It happens to be my dad’s name too, and my wife’s now. So I asked them before I did it. They said go ahead.

2 What structure is

One word. That’s this whole section, and I’m aware of how that sounds, so here’s the word and you can decide whether to stay. Structure.

I used that word for most of my life without once looking at it, which I suspect is true of most people and most words, and it’s fine, right up until you do something stupid. The stupid thing I did was build a machine that keeps a running bill, and the bill has one rule I couldn’t talk it out of. You can arrange things however you like for whatever price you name, including free. But the moment you ask the machine to *vouch* for an arrangement, to put its own name on the claim that these things belong together, there’s a charge, and it isn’t zero. How much, charged in what, why I picked that and not something else, and whether I had any business picking it at all: that’s all later, and some of it I’m still not sure of. What I was sure of, standing there with the thing built, was that I’d made something that charges for standing behind structure and I could not have told you what structure was. It didn’t care. It can’t nod along the way a person does. It needed the word to hold still, and the word had never held still for anyone, least of all me. (The machine itself stays off the page until Part IV, on purpose. Until then, the one rule is all of it that matters, and I’d rather you met the rule in a toy you can hold than in a machine you’d have to take on trust.)

So I went looking at the people who’d wrestled it before me. Some of them have been dead two thousand years and are still, I’d bet money, better at this than I am. I used everything I had: language models, more than one, books, libraries, my own gut, anything in reach. I don’t know if I found enough. I never do. There’s always another shelf behind the one I was reading. And being dead and famous is not the same as being right, so I pushed on each of them, and what follows is what I could reach, pushed on as hard as I know how. The one thing I’m asking is that you push on it with me, out loud, at your own kitchen table, because the first time through I was the only one checking, and that’s not enough people.

2.1 A string, a square, and a drowning that didn’t happen

Start with the oldest one, and start by doing it. Take a string. A guitar string, a rubber band over an open box, anything that twangs. Pluck it. Now press it down exactly at the middle and pluck half. That’s the same note, an octave up, the one that makes a round sound like a round. Press at a third of the way along and pluck the long side:

that's the fifth, the second note of every bugle call you've ever heard. Half. Two thirds. Small whole numbers, and your ear says yes before your brain does. About twenty-five hundred years ago, in the south of Italy, the followers of a man named Pythagoras are credited with finding exactly that, and it's as true in your kitchen tonight as it was on their hillside.

Here's what they did with it, as best anyone can tell. Nothing survives that's surely from Pythagoras's own hand. The earliest fragments are from a later member, Philolaus, and scholars argue over those. Mostly what's left is Aristotle, one to two centuries later, writing in his *Metaphysics* about them the way you'd write about the neighbours' religion, and he says they took the elements of numbers to be the elements of all things: everything is number. Not that everything can be counted. That the parts of numbers are the parts of all things.

Stop and ask what I asked. What does that even mean? Is a chair a number? Does a chair have a number tucked inside it somewhere? Aristotle doesn't say, and he's reporting a belief he doesn't hold. So don't take it on his word, or theirs, or mine. But before you push on it, know what it was holding up, because you can't test a load-bearing wall without knowing what sits on it. These weren't people doing math for fun on weekends. They were a brotherhood, with rules about what you could eat and when you had to keep quiet, and that plucked string was their scripture. Halve the string, hear the octave: to them that was proof that under the mess of the world there were whole numbers in clean ratios, that the cosmos was a chord and they'd heard it. "Everything is number" wasn't a slogan. It was what they'd organized their lives around. Which is exactly the kind of sentence you should be suspicious of, mine included.

Now push, and push where they loved most: the square. Draw one, with a side one unit long, any unit you like, and draw the line from one corner to the opposite corner. Call that the diagonal. If everything is number, then the diagonal is some whole number of parts long when the side is some whole number of parts, so the diagonal divided by the side is a fraction, a whole number over a whole number. Cancel anything the top and bottom share, the way four-sixths becomes two-thirds, until they have no common factor left. I'll call the top p and the bottom q , and the letters are purely so I don't have to keep writing "the top" and "the bottom."

You need one fact about squares, and it's the one Pythagoras is famous for. The square built on the diagonal has exactly twice the area of the square built on the side. If you want to see it rather than believe it, cut a square along both diagonals into four triangles and arrange them around; it's a five-minute job with scissors. So p times p is twice q times q . People write "times itself" as a small raised two, so that's $p^2 = 2q^2$, and that is the only piece of notation in this argument.

Twice anything is even. So p times p is even. An odd number times itself is always odd (three times three, five times five, try a couple), so p itself must be even: it's two

times something, and I'll call that something r . Put it back in. Two r times two r is four times r times r , and that equals twice q times q . Halve both sides: two times r times r equals q times q . So q times q is even, so q is even too, by the same step as before. But you already cancelled everything the top and bottom had in common, so they can't both be even. The argument has eaten itself, and when an argument eats itself the thing you supposed at the start is false. There is no such fraction. The diagonal of a square can't be named by any ratio of whole numbers, ever. Aristotle reaches for this very argument in his *Prior Analytics* as his example of proof by contradiction (suppose the diagonal can be measured, and odd numbers come out equal to even ones), and you just did it with a pencil.

There's a story that's been told for two thousand years: that a member of the brotherhood named Hippasus let this secret out, and his brothers took him to sea and drowned him for it. I went and checked, because it's a good story and good stories are the ones to check. The Stanford Encyclopedia of Philosophy's entry on Pythagoreanism finds no ancient writer who connects Hippasus to this discovery at all. It's a legend. But notice that the legend exists. Somebody, long after, felt that a man *should* have been drowned for this. That's how much was riding on it. To them, the diagonal wasn't a math problem. It was the end of the world.

So the claim is dead, and you killed it, not me. Now the question that matters: what's left? Look at your square again. The diagonal is still there. It still stands in a perfectly definite relation to the side; you drew it, and nobody drowned. That relation was real before anyone tried to put a number on it, and it stayed real after the number failed. The relation came first. The number, when there is one, is a name for the relation, and sometimes there isn't a name. I'm going to keep that, and I want you to keep it too, because you're going to watch the same move get made again and again as this book goes on: somebody says "everything is X ," and it's true enough to do real work with, and it sounds right when you pluck it, and then somebody draws the diagonal. I don't know yet whether the thing I built survives its own diagonal. That's a real question, and I'll get to it.

2.2 Shapes poured into nothing

About a century and a half after the brotherhood, Plato, in a dialogue called the *Timaeus*, tried to build the whole physical world out of geometry. Fire, air, water and earth were four regular solids (the pyramid, the eight-faced one, the twenty-faced one, and the cube), and the faces of those solids were built from exactly two kinds of right triangle. You can make one of each in a minute: cut a square from corner to corner for the first, and cut an equilateral triangle from a corner to the middle of the opposite side for the second. Everything else, in Plato's story, was a receptacle, a featureless medium that takes on shape the way wax takes a seal. Structure, for Plato, is shape laid into something that has no shape of its own.

Why did he care? He wanted a world made on purpose, by a craftsman working from

a plan, and triangles were the plan. Is it true? No, but not for the cheap reason. You can't refute a claim about what things are made of at the smallest scale by picking one up; the smallest scale is exactly where your hands don't reach, and Plato knew that. What sank it is that it made no prediction anyone could check and the people who eventually did reach that scale, two thousand years on, found atoms, and atoms aren't triangles. The idea was a guess with no way to lose, until it lost. But push on it and two things survive. One: structure is something *laid in*, and the stuff it's laid into doesn't carry it by itself. That's a real idea, and I'd ask you to keep it loosely for now, because later I'll have to ask whether the state a machine holds is the wax or the seal. Two: Plato broke his own claim, in a later dialogue called the *Parmenides*, with an objection that Aristotle later nicknamed the third man, and it's the most useful mistake in this section. Take all the large things. Plato says they share one thing, Largeness. But Largeness is itself large, so now there's a further thing that all the large things *and* Largeness share, and that one is large too, and it never stops. Try it with any property you like; it runs away every time. The lesson, and I think it's right though Plato never quite accepted it himself: the structure of a collection is not one more member of the collection. The plan isn't a brick. Hold on to that one. It'll come back in a place you won't expect.

2.3 Prove it from what you and I both accept

Around 300 BC, in Alexandria, Euclid wrote the *Elements*, and the thing to take from it is not the geometry. It's the method, and the method is the whole reason I trust anything in this book, so give me a paragraph on it. He starts with definitions ("a point is that which has no part"), five postulates (things he asks you to grant, like "a straight line can be drawn from any point to any point"), and five common notions (things nobody argues with, like "things equal to the same thing are equal to each other"). Then he derives everything else, four hundred and sixty-five propositions, from only those, in order, each meant to use nothing but what came before (he sometimes leans on things he never stated, and later geometers filled those in). Not "trust me." Not "it's well known." Here are the rules you and I both accept; now watch. If you've ever been in an argument that ended with "well, that's just your opinion," Euclid is the man who found a way out of that room, and I'm not embarrassed to say I learned the way out from someone dead for two thousand three hundred years. Four hundred and sixty-five times he does the same trick, and the trick is everything.

What is structure, to Euclid? Whatever the starting rules force. And when are two shapes the same? Here's the one you can do: cut out two paper triangles and lay one on the other. If they coincide, they're equal. That's his fourth common notion, and it's how he proves his fourth proposition, by picking up one triangle and setting it down on another. Sameness is coinciding under a move. Keep that; it's about to get very general, and I didn't see that coming either.

2.4 The statue and the bronze

Aristotle, Plato's student, pulled the forms down out of heaven and put them inside things, which was a bit rude to his teacher and also, I think, correct. His example, from the *Metaphysics*: a bronze statue is bronze plus a shape, and being made of bronze is no part of what it is to be a statue. The same shape in marble makes a different statue with the same form. Matter and form, and form is what the thing *is*.

Push on it. Is that true? Take anything on your desk. Could the same thing be made of something else? A coffee cup, sure. A sandwich, no; a sandwich made of wood is a sculpture of a sandwich, and if you've ever tried to eat one you know the difference matters. So the claim doesn't hold everywhere. It holds exactly where the material was never doing the work, and that's a sharper thing than Aristotle said: structure is what survives a change of material, in the places where the material didn't matter. Why do I care? Because the machine I keep mentioning and not describing gets built more than once, in more than one material, and I'd like it to be the same machine every time, and I don't get to just say it is. That's a promise I'll have to keep in Part IV (Section 15), and Aristotle is the one who told me what the promise means.

2.5 Where things are, without numbers

In 1679 Gottfried Leibniz, who co-invented calculus and had opinions about everything, wrote to the physicist Christiaan Huygens asking for something that didn't exist: a way to reason about *where* things are directly, the way algebra reasons about *how much*, without ever laying down coordinates. He called it the analysis of situation. Try it yourself. Describe where the furniture is in the room you're sitting in without using a single number. "The couch is against the wall. The table is between the couch and the window. The lamp is next to the chair." You just did it, and what you used was relations between places and nothing else. Huygens wrote back that November and was not impressed. It took roughly two hundred years for Leibniz's idea to become a field, and the field is called topology, and Huygens is remembered for other things.

What survives the push is the same thing that survived the square, coming at you from the other side. Position is a relation among places. The numbers you lay on top are a convenience, and they're a convenience you can take away and still have the room. I'll be honest about why this one landed for me: the thing I built keeps track of which things belong together, and I spent a long time thinking that was a list of numbers before I understood it was a relation that the numbers were merely naming. Leibniz had that in 1679. Huygens told him no. I'd like to think I'd have sided with Leibniz. I probably would have sided with the famous guy.

2.6 Naming a thing by saying when two are the same

In 1884, in a short book called *The Foundations of Arithmetic*, Gottlob Frege noticed something you can check in your head right now. "Line *a* is parallel to line *b*" and "the direction of *a* is the direction of *b*" say exactly the same thing. There is no fact in one that isn't in the other. But the second sentence has a new object in it, a *direction*, that

the first one doesn't. Where did it come from? From the sameness. Once you can say when two things count as the same, you've earned the right to name the thing they share. Frege used exactly this move to say what a number is: the number of things in a pile is whatever every pile you can match up with it, one to one, has in common. That's a residue with teeth: a sameness relation lets you name a new object.

Now push, because Frege pushed too hard, and it cost him. In his big system he also assumed that any condition you can state picks out a collection, the collection of everything that satisfies it. In June of 1902, a young Bertrand Russell wrote him a short letter. Consider the collection of all collections that are not members of themselves. Is it a member of itself? If it is, then by its own rule it isn't. If it isn't, then it qualifies, so it is. Do it in your head; it flips every time you look at it, like one of those drawings that's a vase and then two faces and then a vase. Frege's floor had a hole in it, and he knew it the day he opened the letter, and his second volume was already at the printer. The repair came from Ernst Zermelo in 1908 and it's the one mathematics still stands on: you may only cut a collection out of a collection you were *already given*. You don't get to sort "everything." You sort a pile. I'll tell you now that this rule shows up again, later, in a place where I had to obey it or fall through the same hole, and I'll point at it when the book gets there.

2.7 Choose the changes, keep what survives

By 1872 there were too many geometries. Euclid's, and ones where parallel lines meet, and ones with no such thing as distance, and nobody could say what made them all geometry rather than a pile of unrelated hobbies. Felix Klein, twenty-three years old and taking up a professorship at Erlangen, published an answer in a program paper that people still call the Erlangen program. Pick a set of allowed changes: rotations, stretches, whatever you decide. Then a geometry is the study of the properties that *stay the same* under those changes. Pick a different set of changes and you get a different geometry on the very same page. Euclid's is the one where you may slide and turn a shape but not stretch it. And here's the part I'd underline if this were my book, which it is: in Klein's scheme any group of changes can define a geometry. The choice is free. Nobody tells you which changes to allow.

You can do this one at your desk. Take any drawing and ask what's still true after you turn it upside down (lengths, angles), after you stretch it (not lengths anymore, but which lines cross which), after you crumple it (almost nothing but what touches what). Three answers, three structures, one drawing. The residue: structure is what survives a chosen set of changes, and "chosen" is doing real work in that sentence. There's no structure of a thing from nowhere. You have to say what you're allowed to do to it first. I didn't know that when I started, and I'd been saying "the structure of" things for years.

2.8 An invariance comes with a bill

In 1918 Emmy Noether, who wasn't allowed to hold a professorship because she was a woman and for a while lectured under a male colleague's name, proved something that physicists still treat as close to holy. Take a physical system whose laws can be written the way most of physics is written, as "of all the paths the system could take, it takes the one that makes a certain total smallest" (the total is called the action, and that's all I'll say about it here). For such a system, every continuous symmetry of the laws comes with a quantity that is conserved. If the laws don't care *where* you are, momentum is conserved. If they don't care *when* it is, energy is. A symmetry isn't decoration. It has a consequence downstream that you can measure with instruments. I can't hand you this one to do at the table; the proof needs calculus I'd have to build first and I'd lose you and probably me, and the "written as an action" condition is doing real work that I'm not going to unpack here. What I can hand you is the shape. Structure, once it's really there and the setting is the right kind, has consequences you can measure. I'll admit that's the sentence that made me think a machine could carry a bill for structure in the first place, and I'll also admit I don't yet know whether that hope was reasonable. Noether makes it reasonable to ask. She doesn't make it true, and nothing I build in this book is a system with an action.

2.9 All of mathematics as structures

Starting in the 1930s, a group of young French mathematicians began publishing a vast treatise under one made-up name, Nicolas Bourbaki, as if he were a single person. He wasn't. He also applied for membership in the American Mathematical Society and was turned down, which I find comforting. Their goal was to rebuild all of mathematics on one idea: a structure is a set of elements "whose nature has not been specified," together with one or more relations on them that satisfy stated axioms. They believed three kinds of structure were the mothers of all the rest: algebraic, order, and topological.

Push on it, and here's what the historian Leo Corry found when he went through the entire treatise looking for the place where this formal definition of structure actually does any work: it "plays no actual role in the presentation of theories within the treatise." Outside the chapter that defines it, almost nowhere. And I found no argument in their writing for why the mother structures number three. And the chapter on category theory, which they announced, never appeared. Why? The thing their definition left out was the *maps*, the ways of carrying a structure from one set to another, and it turned out the maps were where the weight was. So I keep two things from Bourbaki. The right thing: relations on elements whose nature doesn't matter, which is Aristotle's bronze in different words. And the warning: a structure stated without its maps won't carry the work. I learned that one the slow way, and the origin story you just read tells it.

2.10 A set with some named relations

What mathematical logic eventually settled on, with Alfred Tarski giving the field its name, “the theory of models,” in 1954, is almost boring, and the boring ones are the ones that last. A structure is a set, called the domain, plus whatever relations and functions a chosen vocabulary names on it. Pick the vocabulary (Klein again: you choose), and a structure is anything that gives those names a meaning. Now I want to say “the smallest one,” and the word “smallest” has no meaning until I say what I’m measuring, so here is the ruler: count the vocabulary. One name, taking one thing at a time, with two possible answers. That’s a domain with one yes-or-no property named on it. One question, put to each thing. Two honest limits, and I’d rather give them to you than have you find them. A vocabulary with one named constant (“this one, here”) is exactly as small by my ruler and it doesn’t sort anything, so “smallest that sorts” is the claim, not “smallest.” And one question only splits the domain into two groups when some things answer yes and some answer no; if they all agree there’s one group and nothing was sorted. With those limits said, and with the ruler said: by the count of names, the smallest structure that sorts anything is one question. Someone with a different ruler (fewest symbols to write down, say, or least information) might land somewhere else, and I’d want to see where. Keep it. It’s about to turn into the whole section.

2.11 The structure is in the arrows

In 1945 Samuel Eilenberg and Saunders Mac Lane published a paper that founded a field now called category theory, and they said in plain words that it was a continuation of Klein. The move: stop asking what a thing is, and ask how it’s carried. A category is some objects and some arrows between them, where arrows can be followed one after another and every object has a do-nothing arrow to itself, plus two rules that make the following dependable: following three arrows in a row gives the same result whether you pair up the first two or the last two, and the do-nothing arrow really does nothing when you tack it onto either end. That’s the whole definition, and I know it sounds like nothing. The two rules are the part people skip, and they’re the part you’ll watch me have to prove later. What a thing *is* is how it maps. You’ve already read how I came at this backwards, the way I come at everything: built by feel, in a rendering engine, as “digital DNA,” with the name and the founding paper turning up only after the thing already ran. For now the residue is the one Bourbaki missed: structure lives in the maps. Whether the thing I built honours that, you’ll be able to judge when I build the category in front of you, from scratch, later in this book.

2.12 Structure is the part that compresses

Play this one with me. I write “th” and you guess the next letter. You said “e,” and you were right. Now “the q”: you said “u.” Congratulations, you just measured the structure of English. In 1948 Claude Shannon measured it properly, in a paper that founded information theory, and found that about half of written English is “determined by the structure of the language.” Be careful what that says. It does not

say that every other letter is fixed by the ones before it; you just guessed “e” after “th” and you could have been wrong (think “than”). It says that, on average and over long stretches, about half of what’s written could be rebuilt from the rest: the text is roughly twice as long as the choices in it. And he said why that matters in one line that I’d frame if I framed things: the structure “allows one to make a saving in time.” The more you can predict, the less you have to read.

Thirty years later Jorma Rissanen turned that saving into a price. His idea is called the minimum description length principle, and it says: the best model of some data is the one that makes the total description shortest, where the total is the length of the model *plus* the length of the data once the model has explained what it can. A model is paid for in bits. It’s worth stating only if it saves more than it costs. Peter Grünwald, in a tutorial you can find online, adds the footnote that keeps this honest: the price of a model depends on the code you chose to write it in, so the price itself needs a principle behind it, and nobody hands you that principle for free.

That’s a lot of “length of” to swallow in one paragraph, so do one with me, with numbers small enough to check on the back of an envelope. Every number in it comes from a code I’m about to make up on the spot, and I’ll say every rule of the code out loud, because Grünwald’s footnote is the whole point: change the rules and every number moves.

Here’s the data. A row of thirty-two digits, each a 0 or a 1, and you and I agree in advance that the row is thirty-two long, so nobody has to spend anything saying so:

0101010101010101 0101010101010101

(The space is only there to rest your eyes.) Now the code. Every description starts with one digit that says which kind of description follows: 0 means “here’s the row, written out,” 1 means “here’s a pattern, then the exceptions to it.” Each digit is one yes-or-no answer, one bit, which is what Shannon’s unit is. Written out, the row costs that first digit plus the thirty-two digits themselves: thirty-three bits, and there’s no model in it at all. A pattern description goes like this. First five digits giving the length of a little block, anything from 1 to 32, because five yes-or-no answers can pick out one of thirty-two lengths. Then the block itself. The rule is “repeat the block until you have thirty-two digits.” Then five digits for how many places the row disagrees with the pattern (up to thirty-one, which is plenty here), and five more digits for each of those places, saying which of the thirty-two it is.

Do it for the row above. The block is 01, two digits long. So the model costs 1 (which kind) plus 5 (the block’s length) plus 2 (the block): eight bits. Then what’s left to say about the data once the model has said its piece: the row disagrees with “repeat 01” nowhere, so that’s five digits saying “zero exceptions” and nothing else. Five bits. Total, thirteen, against thirty-three for writing it out. The pattern pays for itself, with

twenty bits to spare. That's Rissanen's sentence happening on paper: a model is worth stating when it saves more than it costs.

Now spoil the row. Flip two of its digits anywhere you like. The model doesn't change, eight bits, but now the leftover says "two exceptions, here and here": five plus five plus five, fifteen. Total twenty-three. Still cheaper than thirty-three; the pattern still pays, it just pays less, because the data has more to say that the model can't. Flip four, and the total is $8 + 5 + 4 \times 5 = 33$, a dead tie. Flip five and it's thirty-eight, and now writing the row out is cheaper than explaining it, and by Rissanen's rule you shouldn't state the pattern at all, even though it's still sort of there if you squint. So "is there structure here?" has turned into a question with a number on each side, and a break-even point you can find yourself: somewhere between four and five spoiled digits, in this code.

And now change the code, because I promised you'd see the numbers move. Say I'd used six digits for each exception's place instead of five (wasteful, but nobody stopped me). Then four spoiled digits cost $8 + 5 + 4 \times 6 = 37$, and the tie is gone; the pattern stopped paying one spoiled digit sooner. Same row, same pattern, different verdict, and the only thing that changed was a convention I chose. That's Grünwald's footnote, and it's why I won't let you take any of those numbers anywhere else. They're the price of a description in one made-up code, a length of a written-down message, and nothing more. They are not the charge I opened this book with. That charge, when the book finally states it, is on one step a machine takes, not on how long anybody's description is, and if the two ever turn out to be related, that has to be shown, not assumed from both of them being counted in whole numbers.

Now push, because this is where "structure means information" gets born, and it's wrong, and it's an easy thing to believe. Flip a fair coin two hundred times and write down the string. Almost always, there's no shorter way to say that string than to say the whole thing. You can watch it happen in the little code above, cut to thirty-two digits: a coin-flip row disagrees with "repeat 01" in about sixteen places, which would cost eighty-odd bits to list, and even the one block guaranteed to fit with no exceptions, the whole row used as its own block, costs $1 + 5 + 32 + 5 = 43$. The written-out row at thirty-three wins, and no pattern earns its keep. Nikolai Vereshchagin and Paul Vitányi, in a 2004 paper on exactly this question, put it without mercy: every bit of a string like that is "meaningless information, random noise." Maximum information. Zero structure. So in this sense structure is the part that compresses, the part where the "th" tells you about the "e," and most information and most structure are different things. Hold that one tightly, because if you built something that charged for information, you'd be charging for the wrong thing, and for a while I wasn't sure which one I'd built.

And now notice that the word just changed meaning on you, and on me, and I'd rather point at it than let it slide. Shannon and Rissanen and Vereshchagin mean *regularity*:

a pattern that lets you say something shorter. Tarski, a few pages back, and Halmos, coming up, mean a set with named relations on it, and those two are not the same thing. Here's the test, and you can run it: number a thousand objects, flip a coin for each, heads goes in group one, tails in group two. That's a perfectly good sorting in the set theorist's sense, every object in exactly one group. And the list of who went where is a thousand coin flips, which is to say it compresses about as well as a thousand coin flips, which is to say not at all. Feed it to a code built like the little one, with the fields widened to fit a thousand, and you get the coin-flip row again, only longer: no block earns its keep, the written-out list wins, and by Rissanen's rule there's no pattern worth stating. By the set theorist's rule, the sorting *is* the structure, and it's every bit as much a sorting as "evens in one group, odds in the other," which compresses to almost nothing. Same kind of object, wildly different description lengths. So the two meanings don't even rank things the same way: one asks "is there a sorting?" and the answer is yes for both; the other asks "does a short rule say who went where?" and the answer is yes for evens and odds and no for the coins. Structure in the set theorist's sense, none in Rissanen's. I'll keep both meanings, and I'll try to say which one I'm using, and when I slip you'll know what to look for.

2.13 The four parts of every computer

In 1936 Alan Turing described the machine that every computer you've ever touched is a version of, and he did it before any of them existed. Write down its parts; there are four. A tape, divided into squares. A head that reads one square at a time and can write on it and move left or right. A finite list of states the head can be in. A table of rules: in this state, reading this symbol, do this. That's all, and two different things are true about it that get run together. Every other formal recipe for "a procedure" that anyone has written down since computes exactly what this one does; those are theorems, proved one pairing at a time, and Turing proved the first of them himself. That it also captures everything a person could ever carry out by rote, everything "computable" in the plain-English sense, is a claim, called the Church-Turing thesis, and nobody has proved it, because the plain-English side isn't a mathematical object. Turing himself kept those two apart in the 1936 paper, and so will I. There's a third claim that gets stacked on top, and it's a different kind again: the physical Church-Turing thesis, that every device you could build out of real stuff computes no more than this machine does. That one is a claim about physics, not mathematics. Nobody has built a device that breaks it. Your laptop, your phone, every server in every data center compute nothing a Turing machine can't; what differs between them is speed, space, energy, and cost. Hold on to that last word. Cost is where this book lives.

Now do the thing I did, and do it honestly. Look through those four parts for a place that says "these squares belong together." I couldn't find one. Any grouping has to live in the table, as a rule, or on the tape, as marks, which is to say as bytes like any other bytes. Maybe you can find a slot I missed. I'd want to know. I looked for a long time and I kept not finding it, and not finding something is weaker than proving it

isn't there, so I'll be careful about what I claim from that.

In 1972 David Parnas, a software engineer, wrote a short paper that every programmer should be made to read and most haven't, on how to cut a program into modules. He took one small program and cut it two ways. Both worked. But one was easy to change later and the other wasn't, and the second could be "much less efficient" if talking between modules cost anything. His point: a module is "a responsibility assignment," a decision about who answers for what, and how you cut decides what every later change costs. If you've ever maintained someone else's code, you've paid Parnas's bill, probably at two in the morning. What survives the push: a division is a commitment, not a caption. It changes what the work costs. It is not a label stuck on afterwards. I'd like to say the thing I built understood that from the beginning. It didn't. I did, eventually.

2.14 Laying them all down

Now lay every one of those on top of the others and see what shows through. Here are the seven things I think I see. Be clear about what this is: it's my overlay, the places where I see the sources agreeing, and not a thing the sources proved together. They didn't; some of them never met, and some of them disagree (Bourbaki left out the maps that Eilenberg and Mac Lane built everything on; Shannon's structure and Tarski's are two different words, as I just showed you). You've pushed on each source yourself by now, so you're in a position to tell me if I'm seeing things.

A list of seven slogans would be easy to nod at and impossible to check, so I'm going to carry one thing through all seven and let you watch each one add to the one before it. The thing is the square you drew for the Pythagoreans, side, diagonal and all. Draw a few more on the same sheet: one small, one big, the small one again but tilted, and one stretched out into a long rectangle. You'll want a pile.

Structure is about relations among things, not the things. The diagonal, the bronze, Leibniz's room, Bourbaki's unspecified elements, Tarski's named relations, the arrows. On your sheet, what makes a square a square was never the ink. It's that the four sides match and the corners are square, and that the diagonal stands to the side the way it does whether or not any number names it. Every one of those is a relation among parts. But on its own this first thing is too loose to use, because a sheet of paper is drowning in relations: this square is left of that one, this one is nearer the coffee ring. Which relations count?

Structure is what stays the same under a chosen set of changes. Euclid's triangles laid together, Klein's allowed moves, Noether's symmetry, Frege's direction that every parallel line shares. That's the answer the first thing was missing. The relations that count are the ones that survive the moves you've decided to allow. Slide the small square across the sheet, turn it, and the diagonal still stands to the side exactly as before; "left of the coffee ring" doesn't make it across. The second thing is a filter on the first.

The same structure can be carried by different stuff. The statue in marble. Two sets that answer the same vocabulary the same way. This is the second thing again with one more change allowed: swap the material. Cut the square out of cardboard, or lay it out in string on the table, and if the relations survive, it's the same square in every way you agreed to care about. Aristotle's bronze turns out to be one more thing you agreed not to look at, and the wooden sandwich is what happens when the material was never on that list.

The first act of structure is a sameness that sorts a pile you were already given. Euclid's first common notion, Frege's direction, Tarski's one question, and Russell's letter saying you must start from a pile. This is where the overlay stopped being a list for me and turned into a shape, so slow down here with me. Look at your pile of drawings and say: two of them count as the same when an allowed move carries one exactly onto the other. Watch what happens to the pile. It falls into groups. The small square and its tilted twin go together; the big one sits on its own; the rectangle sits on its own. And it falls cleanly, nothing half in one group and half in another, for a reason you can check. Klein's allowed moves always come with three guarantees: doing nothing counts as a move, every move can be undone, and two moves in a row is again a move. Those three are, word for word in different clothes, "everything is the same as itself," "if this matches that, then that matches this," and Euclid's first common notion, "things equal to the same thing are equal to each other." So the second thing, sameness under chosen moves, *is* a sorting of the pile the moment you ask which drawings it calls the same. Frege did it with lines and parallels. Tarski's one question does it in the crudest way there is, by cutting the pile in two. And the pile has to be there first, because a sameness has to be a sameness among something; that's Russell's letter, and Zermelo's repair.

Adding structure rules possibilities out. Every one of Euclid's postulates is a door closed. Half of English is already decided. A conserved quantity limits where a system can go. On your sheet: once the small square and its tilted twin are in one group, you've given up the right to answer any question the moves respect differently for the two of them. Tell me the side of one and the diagonal of the other isn't an open question any more. A sorting is a bundle of doors closed at once.

Held structure gets used without re-deriving it. Shannon's saving in time. Rissanen's compression. Parnas's one-module-at-a-time. Leibniz, who hoped his algebra of position would spare people a great many fruitless tries. This is the fifth thing paying out. The doors you closed are doors you never have to check again: know which group a drawing is in and you know everything the group shares without getting the ruler out, the same way the row of 01 never needed writing out. With one catch that I'll need a whole section for: it only pays if you kept the sorting somewhere you can read it back.

And structure is always relative to a chosen frame. Klein's group. The vocabulary.

The code you compress against. Parnas's criteria. Nobody gets to say "the" structure of a thing without saying from where, and I've caught myself doing it twice while writing this. On your sheet: allow stretching as one of the moves and the rectangle walks over and joins the squares; allow only slides and turns and it doesn't. Same pile, same ink, different groups, because I chose different moves. The groups were never in the paper. They were in the paper and the choice, together.

So here's how I think they hang together. Relations are what structure is made of. The chosen moves pick which relations count, and a change of material is just one more move. Asking which things the moves call the same sorts the pile. The sorting closes doors, and closed doors are what you get to skip later. And because the moves were chosen, all of it is relative. Read that way, the fourth thing sits in the middle: it's where the first three turn into something you can write down, and where the last three get their start.

That's my reason for starting the next section with a sorting, and I want to be exact about what kind of reason it is, because it's mine and not theirs. None of these people said "start with a partition." Klein would tell you to start with the moves, Eilenberg and Mac Lane with the arrows, Shannon with the probabilities. What I can actually point at is narrower. In the framings about when two things are the same shape or the same kind (Euclid laying one triangle on another, Klein's moves, Frege's directions, Tarski's question) there's a moment where somebody decides what counts as the same, and that decision, written down, is a sorting of a pile. In Shannon, Rissanen and Noether I can't find that moment as cleanly, and I'm not going to bend them until I do. So the overlay doesn't prove that a sorting comes first. It shows me where the framings I trust most keep putting their first foot down, and I'm putting mine there too.

I looked for a framing that denies any of the seven and didn't find one, which could mean there isn't one or could mean I didn't look where it lives. If you know of one, I'd like to see it.

2.15 The smallest piece

Now the sentence I wanted to say at the top and had no right to until here, and even here I have to split it in two, because half of it is a textbook fact and the other half is me. The piece of structure I'm going to start from is a partition.

The textbook half: a partition and an equivalence relation are the same object. That's Paul Halmos, *Naive Set Theory*, 1960, section seven, and you could read that section before dinner. Here it is in words. Take any pile of things. An equivalence relation on it is a way of saying "these two count as the same" that behaves the way sameness should: everything is the same as itself; if a is the same as b then b is the same as a ; and if a is the same as b and b the same as c , then a is the same as c . Every such relation sorts the pile into groups, the things that count as the same go together, and every sorting into groups hands you back a relation, "same group." Do one and then

the other and you land where you started, in both directions. So a partition and an equivalence relation are one object seen from two sides, and that's the fourth thing on the overlay, the sorting of a given pile, now with a name and a page number.

Here it is in the exact form, since this is the first place the book uses one. A *set* is a pile, $x \in X$ means “ x is in the pile X ,” and a *relation* on X is any rule $\sim$ that, for each pair of things, says whether $x \sim y$ holds. A few more marks show up below, and each is a word in disguise. $S_1, \dots, S_k$ are the buckets, numbered one to k . A cap, $S_j \cap S_l$, is “what the two buckets share,” a cup, $S_1 \cup \dots \cup S_k$, is “everything in any bucket,” and $\emptyset$ is the empty pile. And curly braces with a colon, $\{y \in X : y \sim x\}$, mean “the pile of every y in X for which $y \sim x$ holds”; the theorem writes that pile $[x]$, the things that count as the same as x .

Definition 2.1 (Equivalence relation, partition). A relation $\sim$ on a set X is an *equivalence relation* when it is reflexive ($x \sim x$), symmetric ($x \sim y$ implies $y \sim x$), and transitive ($x \sim y$ and $y \sim z$ imply $x \sim z$). A *partition* of X is a collection of non-empty subsets of X , the blocks, such that every member of X lies in exactly one block. Spelled out for blocks $S_1, \dots, S_k$: each S_j is non-empty, no two share a member ($S_j \cap S_l = \emptyset$ when $j \neq l$), and together they cover everything ($S_1 \cup \dots \cup S_k = X$).

Theorem 2.1 (Sameness and sorting are one object). *Every equivalence relation on X determines a partition, whose blocks are the classes $[x] = \{y \in X : y \sim x\}$; every partition determines an equivalence relation, “ x and y lie in the same block”; and the two constructions undo each other.*

Proof. The classes are non-empty ($x \in [x]$ by reflexivity) and cover X . If z lies in $[x]$ and in $[y]$, then $z \sim x$ and $z \sim y$, so $x \sim y$ by symmetry and transitivity, and then $[x] = [y]$ by transitivity again; so the blocks don't overlap. “Same block” is plainly reflexive, symmetric, and transitive. Going relation to partition to relation returns $x \sim y$ exactly when x and y share a class, which is the original relation; going partition to relation to partition returns the original blocks. $\square$

This is textbook mathematics (Halmos, section 7), and the book uses it as a foundation, not as a contribution.

And the smallest structure that sorts, by my ruler in Tarski's setup, the one question, is the same object a third time. Put a yes-or-no question to every thing in the pile. The ones that say yes are one group and the ones that say no are the other. Going the other way, a two-group partition becomes a question the moment you point at one group and call it “yes.” The only thing a question has that the partition doesn't is which group got the label. So the set theorist's first step and the logician's first step are the same step, and nobody has to pick a side, which is a relief, because it's an argument you can lose with yourself for a long time.

That's the textbook half. Now mine, and hold it looser, because Halmos says nothing about "smallest" or "first" and I'd be lying if I let his page number carry my sentence. In what I'm going to build, the order goes: a pile, then groups in the pile, then whatever connects one group to another, and the groups come first because a question needs something to be about and a connection needs somewhere to land. That's an order for *my* construction. It is not a law that every account of structure has to start by sorting. You could start with an ordering instead ("this one comes before that one"), or with arrows between the things themselves and no groups at all, and people do, and nothing I've shown you rules that out. So the honest sentence is: a partition is where I chose to start, it is the smallest start by the ruler I gave you, and I chose it because it's the first thing that can be right or wrong about a pile and the last thing I could take away without everything I build on it coming down. If you have a smaller start by a ruler you can state, I want to see it. It would change this book.

2.16 Buckets

So here's the smallest piece, done once, for real. Take everything in front of you and sort it into buckets: nothing in two buckets, nothing left on the floor. That's the whole of the definition. It's the thing you do with a kindergartner and a pile of blocks, and I am aware that I've just marched you past Pythagoras, Plato, Euclid, Frege and Noether to arrive at a pile of blocks. Keep the blocks anyway. They're about to cost you something.

Ten blocks. Red ones in one bucket, square ones in another, everything else in a third. Now a red square turns up, as they do, and you've got your first crisis: it belongs in two buckets and the rule says one. So this isn't a partition yet, and you have to decide which bucket wins. That decision is the whole thing. A partition is a pile of small rulings like that one, and once you've made them all, the pile is holding a fact it wasn't holding a minute ago. The formal version says the same thing with fewer words and more squiggles, and you've already seen it: it's the definition of a partition a few pages back (2.1). Non-empty buckets, nothing in two, nothing on the floor.

Here's the part that isn't kindergarten, and it's the part I had to sit with longest. Why would that fact cost anything? Watch what it does to one question. Before the buckets, "are these two in the same group?" means working it out: go find the first one, go find the second one, apply the rule to each, compare. After the buckets, it's a glance: read the first one's bucket, read the second one's bucket, done. And the glance costs the same when the pile goes from ten blocks to a billion. That's Shannon's saving in time and Parnas's one-module-at-a-time, the sixth thing on the overlay, showing up in a kindergarten.

Now push on it, because that paragraph sells more than it should, and I want you to see where. The partition didn't give you the glance. A *table* gave you the glance: a written-down line for each block saying which bucket it went in. The partition is the fact; the table is one way of holding the fact so it can be read back fast, and you had to

build the table, and you have to store it, and it has to be the kind of thing you can read in one look (a bucket number, say, and not a list of every block in the bucket that you'd have to search through again). Take the table away and keep the partition and you're back to working it out. And sometimes there was never a search to save: if the rule is "evens here, odds there," you can answer "same group?" by looking at the last digit of each, no buckets needed. So the honest residue is smaller than the sentence I wanted: holding a sorting in a form you can read back is worth something, and building that form cost something, and those are already two different events. Building the table, storing it, checking that it's right, and deciding to *rely* on it are four different things. The question this book is about lives in the gap between the third and the fourth, and nothing in the kindergarten has shown you yet that there's a charge in that gap. It's shown you there might be a place for one.

So who, in everything I read, says what the bill is? I found three answers that come close, and I'll be exact about how close, because this is the edge of what I know. Rolf Landauer, in 1961, and Charles Bennett after him, say that any step which takes two possible situations and makes them one, erasing a bit or merging two paths a computation could have taken, has to push some disorder out into the world, which in a real machine means heat. That's not a metaphor; it's the one place where a fact about information turns into a fact about your electricity bill, and it's been measured. Is sending blocks to buckets a step of that kind? Push, because it depends on what you keep. If you keep each block and write its bucket next to it, you've lost nothing; you can read the block right back off the pair, and Bennett's whole point in his 1973 paper was that a computation which keeps its inputs can be run without any such step at all. If instead you throw the block away and keep only the bucket label, then two different blocks can leave the same label behind, and that is a many-to-one step, the kind Landauer charges for. Both are things people call "sorting into buckets." Only the second loses a distinction. So the bucket by itself doesn't hand you a Landauer bill; the *forgetting* does, when there is forgetting, and whether a given machine forgets is a question about that machine, which has to be answered on the page and not waved at here. Rissanen says a model is paid for in the bits it takes to state, and Grünwald says the price depends on the code. Parnas says a bad cut is paid for later, at two in the morning. And that's where it stops. I couldn't find anyone who says that the moment you *vouch* for a grouping, the moment you stand behind "these belong together" instead of just arranging them that way, has a price of its own. That could mean nobody says it. It could mean I didn't find the person who does. I honestly can't tell you which.

So that's where I am at the end of one word. Holding a sorting in readable form is worth something and cost something, forgetting where things came from costs heat, and standing behind a sorting is a different act from either, and nobody I found has priced it. You can walk back through all of that and tell me where it breaks. Maybe there's no such bill. Maybe I'm about to spend a book chasing a charge that nobody

owes, which would be an extremely me thing to do. The next section is me trying to find out whether that charge is real on any machine at all, not mine, any machine, and if the answer comes back no, you should stop reading there. I would have stopped writing there too.

Part II

The axiom

3 A record and a toll

I left you with a question I couldn't answer: something pays for the glance, and I couldn't find anyone who says the moment of vouching for a grouping has a price of its own. Before I go looking for that price in anything complicated, I want to try the opposite. Build the smallest thing that could possibly carry such a bill, on paper, with a pencil, write the bill into its rules, and see what follows from it. If nothing follows in something with five parts, nothing is going to follow in something with five thousand, and I'd rather find that out on page one than on page four hundred.

But first the word, because I've got a new one and I don't trust it either. Record. Receipt. Certificate. Vouch. People have been making marks that other people are supposed to rely on for a very long time, and I wanted to know what they thought they were doing, and what it cost them, and whether any of them ever said the thing I need said. So, same deal as before: here's what I could reach, pushed on with you, and I'll tell you where the search ran out.

3.1 Clay, seals, and seven witnesses

Start about five thousand years ago, in the Near East, with people who counted sheep with little clay tokens. To send a count across a distance without the courier changing it, they put the tokens inside a hollow clay ball, pressed the shapes of the tokens onto the outside while the clay was soft, and sealed it. The archaeologist Denise Schmandt-Besserat has spent a career on these; the ball is called a bulla. Here's the push, and you can do it with a cookie tin: the thing that makes it a record isn't the tokens, it's that the outside shows what's inside and you have to break it to get in. Opening it destroys it. Tampering shows. That's a real idea and it will show up again in a form you won't expect: a record is worth something only if it can't be silently undone.

Now law, which is where people get serious about who's allowed to say what happened. The book of Deuteronomy, chapter nineteen: nobody is convicted on the word of one witness; it takes "two or three." The Roman jurist Gaius, writing in the second century, says a will the praetor will honour is one sealed by seven witnesses. Seven. Why did they care that much? Because one person can lie, and a will moves a fortune,

and the dead man can't object. So the record isn't the paper. It's the number of independent people who have put their own seal, their own name, their own risk, next to the claim. Try it: write "paid five dollars, signed A" on a napkin and ask a friend to forge it. Takes them ten seconds. Now require two other people to initial it at the moment of payment and each keep a copy. Your forger needs two accomplices or has to match two copies held in two other pockets. The claim didn't change. What stands behind it did.

Which brings me to seals, and the medieval historian Michael Clanchy, whose book is called *From Memory to Written Record*. England between the Norman Conquest and the early 1300s was learning to trust writing instead of trusting people who remembered, and the lesson was painful: seals were forged, charters were forged, and historians today have a whole craft of telling real charters from fake ones because the fakes are so many. A seal is only as good as whoever holds the stamp. Keep that one too, because in 2011 a certificate authority called DigiNotar found out, at the cost of its existence, that a mathematically perfect signature is worth exactly what the custody of the key is worth, and intruders had taken control of the machines that signed. Nine hundred years, same failure. I find that oddly comforting.

3.2 Two columns and a stamp

Here's the one I'd never have guessed mattered. Bookkeeping. Around 1300, the ledger of a Florentine company called Farolfi, kept at its branch in Provence, shows the earliest complete example anyone has found of a trick merchants were working out: write every transaction twice, once where the money came from and once where it went, and the two columns have to balance. Genoa's public accounts of 1340 do it. And in 1494 a friar named Luca Pacioli printed the method in a mathematics book, and it spread. That's double-entry bookkeeping, and it's easy to take it for just a tidy habit.

Do it for one day. Keep two columns of every dollar that moves, and total them at night. Now have a friend secretly change one entry in one column. The totals disagree; you catch it. Now have them change the matching entry in both columns. The totals agree. You've caught nothing. Push on that and you've found what double entry actually is: a trap for slips, not for careful liars. Pacioli knew. The scholar Forrester reads him as reporting that in many places the books were taken to the local consul to be authenticated before a single entry went in. The stamp is a witness again, from outside. The two columns are the discipline; the stamp is the claim that someone other than you will stand behind the fact that these books, and not some others, are the books. Keep both halves separate in your head. One is a method. The other is a commitment.

And then I found the place where this got turned into a motto. The Royal Society, founded in London in 1660 and still running, picked as its motto *Nullius in verba*: take nobody's word for it. The historian Steven Shapin wrote a paper about what that

meant in practice, and the answer is more interesting than the slogan. Robert Boyle, pumping the air out of a glass globe to see what happens to a candle or a bird, had three problems: the machine, the report, and the witnesses. He solved the third by having gentlemen present who'd sign their names, and the second by writing the report so richly that a reader who wasn't there could picture the scene and count as a witness anyway. Shapin calls that virtual witnessing, and he notes, dryly, that the air pump was very expensive, seventeenth-century big science. So there *was* a cost. It just wasn't a cost of declaring the fact. It was the cost of the instrument and of getting enough people into the room. Hold on to that distinction. It's going to turn out to be the whole question.

3.3 A demon, and the step that pays

Now the strangest entry, and the one that gets closest. In 1867 the physicist James Clerk Maxwell, in a letter to a friend, imagined a "very observant and neat-fingered being" sitting at a tiny door between two halves of a box of gas, letting fast molecules through one way and slow ones the other, so that one side heats up and the other cools with no work done, which is supposed to be impossible. The being got called Maxwell's demon, and physicists have been arguing about it for a hundred and fifty years, which should tell you something.

Szilard's one-molecule engine, in 1929, made information part of the accounting. It is tempting to put the whole bill on finding out which side the molecule is on. Bennett's analysis separates that measurement from restoring the memory for another round. A measurement onto a blank record can, in principle, be thermodynamically reversible; resetting an unknown record is a different operation. That does not make a real measuring device free to build or run. It says that learning and resetting do not have the same unavoidable entropy cost. I'm about to pick a step to charge, so I need that distinction before I do.

Bennett's version of the rule, in a 2003 paper you can read, is that what must cost is "the erasure of a bit or the merging of two computation paths." Erasing, or taking two possible situations and making them one. And in the same paper he lists copying something onto a blank place to write as an operation that can be thermodynamically reversible. So, in the one place in all of this where a record is priced by physics, the unavoidable erasure charge belongs to *forgetting* the unknown record, rather than to every act of making a record. Read that twice. It's not what I expected either.

I'll put the rest of the lineage quickly, because it all points the same way. Regulators who make brokers keep records on media that can't be rewritten (a rule the SEC had for decades) are pricing the hardware, not the act. Haber and Stornetta in 1991 chained time-stamped records together with hashes so that changing an old entry breaks every entry after it (and Ralph Merkle's hash tree of 1979 lets one short value vouch for a great many records); try it with five lines and a made-up sum, change line two, and watch every number below it move. That makes a change *visible*. It doesn't

make it cost. Proof-of-work, which Dwork and Naor proposed in 1992 to make junk mail expensive and which Bitcoin later chained, is the one framing where a record genuinely costs, and what it charges for is a search: how many tries it took to find a lucky number. An empty block costs the same work as a full one. So the work prices the search, not the moment anything becomes certified. Floyd and Hoare, in 1967 and 1969, attached claims to points in a program (“whenever you get here, this is true”), and the claim is backed by a proof that’s paid for off the machine. And the certificates of complexity theory, which go back to Stephen Cook’s 1971 work on NP, are strings designed to be cheap to check: hand me the certificate and I can confirm the claim quickly. Whether they’re also expensive to *find* is the famous open question of the field, P versus NP, and nobody knows; what’s settled is only the cheap half. That’s the opposite of a price on vouching: the whole design goal is that checking a claim costs nearly nothing.

3.4 What they all agree on, and who they rule out

Lay them down, and this time what shows through is not one thing. It’s five different jobs that the word “record” has been doing, and I’d been letting one word carry all five. Each has a corpse next to it, and the corpses are different too.

Durability: the record can’t be silently undone. The bulla, the write-once disk, the hash chain, Bennett’s card that has to be wiped. The corpse is the editable log, and the company that kept its debts off the books: Enron hid its debt in side entities and its auditor shredded the paper, and those books recorded nothing. But notice this job doesn’t need a witness at all, only a copy: a hash chain nobody signed is still hard to change quietly, so long as somebody else holds the latest link, because a chain with no copy anywhere can be rewritten from end to end and recomputed, and nobody would know.

Authenticity: the record came from who it says it came from. Seals, the signature only a key-holder can make, the consul’s stamp. The corpse is the forged charter and DigiNotar. This job is about custody of a stamp, and it’s a different job from durability; a perfectly authentic entry can still be erased, and a perfectly durable one can still be a forgery.

Evidence: enough independent people stand behind the claim that lying is expensive. Two witnesses, seven seals, Boyle’s gentlemen. It rules out anonymous assertion, and it rules out “the universe keeps the receipt,” which I mention because people say it: information spread into the environment is not a record anyone can read or rely on. This one is about numbers of people and their risk, not about paper.

Accounting: the record checks itself for slips. The two columns. And here’s a thing it’s easy to get wrong: the second column is *not* an outside witness. It’s you, checking you. It catches a dropped digit; it doesn’t catch a careful liar, as you saw. It belongs on this list as its own job, not as a cousin of the seven seals.

Cheap checking: a record whose correctness can be confirmed for much less than it cost to make. Cook's certificates, the signature, the hash. This one isn't universal either, and I thought it was. Whether checking needs to be cheaper than making depends on who you're worried about and how much a wrong record costs you; plenty of records are checked by redoing the whole thing, and that's fine when the thing is small.

Now the thing they don't agree on, because none of them says it. Not one of these jobs is pricing the act of certifying. Bennett prices merging and erasing, which a certifying step might or might not be. Proof-of-work prices a search the designer chose. Bookkeeping prices a discipline. Witnesses and seals price showing up and guarding the stamp. I looked, with everything I had, and that's where it ran out. If the bill exists, nobody in this lineage wrote it down, and that's either because it isn't there or because the one person who did write it down is on a shelf I never found. So I'm going to have to try it myself, on the smallest thing I can build, and let you watch. And I'll have to be honest about which of the five jobs the small thing even attempts, because it's not going to be all of them.

3.5 The smallest thing that could carry the bill

Take a sheet of paper. Draw two boxes and label them NO and YES. That's the reading, a single yes-or-no question asked of the state, and I'm going to be careful about what the question is, because after five thousand years of seals and witnesses the word "vouch" is carrying a lot and this toy carries almost none of it. The question is only: "has the designated flag been set." Not "is some claim about a grouping true." Not "did anyone check." There is no claim in this toy, no evidence, no checker, nobody standing behind anything. There is a flag. Whether a flag can ever be made to *mean* that something was checked and can be relied on is a real question, and it's not this section's; I'll build toward it later, and until then please read YES as "the flag is up" and nothing more. Where the state is: which box you're in. That's the whole state, one bit. Then draw a line under the boxes for tally marks. The marks are not part of the state; they're the running total you keep beside it as you play, the way you'd keep score beside a board game.

Now give yourself three moves. STAY: you don't move, no mark. WORK: you don't move either (it's a stand-in for every ordinary step a machine takes), no mark. VOUCH: from NO you move to YES; from YES you stay in YES; either way you make one tally mark, because in this toy the price belongs to the move's name and not to where you were standing. Play. Start in NO, make any sequence of moves you like, and stop in YES. Count the marks. Now try to reach YES with zero marks. You can't, and you can see exactly why: the only move that crosses from NO to YES is VOUCH, and VOUCH makes a mark. Now erase the mark from VOUCH's rule and play again. YES for free, every time. That's the entire content of what I'm about to call a rule, and I want you to see how little it is before I dress it up: I've decided that the move which crosses the line makes a mark, and nothing else about the machine forces that. It's a choice. The next section is about whether I had to make it.

Here is the same toy, stated once, carefully, because this is the one place in the book where I state it and everything later points back here. The thing on your paper has five parts.

A set of states. (Here: NO and YES, and nothing else.)

A set of move names. (STAY, WORK, VOUCH.)

A step: for each state and each move name, where you land.

A cost for each move name: a whole number of marks, zero or more. (Zero, zero, one.)

A reading: for each state, yes or no.

That's it. Add the one rule below and I'll call it a certification system; in the checked record the rule is a sixth field, so nothing counts as one without it. And I want to be exact about what's not in it. There's no counter stored inside the state that's forbidden from going down; the marks are the side tally from the board game, and the tally only ever grows because the costs are whole numbers, zero or more. (A machine could keep the total inside its state too; that's an extra part, not one of the five, and [Section 6](#) builds it.) There's no window, no observer who sees only part of the state; that's a later question and it gets its own section. And the cost depends only on the move's name, not on where you are when you make it, which is a real simplification and I'll say so when it matters.

One rule, which I'll call the toll. For every state and every move: if the reading is NO before the move and YES after it, the move's cost is at least one mark. That's all. It says nothing about any other move. It says nothing about how big a mark is in anything you could measure. One mark is one mark; what a mark is worth, and why the smallest charge is one and not two or a half, and whether I had any right to decide that, come next, and I'm not sure of all of it.

And one theorem, which is the only thing the toy actually proves, and you've already proved it by playing. Take any run of moves that starts in a NO state and ends in a YES state. Then the marks it made add up to at least one. Here's the argument in words, and it's short enough that you can hold the whole thing. Walk the run move by move, reading the box at each step. It read NO at the start and YES at the end, and every move either leaves the reading alone or flips it, so at some move it went from NO to YES. The toll says that move cost at least one. Every other move cost zero or more. Add them up: at least one. Notice what the argument did not need. It didn't need the reading to *stay* YES afterward; if a later move flipped it back to NO and another flipped it to YES again, that's two tolls, not a problem. It didn't need the machine to be finite. It didn't need to know what any move does besides what the reading says before and after. It needed exactly one thing: the toll.

That's the whole theorem. Here it is once more in the exact form, because from here on the book carries two layers side by side: what I mean, in words, and what's stated

and checked, in symbols you can hold me to. The symbols are few. $\mathbb{N}$ is the whole numbers 0, 1, 2, and so on. S and I are just names for “the set of states” and “the set of move names.” Writing $\text{step} : S \times I \rightarrow S$ means “hand it a state and a move name and it hands back a state.” A *run* is a list of move names, $t = [i_1, \dots, i_n]$, played from a start s_0 , one after another. The double arrow $\implies$ reads “then”: the thing on its left forces the thing on its right. And “by induction on the run,” in the proof, is the formal name for the way you’d check it by hand: settle the empty run, then show that one more move at the front can’t break what you’ve settled.

Definition 3.1 (Certification system). A *certification system* consists of a set S of states, a set I of move names, a step function $\text{step} : S \times I \rightarrow S$, a cost $\text{cost} : I \rightarrow \mathbb{N}$, and a reading $\text{cert} : S \rightarrow \{\text{no}, \text{yes}\}$, subject to one rule, the *toll*: for every state s and move i ,

$$\text{cert}(s) = \text{no} \text{ and } \text{cert}(\text{step}(s, i)) = \text{yes} \implies \text{cost}(i) \geq 1.$$

The run of $t = [i_1, \dots, i_n]$ from s_0 ends at $\text{run}(t, s_0)$, the result of applying the steps in order, and its total cost is $\text{cost}(t) = \text{cost}(i_1) + \dots + \text{cost}(i_n)$, with the empty run costing 0.

Defined as the record `CertificationSystem` (`UniversalCertificationCost.v`).

Theorem 3.1 (The universal floor). *In any certification system, every run that starts at a state reading no and ends at a state reading yes has total cost at least 1: if $\text{cert}(s_0) = \text{no}$ and $\text{cert}(\text{run}(t, s_0)) = \text{yes}$, then $\text{cost}(t) \geq 1$.*

Proof. By induction on the run. If t is empty, the run ends where it starts, at a state reading no, so the hypothesis “ends at yes” can’t hold and there’s nothing to prove. Otherwise $t = i :: t'$ (the first move, then the rest); let $s_1 = \text{step}(s_0, i)$. If $\text{cert}(s_1) = \text{yes}$, the first move is a crossing, the toll gives $\text{cost}(i) \geq 1$, and the rest cost at least 0, so $\text{cost}(t) \geq 1$. If $\text{cert}(s_1) = \text{no}$, the rest of the run starts at no and ends at yes, so by the induction hypothesis $\text{cost}(t') \geq 1$, and $\text{cost}(t) = \text{cost}(i) + \text{cost}(t') \geq 1$. Nothing else is used: not finiteness, not that the reading stays yes, not what any move does beyond the reading before and after. $\square$

Where to check it: the record’s six fields are `cs_state`, `cs_instr`, `cs_step`, `cs_cost`, `cs_cert`, and the toll `cs_cert_costs`; the theorem is `universal_nfi_any_substrate`, in the same file; and the checked proof is the induction above, step for step. These names identify the companion formalization. The argument is the induction on this page. One honest edge: the theorem does not assume that a YES state exists. If no state ever reads YES, the theorem is true for no runs at all, which is a mathematician’s way of saying it’s true and useless. The toy on your paper has a YES box, so it isn’t useless there. It would be natural to assume the system has to have at

least one NO state and one YES state; the checked statement has no such clause, and that's on purpose, because a theorem that holds for no runs is still a theorem and the toy is where you supply the runs.

3.6 What this does and doesn't buy

I want to be very plain about how small this is, because the temptation is to look at the word "theorem" and feel something has been established about the world. Nothing has. What's been established is that *if* you adopt the toll, *then* the flag can't be raised for free, in any system with those five parts that obeys it, no matter what the states and moves are. The "if" is doing all the work. I chose the rule. A reasonable person could ask why, and the only honest answer right now is "because that's the thing I wanted to be true," which is not a reason.

Notice also what it doesn't ask about how strong the thing is, in either direction. A system is *Turing-complete* when it can imitate a Turing machine, and no stronger than one when a Turing machine can imitate it. The theorem asks neither. Stronger than a Turing machine, weaker, dead even: none of it reaches the floor. Bring the five parts, charge the toll on the crossing, and you're in scope. The floor doesn't check what you can compute, because it was never about what you can compute. It's about what a move is allowed to do for free.

And the other side of that: nothing is in scope just for being a computation. A Turing machine's run isn't an instance of this theorem because it's a run. It becomes one when somebody supplies all five parts for it, says which states read yes, and proves the toll for that cost. Until then the theorem says nothing about it, and I won't pretend it does.

The same goes for a system with no reading at all. If nothing in it answers "is the flag up," then nothing in it can say, from the inside, that anything is entitled to be relied on. Whatever entitlement it ends up with gets carried in from outside, by somebody who signs their name to it, and nothing in the system priced it. Smuggled entitlement isn't entitlement, no matter whose name is on it.

So why bother with the toy at all? Not because the lineage was looking for it. The bulla was after durability, the seven seals after evidence, the consul's stamp after authenticity, the two columns after catching slips; none of them was trying to put a price on the moment a flag goes up, and I shouldn't pretend they were. The reason is narrower. Everywhere in that lineage where somebody *did* locate a cost, it sat on a step next to the flag and not on the flag: Szilard on the measurement, Bennett on the wiping, proof-of-work on the search, Boyle on the instrument and the room. The toy is the first thing I could build where the cost sits on the crossing itself, and it proves that if you price that step, the floor follows everywhere, for any machine with the five parts. That's not an answer. It's the question made exact enough to be wrong. And of the five jobs a record does, the toy attempts none of them. Not even durability: the flag can be lowered by any move you care to write, and the marks that don't go away are kept by *you*, the scorekeeper, beside the board, not by the machine. What the

toy has is a scorekeeper's receipt for the crossing. Whether the machine can be made to keep that receipt itself, where a step could reach in and change it, is Section 6's question. Authenticity, evidence, accounting, cheap checking: none of them is in here, and I'll say so again when it's tempting to forget.

And here's what I'll owe you by the end. Whether the toll is a choice or a necessity: the next section, and I'll tell you now that the answer surprised me. What a mark is worth in anything physical: later, and only partly. What an observer who sees only a window can and cannot recover, including whether they can even tell which step paid: its own section. And whether a full-size machine, the kind you could actually run a program on, can be made to obey the toll without cheating, and what that machine looks like, and what it costs to vouch for structure in it: that waits for Part IV, on purpose, and takes most of the rest of the book, and I'm still not sure I pulled it off.

For now you have a sheet of paper with two boxes, three moves, one rule you were handed without a reason, and one theorem you proved yourself in about a minute. Don't let go of it. It's the smallest version of everything that follows, and if it's wrong here, it's wrong everywhere.

4 Why the toll isn't arbitrary

I handed you a rule without a reason. The move that raises the flag makes a mark, because I said so. If that's all it is, this book is a long way of describing a preference, and you should treat it the way you'd treat someone telling you their favourite colour is a law of nature. So this section is me trying to find out whether I had to pick it. I'll tell you the shape of the answer up front, because I don't want you reading forty paragraphs to find out whether there's a point: on a machine with a finite memory, where the flag once raised stays up, the step that raises it has to forget something, and forgetting is the one step Landauer's principle puts a price on. Three "ifs" in that sentence. Every one of them is a door you can walk out of, and I'm going to show you each door, because the doors are where the actual content is.

4.1 A closed room with no spare chair

Do this one with real objects, because the whole section rests on it and it's about ninety seconds of work. Draw a box on the table and call it the certified room. Put three chairs inside the box and one chair outside, by the door. Four people, one per chair; the people are the states your machine could be in, and the three inside are the ones whose flag is already up. One move of the machine moves everybody at once: each person stands up and goes to the chair the rule sends them to. Two rules. First: nobody inside the room may end up outside, because the flag, once up, stays up. Second: the person by the door must end up inside, because this move is the one that raises their flag.

Now try to finish the move with one person in each chair.

You can't. Three people start inside and have to stay inside, so they take the three inside chairs between them, with nobody left out, because nobody may leave. The person from the door has to come in. There is no fourth chair inside. Somebody shares. Two of the machine's possible states land on one state, and after the move you can no longer tell, from where everyone is sitting, which of the two it was. The move forgot something.

That's the whole argument, and I want to name the piece of mathematics that did the work, because it's old and it's not mine. If you put more things into fewer boxes than there are things, some box gets two. People call it the pigeonhole principle; the earliest written version I could find is a 1622 book by a Jesuit named Jean Leurechon, who used it to argue that two people must have exactly the same number of hairs, and the name comes from Dirichlet calling it the drawer principle in 1834. It's the kind of fact a child accepts and a mathematician spends a career finding new uses for. Here the use is: a step that forgets nothing is a one-to-one shuffle, a one-to-one shuffle of a finite set of chairs hits every chair, so the inside chairs are all taken by insiders, so there's no room for the newcomer. Finite. Permanent. Flip. Those three and nothing else, and the conclusion is "two states became one."

The checked version of the chairs is a theorem a few lines down, and it needs one word first. "Injective" is the mathematician's word for "forgets nothing": a step is injective when two different starts always give two different ends. The theorem says exactly what the chairs said and takes exactly the three premises: a duplicate-free list that names every state (finite), a reading that no step ever turns off (permanent), and one state that reads NO before the step and YES after it (the flip). There is no price in it anywhere. It is pure counting. I'll come back to what counts as physics, but notice that nothing so far has been.

Here are the words, exactly. For this whole section the setting is the five parts from Section 3 without the toll: states S , move names I , a step, a reading, and (later) a cost.

Definition 4.1 (Finite, permanent, injective). The machine is *finite* when there is a list of states with no repeats that contains every state. The reading is *permanent* when no move ever turns it from yes back to no: for all s and i , $\text{cert}(s) = \text{yes}$ implies $\text{cert}(\text{step}(s, i)) = \text{yes}$. A move i is *injective* (forgets nothing) when different starts give different ends: $\text{step}(a, i) = \text{step}(b, i)$ implies $a = b$. A move that is not injective *merges*.

Defined as `permanent` and `step_injective` (PermanentCertification.v).

Theorem 4.1 (A permanent flip merges). *On a finite machine with a permanent reading, if $\text{cert}(s) = \text{no}$ and $\text{cert}(\text{step}(s, i)) = \text{yes}$, then the move i merges.*

Proof. Suppose i were injective. Let C be the list of states that read yes; it is finite

and has no repeats. By permanence, $\text{step}(\cdot, i)$ sends every member of C into C . Being injective, it sends the members of C to members of C with no two landing on the same one, so its images are as many as C itself, and a list of that many distinct members of C is all of C . So the yes-state $\text{step}(s, i)$ equals $\text{step}(c, i)$ for some c in C , and injectivity gives $s = c$, so s reads yes. It reads no. That's the contradiction, and it's the chairs. $\square$

Checked as `permanent_flip_is_not_injective` (PermanentCertification.v). The proof there uses the standard-library fact that a duplicate-free list of n members inside a list of n is the whole list, which is the pigeonhole in the form Leurechon would recognise.

4.2 Now charge for forgetting, and the toll falls out

Here's the step I'd been circling since the kindergarten buckets. In the last section I told you Bennett's rule as he states it: what has to cost is erasing a bit or merging two paths a computation could have taken. I didn't ask you to believe it. I'm not going to ask you now either. I'm going to write it down as a rule you can accept or refuse, in the smallest form that does any work: *any move that forgets something costs at least one mark*. Any move that is not injective has cost at least one. That's it. Note what it does not say. It says nothing about certifying. It says nothing about flags. It's a price on forgetting and only on forgetting.

Put the two together. The room says: on a finite machine with a permanent flag, the move that raises the flag forgets something. The price says: a move that forgets something costs at least one. So the move that raises the flag costs at least one. That is the toll, the rule I handed you without a reason, and now it has one. The proof is two lines: the first theorem says the flipping move merges, the premise says merges are priced, done. I'll keep saying "premise," because that's what it is, and because the next section of this book is about what happened to some very smart people who put a price where a premise goes and then treated it as a law of nature, and about half of this one is me trying not to do that.

Premise 4.2 (Merge pricing). Every move that merges costs at least one: for all i , if i is not injective then $\text{cost}(i) \geq 1$. (Checked name: `merging_steps_priced`.)

Theorem 4.2 (The toll from finiteness, permanence, and merge pricing). *On a finite machine with a permanent reading whose cost satisfies merge pricing, every move that turns the reading from no to yes costs at least one. That is, the machine with that cost is a certification system in the sense of Definition 3.1, and the universal floor (Theorem 3.1) applies to it.*

Proof. A move that flips the reading merges, by Theorem 4.1; merge pricing charges it at least one. $\square$

Checked as `a2_from_merging_price_and_permanence`; the packaging of such

a machine as a certification system is `certification_system_from_merging_price`, and the run-level floor for it is `permanent_certification_trace_floor` (`PermanentCertification.v`).

One more way to see the same thing, because it answers a rival I'd otherwise owe you. The rival says: don't price certification, price *erasure*, and keep an honest flag that reports when something was erased. Fine; but a system can set its certificate at cost zero with its erasure flag reporting nothing, exactly because the certifying move merged two states and the flag didn't say so (`commit_without_erasure_system_is_not_honest`). Make the erasure flag honest, so that it reports every move that forgets, and on a finite machine with a flag that stays up the erasure law sends the same bill the toll does (`honest_erasure_accounting_implies_a2`). The toll isn't a rival to honest erasure accounting. It's what honest erasure accounting says about the certifying move.

So, is the toll arbitrary? Here's the honest version. The toll is a consequence of three things: a finite memory, a flag that stays up, and a charge on forgetting. If you accept all three, you don't get to refuse the toll; it's forced. If you refuse any one of them, the toll goes back to being my preference. So the question "did I have to pick it" has turned into three smaller questions, which is what progress usually looks like: not an answer, but a better set of things to argue about. Let me take the three doors in order, and for each one, build the thing that walks out through it, because a premise you can't see the counterexample to is a premise you're taking on faith.

4.3 Door one: keep everything, and nothing ever has to merge

Take a strip of paper that never runs out of boxes. The machine's whole state is the strip. Each move writes a 1 in the next empty box, and the flag is up as soon as any box has a 1 in it. Check the three premises. Permanent: once there's a 1, there's always a 1. Flip: the empty strip reads NO, and after one move it reads YES. Forgets nothing: every move can be undone by erasing the last 1, so two different strips never become the same strip. So here is a machine that raises a flag, keeps it up, and never merges anything. The room argument has no grip on it, and the toll doesn't follow. The checked version is `unbounded_history_escapes`, and its state is a list of bits of any length, which is the formal way of saying the strip has no end.

What walked out the door is the finiteness premise. Now cut the strip to eight boxes and run it for real: eight boxes, nine moves. On the ninth move there's nowhere to write. To keep going you have to reuse a box, and the moment you reuse a box two different histories end on the same strip, and you're back in the room with a shared chair. This is exactly the move Bennett made in 1973, when he showed that any computation can be done without forgetting if you let it keep a copy of everything it did; Landauer, back in 1961, had already written that for a program that doesn't stop, the space to hold every intermediate result "cannot be there." The history escape is real. It costs you a memory that never fills. Every machine you can buy has a memory

that fills.

One thing I have to say plainly, because the file is more careful than I'd like to be. The theorem shows the strip machine has the other two properties and forgets nothing. It does not contain a sentence saying "and this machine is infinite"; that's something you and I can see about a list of unbounded length, but no Coq statement says it. And it contains no cost at all, so "certifies for free" is an inference I'm drawing (a move that forgets nothing is allowed to cost zero under the price premise), not something written in the theorem. I'd rather mark where the words outrun the file than let them run.

4.4 Door two: let the flag come back down

Take one coin. The rule is: turn it over. Heads is YES, tails is NO. Every move is undoable (turn it over again), so nothing is forgotten. Tails goes to heads: that's a flip, NO to YES. And heads goes to tails: the same move takes the flag back down somewhere else. Two states, finite as it gets, and the move that raises the flag forgets nothing, so a schedule that charges only for forgetting is allowed to charge it nothing. The file states the one move's bijection and its on-then-off behaviour (`revocable_certificate_escapes`); that the coin is finite and that the price may be zero are read off the picture, not written in the theorem. The thing that walked out the door is permanence.

Now here's a fact that I think is sharper than the premise it replaces, and it was the thing in this section that surprised me. Drop permanence entirely. Keep only finiteness. Take any move that raises the flag anywhere. Then one of two things is true: the move forgets something, or the same move lowers the flag somewhere else. There's no third option, and the proof below is a search: go through the finitely many YES states looking for one this move turns to NO. Find one, and that's the revocation; find none, and the move is permanent at least for itself, which is all the room argument needs. So a flag that goes up for free is a flag the same move can take back. A flag that nothing takes back is paid for. You can't have a free raise that stays raised, not on any finite machine, not with any price schedule that charges for forgetting. That's a narrower sentence than "every certificate costs," and I'll take narrow and true over broad and mine.

Theorem 4.3 (Merge or revoke). *On a finite machine (no permanence assumed), if $\text{cert}(s) = \text{no}$ and $\text{cert}(\text{step}(s, i)) = \text{yes}$, then either i merges, or there is a state t with $\text{cert}(t) = \text{yes}$ and $\text{cert}(\text{step}(t, i)) = \text{no}$.*

Proof. Check every yes-state on the finite list for one that i sends to no. If there is one, that is the second alternative. If there is none, then i keeps every yes-state at yes, which is permanence for the single move i ; the argument of Theorem 4.1 uses permanence only for the move it is about, so it applies, and i merges. $\square$

Checked as `flip_merges_or_revokes`, with the single-move form of the room argument as `permanent_at_flip_is_not_injective` (PermanentRecordPricing.v).

I'll give you the three-card version too, because it shows the converse fails and I don't want you to think the picture is tidier than it is. Three cards, T0, T1, T2, with a star on T1. The rule: T0 goes to T1, T1 goes to T2, T2 goes to T1. T0 gains the star (a flip). T1 loses it (a revocation). T0 and T2 both land on T1 (a merge). So this move is priced, by the forgetting rule, and yet it writes nothing permanent; it takes a star away in the same breath. "The priced moves are exactly the permanent records" is false, and the file proves it false with these three cards. What's true is one direction: every permanent raise is a priced move. The other direction has the three cards standing in it. And "priced" here has an exact meaning worth one sentence: a move is *forced to be priced* when every schedule that charges for forgetting charges it, and that holds exactly when the move forgets something, with no finiteness needed at all, only a way to tell two move names apart.

Definition 4.3 (Forced to be priced). A move i is *forced to be priced* when every cost function satisfying merge pricing (Premise 4.2) charges i at least one.

Defined as `forced_priced` (PermanentRecordPricing.v).

Theorem 4.4 (Forced price is merging). *If any two move names can be decided equal or different, then a move is forced to be priced exactly when it merges. No finiteness is needed.*

Proof. If i merges, merge pricing charges it at least one, whatever the cost function. If i does not merge, define the cost that is 0 on i and 1 on every other move (this is where telling move names apart is used). It satisfies merge pricing, since every merging move is some move other than i and costs one, and it charges i nothing, so i is not forced. $\square$

Checked as `forced_priced_iff_merges` (PermanentRecordPricing.v).

Corollary 4.5 (Permanent records are forced). *On a finite machine, a move that never turns the reading off and turns it on somewhere is forced to be priced. (By Theorem 4.3 it merges; by Theorem 4.4 it is forced.) The converse fails: the three cards are forced, because they merge, and they revoke.*

Checked as `forced_priced_iff_merges, permanent_record_write_is_forced_priced`, and the three-card counterexample `forced_price_without_permanent_record` (PermanentRecordPricing.v).

4.5 Door three: forget for free

The third door is the easiest to describe and the hardest to argue about, so I'll describe it and then spend the rest of the section on the argument. Take two sheets of paper, one blank and one stamped. One move: stamp whatever you're handed. Finite, two states. Permanent: stamped stays stamped. Flip: blank goes to stamped. Forgets: both sheets end up as the same stamped sheet. Everything the room argument needs is here, and the move merges. Now set the price of that move to zero. The toll fails, and you can watch it fail: the flag went up for nothing. That's `free_merge_escapes`, and what walked out the door is the charge on forgetting. (The same file shows that a price of one on that move satisfies all three premises, `priced_reset_satisfies_premises`, so the premises can hold together; door three is about whether they must.) Nothing in counting stops you from setting that price to zero. Only a claim about the world does. The nearest such claim is Landauer's, and I have to be exact about the gap between the two: the premise as written charges *every* forgetting move, whether or not you knew which state you were in, and Landauer's own heat, as the coin under the cup below will show, is owed only for what you didn't know. So the premise is Landauer's claim taken in its worst case over what the machine might be holding, and that's the version the counting needs.

Here is what Landauer said, in 1961, as exactly as I can put it after reading the paper rather than the summaries. A logical operation with no single-valued inverse (a step that forgets, in this book's words) must produce heat, "typically of the order of kT " per operation, where k is Boltzmann's constant and T is the temperature; and for the specific case of resetting one bit whose value you don't know, the minimum is kT times the natural log of 2, about $0.69 kT$. I want you to hear two things in that. First, "of the order of": he was not handing out a number to three decimals. Second, and this surprised me, he did not measure anything. The paper is argument, with models. He writes that he can "show, or at least very strongly suggest" the conclusion, and that his reasoning gives "no guarantee that this minimum is in fact achievable." That's the founding paper of the whole subject, and it's a careful man saying "I think this is right and here's why." I like him more for it.

Was it measured, eventually? Yes, and only recently. In 2012 a group at the École Normale Supérieure in Lyon, Antoine Bérut and Sergio Ciliberto's lab working with Eric Lutz, trapped a single glass bead, two millionths of a metre across, in a double well made of laser light, and used the two wells as the two values of a bit. Then they erased the bit, by lowering the barrier, tilting the trap, and raising it again, many times over, slower and slower, and measured the heat each time. The average heat came down toward Landauer's $kT \ln 2$ and sat there as the erasure got slower. On average. Single runs went under it; the bound is about the average, not any one erasure, and if you only succeed in erasing half the time you owe nothing, which the paper's formula shows and which I think is the most honest line in it. Others have done it since with different devices. So: predicted in 1961 by argument, seen in a laboratory fifty-one

years later.

And is it settled? No, and I'd be lying by omission if I left that out, because the people who doubt it are not cranks. John Norton, a philosopher of physics, has argued since 2005 that the usual proofs of the principle quietly assume what they're trying to prove, and that the kinds of process they assume, combined, could be made to break the very law the principle is supposed to guard. Charles Bennett, who did more than anyone to build the subject, rated Earman and Norton's 1999 form of this objection the one of "greatest merit," and then disagreed with it. Others replied to Norton; Norton replied back. The Stanford Encyclopedia entry on the topic, written by a philosopher of physics who works on exactly this, ends without a verdict. I've read what I could of both sides and I'm not qualified to referee them, and I'm not going to pretend the laboratory settles a question about what the principle means in general, because measuring one bead doesn't do that.

So here's what I did with it, and I think it's the only honest thing to do. The charge on forgetting is written into the checked work as a *named premise*. It's a definition the theorems take as an assumption, and nowhere is it proved as a general fact, because it isn't provable from counting; for particular machines with particular price lists it's checked (the stamp at cost one, above), which is a different thing. If Norton turns out to be right, every theorem in this section still holds; they just have one premise fewer that the world agrees to. If Landauer's side turns out to be right, the premise is physics. Either way, nothing in this section says a device does anything until someone measures the device. The counting is mine to prove. The physics is a door I've labelled and left open.

4.6 How much: the price grows with the squeeze

The toll says "at least one mark." That's a floor for raising one flag. The room argument says more if you let it. Say there are m people already inside and k more come in through the door in one move. (That's a different k from the one in Landauer's kT ; when the two meet, below, Boltzmann's constant gets a little B hung under it, k_B , so you can tell it from a head count.) All m plus k of them end up in m chairs. That's a squeeze, by a factor of $(m + k)$ over m , and if you accept a slightly stronger version of the forgetting price, that each mark pays for at most one *halving* of the number of distinguishable states, then the move costs at least the number of halvings in that squeeze, which is the base-two logarithm of $(m + k)/m$.

Back to door three's stamp, with four sheets this time: three blank, one stamped, and the stamp-everything move. Four states go to one. Before the move, to say which sheet you had took two yes-or-no questions (is it one of the first two? then which of those two?); after the move, it takes none. Two bits gone. Two halvings. So that move costs at least two marks, and two marks is enough: a cost of exactly two satisfies the halving rule (both facts are checked for these four sheets, `quad_stamp_needs_two` and `quad_stamp_cost_two_is_priced`). The general statement is $m + k$ at most

2 to the cost times m , and next to it in the file sits a logarithm form that rounds both logarithms up and is weaker for it (three states and one newcomer round to the same two bits on both sides, and the rounded form says nothing). The halving premise, like the one before it, is a definition used as an assumption. The units are halvings, not joules. To turn a halving into heat you multiply by $k_B T \ln 2$, and that multiplication is the physics, not the counting.

Exactly, with one new piece of notation: for a list D of states, write $|D|$ for how many it has, and $\text{img}_i(D)$ for the number of *different* states the move i sends them to.

Premise 4.4 (Halving price). Each mark pays for at most one halving: for every move i and every list D of states with no repeats, $|D| \leq 2^{\text{cost}(i)} \cdot \text{img}_i(D)$. (Checked name: `compression_priced`. It implies merge pricing, since a merge sends two different states to one and so needs $\text{cost}(i) \geq 1$.)

Theorem 4.6 (The squeeze bound). *On a finite machine with a permanent reading and a cost satisfying the halving price, let m be the number of states that read yes, and let a move i turn k different no-states to yes. Then*

$$m + k \leq 2^{\text{cost}(i)} \cdot m, \quad \text{equivalently} \quad \text{cost}(i) \geq \log_2 \frac{m + k}{m}.$$

Proof. Put the m yes-states and the k newcomers in one list; it has $m + k$ different members. By permanence every yes-state lands on a yes-state, and by hypothesis every newcomer does too, so all $m + k$ land among the m yes-states: $\text{img}_i \leq m$. The halving price gives $m + k \leq 2^{\text{cost}(i)} \cdot \text{img}_i \leq 2^{\text{cost}(i)} \cdot m$. $\square$

The four sheets are the case $m = 1, k = 3$, where the bound is reached. With $k = 1$ the bound says $\text{cost}(i) \geq 1$, which is the toll again (`a2_from_compression_price_and_permanence`). Checked as `permanent_flips_compression_bound`, with the rounded-logarithm form `permanent_flips_log_bound` and the counting-only cousin `permanent_flips_collapse_at_least` (at least k states collapse, with no price premise at all).

4.7 The same bill, written in bits, and when it's zero

There's one more way to say all this, and I include it because it's the way physicists say it and because it hands you the one case where the bill is exactly zero, which I find more instructive than any case where it isn't.

Claude Shannon, in 1948, gave a number to "how much you don't know": if something could be in any of n states and you have no idea which, your uncertainty is log base two of n bits, and that's the most it can be for n states. If you push your uncertainty through a step that forgets nothing, it stays the same; through a step that merges two states you might have been in, it drops. Now run the room. Before the move,

you might be in any of the $m + k$ states in play. After it, you're in one of m . So your uncertainty afterward is at most $\log_2$ of m , whatever you believed before. If you started with no idea at all, every one of the $m + k$ states equally likely, your uncertainty was exactly $\log_2$ of $(m + k)$, and the move removed at least $\log_2$ of $(m + k)/m$ bits: the same number as the halving count, now in Shannon's units. Landauer's principle, in the form physicists write it, then says a step that removes that many bits from the state gives off at least $k_B T \ln 2$ of heat per bit. Put the two together and the flip gives off heat, and the exact statements below say how much and under which side conditions, one of which (the temperature has to be taken as zero or above) is easy to skip and shouldn't be.

Exactly, with the notation built first. A *spread* (probabilists say *distribution*) is a way of giving each state a chance $p(x) \geq 0$, with the chances adding to 1. Its Shannon entropy, in bits, is

$$H(p) = - \sum_{x: p(x) > 0} p(x) \log_2 p(x),$$

where the big sigma means "add up, over every state x with a chance above zero," and each term is a state's chance times the bits it would take to single that state out. That comes to $\log_2 n$ when n states share the chance evenly and less when the chance is lopsided. Pushing a spread through a move i gives the spread of where you land: $q(y) = \sum_{x: \text{step}(x,i)=y} p(x)$. The bits the move *removes* are $H(p) - H(q)$.

Start with the half of the claim that costs nothing, Bennett's half. A move that forgets nothing sends each state somewhere different, so the chances just get carried to new addresses, one for one. Nothing piles up and nothing spreads out. The uncertainty is the same number before and after.

Theorem 4.7 (A move that forgets nothing removes no bits). *Let the states be finite, listed once each, and let f send states to states with $f(a) = f(b)$ only when $a = b$. Then for every assignment of chances p , the pushed spread q has $H(q) = H(p)$.*

Proof. An injective map on a finite list of distinct states is a reshuffle of that list: every state is hit exactly once. So $q(f(x)) = p(x)$ for every x , and the sum defining $H(q)$ is the sum defining $H(p)$ with its terms in a different order. $\square$

Checked as `step_entropy_invariant_if_injective` (PermanentCertificationEntropy.v). That's why a price on removed bits can leave every reversible move free, which is the shape the toll needs: the bill lands only where something was forgotten.

Theorem 4.8 (Entropy after a permanent flip). *On a finite machine with a permanent reading, let a move i turn $k \geq 1$ different no-states yes, beside m yes-states. For any spread p whose chance lies on those $m + k$ states, the spread after the move has $H(q) \leq \log_2 m$. For the even spread on the $m + k$ states, the move removes at least $\log_2 \frac{m+k}{m}$ bits.*

Proof. After the move, all the chance sits on the m yes-states (permanence for the old ones, the hypothesis for the new). There is at least one yes-state: the image of a state that flips. Thus $m \geq 1$. Write $q_1, \dots, q_r$ for the positive output probabilities, where $r \leq m$ and their sum is 1.

For $y > 0$, $\ln y \leq y - 1$: the derivative of $y - 1 - \ln y$ is $1 - 1/y$, so its minimum is 0 at $y = 1$. Apply this with $y = 1/(mq_j)$, multiply by q_j , and add:

$$-\ln m - \sum_{j=1}^r q_j \ln q_j \leq \sum_{j=1}^r (1/m - q_j) = r/m - 1 \leq 0.$$

Dividing by $\ln 2 > 0$ gives $H(q) \leq \log_2 m$. The even input spread has $m + k$ probabilities each $1/(m + k)$, hence $H(p) = \log_2(m + k)$. Subtracting the output bound gives the claimed loss $\log_2((m + k)/m)$. $\square$

Checked as `permanent_step_entropy_ceiling` and, for the even spread, `permanent_flip_uniform_entropy_drop` (`PermanentCertificationEntropy.v`).

Premise 4.5 (Landauer’s principle, as a premise). A step that lowers the entropy of the machine’s state by ΔH bits, at temperature T , gives off heat $Q \geq k_B T \ln 2 \cdot \Delta H$, where k_B is Boltzmann’s constant. (Checked name: `landauer_heat`, with $k_B T$ as one variable.)

Corollary 4.9 (Heat floor of a permanent flip). *Under Premise 4.5, with $k_B T \geq 0$, the flip of Theorem 4.8 from the even spread gives off at least $k_B T \ln \frac{m+k}{m}$; with $k_B T > 0$ that is strictly positive.*

Proof. Theorem 4.8 gives an entropy loss of at least $\log_2((m + k)/m)$ bits. Multiply by the nonnegative factor $k_B T \ln 2$ and use Premise 4.5. The identity $\ln 2 \log_2 x = \ln x$ gives the displayed floor. Here $m > 0$ and $k > 0$, so $(m + k)/m > 1$ and its logarithm is positive; if $k_B T > 0$, the heat floor is strictly positive too. $\square$

Checked as `permanent_step_entropy_ceiling`, `permanent_flip_uniform_entropy_drop`, `entropy_le_log_support`, `permanent_flip_heat_floor`, and `permanent_flip_heat_positive` (`PermanentCertificationEntropy.v`).

How big is that? Take the smallest case, one state flipping in beside one that was already yes, so $m = k = 1$ and the floor is $k_B T \ln 2$. At room temperature, about 300 kelvin, that’s roughly three zeptojoules: three thousandths of a billionth of a billionth of a joule. That’s about as small as a bill gets and still be a bill. And it’s a bill only if you grant the premise; nothing on this page measured anything.

The even spread is the easy case, and you might worry the heat depends on picking it. It doesn’t. Any spread that gives a real chance to every yes-state and to the state that

flips will do.

Corollary 4.10 (Positive heat for any full spread). *On a finite machine with a permanent reading, let move i turn a no-state s yes. Take any spread p that gives positive chance to s and to every yes-state. Under Premise 4.5 with $k_B T > 0$, the heat the move gives off is strictly positive.*

Proof. After the move, s and the yes-states all land among the yes-states, and there is one more of them than there are yes-states, so two distinct states with positive chance land on the same state. That merge removes a positive number of bits (Theorem 4.13 below), and a positive temperature times a positive number of bits is positive heat. $\square$

Checked as `permanent_flip_full_support_entropy_drop_positive` and `permanent_flip_full_support_heat_positive` (PermanentCertificationEntropy.v).

There's a third road to the toll here too, and it's the same shape as the merge road, with bits in place of marks for forgetting.

Definition 4.6 (Entropy-priced). *On a finite machine, a cost is *entropy-priced* when, for every move i and every spread p , the cost of i is at least the bits the move removes: $\text{cost}(i) \geq H(p) - H(q)$, where q is p pushed through i .*

Theorem 4.11 (The toll from entropy pricing). *On a finite machine with a permanent reading and an entropy-priced cost, every move that turns the reading from no to yes costs at least one. So the machine with that cost is a certification system (Definition 3.1), and every run from a no-state to a yes-state costs at least one.*

Proof. Let move i turn s yes, and let m be the number of yes-states; $m \geq 1$, since $\text{step}(s, i)$ is one. Take the even spread on the m yes-states and s . By Theorem 4.8 with $k = 1$, the move removes at least $\log_2 \frac{m+1}{m} > 0$ bits, so the cost of i is a whole number above a positive number, which makes it at least one. The run-level floor is the universal floor (Theorem 3.1) applied to that system. $\square$

Checked as `a2_from_entropy_price_and_permanence`, with the packaging `certification_system_from_entropy_price` and the run-level floor `entropy_priced_trace_floor` (PermanentCertificationEntropy.v).

Now the zero. Put a coin under a cup without looking. Rule: set it heads up. You might have had tails; now you have heads; one bit of what you didn't know is gone, and that's the bit the principle charges for. Now lift the cup, see heads, put the cup back, and apply the rule again. Nothing you didn't know changed. No bit was lost. And the principle, applied to a state you already knew, demands nothing: the premise

holds exactly when the heat is zero or more, which is to say it sets no floor at all. The principle charges for *uncertainty removed*, not for the shape of the move. This is the generalized form of Landauer’s principle, worked out by Maroney in 2009 and Sagawa in 2014, and Landauer himself had the seed of it in 1961: a row of bits all known to be one can be reset with no entropy change and no heat.

Theorem 4.12 (A known state owes nothing). *On a finite machine, if the spread puts all its chance on one state s , then any move removes zero bits, and Premise 4.5 for that move holds exactly when $Q \geq 0$.*

Proof. A spread on one state has entropy 0, and so does its image, which is again on one state. So $\Delta H = 0$ and the premise reads $Q \geq 0$. $\square$

Checked as `known_state_step_removes_no_entropy` and `known_state_flip_forces_no_heat` (PermanentCertificationEntropy.v).

Be careful with that one, because it’s easy to overread. “A known state owes nothing” does not mean a stamping machine run on a known-blank sheet gives off no heat. A fixed device that merges states dissipates on known input too; Bennett calls that wasteful, because entropy goes out into the room with nothing taken from the data. What the theorem says is only that the *premise* sets no floor there. The floor comes from what you didn’t know. That’s a sentence about the bill, not about the device. And notice that the toll still charges the known-state stamp one mark, because the toll is a rule about the *move*, and the move doesn’t know what the observer knows. That’s the worst-case gap from door three showing up again, and it’s the price of having a rule a machine can follow without asking who’s watching.

Here’s the general case, and it’s the one that shows you where the bits go. Split the bill state by state. Each state x you might be in carries its chance $p(x)$ along to where it lands, $\text{step}(x, i)$, and finds whatever other chance landed there too. If nobody else landed there, it owes nothing. If others did, it owes the difference between how crowded its landing spot is and how much of the crowd it brought.

Theorem 4.13 (The bits removed, one term per state). *On a finite machine, for any move i and any chances $p(x) \geq 0$, with q the pushed spread,*

$$H(p) - H(q) = \sum_{x: p(x) > 0} p(x) (\log_2 q(\text{step}(x, i)) - \log_2 p(x)).$$

Every term is zero or more, so a move never adds bits. If two different states $x \neq x'$, both with positive chance, land on the same state, the sum is strictly positive.

Proof. Group the sum for $H(q)$ by where things land: the chance $q(y)$ at a landing spot is the sum of the $p(x)$ that land there, so $H(q) = - \sum_{x: p(x) > 0} p(x) \log_2 q(\text{step}(x, i))$.

Subtract from $H(p)$ term by term. Each term is non-negative because the landing spot holds at least what x brought, $q(\text{step}(x, i)) \geq p(x)$, and the logarithm keeps order. If x and x' share a landing spot, that spot holds at least $p(x) + p(x') > p(x)$, so the term for x is strictly positive. $\square$

Checked as `entropy_drop_as_point_sum`, `entropy_drop_nonneg`, and `entropy_drop_pos_of_support_merge` (PermanentCertificationEntropy.v).

4.8 What you're entitled to, and what you're not

Let me total it: what have you earned the right to conclude, and what is still a door?

You've earned this: on any machine with finitely many states, a move that raises a flag either forgets something or takes a flag down somewhere else. Pure counting, no physics, no premise beyond finiteness, and the half of it where nothing comes down is the four chairs. You've earned this: on a finite machine, if a flag, once up, stays up, then raising it forgets something, and a move that raises k states collapses at least k of them. Counting again. And you've earned this: if forgetting is charged for, the toll follows, and if forgetting is charged per halving, the size of the toll follows too; each of those derivations hands back a system with the five parts and the toll, so Section 3's floor applies to it as it stands. That's the rule from Section 3, derived rather than declared, with every premise named, for any finite machine with a flag that stays up.

Here's what you haven't earned, and I'd rather list it than have you assume it. You haven't earned that forgetting costs anything in the world; that's Landauer, it's measured for beads and argued over for the general case, and in the checked work it's an assumption with a name. You haven't earned that any machine you'd actually want to build is of this kind, either, and there are two easy ways to miss. A machine that keeps its running total inside its own state, as a number with no ceiling, has infinitely many states, so the finite premise fails for it as a whole. And a price list that charges the step that raises the flag, but lets some other step merge two states for nothing, raises the flag honestly and still doesn't charge for forgetting in general, so the merge-pricing premise fails for it. Either one puts a machine outside these theorems as stated, though a finite piece of it can still sit inside. The machine I build later misses in both ways, and Part IV says exactly which finite piece of it is a machine of this kind (Section 23). And you haven't earned anything about *what* the flag means. Four chairs don't know what they're certifying. The bill is for raising a flag that stays up. Whether raising that flag ever amounts to standing behind a structural fact, the "vouching" I keep promising and keep not delivering, is still the same open question it was in Section 3, and it has moved exactly this far: if that flag stays up, it was paid for.

One last thing, because it should nag at you. The toll is forced by the premises, but the premises were chosen, and "permanent" in particular was chosen by me. Why should a certificate stay up? Why not let it come down, pay nothing, and be honest about it? That's a fair question, and the three-card machine is the start of an answer:

a flag that comes down for free is a flag the same move takes back, and a record that can be silently taken back is the thing the bulla and the hash chain and the write-once disk were all built against. “Permanent” here is exactly that: the flag can’t be lowered. The toy in Section 3 didn’t have it; the lineage in Section 3 was built around it; I chose it. Whether that was the right job to pick out of the five is something you can argue with me about, and I’d want you to. The next section is about a different door entirely: not whether forgetting costs, but whether *finding out* does, because the demon’s whole trick was that it knew which way the molecule was going, and I owe you a straight answer on whether knowing is free.

5 Learning is free, committing is not

I ended the last section owing you an answer about knowing. The bill so far sits on forgetting: a flag that stays up was paid for because raising it merged two states. But there’s a thing that looks like the opposite of forgetting and feels like it ought to cost more, not less. Finding out. Every time a machine rules out a possibility, it knows more than it did. If forgetting a bit costs, does learning one cost too? There’s a strong feeling about this, and the feeling is wrong, and the way it’s wrong is the content of this section, so here’s the feeling first: learning is the expensive part. It’s a common feeling, and most physicists had it too. It’s why the demon took a hundred and fifteen years to exorcise.

5.1 A small being at a small door

You met Maxwell’s demon in Section 3, in one paragraph. It gets a whole section here, because the step it seemed to pay for and the step it actually pays for are what this section is about.

In December 1867 James Clerk Maxwell, who had published his equations for electricity and magnetism two years before and had as good a claim as anyone to being the most important physicist alive, wrote a letter to his friend Peter Tait describing a problem he couldn’t get rid of. Take a box of gas at one temperature, with a wall down the middle and a tiny door in the wall. Now imagine, he said, a “very observant and neat-fingered being” sitting at the door, who can see individual molecules coming. When a fast one approaches from the left, it opens the door and lets it through. When a slow one approaches from the right, same. Otherwise it keeps the door shut. It never pushes anything; it only opens and closes a frictionless door at the right moments. After a while the right side is hot and the left side is cold, and you’ve got a temperature difference you could run an engine off, made out of nothing but *looking*. That’s supposed to be impossible. It’s the thing the second law of thermodynamics says can’t happen, and the second law is the law physicists are least willing to give up; Eddington said that a theory found to contradict it has nothing for it but to collapse in deepest humiliation, and nobody has had to test that yet.

Maxwell put it in his textbook in 1871, and it got named Maxwell’s demon, and

physicists spent the next century trying to find where the being was cheating. I want you to feel the pull of the obvious answer, because I felt it and so did most of them: the being has to *see* the molecules. Seeing costs. Case closed. In 1929 Leo Szilard built a one-molecule version of the box and did the arithmetic, and said the second law survives if finding out which side the molecule is on costs at least as much entropy as the being gains. In 1951 Léon Brillouin made it concrete: to see a molecule you have to bounce light off it, and bouncing a photon costs at least $kT \ln 2$, Landauer's number before Landauer, and there was your toll on looking. For thirty years that was the answer. Information costs because getting it costs.

That does not establish a universal charge on measurement. An optical device can spend energy without showing that every possible way of making a record must do so.

5.2 Bennett moves the bill

In 1982 Charles Bennett, who'd already shown in 1973 that computation can be done without forgetting, took the demon apart one step at a time. The being looks at a molecule and writes down which side it's on. Is writing that down a merge? Picture the being's notepad, which starts blank. After looking, the notepad says "left" or "right." Two possible worlds before (molecule left, pad blank; molecule right, pad blank). Two possible worlds after (left, "left"; right, "right"). Nothing merged. Two in, two out, and you can read the before off the after. Copying what you see onto a blank page forgets nothing, and a step that forgets nothing is a step Landauer doesn't charge for. Brillouin had priced one particular way of looking (bouncing light) and then said all looking costs. Bennett showed a way of looking that doesn't.

So where is the bill? The being has one notepad and a lot of molecules. After the first molecule the page is full. To do the next one it has to wipe the page. And *wiping* takes two possible worlds (pad says "left"; pad says "right") to one (pad blank). That's the merge. That's the step with no way back. The demon pays, every cycle, not for seeing but for making room to see again, and the amount it pays is at least what it gained, and the second law sleeps soundly. That's the rule from his 2003 notes I quoted in Section 3: erasing and merging cost, and copying onto a blank place to write doesn't.

Section 3 separated making a record from resetting it. Now you can see exactly which step and why it was tempting, because looking *feels* like the active part. It's the thing the being does. Wiping feels like housekeeping. The housekeeping is where the money goes. If there's a sentence in this section I'd want you to carry out of it, that's the one, and I didn't write it, Bennett did.

5.3 Two different things called "narrowing"

Here is what I had to get straight in my own head before any of this made sense, and I'll give it to you as a picture because that's how I eventually got it.

Draw the machine's possible states as a row of dots. Now draw a second row above it:

the dots that *you*, standing outside, still think it might be in. Those are two different rows. The first is a fact about the machine. The second is a fact about you, about what you’ve seen and what you’ve ruled out. Both rows can shrink. They shrink for different reasons, and only one of them costs.

The machine’s row shrinks when two of its states land on the same state. That’s a merge. That’s what the price charges. Under the halving rule from Section 4, a run can’t shrink the machine’s own row by more halvings than it paid for. That’s the theorem just below, and it’s the Section 4 squeeze bound run forward along a whole program instead of one step. No new idea; the same chairs.

Exactly. Write $\lceil \log_2 n \rceil$ for “log base two of n , rounded up,” the number of yes-or-no questions it takes to pick one of n things. For a run t and a list D of starting states, let $\text{img}_t(D)$ be the set of states the run can end in, starting from somewhere in D , and write $|\text{img}_t(D)|$ for how many there are (in Section 4 the symbol for one move already meant the count; here I need the set and its size both). Let $\text{cost}(t)$ be the run’s total cost.

Theorem 5.1 (The machine’s own narrowing is priced). *If the cost satisfies the halving price (Premise 4.4), then for every run t and every list D of starting states with no repeats,*

$$\lceil \log_2 |D| \rceil \leq \text{cost}(t) + \lceil \log_2 |\text{img}_t(D)| \rceil.$$

Proof. By induction on the run. A single move of cost c shrinks a list with no repeats by at most a factor 2^c (that’s the halving price), which costs at most c in rounded logarithms; the states reachable after the rest of the run from the shrunk set are exactly those reachable after the whole run from the original. Add the costs. $\square$

Checked as `run_narrowing_priced_log` (KnowledgeNarrowing.v). The run itself is the witness; nothing else is handed in. That will matter in Section 8, and I’ll say why at the end of this section.

Your row shrinks when you see something that some of your candidates couldn’t have shown you. You cross those candidates off. Nothing happened to the machine. It didn’t merge anything; it didn’t know you were watching; it would have done the same thing with nobody in the room. The physicist Edwin Jaynes had a name for mixing these up, and I think it’s one of the most useful names in all of science: the mind projection fallacy, treating what you know about a thing as if it were a property of the thing. Your list of candidates is in your head. The machine’s row is in the machine. The second law has opinions about the machine.

5.4 The demon on a sheet of paper

Build it, because the whole claim is four dots and two arrows and you should see it merge or not merge with your own eyes.

The machine has a hidden bit (think of it as the molecule's side) and a display (the notepad). Four states: write them as pairs, hidden then display. 00, 01, 10, 11. Two moves.

MEASURE copies the hidden bit onto the display, by adding it in the way that a light switch adds: if the hidden bit is 1, flip the display; if it's 0, leave it. Draw the arrows. 00 goes to 00. 01 goes to 01. 10 goes to 11. 11 goes to 10. Look at the arrowheads: every state has exactly one arrow landing on it. Nothing merged. MEASURE forgets nothing.

WIPE sets the display to 0 whatever it held. 00 goes to 00; 01 goes to 00 as well. 10 goes to 10; 11 goes to 10 as well. Two arrows land on 00 and two on 10. That's a merge, twice over. Under any price that charges per halving, WIPE costs at least one. MEASURE is allowed to cost zero.

Now be the observer. You can see the display and not the hidden bit. What you're shown, the display here, is what I'll call a *window*: the part of the state an outside observer gets to see. Section 7 is about windows in general; here the window is just the display. The display starts blank, so your list of candidates is {00, 10}: two possibilities, one bit you don't know. The machine runs MEASURE. You look at the display. If it shows 1, the only candidate that could have produced it is 10. If it shows 0, only 00. Your list went from two to one. You learned the hidden bit. The machine's price for that move: zero. Its own row of possible states went from {00, 10} to {00, 11}, still two; nothing it could have been got merged with anything else. You narrowed. It didn't. That's the measuring half of Bennett's argument on four dots, and WIPE's charge is the wiping half. So the demon just broke something: one bit gained, nothing charged. Here's the thing it broke, written out so you can see exactly where the crack runs: for every list of starting candidates, every run, and every actual start in the list, the index bits the observer gains ($\log_2$ of the candidates before, rounded up, minus $\log_2$ of the candidates after) are at most what the run was charged. Four dots, and that sentence is false.

Definition 5.1 (What an observer knows). Fix a window w from states to what can be seen. Given a list Ω of candidate starting states, a run t , and the actual start $s_0 \in \Omega$, the observer's *knowledge* $K(\Omega, t, s_0)$ is the list of candidates $s \in \Omega$ whose run shows the same thing through the window at every step as the actual run does.

Defined as `knowledge (KnowledgeNarrowing.v)`.

Theorem 5.2 (An observer can learn for free). *There is a four-state machine (the demon), a cost satisfying the halving price, a run of cost 0, and candidates with $|\Omega| = 2$, for which $|K(\Omega, t, s_0)| = 1$. So the claim "for all Ω, t, s_0 : $\lceil \log_2 |\Omega| \rceil - \lceil \log_2 |K(\Omega, t, s_0)| \rceil \leq \text{cost}(t)$ " is false for that machine and that cost. In the same machine, every cost satisfying the halving*

price charges WIPE at least one.

Proof. The machine and run are the four dots above: MEASURE is a bijection, so cost 0 satisfies the halving price; the observer's two candidates show different displays after MEASURE; WIPE sends two pairs of states to one each, so the halving price needs its cost ≥ 1 . $\square$

Checked as `observer_narrowing_can_be_free`, `demon_observer_learns`, `demon_machine_spread_kept`, `measure_forgets_nothing`, `wipe_merges`, and `wipe_costs_at_least_one` (KnowledgeNarrowing.v).

I have to be exact about one thing the file is exact about. The halving rule is a floor. It says merges cost at least this much; it does not say anything costs zero. So what the theorem exhibits is *one* price schedule that meets the halving rule and charges MEASURE nothing. A different schedule could meet the rule and still charge MEASURE; nothing in counting forbids overcharging. The claim is “learning is not *forced* to cost,” not “learning is free on every machine.” That’s weaker than the section title, and the section title is what the strong feeling says before you check, so the title stays and this paragraph stays next to it.

5.5 Who holds the pen changes the bill

Now the part that I think is the most beautiful fact in here, and the way to find it isn’t by thinking; it’s by playing with the four dots. Run MEASURE *twice*. The first copies the hidden bit onto the display. The second adds the hidden bit in again, and adding a bit twice with a light-switch add gets you back where you were: the display is blank again. Check the arrows: 10 to 11 to 10. The display went full and then blank, and nothing merged, and the price was zero both times. So a display that was blank before the first MEASURE *can* be put back to blank for free, by a second MEASURE. Just not by WIPE.

What’s the difference? MEASURE *consults the hidden bit* when it clears the display. It knows what’s written there, because it has access to the thing it was copied from, and so it can undo the copy instead of erasing it. WIPE doesn’t look. It sets the display to 0 regardless, and “regardless” is exactly the many-to-one. Bennett has the general form of this in the 2003 notes: erasing one of two copies that you *know* match is reversible, because you’re not destroying information, you’re removing a duplicate. And it goes further than Bennett in work that came after him: del Rio and colleagues in 2011 showed that the cost of erasing a system depends on how much the eraser already knows about it, down to zero and in the quantum case past zero. The bill for clearing a record is sent to whoever clears it, and it’s smaller the more they know. Try it with a card under a cup and a sticky note: write the colour on the note, then erase the note *without lifting the cup* (two possible notes become one blank: a merge) versus lift the cup, check the note matches, and then erase (nothing unknown was destroyed). Same blank note. Different bill.

5.6 The smallest machine that teaches for free, and the one that can't

Two more small facts, and then the point. The demon measures what you learn by comparing the start to the end, but an observer's very first look already rules things out before any step runs, so the fairer question is: after that first look, can a *step* teach you more, for free? Yes, and the demon still does it. And the smallest machine that can is three states in a ring: U goes to V , V goes to W , W goes to U , and the window shows "true" only at W . Start with candidates $\{U, V\}$; both show false. Take one step. From U you now see false; from V you now see true. One step told them apart, and a ring forgets nothing, so a zero price meets the halving rule, and the file takes it.

Two states can't do it, ever, with any arrows, and this is the one that needs no price premise at all. If a machine has only two states and you can't tell them apart at the first look, then the window shows the same thing on both, and those two are *all the states there are*, so the window shows the same thing everywhere, forever, and you never learn anything. Try to draw a counterexample. You'll draw the same two dots a few times and give up, and the giving up is the proof. Three is the smallest number of states on which two states that look alike at the first look can come apart at a later look, and that's what teaching is.

Theorem 5.3 (Three states are the minimum for free teaching). *Call a machine a free teacher when it has a cost satisfying the halving price, a list of candidates with no repeats containing the actual start, and a run of cost 0 after which the observer knows strictly more than after the first look, $|K(\Omega, t, s_0)| < |K(\Omega, [], s_0)|$. There is a free teacher with exactly three states, and there is none with two or fewer.*

Proof. Three: the ring $U \rightarrow V \rightarrow W \rightarrow U$ with the window true only at W , candidates $\{U, V\}$, one step, cost 0 (a ring is a bijection, so cost 0 satisfies the halving price). Two or fewer: two candidates that agree at the first look are, on a two-state machine, all the states there are, so the window takes one value everywhere and no later look can separate them. This half never uses the price at all. $\square$

Checked as `free_incremental_narrowing_with_three` and `no_free_incremental_narrowing_below_three` (KnowledgeNarrowingMinimal.v), and the demon-against-the-first-look form as `demon_refutes_incremental`.

5.7 What the toll prices, said at last

Now I can say, more precisely than I could two sections ago, what the mark in Section 3 is a mark for. Not for learning. What you know can be narrowed at no cost, with nothing merged, on some machines under some honest price lists, and the demon is one. Not for looking, not for copying, not for finding out. The mark is for the one thing in Section 4 that couldn't be done without merging, on a machine with finitely many states: raising a flag that stays up. Learning is the demon writing on a blank pad, which merges nothing. What the demon pays for is the wipe, a step with a shared

chair. The mark in Section 3 is paid for the same reason, because raising a flag that stays up also has a shared chair, but it's a different step: the demon machine has no flag, and its file prices merges, not commitments. Same chair. Different room.

So "No Free Insight," the name I gave the toll before I understood it, is narrower than it sounds, and I'd rather tell you the narrowness than let the name do work it hasn't earned. Insight in the sense of *finding out* can be free. The insight the toll prices is the kind you've *committed to*, written into a record the machine can't silently revise. Whether that narrower thing deserves the name is a fair thing to argue about; I kept the name because it's the question I started with, and because the narrowing is the finding, not a retreat.

It also tells you how to read a bound that's coming. Section 8 counts what it takes to narrow a list of candidates down to a shorter one: at least as many yes-or-no questions as it takes to point at the survivors among their rivals, a counting fact that holds for anything with a list in it. Turning that count into a *bill*, saying a run actually paid that many marks, is a different claim, and when the book makes it, for any system with a counter in Section 8 and on the machine I build later in Section 22, the payment goes in as a hypothesis: a decision tree (a branching chart of yes-or-no questions, which Section 8 builds) no deeper than the number of paid steps the run took is handed in, along with the lists themselves. This section is the reason it has to go in that way. The narrowing alone doesn't force anyone to pay; the demon just showed you a narrowing that cost nothing. So wherever narrowing gets a price in this book, the payment is a premise, and I'll keep calling it one.

And one more boundary. This whole section is about a machine that's finite and deterministic and a watcher who crosses candidates off a list. There's no quantum anything in it. People sometimes reach for "observation collapses the state" when they hear "watching narrows things down," and I want to head that off: the narrowing here is bookkeeping in the watcher's list, the machine never notices, and the result would be the same if the watcher were a filing cabinet. In 2010 a group in Tokyo (Toyabe, Sagawa, Ueda, Muneyuki and Sano) did run a real demon, a tiny particle walked up a staircase of potential by feedback on what a camera saw, and the energy it gained was paid for by the information the camera took, with no violation of anything. The demon is real, it works, and it pays. Where it pays is the part that took a century.

What I still owe: everything so far says "at least one mark" and I've been keeping the marks on the side of the page, a running total you tally as you go. What happens when the machine keeps that total itself, inside its own state, where a step could in principle reach in and change it? That's the counter, and it's the next thing I have to build.

Part III

The logic

6 The counter

Everything so far has kept the marks on the side of the page. You played the toy, you made a tally as you went, and the tally lived in your hand and not in the machine. That's fine for a toy. For anything that's going to run on its own, with nobody tallying, the total has to live somewhere the machine can reach, which means the machine can, in principle, reach it. So this section is about what a counter has to be before you're allowed to call it a receipt. Not the counter of some particular machine: the counter of *any* machine whose state carries a running total that every step bumps by whatever a price list says that step costs. Section 3's five parts don't need one (there the total was a sum you did on the side), so this is one piece added to the five, and the facts below are about anything that has it. I'll give you the answer first, because it's short, and then spend the section on why each word is there: a receipt is a total that lives in the state, that only climbs, that equals the sum of what was charged, and that nothing else could have been.

6.1 A stick, split down the middle

The oldest receipt I could find that you can still hold is a piece of wood. From about 1100, under Henry the First, until 1826, the English treasury recorded what it was owed on tally sticks: a strip of hazel, notched across with cuts of set widths (a thumb's width for a hundred pounds, a swollen barleycorn for a pound, a bare cut for a penny), then split lengthwise so that each half carried the same notches. The payer kept one half, called the stock, which is one story for where the word stockholder comes from. The treasury kept the other. When the time came to settle, you laid the two halves together, and only the two original halves fit, because wood splits along a grain nobody can copy. Add a notch to your half and the other half shows a gap where the partner should be. Try to remove a notch and you find that wood can't be uncut. When they finally stopped using them, the clerks burned centuries of old tallies in the furnaces under the House of Lords, overheated the flue, and burned down the Palace of Westminster, which I mention because it's the best fact I picked up for this book and because it tells you how many of them there were.

Do it: notch a strip of card once for every coin you hand a friend, split it, keep half each. Try to cheat, either direction. The three things you'll find are the three things I want. The total is the notches, added up, and nothing else (I'll call that conservation). You can add a notch but you can't take one away (it only climbs). And two halves that were notched together must agree; a half that disagrees is a forgery, and the wood says which one (there is only one honest count). The treasury didn't have the words. They had the stick.

6.2 Why it has to climb

Start with the second property, because it's the one that does the most work and the one people fight hardest. Why can't the counter ever go down?

Because of what the toll is for. The floor it buys, back in Section 3, says a run from NO to YES costs at least one. That sentence is a *floor*: it says the run paid at least this much. If the counter could come back down, a run could pay the toll, get its flag, and then take the payment back, and the floor would be a floor on nothing; "this run cost at least one" would be true of a run that ended having cost nothing. So a counter that can fall is an accountant you can lean on to revise the books, and a floor stated in terms of it isn't a floor. The counter only climbing is not a flourish. It's what makes "at least" mean something. (In Section 3's abstract theorem the total is a sum of whole numbers and climbing comes for free; this paragraph is about why a total *stored in the state*, where a step could in principle write it, has to be designed so that no step ever lowers it.)

I found the same rule in two places that had nothing to do with each other, pointing in opposite directions, and that's when I started to trust it. In 1978 Leslie Lamport asked how a set of computers that pass each other messages, with no shared clock, could ever agree on what happened before what. His answer is a counter per machine with two rules: add one before you do anything, and when a message arrives carrying the sender's count, jump your own counter above it (never below your own). Those counters, he showed, respect cause and effect: if this caused that, this has the smaller number. And they do it "with no actual timing mechanism," his words; they're not clocks at all. Try it with three people passing notes and you'll see it work in two minutes, and then let one of them reset their counter to zero after reading a note, and watch a reply come out numbered *before* the note it replies to. A counter that goes down can't be leaned on for order.

Now the other direction. In 1949 Alan Turing, and in 1967 Robert Floyd, each asked how you prove a program *stops*. The answer, both times: find a quantity that goes *down* on every step and can't go down forever (a whole number that can't go below zero, say). If every step lowers it and it can't fall forever, the steps can't go on forever. Up-only gives you order. Down-only gives you stopping. They're not the same rule (one needs the jump-above-the-message step, the other needs a floor it can't fall below), but they're the same shape seen from the two ends: a one-way quantity is something you can build on, and a two-way quantity is a number that happens to be written down somewhere.

And the real-world case where a counter was allowed to go down, and got gamed exactly there: Ethereum, one of the largest blockchains, charges "gas" for every step of a program, and for years it *refunded* gas when a program freed up storage. Someone noticed you could fill storage when gas was cheap, hold it, and cash the refunds when gas was dear, and a token was built to do exactly that, and the real work in a block

could reach nearly twice the block's stated limit. In 2021 a change called EIP-3529 cut the refunds sharply (none for self-destruct, a smaller one for clearing storage, and a cap of one fifth of the gas used). The written reasons are the gas token and those overfull blocks. I read both as the same lesson: a counter that goes down was being leaned on at the place where it went down. Ethereum comes back in Section 12 as one of five small models of systems people actually run, and it's the only one of the five that prices a write at all, and it's instructive that the part they had to fix was the part that could fall.

6.3 Conservation: the total is the charges and nothing else

The first property sounds too obvious to state. The counter after a step is the counter before plus what that step was charged. Add up a whole run and the middle values cancel out, the way a sum of differences collapses to last minus first, and the total is the sum of the per-step charges. Why state it?

Because it's the property that fails quietly. Say a machine charges one for raising a flag, and also, somewhere in its error handling, forgets to add the charge when the step fails halfway through. Or charges two in one code path and one in another for the same move. Or has one step that writes the counter directly. Then the counter is *mostly* the sum of the charges, and "mostly" is the whole gap between a receipt and a note that says "trust me, about this much." Conservation is the sentence that there is no "mostly": every step, every outcome, every error, adds exactly its charge, and a step that fails is charged the same as one that succeeds, because the charge is on the attempt. For a machine built the way I'm describing, that sentence is the rule the counter is updated by, so it can't fail by definition. What can fail is the claim that a real machine *is* built that way, and checking that means going through every move it has, the failure branches included, one at a time, which is dull and is the whole job. The run-long form, counter at the end equals counter at the start plus the sum of the charges, then follows from the one-step rule by the cancelling sum; it's below. It's the kind of theorem that looks boring and is the one you'd most want if you were going to rely on the number. One thing it doesn't say: that every mark bought something. The ledger is exact about what was spent, not about what the spending was worth; whether a mark bought a real narrowing is Section 8's question, and it needs a witness on top of the ledger entry.

6.4 Only one honest count

Here's the property that surprised me by being provable at all. Suppose you fixed the schedule: this move costs one, that move costs zero, and so on, for every move. And suppose someone else keeps their own books: their own function from states to totals, which starts at zero and, on every step, goes up by exactly what the schedule says. Could their books disagree with the machine's counter, somewhere, on some state you can actually get to?

No. And the proof is the tally stick. Both totals start at zero. On every step both

go up by the same amount. So on every state you can reach by steps from the start, they're equal. That's it; it's an induction, the one-step-at-a-time argument from Section 3's floor. The mathematician's word for this is *initiality*: the counter is the one bookkeeping that all the others, if they're honest about the schedule, turn out to be. Given the schedule, the count is forced. Two things about that sentence I had to be careful with, and they're worth saying before the symbols. It doesn't assume the other books climb: climbing follows from the schedule, since the schedule never charges a negative amount, so "only one honest count" and "it only climbs" aren't two separate demands on the other bookkeeper. And it holds on *reachable* states only. A state you can't get to from the start is one where it says nothing about whether the other books agree, which is the right thing for a receipt to say.

Here are the properties exactly. First the piece being added to Section 3's five, a counter and a price list, and then the two facts about it. The counter gets the Greek letter μ (mu), which is what the checked work calls it.

Definition 6.1 (A counter kept by a price list). A *counter machine* has a set of states, a set of moves, a step function apply that takes a state and a move, a start state s_{init} , a schedule c giving each move a cost in $\mathbb{N}$, and a counter μ from states to $\mathbb{N}$, such that $\mu(s_{\text{init}}) = 0$ and $\mu(\text{apply}(s, i)) = \mu(s) + c(i)$ for every state s and move i , with no exceptions. A *run* of a list t of moves from s applies them in order, and $c(t)$ is the sum of their costs. A state is *reachable* when it is the start or is $\text{apply}(s, i)$ for a reachable s and some i .

Theorem 6.1 (Conservation). In a counter machine, for every state s and every list t , μ at the end of the run of t from s equals $\mu(s) + c(t)$. In particular μ never decreases along a run.

Proof. Induction on t . For the empty list both sides are $\mu(s)$. For a list i followed by t' , the one-step rule gives $\mu(\text{apply}(s, i)) = \mu(s) + c(i)$, and the induction hypothesis from $\text{apply}(s, i)$ adds $c(t')$. Since every $c(i) \geq 0$, the counter never falls. $\square$

Theorem 6.2 (Only one honest count). In a counter machine, let B (for "books") be any function from states to $\mathbb{N}$ with $B(s_{\text{init}}) = 0$ and $B(\text{apply}(s, i)) = B(s) + c(i)$ for all s and i . Then $B(s) = \mu(s)$ for every reachable state s .

Proof. By induction on why s is reachable. At the start both are 0. If $s = \text{apply}(s', i)$ with $B(s') = \mu(s')$, then $B(s) = B(s') + c(i) = \mu(s') + c(i) = \mu(s)$, the last step by the counter's rule. $\square$

Those two proofs are the whole of it, and they're about any counter machine. There's no Coq statement of them over arbitrary states, so here they're argued, on the page, where you can check every line. The checked statements are about the machine I build later, which is one of these: Section 16 goes through every move it has and checks the

one-step rule on every branch, and Section 23 gives the run form and the uniqueness under their checked names.

I want to flag what initiality does *not* do, because the word sounds grander than the fact. It doesn't pick the prices. The schedule is an input: somebody decided that raising the flag costs one and adding two numbers costs zero, and nothing here says that decision was right (Section 4 is where I argued the floor isn't arbitrary; the rest of a schedule has to be defended by whoever writes it). It doesn't turn marks into anything physical. And it isn't the category theorist's "initial object," though the word is borrowed for a related reason.

There *is* a universal property nearby, and I'll say exactly how far it reaches so the borrowed word doesn't reach further. Take any other machine that understands the same moves (call it the *target*), and pick a state of it to start from. Walking a list of moves through it, one at a time, is the one and only way to send lists to its states that starts where you said and keeps in step as the list grows by one. Programmers call that walk down a list, one item at a time, a *fold*, and I'll borrow the word. That's an induction on the list from its end, the same shape as the proof just above, and it never looks at the other machine's reading or its prices, which tells you how much the toll is doing here: nothing. It's a fact about *lists of moves*, not about states.

Coming down from lists to the states of the machine running them is where it gets interesting, and where the borrowed word stops. Two different lists can leave a machine in the same state. If they leave the target in two different states, there's no single place to send that state. That's the whole obstruction, and it's exact: the map from states exists on the reachable ones precisely when equal states always come from lists that agree on the other side (and the readings agree). Call that the *descent* condition, because it's what lets the fold come down from lists to states. A toll plus a reading that agrees doesn't force it to happen. A machine that keeps its whole history as its state obeys any toll you like and still never comes down. So the fold is unique on lists, and that's what the word "initial" earns. It doesn't make anybody's states initial among everything that obeys the toll, and I won't say it does. The checked statements (the fold, the descent condition, and three example targets that show where its edges are) are written for the machine I build later, so they're Section 23's.

6.5 The toll is an old trick with a new name

Now a thing that sounds like a let-down and isn't. It would be nice to have a new kind of cost argument. This isn't one. The toll is a case of a technique that algorithm people have used since 1985 and that physicists had, in a different costume, in 1892, and I'd rather tell you it's old than let you think I invented it.

The trick is called the potential method, and here's the whole of it. Give every state a number, call it the potential, that measures how much "owing" is stored up in that state. Then look at each step and ask: does the cost of the step, plus the change in potential it causes, ever come out negative? If it never does, you've got a bound: any

run costs at least the amount the potential *dropped* from start to finish, because the drop had to be paid for by the steps along the way. Robert Tarjan and Daniel Sleator used it in 1985, going the other way (potential as prepaid credit, to bound a run from above), to analyse data structures that are occasionally expensive and usually cheap; and the decreasing-function idea underneath it is the one Aleksandr Lyapunov used in 1892 to prove that a system settles down. Do the coin version: a stack of coins, PUSH adds one and costs one, POP-ALL removes them all and costs as many as it removed, potential equals the number of coins. PUSH costs one plus one potential; POP-ALL of k coins costs k and drops the potential by k , net zero. So any sequence from an empty stack costs at most twice the number of pushes, however many expensive pops are in it. That’s the method. Three lines.

Now point it at the flag. Potential: one if the flag is down, zero if it’s up. A step from down to up drops the potential by one, so for the step’s cost plus potential change to be non-negative, the step has to cost at least one. That’s the toll, exactly, and the two are the same statement: the toll holds for a machine if and only if every step has non-negative amortized cost under the flag potential (“amortized cost” is just that sum, the step’s cost plus the change in potential). Then the universal floor from Section 3, “any run from NO to YES paid at least one,” is just the potential method’s standard conclusion, the drop in potential from one to zero, and every system with the five parts that obeys the toll is an instance (the checked record carries the toll as a field, so that’s every certification system). Two more facts fall out of the same picture. Under the toll, a run raises the flag at most as many times as its total cost; each raise uses up a unit and a unit isn’t used twice. And if you demand the upper-bound version too, that the potential before a step covers its cost plus the potential after, then the two together hold exactly when raising the flag costs exactly one, everything else costs zero, and the flag never comes down; that’s the sharpest version of the schedule, and notice that Section 3’s toy doesn’t meet it: the toy’s VOUCH move, played from YES, makes a mark although it raises nothing, which the floor allows and the exact version forbids. (These two theorems let the price depend on the state as well as the move’s name, which is wider than Section 3’s per-name cost; a per-name cost is a special case.) Two older tools sit beside these, named for completeness: running a list of moves in a “writer” that accumulates a number returns the final state and the run’s cost (`run_writer_is_run_and_cost`), and a per-move schedule is a “grade” fixed before the run (`run_graded_is_run`); neither embeds those frameworks’ full type systems, and the files say so. The writer is the move Danner and Licata make in their cost semantics, a translation that carries a cost alongside a program’s result (Danner, Licata and Ramyaa, ICFP 2015; Danner and Licata, *Journal of Functional Programming*, 2022). The grade is Katsumata’s: a computation carries its resource as a grade, and the grades compose as the computations do (“Parametric effect monads and semantics of effect systems,” POPL 2014). Orchard, Liepelt and Eades put grades into the types of a working language (“Quantitative program reasoning with graded modal types,” ICFP 2019). Read against them, the counter is a grade carried in the state by the step

rule, not in a type. And what does that difference even mean? A cost that lives in a type gets checked before the program runs, by whoever checks the types; a cost that lives in the state gets written during the run, by the step itself. Same number, different moment of truth. Whether mine is an embedding of their frameworks or something weaker is a question this book leaves open, and I'd love for someone who knows those frameworks better than I do to settle it.

Exactly, for any certification system with states S , moves I , step, per-name cost, and reading:

Definition 6.2 (Potential, amortized cost). A *potential* is a function $\Phi : S \rightarrow \mathbb{N}$. A step has *non-negative amortized cost* under Φ when $\text{cost}(i) + \Phi(\text{step}(s, i)) \geq \Phi(s)$. The *flag potential* is $\Phi_{\text{cert}}(s) = 1$ if $\text{cert}(s) = \text{no}$ and 0 if it reads yes.

Defined as `amortized_nonneg` and `cert_potential` (`CostSemanticsComparison.v`).

Theorem 6.3 (The toll is the potential method). (1) *The toll holds exactly when every step has non-negative amortized cost under Φ_{cert} .* (2) *For any Φ with non-negative amortized cost and any run t from s , $\text{cost}(t) + \Phi(\text{run}(t, s)) \geq \Phi(s)$.* (3) *Hence, under the toll, a run from a no-state to a yes-state costs at least $1 - 0 = 1$.*

Proof. (1) Check the four combinations of the reading before and after: only no-to-yes has Φ_{cert} dropping, by one, so the inequality there is exactly $\text{cost}(i) \geq 1$; in the other three it holds for any cost. (2) Induction on the run, adding the per-step inequalities so that the middle potentials cancel. (3) Apply (2) with Φ_{cert} , which is 1 at the start and 0 at the end. $\square$

Checked as `a2_iff_nonnegative_amortized_cost`, `potential_telescoping`, `nfi_by_potential`, and `certification_system_is_potential_method` (`CostSemanticsComparison.v`), with the state-dependent-cost results `flips_le_cost` and `a2_and_aara_iff_exact` (`CostFrameworks.v`).

So what's new, if the method isn't? Only where it's pointed. The potential method is a way of proving things about costs. What this book adds is a claim about *which* potential, the flag, and about what happens to that claim when you can only see a window of the state. The method is Tarjan's. The reading is mine, and I'm about to find out what it can and can't survive.

6.6 What the counter is, said once

A receipt is a total that lives in the state, only climbs, equals the sum of the charges on every path including the failures, and is the only such total for its schedule. Each of those four, counting "lives in the state" as the one the others are about, had a

home before this book: the stick, Lamport's clocks and Floyd's ranking functions, the telescoping sum, the induction. None of them is physics. None of them says what the charges should be. What they say is that once you've chosen the charges, the counter isn't a second opinion; it's the arithmetic. And the toll on top of it is the potential method with the flag as potential, which is to say a trick from 1985 wearing the question from Section 2.

Here's one thing the counter can't do just by existing: if you can only see *part* of the state, can you tell what the counter is? Can you even tell, from outside, which step paid? The stick works because you can lay the two halves together. What if you're only ever shown one half, through a window, and the window doesn't show the notches?

One piece of that has a short answer, and I'll give it now so the next section can start from it. A window, for now, is just the part of the state an outside observer is shown (Section 5's display was one, and Section 7 is about windows in general). Call a window *counter-complete* when some rule, applied to what the window shows, returns the counter on every state; *flag-complete* likewise for the flag. Say a window *forgets the counter* when changing the counter and nothing else never changes what the window shows, and likewise for the flag.

The short answer: a window that forgets the counter is never counter-complete, whatever else it shows, and a window that forgets the flag is never flag-complete. Take any state and make two copies of it, one with the counter set to 1 and one with it set to 2, everything else held still. The window shows the same thing for both, so any rule gives the same answer for both, and one of the two answers is wrong. The flag is the same argument with yes and no. On the stick: a window that shows you the wood's colour and its length but not the notches can't tell you what's owed, and staring harder at the colour won't help. (The argument needs a state space where the counter can be changed on its own, the rest held still; that's what "forgets" quietly assumes. The machine I build later is such a space, and Section 23 has its checked version, with the machine's own full window.)

Be careful what that doesn't say. It says neither part can be dropped. It doesn't say a window showing both is the smallest window that works, because nothing here orders windows, so there's nothing for it to be smallest in. And it's about reading the counter back, not about which step paid. That second question is harder, and it's the next section's.

7 Windows and shadows

I left you with the stick split in half and asked what happens if you can only ever see one half through a window that doesn't show the notches, and with a harder question under that one: can you tell, from outside, which step paid? This section is about windows. It's a section where the mathematics is so small that I'm going to state it in full and then spend most of the pages on what it's *not*, because the smallness is the

trap: the fact is a one-liner and people keep trying to climb over it with cleverness, and you can't, and seeing why you can't is the content.

7.1 A cave, and the people who got good at it

You've probably heard a version of this, so let me tell it carefully, because the version that usually gets told is the wrong half. In the seventh book of the *Republic*, Plato has Socrates describe prisoners chained in a cave since childhood, facing a wall, with a fire behind them and people carrying objects along a walkway between the fire and their backs. All the prisoners have ever seen is the shadows on the wall. To them, Socrates says, the truth would be nothing but those shadows. That's the part people remember, and it usually gets used to say "the real world is hidden from us," which is a thing Plato did believe and which is not what I need from the story.

Here's the half that matters for this book, and it's one line further on. The prisoners, Socrates says, hand out honours to whoever is quickest at spotting the shadows as they pass and best at guessing which comes next. Read that again. The prisoners *get good at it*. They build a whole science of shadows, with experts and prizes, and it works, on shadows. What they can't do is say anything about a carried object that leaves no mark on the wall. Not because they're stupid. Because the information isn't on the wall. Skill inside a window doesn't reach past the window, no matter how much skill. That's the fact. Plato dressed it as a lesson about education, and it is one, but strip the dressing and it's a sentence about functions, and it's going to be the whole section.

7.2 The shadow of a cube

Do this one tonight with a lamp. Hold a cube face-on to the light: its shadow is a square. Now hold up a square of card the same size: same shadow. A person who sees only the wall can't tell you whether what's casting the shadow has any thickness at all. Ask them, and they'll tell you, truthfully, that they can't know, and if they're honest they'll also tell you that no amount of staring at that square will change it.

Now turn both a quarter turn. The card's shadow collapses to a line. The cube's stays a square. Now the wall-watcher knows. What changed? Not their cleverness. The *window*. A second look from a different angle is a bigger window than one look, and it was bought with a turn and a second look. Edwin Abbott told this as a novel in 1884, *Flatland*: a sphere visits a world of flat shapes and the only thing the flat shapes can see is a circle that grows and shrinks as the sphere moves through their plane; from one slice they can't reconstruct a sphere, and from the sequence of slices over time, they can start to. The escape from a window is never "think harder." It's always "a different window," and a different window always costs something: a turn, a second look, time.

7.3 The whole fact, in one paragraph

Here's the mathematics, and I promise it's all of it. You have a set of states, the real things. You have a window, which is a function that takes a state and gives you what you can see of it: the shadow, the slice, the half-stick without notches. And you have a question about the state, which is also a function: it takes a state and gives an answer. The thing you want is a *decoder*: a rule that takes what you can see and gives the right answer to the question, every time. When does one exist?

Think about two states that cast the same shadow. The decoder sees the same shadow for both, so it gives the same answer for both. So if the question has *different* answers on those two states, the decoder is wrong on at least one of them, and there's nothing it can do about it, because it never saw the difference. That's the whole fact. If a decoder exists, then the question must already give the same answer on every pair of states that look the same through the window. Mathematicians call the set of states that cast one particular shadow the *fibre* over that shadow (think of the shadow as a point on the wall and the fibre as the bundle of all the things that could have cast it), and the sentence is: a decoder exists only if the question is constant on every fibre. That's the theorem just below, and the "only if" is the direction that bites. The other direction, "if the question is constant on fibres then a decoder exists," is true too, but it needs you to be handed, for every possible shadow, one actual state that casts it, so you can look up the answer there, and the handing is not free either, since someone has to supply a representative for every shadow. (You can always fill a gap with a default answer. The default doesn't recover what was really there.)

Exactly. Let S be any set of states, V any set of views, $w : S \rightarrow V$ a window, and $q : S \rightarrow A$ a question with answers in any set A . A *decoder* is a function $d : V \rightarrow A$ with $d(w(s)) = q(s)$ for every state s . The question is *constant on fibres* when $w(s) = w(s')$ implies $q(s) = q(s')$.

Theorem 7.1 (Decoders need constant fibres). *If a decoder exists, the question is constant on fibres. Conversely, if the question is constant on fibres and you are handed, for every view v , a state $r(v)$ with $w(r(v)) = v$, then $d(v) = q(r(v))$ is a decoder.*

Proof. If $w(s) = w(s')$ then $q(s) = d(w(s)) = d(w(s')) = q(s')$. Conversely, $d(w(s)) = q(r(w(s)))$, and $r(w(s))$ has the same view as s , so by constancy its answer is $q(s)$. $\square$

Checked as `decoding_requires_fiber_constancy` and `selected_representatives_give_decoder` (`ObservationPolicy.v`), for arbitrary types.

Try the smallest case, by hand, because it'll keep coming back. A state is a pair: a position, 0 or 1, and a hidden flag, up or down. The window shows the position only. Two moves from (0, down): one sets the position to 1 and raises the flag; the other sets the position to 1 and leaves the flag alone. Both land on position 1. Now write down every possible decoder from "what I see" to "is the flag up": there are exactly four

functions from two positions to two answers. Check each against both results. None of the four gets both right, because both results show position 1 and one has the flag up and one doesn't. That exhaustive check is the theorem on this example; the general statement is the one-line argument above, and that's what's checked. There's no fifth function hiding somewhere.

7.4 People who found this on their own

The fact is so plain that it got discovered separately in every field that has to deal with seeing part of something, and I want to lay them beside each other, because that's the kind of evidence I trust: not that one smart person said it, but that people who'd never met kept walking into the same wall.

In 1956 Edward Moore, working on the small switching machines that became computer hardware, asked when an experimenter who can only feed a machine inputs and watch its outputs can tell two of its internal states apart. His answer gave a precise meaning to "indistinguishable from outside," and a theorem with a sting in it: even when every pair of states can be told apart by *some* experiment, there are machines where no single experiment tells you which state you started in. What you can always do is run an experiment after which you know which state you're in *now*. In 1960 Rudolf Kalman, whose filter is in the navigation of aeroplanes and phones, asked the same question for continuous systems, called it *observability*, and gave a test (a matrix rank, which you can compute) for whether watching the output over time pins down the internal state. His unobservable states are exactly a fibre: states whose difference never reaches the output, and, this is the part people miss, never will, however long you watch. More samples of the same window only repeat the same fibre. Try it on a car: if your only instrument is the speedometer, two cars driving side by side at the same speed read the same forever, and nothing on that dial will ever tell you which mile marker either one started at. Swap the speedometer for an odometer and you can recover both. Same car, different window.

In 1977 Patrick and Radhia Cousot built a whole discipline of program analysis, called abstract interpretation, on throwing information away *on purpose*: run the program over coarse descriptions of its values instead of the values, and accept that the answer you get has to be true of *every* real value the coarse description could stand for. That last clause is fibre constancy as a design rule. And in 1970 Ted Codd, inventing the relational database, made programs independent of how the data is stored, and the relational systems that followed gave users *views*, deliberately partial windows onto the stored data. A decade after Codd, Bancilhon and Spyrtos set out the view-update problem: if a user changes what they see through a view, which of the many underlying states that cast that view should the database move to? Several are legitimate. The fibre, from the writing side.

And Parnas, from Section 2, hid things on purpose: a module is characterised by the design decision it *hides* from its callers. Hiding is a window. The caller can't later read

back what was hidden, including the things it later wishes it could check, and that's the price side of the coin whose benefit side Parnas was selling. None of these people were doing philosophy. They were trying to build filters, databases, compilers, and modules, and the wall is the same wall in each.

7.5 The clerk with two photographs

Now point it at the counter, because that's why this section is here. Suppose you want to bill for raising the flag, exactly: at least one mark for every step that raises it, and nothing for a step that doesn't. And suppose the person doing the billing is a clerk who sees only a *window* on the state, a photograph before the step and a photograph after. The clerk's bill is a function of the two photographs. Can the clerk bill exactly?

Not if the window has the wrong kind of blind spot. Here's why, and it's the cube again. Suppose there are two steps that give the clerk the same before-photo and the same after-photo, and one of them raised the flag and the other didn't. (The two-move toy above is exactly such a pair; so are "a stamp goes onto a document inside a closed drawer" and "someone opens and closes the drawer.") The clerk has one bill for that pair of photographs. If the bill is at least one, it overcharges the step that didn't raise the flag. If it's zero, it undercharges the step that did. Any bill that never misses a raise bills something that wasn't a raise, and no bill from those photographs is exact. The fix is the plain one: put the flag in the photograph. If the window shows the reading, then "one when the reading goes from down to up, zero otherwise" is exact, and such a window has no pair like the one above. That's enough. It isn't the only way: a window can hide the flag and still have no colliding pair, in which case a bill from the photographs can be exact; what the file proves the other way is only that a colliding pair rules exact pricing out. And "exact" here has a definition: at least one on every step that raises the flag, nothing on any step that doesn't. That's all the clerk's problem needs, and the bill "one for a raise, zero otherwise" meets it (`step_price_is_exact`). So it's exact about *which* steps get charged. It isn't exact about *how much*, and it doesn't pick out one bill: "two for a raise, zero otherwise" meets it too. Pinning the bill down takes one more demand, a ceiling: over any run, the total never exceeds the number of raises. With the floor and that ceiling there's exactly one bill left, one per raise and zero otherwise; that's an if and only if, proved for any system whose bill is decided step by step (`exact_commitment_pricing_characterization`, `CommitmentPredicateAdequacy.v`). These are stated for any states, any steps, any window; they're the cube theorem with a price in place of a decoder.

Same clerk, now in symbols, so you can check every word I just used. Take the five parts of Section 3 with a window $w : S \rightarrow V$, and say a step (s, i) raises the flag when $\text{cert}(s) = \text{no}$ and $\text{cert}(\text{step}(s, i)) = \text{yes}$. A *price* is any function of a state and a move. It *meets the floor* when every raise costs at least one, and *never overcharges* when every non-raise costs zero; together, it is *exact* (about which steps are charged; the amount is pinned only by the whole-run ceiling above). A *window price* is one computed from the two photographs only: $\text{price}(s, i) = b(w(s), w(\text{step}(s, i)))$ for some

bill $b : V \times V \rightarrow \mathbb{N}$. A *collision* is two steps (s_1, i_1) and (s_2, i_2) with $w(s_1) = w(s_2)$ and $w(\text{step}(s_1, i_1)) = w(\text{step}(s_2, i_2))$, the first a raise and the second not.

Theorem 7.2 (No exact window price across a collision). *If the window has a collision, every window price that meets the floor charges at least one on some step that is not a raise; so no window price is exact. If instead the reading is a function of the view, $\text{cert}(s) = r(w(s))$ for some r , then the window price “one when r goes from no to yes, zero otherwise” is exact, and the window has no collision.*

Proof. Across a collision both steps get the same bill $b(v, v')$. The floor makes it at least one because the first step is a raise; the second step, not a raise, pays it. If the reading is r of the view, the window price reads the raise directly, and two steps with the same photographs either both raise or both don’t, so there’s no collision. $\square$

Checked as `shadow_floor_overcharges`, `shadow_cannot_price_exactly`, `window_showing_reading_prices_exactly`, and `window_showing_reading_has_no_collision` (`ShadowPricing.v`), for arbitrary types.

So the exact price of raising the flag lives in the step itself, or in a window that shows the flag. A window that contains a pair like that one can’t carry it. Whether a particular machine has such a pair, for a particular window, has to be checked machine by machine and window by window; the theorem only tells you what a pair does once you’ve found one.

7.6 The three escapes, and what each one costs

Suppose you’re the person outside, and you want to be convinced that a run paid. The window you’ve got doesn’t show the counter. Here’s how bad that is, in the smallest setting I can build. Take a counter machine from Section 6, give its states a position and a flag as well, and give it a window that shows the position and nothing else. Now take two runs from the same start, with the position at 0, the flag down and the counter at 0. Run *A* makes one move that raises the flag, costs one, and lands at position 1; its counter reads 1. Run *B* makes one move that does nothing but step to position 1, at cost zero; its counter reads 0. Call the pair of photographs, position 0 then position 1, the *transcript*. Both runs leave exactly that transcript. A *verifier* is a rule that takes a transcript and says accept or reject. It is *sound* when every transcript it accepts comes only from states where the claim holds, and *complete* when every transcript from a state where the claim holds is accepted. The claim is “the counter is 1.”

No verifier on these transcripts is both, and the proof is three lines. Suppose one were both. Run *A* ends with the counter at 1 and leaves the transcript, so completeness says the verifier accepts it. Now soundness, on that same accepted transcript: every state that leaves it satisfies the claim, and run *B*’s does. So run *B*’s counter is 1. It is 0. That’s the cube again, with a verdict in place of an answer: a verifier sees only

what the window kept, and two states that the window can't tell apart get one verdict between them.

Now dress the transcript up. Take any richer kind of transcript, with a way to squash one down to its bare photographs, and say a verifier on the rich ones *only looks at the bare part* when two rich transcripts that squash to the same photographs always get the same verdict. If some rich transcript left by run *A* and some rich transcript left by run *B* squash to the same photographs, then no verifier that only looks at the bare part is sound and complete either: it gives those two the same verdict, completeness makes that verdict accept, and soundness then lies about run *B*. Notice what that needs. The pair has to survive into the richer kind of transcript; a design whose extra part tells *A* from *B* doesn't meet the premise, and the argument says nothing about it. So it doesn't classify every transcript design. What it says is that a verifier that only reads what the bare window keeps fails on that pair.

What can you do? Here are three things that work, and nothing says they're the only ones; cryptography uses all three. Each rests on something you have to trust, and I'll say what, because "a price" in the counter's sense is not what any of them costs.

You can be shown the *whole state*. Then there's nothing to decode; you read the counter off the state. The price is trust: you have to believe the state you were shown is the state there was.

You can be shown a *commitment*: the ordinary window plus one extra bit that is set when, and only when, the claim holds (here, that the counter reads exactly one). Reading that bit gives a verifier that is sound and complete for that claim, *if* the bit is binding (set only when the claim holds) and honest (set whenever it holds). The trust is in those two "ifs," which are a contract about who can set the bit. The contract can be met, and a bit anyone can set breaks it, which is the one-line version of why an unsigned log is just another window. Nothing in that picture has a hardness assumption in it; whether a real commitment scheme meets the contract is a question for cryptography, not for this book. In 1999 Bruce Schneier and John Kelsey worked out how to make a log that an intruder who takes over the machine can't quietly rewrite, and the honest scope in their paper is the honest scope here: it protects what was written *before* the break-in, through keys, and nothing after.

Or you can *ask*. Interaction: the outside party gets to put a question and gets an answer that pins the thing down. The price is the honesty of the answerer; a liar's answer is another shadow. Moore's experimenter chose inputs; the zero-knowledge proofs of Goldwasser, Micali and Rackoff in 1985 are the refined form, where a verifier is convinced by a conversation designed to cut exactly the line between true and false and nothing else, with a mathematical guarantee against a cheating prover. I mention zero-knowledge to say what I'm *not* claiming: the asking in this section is one round, with one fixed question ("tell me your counter"), and nothing in it models a cheating prover; the honesty is assumed, not enforced. That's a toy of the real thing, and I'm

not going to let the real thing's reputation stand behind the toy.

Each escape is a window with something added: the whole state, a bound bit, an honest answer. None of them is "a smarter verifier on the old window." Cleverness lives on the wrong side of the projection. What's true of any machine is the cube. The machine I build later has pairs like these, and checked versions of all three escapes; they're stated on it, so they're Section 23's.

7.7 What this isn't

Three things people reach for, and I'd rather take them off the table now.

It isn't mysticism. Nothing here says the window is false or the state is unknowable. The wall is a function, the objects are its inputs, and the only claim is that the function isn't one-to-one on the thing you asked about. A richer window recovers it. Plato's prisoners could have turned around.

It isn't Rice's theorem, though it rhymes with it. The logician Henry Rice, in 1953, proved that no program can decide any interesting property of what another program *does* from its code, and a cousin of the fact here is that a run which hasn't halted yet looks, so far, exactly like one that never will. But Rice is about what's *computable*. The cube is about what's *there*. The cube theorem holds for a toy with two states and nothing to compute; the obstruction is missing information, not missing cleverness, and I'd be overclaiming if I let the famous theorem lend its weight to the small one.

And it isn't "with enough data you can recover anything." Kalman's unobservable states stay unobservable for all time. The question is never how much of the window you've got. It's which window.

So: the counter lives in the state, and whether anyone outside can read it depends entirely on what they're shown, and a window with a pair like that in it can't price the flag exactly, not with any rule, ever. Which raises the question I've been pushing in front of me since the buckets. Fine, the flag went up and was paid for and I can see it in the right window. What does that *entitle* me to? A paid flag is a paid flag. What's the claim behind it, and when am I allowed to lean on it?

8 Structural entitlement

I keep writing the word "vouch" and then taking it back. Let me stop doing that and say what it would take. A reading went from NO to YES, the toll charged the move that did it (Section 3), maybe the reading stays up (Section 4 says what it costs to make it stay), and the right window shows it (Section 7). Now: what is a later step *allowed to do* because of that? That's a different question from "is the thing true," and the gap between those two is the subject of this section, and I'd argue it's the subject of most of the arguments people have about evidence, so give me the long way round.

8.1 The coin that moved

Here's an experiment you can run on a friend, and I'd ask you to actually run it, because the feeling of it is the point. Put a coin under one of three cups while they watch. Say, cup one. Send them out of the room. Move the coin to cup three. Then, and this is the trick, put a *second* coin under cup one. Bring them back and ask: "Is there a coin under cup one?"

They say yes. They're right; there is. And they have a reason: they watched you put it there. True, and believed, and justified by what they saw. And it's not knowledge, and you can feel that it isn't, because their reason points at a coin that left. The coin that makes them right is not the coin they're thinking of.

That's a three-page paper from 1963 by Edmund Gettier, "Is Justified True Belief Knowledge?", which broke a definition people had leaned on since Plato, in about the time it takes to read it. The definition, which people read into Plato's *Theaetetus*, was that you know something when it's true, you believe it, and you've got good reasons. Gettier's two cases, and the cups, show a belief that passes all three tests and still isn't knowledge, because the reasons and the fact aren't connected; they just happen to agree. Bertrand Russell had a related case in 1948: somebody looks at a stopped clock at the one moment it happens to be right, and they're right about the time, and they know nothing.

What's missing is a wire from the reason to the fact. Ask your friend what they'd need, to be *entitled* to "there's a coin under cup one," and they'll tell you: lift the cup. Not "remember putting it there." Lift it, now, and look. That lifting is an event, it happens on the record, and after it the reason and the fact are the same object.

8.2 Plato nearly had it

I want to be fair to the *Theaetetus* here, because it's easy to be quick with the ancients and they deserve better. Socrates, in that dialogue, doesn't just say "true belief plus reasons." He tries three different senses of what the "account" behind a true belief could be: a statement you can say aloud; a list of the thing's parts; and, third, the mark that sets this thing apart from everything else it could be confused with. He tries each, and each fails, and the dialogue ends with nobody knowing what knowledge is, which is one of the more honest endings in philosophy. But the third sense is the one I'd underline. An account as *the thing that distinguishes*: the test that gives the real case a value no rival gives. He couldn't make it work because he had no way to say what the account had to be attached to. Two and a half thousand years later, the thing it has to be attached to turns out to be a state, and the account turns out to be an object sitting in it. I'm not claiming that settles Plato. I'm claiming he was pointing at the right thing.

8.3 The shape, said once

So here's what I mean by entitlement, and it has three parts, and I'll try to earn each.

A run is entitled to a claim when three things are present. First, an *object*, carried in the state: a grouping, a relation between groups, a counter, a package of constraints. Not a belief; a thing you could print. Second, a *checked relation* between that object and the claim: a test that was actually run, with its result on hand, not “a test exists.” Third, an *event on the record* that says this object was put there and paid for, so that anyone auditing the run can point back at the moment the claim was marked.

Said as a definition, so the rest of the section has something to point at:

Definition 8.1 (Entitlement). A run is *entitled* to a claim when three things are on hand: an *object* carried in the run’s states; a *checked relation* between that object and the claim, meaning a test whose result is on the record, not a test that merely exists; and a *paid event*, a move of the run at which the object was put in place and which the price list charged at least one. In a certification system (Definition 3.1) a move that takes the reading from no to yes is such a move, because the toll charges it. This is the book’s word, not a Coq definition.

Without the third part, the object may be sitting right there and the claim may be true, and the run still hasn’t *earned* it. That’s the cups: the coin is under cup one; the reason is disconnected from it; nobody lifted the cup. Put it on Section 3’s sheet of paper, the one with the NO box, the YES box and the tally line, and grow the toy by one thing: the state is now the box you’re in plus whatever is written on the sheet beside it. Next to the boxes, write a note: “coin under cup one, checked.” Writing the note is a WORK move, and WORK costs nothing. The note is an object, it’s on the paper, and it might even be true. What it doesn’t have is a crossing on the record: no move took the reading from NO to YES when the note went down, and there’s no mark under the line for it. The note is raw data, forged in one write, and any system with the five parts lets you write one, because nothing in the five parts says what an ordinary move may put in the state. A note put down by the move that crosses has a receipt, and the receipt is the event. One limit on that word “receipt”: the crossing records that something was paid for at that moment. What the crossing move actually looked at before it crossed is up to whatever the move does, and the five parts don’t say.

And here’s the part to hold on to. Nothing in the five parts makes a later move *read* the reading. The step function is handed the state and a move name, and it’s allowed to ignore the YES box completely; plenty of systems will. So when I say a run is *entitled*, I mean what someone auditing the run may conclude from the reading and the receipt. The system doesn’t have to enforce it, and unless somebody builds it to, it won’t.

The engineering version of this was already built, in 1997. George Necula’s *proof-carrying code*: when you ship a program to a host that doesn’t trust you, you ship a *proof* with it that the program obeys the host’s safety rules, and the host *checks the proof that arrived* before running anything. Not your reputation. Not “a proof exists somewhere.”

The one in the box. A quarter century earlier Stephen Cook's certificates had the same bones, from Section 3: a claim comes with a witness that's cheap to check, and the checker checks *that* witness. And David Hume, in 1748, arguing about miracles, laid out the thing the receipt replaces: he weighed testimony against evidence, and said a wise person believes in proportion to the evidence, and the whole of his argument is a method for how much to trust a witness when you have no receipt. The move in this book is to stop weighing. A claim on the record is either backed by its object and its paid event, or it's testimony from outside, and a run that leans on testimony hasn't earned anything.

Three things entitlement is *not*, which people reach for. It isn't permission: an access control system says who may write to what, and that's granted from outside and says nothing about truth; a process with permission to write the note still has no entitlement. It isn't "true, therefore usable": that's the cups. And it isn't "sound by luck": a shortcut that gives the right answer because the inputs happened to cooperate is the stopped clock. The count below has one condition aimed at it, and I'll tell you what the condition says and how much it actually catches.

8.4 What narrowing costs: count the rivals

Now the number, because a book about a bill has to say how much. Here's the idea in a form you can do with three playing cards. Shuffle cards marked 1, 2, 3 face down. Sort them using only questions of the form "is this one bigger than that one?", and count the questions. There are six orders the cards could be in. Each question has two answers, so after one question you've split the six into at most two piles, after two questions at most four, and you need to get down to one. Four is less than six, so two questions can't always do it, and three can, and you can draw the tree of questions on paper and see it: six leaves, three deep. The number of questions you need is at least the base-two logarithm of the number of possibilities you started with, rounded up: that's $\log_2 6$, which is a bit under 3, rounded to 3. Donald Knuth, in the volume on sorting of *The Art of Computer Programming*, gives this as the lower bound for all comparison sorting, and the shape of the argument is just: a tree f questions deep has at most 2 to the f leaves, so it can't tell apart more than 2 to the f cases.

Ralph Hartley in 1928, before Shannon, measured information exactly this way: the logarithm of the number of possibilities, no probabilities, just a count. That's the measure here, and I'll say plainly it's Hartley's and not Shannon's: there's no distribution over the list, only the list's length.

Now put it on a run. Say a run comes with a list of states that are "still on the table," the candidates, and a shorter list, with some ruled out. Call the first the prior and the second the posterior, and I want you to hear those as *a list you could print*, not "the space of possible worlds." Picture the ruling-out as observations, each observation that ruled something out being one branch point in a tree like the card tree. And here's the claim: if that tree is no deeper than the number of paid steps the run took, and

the tree has enough leaves to cover the narrowing in a way I'll spell out in a moment, then the counter rose by at least $\log_2$ of the prior's length minus $\log_2$ of the posterior's length. At least as many marks as *index bits* the list lost, where the index bits of a list are the yes/no questions it takes to point at one entry of it, the card count again. That's the bound.

"Paid step" needs its own sentence. It means a step the price list charges at least one for. In a certification system the toll guarantees that every crossing of the reading is one; a price list is free to put a floor under more steps than that (a look, a send, a check), and the count takes every paid step there is. The tree is a bare shape, branches and leaves with nothing written on them, and "no deeper than the number of paid steps" is a count along the run that happened. It isn't a walk down the tree. A paid step doesn't have to have asked any question at all.

Notice, too, what the bound does and doesn't say about the run. It takes the two lists, the tree and the covering as given. It doesn't say the run produced the posterior from the prior; nothing in it reads the lists off the run. What it says is that, given lists, a tree and a covering that someone supplies, the paid steps of the run cover the rounded drop in list length. That's Section 5's warning come due: watching can narrow for free, so the payment can't be dug out of the narrowing. It goes in as a premise, the tree's depth against the paid steps, and I'll keep calling it one.

8.5 What "narrower" buys: a stronger test

Before the count, one link that's pure logic, and Plato's third account is the whole of it. Take a list of candidates and an observation, a function that says what each candidate looks like from outside. The list's *membership test* accepts an observation when some candidate on the list shows it. Shrink the list, and the test can only get pickier: anything the short list's test accepts, the long list's accepts too. The word for that, and it runs backward from English, is *stronger*: the stronger test is the one that lets *fewer* things through.

Pickier, though, isn't the same as strictly pickier, and the cards show the gap. Take all six orders as the prior, and as the posterior the three where card 1 sits left of card 2: 1-2-3, 1-3-2 and 3-1-2. First observation: "is 1 left of 2?" Every survivor says yes, and the removed 2-1-3 says no, so the prior's test accepts yes and no, the posterior's only yes. Strictly stronger. Second observation, same two lists: "is the first card a 3?" Now 3-1-2 says yes and the other two survivors say no, and among the removed orders 3-2-1 says yes and 2-1-3 says no. Both tests accept yes and no. Same strength, even though three orders are gone. What made the difference is a removed candidate whose observation *no survivor shares*. That's Plato's mark, the thing that sets this case apart from everything it could be confused with, turned into a function. Narrow the list and keep one distinguishing removed candidate, and the test is strictly stronger; narrow it without one, and the test may not have moved at all.

8.6 The count, exactly

Here are the words and the count, for any system with Section 6's counter. A *list* is a finite list of candidates, repeats allowed, and $|\Omega|$ is the length of the list Ω , repeats counted. $\lceil \log_2 x \rceil$ is the base-two logarithm of x rounded up: the smallest $k \in \mathbb{N}$ with $2^k \geq x$. At $x = 0$ this index-bit count is defined to be 0 as well; the notation $\lceil \log_2 0 \rceil = 0$ here is a convention for empty lists, not the real logarithm of zero. And $a \dot{-} b$ is subtraction on whole numbers that stops at zero, so $3 \dot{-} 5 = 0$.

Definition 8.2 (Decision tree). A *decision tree* is a leaf, or a branch with two subtrees. Its depth is 0 for a leaf and one more than the deeper subtree for a branch; its leaf count $L(T)$ is 1 for a leaf and the sum of the two counts for a branch. Nothing is written on a tree: no question, no answer, no candidate.

Lemma 8.1 (A tree is no wider than two to its depth). *For every decision tree T , $L(T) \leq 2^{\text{depth}(T)}$, and so $\lceil \log_2 L(T) \rceil \leq \text{depth}(T)$.*

Proof. Induction on the tree. A leaf has one leaf and depth 0, and $1 \leq 2^0$. A branch with subtrees of depths d_1, d_2 has at most $2^{d_1} + 2^{d_2} \leq 2 \cdot 2^{\max(d_1, d_2)}$ leaves, which is 2 to its own depth. The second half: $L(T) \leq 2^{\text{depth}(T)}$ means $\text{depth}(T)$ is one of the k with $2^k \geq L(T)$, and the rounded-up logarithm is the smallest of them. $\square$

Checked as `decision_tree_leaves_le_pow2_depth` and `decision_tree_log2_up_leaf_bound` (MuShannonBridge.v). Both are stated over the `tree` type alone, a leaf or a branch, with nothing about any machine in them.

Theorem 8.2 (Paid narrowing, counted). *Take a system with a counter μ and a price list c that obey conservation (Theorem 6.1): each step raises μ by exactly its charge. Let Ω and Ω' be lists with $|\Omega'| > 0$, let T be a decision tree with $|\Omega| \leq L(T) \cdot |\Omega'|$ (the tree covers the narrowing), and let t be a run from s ending at s' in which at least $\text{depth}(T)$ steps are charged at least one each. Then*

$$\lceil \log_2 |\Omega| \rceil \dot{-} \lceil \log_2 |\Omega'| \rceil \leq \mu(s') - \mu(s).$$

Proof. The rounded-up logarithm respects order, so the covering gives $\lceil \log_2 |\Omega| \rceil \leq \lceil \log_2 (L(T) \cdot |\Omega'|) \rceil$. It is also subadditive on products of positive numbers, $\lceil \log_2 ab \rceil \leq \lceil \log_2 a \rceil + \lceil \log_2 b \rceil$, because $2^{\lceil \log_2 a \rceil} \cdot 2^{\lceil \log_2 b \rceil} \geq ab$. So

$$\lceil \log_2 |\Omega| \rceil \dot{-} \lceil \log_2 |\Omega'| \rceil \leq \lceil \log_2 L(T) \rceil \leq \text{depth}(T),$$

the last step by Lemma 8.1. By hypothesis at least $\text{depth}(T)$ steps of the run were charged at least one, and no step is charged less than zero, so the sum of the charges is at least $\text{depth}(T)$. By conservation that sum is $\mu(s') - \mu(s)$. $\square$

Argued here for any system with a counter that obeys conservation. The tree half is the checked lemma above; the rest is arithmetic on three whole numbers, the two lengths and the leaf count, and on the counter. Notice what's missing from the hypotheses: anything tying the lists to the run. That's the honest shape of the bound.

Three honest small prints, all of them visible in the statement. Both logarithms round up separately, and the subtraction stops at zero, so when the prior and the posterior round up to the same number of index bits (four candidates narrowed to three, say) the bound says nothing. The lists are plain lists, not sets, so "index bits" is a difference of rounded list lengths with repeats counted; a list can get shorter by dropping a duplicate without losing a single candidate, and "a candidate was ruled out" and "the list got shorter" are two different claims. And the difference isn't the number of questions anyone asked.

The covering condition, $|\Omega| \leq L(T) \cdot |\Omega'|$, is the nearest thing the count has to an anti-stopped-clock clause, and I have to be exact about how near. Picture each survivor standing for a little crowd of the candidates that were dropped, a crowd no bigger than the tree has leaves. Then the condition says every candidate in the prior fits in some survivor's crowd. That limits how much can be deleted per leaf you paid for. It doesn't say why anything was deleted. So the condition rules out deleting too much for the tree you've paid for. It doesn't rule out deleting for no reason. A run can satisfy every hypothesis of the count and have ruled nothing out on purpose; the count prices the size of the drop, not the honesty of it.

8.7 What you've got now

A paid flag is not a true claim; Section 7 left it at "a paid flag is a paid flag," and Section 9 will say it flatly. What a paid flag with an object and a checked relation behind it gives you is *entitlement*: anyone auditing the run may treat the claim as established, and can point at the event where that became legitimate. (Nothing in the five parts makes the system itself read the flag; I said that above and I'm saying it again because it's the sentence most likely to get lost.) Entitlement costs at least the index bits of narrowing it commits to, given lists, a tree and a covering somebody supplies, and given a run that paid for the tree's depth. The cups say why entitlement and truth are different. Plato says what the checked relation has to do: distinguish. Necula says how to build it. Hartley says what to count.

The machine I build later has all of these pieces at once, and Section 22 puts this whole section on it, every hypothesis written out, with one worked member you can count by hand. Before any of that, I owe you the word the toll is named after, the "insight" in "No Free Insight" (Section 5), because I've been using it as if it meant one thing.

9 What insight is

I named the rule in this book "No Free Insight," and the name ran ahead of any careful account of what insight is. The book has been narrowing it ever since: not learning

(Section 5), not looking, not copying. It's time to say what's left, and to say it in a way that someone who studies actual insight, in actual heads, wouldn't laugh at. So here's what those people found.

9.1 Sultan and the sticks

In 1913 the psychologist Wolfgang Köhler went to Tenerife to run a research station with a colony of chimpanzees, and the First World War then kept him there until 1920 with nothing to do but watch them. The famous one is Sultan. Food is outside the cage, out of reach; two sticks are inside, each too short, but one fits into the end of the other. Sultan tries each stick, fails, sulks, plays with the sticks, and then, by Köhler's account, at a moment you can see, joins them and reaches the food. Köhler's word was *einsichtig*, which came into English as "insight," and the whole school of psychology he belonged to, the Gestalt school, said that's what thinking is: not stacking up tries until one works, but a *restructuring*, the moment two sticks stop being two sticks and become one tool. Karl Duncker, in 1945, gave the human version that everyone's done in a class: a candle, a box of tacks, matches, and the task of fixing the candle to the wall. Almost nobody sees that the *box* is a shelf, until they do. And then he found that if you hand people the box *empty*, with the tacks poured out beside it, they see it right away. Same objects. What changed was which relation among them was on the table.

I like both of these for the same reason: the insight didn't add an object. Sultan had both sticks the whole time. The box was there the whole time. What arrived was a *relation*, which is to say a piece of structure in Section 2's sense, and the arrival was sudden. Hold that, because it's going to turn out that the thing the toll prices also arrives in one step, and at its best it's also a relation.

9.2 It isn't a feeling

But here's the finding that actually changed how I think about the word, and it comes from someone who measured it. In the 1980s the psychologist Janet Metcalfe asked people to solve problems while rating, every fifteen seconds, how "warm" they felt, how close to the answer. On ordinary problems, algebra for instance, the warmth climbs steadily and peaks at the answer, which is what you'd expect. On *insight* problems, riddles of the candle-box kind, the warmth stays flat, flat, flat, and then the answer arrives, and if anything the people who felt warmest beforehand were *more* likely to be wrong. The feeling of being close is not a gauge of being close. Try it yourself: write a number from one to seven every fifteen seconds while doing a long sum, then while doing a riddle, and compare the curves.

So any account of insight that keys on the sensation, the "aha," is keyed to something that the data says is unreliable. Whatever insight is, it isn't the feeling of having it. That's a relief, because a system of states and moves doesn't have feelings, and a word that only meant a feeling would be cheating if I hung it on one. The version this book can price has to be keyed to an *event*, and the question is which.

9.3 What happens when a computation learns

Here's the split, and I'll say what the toll does with each part, and then I'll say who decided the split, because it wasn't me. Keep Section 3's sheet of paper in front of you: the NO box, the YES box, the tally line, and the three moves STAY, WORK and VOUCH.

The first thing: *raw observation*. Something comes in and gets written down. A counter ticks. Nothing is claimed about what it means, nothing is marked as resting on it, and the reading doesn't move because of it. Later moves can read what it wrote, but that reading is a move of its own. This is the demon's pad in Section 5, and it's what the statistician Dennis Lindley measured in 1956 as "the information an experiment provides," the expected drop in your uncertainty: a real quantity, and a quantity about what was *seen*. It can be free. The toll says nothing at all about a move that doesn't cross from NO to YES, so a price list may charge it zero: on the paper, STAY and WORK cost nothing, and Section 5's watcher narrowed a whole row of candidates without paying a mark. "Can be" is the honest word. A price list is allowed to charge for looking anyway, and whether looking costs anything in the physical world is a different question that this sheet of paper doesn't touch.

The second thing: *certified insight*. Some move turns the reading from NO to YES. Here is where the toll lives. In any system with the five parts that obeys the toll, that move costs at least one mark, and every run that starts at NO and ends at YES pays at least one somewhere along the way, which is Section 3's universal floor. What's being charged for, at its best, is exactly the Gestalt thing: a relation, committed to. Not two sticks; one tool, claimed. "At its best," because Section 3 was careful about what YES means: the flag is up, and nothing more. A crossing that writes down which relation it's committing to carries Sultan's tool with it. A bare crossing, VOUCH on the paper, commits to no relation at all, and it pays exactly the same.

The third thing isn't a kind of event. It's an observation about the price: it's *flat*, whatever is certified. Say the YES state also shows something striking, a deep relation, a result nobody could have guessed, Sultan's two sticks become one tool. The toll charges the crossing at least one. It doesn't charge the striking crossing more, because the floor's proof never looks at anything but the reading before and after. On the paper: VOUCH from NO makes one mark whether you were thinking of Duncker's box or of lunch. Meaning is not cost. The depth of what's certified adds meaning. It doesn't add a mark.

Here they are, exactly, for any certification system (Definition 3.1). Runs are written as lists, the way Section 3's proof wrote them: $[]$ is the empty run, and $i :: t$ is the run that starts with move i and carries on with the run t .

Definition 9.1 (Certified insight event, raw observation). In a certification system, a step from state s by move i is a *certified insight event* when it turns the reading on:

$\text{cert}(s) = \text{no}$ and $\text{cert}(\text{step}(s, i)) = \text{yes}$. A move i is *raw observation* when no step by it, from any state, is a certified insight event.

Remark 9.1 (What the toll prices, and what it doesn't). Read off Definition 3.1 and Theorem 3.1; there is no further Coq theorem at this generality. (1) The toll puts no bound on the cost of a raw-observation move; on Section 3's board STAY and WORK are raw observation and cost 0. (2) Every certified insight event is charged at least 1, by the toll, and every run from a state reading no to one reading yes has total cost at least 1, by the universal floor. (3) The price is flat: the floor's proof uses nothing about the end state but its reading, so adding any hypothesis about what else the end state shows leaves the bound at 1 and cannot raise it. A system that wants to charge more for a deeper certification has to write that into its price list, move by move; the toll doesn't do it for anyone.

So the account comes down to two facts and a warning. Looking can be free. Turning the reading on can't. And putting something remarkable behind the reading doesn't make it cost any more.

Who decided the split? Not the costs alone. The toll prices exactly one thing, the crossing. A price list is free to put a floor under more moves than that (a look, a send, a check) and free to leave other moves at zero, so the sorting into raw observation and certified insight is by whether the reading moves, and that sorting is this book's. The part that's *forced* is the floor under the crossing, and Section 4 is the argument for it; anything a price list charges beyond that is a choice, and any concrete machine owes you its whole table so you can see which choices it made.

9.4 Test, then commit, and nothing forces the second to follow the first

The philosopher Karl Popper said a theory earns its standing by being *exposed*: you make a conjecture, you try to kill it, and what survives a real attempt counts for something, where "real" means the test could have failed. His word for that earned credit is *corroboration*. I've leaned on that idea through this whole book, and here's where a certification system has a version of it, and here's where the version is weaker than Popper and I have to say so.

Add a fourth move to the paper: CHECK. It looks at something (a sum, a coin under a cup, a formula against a row of values), and it can come out pass or fail, and it doesn't move the box. Charge it whatever you like; charging it on the attempt, pass or fail, fits Popper, because what made his test count was that it was really run and could have failed. That's the test. It's raw observation in the sense above, because it never turns the reading on. VOUCH is the commit. So the honest protocol is two moves: test, then commit, and the costs are separate, one for the checking work and one for the crossing.

But nothing in the five parts makes the second move wait for the first. The step is

handed the current state and a move name; the toll looks only at whether the reading crossed. Play VOUCH from NO with no CHECK anywhere before it, and you land in YES and make one mark, the same box and the same mark as the player who checked first. The two tally lines differ by exactly what the CHECK was charged, and if you charged it nothing they don't differ at all. The mark for the crossing is the same mark either way. The receipt says a commitment was paid for. It does not say it was earned. Popper's corroboration needs the test; the toll doesn't.

You can build a system that makes the commit wait. Let CHECK leave a mark in the state when it passes, and write the step so VOUCH only crosses when that mark is there. That's a design, written into the step function, and it's a perfectly good one. Nothing in the five parts or the toll asks for it. That's a real gap between the word and the bare system, and the only honest thing is to leave it visible: the toll prices the commit, and whether the commit was preceded by a test is a fact about the run, or about a design somebody chose, not a guarantee of the five parts.

Remark 9.2 (Nothing makes the crossing wait). Read off Definition 3.1; there is no Coq theorem. The toll constrains $\text{cost}(i)$ for moves that cross, and nothing else; the step function is any function of the current state and the move. So a certification system may have a move that crosses from a state no check has touched, and the toll still holds. On Section 3's board extended with CHECK, the run [VOUCH] from NO crosses with one mark, and so does [CHECK, VOUCH]. Making a crossing depend on an earlier check is a condition on the step function, which a particular system can impose and the definition does not.

9.5 What the word means now

So: insight, in the sense this book prices, is a relation committed to, in a move that flips a reading from off to on, and that move costs at least one mark, in any system with the toll, for any relation, whatever its depth. (And one edge from the paper: a bare crossing that commits to nothing pays the same mark, so "a relation committed to" is what the crossing is at its best, not what the toll guarantees.) Three boundaries, each from a different place. From Metcalfe: it isn't a feeling, and a system's version has to be an event. From the proof: the price is flat; a shallow commit and a deep one both cost at least one, and cost is not depth. A system can pay heavily to check a claim that turns out trivial, and Duncker's box is one small commit. And from the two-move protocol: a paid flag proves nothing on its own. Price and correctness are separate books.

Which leaves a suspicion I'd have in your place. Everything so far has been one reading on one sheet of paper, with a rule I chose and a count I bolted on. Is the shape of that record something I picked, or is it what any honest record has to look like, on any machine at all? That's the next section: take *any* machine that steps, bolt a record onto it, and see what shape it's forced into. How a concrete machine hangs actual

claims on its parts, formulas and labels and where they sit in the state, comes in Part IV.

10 Records over any base: the latch

For the whole book so far I've been building up a thing with a flag and a counter, and a fair reader is entitled to suspect that the whole apparatus is special pleading for one machine, the one I keep promising to build. So here's the section that answers the suspicion with no particular machine in it at all, and sees what's left. Take *any* machine that steps deterministically: a Turing machine, a register machine (a few numbered boxes that each hold a whole number, and a program that adds, subtracts and jumps), a lambda calculus (Alonzo Church's way of computing by nothing but plugging expressions into expressions; Section 11 builds one), anything. Call it the base. Now bolt a record onto it, under rules I'll state, and ask: what can the record be?

The answer is one shape, and I'll show you the shape first because it's a thing you can build from a cup and a die.

10.1 A cup and a die

Put a cup upside down on the table. The rule: the first time a coin lands in front of it, turn it right side up. There is no rule for turning it back over. Now roll a die, and put a coin down on every six. Roll twenty times. The cup turns over on the first six and stays turned over. Write down which roll did it. That number, the roll that turned the cup, is where the record happened, and the cup afterward is the record.

That's a *latch*. Electrical engineers have had the circuit since 1918, when William Eccles and Frank Jordan applied for a patent on a two-valve arrangement that holds whichever state you last put it in; it became the flip-flop, and the registers and caches in the device you're reading this on descend from it. The simplest version has one input, "set," and the rule *new state = old state or set*: once set, it stays set, and there's no reset wire. The cup is that circuit: a flip-flop with its reset wire cut off.

Now the claim, stated loosely and then exactly: *every honest record over any base is a cup with no reset wire, turned by some event the base passes through.*

10.2 What "honest" has to mean

Here's where the content is, and I'll give you the five rules, because the theorem is only as strong as they are.

A machine *extends* a base when there's a map from the big machine's states down to the base's, and that map starts in the base's start states, reaches every one of them, moves in step with the base (one step up top is one step down below, through the map), and agrees about halting. The big machine may carry as much hidden state as it likes beyond the base. That's the "any base" part: the base doesn't get to see the extension, and the extension doesn't get to change what the base does.

The extension carries a *record*, a yes/no reading, and a *ledger*, a counter. It's an honest extension when five things hold. One: the record's next value depends only on the base's current state and the record's current value. (I'll call this *driven by the computation*. It says the record answers to what the machine is doing, not to a clock on the wall or someone's hand.) Two: the ledger never goes down (Section 6). Three: the step that turns the record on costs at least one; that's the toll (Section 3). Four: once on, the record stays on; that's permanence (Section 4). Five: some run from a start state actually reaches a step that turns the record on; otherwise the record is decoration.

Here are the words exactly. On/off is true/false, $\vee$ is "or" (on when either side is on), and next^k is k steps.

Definition 10.1 (Base, record-carrying machine, cover). A *base* B is a set of states with a step function next_B , a set of starting states, and a halting predicate. A *record-carrying machine* M is a set of states with a step function next_M , a set of starting states, a *record* rec from states to Booleans, a *ledger* μ from states to whole numbers, and a halting predicate. The cost of a step is $\text{cost}(m) = \mu(\text{next}_M(m)) - \mu(m)$. A *cover* of B by M is a map π from states of M to states of B that sends starting states to starting states, has every starting state of B as the image of some starting state of M , commutes with the steps, $\pi(\text{next}_M(m)) = \text{next}_B(\pi(m))$, and agrees about halting: m halts exactly when $\pi(m)$ does.

Definition 10.2 (Honest extension, latch). M with cover π is an *honest extension* of B when, for every state m : (R1, driven by the computation) there is one function f with $\text{rec}(\text{next}_M(m)) = f(\pi(m), \text{rec}(m))$; (R2, ledger) $\mu(m) \leq \mu(\text{next}_M(m))$; (R3, toll) if $\text{rec}(m)$ is off and $\text{rec}(\text{next}_M(m))$ is on, then $\text{cost}(m) \geq 1$; (R4, permanence) if $\text{rec}(m)$ is on, so is $\text{rec}(\text{next}_M(m))$; and (R5, a reachable write) some starting state m_0 and some k have $\text{rec}(\text{next}_M^k(m_0))$ off and $\text{rec}(\text{next}_M^{k+1}(m_0))$ on. An *event* is a function h from base states to Booleans, and its *latch* is the step $(b, r) \mapsto (\text{next}_B(b), r \vee h(b))$. M *factors as the latch of h* when, for every state m (reachable or not),

$$(\pi(\text{next}_M(m)), \text{rec}(\text{next}_M(m))) = (\text{next}_B(\pi(m)), \text{rec}(m) \vee h(\pi(m))).$$

Now the theorem. Under those rules, the base state and the record move *exactly* as a latch: the record switches on at the first base state where some event holds, and never off, and the event is read off the extension's own rule by asking it what it does when the record is off (stated exactly below). Everything else the extension carries, all the hidden state, shows up only in its ledger; it can't leak into the record's behaviour.

I promised the content was in the definition, and here's what that means: the proof is a short case split on whether the record is already on, and it uses only *two* of the five rules, "driven by the computation" and "permanent." The ledger, the toll, and the reachable-write clauses play no part in it. A latch is what you get from "answers only

to the base” plus “never comes back down,” full stop. The other three rules matter for the *converse*: for any base and any event the base can reach, the latch of that event, priced at one unit per write, is an honest extension. Put the two together and you’ve got: honest records are exactly latches. One shape, no others, over any base.

Theorem 10.1 (The record is a latch). *Let B be any base and M an honest extension of it through π , with f the function of (R1). Then M factors as the latch of the event $h(b) = f(b, \text{off})$.*

Proof. The base half of the equation is the cover’s step clause. For the record, split on $\text{rec}(m)$. If it is on, permanence (R4) makes the next value on, and on $\vee h(\pi(m))$ is on. If it is off, (R1) makes the next value $f(\pi(m), \text{off}) = h(\pi(m)) = \text{off} \vee h(\pi(m))$. Only (R1), (R4) and the cover are used; (R2), (R3) and (R5) are discarded. $\square$

Checked as `record_axis_is_latch_holds` (StructuralRecordAxis.v), with the definitions `BaseMachine`, `BaseCover`, `computation_driven`, `HonestBaseExtension` and `latch_factorization` (StructuralCoreAnyBase.v), and the clauses (R2) to (R5) as `ledger_carried`, `rc_a2` (StructuralCore.v), `record_permanent` and `reachable_record_write` (StructuralCoreCover.v).

Theorem 10.2 (Every reachable latch is an honest record). *Let B be any base and h any event. Let Latch_h have states (b, r, x) , starting states $(b, \text{off}, 0)$ with b starting in B , step $(b, r, x) \mapsto (\text{next}_B(b), r \vee h(b), x + \delta)$ with $\delta = 1$ when r is off and $h(b)$ is on and $\delta = 0$ otherwise, record r , ledger x , halting as b halts, and cover $(b, r, x) \mapsto b$. Suppose some starting b_0 and some k bring Latch_h from $(b_0, \text{off}, 0)$, in k steps, to a state whose record is still off and whose base state satisfies h . Then Latch_h is an honest extension of B .*

Proof. (R1) holds with $f(b, r) = r \vee h(b)$. (R2): each step adds 0 or 1. (R3): a step that turns the record on has r off and $h(b)$ on, so it adds 1. (R4): $\text{on} \vee h(b)$ is on. (R5) is the supposition. The supposition asks for the first time h holds on some run; any run from a start that reaches an h -state has such a first time, though the file states only the first-time form. $\square$

Checked as `latch_core_honest` (RecordAxisDiscrimination.v).

A reversible base needs one more thing to carry a record, and the files say exactly what. If the base never merges two states, keep every earlier record value in a growing history beside the latch, and the whole step merges nothing either; but on a finite state space a permanent record can’t be written by a step that merges nothing at all. A step that merges nothing is called *injective*: no two states go to the same next state. And $r :: H$ is the history H with r put on the front, as in Section 9.

Theorem 10.3 (Records on reversible machines). (1) Let B be a base whose step is injective. The machine with states (b, r, H, x) , step $(b, r, H, x) \mapsto (\text{next}_B(b), r \vee h(b), r :: H, x + \delta)$ (δ as in Theorem 10.2), starting states with r off, H empty and $x = 0$, has an injective step, and under the supposition of Theorem 10.2 it is an honest extension of B . (2) On a finite state space, a single injective step with a permanent reading never turns that reading from off to on.

Proof. (1) Equal images have equal base parts, so equal base states by injectivity; equal histories $r :: H$, so equal records and histories; and equal ledgers once the equal δ is subtracted. Honesty is the proof of Theorem 10.2 with the history carried along. (2) is Theorem 4.1 of Section 4 with a single move. $\square$

Checked as `history_latch_injective`, `history_latch_honest` and `finite_reversible_cannot_write` (RecordAxisDiscrimination.v), the last from `permanent_flip_is_not_injective` (PermanentCertification.v).

So a base that never merges can still carry a record, if it's allowed to grow a history that never ends. A finite one can't. On a finite machine the record's first write is a merge, and Section 4 already put the price on that.

10.3 Both rules do work: the toggle and the clock

A theorem with two hypotheses should have two counterexamples, one for each hypothesis dropped, and it does, and they're the two ways a record goes wrong in the world.

Drop permanence. Take a base that just counts, and a record that flips at every step: on, off, on, off. It's driven by the computation; it's not permanent; and it's not a latch of any event. This is a *toggle*, a piece of working state. A record you can revoke is working state wearing a record's clothes, and if a revocation has to stick, that's a *second* latch ("revoked"), which is how certificate revocation lists actually work: you add an entry; you never delete the certificate. The pair theorem (stated below) covers two latches that can each read the other; do it with two cups where the second only turns over on an even roll *if the first is already up*, and you'll see the second can never turn before the first.

Drop "driven by the computation." Take a base that counts, and a record that switches on at the fifth tick of a *hidden clock*, whatever the base is doing. It's permanent; it's not driven by the computation. This is a *timestamp*. A timestamp is not evidence; it says *when*, not *what happened*, and a record switched by an outside party or a clock records the party or the clock, not the computation. Try it with the die: play twenty rolls with the rule "turn the cup after roll seven no matter what," and notice you've recorded nothing about the dice.

The file proves exactly the facts I named for each: the toggle is driven, not permanent and not a latch; the clock is permanent and not driven. That each also keeps the other

rules (the ledger is the step count, the first switch-on costs one, a run reaches it) is true by inspection, and isn't a separate theorem.

So: permanent but undriven is a clock, driven but impermanent is a toggle, and a record is the thing that's both. That's the whole discrimination, and I think it's the most useful three-sentence thing in this book for looking at systems other than the one I build, which is why it's here.

The pair theorem and the two counterexamples, exactly. The base for both counterexamples is the counter: states are whole numbers, the step adds one, every state is a starting state, and nothing halts. Two more bits of shorthand: $\neg r$ is r flipped, and a statement in square brackets, like $[k = 5]$, reads on when it's true and off when it isn't.

Lemma 10.4 (Two records are two latches). *Let c_1, c_2 be two records on M , with cover π onto a base B , such that: there is one function F with $(c_1(\text{next}_M(m)), c_2(\text{next}_M(m))) = F(\pi(m), c_1(m), c_2(m))$; the ledger never goes down; a step that turns either record on costs at least one; and each record is permanent. Then there are h_1, h_2 with $c_1(\text{next}_M(m)) = c_1(m) \vee h_1(\pi(m), c_2(m))$ and $c_2(\text{next}_M(m)) = c_2(m) \vee h_2(\pi(m), c_1(m))$ for every m .*

Proof. Take $h_1(b, r_2)$ to be the first part of $F(b, \text{off}, r_2)$ and $h_2(b, r_1)$ the second part of $F(b, r_1, \text{off})$, then split on each record as in Theorem 10.1. $\square$

Checked as `record_pair_is_two_latches_holds` (StructuralRecordAxis.v).

Lemma 10.5 (The toggle). *Let the toggle have states (n, r) , step $(n, r) \mapsto (n + 1, \neg r)$, every state starting, record r , ledger n , cover $(n, r) \mapsto n$ onto the counter. It is driven by the computation, with $f(b, r) = \neg r$; it is not permanent; and it factors as the latch of no event.*

Proof. At $(0, \text{on})$ the next record is off, which breaks permanence; and any latch gives on $\vee h(0) = \text{on}$ there. $\square$

Checked as `toggle_not_permanent` and `toggle_not_latch` (StructuralRecordAxis.v).

Lemma 10.6 (The clock). *Let the clock have states (b, k, r) , step $(b, k, r) \mapsto (b + 1, k + 1, r \vee [k = 5])$, every state starting, record r , ledger b , cover $(b, k, r) \mapsto b$ onto the counter. It is permanent and not driven by the computation.*

Proof. Permanence: $\text{on} \vee x$ is on. Not driven: the states $(0, 5, \text{off})$ and $(0, 0, \text{off})$ have the same base state and the same record, and their next records are on and off, so no f of the base state and the record can give both. $\square$

Checked as `toggle_computation_driven`, `toggle_not_permanent`, `toggle_not_latch`, `clock_record_permanent` and `clock_record_not_driven` (`StructuralRecordAxis.v`). “Driven” is a condition on every state, reachable or not, and that is what the clock’s proof uses.

10.4 Is there only one such machine? No. One structure, two free choices.

Here’s a sentence that’s tempting and wrong, and the way it’s wrong is more interesting than the sentence: “there’s only one honest way to carry a record.” Three different things get squeezed into that breath, and I’ll pull them apart.

First claim: given the toll and a fixed price list, is there one way to keep the books? Yes; that’s Section 6’s initiality, and it assumes the price list.

Second claim: is every honest extension of a base *the same machine*, charges and all? No, and the counterexample is embarrassing in its simplicity. Take any honest extension and add one more coin to the bill on every step, whatever the step is. Now walk the five rules. The record still answers only to the base and its own value (R1); the bill never touches it. The ledger still never goes down (R2); it climbs faster, if anything. The step that turns the record on still costs at least one (R3), because the toll is a *floor* and a surcharge only raises the price. Once on, the record stays on (R4), and the same run still reaches the write (R5). So the billed machine is honest, and it isn’t the machine you started with: wherever the original sat still for free, the billed one pays a coin a tick for sitting still. Do it with the cup and the die. Charge a coin for every roll, six or not. The cup turns over on the same roll as before. You’ve changed the bill and nothing about the record. A machine that charges for CPU time is honest and isn’t the one that doesn’t.

But the billed machine isn’t a different *structure*. It’s the same machine with a different price list. So the claim worth keeping is sameness *up to the price schedule*: compare two machines by their record, their halting and their steps, through their maps down to the base, and don’t compare charges. Fix which event the record is, and the bill stops mattering; a flat surcharge, or any surcharge that depends on the current state, sits on the same structure. That’s the first free choice. The walk above is the whole argument, and it’s argued here, not checked over an arbitrary base: the checked statements, the exact uniqueness refuted and the uniqueness up to schedule proved, are written for the machine I build later, with its own certification reading as the record, and they’re Section 23’s.

And then the second free choice, and this one still surprises me. *Which event* is the record? Nothing in the five rules picks it. The converse above already says so, if you read it twice: for any event the base can reach, the latch of that event, priced at one unit per write, is an honest record. Do it with the die. One cup turns over on the first six. Another turns over on the first one. Both are cups with no reset wire; both are honest. Roll a one first, and until a six turns up the two cups disagree about whether anything has been recorded. So *which permanent reading counts as the record* is a second thing

nobody forces on you. The price list is one free choice. The event is the other. That second choice is the question Section 13 is about: what, if anything, makes certification the event worth recording, rather than “the ledger passed one” or anything else.

Corollary 10.7 (The event is a free choice). *Let B be any base and h_1, h_2 two events, each meeting the supposition of Theorem 10.2. Then Latch_{h_1} and Latch_{h_2} are both honest extensions of B . If some starting base state b has $h_1(b)$ on and $h_2(b)$ off, then one step from $(b, \text{off}, 0)$ leaves the first record on and the second off.*

Proof. Honesty is Theorem 10.2 applied twice. The step is the latch’s own rule: $\text{off} \vee h_1(b)$ is on and $\text{off} \vee h_2(b)$ is off. $\square$

Argued from `latch_core_honest` (`RecordAxisDiscrimination.v`) applied to each event; the corollary itself is not a separate Coq statement.

Now let the cup wobble. Say whatever turns it over sometimes does and sometimes doesn’t, with odds you could write on a card. Does fixing the price list fix the odds? No, and the counterexample is two cups with the same price tag and differently loaded coins. Here’s the setup, said carefully. Let a record be a single bit that moves at random: from each value, a list of weighted next values, every weight positive, never empty, never leaving on once on, and with the write from off to on charged at least one. Call two such records *the same schedule* when the same moves are possible and every move is charged the same, and *the same in probability* when the weighted lists are equal.

Theorem 10.8 (The schedule doesn’t fix the probabilities). *(1) There are two honest probabilistic records with the same schedule that are not the same in probability. (2) There is an honest probabilistic record that no deterministic latch on its current value describes.*

Proof. (1) From off, one record moves to off or on with weights 1 and 1, the other with weights 1 and 2; from on, both stay on with weight 1; both charge 1 for the write. Same possible moves, same charges, different weights. (2) The first record moves from off both to off and to on, while a latch $r \vee h(r)$ gives one value. $\square$

Checked as `schedule_determines_probabilities_refuted` and `deterministic_latch_handles_branching_refuted` (`ProbabilisticRecord.v`), with the definitions in `ProbabilisticRecordCore.v`. These are finite-weight models on one bit with no base underneath; they say nothing about any particular machine.

Third claim: is there one state space, up to renaming, that every honest record-carrier has to have? I don’t claim it. Nothing needs it.

10.5 Bases from outside, and a relation I have to be honest about

The theorem is stated for any base, and the checked work instantiates it on three models of computation that other people built: a small Turing machine (any program); a register machine of the kind Cook and Reckhow described in 1973, with unit-cost indirect addressing (any program); and a lambda calculus (call-by-value, meaning an argument is worked out before it's plugged in, and made deterministic by letting a term with no step left to take step to itself). On each, the latch form holds, by the one generic theorem applied to that base (the checked names are listed with the formal statement below). So a register machine with an honest record is a register machine plus a cup. So is a lambda calculus. (There's a fourth instance, the machine I build later; it's Section 23's.)

Here are the three bases explicitly. A base needs a state, a next-state function, a set of starting states and a halting predicate; the latch theorem asks for nothing else about its instruction vocabulary.

Definition 10.3 (Three concrete bases). The **tape base** has a finite Boolean list t , a head index h , a program counter q and a natural-number cost field. Reading beyond the list returns false. Writing at h extends the list with false cells if needed, then changes that cell. Moving right increments h and extends the list through the new head position. Moving left decrements a positive h ; at $h = 0$, it prepends a false cell and keeps the head at zero. A fixed finite program contains these instructions:

Instruction	Next state
Write a bit	Perform the write and advance q by one.
Move left or right	Perform the move and advance q by one.
Branch to j	Set $q := j$ if the current cell is true; otherwise advance q by one.
Halt	Leave the entire state unchanged.
ClaimTapeIsZero with parameter δ	Advance q by one and do nothing else.

An out-of-range program fetch is Halt. Every state is allowed as a starting state, and halting means the fetched instruction is Halt. The cost field is preserved by every instruction in this base. In particular, its ClaimTapeIsZero instruction neither erases the tape nor charges the cost field; the name supplies no extra semantics.

The **random-access base** has a register function $X : \mathbb{N} \rightarrow \mathbb{N}$ and a program counter q . For a fixed finite program its instructions are:

Instruction	Next state
Constant (i, c)	Set $X(i) := c$.
Add (i, j, k)	Set $X(i) := X(j) + X(k)$.
Subtract (i, j, k)	Set $X(i) := \max(0, X(j) - X(k))$.

Instruction	Next state
Indirect load (i, j)	Set $X(i) := X(X(j))$.
Indirect store (i, j)	Set $X(X(i)) := X(j)$.
Positive branch (j, a)	Set $q := a$ if $X(j) > 0$; otherwise advance q by one.
Jump to a	Set $q := a$.
Halt	Leave the entire state unchanged.

Each register-writing instruction advances q by one; its operands are evaluated in the old state, and all other registers are preserved. A fetch outside the program also leaves the state unchanged. Starting states have $q = 0$ and arbitrary register contents. A state is halted when it fetches Halt or is outside the program. Unit cost here concerns instruction counting, not the certification ledger.

The **lambda base** uses the terms and weak call-by-value rule defined in Definition 11.3. To compute its next state, first try to reduce the function part of an application. If that part cannot step and is an abstraction, either substitute an argument that is already an abstraction or take a step in the argument. If neither applies, the term cannot step. Variables and abstractions cannot step by themselves, and there is no reduction under an abstraction. The base leaves a term unchanged when it cannot step and calls precisely those terms halted. Every term is allowed as a starting state, including open terms.

These definitions make the examples instances of the base record. Applying the latch theorem to them does not assert universality, an execution-time bound, or a simulation between the three bases.

There's also a relation defined between bases, *weak base equivalence*: two bases are equivalent when some correspondence covers their start states both ways, related states agree on an observation and on halting, and one step on one side is matched by some number of steps, possibly zero, on the other. That "some number of steps" is the *stuttering* idea Leslie Lamport (the same Lamport as Section 6's message counters) put forward in 1983: a fine-grained machine and a coarse one shouldn't count as different because one takes three ticks to do what the other does in one. The relation carries no cost; a machine that multiplies huge numbers in one step and a Turing machine differ in price, not in records. It's proved to be an equivalence relation, and the latch form is preserved across it.

There is a limit to that statement. The "preserved across it" proof throws away its equivalence hypothesis; the latch holds on every base regardless, so the preservation is true for a boring reason. And nowhere in the checked work are *two* of the bases actually related by this relation. The relation is defined, it's an equivalence, and it is not used to connect the Turing machine to the register machine to the lambda calculus,

or any of them to the fourth instance. It would be easy to read that list as if it did. What's proved is that each base, separately, carries a latch. That's the honest sentence.

Definition 10.4 (Weak base equivalence). Bases B_1 and B_2 , with observations o_1 and o_2 into one common set, are *weakly equivalent* when some relation R relates every starting state of each to some starting state of the other, and any related x, y have $o_1(x) = o_2(y)$, the same halting, some $n \geq 0$ with $R(\text{next}_{B_1}(x), \text{next}_{B_2}^n(y))$, and some $n \geq 0$ with $R(\text{next}_{B_1}^n(x), \text{next}_{B_2}(y))$.

Theorem 10.9 (Outside bases, and the relation). (1) *Weak base equivalence is reflexive, symmetric and transitive.* (2) *If two bases are weakly equivalent, Theorem 10.1 holds over one exactly when it holds over the other.* (3) *Theorem 10.1 holds over each of the three bases in Definition 10.3, for every fixed program where the base has one.*

Proof. (1) Reflexive with equality as R and one step against one. Symmetric by reversing R . Transitive by composing the two relations; a step on the first base is matched by n steps on the second, and those n steps by some number on the third, by induction on n . (2) and (3) apply Theorem 10.1 directly. In (2) the equivalence hypothesis is discarded. $\square$

Checked as `weak_base_equiv_refl_holds`, `weak_base_equiv_sym_holds`, `weak_base_equiv_trans_holds` (through `weak_match_left_runs` and `weak_match_right_runs`), `weak_equiv_preserves_record_latch_holds` and `record_axis_is_latch_on_tm_holds` (`CrossBaseGranularity.v`), `record_axis_is_latch_on_ram_holds` (`CrossBaseGranularityRAM.v`) and `record_axis_is_latch_on_l_holds` (`CrossBaseGranularityL.v`), with the definitions in `CrossBaseGranularityCore.v`.

10.6 Records with more than two values

A cup has two positions. Some records have more: a counter of how many checks passed, a level that only rises. The same reasoning goes through one threshold at a time. For each value a , “the record has reached a ” is a yes/no reading, and each of those readings is a latch. What doesn't go through is squeezing the whole record into one cup. “Rises” needs one word: the values come with an order $\leq$, “is at or below,” which needn't compare every pair (that's what makes it a *partial* order), and $[a \leq x]$ is the yes/no reading “ x has reached a .”

Definition 10.5 (A growing record). Let the record take values in a set A with a partial order $\leq$ decided by a Boolean test. A record ρ on M , with cover π onto a base B , is an *honest growing record* when: there is one function f with $\rho(\text{next}_M(m)) = f(\pi(m), \rho(m))$; it grows, $\rho(m) \leq \rho(\text{next}_M(m))$; the ledger never goes down and every step that changes ρ costs at least one; and some run from a starting state reaches a step

that changes it.

Theorem 10.10 (Growing records are families of latches). (1) For every honest growing record there is an h with

$$[a \leq \rho(\text{next}_M(m))] = [a \leq \rho(m)] \vee h(a, \pi(m), \rho(m))$$

for every value a and state m : each threshold is a latch, whose event may read the current value. (2) The thresholds determine the value: if $[a \leq x] = [a \leq y]$ for every a , then $x = y$. (3) For a growing record, charging every change is the same as charging every threshold that switches on. (4) One latch is not enough: a three-valued record that climbs $0 \rightarrow 1 \rightarrow 2$ over a one-state base, each climb charged one, is honest, and no single Boolean latch with a decoding from the base state and the latch reproduces it. (5) A chain of distinct k -bit words, each word's set bits contained in the next one's, has at most $k + 1$ members.

Proof. (1) Take $h(a, b, x) = [a \not\leq x] \wedge [a \leq f(b, x)]$. If $a \leq \rho(m)$ already, growth and transitivity give $a \leq \rho(\text{next}_M(m))$; otherwise the equation is the definition of h . (2) Antisymmetry, with $a = x$ and $a = y$. (3) Growth plus antisymmetry turn “the value changed” into “some threshold switched on,” and back. (4) Over one base state, a latch and a decoding can produce at most two values, and the record takes three. (5) Each strict step up the chain adds at least one set bit, and there are only k bits. $\square$

Checked as `growing_record_decomposes_holds`, `thresholds_determine_record_holds`, `record_price_iff_threshold_price_holds`, `one_latch_refuted` and `chain_needs_bits_holds` (GrowingRecord.v), with the definitions in GrowingRecordCore.v. The order need not be finite; (5) is the finite-memory bound.

10.7 What the base being free buys you

The reason this section exists is to answer the suspicion I started with. The record isn't special pleading for one machine. Over any base, under rules you can read, a record that answers to the computation and never comes down is a latch on an event, and the proof doesn't care what the base is. What a particular machine adds is not the latch; it's a *choice of event* and a *choice of price list*, and both are choices. The surcharged cup shows the first: the bill can change and the record not notice. The two cups on one die show the second: the rules are happy with either event. Any machine I build will have to make both choices, and whether its choice of event is the right one is an open question, and I'll get to it. Before that, the question every machine with a record has to face, and that no amount of good design makes go away. A record is worth keeping because something later leans on it instead of working the thing out again: that's Section 2's glance, reading a block's bucket off the table instead of applying the rule from scratch. Call any move like that a *shortcut*, and call it *sound* when it gives the answer the long way would have given. So: can a machine *decide*, from inside, whether its own shortcuts are sound? That's a wall, and it's the same wall Gödel and

Turing hit, and I have to show you that it's there before I show you anything that pretends to be on the other side of it.

11 The diagonal

In Section 2 I said you'd watch the same move get made again and again as the book went on: somebody says "everything is X," it's true enough to do real work with, and then somebody draws the diagonal. I owe you the diagonal itself now, because Section 10 just handed a record to *any* system that steps, and the next thing anyone wants from a system with a record is for it to tell sound shortcuts from unsound ones by itself. Before I let anything claim that, I have to show you the wall every such claim runs into, and then show you exactly where a system with a record stands relative to the wall. Not on the far side. Nobody's on the far side.

11.1 Five names and a table

Do this first, with a pencil, because the whole family of arguments is this one trick and you should own it. Write five friends' names down the left of a page. Next to each name, write a five-digit string of zeros and ones, anything you like. Now build a new string: its first digit is the *opposite* of the first digit of the first row; its second digit is the opposite of the second digit of the second row; and so on down the diagonal. Look at what you've made. It differs from row one in position one, from row two in position two, from every row at that row's own position. So it's nobody's string. Your table of five missed something.

With five rows and five digits that's a cute fact. Georg Cantor's point, in 1891, was that it works for *any* list, however long, even infinite: hand me a complete list of all infinite strings of two symbols, I'll walk the diagonal flipping as I go, and I'll hand you back a string that's on no row. So there is no complete list. There are more such strings than there are places on any list, and that's where "some infinities are bigger than others" comes from. (He wrote the two symbols as m and w, not 0 and 1, and that paper wasn't even his first proof of the fact; the 1874 one was harder. The diagonal was him finding the easy way.)

Now look at what the argument *did*, because it's going to do the same thing again and again, in new clothes, and I want you to recognise it each time. (Not every name below is a flipped diagonal in the table sense. Kleene's theorem is the tool that makes the diagonal buildable, Rice's is the general form, and Thompson's is an attack that uses the same trick.) There was a table: rows are the things being listed, columns are questions, each cell says "does row i do column j." There was the diagonal: the cells where a thing is asked about *itself*. There was a flip: something with no fixed point, like turning yes into no, applied along the diagonal. And the thing that got refuted was *the assumed object*, the complete list. Nothing outside the assumption was touched. Table, diagonal, flip, and the assumption eats itself. Hold those four.

11.2 The same move, again and again

Russell, 1902, from Section 2: the collection of all collections that don't contain themselves. That's Cantor's flipped diagonal with "contains" as the table, and Russell said as much; he got there by running Cantor's argument on a supposed collection of everything. What got refuted was the assumed object, the collection of everything, and Zermelo's fix was to stop assuming it.

Jules Richard, 1905: some English phrases define real numbers ("the ratio of a circle's circumference to its diameter"); there are only countably many phrases, so list the numbers they define in order; run Cantor's diagonal down their decimal expansions and you get a real number no phrase defines, except that you just defined it, in the phrase that describes the diagonal. The lesson that came out of that one is the lesson this whole section turns on: keep apart the language a system *talks in* and the language you use to talk *about* it. Mix them and the diagonal bites you.

Kurt Gödel, 1931, with an improvement by Barkley Rosser in 1936: the big one, and I'll say exactly what it says because it's misquoted more than any theorem alive. Take any consistent formal system with a mechanically checkable list of axioms that can do a bit of arithmetic. Then there are sentences it can neither prove nor refute. The engine is a lemma that lets you build, for any property you can write down, a sentence that the system proves *equivalent* to that property holding of the sentence's own code number. Apply it to "not provable" and you get a sentence equivalent, inside the system, to its own unprovability. Table: sentences by code. Diagonal: a sentence about its own code. Flip: provable to not-provable. Two cautions the Stanford Encyclopedia stresses and I'll repeat: the sentence doesn't literally "say" anything about itself, it's provably *equivalent* to a claim about its code, which is less mystical and more exact; and the theorem doesn't say the sentence is unknowable, it says *this one system* can't settle it.

Alan Turing, 1936: and here the historical record is not the story you usually hear. The usual story is that Turing proved the halting problem undecidable. Joel David Hamkins and Theodora Nenu went back to the 1936 paper and found that the word "halting," the modern statement, and the self-referential proof everyone teaches are all, strictly, absent from it. What Turing proved undecidable were two other questions about his machines (whether a machine prints infinitely many digits; whether it ever prints a given symbol), which are *equivalent* to halting, and the name "halting problem" came later, probably from Martin Davis. The textbook version is still right as mathematics, and here it is, on two index cards. Card A is the Checker: hand it any program card and it announces "halts" or "runs forever." Card B is the Contrarian: "Hand me to the Checker. If it says I halt, run forever. If it says I run forever, stop." Hand B to A. Either answer is wrong. There's no Checker. Table: programs by programs. Diagonal: a program run on itself. Flip: halt to loop.

Stephen Kleene, 1938, and this is the one that turns the paradox into a tool, so slow down. Everything above needs the self-referring object to *exist*: Gödel's sentence has

to be buildable, the Contrarian has to be able to “hand me to the Checker.” Kleene’s second recursion theorem says: for any transformation of programs you can compute, there’s a program that behaves exactly as if it had been handed its own source code. Any program may be written as though it already had a copy of itself in hand. He stated it almost in passing, in a paper on something else, and proved it in two lines, and Yiannis Moschovakis calls it a little gem, and it’s the reason “programs that print their own source” exist in every programming language (they’re called quines, after the philosopher; try “Print the following twice, the second time in quotes: ‘Print the following twice, the second time in quotes:’” and obey it). Self-reference in computation isn’t a paradox. It’s a construction. The paradoxes happened only when someone assumed an *unrestricted* collection or language; the theorem happens when you build the self-referrer honestly.

Henry Rice, 1953, whom you met briefly in Section 7: the general form. Any property that depends only on what a program *does* (not how it’s written), and that’s true of some programs and false of others, has no algorithm that decides it for all programs. “Does this program halt” is one such property. “Does this program ever output 7” is another. “Does this program contain an if-statement” isn’t, because that’s about the text. One standard proof goes straight through Kleene.

And the one that’s actually an attack: Ken Thompson, in his Turing Award lecture, published in 1984. Build a compiler that, when it compiles the login program, plants a back door, and when it compiles *a compiler*, plants both habits into that one too. Then delete the trick from the source. The compiled compiler still carries it, and recompiling a clean source with the poisoned binary re-plants it, forever. Reading the source never reveals it. A locksmith teaches an apprentice to cut a spare for every town-hall key and to teach *that* to every apprentice they train, and never to write it in the manual; years later the manual is spotless and every locksmith in town cuts the spare. That’s the recursion theorem with a knife in it, and his moral was that you can’t trust code you didn’t write entirely yourself, which is a sentence I’ll come back to in Section 28 when I tell you what the trusted base of this book is.

Bill Lawvere, 1969, showed all of these are one theorem in disguise: if a set can be mapped onto all the functions from itself to some target, then every function on the target has a fixed point; so if the target has a function with no fixed point (a flip), no such covering exists. Cantor, Russell, Gödel, halting, as instances of one shape. Noson Yanofsky retold it in 2003 without the category theory. I mention it because “it’s the same wall” is a claim I make below, and I want you to know it’s fair at the level of *proof shape*, and that every instance has its own boundary of *who counts as a decider*, and that boundary is exactly where most overclaims live.

11.3 What it doesn’t say

Before I point this at a system with a record, the four wrong turns, because they’re the ones people take.

“Gödel shows minds aren’t machines.” John Lucas argued it in 1961 and Roger Penrose revived it twice. The standard replies have held up: the Gödel sentence is true *only if the system is consistent*, so a human who “sees” its truth has to know the system is consistent, which is the hard part; and nobody can build the Gödel sentence of a system they can’t specify, so humans might be machines too complicated to diagonalise. The theorem has a conditional in it and the argument drops the conditional.

“Undecidable means unknowable.” No. Two senses share the word: a problem with no single algorithm correct on every instance, and a sentence independent of one particular system. Neither means nobody can know a given answer. Any single instance is decidable, trivially: for any one program, one of “say yes” and “say no” is right. What’s ruled out is one *inside* procedure that’s right on *every* case.

“Everything interesting is undecidable.” Rice is about behavioural properties over *all* programs. It says nothing against checking the text, nothing against deciding for specific programs, nothing against analyses allowed to say “I don’t know,” nothing against finite-state model checking or type checking. Real compilers are limited by cost and by choosing to be sound-but-incomplete, not by the diagonal.

“Self-reference is a paradox.” It’s a theorem with a construction. Kleene, quines, Thompson.

11.4 Now point it at the record

Here’s the question Section 10 left open, with nothing in it but what Parts I to III have built. A system steps; a record rides along and latches on an event (Section 10); when the event is a certification, Section 8 said the reading, with its object and its paid event behind it, is what lets an auditor treat a claim as established, and for a claim that a shortcut is sound, that’s what makes the shortcut *admissible*. So: can a system like that have, as one of its own programs, a procedure that correctly decides, for every program, “does this program admit a sound shortcut”? That would be autopilot. Hand it any program and it says yes or no, and it’s always right.

That’s the question in plain words, and the checked statement is narrower, so here it is before the proof. In the theorem, “admits a shortcut” is any property of programs with two features: two programs that behave alike agree on it, and there’s one program that has it and one that doesn’t. That’s all the argument gets to look at. The diagonal doesn’t see soundness. Each system that wants the wall picks its own property, and whether that property lines up with being entitled to soundness in Section 8’s sense is a separate fact about that system, checked one system at a time if it’s checked at all. A procedure that returns soundness records for some programs, or only for restricted inputs, or with outside evidence, is not what the wall rules out.

The answer is no, under premises. The proof is the two index cards, and I’ll give you the premises because they’re the whole content. Suppose there were such a decider, one that says yes exactly when a program admits a shortcut. Build the flip: “if the

decider says yes about me, behave like a program that admits no shortcut; if it says no, behave like one that does.” If the system has a recursion theorem (a program can be handed its own code), and if that flip is one of the transformations the system can carry out, and if “admits a shortcut” depends only on behaviour (two programs that behave the same agree on it), and if there’s at least one program each way, then there’s a program that behaves exactly like its own flip, and whatever the decider says about it is wrong. Both cases close in a line each. That’s `structural_shortcut_undecidable`, stated for *any* substrate with those fields. (A substrate here is the bare shape with exactly the pieces this theorem asks for: programs, a run that may fail to give back a state, a counter, an encoding of programs as data, a notion of “representable” transformation, and the recursion theorem for those. Nothing about what the states are made of. The smallest checked instance is the nat family below.) It’s Rice’s trick one floor over, and I should be careful with that. Rice’s theorem is stated for the functions Turing machines compute. This wall is the same diagonal with different premises: it carries its own recursion theorem and its own list of representable transformations, and it picks the property. So it isn’t a corollary of Rice as Rice stated it, and it doesn’t separate anything from classical halting. And here’s something about its proof that I want you to have: it never touches the counter, the toll, or the step relation. It uses the recursion theorem, representability of the flip, the two witnesses, and extensionality, and nothing else. (The file’s header calls the substrates it covers ones that respect the toll, which reads as if the toll were needed. The proof uses neither the toll nor the counter’s climbing. It’s the diagonal, clean.)

Here’s the setting and the wall, with every word pinned down so you can poke at it.

Definition 11.1 (Substrate). A *substrate* has a type of states, a type `Prog` of programs, and a run map that takes a program and a state and either gives back a state or gives nothing. It has a counter μ on states and a step relation along which μ never decreases; an encoding `enc` of programs as states with a decoder `dec` such that $\text{dec}(\text{enc}(p)) = p$; and a predicate `Rep` on maps $f : \text{Prog} \rightarrow \text{Prog}$, the transformations the substrate counts as its own. Two programs are *equivalent*, $p \simeq q$, when $\text{run}(p, s) = \text{run}(q, s)$ for every state s . The last field is the *recursion theorem*: for every f with `Rep`(f) there is a program p with $p \simeq f(p)$.

Definition 11.2 (Shortcut predicate). A *shortcut predicate* on a substrate is a property A of programs with two programs y and n such that $A(y)$ holds, $A(n)$ fails, and A is extensional: $p \simeq q$ implies $A(p) \iff A(q)$. For a Boolean function d on programs, its *flip* is $f_d(p) = n$ if $d(p)$ is true and $f_d(p) = y$ if $d(p)$ is false.

Theorem 11.1 (No inside decider). Let a substrate carry a shortcut predicate A with witnesses y and n . Then there is no Boolean function d on programs such that `Rep`(f_d) holds and, for every program p , $d(p) = \text{true} \iff A(p)$.

Proof. Suppose d is such a function. Since $\text{Rep}(f_d)$, the recursion theorem gives a program p with $p \simeq f_d(p)$. Split on $d(p)$. If $d(p)$ is true, then $A(p)$ holds by correctness, and $f_d(p) = n$, so $p \simeq n$, and extensionality turns $A(p)$ into $A(n)$, which fails. If $d(p)$ is false, then $A(p)$ fails by correctness, and $f_d(p) = y$, so $p \simeq y$, and extensionality turns $A(y)$ into $A(p)$. Both cases are contradictions. The argument uses the recursion theorem once, the two witnesses, extensionality, and nothing about μ , the step relation, or the encoding. $\square$

Checked as `structural_shortcut_undecidable` (`StructuralUndecidability.v`; the correctness it rules out is $d(p)$ true exactly when $A(p)$), stated over the class `Substrate` (`Substrate.v`) and its extension `WithShortcutPredicate`. Every field named above is a premise; the theorem is about every substrate that supplies them.

So the wall is there. Now the honest part: *who counts as a decider*. The theorem rules out a decider whose flip the substrate can itself represent. It says nothing about a decider standing *outside*, and an outside decider can exist for the very same property, correct on every program. That's not a loophole; it's the boundary of the sentence, and the boundary is Richard's lesson: the language you talk about a system in is not the language it talks in. You'll see it in miniature in a minute, on the `nat` family, with a pencil.

Two forms of the wall live in this chapter, and they're honest in different ways.

The *nat family* (`nat_self_undecidable`): a tiny substrate built so that program two is the Contrarian for a given candidate decider d , and for every d , d is wrong about program two. It's a *family*, one instance per d , and its recursion theorem holds by construction. It's the index cards made formal and nothing more. Don't let it sound like more.

Theorem 11.2 (The *nat* family). *Fix any Boolean function d on $\mathbb{N}$. Let the substrate N_d have programs and states $\mathbb{N}$; program 0 returns 1, program 1 returns 0, program 2 returns 0 if $d(2)$ is true and 1 if it is false, and every other program gives nothing; μ is constantly 0; $\text{Rep}(f)$ holds when $f(p) = 1$ for every p with $d(p)$ true and $f(p) = 0$ for every other p ; and $A(p)$ holds when program p returns 1 on input 0. Then it is not the case that $d(p) = \text{true} \iff A(p)$ for every p .*

Proof. The recursion theorem of N_d holds with the fixed witness 2: for a representable f , $f(2)$ is 1 or 0 according to $d(2)$, and program 2 returns exactly what that program returns. The witnesses for A are 0 and 1. The flip of d is representable by the definition of Rep , so Theorem 11.1 applies to d itself. Directly: $A(2)$ holds exactly when $d(2)$ is false. $\square$

Checked as `nat_self_undecidable` (`NatSubstrateInstance.v`), with the built-in recursion theorem `nat_recursion_theorem`; the general form, no decider whose

flip is representable in N_d , is `nat_structural_shortcut_undecidable` in the same file.

Now push on it once more, because the `nat` family shows the outside decider in three lines, and you can do them yourself. Stand outside N_d , knowing d . Write down a new decider e : say yes on program 0, say “not $d(2)$ ” on program 2, and say no on everything else. Check it against A . Program 0 returns 1, so $A(0)$ holds and e says yes. Program 1 returns 0, so $A(1)$ fails and e says no. Program 2 returns 1 exactly when $d(2)$ is false, and that’s what e says. Every other program gives nothing, so A fails and e says no. So e is right about every program. Does that break the wall? No, and here’s the bit that matters: in N_d a flip is representable only if it says 1 exactly where d says yes, so the flip of e is representable only if e and d agree everywhere, and they disagree at program 2. The correct decider exists; its flip just isn’t one of the substrate’s own moves. (That one isn’t a separate theorem in the Coq. It’s three lines and a pencil, and you just did it.) Undecidable from inside and decided cleanly from outside, at the same time, about the same property. That’s not a paradox. It’s two different people asking.

And the one where the recursion theorem is *proved*, not assumed, which is why it exists: L , the weak call-by-value lambda calculus that Yannick Forster and Gert Smolka presented in 2017 as a model of computation in Coq, and which a group at Saarland has since used as the seed of a whole library of undecidable problems. The development here builds its own copy of L from the reduction rules, so the L results below assume nothing from that library. There, `L_recursion_theorem` is a theorem with a proof, Rice’s theorem follows (`L_rice`), halting follows (`L_halting_undecidable`), and the diagonal is reproved in the shape of the substrate theorem (`L_structural_shortcut_undecidable`), for any property of programs that depends only on what they compute and that holds of one program and fails of another. The statement mentions no shortcut. That’s the diagonal for a fixed model with nothing assumed. It isn’t an instance of the generic class (L ’s runs aren’t total in the way the class wants), so it’s the argument *reproved*, not the generic theorem *applied*. Both are true; they’re different sentences.

Here’s what that string of words means, because L is the strangest thing in this book that computes, and the smallest. The lambda calculus is Alonzo Church’s, from the 1930s, and it’s a programming language with one idea in it: functions. A term is one of three things. A variable. A function, written λ followed by its body, the body saying what to do with whatever you hand it. Or one term applied to another, written side by side. Running a program is one move, over and over: when a function meets an argument, put the argument into the body where the variable was. No numbers, no memory, no instructions, and Turing showed in 1937 that it computes exactly what his machines compute. “Weak call-by-value” is two house rules about the order of moves: an argument is run down to a finished function (a *value*) before it’s put in, and nothing runs inside a function that hasn’t been handed anything yet. Variables

are written as numbers counting how many λ s back they were bound (Nicolaas de Bruijn's trick, so two variables can never get mixed up by sharing a name), and a term is *closed* when every variable in it is bound inside it, so it leans on nothing outside. Last piece: to hand a program to a program, L needs a way to write any term as data, the way a program's source is a string you can pass around. The one used here is the *Scott encoding*, named for Dana Scott, and all you need about it is that every term t gets one *code*, written $\ulcorner t \urcorner$, and the code is itself a closed value. Two tools get built on top of that in the proof: a *quoting* term Q that takes the code of a term to the code of that code (a program that prints its own source needs exactly this move), and a *fixed-point combinator*, a term that lets a function call itself, which is how Q works its way through a code of any size.

Definition 11.3 (L, as built here). A term of L is a variable (a de Bruijn number), an application st , or an abstraction λs ; the values are the abstractions. One step is weak call-by-value, left to right: $(\lambda b) v$ steps to b with v substituted when v is a value, a step inside the left part of an application is a step of the application, and so is a step inside the right part once the left part is a value. A term s *evaluates to* v when s reaches v in some number of steps and v is a value; $p \equiv q$ when p and q evaluate to the same values. Every term t has a code $\ulcorner t \urcorner$, its Scott encoding, which is a closed value. A map f on terms is *L-computable* when some closed value s has $s \ulcorner p \urcorner \equiv f(p)$ for every p . A term D *decides* a property P of terms when, for every p , $D \ulcorner p \urcorner$ reaches $\lambda \lambda.1$ (true) if $P(p)$ holds and $\lambda \lambda.0$ (false) if it fails. P is *extensional* when $p \equiv q$ and $P(p)$ give $P(q)$.

Theorem 11.3 (Recursion, Rice and halting for L). (1) For every closed value s there is a closed term t that reaches $s \ulcorner t \urcorner$; hence every L-computable f has a term p with $p \equiv f(p)$. (2) Let P be extensional, with $P(y)$ and not $P(n)$ for closed terms y and n . Then no closed term decides P . (3) No closed term decides halting, the property of evaluating to some value.

Proof. (1) A closed quoting term Q takes $\ulcorner t \urcorner$ to the code of $\ulcorner t \urcorner$; its constructors, recursive program and terminating induction are given in Section G. A closed term mkapp takes two codes to the code of their application. Let $A = \lambda x. s (\text{mkapp } x (Q x))$ and $t = A \ulcorner A \urcorner$. One step gives $s (\text{mkapp } \ulcorner A \urcorner (Q \ulcorner A \urcorner))$, and the argument reduces to $\ulcorner A \ulcorner A \urcorner \urcorner = \ulcorner t \urcorner$. For the second half, take s computing f ; then $t \equiv s \ulcorner t \urcorner \equiv f(t)$. (2) Given a closed decider D , the flip $\lambda x. D x (\lambda _. n) (\lambda _. y) (\lambda z. z)$ is a closed value; on the code of p it reaches n when D answers true and y when D answers false. Take its t from (1). If $P(t)$, then $t \equiv n$ and $P(n)$; if not, $t \equiv y$ and $P(t)$. Both are contradictions. (3) Halting is extensional; apply (2) with $y = \lambda z. z$ and $n = (\lambda x. x x)(\lambda x. x x)$, which steps only to itself and so never reaches a value. $\square$

Checked as `second_recursion`, `L_recursion_theorem`, `L_rice`, and `L_halting_undecidable` (LRecursion.v), with the quoting term's specification `Q_spec` and the substrate-shaped form `L_structural_shortcut_undecidable` (no decider

whose flip L can compute says true exactly when the property holds) in the same file. The file uses only Coq's standard library.

That's the wall on paper, on a family built to fit it, and on one outside model with nothing assumed. The machine built in Part IV fills in this same shape, with its recursion theorem as a premise rather than a result, and a small fragment of it gets a recursion theorem by proof, with L as one of the tools; Section 24 does both, once there's something built to point at.

Here's the picture all of that fills in, lined up against Turing's, one column per axis.

Time axis	Structural axis
counts: steps	counts: μ
asks: does it halt?	asks: does it admit the shortcut?
the wall: no Turing machine decides halting	the wall: no decider whose flip is representable, under a recursion theorem

Read the table as two uses of one diagonal, each under its own premises, and not as more. The structural wall never uses μ ; the μ entry only says what that axis counts. The nat result is a family indexed by the candidate decider, and L 's results are about one fixed model and properties that mention no shortcut. Neither hands you a second fixed undecidable problem, and neither separates Turing degrees: a property can sit behind the wall from inside and be decided cleanly from outside, as the pencil argument on N_d showed.

11.5 How to kill it

Three ways, and I mean them as instructions.

One: produce, for some substrate with a shortcut predicate, a decider whose flip the substrate can represent and that's correct on every program. That breaks the generic proof at a named step, and I've named the steps.

Two: for the nat family, produce a d that correctly decides the predicate of the substrate built from d .

Three: for L , produce a closed term that decides some extensional property holding of one closed term and failing of another. That breaks L_{rice} , which rests on the reduction rules and Coq's standard library and nothing else.

An outside decider alone doesn't count against any of the three; the pencil argument on N_d is why.

11.6 What this does to the book

It does two things. It keeps me honest: nothing in what follows claims that a system with a record can check its own shortcuts on autopilot, because that's a wall, the same

wall, and I showed you the bricks. And it tells you what a record *can* do, which is the thing Part III has been about since the counter: it can carry a shortcut *with its receipt*, so that the question “was this earned” is answered by an event you can point at rather than a decider that can’t exist. The diagonal says you can’t automate the judgement. The latch says you can record the commitment. Those are different jobs, and a record does the second.

One last distinction, because the two are easy to conflate and conflating them is the easiest way to misread this book. There’s a *second* wall, nothing to do with the diagonal. It’s Section 7’s kind of wall, a window with a colliding pair in it. Two states that differ in their certification reading can look identical through a window that doesn’t show that reading, the stamp inside the closed drawer and the drawer opened and shut, and then no function of the window recovers the reading (Theorem 7.1: a decoder has to give one answer on everything the window lumps together). That’s missing information, not an undecidable question; it holds for finite things with nothing to compute; and it sits *beside* Rice, not under it. The property the diagonal uses (any behavioural property of programs with a yes and a no) is a different kind of thing from the reading the window hides, and nothing here says the two are the same. Two walls, two axes. The diagonal is about what no inside procedure can decide. The window is about what no outside observer can see. I’ll need both, and I’ll try not to call one by the other’s name again. The machine I build later has a colliding pair of exactly that kind; Section 23 shows it.

12 Five small models

A toll, a counter, two kinds of wall and a latch, and every one of them shown on paper, on cups and cards and dice I made up. I’ve been building a toy and claiming it’s the smallest version of something people actually do. Time to go and look at what people actually do. I picked five systems that keep records other people rely on, with real money or real security behind them, and asked each the one question this book is about: *is there a price on the moment the record is written?* Not on the work around it. On the write. I’ll tell you the answer first, because it’s a single sentence and it surprised me: of the five specifications, read through the transcriptions I made of them, only one prices the write, and that one has a leak at exactly the place Section 6 said a counter leaks, where it goes down.

Three things are easy to blur in a chapter like this, so I’ll name them now and keep them apart all the way through. There’s the *real specification*: the document real money and real security run on. There’s my *transcription*: one mechanism copied out of that document into Coq, small enough to check. And for some of the five there’s a *wrapper*: a tiny abstract model that takes one premise as given and says what follows from it. A claim proved at one level isn’t a claim at the level above it. Near the end I’ll walk one system, gas, through all three, so you can see exactly where each claim stops.

What I built for each is a *fragment*: a small Coq model of the one mechanism I’m

asking about, transcribed from the system’s own specification where there was one, and nothing more. Not the system. I’ll say for each one what the fragment is and what it isn’t, because “Ethereum has been formalised” is the kind of sentence that gets a book thrown across a room, and rightly.

12.1 Proof-of-stake finality: the price is on going back

Ethereum and chains like it decide which blocks are final by having validators, people who’ve locked up money (their *stake*), vote on checkpoints: blocks picked out at regular numbered rounds called *epochs*. A vote isn’t for one checkpoint. It’s a *link* from a checkpoint the validator already trusts, the source, to a later one, the target, and every chain starts from one first block, its *genesis*, which counts as trusted from the start. In the design Vitalik Buterin and Virgil Griffith published in 2017, called Casper, a checkpoint is *justified* when validators holding two thirds of the stake vote a link to it from a justified source, and *finalized* when a justified checkpoint’s direct child is also justified. The record is “this checkpoint is final.”

What’s priced? Two rules: don’t vote for two different targets in the same epoch, and don’t cast a vote whose span, source to target, surrounds the span of another of your votes. Break either and your deposit can be destroyed, which is called *slashing*. The theorem they prove, called accountable safety, says that if two *conflicting* checkpoints are ever both finalized, at least a third of the stake broke a rule and can be slashed. So the price is on *reversal*, on making the record contradict itself. The write itself, finalizing, is free: honest validators finalize all day and nobody loses anything.

Do it with three validators. Let A and B vote one branch, B and C vote a competing branch. Each pair is two of three. Both branches finalize. Now read the rule: B cast two different votes with the same target epoch. B is the third that gets slashed; A and C are clean. That’s what a fork costs and who pays, and the checked work has exactly this instance (`casper_fork_exists`, `casper_fork_slashable`), so the theorem’s premise isn’t vacuous: forks of that kind can be built, and the theorem fires on them.

The fragment: a transcription of a published Coq proof of Casper’s accountable safety (the casper-proofs development from Runtime Verification and the University of Texas at Austin, itself based on Yoichi Hirai’s proof in Isabelle, another proof checker; proofs redone on the standard library), with validator sets as predicates and quorum intersection as a premise. A *quorum* is a set of validators big enough to act, here two thirds of the stake, and *quorum intersection* is the arithmetic of the three-validator example: two thirds plus two thirds is four thirds, so any two quorums share at least a third. The fragment doesn’t do that arithmetic; it takes the overlap as given. No deposits, no rewards, no fork choice (the rule for picking a branch to build on), no connection to anything else in this book. `accountable_safety`: a finalization fork implies a slashed quorum. And `finalization_without_slashing`: a one-validator chain finalizes and nobody’s slashed, which is the “write is free” sentence in a degenerate

setting; I'll say so rather than let it sound like a result about every setting.

Here's the fragment, small enough to read in one sitting. Every name below is from the transcription.

Definition 12.1 (Casper setting and votes). A *setting* has a type of validators, a type of block hashes with a parent relation and a genesis hash, and two classes of validator sets, the “two-thirds” sets Q_1 and the “one-third” sets Q_2 . It carries two premises: *quorum intersection*, for any two sets in Q_1 some set in Q_2 lies inside both; and *at most one parent* per hash. A *state* says, for each validator, target hash h , target epoch t and source epoch u , whether that validator voted the link (h, t, u) . A link from p at epoch u to h at epoch t is *justified by* q when $q \in Q_1$, every member of q voted (h, t, u) , h is the $(t - u)$ -th descendant of p , and $t > u$. Genesis is justified at epoch 0, and a block reached from a justified block by a justified link is justified. Block h is *finalized* at epoch v with child c when c is a child of h , h is justified at v , and the link from h at v to c at $v + 1$ is justified. A *finalization fork* is two finalized blocks $h_1 \neq h_2$, neither an ancestor of the other. A validator is *slashed* when it voted two different targets with the same target epoch (a double vote), or voted links (h_1, t_1, u_1) and (h_2, t_2, u_2) with $t_1 > t_2$ and $u_2 > u_1$ (a surround vote). The state is *quorum-slashed* when some set in Q_2 consists of slashed validators.

Theorem 12.1 (Accountable safety). *In every setting (so under quorum intersection and at most one parent), every state with a finalization fork is quorum-slashed.*

Proof. Let the fork be h_1 finalized at v_1 and h_2 at v_2 . If $v_1 = v_2$, the two links into epoch $v_1 + 1$ are each voted by a set in Q_1 ; quorum intersection gives a set in Q_2 that voted both; their targets differ, since equal children would give h_1 and h_2 the same child and so, by at most one parent, $h_1 = h_2$. That set double-voted. If $v_1 \neq v_2$, say $v_1 < v_2$, walk down the chain of justified links that justifies h_2 , by strong induction on the epoch gap. Every block on that chain is an ancestor of h_2 , so none descends from h_1 . Some link in the chain starts at or below v_1 and lands above v_1 . If it lands at $v_1 + 1$, it and the finalization link of h_1 have distinct targets at that epoch, and the intersection set double-voted. If it starts at v_1 , its source is a justified block at v_1 other than h_1 , and the links that justified those two blocks give a double vote the same way. Otherwise it starts below v_1 and lands above $v_1 + 1$, so it surrounds the finalization link of h_1 , and the intersection set made a surround vote. $\square$

Checked as `accountable_safety` (CasperFFG.v), a transcription of the casper-proofs Core/AccountableSafety file at a pinned commit with proofs redone on the standard library; validator sets are predicates, and both premises are fields of the setting record, so they are hypotheses of every theorem. The record reading is `conflicting_records_are_priced` (CasperRecordReading.v).

Theorem 12.2 (The premise can hold, and the write is free). (1) In the three-validator setting where each validator has stake 1, Q_1 is the sets of stake at least two thirds and Q_2 the sets of at least one third, the state where A and B vote one branch and B and C the other has a finalization fork; B is slashed, A and C are not, and $\{B\}$ is in Q_2 . (2) In the one-validator chain setting, one vote finalizes genesis and no validator is slashed.

Proof. Both are computations on fixed finite data. In (1) B voted two different targets at epoch 1, which is the double vote; A's and C's votes are one link at epoch 1 from source 0 and one at epoch 2 from source 1, which neither double-vote nor surround. $\square$

Checked as `casper_fork_exists` and `casper_fork_slashable` (CasperFork-Witness.v), and `finalization_without_slashing` (CasperRecordReading.v).

Wrong framing to take off the table: "a blockchain is immutable truth." Finality is revocable at a price, the price is named, and a final block can hold a false statement.

12.2 Gas: the one that prices the write

Every operation on Ethereum costs *gas*, paid in ether by whoever sent the transaction. The yellow paper, Ethereum's formal specification, gives the reason in one line: every computation is fee-bearing so the network can't be abused and halting questions are sidestepped. The record I care about is a nonzero slot in the shared storage, which every full node in the world (every computer keeping its own complete copy of the chain) has to keep forever.

And here it is: writing a fresh storage slot from zero to nonzero costs twenty thousand gas, by the table in the paper's appendix, plus twenty-one hundred for touching a *cold* slot, one the transaction hasn't touched yet (after the first touch it's *warm*). The stated reason is to discourage storage growth, since storage grows every node's database. That's a price on the write, at the moment of the write, and it's the only one in the five.

Now the leak, and it's Section 6's leak. Set a slot and then clear it *in the same transaction* and you get a refund. Count it through: first write twenty-two thousand one hundred; clearing it costs a hundred and credits a refund of nineteen thousand nine hundred; net, twenty-three hundred. That's `revoked_write_nearly_free`, and `persistent_write_priced` is the other half: leave it set and you pay at least twenty thousand. The write is priced; the *revoked* write nearly isn't. But I have to add what the fragment leaves out, because twenty-three hundred on its own misleads: the realized refund is capped at one fifth of the gas the whole transaction used (that's the 2021 change from Section 6, the one that ended the refund-hoarding token's advantage). So a real set-then-clear pays a lot more than twenty-three hundred. How much more? The cap is a fifth of everything the transaction used, and that includes the twenty-one thousand every transaction pays just for existing. Take a transaction that does nothing but set and clear the slot: it uses 43,200 gas, so the cap is 8,640, so it pays 34,560, which is 13,560 on top of the base. (Look at the slot alone and forget the base, which the rule

doesn't do, and you'd get 17,760.) Either way it's far above twenty-three hundred. The counter-level net is twenty-three hundred. The bill is not.

Here's the arithmetic the way the specification has it, so you can redo it with a pencil.

Definition 12.2 (One slot in one transaction). A slot run records the slot's value at the start of the transaction (original), its current value, whether it has been touched (warm), the gas charged so far, and the refund counter. A store of a new value charges 2100 if the slot is cold, plus 20000 if the slot still holds its original value, the new value differs, and the original is 0 (2900 if the original is nonzero), and otherwise 100. It adjusts the refund counter by the specification's rules; in particular, when the new value equals the original value 0, the refund grows by $20000 - 100 = 19900$. The run starts fresh: original and current 0, cold, gas and refund 0. The *net* charge is gas minus refund.

Theorem 12.3 (The persistent write is priced; the revoked one nearly is not). (1) For every list of stores from a fresh slot, if the slot ends nonzero then the net charge is at least 20000. (2) Storing 1 then 0 leaves the slot at 0 with net charge 2300; storing 1 alone has net charge 22100.

Proof. (1) By induction on the list, with the invariant: while the original value is 0, the net charge is at least 0 when the current value is 0 and at least 20000 when it is not. Each store keeps it, which is a finite case split on whether the slot is warm and which of the original, current and new values are equal. A fresh slot satisfies it. (2) Direct computation: $2100 + 20000 = 22100$ for the first store, 100 for the second, refund 19900, and $22200 - 19900 = 2300$. $\square$

Checked as `persistent_write_priced` and `revoked_write_nearly_free` (EVMStorageGas.v), transcribed from the execution specification's storage instruction and gas table at a pinned commit. The end-of-transaction refund cap of one fifth is not in the fragment.

The fragment: one slot, one transaction, the gas and refund arithmetic transcribed from the Ethereum execution specification at a named commit. No stack, no call context, no other slots, and *no refund cap*; the file says so.

Then the wrapper, which is a different kind of object, and the walk near the end is about exactly that difference. It's a *gas schedule* in the abstract: any states, any steps, a cost per step, a yes/no commitment flag, and a charge predicate saying which steps get billed. It formalises no EVM (the Ethereum virtual machine) at all. It is Section 7's result with the whole-run ceiling (exact about the amount, not just about which steps pay) renamed, and the file says that too. Its two halves are worth reading separately. Leave one committing step unbilled, and a one-step run commits for nothing (`unde` `rcharged_opcode_admits_free_commitment`), so the floor fails outright; one

missed flip is enough. Bill one step that doesn't commit, and the schedule charges more than the number of commits on that one-step run (`overcharge_breaks_exactness`), so the ceiling fails. And the class isn't empty: in any system, the bill that charges one exactly on a step that raises the flag and nothing on any other meets both halves (Section 7's step price).

And the wrapper itself, which is the cleanest thing in this section and has the least Ethereum in it.

Definition 12.3 (A gas schedule). A *gas schedule* has states, instructions, a step, a cost $c(s, i) \in \mathbb{N}$ that may depend on the state, a Boolean commitment flag on states, and a Boolean charge predicate $\chi(s, i)$, with two premises: a charged step costs at least 1, and an uncharged step costs 0. A step *commits* when it takes the flag from false to true. For a run t from s , let $c(t)$ be its total cost and $n(t)$ its number of committing steps. The schedule has the *floor* when $c(t) \geq n(t)$ for every run, and *no overcharge* when $c(t) \leq n(t)$ for every run. It has *unit pricing* when $c(s, i) = 1$ if $\chi(s, i)$ and 0 otherwise.

Theorem 12.4 (Gas schedule exactness). *For every gas schedule: it has the floor and no overcharge if and only if $\chi(s, i)$ holds exactly when the step from s by i commits, and it has unit pricing. Moreover, (1) if some committing step from an uncommitted state is uncharged, that one-step run costs 0 and the schedule has no certification floor; (2) if some charged step does not commit, the schedule overcharges.*

Proof. The equivalence is Section 7's characterization with the whole-run ceiling, under another name. From right to left, each step's cost is 1 on a committing step and 0 otherwise, so a run's cost counts its commitments. From left to right, apply the floor and no overcharge to one-step runs: a committing step must cost at least 1, so it is charged; a charged step costs at least 1 and may not exceed its commitment count, so it commits and costs exactly 1. Parts (1) and (2) are those one-step runs read the other way. $\square$

Checked as `gas_schedule_exactness`, which proves both directions, `undercharged_opcode_admits_free_commitment`, `undercharged_opcode_breaks_certification_floor` and `overcharge_breaks_exactness` (`GasMetering.v`); `gas_schedule_exactness` is `exact_commitment_pricing_characterization` (`CommitmentPredicateAdequacy.v`) carried over unchanged.

Wrong framing: "gas is a fee for truth." Gas prices storage growth and computation. It says nothing about whether what's stored is true.

12.3 Attestation: hash chains, and a quote about the past

A trusted platform module, the small chip on most laptop motherboards, keeps a handful of registers that can only be *extended*: the new value is a hash of the old value

together with a digest (a short hash) of whatever software just loaded. You can't set them; you can only fold more in. A *quote* is the chip signing those register values together with a number the asker chose, a *nonce*, so the asker knows the quote is fresh. Intel's SGX does the equivalent for an *enclave*, a walled-off region a program runs in. The record is the extended register.

What's priced? Nothing in the specification. Reversal is *infeasible*, not expensive: you'd have to invert a hash or steal a key. That's the difference between a price and a wall, and it's easy to blur. A price can sit at three levels. It can be economic, a fee or a deposit the protocol takes (Casper and gas are here). It can be cryptographic, a reversal made infeasible by a hardness assumption, which is a wall and not an exact price (attestation and transparency are here). Or it can be physical, energy spent, which none of these five specifications touches. And the record's permanence is only as long as the register lives: a platform's registers are cleared when it resets, so the permanence is per boot.

There's a fourth level under those three, and it's the one that always pays. At the logical level, Section 4's room of chairs applies to any of these records: on a finite state space, a step that switches a record on somewhere, and never switches it off anywhere, forgets something. That needs no premise about money, keys or heat. The economic, cryptographic and physical payments each need a premise of their own about their level (a fee rule, a hardness assumption, Landauer's principle), and they hold exactly where that premise holds.

Theorem 12.5 (The logical level always pays). *For any states S , instructions I , step and Boolean reading: if a duplicate-free list names every state, the instruction i never turns the reading from on to off, and i turns it on at some state s , then the step by i is not injective.*

Checked as `permanent_write_has_logical_payment` (PricingPhysicsAudit.v), from `permanent_at_flip_is_not_injective` (PermanentRecordPricing.v); the proof is Theorem 4.1 with permanence required only of the one instruction i .

What the quote can't do, which is the thing people get wrong: it says what was *measured*. It can't say the measured code is correct, and it can't say anything about runtime state that was never folded in. A correctly measured buggy program attests beautifully. The fragment proves that exact limit: one register, a symbolic hash, a quote as a pair, and no function of the quote decides a runtime flag the quote ignores (`quote_cannot_attest_unmeasured_state`), which is Section 7's cube again. And a companion shows that a sign-and-verify interface *alone* gives no authenticity (a scheme where verify always says yes passes the interface), so a cryptographic premise has to be added; the fragment doesn't model signatures, formats, or a freshness check (the nonce is just a field the verifier fixes).

Definition 12.4 (Platform and quote). Fix any functions $H : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ (the extend

hash) and $\delta : \mathbb{N} \rightarrow \mathbb{N}$ (the digest); nothing is assumed about either. A *platform* is a measurement log (a list of numbers) and a Boolean runtime flag. Its register is the log folded from 0 by H . The *quote* with nonce n is the pair $(\delta(\text{register}), n)$.

Theorem 12.6 (The quote cannot attest unmeasured state). *For every nonce n there is no function D from quotes to Booleans with $D(\text{quote}_n(P)) = \text{flag}(P)$ for every platform P .*

Proof. Take two platforms with the empty log, one flagged true and one false. Their quotes are equal, since the quote never reads the flag, so D would have to return both values on one input. $\square$

Checked as `quote_cannot_attest_unmeasured_state` (TPMQuoteGap.v). In the same file, `quote_decides_measured_claims` says any Boolean claim about the retained digest can be read off the quote. The companion is `tpm_interface_authenticity_refuted` (TPMQuoteAuthenticity.v): a scheme whose verify always says yes satisfies the sign-and-verify interface while accepting a signature no key produced.

12.4 Certificate transparency: visible, not priced

The certificates that make your browser show a padlock are issued by hundreds of authorities, and a bad or compromised one can issue a certificate for a domain it shouldn't. Certificate Transparency, now RFC 9162, makes authorities submit every certificate to public append-only logs built as Merkle trees, so that anyone can check that a certificate was logged and that the log never rewrote its past. A Merkle tree is Section 3's hash chain folded into a tree: hash each certificate (those are the *leaves*), hash the leaf hashes in pairs, hash those in pairs, and so on up to one *root* hash. To prove one leaf is in, the log hands you only the few hashes sitting beside the climb from that leaf to the root, the *path*, and you redo the climb yourself. A second check, *consistency*, shows that a bigger tree is a smaller one with leaves added on the end and nothing rewritten. The record is a leaf in the tree.

Priced? Nothing. The RFC says it in its first section: logs don't *prevent* misissuance; they let interested parties *detect* it afterward. And a log that shows different views to different people (a *split view*) can defeat the auditing, so logs are still trusted parties. Visibility, not price.

Do the inclusion proof by hand, because it's the most satisfying ten minutes in this section. Take the RFC's own seven-leaf example tree and verify that leaf d3 is in it from the path the log hands you: three hashes and a rule about which side to hash on, depending on whether your position is odd. Three hashing steps and you land on the root. The checked work runs the RFC's own examples (`rfc9162_example_inclusion_d3` and its siblings, including the awkward lone-leaf case) through a transcription of the RFC's verifier control flow with a symbolic hash. Verifier logic

only: no signatures, no gossip (auditors comparing the views they were shown), no misbehaving log.

Definition 12.5 (Inclusion check, RFC 9162 section 2.1.3.2). Given a leaf index m , a tree size N , a leaf hash, a root hash and a path, the check rejects unless $m < N$. Otherwise it walks the path from the leaf with two counters starting at m and $N - 1$, hashing each path element on the left or the right according to the counters as the RFC specifies and halving the counters, and accepts when the second counter ends at 0 and the result equals the root. A *symbolic hash* makes a leaf hash $\ell(x)$ and a node hash $v(a, b)$ free constructors, so two hashes are equal only when they are built the same way.

Theorem 12.7 (The RFC’s own examples pass). *With the symbolic hash, in the RFC’s seven-leaf tree with leaves $d_0, \dots, d_6$ (writing d_k also for its leaf hash $\ell(d_k)$), the inclusion check accepts leaf 3 with path $[d_2, v(d_0, d_1), v(v(d_4, d_5), d_6)]$, and likewise leaves 0, 4 and 6 with their paths; the consistency check from size 4 to size 7 accepts. For every input with $m = N$, the inclusion check rejects.*

Proof. The examples are evaluation of the transcribed functions on constructor terms. The boundary case is the first test, $m < N$, failing. $\square$

Checked as `rfc9162_example_inclusion_d3`, its siblings `rfc9162_example_inclusion_d0`, `rfc9162_example_inclusion_d4` and `rfc9162_example_inclusion_d6`, `rfc9162_example_consistency_4_7`, and `rfc9162_inclusion_boundary_safe` (RFC9162Merkle.v). The append-only history is `list append`, and `ct_extension_preserves_entries` says appending keeps every old entry.

12.5 Proof-carrying code: no price, no permanence

Necula’s idea from Section 8: ship untrusted code with a proof that it obeys the host’s policy; the host computes what would need to be true and checks the proof with a small checker. The record is the checked proof, and it’s held only for the moment of loading. Priced? No; checking is cheap by design, that’s the whole point, and the producer paid to build the proof off the host. Permanent? No; nothing’s written. The fragment is a three-instruction memory-bounds language with the policy “every address is below the limit”: the checker accepts exactly when the condition holds, and a certificate implies the condition. One example is checked on top of that: a program that reads address two under a limit of two is rejected. Not the paper’s real case study, and no bridge to anything else in this book. And the wrong framing: a program that writes garbage to address zero passes if zero is below the limit. The proof is of the stated policy and nothing else.

Definition 12.6 (The bounds fragment). Instructions are `READ  $a$` , `WRITE  $a$   $v$` , and `HALT`. For a memory limit L , the policy allows `READ  $a$` and `WRITE  $a$   $v$` when $a < L$,

and always allows HALT. The *verification condition* of a program is that the policy allows every instruction in it. A *certificate* is a list of per-instruction policy facts, one for each instruction in order. The *checker* tests the policy on each instruction and returns the conjunction.

Theorem 12.8 (The checker is the verification condition). *For every limit L and program P : the checker accepts P exactly when the verification condition holds; any certificate for P gives the verification condition; and with $L = 2$ the checker rejects READ 2 followed by HALT.*

Proof. The first two are induction on the program and on the certificate, one instruction at a time. The third is a computation: $2 < 2$ fails. $\square$

Checked as `pcc_checker_accepts_iff_vc`, `pcc_certificate_implies_vc`, and `pcc_unsafe_program_rejected` (NeculaPCC.v, definitions in NeculaPCC-Target.v).

12.6 One example, three levels

Before I lay all five side by side, let me walk one of them up and down the three levels from the start of the chapter, because every claim in here lives at exactly one level and it's easy to let a claim slide up a floor without noticing. Gas, since it's the one place all three line up. Think of a restaurant's real recipe, my photocopy of one page of it, and a cookbook chapter on what any recipe has to contain. I can check my photocopy line by line. That tells you nothing about what the kitchen cooks tonight.

The real specification. Ethereum's execution specification says, in a document that runs real money, what a storage write costs, what clearing it refunds, and how the refund is capped at the end of the transaction. None of what I checked is at this level. What the network actually charges is whatever the clients people run actually compute, and the evidence for that is the clients and the chain, not this book.

The transcription. I copied one mechanism out of that document, one slot in one transaction, gas and refund arithmetic only, at a named commit (Definition 12.2, Theorem 12.3). What the result tells you: *if* the storage rule is what I copied, then leaving a fresh slot set costs at least twenty thousand, and set-then-clear nets twenty-three hundred at the counter. What it can't tell you: anything about the refund cap (not copied, which is why the bill came out so different from the counter earlier), the stack, calls, other slots, or whether any client implements the rule as written. The boundary of the claim is the edge of the copy.

The wrapper. The gas schedule of Definition 12.3 isn't Ethereum. It's any system with steps, a cost per step and a yes/no commitment flag, plus one thing *stipulated*: a charge predicate, with the premise that a charged step costs at least one and an uncharged one costs nothing. That premise is the whole of what the wrapper adds, and it's added by hand, not found in any specification. Given it, the theorem says the bill is

exact, never letting a commit go free and never billing a non-commit, exactly when the charge predicate is the commit predicate at unit price. What that tells you is what exact pricing would have to look like, in any system. It doesn't say Ethereum meets it, and Ethereum doesn't: real gas bills computation that commits nothing, which the transcription shows in numbers. So the wrapper prices a definition, the transcription prices a copied rule, and neither prices the chain.

The other four sit on the same three levels, and only one more wrapper lives in this chapter. Casper's transcription is the accountable-safety proof above; its wrapper is a small gadget with the stake at risk handed in. A finalize step that risks nothing turns the record on for free, which breaks Section 3's toll (the file calls that free forgery), and one that risks at least one unit has a floor (`nothing_at_stake_is_free_for_gery, slashing_finality_floor, PoSFinality.v`). It records exposure to loss, not destroyed collateral, has no Casper checkpoints in it at all, and is priced by stipulation, the same way the gas wrapper is. Attestation, transparency and proof-carrying code have transcriptions that prove what their verifiers do and no price theorem, because their specifications carry no price. Their wrappers are Section 7's verifier argument told in each system's vocabulary, and they're written for the machine I build later, with its own pair of colliding states, so they're Section 23's.

How to knock each one down, since the book's rule is that every claim gets a way to break it. For the gas transcription: a list of stores from a fresh slot that ends nonzero with net charge under twenty thousand. For the gas wrapper: a schedule that never lets a commit go free and never bills a non-commit, whose charge predicate still differs from the commit predicate at some step. For the finality gadget: a finalizing trace from unfinalized to finalized whose charged stake is zero while the gadget claims a floor. For the attestation fragment: a verifier reading only the quote's fields that decides a runtime flag the quote ignores.

12.7 The same table, five ways

Lay them out. Casper prices *reversal* (slashing). Gas prices *the write* (and refunds the revocation, capped). Attestation and transparency price *nothing*, and make reversal infeasible or visible instead. Proof-carrying code prices nothing and keeps nothing.

System	What its specification prices	What the transcription checks
Casper finality	reversal, by slashing	a finalization fork means a slashable third
Gas	the write, refunded on revocation	a persistent write nets at least 20000
Attestation	nothing; reversal is infeasible	the quote can't decide what it never measured
Transparency	nothing; misissuance is visible	the RFC's own inclusion examples pass
Proof-carrying code	nothing, and nothing is kept	the checker is the verification condition

Each row proves what its own transcription says. Reading a row as a claim about a deployed system takes evidence from that system, and no row shows that certification is the one event every model has to price.

So the thing my toy charges for, the moment the record is written, is charged for in one of five real systems, and in that one the designers had to patch the place where the charge could be clawed back. I find that neither a vindication nor a refutation. It's a fact about what people built, and what they built was mostly walls (hash chains, signatures) and penalties for contradiction, not tolls on commitment. Whether a toll on commitment is a *better* design than a wall is not something I can prove, and I won't pretend the five say so. What I read in them, and it's a reading of five fragments I chose, is that the record axis is real in each of them (each has a bit that goes one way) and that the price axis is mostly empty.

That leaves the question Section 10 raised and Section 13 has to face. Five systems, five different events chosen as "the record": finalized, committed, attested, logged, checked. Is there anything that picks out one event as *the* one worth recording, or is it a free choice every time? I have a guess. It's a conjecture, and I'll show you how to kill it.

13 The pointer conjecture

Here's where I am. A record over any base is a latch on an event (Section 10), and the event is a free choice (Section 10 again, the two cups on one die). Five real systems each picked a different event (Section 12). The toll in this book sits on certification. Why that one? I don't have a proof. I have a conjecture, and the honest shape of a conjecture is: here's the claim, here's where I got the idea, here's what's checked, here's what isn't, and here's how to kill it. In that order.

13.1 The claim

Of all the events a machine could put on its meter, the ones its price list charges for and its counter adds up, certification is the one whose records get *copied*: checked, stored, and relied on across independent places that share no other state. Rival events leave no such trail. If that's true, certification isn't a free choice; it's the one event the world around the machine keeps redundant copies of, and that's a reason to price it and not the others.

There's a second handle on the same question, and it comes from Section 4. The permanent-certificate theorem never uses the word certification for anything special: given Section 4's rule that forgetting is charged, it prices any yes-or-no reading that, once on, stays on, on a machine with finite memory. So it points at a narrower candidate: the events to price are the ones that write a record no later step can take back. Certification is one of them. As an exact answer that candidate is false, because a merging step can also revoke (Section 4's three cards). Whether the records that proliferate are the permanent ones, and whether proliferation picks out anything the merge

criterion doesn't, is what the two handles have to settle between them.

That's the whole conjecture. Now the rest of what a conjecture owes you.

13.2 Where the shape comes from, and how much of it survived

The shape is from physics, and I want to tell you where exactly, because I borrowed a word and I didn't get to keep the whole thing.

Quantum mechanics has a famous problem: a system can be in a blend of states, and when you measure it you find one definite state, and nobody agrees on what "measuring" did. In 1981 Wojciech Zurek asked a sharper question: of all the ways a measuring device *could* end up, why does it end up recording one particular kind of quantity (the position of a pointer on a dial, say) and not some blend? His answer, developed over twenty years and summarised in a 2009 paper called "Quantum Darwinism," is that the *environment* decides. The air, the light, the table, everything the device touches, is constantly interacting with it, and only certain states of the device survive that interaction; those are the *pointer states*. And more: the environment doesn't just select them, it *copies* them. The light bouncing off a dial carries the dial's reading to every corner of the room, redundantly, so that a thousand observers can each read a different scrap of the environment and all agree on where the pointer is, without touching the dial. Objectivity, on this view, *is* redundancy: a property is objective when many independent fragments of the world each carry it.

That's the shape I borrowed. Certification is to a machine's ecosystem (the machine plus everything around it that reads, stores and relies on its records) what the pointer reading is to the room: the thing that gets copied everywhere. It's a good shape and I think it's the right intuition. Here's what survived into the checked work, and I'm going to be exact because the word "redundant" is doing less than it does for Zurek.

Zurek's redundancy is *graded* (each fragment carries almost all the information, with some small loss) and it requires the fragments to be *disjoint*: different observers reading different parts of the environment. The checked definition is the zero-loss case with *no disjointness at all*. An ecosystem is a state type, some number of observers, and one bit per observer; an observer *records* an event when its bit is true exactly when the event holds; an event *proliferates* when every observer records it. Nothing in that says two observers draw on separate parts of the state. They could all be reading the same bit. The file says so itself. So I kept the word and not the independence that makes Zurek's version mean "objective." That's a gap between the conjecture's prose and its formal object, and the prose is where the conjecture lives.

13.3 What's checked

Definitions, a toy, a survey, and two refutations of things that would have been stronger.

The definitions above, plus "unique pointer among rivals": the event proliferates and

none of a listed set of rival events does. Uniqueness is *relative to the list*, necessarily, since any event that holds on exactly the same states as a proliferating event proliferates whenever it does: the same fact under another name, so someone has to say which names count as rivals. And the conjecture itself, as a schema: for every ecosystem in some class, its metered event is the unique pointer among its rivals. The class and the rivals are inputs. Nothing about them is proved.

Here's the formal object, small enough to hold in one hand.

Definition 13.1 (Ecosystem, records, proliferates, pointer). An *ecosystem* is a type of states, a number N of observers, and for each observer i a Boolean fragment F_i on states. Observer i *records* an event E (a property of states) when, for every state s , $E(s)$ holds exactly when $F_i(s)$ is true. E *proliferates* when every observer $i < N$ records it. E is the *unique pointer among* a list of rival events when E proliferates and no rival in the list does. Given a class C of ecosystems, and for each member a designated metered event and a list of rivals, the *pointer criterion holds* for C when every member's metered event is the unique pointer among its rivals.

Nothing in the definition makes two fragments read different parts of the state, and "metered" is only a label: the definition has no metering in it.

Conjecture 13.1 (The pointer conjecture). *Let C be the class of ecosystems that faithfully model deployed record-keeping disciplines, observed through independent carriers (separate parts of the world, each holding its own copy), with each discipline's certification event as its metered event and the other events a step rule could meter as rivals. Then the pointer criterion holds for C .*

The conjecture leans on four notions left without precise mathematical definitions: what it is to "faithfully model" a system, which systems count as "deployed," what makes carriers "independent," and which events "a step rule could meter." That's why it's a conjecture and not a theorem with a missing proof.

A toy: three observers on a replicated ledger, where certification proliferates and "work done" doesn't. The file labels it a sanity check, and it is one, not evidence.

Theorem 13.2 (The toy). *Let the toy have states (a certified bit, a work count), three observers, and every fragment equal to the certified bit. Then "certified" is the unique pointer among the one rival "work at least 1".*

Proof. Every fragment is the certified bit, so every observer records "certified" by definition. For the rival, take the state with the bit false and work 1: the event holds and observer 0's fragment is false, so observer 0 does not record it. $\square$

Checked as `toy_cert_unique_pointer` and `toy_work_not_proliferating` (`PointerObservable.v`).

A survey of twelve events, one checked theorem: for each of the five systems in Section 12, the designated event (finalized, committed, attested, logged, checked) proliferates in the observer map I wrote for that system, and a rival (proposer work, a scratch counter, measurement noise, submission latency, prover steps) doesn't. The five together are one statement (`five_labeled_models_have_selected_pointer`), an instance of the schema above. Plus a symmetric message-authentication code, which does *not* proliferate, and a digital signature, which does. Twelve rows. And a thirteenth that I think is the most important one: swap which bit every observer copies, and the criterion picks the *other* event. The criterion selects whatever the observer map copies. The map is where the argument lives, and I wrote the maps. So the five "confirmations" are five instances of one construction with the fields renamed, and I'd rather say that twice than let it pass once.

Theorem 13.3 (The survey, and the swap). *(1) In each of the five observer models (proof-of-stake, gas, attestation, transparency, proof-carrying), the designated event proliferates and the designated rival does not; in the symmetric-MAC model the authentic-tag event does not proliferate; in the signature model the authentic-signature event does. (2) Let the states be pairs of bits (first, second), with three observers whose fragments all equal the second bit. Then "second is true" is the unique pointer among the one rival "first is true".*

Proof. Each confirmation holds by definition: each observer's fragment is true exactly when the event holds. Each refutation exhibits one state where the event holds and some observer's bit is down. In (2) that state is (true, false). $\square$

Checked as `twelve_candidate_measurements_checked` and `swapped_event_is_pointer_checked` (`RecordProliferationSurvey.v`, statements in `RecordProliferationSurveyTarget.v`); the five designated models together are `five_labeled_models_have_selected_pointer` (`PointerObservableReductions.v`), an instance of the indexed form of the criterion, `pointer_criterion_holds_indexed`.

Two stronger claims, both refuted, and I'd ask you not to merge them because they fail for different reasons.

The first stronger claim: *every forgery-resistant commitment proliferates*. If that were true, certification would be the pointer because it's the commitment. It's refuted in three observer models (deniable authentication, where only the designated verifier's bit tracks the event; a symmetric MAC, where the key holders share a secret and nobody else can check; an object capability, an unforgeable token that grants access to whoever holds it) and confirmed in two (a public log; a digital signature). Build the first with three observers: give one a copy of the event's bit and the other two a constant false. The one records it; it doesn't proliferate. The checked half is the map computation.

Whether my maps are *fair* to the real designs is argued in prose, and you should argue back.

Theorem 13.4 (One blind observer is enough). (1) *In any ecosystem, if some observer $i < N$ has $F_i(s_0)$ false at a state s_0 where E holds, then E does not proliferate.* (2) *Let the deniable model have states (authentic, verifier-accepts), three observers, observer 0's fragment the authentic bit and observers 1 and 2 constantly false. Then observer 0 records "authentic", and "authentic" does not proliferate. The symmetric-MAC and object-capability models also fail to proliferate their events; the public-log and signature models proliferate theirs.*

Proof. (1) Recording would require $E(s_0) \iff F_i(s_0)$, and the left side is true while the right is false. (2) Observer 0's fragment is true exactly when the event holds. Apply (1) with observer 1 and the state (true, true). The other three refutations apply (1) to a blind observer of their own model, and the two confirmations hold by definition. $\square$

Checked as `blind_observer_blocks_proliferation` and `labeled_model_verdicts` (PointerObservableCounterexamples.v). The theorem is about the observer maps. That a third party in deniable authentication really cannot be convinced is the modelling claim, and the model has no transcript, key or verifier in it to check that against.

The second stronger claim: *agreement plus accuracy plus locality imply permanence*. That is, if all observers agree, each sees the event exactly, and each updates by the same local rule with no coordinator, then the recorded event can't be undone. Refuted by the smallest thing imaginable: one bit, two observers who both see it, and the rule "flip." All three conditions hold and true becomes false in one step (`toggle_game_refutes_strong_pointer_necessity`). Do it on paper; it takes a minute and it's humbling. What *does* give permanence is durability, "an observer's true view stays true," added as a premise, and then it follows in a few lines (`durable_consensus_implies_permanence`). So durability is the premise doing the work, and durability is Section 4's permanence wearing a different coat. Agreement doesn't buy it. Do the paper version again with one change: the rule is "stay true once true" instead of "flip," and the bit can't fall, because the rule won't let it, not because the observers agree. (The machine I build later has a reading that's durable in exactly this sense; that instance is Section 23's.) None of this rules out a narrower theorem with real assumptions about the network, the adversary or durability; it rules out these two strong forms.

Definition 13.2 (Observation game). A *game* has states, a number N of observers, a Boolean event e , a Boolean view V_i for each observer, and a step map. For $i, j < N$ it has *consensus* when $V_i = V_j$ on every state; *authenticity* when $V_i(s) = e(s)$ for every state and every $i < N$; *coordinator-free update* when one function ℓ on Booleans has $V_i(\text{step}(s)) = \ell(V_i(s))$ for every state and every $i < N$; *durable views* when $V_i(s)$ true

gives $V_i(\text{step}(s))$ true for every $i < N$; and a *permanent event* when $e(s)$ true gives $e(\text{step}(s))$ true.

Theorem 13.5 (Agreement does not buy permanence; durability does). (1) *It is false that every game with $N > 0$, consensus, authenticity and coordinator-free update has a permanent event.* (2) *Every game with $N > 0$, authenticity and durable views has a permanent event.*

Proof. (1) The toggle game: states are Booleans, two observers, event and both views the identity, step “flip”. Consensus and authenticity hold trivially, the local rule is “flip”, and the state true steps to false. (2) If $e(s)$ is true, observer 0’s view is true by authenticity, stays true by durability, and authenticity turns it back into $e(\text{step}(s))$. $\square$

Checked as `toggle_game_refutes_strong_pointer_necessity` and `durable_consensus_implies_permanence` (EcosystemGame.v, definitions in EcosystemGameTarget.v).

13.4 What isn’t checked, and must stay that way on the page

That certification *in deployed systems* is singled out by redundant proliferation across *independent* carriers. The proposed criterion has no precise definition of “independent” or “deployed,” and no cost law connecting proliferation to price. Every correspondence between a Coq observer map and a real system is a sentence I wrote. The conjecture is a conjecture, and anyone who says the formal object proves it is reading more than is there.

13.5 How to kill it

Three ways, in the order I’d try them.

One: find a deployed discipline that prices a *non*-proliferating event. A real system with a toll on something nobody copies. That breaks “the priced event is the copied one.”

Two: find a rival event that proliferates *as well as* certification in a real ecosystem. Then certification isn’t unique and the conjecture’s “the one” is wrong. Or do it in a model: show that something like a multiplication count or a head reversal proliferates by the same standard I used, so the criterion picks out more than one event and decides nothing.

Three: formalise an ecosystem honestly, with independence defined, where certification *fails* the test. The symmetric-MAC row is already most of the way there for a different claim; push it.

The checked work has already refuted two stronger forms of its own claim, which I take as a sign the method works and as a warning about the main claim.

13.6 The end of the logic

That's Part III, and the end of the logic that started in Part II. Here's what it earned. A toll on raising a flag is forced by three premises (Section 4). Learning is free, committing isn't (Section 5). The counter is arithmetic once the schedule is fixed (Section 6). A window with a colliding pair in it can't price the flag exactly (Section 7). Entitlement is an object, a check, and a paid event, and, given lists, a tree and a covering that someone supplies, it costs the index bits it commits to (Section 8). Insight is a committed relation, not a feeling (Section 9). Over any base a record is a latch, with two free choices, the price list and the event (Section 10). No inside procedure whose flip a substrate can represent decides a shortcut property, given a recursion theorem, which is proved for L and assumed elsewhere; a record carries the commitment instead (Section 11). Of five specifications, read through my transcriptions, one prices the write (Section 12). And which event to record is a conjecture with a borrowed shape and a listed set of refutations (Section 13).

What none of that is: a machine. Every theorem so far quantifies over "any states, any steps," or is about a small copy of somebody else's system, or points forward at a thing I keep promising. That's the abstract part of the book, and it's done. From the next page on I stop saying "any machine" and build one, mine, with every part named and every price written down, so that everything above finally has something to bump into. If a statement up there doesn't mean what I think it means, that's where it'll show. Part IV is the thing. Let's see if it holds weight.

Part IV

The machine

14 The smallest honest machine

Parts I to III never built anything. They said "any states, any steps," and whenever a picture was needed, a board or some chairs or a cup stood in. Now I have to build a machine, and the first worry I have about that is a slightly embarrassing one. Say I build a big machine, with dozens of instructions, some picked for reasons and some picked because I had to pick something, and every theorem holds of it. What did that show? That the model works, or that I'm good at stacking instructions until the proofs go through? A big machine is a wonderful place to hide a mistake. It's a garage so full of stuff that you could lose a car in it.

So before the big one, the small one. The least I can build that still shows the whole model, every piece of it, with nothing waved at. If the idea only stands up with dozens of instructions holding it, it's their idea and not the model's. If it stands up on six, the six are the idea, and you can hold all of them in your head at once, which is more than

I can say for most things I've built.

14.1 What the model actually asks for

Here's what Parts II and III say the model adds to a machine. A *record*: a flag that reads no or yes, "certified" or not. A *toll*: the step that raises the flag costs something (Section 3). A *ledger*: the running total of those costs, a whole number that only climbs (Section 6). And Section 10 proved a thing that turns out to make this whole chapter possible: over any base at all, an honest record is a *latch*, a bit that switches on at an event the base decides and never switches off (Theorem 10.1). The record watches the base. It doesn't compute anything the base couldn't compute without it.

Push on that for a second, because it cuts the job in half. If the record adds no computing power, then all the computing power comes from the base, and the question "how small can the machine be" splits into two questions that don't touch each other. For the base: what's the smallest thing that can compute everything there is to compute? For the record: what's the smallest set of pieces that makes the flag *mean* something?

That second question is the one Part III left open, and it's open on purpose. Section 9 said that testing a claim and committing to it are two different acts, and that nothing in the five parts of the model makes the second wait for the first. Read Section 3's definition again with that in mind: a machine where you pay one unit and raise the flag over nothing in particular obeys every rule in it. That machine is honest about the price and silent about what was bought. A receipt for nothing. The toll alone can't fix that, because the toll is a rule about *cost*, and "was this earned" is a question about *order*: did a check happen, of this claim, before the commitment, with nothing changing underneath in between?

This machine is my attempt to make the order part of the machine, in the smallest place I could find to do it. I'll show you the pieces, then the proofs, then the places it falls short, which there are, and which I'd rather you hear from me.

14.2 Where the counters come from

The base first, and the base isn't mine. It's Marvin Minsky's. In a 1961 paper in the *Annals of Mathematics*, and then in his 1967 book *Computation: Finite and Infinite Machines*, Minsky studied a machine so bare it's almost a joke. Picture a few bins of pebbles and a numbered list of instructions. There are only two kinds of instruction: put a pebble in bin c and go to the next line; or, if bin c has a pebble, take one out and jump to line j , and if it's empty, just go to the next line. That's it. No memory, no tape, no arithmetic beyond plus one and minus one. When the line you're on has no instruction, the machine stops.

What Minsky showed is that two bins are enough to compute anything a Turing machine can compute, as long as you're willing to pack the input in a strange way. The trick is to keep many numbers inside one number, as the powers of different primes ($2^x 3^y 5^z$ holds three numbers in one), and to do arithmetic on the exponents

by multiplying and dividing with the second bin as scratch space. It is as slow as it sounds. Exponentially slow. A two-counter machine doing your taxes would finish sometime after the sun doesn't. But "universal" is a claim about what's *possible*, not about what's pleasant, and on that claim two bins of pebbles are as strong as anything.

Who cares? Minsky did, because the question then was where the edge of computation sits and how little machinery reaches it. You should, because it means the base of this chapter's machine hides nothing: if it can compute everything with plus one, minus one and a jump, then nothing about the record can be smuggling computing power in through some instruction I made fancy. And I care because it's the smallest universal base I know of that is already in a Coq library, so the hard part (proving that its halting question is undecidable) is someone else's checked work and not my say-so.

One word clash, before it bites. Minsky's machine is usually called a *counter machine*, and its bins are *counters*. Section 6 also had a counter: the ledger μ , the number that only climbs. Same word, two completely different things, and I'd love to say I set that up on purpose. In this chapter, *counter* means one of Minsky's two bins, called *A* and *B*, and the climbing total is always *the ledger μ* .

14.3 The machine

Here's the whole state. I'll say each piece in words first.

- Two counters, *A* and *B*. Each holds a whole number, with no upper limit.
- A *version* for each counter, also a whole number with no limit. It starts at 0 and goes up by one every time its counter is written. Think of it as the counter's edition number: a new value is a new edition.
- A *line number*, which instruction comes next, counting from 1 the way Minsky does.
- A *fact table*: a list of facts. A *fact* is three things stuck together: a property, a counter, and a version. "*A* is even, as of edition 3" is a fact. The table has room for sixteen.
- A *channel*: either empty, or holding one fact. This is where a commitment sits.
- A *trap*: one bit, down at the start. When it goes up, the machine has refused something, and it stays up.

Everything above is the *core*, the part the base moves. Outside the core sit the two pieces from the model: the ledger μ and the flag. The machine never reads either of them to decide what to do next. That's not a promise, it's how the step is written, and it shows up as a theorem below.

The properties a fact can name come from a tiny fixed language with three words: *is zero*; *is even*; and **is at least n^** , for any whole number n you like. Each one comes in two forms. There's the *checker*, a little program that computes yes or no: compare with

0, look at the last bit, compare with n . And there's the *meaning*, the plain arithmetic statement the checker is supposed to be checking: the number is 0; the number is two times something; n is less than or equal to the number. Why both? Because a checker is code, and code can be wrong in ways that look right, and "the checker says yes" is only worth anything once somebody has proved it says yes exactly when the meaning is true. That proof is one line per property, and it's the first block below.

Now the six instructions.

Instruction	What it does, when the trap is down	Cost
INC c	Adds one to counter c , adds one to its version, goes to the next line.	0
DEC $c\ j$	If c is above 0: subtracts one, adds one to its version, jumps to line j . If c is 0: just goes to the next line.	0
HALT	Nothing. A program that reaches it stops.	0
CHECK $p\ c$	Runs the checker for p on c 's value. If it says yes and the table has fewer than sixteen facts, writes the fact $(p, c, c$'s current version) into the table and goes to the next line. Otherwise, trap.	1
COMMIT $p\ c$	Looks in the table for exactly the fact $(p, c, c$'s current version), field by field. If it's there, puts it in the channel and goes to the next line. Otherwise, trap.	1
CERTIFY	If the channel holds a fact, goes to the next line, and the flag goes up. Otherwise, trap.	1

The first three are Minsky's machine, plus a HALT so a program can say it's done. The last three are the record's. Read them as a chain: you can only certify with a commitment in the channel, you can only commit to a fact in the table, and the only way into the table is a check that passed. Three links, and each one costs.

One of what? One unit of μ , the same unit as Section 3's toll. Why one and not ten? Because one is the smallest positive whole number, which is honestly the whole defence, and Section 4 is the section where I tried to do better than that for the flag. The zeros matter just as much: computing is free on this ledger. Plus one, minus one and jumping around cost nothing at all. Only the three acts that make a claim cost, and each of them costs whether it passes or traps. Failing is not a discount.

The version rule. Here's why versions exist, with numbers. Start with $A = 4$, version 0. CHECK *is even* A passes, and the table gets the fact (*even*, A , 0). Now INC A : A becomes 5, and its version becomes 1. Now COMMIT *is even* A . The machine looks for (*even*, A , 1), the claim about the current edition. The table only has (*even*, A , 0). No match. Trap. The old fact is still sitting in the table, still true about edition 0, and completely useless for committing to anything about edition 1. A fact is about one edition of one counter, and an edition never comes back, because versions only go up and never wrap. That's the whole trick against stale evidence: you can't check a number, change it, and then commit to the check.

Notice what *isn't* a write. DEC on a counter that's already 0 doesn't change the counter, so it doesn't change the version. It's a test that happens to move the line number.

The fact table. Facts go in and never come out. Nothing erases them and nothing overwrites them. When the table is full, a seventeenth CHECK doesn't push an old fact out to make room; it traps. That's a choice with a bill attached, and I'll put the bill in the section on what this machine doesn't do.

Traps. Any failure puts the trap up and changes nothing else. Once it's up, every instruction leaves the core alone. A stored program stops at a trap. But I also prove things about *any* list of instructions, not just programs, because an adversary doesn't have to run my programs; they can feed the machine whatever they want. For those lists, an instruction after a trap is still charged its cost, and still changes nothing.

Now the exact version.

Definition 14.1 (Properties, checker, meaning). A *property* is one of: Zero, Even, or $\text{Ge}(n)$ for a whole number n . The *checker* $\text{eval}(p, v)$ computes yes or no: $v = 0$ by equality, evenness by the parity function, $n \leq v$ by comparison. The *meaning* $\text{holds}(p, v)$ is the arithmetic statement: $v = 0$; there is an m with $v = 2m$; $n \leq v$.

Lemma 14.1 (The checker says what it means). *For every property p and every whole number v , $\text{eval}(p, v) = \text{yes}$ if and only if $\text{holds}(p, v)$.*

Proof. Split on the three properties. Equality with zero returns yes exactly at zero; comparison with n returns yes exactly when $n \leq v$. For evenness, the parity checker returns yes at 0, no at 1, and the same answer at $v + 2$ as at v . Induction in steps of two gives yes exactly at $0, 2, 4, \dots$, the numbers $2m$. Those are precisely the three meanings. $\square$

Checked as `eval_iff` (EarnedCore.v).

Definition 14.2 (Core, state, claim, start). A *core* k holds two counters A, B and their versions (whole numbers), a line number, a fact table (a list of facts, a fact being a

triple of a property, a counter and a version), a channel (empty or one fact), and a trap bit. Write $\text{val}(k, c)$ and $\text{ver}(k, c)$ for counter c 's value and version. A *state* s is a core, written $\text{core}(s)$, with a ledger $\mu(s)$ and a flag $\text{cert}(s)$. The *claim* p of c in k is the fact $(p, c, \text{ver}(k, c))$. The table's room is 16. The start $\text{start}(a, b)$ has counters a and b , versions 0, line 1, an empty table, an empty channel, the trap down, $\mu = 0$ and the flag down. A state is *clean* when its table and channel are empty and its flag is down.

Definition 14.3 (Instructions, costs, the step). The instructions are INC c , DEC c j , HALT, CHECK p c , COMMIT p c and CERTIFY, with effects on a core with the trap down as in the table above. Passing any check, commitment or certification requires the trap to be down. With that condition, CHECK passes when $\text{eval}(p, \text{val}(k, c)) = \text{yes}$ and the table holds fewer than 16 facts, COMMIT passes when the claim p of c is in the table, and CERTIFY passes when the channel holds a fact; a failing one sets the trap and changes nothing else. On a core with the trap up, every instruction changes nothing. Costs: CHECK, COMMIT and CERTIFY cost 1, the rest 0. One step on a state is

$$\text{exec}(s, i) = (\text{cexec}(\text{core}(s), i), \mu(s) + \text{cost}(i), \text{cert}(s) \vee \text{fires}(\text{core}(s), i)),$$

where cexec is the core's step and $\text{fires}(k, i)$ is yes exactly when i is CERTIFY and passes at k . For any list t of instructions (a *trace*), $\text{run}(t, s)$ applies them in order and $\text{cost}(t)$ adds their costs. A *program* is a list of instructions numbered from 1; a program run fetches the instruction at the line number and executes it, and stops when the trap is up, the line holds HALT, or the line holds nothing.

Defined as `cexec`, `exec`, `run`, `total_cost`, `run_prog`, `start` and `Minimal.EarnedCore.clean_start` (`EarnedCore.v`).

Read the step line slowly, because the rest of the chapter leans on its shape. The core moves by its own rule and nothing else. The ledger adds the cost. The flag becomes "it was up, or this is a CERTIFY that passed." The core's move doesn't mention μ or the flag anywhere. The base is blind to the record by construction.

A worked run, the shortest one there is. The program: CHECK *at least 0* A ; COMMIT *at least 0* A ; CERTIFY. Start anywhere, $\text{start}(a, b)$ for any a and b . Line 1: every whole number is at least 0, the checker says yes, the table is empty so there's room, and the fact (*at least 0*, A , 0) goes in. $\mu = 1$. Line 2: the claim about A 's current edition is (*at least 0*, A , 0), and it's right there. Into the channel. $\mu = 2$. Line 3: the channel holds a fact, the flag goes up. $\mu = 3$. Line 4 has no instruction, so the program stops. Three paid acts, in order, and a flag that can tell you which fact it stands on. It's the dumbest possible claim (a number is at least zero, stop the presses), and that's fine, because the point is the chain, not the claim.

14.4 It computes everything two counters can

First the base, because if the base can't compute, nothing else here matters.

Definition 14.4 (Two-counter program, compile, window). A *two-counter program* M is a list of instructions, each either $\text{MINC } c$ or $\text{MDEC } c \ j$. A configuration is a triple (line, a, b) . One step of M : if the line holds no instruction, M has *stopped*; $\text{MINC } c$ adds one to c and goes to the next line; $\text{MDEC } c \ j$ subtracts one from c and goes to line j if c is above 0, and goes to the next line if c is 0. $\text{mrun}(n, M, x)$ is n steps from x , staying put once stopped. The *compile* of M replaces each $\text{MINC } c$ by $\text{INC } c$ and each $\text{MDEC } c \ j$ by $\text{DEC } c \ j$. The *window* of a core k is the triple $(\text{line number}, \text{val}(k, A), \text{val}(k, B))$.

Theorem 14.2 (Step for step). *For every two-counter program M and every core k with the trap down: M has stopped at $\text{window}(k)$ if and only if the program $\text{compile}(M)$ has halted at k ; and if M steps from $\text{window}(k)$ to y , then one program step of $\text{compile}(M)$ from k lands on a core with window y and the trap still down. So for every n , n program steps of $\text{compile}(M)$ from k land on a core whose window is $\text{mrun}(n, M, \text{window}(k))$, with the trap down.*

Proof. Look up the instruction at the line. If there is none, both sides stop. If it's $\text{MINC } c$, the compiled $\text{INC } c$ adds one to c and goes to the next line, which is exactly the window's move (the version also moves, but the window doesn't show versions). If it's $\text{MDEC } c \ j$, split on whether c is 0, and both sides do the same thing in each case. Neither instruction can trap. The run statement is induction on n , one step at a time. $\square$

Checked as `simulation_step` and `simulation_run` (EarnedCore.v).

Theorem 14.3 (Halting matches). *For every two-counter program M and whole numbers a, b : M stops after some number of steps from $(1, a, b)$ if and only if the program $\text{compile}(M)$ halts after some number of program steps from $\text{start}(a, b)$.*

Proof. The start has window $(1, a, b)$ and its trap is down. Apply Theorem 14.2 after any number n of steps: the two configurations still have the same window, the trap is still down, and one is stopped exactly when the other is halted. The same n witnesses termination in either direction. $\square$

Checked as `halting_correspondence` (EarnedCore.v).

Now the part that isn't mine. Proving that halting is undecidable for two-counter machines is real work, and it's already done, checked, in a shared Coq project called the Coq Library of Undecidability Proofs (presented by Yannick Forster, Dominique Larchey-Wendling and coauthors at CoqPL 2020). The two-counter result in it is

Larchey-Wendling’s, building on his and Forster’s work on Minsky machines (CPP 2019). Their two-counter machine has the same two instructions as Minsky’s, with the same rule for zero. So the only thing I have to prove is that their machine and my INC/DEC fragment are the same machine, step for step, and then borrow their theorem.

One honest wrinkle about what “undecidable” means in that library, because it isn’t the textbook sentence. Every function you can write in Coq is a program, so “decidable” there means “some Coq function answers it.” Coq by itself can’t prove that *no* Coq function answers a question without an extra assumption about what programs are. So the library says the strongest thing it can without one: if anything decided this, then the questions on which a single-tape Turing machine *doesn’t* halt could be listed by a program, which is the classic thing that can’t be done. That’s their convention, and the theorem below inherits it exactly.

Theorem 14.4 (Halting is undecidable for this machine). *The library’s two-counter machine agrees with the INC/DEC fragment: its step, its stop, and its termination match those of the compiled program, so a library problem (P, a, b) halts if and only if the compiled program halts from $\text{start}(a, b)$. Consequently, the question “does program P halt from $\text{start}(a, b)$?” is undecidable in the library’s sense: a decider for it would make the complement of single-tape binary Turing machine halting enumerable.*

Proof. The agreement is a case split on the library’s instruction at the current line, and then induction along the run, ending in Theorem 14.3. The compile map is then a reduction from the library’s two-counter halting problem, which the library proves undecidable, and the library’s own lemma carries undecidability across a reduction. $\square$

Checked as `mm2_step_iff`, `mm2_stop_iff`, `mm2_terminates_iff`, `mm2_halt ing_iff` and `earned_core_halting_undecidable` (`EarnedCoreLinks.v`), the last applying the library’s `MM2_HALTING_undec`.

What that buys, and what it doesn’t. It buys: the base of this machine is a universal machine in Minsky’s sense, and nobody (in the library’s sense) can predict in general what it’ll do. What it doesn’t buy is a fast machine, or one where your input arrives conveniently. The counters start holding a and b , given directly; there’s no encoding layer in my files. Minsky’s prime-power packing is how you’d get from “any Turing machine” to “two counters,” and that construction is his and the library’s, not something I re-proved.

14.5 It’s a certification system, and the flag is a latch

Now the record. Three facts first, all of them read straight off the step line.

Theorem 14.5 (Ledger, toll, permanence). *For every state s and instruction i , $\mu(\text{exec}(s, i)) = \mu(s) + \text{cost}(i)$, and for every trace t , $\mu(\text{run}(t, s)) = \mu(s) + \text{cost}(t)$. If $\text{cert}(s)$ is down and $\text{cert}(\text{exec}(s, i))$ is up, then i is CERTIFY, it passes at $\text{core}(s)$, and $\text{cost}(i) \geq 1$. If $\text{cert}(s)$ is up, so is $\text{cert}(\text{exec}(s, i))$.*

Proof. Read the ledger component of Definition 14.3: it adds exactly $\text{cost}(i)$. Repeated substitution along a trace gives the sum of its costs, or, equivalently, induction on the length of the trace does. The flag's new value is its old value or $\text{fires}(\text{core}(s), i)$. A false-to-true change therefore requires that event, whose definition requires a passing CERTIFY. CERTIFY costs 1. An already true flag stays true under the same disjunction. $\square$

Checked as `mu_conservation`, `mu_conservation_trace`, `only_certify_certifies`, `a2` and `cert_permanent` (EarnedCore.v).

That third clause in the middle is Section 3's toll, for this machine. Which means this machine is one of the things Section 3 was about, and Section 3's theorem applies to it without my lifting a finger.

Theorem 14.6 (An instance of the certification system). *The states, the instructions, the step exec , the cost and the flag, with the toll of Theorem 14.5, form a certification system (Definition 3.1). So the universal floor (Theorem 3.1) holds here: if $\text{cert}(s_0)$ is down and $\text{cert}(\text{run}(t, s_0))$ is up, then $\text{cost}(t) \geq 1$.*

Proof. Take the state and instruction sets of Definitions 14.2 and 14.3, their step and cost, and the flag as the reading. These provide the five data fields of Definition 3.1. Theorem 14.5 supplies its one required rule. Applying Theorem 3.1 gives the claimed bound. $\square$

Checked as `earned_cs` and `earned_core_floor` (EarnedCoreLinks.v); `earned_core_floor` is `universal_nfi_any_substrate`, the checked name of Section 3's theorem, applied to `earned_cs`. The same floor is proved directly, without the link, as `nfi_floor` (EarnedCore.v).

Then Section 10. Section 10 said: take any base, and any record that answers only to the base and never comes back down, and the record is a latch. This machine has a base: a program together with a core, stepping by the program step. And it has a record, the flag, with the ledger riding along. So the question is whether it's an *honest* extension in Section 10's five-rule sense (Definition 10.2), and if it is, Section 10 does the rest.

Theorem 14.7 (The flag is a latch of the base). *Let the base have states $(\text{program}, \text{core})$, stepping by the program step, starting at $\text{start}(a, b)$'s core for any a, b , and halting when the*

program has halted. Map each (program, state) to (program, core). Then the machine, run as a closed machine on (program, state), is an honest extension of that base, and so its flag factors as the latch (Theorem 10.1) of the event “the next instruction is a CERTIFY that passes.”

Proof. The rules of Definition 10.2 one at a time. Driven by the computation: the next flag is the old flag or “the next instruction is a passing CERTIFY,” which is a function of the base state and the old flag (that’s the step line). Ledger: μ only adds. Toll: Theorem 14.5. Permanence: Theorem 14.5. A reachable write: the three-line program of the worked run raises the flag from a start. Section 10’s theorem then gives the factorization. $\square$

Checked as `earned_core_honest` and `earned_core_is_latch` (EarnedCore-Links.v), the latter by `record_axis_is_latch_holds`; the step-level facts are `cert_latch` and `base_blind` (EarnedCore.v).

So far, nothing new. The toll, the floor and the latch are things Part III already promised about any machine of this kind. The next section is the part Part III couldn’t give.

14.6 Commitments are earned

Here’s the shape before the words. Pick any run from a clean start, any list of instructions at all, chosen by somebody who’d love to cheat. Look at the moment a COMMIT would pass. Then, somewhere earlier in that run, there’s a CHECK of exactly the same claim that passed and paid, at the same edition of the counter, and between that CHECK and now, nothing touched the counter. Not “probably.” Not “on the programs I wrote.” On every list of instructions anybody can write.

To say “nothing touched it” exactly, one word.

Definition 14.5 (Untouched). A list of instructions *mid*, run from a state *s*, leaves counter *c* *untouched* when, for every way of splitting *mid* as *t*₁, then *i*, then *t*₂, executing *i* at the core reached by running *t*₁ from *s* leaves both `ver(·, c)` and `val(·, c)` as they were.

Defined as `untouched` (EarnedCore.v).

Theorem 14.8 (A commitment was checked first). *Let s_0 be clean and pre any trace after which COMMIT $p\ c$ would pass. Then pre splits as pre_1 , then CHECK $p\ c$, then mid , where: the CHECK passes at the core reached by pre_1 ; CHECK $p\ c$ and COMMIT $p\ c$ each cost at least 1; c ’s version after pre_1 equals its version after pre ; and mid , run from just after the CHECK, leaves c untouched.*

Proof. COMMIT passes, so the claim $(p, c, \text{ver}(c))$ for the current version is in the table. By the no-forging invariant (Theorem 14.12 below), that fact was written by a passing

CHECK $p\ c$ at some point pre_1 , with the version c had then; since it's the current version too, the two versions are equal. Versions never go down, and any instruction that changes a counter's value also raises its version, so an instruction in mid that touched c would have left c 's version higher at the end than at the CHECK. It isn't. So nothing in mid touched c . $\square$

Checked as `earned_commitment_provenance` (`EarnedCore.v`).

Theorem 14.9 (A raised flag was checked, then committed, then certified). *Let s_0 be clean and t any trace with $\text{cert}(\text{run}(t, s_0))$ up. Then for some p and c , t is*

$$pre_1, \text{ CHECK } p\ c, \ mid_1, \text{ COMMIT } p\ c, \ mid_2, \text{ CERTIFY}, \ post,$$

where the CHECK passes at its state, the COMMIT passes at its state, the CERTIFY passes at its state, c has the same version at the CHECK as at the COMMIT, and mid_1 leaves c untouched.

Proof. The flag starts down, so some first instruction raised it, and by Theorem 14.5 it's a passing CERTIFY. A CERTIFY passes only with a fact in the channel, the channel starts empty, and only a passing COMMIT puts a fact there, so some earlier COMMIT $p\ c$ passed. Theorem 14.8 at that COMMIT gives the CHECK, the equal version, and the untouched stretch. $\square$

Checked as `earned_certification_provenance` (`EarnedCore.v`).

And the failure side, so you can see the refusals are clean and not half-done:

Theorem 14.10 (Unearned steps trap and change no claim). *If COMMIT $p\ c$ would not pass at a core k , executing it puts the trap up and leaves the channel and the table as they were. If CERTIFY would not pass at a state s , executing it puts the trap up and leaves the flag as it was.*

Proof. If the trap is already up, the core is unchanged and its trap remains up. Otherwise, the failed COMMIT takes the failure branch of the instruction table: set the trap, leave every other core field alone. A failed CERTIFY takes the same branch, and its fires value is false. The flag update is then its old value or false, which leaves it unchanged. The instruction's charge still applies to the ledger. $\square$

Checked as `unearned_commit_traps` and `uncommitted_certify_traps` (`EarnedCore.v`).

Read Theorem 14.9 once more for what it *doesn't* say. Between the CHECK and the COMMIT, the counter is untouched. Between the COMMIT and the CERTIFY, it isn't promised anything. You can check A is even, commit to it, add one to A , and then

certify. The flag goes up, and what it stands on is a commitment about the edition that was current when you committed. The certificate is historical: “this was checked and committed about this edition,” not “this is still true.” I think that’s the right meaning for a certificate (a birth certificate doesn’t expire when the baby grows), but it’s a meaning, and you should know which one you’re getting.

14.7 What a passing check means

Provenance says a CHECK happened. It doesn’t yet say the CHECK was *right*, and that’s a separate question with a separate proof. A machine could have a perfectly ordered chain of check, commit, certify, all standing on a checker that says yes to everything. So: what does a fact in the table actually tell you about the counter?

Theorem 14.11 (Checker soundness). *Let s_0 be clean and t any trace, and let k be the core of $\text{run}(t, s_0)$. For every fact (p, c, v) in k ’s table with $v = \text{ver}(k, c)$, the property holds of the counter’s current value: $\text{holds}(p, \text{val}(k, c))$. In particular, if COMMIT $p\ c$ would pass at k , then $\text{holds}(p, \text{val}(k, c))$.*

Proof. Carry one sentence through every instruction: every fact in the table has a version no higher than its counter’s, and if its version equals its counter’s, its property holds of its counter’s value. A clean start has an empty table, so the sentence holds there. A passing CHECK adds a fact at the current version, and the checker said yes, so by Lemma 14.1 the property holds. A write raises one counter’s version, so every old fact about that counter falls strictly behind and the second half no longer applies to it. Nothing else changes a value or a version. The sentence holds after every instruction, so it holds at the end. The part about COMMIT is the case where the claim is in the table at the current version. $\square$

Checked as `checker_soundness` and `committed_claim_holds` (EarnedCore.v), through the invariant `sound` and its one-step form `sound_step`.

So a live fact means exactly this: one of three arithmetic properties holds of one counter’s current value. Not “the program is correct.” Not “the number is interesting.” The property, of the value, now. That’s all “checked” means on this machine, and it’s enough to be true.

14.8 Nothing gets forged

The proof of Theorem 14.8 leaned on an invariant I promised to show you. Here it is. The fear it answers: could some clever sequence of instructions get a fact into the table without a CHECK writing it? A DEC with a funny jump target, a COMMIT after a trap, something I didn’t think of? The answer is checked one instruction at a time, which is the only way I trust myself to answer it.

Definition 14.6 (Earned fact; no forgery). After a trace t from s_0 , a fact $f = (p, c, v)$ is

earned when t is *pre*, then CHECK p c , then *mid*, where the CHECK passes at the core reached by *pre*, f is exactly the claim p of c at that core, and, if f is live at the end (v equals c 's version after t), *mid* leaves c untouched. The trace has *no forgery* when every fact in the table after it is earned.

Theorem 14.12 (No forging). *For every core k , instruction i and fact f in the table after executing i at k : either f was already in k 's table, or i is CHECK of f 's property and counter, it passes at k , and f is the claim it writes. Consequently, if a trace has no forgery, so does that trace followed by any one instruction; and from every clean start, every trace has no forgery.*

Proof. The first part is a case for each of the six instructions: only a passing CHECK adds anything, and it adds exactly its claim. For the second, a new fact is earned by the CHECK that just wrote it; an old fact stays earned, and if it's still live after the new instruction, its counter's version didn't move, so the new instruction didn't touch the counter and the untouched stretch grows by one. A clean start has an empty table, which has no forgery, and the step carries it along any trace. $\square$

Checked as `facts_step`, `no_forging_step` and `no_forging` (EarnedCore.v).

14.9 A full table traps

Sixteen facts of room, and no eviction. Two halves to prove: nothing ever leaves, and nothing ever overflows.

Theorem 14.13 (Bounded table, no eviction). *Every instruction keeps every fact already in the table. If the table holds 16 or more facts, CHECK p c puts the trap up and leaves the table as it was. If the table holds at most 16 facts, it still holds at most 16 after any instruction.*

Proof. With a raised trap no instruction changes the table. With the trap down, only CHECK can change it, and a successful CHECK adds one fact without removing an entry. Success requires a previous length strictly below 16, so the new length is at most 16. At length 16 or greater that branch is unavailable: CHECK raises the trap and preserves the table. Every other instruction preserves it as well. $\square$

Checked as `facts_keep`, `full_table_traps` and `facts_bounded_step` (EarnedCore.v).

Why trap instead of making room? Because “making room” means deciding which fact dies, and a fact that can die is a fact whose absence you have to explain later. A table that only grows is a history you can read back, and the provenance proofs above lean on exactly that. The bill for it is real, though, and it's in the section after next.

14.10 The window can't see the record

Section 7 was about windows: a view of a state that shows some of it, and the theorem that a window which can't tell two states apart can't price the step between them exactly (Theorem 7.2). Here's the smallest pair I could build where the window is Minsky's view of the machine (line number and the two counters) and the record is everything else.

Run two traces from $\text{start}(0, 0)$. The *earned* run is CHECK is zero A; COMMIT is zero A; CERTIFY. The *idle* run is DEC A 0, three times. In the idle run, A is 0 every time, so each DEC just goes to the next line and writes nothing. Both runs end on line 4, with $A = 0$ and $B = 0$, both versions 0. Through the window, identical. One of them has paid 3, holds a fact, could commit again, and has its flag up. The other has paid nothing, holds nothing, can't commit, and its flag is down. Two photographs, same picture, different people.

Theorem 14.14 (Same window, different record). *The earned run and the idle run from $\text{start}(0, 0)$ end at cores with the same window and the same versions of A and B, and with different ledgers, different flags, different tables, and different answers to "would COMMIT is zero A pass."*

Proof. In the earned run, CHECK adds (Zero, A, 0), COMMIT puts that fact in the channel, and CERTIFY raises the flag. The ledger is $1 + 1 + 1 = 3$. No counter is written. In the idle run each DEC sees zero, so it advances the line without writing a counter or raising a version. Its three costs are zero and it adds no fact or commitment. Thus both final windows are (4, 0, 0) with versions (0, 0). The earned table contains the current zero claim and the idle table is empty, giving the stated difference in whether another COMMIT would pass, as well as in the ledger and flag. $\square$

Checked as `receipt_separation` (EarnedCore.v).

Theorem 14.15 (No window oracle). *There is no function g from windows to whole numbers with $g(\text{window}) = \mu$ at the end of every trace from $\text{start}(0, 0)$. Likewise there is no function from windows to yes or no that gives the flag at the end of every such trace, and none that gives whether COMMIT is zero A would pass.*

Proof. Such a g would have to give both 3 and 0 on the one window the two runs of Theorem 14.14 share. The same for the flag and the commit right. $\square$

Checked as `no_mu_oracle`, `no_cert_oracle` and `no_commit_oracle` (EarnedCore.v).

This is Section 7's shape, made of pebbles: the record lives off the window. Anybody who only watches the counters and the line number, which is everything Minsky's

machine *is*, can't tell you what was paid, what was committed, or whether the flag is up. Not because they looked carelessly. Because it isn't there to see.

14.11 The exact price is three

Section 3 said a raised flag costs at least one. On this machine that's true and too modest. A raised flag needs a CERTIFY, a CERTIFY needs a COMMIT, a COMMIT needs a CHECK, and each of those is its own instruction costing one. So at least three. One of what: three units of μ , one per paid act, each one the "one" I picked back in the table. And three isn't a guess that happens to hold; it's the number of links in the chain.

Theorem 14.16 (The minimum price of a certificate). *If s_0 is clean and $\text{cert}(\text{run}(t, s_0))$ is up, then $\text{cost}(t) \geq 3$ and $\mu(\text{run}(t, s_0)) \geq \mu(s_0) + 3$. For programs: if the flag is up after n program steps from $\text{start}(a, b)$, then $\mu \geq 3$ there.*

Proof. Theorem 14.9 finds a CHECK, a COMMIT and a CERTIFY at three different places in t , each costing 1, and every other instruction costs at least 0. The ledger statement is Theorem 14.5. A program run is the run of the instructions it actually executes, so the program form follows. $\square$

Checked as `certified_run_min_cost` and `program_certified_min_cost` (EarnedCore.v).

Theorem 14.17 (Three is tight). *For all whole numbers a and b , the program CHECK at least 0 A; COMMIT at least 0 A; CERTIFY, run for 4 program steps from $\text{start}(a, b)$, ends with the flag up, $\mu = 3$ exactly, and the program halted.*

Proof. Every natural number a satisfies $a \geq 0$. The first instruction therefore adds $(\text{Ge}(0), A, 0)$ to the empty table and charges 1. No counter changes, so the second instruction finds that same fact, puts it in the channel and charges 1. The third finds the channel occupied, raises the flag and charges 1. The line is now 4, beyond the three-instruction program. The fourth program step stays there and adds no charge. The final ledger is exactly 3 and the program is halted. $\square$

Checked as `min_cost_tight` (EarnedCore.v), with the program named `witness`.

That's the worked run from the start of the chapter, now as a theorem. The floor is 3 and something pays exactly 3, so 3 is the price, not a bound with slack in it.

And that's every piece of the model, standing on six instructions: a base that computes everything two counters can, a ledger that's exact, a toll the universal floor applies to, a flag that's a latch of the base, a window that can't see the record, and a commitment that can't happen unless a check of the same claim, on the same edition, came first and was right. Every result above is closed: Coq reports no assumptions for any of

them. The base file uses nothing but Coq's standard library, so you can check it with nothing else installed; the link file adds only the book's own abstract records and the library of undecidability proofs.

14.12 What it doesn't do

Here's where it falls short, in plain words, because a small machine is only honest if its edges are as visible as its middle.

- **The check price is flat.** CHECK costs 1 whether it's asking "is this 0" or "is this at least a billion." The property language is fixed and each property is one evaluation, so "pays for its work" here means that flat price, not a price that grows with how hard the question was. A machine that charged by the difficulty of the check would be a different machine, and I haven't built it.
- **DEC on zero isn't a write.** It doesn't change the counter, so it doesn't change the version. That's the right call for versions (nothing changed), but it means DEC on zero is a free way to move the line number without leaving any mark. The idle run in the window section uses exactly that.
- **After a trap, a trace still pays.** On programs, a trap stops the run. On arbitrary traces, instructions after a trap are each charged their cost and none of them changes the core. I chose to keep charging so that the ledger stays the plain sum of what was issued (Theorem 14.5 has no exceptions); the price of that choice is that a trapped machine can still be billed for nothing.
- **"Checked" means three properties and nothing more.** Is zero, is even, is at least n . A live fact means one of those holds of one counter's current value. It doesn't mean the program is correct, or that anything follows from the property, or that the property was worth checking.
- **The certificate is historical.** Between the COMMIT and the CERTIFY, the counter can change, and the flag still goes up. The flag says a claim was checked and committed about an edition, not that the claim is still true.
- **Sixteen checks per run, ever.** Facts never leave and the table holds sixteen, so a run gets at most sixteen passing CHECKs in its whole life, and the seventeenth traps. A certificate needs only one, so this costs nothing for the theorems, but it's a hard wall for any program that wants to keep checking forever.
- **The clean start matters.** Provenance, soundness, no forging and the price of three are all stated from a clean start. Hand the machine a table pre-loaded with facts nobody checked, and of course none of that follows. The theorems are about what the machine produces, not about what you smuggle in at the door.
- **Slow, and fed directly.** Universality is Minsky's, with the inputs placed straight in the counters. Getting from an arbitrary Turing machine to two counters goes through his exponential packing, which is not in my files.

14.13 What comes next

The next section opens a much bigger build. It carries far more than this one: memory and registers, modules and the arrows between them, a two-player game and its check, and eventually hardware. I'll introduce all of it there, from scratch, and I won't lean on any of it before then.

But I want one thing said plainly before you meet it, so it doesn't surprise you later. The big build prices commitments. Its toll on the flag is there and proved, and every paid step is on its ledger. What it doesn't do is *enforce that a commitment was earned*: in it, the act that checks a claim commits to nothing, and the act that commits doesn't look for a passing check (Section 22 shows exactly where). Theorem 14.9, the one that says a raised flag was checked, then committed, then certified, on the same edition with nothing in between, is a theorem about *this* machine, the small one. The earned version lives here.

So Part IV has two machines, and neither of them is the model. The small one shows the model whole, with earned commitments, on the least I could build. The big one shows how much a single build can carry while the toll still holds, and where it stops short. If you ever find yourself wondering whether a property of the big one is the model's or just the build's, this chapter is the one to hold it up against. Six instructions. If the model needs more than that, the big one is where it'll show.

15 One machine, built three times

That was the small machine: six instructions, the whole model, earned commitments and all, on the least I could build. Now the big one. Parts I to III were about *any* system with the named pieces: a state, a step, a reading, a toll, and whatever extra a theorem asked for, with boards and chairs and cups standing in whenever a picture was needed. Section 14 showed those pieces can stand up on six instructions. From here on, through the hardware, everything is about one much larger concrete build, the test bench where I find out how much a single machine can carry while the toll still holds. When I say "my machine" or "the machine" from now on, I mean exactly one thing: that build, with its state record, its fifty-one instructions and the price of each, written down in Coq and then turned into a hardware design. The six-instruction one keeps its own name, the small machine. Not a toy, not "any system." This one, warts included. I want to say three things about it before I show you a single field, because the way people misread a concrete machine is to mistake it for the idea.

First: this is one try. The theorems of Part III quantify over any states and any steps; the thing in Part IV is one particular choice of states and steps, with sixteen registers and a hundred and twenty-eight words of memory and fifty-one instructions, and every one of those numbers was a decision I made, some of them for reasons and some of them because I had to pick something. Because those theorems are stated for any states and any steps, none of them mentions these numbers. If you built a different machine that

obeyed the toll, it would be a different try at the same idea, and the latch theorem in Section 10 says its record would have the same shape underneath. Section 14 is exactly that: a different try, much smaller, and its record is a latch. One difference I'll keep saying out loud: this build prices commitments but doesn't enforce that they were earned. That guarantee is the small machine's (Section 14), and Section 22 shows where this one stops short of it.

Second: why build one at all, if the abstract theorems are the point? Because an abstract theorem about "any machine" is checked against nothing until there's a machine to check it against, and an abstract statement can be wrong in ways nobody notices until it meets one. The concrete machine is where the abstract claims get *instantiated*, and each one meets it in a place I can name now. Section 3's universal floor becomes a theorem about these fifty-one instructions (Section 17, and its whole ladder in Section 23). Section 6's counter rule becomes a statement about one field of the state (Section 16, with the run forms in Section 23). Section 7's windows become two states that look the same and aren't (Section 20), and the clerk's collisions and the verifier's escapes become named pairs (Section 23). Section 8's entitlement and Section 9's insight land in Section 22, after Section 19 has shown what a claim on this machine physically is. Section 4's finite-room merge argument becomes a statement about a finite window onto this machine, because the whole record is not finite (Section 16 says why, Section 23 builds the window). Section 5, Section 10, Section 12 and Section 13 land in Section 23 too, and Section 11's diagonal comes home in Section 24. Each of those places names the Part III claim it instantiates and points back to it rather than arguing it again. When an instantiation goes through, it's evidence the abstract statement means what I think. When one doesn't, the abstract statement is the thing under suspicion, not the machine. A machine is how a person who thinks in shapes checks that the shapes close.

Third: I built it three times, in three materials. Once as a mathematical object in Coq, a step relation on a record of twelve fields, where every theorem about the machine lives; I'll call that version the *kernel*, because it's the reference definition the other two are held up against. Once as a hardware model in a language called Kami, which is Coq again but shaped like a circuit, with a proof, instruction by instruction and under stated side conditions on the hardware's state, that the hardware's step, read back through an abstraction function, lands on the same state as the kernel's step. And once as the thing that falls out the far end of that model: Verilog, the language chips are described in, which a tool chain turned into a file that would configure a real programmable chip, if I owned one, which I don't. Aristotle's bronze from Section 2: the statue in three materials, and the promise that it's the same statue is a promise I have to keep in Section 27, not assert here.

What the next eleven sections are, in the order you'll meet them, after the small machine in Section 14. Section 16 is the state: the twelve fields and what each promises. Section 17 is the instruction set and the cost schedule, all fifty-one, in a table, with

the twelve that are never free picked out. Those two are the reference chapters: every name the rest of the machine’s story uses is handed over there first. Section 18 is the category, built from scratch, with the two laws I made you wait for in Section 2. Section 19 is about the claims a module can carry: a formula, a label, and the step that checks one against the other. Section 20 is two states that look the same and aren’t. Section 21 is a receipt, worked through in four acts. Section 22 puts Section 8’s entitlement and Section 9’s insight on the machine. Section 23 is the machine as the *witness* for the rest of Parts II and III, where the remaining forward pointers land on named theorems. Section 24 is the interpreter and the guest, where the diagonal from Section 11 comes home. Section 25 is the two-player game and its check, built from zero. Section 26 is the accounting against physics and the hierarchy of what costs more. Then the machine leaves the page: Section 27, in Part V, is the third material, the proof turned into a chip, and the place where the same-statue promise gets kept or broken.

And one rule for all of them: no sentence about what the machine does stays on the page unless I’ve read the step rule it describes. Where I’ve simplified, I say so. Where a comment in the source and the rule next to it disagree, I follow the rule. The machine is the one thing in this book I can be *exactly* right about, because it’s written down, and being exactly right about it is the only way the abstract parts earn their keep.

16 The state record

A state of the machine is a record with twelve fields. I’m going to give you all twelve, because the whole of Part III was about “the state” as an abstract thing and you’ve earned the right to see what one looks like when it’s written down. Then I’ll tell you what *isn’t* in it, which turns out to be as informative, and then the one piece of structure that the rest of the book hangs on, the partition graph.

16.1 The twelve fields

Here they are, in the order they appear in the source, with what each one is for. A few instructions get named in passing, by the names the source gives them (the ones in capitals). All fifty-one are in Section 17’s table with exactly what they do; here each one gets a few words the first time it shows up, enough to follow the field it touches.

The *partition graph*: the modules (the groups from Section 2, each owning a range of addresses), the arrows between them, and two counters that hand out fresh identifiers and only ever go up. Its own record is below.

Four *control and status words*, held together in a small sub-record: the *assertion channel* (the record calls this field the certification address, `csr_cert_addr`, and the rest of the book just says assertion channel), one of the machine’s two certification readings, holding the checksum (the sum of the character codes) of the most recent property asserted on an arrow (the name says address and the contents say fingerprint; it’s nonzero after an assertion unless that property’s checksum is zero, and each assertion overwrites it rather than adding to it, so a later one can put it back to zero); a general

status word, which no step writes; an error code, which every fault in the kernel sets to one (the hardware has a richer set of codes, and the kernel's comment lists them, but the kernel writes one and lets the hardware tell faults apart); and a heap base, used by two instructions that address memory relative to it, which no step writes either.

The *registers*: a list of sixteen natural numbers, read and written through an index taken modulo sixteen, so an out-of-range register name wraps rather than faults.

The *data memory*: a list of a hundred and twenty-eight natural numbers, addressed modulo a hundred and twenty-eight, same wrapping rule.

The *program counter*: a natural number.

The *counter* from Section 6, which the source calls *mu*: a natural number, no upper bound in the type. It starts at zero, and every step raises it by exactly the instruction's cost (Section 17), so no step lowers it. This is the field the whole book is about, and it has no ceiling, which matters for Section 23 and I'll flag it there.

A *mu tensor*: sixteen numbers, a flattened four-by-four grid, which one instruction adds into. I'll tell you what it's for in Section 26, and I'll tell you now that only one instruction ever writes it. Don't confuse it with the little tensors the modules carry inside the graph (below): the one in the record is global and REVEAL (an instruction that discloses a count of bits and pays for them) adds into it, while each module's own is written by a different instruction, TENSOR_SET.

The *error flag*: a bit that, once set, stays set; no step ever writes it back to false. The faults that step on pass through a function that ors the new value with the old, and the faults that jump to the trap address set it to true outright, so either way there's no path back. A latch, in Section 10's sense (Definition 10.2), on the event "something went wrong."

A *logic accumulator* and a *mode word*, which I include because they're in the record and not because they do anything: no step of the machine changes either. They mirror two registers in the hardware, which gives them starting values (the mode word boots at one; the kernel's starts at zero), lets you read them back, and never writes them either. The mode word's comment says zero is "Turing mode" and one is "Thiele mode," and there's a constant defined that the comment says unlocks operations, and no step rule reads it. Two fields and a constant that are scaffolding, honestly labelled.

The *witness counts*: eight counters for the two-player game of Section 25, two for each of the four ways the referee can ask the question, one for "the players agreed" and one for "they didn't." Exactly one instruction writes them, CHSH_TRIAL, one round of the game (the five instructions that run the game's check, the CHSH_LASSERT family, read them).

And the *certified flag*: the top-level yes/no reading of Section 3's certification system (Definition 3.1), the other of the two readings, set by exactly one instruction (CERTIFY)

and never cleared by any.

That's the state. Every value written into a register or a memory word is truncated to sixty-four bits on the way in; the kernel's word is sixty-four bits wide. (The one exception is XOR_SWAP, which swaps two registers and so only moves values already in the register file.) The other numeric fields, the counter, the program counter, the tensor entries, the witness counts and the assertion channel, are plain natural numbers with no truncation. (The hardware's datapath word is thirty-two. That's a difference I'll have to account for in Section 27.)

Here is the record written out, with the two sizes and the word.

Definition 16.1 (The state record). A *state* s is a record with twelve fields, in this order: the partition graph $\text{graph}(s)$ (Definition 16.3); the control and status words, a record of four natural numbers ($\text{certaddr}, \text{status}, \text{errcode}, \text{heapbase}$); the registers $\text{regs}(s)$ and the data memory $\text{mem}(s)$, each a list of natural numbers; the program counter $\text{pc}(s) \in \mathbb{N}$; the counter $\mu(s) \in \mathbb{N}$; the global tensor, a list of natural numbers; the error flag $\text{err}(s)$, a boolean; the logic accumulator and the mode word, both in $\mathbb{N}$; the witness counts, eight natural numbers same_{xy} and diff_{xy} for $x, y \in \{0, 1\}$; and the certified flag $\text{cert}(s)$, a boolean. No field's type bounds a number or fixes a list's length.

Definition 16.2 (Registers, memory, and the word). The sizes are the literals $R = 16$ and $M = 128$. Write $w(x) = x \bmod 2^{64}$ for the 64-bit truncation. Reading register r returns entry $r \bmod R$ of the register list, or 0 when the list is too short. Writing v to register r puts $w(v)$ at position $r \bmod R$; on a list of length R it changes that entry and no other. Memory reads and writes are the same with M in place of R . Every register and memory write in the step rules goes through these two writers, with one exception: XOR_SWAP exchanges two register entries as they are, without truncating.

Checked as `REG_COUNT, MEM_SIZE, word64, read_reg, write_reg, read_mem, write_mem, and swap_regs (VMState.v)`; `word64` is written as a bitwise and with sixty-four ones, which is the remainder mod 2^{64} .

Here's the width difference in one sum. Add $2^{32} - 1$ and 1: the kernel stores $w(2^{32}) = 2^{32}$, since that's below 2^{64} , and a thirty-two-bit word wraps the same sum to 0. The counter is the sharper case. In the kernel it has no width at all; in the hardware it's a thirty-two-bit word. So the step-by-step hardware correspondence carries a premise that the old counter plus the step's cost is below 2^{32} (the refinement lemmas in `ChshRetire.v` and `CouplingFaults.v`, for example), and a run whose counter outgrows that is outside what's proved there (Section 27).

The counter's rule, which Section 6 stated for any machine whose counter is kept by a price list (Theorem 6.1), holds on this one exactly.

Theorem 16.1 (Conservation on this machine). *For every state s and every instruction i ,*

$$\mu(\text{apply}(s, i)) = \mu(s) + \text{cost}(i),$$

where apply is the step function and cost is the schedule of Section 17 (Definition 17.4). No premise is needed: it holds on every state, reachable or not, and on every branch, the faulting ones included.

Proof. Coq’s proof is a case split over the fifty-one instructions and, inside each, over the branches of the step (success, latch, trap). In every branch the new counter is written as the old counter plus the instruction’s cost, either through one of the shared state builders or, for PNEW, PSPLIT, PMERGE and CERTIFY (the steps that create a module, split one, merge two, and raise the certified flag), written out in the rule. Each case closes by computation. This is the one-step rule of Section 6’s Definition 6.1 with the concrete schedule. $\square$

Checked as `vm_apply_mu` (MuLedgerConservation.v).

Remark 16.1 (The error flag latches, argued from the rules). If $\text{err}(s)$ is true then $\text{err}(\text{apply}(s, i))$ is true for every i . The argument is a reading of the rules, not a named theorem: every rule either copies the flag, sets it to $\text{err}(s) \vee \text{true}$ through `latch_err` (VMStep.v), or, on a trap, sets it to true. No rule writes false.

16.2 What isn’t in it

Three things people assume are in a machine state and aren’t in this one, each on purpose.

There’s no *program* in the state. The step relation takes the instruction as an argument; the program lives outside, and a runner fetches from it by program counter. So “the state” never knows what it’s about to do, which is the right design for a thing whose theorems quantify over all instruction sequences.

There’s no *output* in the state. The one instruction that emits a payload to the outside world, EMIT, leaves the payload outside the Coq state entirely; what the state keeps is the charge, at eight marks per character plus one more (the floor, which Section 17 explains), added into the counter with every other charge, and only as part of that total. It can’t tell you that anything was sent, let alone what it said, because the same total can come from other instructions.

And there’s no *heap* as a separate thing. Heap access is the base word plus an offset into the same hundred and twenty-eight words of data memory. One memory.

Also absent, and this one is subtle: the record doesn’t fix the *lengths* of the register and

memory lists. They're plain lists in the type. That the register list has sixteen entries and the memory a hundred and twenty-eight is proved as a separate invariant, not enforced by the record. I mention it because "sixteen registers" is a theorem about reachable states, and the type would let you write a state with three. (The invariant is `eg_size_permanent`: every step keeps both lengths, from a state that has them.)

Lemma 16.2 (The two lengths are kept). *Let s be a state with $|\text{regs}(s)| = 16$ and $|\text{mem}(s)| = 128$. Then for every instruction i , $|\text{regs}(\text{apply}(s, i))| = 16$ and $|\text{mem}(\text{apply}(s, i))| = 128$.*

Proof. Case split over the instructions and their branches. A step either keeps the two lists as they are or writes them through the register writer, the memory writer, or the swap. Each of those rebuilds the list as the part before position k , one new entry, and the part after it, with $k = r \bmod 16$ or $a \bmod 128$; since k is below the length, the result has the same length. $\square$

Checked as `eg_size_permanent` (EventGeneralization.v), stated there as "the size reading is permanent": if the reading holds before a step, it holds after.

One scope note on all of this. Everything the book proves about the kernel machine is a statement about this record and the step rules, nothing more. What a real run has beyond them, the program text, the stream an EMIT sends, the files a runner writes, lives in the runner, the trace and the hardware's test harness, not in the state.

16.3 The partition graph

This is the field that carries Section 2's buckets.

The graph has four parts: a next-module counter, the list of modules, a next-arrow counter, and the list of arrows. The two counters only go up (where they start is a few paragraphs down); an identifier is never reused, so a deleted module's number is dead forever. A *module* has three parts. A region, the list of addresses it owns. A list of strings the source calls axioms, which are labels no step reads (Section 19 shows what the steps do with them); no step writes a nonempty list into one either, so every module on a reachable state carries the empty list. And its own four-by-four tensor, sixteen numbers, zero by default (the comment says "flat space"; Section 26). An *arrow* is a source module, a target module, a coupling (a list of pairs of cells, one from each side, plus a label), a bit that marks it as an identity arrow, and a cost field the comment says records what was paid to certify it, which under the step rules is always zero because the asserting step never writes it. That last one is a field with a promise in its comment that the rules don't keep. The module layer and the arrow layer are separate things: two states can agree on every module and differ in their arrows, which is exactly what Section 20 builds.

Definition 16.3 (Partition graph, module, arrow). A *partition graph* G is a record $(n_G, \text{mods}_G, a_G, \text{arrows}_G)$: a next-module number $n_G \in \mathbb{N}$, a list mods_G of pairs (identifier, module), a next-arrow number $a_G \in \mathbb{N}$, and a list arrows_G of pairs (identifier, arrow). A *module* is a triple (region, axioms, tensor): a list of natural numbers, a list of strings, and a list of natural numbers, sixteen zeros by default. An *arrow* is a record (source, target, coupling, identity, certcost): two module identifiers, a coupling made of a list of pairs of natural numbers and a label string, a boolean, and a natural number. Adding a module to G gives it identifier n_G , puts it at the front of the list, and raises the counter to $n_G + 1$; the arrow list is copied unchanged.

Checked as `PartitionGraph, ModuleState, MorphismState, CouplingData,`
and `graph_add_module (VMState.v)`.

Now the regions, and here's the design decision, because there's a cheaper one. The cheap way to give modules regions is to let every module own the prefix of memory from zero up to its size, so two modules of size four own the *same* four cells; that's enough for every theorem about the counter and useless for anything about locality, and it isn't what Section 2's buckets said. The kernel says what the buckets said: *nothing in two buckets*. A region is a run of consecutive addresses, a base and a length, and it lies inside the hundred and twenty-eight words that exist; the regions of distinct modules are pairwise disjoint; and the step that creates a module, `PNEW`, *traps* (jumps to a fixed error address and raises the error flag; Section 17 has the details) if the range it asks for overlaps a range some other module owns (unless it's exactly a range a module already owns, in which case nothing changes and nothing's charged beyond the declared cost), or runs past the end of memory. Splitting a module, `PSPLIT`, gives its first half to one child and the rest to the other, in place. Merging two modules, `PMERGE`, requires their ranges to touch end to end, and traps otherwise. And all three trap when the machine is out of module numbers, which is a story of its own below. Every step preserves the properties, so from a state with no modules, every reachable state has disjoint, contiguous regions (`vm_reachable_regions_disjoint`, with the preservation theorems under it), each inside memory (`vm_reachable_partition_in_bounds`). Those are invariants proved about reachable states; they aren't part of the record type, the same way the list lengths aren't.

Two properties and a theorem, then, with nothing left to the imagination. Write $[b, b + n)$ for the list $b, b + 1, \dots, b + n - 1$, and $\text{reg}(m)$ for a module's region.

Definition 16.4 (Disjoint and contiguous regions). A graph G has *disjoint regions* when, for any two entries of mods_G at different positions in the list, no address lies in both regions. A list r is *contiguous* when $r = [\text{hd}(r), \text{hd}(r) + |r|)$, where $\text{hd}(r)$ is its first element (0 for the empty list); the empty list is contiguous. G has *contiguous regions* when every module's region is contiguous. A state s' is *reachable from* s when it arises from s by zero or more steps of the step relation.

Theorem 16.3 (The regions stay a partition of addresses). (1) If $\text{graph}(s)$ has disjoint regions and s steps to s' on any instruction, then $\text{graph}(s')$ has disjoint regions. (2) The same holds for contiguous regions. (3) If $\text{graph}(s)$ has no modules and s' is reachable from s , then $\text{graph}(s')$ has disjoint and contiguous regions.

Proof. (1) and (2) are a case split over the step relation. Steps that leave the module list alone (every instruction other than PNEW, PSPLIT and PMERGE; TENSOR_SET does change a module, but only its tensor) keep both properties trivially; so does every trap, since a trap keeps the old graph. A successful PNEW has passed the conflict test, so the new range meets no existing region except one it equals, and in that case the graph is unchanged; the new range is $[b, b + n)$ by construction. PSPLIT removes one module and adds its first half and its rest, two disjoint subsets of the removed region, and on a range $[b, b + n)$ they are $[b, b + \lfloor n/2 \rfloor)$ and $[b + \lfloor n/2 \rfloor, b + n)$. PMERGE removes two modules and adds one region inside their union, so disjointness survives without any test; contiguity of the joined region is exactly what the adjacency test checks, and without it the step traps. (3) is induction on the number of steps, starting from the fact that a graph with no modules has both properties. $\square$

Checked as `vm_step_preserves_regions_disjoint`, `vm_step_preserves_regions_contiguous`, and `vm_reachable_regions_disjoint` (VMStep.v), with the per-operation lemmas `graph_pnew_preserves_regions_disjoint`, `graph_hw_psplit_preserves_regions_disjoint`, `graph_hw_pmerge_preserves_regions_disjoint`, `graph_hw_pmerge_preserves_regions_contiguous` and `psplit_halves_of_range` under them. Nothing in these statements mentions the hundred and twenty-eight words of memory; staying inside them is a separate theorem, which needs the well-formedness condition below and comes right after it.

Now an honest limit on the word “owns.” Nothing in the kernel enforces ownership. LOAD, STORE, HEAP_LOAD and HEAP_STORE take an address and read or write that cell without looking at any module’s region. The regions are read by the instructions that make and reshape modules (PNEW, PSPLIT, PMERGE) and by the two that build arrows from regions (MORPH keeps only the coupling pairs whose ends lie inside the two ranges; MORPH_TENSOR compares ranges), and no other step rule reads one. Locality, a program touching only its own module’s cells, is a check the hardware’s partition table makes, and the kernel doesn’t (Section 27).

A few edge cases the definitions settle that no theorem names, since you’ll find them if you play: splitting a module that doesn’t exist (after its number is taken modulo sixty-four) adds two *empty* modules, using up two module numbers; merging two that don’t exist passes the adjacency check (the empty list is a run) and adds one empty module, using up one; and the creating step reads its region as “first listed address, then the count of distinct addresses,” so asking for cells 5 and 3 gets you 5 and 6, not

3 and 5. The hardware's encoding carries base and length, and the kernel reads lists that way to match.

The well-formedness condition is three clauses: every module identifier is below the next counter, every arrow identifier is below its counter, and every arrow's two ends name modules that exist. Every step preserves it, and so does a whole run (`vm_step_preserves_well_formed_graph`, `run_vm_preserves_well_formed_graph`). That's the reason PSPLIT and PMERGE delete every arrow that touches a module they remove: an arrow left behind would name a module that's gone, and the third clause would fail.

Definition 16.5 (Well-formed graph). A graph G is *well formed* when three things hold: every identifier in mods_G is less than n_G ; every identifier in arrows_G is less than a_G ; and for every arrow in arrows_G , its source and its target each appear as an identifier in mods_G .

Theorem 16.4 (Well-formedness is kept). (1) If $\text{graph}(s)$ is well formed and s steps to s' on any instruction, then $\text{graph}(s')$ is well formed. (2) If $\text{graph}(s)$ is well formed, then for any fuel k and any program P , the graph of the state reached by running P from s for at most k steps, fetching each instruction by program counter and stopping when the counter leaves the program, is well formed.

Proof. (1) is a case split over the step relation. Adding a module uses the counter as the new identifier and then raises it, so every identifier stays below the counter; the same holds for arrows. The steps that add an arrow (MORPH, COMPOSE, MORPH_ID, MORPH_TENSOR) take its ends either from modules they have just looked up or from arrows already in the graph, whose ends exist by the third clause; when a lookup fails they latch an error and keep the graph. MORPH_DELETE only removes. PSPLIT and PMERGE first delete every arrow whose source or target is a module about to be removed, then remove it, so the third clause cannot break; a PMERGE that traps keeps the old graph. (2) is induction on the fuel, using the function form of (1) at each step. $\square$

Checked as `vm_step_preserves_well_formed_graph` (VMStep.v) for (1), and `vm_apply_preserves_well_formed_graph` with `run_vm_preserves_well_formed_graph` (SimulationProof.v) for (2). Both are proved for the partition rules of this chapter: ranges, the overlap trap, the adjacency trap, and the two traps of the next part.

16.4 Inside memory, and sixty-four numbers

Two more promises about the buckets, and both are about running out.

The first is memory. There are a hundred and twenty-eight words, numbered 0 to 127,

and a module that owns cell 200 owns a cell the machine doesn't have. So PNEW looks at every address in the range it's about to claim and traps if any of them is 128 or more. Ask for cells 120 to 127 from the initial state and you get a module of eight cells, the last eight words of memory. Ask for cells 126, 127 and 128 and the step traps: the error flag goes up, the program counter jumps to the trap address, the counter is charged the declared cost anyway, and the graph is exactly what it was. Ask for the empty range and there's nothing to check, so the empty range is never past memory.

The second is module numbers, and it's the one I find a little funny, because sixty-four turns out to be a lifetime allowance and not a parking lot. Every module number comes from the graph's counter n_G , which only goes up, and no number is ever handed out twice: PSPLIT gives its two halves two fresh numbers and retires the old one, and PMERGE gives the joined module a fresh number and retires both. So n_G counts every module number the run has ever issued, and a split followed by a merge that puts the cells back exactly where they started has still spent three. The hardware's partition table has sixty-four slots, so the kernel's rule is: a step that needs k fresh numbers runs only when $n_G + k \leq 64$, and traps otherwise. PNEW needs one (and asks for it even when the range is one a module already owns, so on a spent budget every PNEW traps), PSPLIT needs two, PMERGE needs one. A run from the initial state can make sixty-four module numbers in its whole life. The extracted runner starts its counter at one (next part), so it gets sixty-three.

That has a nice side effect. PSPLIT and PMERGE read their module operands modulo sixty-four, which is how the hardware's six-bit slot field reads them. If a module numbered sixty-four could exist, the operand 64 would land on module 0 and module 64 couldn't be named at all. No such module exists: every number issued is below sixty-four, so reading a live module's number modulo sixty-four gives the number back, and every module can be named by its own number. An operand of 64 or more still names the module whose number is the remainder, so PSPLIT of 64 splits module 0. That's arithmetic on the operand, not a module hiding behind another one.

Definition 16.6 (A partition in bounds). A graph G is *in bounds* when it is well formed (Definition 16.5), every address in every module's region is less than 128, $n_G \leq 64$, and no identifier appears twice in mods_G .

Checked as the definitions `partition_in_bounds`, `regions_in_memory` and `module_ids_distinct` (VMStep.v).

Theorem 16.5 (The partition stays in bounds). (1) If $\text{graph}(s)$ is in bounds and s steps to s' on any instruction, then $\text{graph}(s')$ is in bounds. (2) If $\text{graph}(s)$ is well formed, has no modules and has $n \leq 64$, and s' is reachable from s , then $\text{graph}(s')$ has disjoint, contiguous regions, every address in every region is less than 128, its next module number is at most 64, it has at most 64 modules, and every module identifier m in it satisfies $m \bmod 64 = m$. (3)

The same holds for the step function, and for a bounded run of a program from such a state.

Proof. (1) A case split over the step relation, one part at a time; a trap keeps the old graph, so only successful steps need an argument. *Memory:* a successful PNEW passed the memory test; PSPLIT's two halves are pieces of a region already inside memory; PMERGE's joined region is the two old regions laid end to end. *The counter:* each step that takes fresh numbers checked that it had room for them, and no other step touches the counter. *Distinct numbers:* a new module gets the counter as its number, and well-formedness says every existing number is below the counter, so the new one is different from all of them; removing modules can't create a repeat. Well-formedness itself is Theorem 16.4. (2) graph(s) is in bounds (no modules, so nothing to check but the counter), and induction on the steps with (1) carries it to s' . Then s' has distinct module numbers, all below $n \leq 64$, so there are at most sixty-four of them, and each one is below sixty-four, so taking it modulo sixty-four changes nothing. The disjoint, contiguous regions are part (3) of Theorem 16.3. (3) The same case split, on the function. $\square$

Checked as `vm_step_preserves_partition_in_bounds` for (1), with its parts `vm_step_preserves_regions_in_memory`, `vm_step_preserves_modules_bounded` and `vm_step_preserves_module_ids_distinct`, and `vm_reachable_partition_in_bounds` for (2), using `modules_count_bounded` and `module_id_mod_64`, with the two tests `module_room` and `region_in_memory` (all `VMStep.v`). For (3), `vm_apply_preserves_partition_in_bounds` and `run_vm_partition_in_bounds` (`SimulationProof.v`). The chip makes the same two tests and traps the same way (Section 27).

16.5 Where a run starts

I've been saying "reachable," so here's the starting state, the kernel's `init_state`: every register and memory word zero, program counter zero, counter zero, the tensor zeros, the error flag down, the certified flag down, the assertion channel zero, the witness counts zero, and a graph with no modules and no arrows. The module counter starts at zero and the arrow counter at one. These are conventions of this initialization. The theorems about reachable states start from this state, or from any state with no modules. Three other initialization conventions used by the implementations differ from this one: the library's `empty_graph` starts both counters at one; the extracted runner starts both at one (Coq can turn its definitions into an ordinary program, here in the language OCaml, a step called *extraction*, and the runner is that program, the one the demonstration in Section 21 drives); and the standalone copy of the kernel starts both at zero. That copy is one long file, `ThieleMachineComplete.v`, that carries its own copy of the machine so the whole build can be read top to bottom; a test checks that its definitions match the kernel's text, which is a test and not a proof. When I say "initial" below I mean the kernel's.

Definition 16.7 (The initial state). The initial state s_0 has $\text{graph}(n, \text{mods}, a, \text{arrows}) = (0, [], 1, [])$; all four control and status words 0; registers sixteen zeros and memory a hundred and twenty-eight zeros; $\text{pc} = 0$; $\mu = 0$; the global tensor sixteen zeros; the error flag false; the logic accumulator and the mode word 0; all eight witness counts 0; and the certified flag false.

Corollary 16.6 (What every run from the start keeps). *For every state s' reachable from s_0 : the graph of s' has disjoint, contiguous regions, each inside memory; it is well formed; it has at most sixty-four modules, each numbered below sixty-four; and $|\text{regs}(s')| = 16$ and $|\text{mem}(s')| = 128$.*

Proof. s_0 has no modules, so part (3) of Theorem 16.3 gives the regions. The graph of s_0 is well formed because all three clauses are about empty lists, and Theorem 16.4 carries it along every step. Its module counter is $0 \leq 64$, so part (2) of Theorem 16.5 gives memory, the count and the numbers. The lengths hold at s_0 ; each step of the relation lands on the value of the step function (the bridge between the two, which Section 17 states as Lemma 17.1), so Lemma 16.2 carries them. $\square$

Checked as `init_state` (MuInitiality.v) for the values. The corollary is the four theorems instantiated at s_0 ; no single Coq name packages it, and the step “the empty graph is well formed” is checked for the library’s `empty_graph` (`empty_graph_well_formed`, VMState.v) rather than for this graph, whose counter starts at zero instead of one.

16.6 The size constants

Sixteen registers, a hundred and twenty-eight words, sixty-four module slots, a region size of sixteen. All four are definitions with literal values, and it’s easy to read the first two as parameters. They aren’t: the proofs unfold the literal and compute with it (`unfold REG_COUNT`; `lia` and its cousins appear throughout), no file binds either as a variable or hypothesis, and no theorem is stated for other values. The proofs are about sixteen and a hundred and twenty-eight. Retargeting means editing one definition *and* every tactic that computes on the literal, which the hardware side’s own comment admits for its table sizes. The sixty-four is the module-number budget of the last part, the one the partition rules check, and the same number the hardware’s table has slots for; the split and merge rules still write their operand reduction as “mod 64” with the literal. The region-size constant is used nowhere at all.

Remark 16.2 (The four constants). The kernel defines $R = 16$ (`REG_COUNT`), $M = 128$ (`MEM_SIZE`), 64 (`NUM_MODULES`) and 16 (`REGION_SIZE`), each as a literal in VMState.v. R and M enter the semantics through $r \bmod R$ and $a \bmod M$ (Definition 16.2), the initial lists (Definition 16.7), and one cap: MORPH reads at most $M/2 = 64$ coupling pairs from memory. M also enters through PNEW’s memory test, and `NUM_`

MODULES through the module-number test of PNEW, PSPLIT and PMERGE (Definition 16.6 and Section 17). The operand reduction of PSPLIT and PMERGE writes $m \bmod 64$ with the literal, not the constant, and `REGION_SIZE` appears nowhere else. Every theorem in this chapter that mentions a size is about these values and no others.

16.7 What the record promises, and what it doesn't

It promises: a counter that's a natural number, a flag that's a bit, an error that latches, a graph whose regions don't overlap and stay inside memory on any reachable state, at most sixty-four module numbers per run, and sixty-four-bit truncation on every register and memory write. It doesn't promise: what the program is, what was emitted, that the lists have their intended lengths (that's a separate invariant), that the axiom strings mean anything, or that the per-arrow cost field records anything. And it's one choice of record. The abstract theorems needed only "a state, a step, a reading, a cost," and this is one way to fill those in, with twelve fields, two of which are scaffolding.

Next: what you can *do* to one of these. Fifty-one things.

17 The instruction set and the cost schedule

Fifty-one instructions. I'm going to give you all of them, because a reader who's been told "the machine charges for certification" deserves to see the whole price list and check that nothing else is charged for in secret, and because the table is the thing that makes Section 4's argument (Theorem 4.2: finiteness, permanence and merge pricing force the toll) checkable on *this* machine: here are the steps, here are the prices, go and see which ones merge. Three things to know before the tables.

Every instruction carries a declared cost, which I'll write d , chosen by whoever writes the program; it can be zero. The counter rises by exactly the instruction's cost on every path, success or failure, because the cost is on the attempt (Section 6's conservation, Theorem 6.1, checked on this machine as Section 16's Theorem 16.1, `vm_apply_mu`). (A housekeeping fact about every theorem name below: the rows are step *relations*, but the theorems are stated on `vm_apply`, the function that computes a step, and `vm_step_vm_apply` is the bridge. If the relation says one state steps to another on an instruction, the function returns exactly that state.) Thirty-nine instructions cost exactly d . Twelve cost at least one more than d , and those twelve are the cost-positive class; the predicate that picks them out is called `is_cert_setterb`, and its own comment says it is not the literal "sets `csr_cert_addr`" predicate (`csr_cert_addr` is the source's name for the assertion channel): it marks the positive-cost policy. In the step rules only *two* instructions actually flip a certification reading (one sets the assertion channel, one sets the flag), and I'll point at them. For the channel, the fact is `morph_assert_step_sets_supra_cert_iff`: from a clear channel, one step leaves it nonzero exactly when that step is a MORPH_ASSERT on an arrow that exists, with a property whose checksum isn't zero. For the flag it's the one line in CERTIFY's rule. The class is wider than the flippers, which is a design choice I'll defend after the tables.

Register names wrap modulo sixteen and memory addresses modulo one hundred and twenty-eight, and every value written to a register or a memory word is truncated to sixty-four bits (Section 16).

And how to read the tables. In the cost column, $S\ d$ means d plus one: S is Coq’s name for “the next number,” which is the most Coq way possible of saying plus one, and that plus one is the floor. The fault column says what a failure does. TRAP jumps to a fixed trap address, 3840, and sets the error code and the error flag. LATCH steps on to the next instruction, sets the error code and the flag, and changes nothing else. The last column says whether the instruction is in the cost-positive class.

Here is the shape of a step, exactly, before the tables fill it in.

Definition 17.1 (Instructions, step relation, step function). An *instruction* is one of fifty-one constructors, each with its operands and a declared cost $d \in \mathbb{N}$. The *step relation* $s \xrightarrow{i} s'$ is an inductive relation on states with one rule per outcome of each instruction: one rule for an instruction that cannot fail, and a success rule and a failure rule for one that can. The *step function* $\text{apply}(s, i)$ is a total function from a state and an instruction to a state. A *run* from s on a program P (a list of instructions) with fuel k fetches the instruction at position pc of P , applies it, and repeats, stopping after k steps or when pc falls outside P . The program is never part of the state.

Lemma 17.1 (The relation computes the function). *For all states s, s' and every instruction i : if $s \xrightarrow{i} s'$ then $\text{apply}(s, i) = s'$. In particular the relation is deterministic: if $s \xrightarrow{i} s_1$ and $s \xrightarrow{i} s_2$ then $s_1 = s_2$.*

Proof. Invert the step: each rule of the relation names a right-hand state, and the matching branch of the function computes to the same record once the rule’s side conditions are substituted. Determinism follows because both results equal $\text{apply}(s, i)$. $\square$

Checked as `vm_step` (VMStep.v), `vm_apply`, `run_vm`, `vm_step_vm_apply` and `vm_step_deterministic` (SimulationProof.v). Coq states this direction only; that every value of the function is also a step of the relation is not a named lemma. Theorems stated on the function therefore cover every state the relation reaches.

17.1 Group 1: partition (4)

Instruction	Effect	Cost	Fault	Positive
PNEW region, d	Claims a range of cells as a fresh module.	d	TRAP on overlap with another module's range, on a cell past memory, or with no module number left	no
PSPLIT module, left, right, d	Replaces the module with two holding the first half and the rest of its range.	d	TRAP with fewer than two module numbers left	no
PMERGE m1, m2, d	Replaces two modules with one holding the joined range.	d	TRAP unless the ranges touch end to end and a module number is left	no
PDISCOVER module, evidence, d	Steps on. The evidence is not attached to anything.	d	none	no

The table is a diet version, so here's what it left on the plate. PNEW claims the range from the first listed address for as many cells as there are distinct addresses, and traps if any cell of that range is past the hundred and twenty-eight words of memory. If some module owns exactly that range, nothing changes; otherwise a fresh module with no axioms is added; and a PNEW that traps leaves the graph unchanged. All three take fresh module numbers from a budget of sixty-four per run that is never refunded (Section 16 tells that story): PNEW asks for one, PSPLIT for two, PMERGE for one, and each traps when the budget can't cover it. PSPLIT and PMERGE remove every arrow touching a module they remove. Both take their module operands modulo 64 first; every module that exists is numbered below 64 (Theorem 16.5), so this names the module itself, and an operand of 64 or more names the module numbered by its remainder: 64 splits module 0. PSPLIT accepts its left and right operands and ignores them, and a missing module splits into two empty ones. PMERGE's joined range starts from the lower base.

The three rules that reshape modules, exactly. They use the region notions of Section 16 (Definition 16.4).

Definition 17.2 (PNEW, PSPLIT, PMERGE). For a list r , let $\bar{r}$ be r with repeats removed (first occurrences kept) and $q(r) = [\text{hd}(\bar{r}), \text{hd}(\bar{r}) + |\bar{r}|)$. A range q *conflicts* with G when some module's region shares an address with q without having the same set of addresses. Write n_G for the graph's next module number; G *has room for* k when $n_G + k \leq 64$. A *trap* keeps the graph, sets the error code to 1, sets the error flag, and sets pc to 3840. Every branch below adds the instruction's cost to μ , and every branch that does not trap sets pc to pc + 1.

PNEW r : if G has no room for 1, or some address of $q(r)$ is 128 or more, or $q(r)$ conflicts with the graph, trap. Otherwise, if some module's region has the same addresses as $q(r)$, keep the graph; if not, add a module with region $q(r)$, no axioms, and a zero tensor.

PSPLIT m : if G has no room for 2, trap. Otherwise let $m' = m \bmod 64$ and ρ the region of m' (empty if m' is absent). Delete every arrow whose source or target is m' , remove m' if present, then add a module with the first $\lfloor |\rho|/2 \rfloor$ addresses of ρ and another with the rest. Its two region operands are not read.

PMERGE m_1, m_2 : let $m'_j = m_j \bmod 64$ and ρ_j their regions (empty if absent). If G has no room for 1, or neither $\rho_1\rho_2$ nor $\rho_2\rho_1$ (concatenation) is contiguous, trap. Otherwise delete every arrow touching m'_1 or m'_2 , remove both, and add one module whose region is $\rho_1\rho_2$ if that is contiguous and $\rho_2\rho_1$ if not.

Checked as `step_pnew`, `step_psplitt`, `step_pmerge`, `pnew_region`, `region_conflict`, `module_room`, `region_in_memory`, `pnew_ok`, `pmerge_ok`, `partition_step_state`, `graph_hw_psplitt`, `graph_hw_pmerge`, `pmerge_adjacent`, `pmerge_region`, `psplitt_left` and `psplitt_right` (VMStep.v), with `graph_pnew` and `graph_add_module` (VMState.v).

17.2 Group 2: logic and certification (3)

Instruction	Effect	Cost	Fault	Positive
LASSERT freg, creg, kind, flen, d	The checking step (Section 19 takes it apart): reads a formula at the address in register freg and two witness rows at the address in register creg, checks the first satisfies and the second falsifies (a tautology has no falsifying row, so it can't pass), and that the length operand matches the length word in memory. On success, steps on; nothing else changes.	flen times 8, plus S d	TRAP on a failed check, a header mismatch, and always when kind is false (the unsatisfia- bility path is not implemented).	yes
LJOIN c1reg, c2reg, d	Steps on. Does not compare the certificates or touch any status word.	S d	none	yes
MDLACC module, d	Steps on. The comment says it charges for looking up a module.	d	none	no

About LASSERT's price: the declared length has to equal the length word in memory, so declaring a smaller formula only gets you trapped (`lassert_honest_cost`; its corollary `lassert_honest_mu_cost` says a successful LASSERT is charged the length actually read, times eight, plus the floor). The eight is a pricing convention. Nothing says each unit is a byte or that the charge counts physical information. So a

formula whose header says two units, checked with declared cost zero, costs $2 \times 8 + 1 = 17$, and a declaration of zero units for the same formula costs 1 and traps.

Theorem 17.2 (A successful LASSERT pays for what it read). *Let $s \xrightarrow{i} s'$ with $i = \text{LASSERT}$ with operands f, c , kind k , declared length ℓ and declared cost d . Let $h(s, f)$ be the length word the checker reads from the formula header at the address held in register f . If $\text{pc}(s') \neq 3840$, then $\ell = h(s, f)$ and*

$$\mu(s') = \mu(s) + 8h(s, f) + d + 1.$$

Proof. The rule's success test is the conjunction of " $h(s, f) = \ell$ " and the satisfying and falsifying checks; when the test fails the rule sets pc to 3840. So a step that lands elsewhere passed the test, which gives $\ell = h(s, f)$. The counter equation is then conservation (Theorem 16.1) with $\text{cost}(i) = 8\ell + d + 1$. $\square$

Checked as `lassert_honest_cost` and `lassert_honest_mu_cost` (StateSpace-Counting.v). The premise is about the landing address: a success whose next address happens to be 3840 is outside the statement. Two more names sit nearby and are about a different object. `lassert_cost_from_component_floors` and `lassert_cost_is_its_formula` (MuCostDerivation.v) concern a separate model of a constraint, with a state count before and after and a description length: its total is defined as $\log_2 \Omega - \log_2 \Omega' + \text{bits}$, and the two theorems say that this total is that sum and that any charge paying at least each term pays at least the total. Both hold by unfolding the definition; neither is about the machine's schedule.

MDLACC's name is short for minimum description length, the idea that a model's bill is the bits to write the model down plus the bits to write the data given the model. The step doesn't know any of that. It charges its declared d and steps on, with no graph change and no register read. If a program uses it to enter a description-length bill, the number is whatever the program declared, and nothing checks it against a description.

17.3 Group 3: compute and data transfer (14)

All fourteen step on; thirteen write one register and XOR_SWAP writes two; none touches the graph or the status words; none faults. Cost d for every one; none is in the positive class. None can move the assertion channel either: over any run of the instructions the source calls classical, which includes all fourteen, the assertion channel ends where it started (`classical_trace_preserves_cert_addr`). Arithmetic doesn't mint certified structure.

Instruction	Effect
XFER dst, src	dst := src
LOAD_IMM dst, imm	dst := imm (truncated)

Instruction	Effect
ADD, SUB, MUL dst, rs1, rs2	arithmetic modulo 2 to the 64
AND, OR dst, rs1, rs2	bitwise
SHL, SHR dst, rs1, rs2	shifts, 64-bit; the shift amount is taken modulo 64
LUI dst, imm	dst := imm shifted left by eight (eight, not the twelve of the RISC machines it's named after)
XOR_LOAD dst, addr	dst := memory at the absolute address; no exclusive-or involved despite the name
XOR_ADD dst, src	dst := dst xor src
XOR_SWAP a, b	swap two registers, as a direct exchange of the two values (no XOR sequence involved either)
XOR_RANK dst, src	dst := number of one-bits in src

17.4 Group 4: memory and control (8)

Instruction	Effect	Cost	Fault	Positive
LOAD dst, rs_addr, d	dst := memory at the address held in register rs_addr	d	none	no
STORE rs_addr, src, d	memory at that address := src	d	none	no
JUMP target, d	pc := target	d	none	no
JNEZ rs, target, d	pc := target if register rs is nonzero, else step on	d	none	no
CALL target, d	memory at register 15 := pc plus one; register 15 rises by one; pc := target. The stack grows upward and register 15 is the stack pointer.	d	none	no

Instruction	Effect	Cost	Fault	Positive
RET d	register 15 falls by one; pc := memory at register 15	d	none	no
HALT d	steps on, nothing else. The step relation never stops the machine; the runner does.	d	none	no
CHECKPOINT label, d	steps on; the label is not stored	d	none	no

LOAD and STORE above, and the heap pair in the next group, take any address and touch that cell. None of them looks at a module's region (Section 16), so the kernel doesn't enforce the locality the regions suggest; the hardware's table does (Section 27).

17.5 Group 5: revelation, ports, heap, tensor (7)

Instruction	Effect	Cost	Fault	Positive
EMIT module, payload, d	Steps on. The payload leaves the Coq state (Section 16).	eight per character of payload, plus S d	none	yes
READ_PORT dst, channel, value, bits, d	dst := value, where the value is fixed in the instruction when it was decoded. Nothing checks that the declared bits match the value.	bits plus S d	none	yes
WRITE_PORT channel, src, d	Steps on.	d	none	no
HEAP_LOAD dst, rs_addr, d	dst := memory at heap base plus the address in rs_addr	d	none	no

Instruction	Effect	Cost	Fault	Positive
HEAP_STORE rs_ addr, src, d	memory at heap base plus that address := src	d	none	no
TENSOR_SET module, i, j, value, d	writes the module's tensor entry (i, j); a missing module is a silent no-op	d	LATCH if i or j is 4 or more	no
TENSOR_GET dst, module, i, j, d	dst := that entry (zero for a missing module)	d	LATCH on a bad index	no

The payload prices in numbers, at declared cost zero: an EMIT of one character costs $8 + 1 = 9$ and of ten characters $80 + 1 = 81$ (the payload's bit length is defined as eight times its character count, `payload_bit_length_ascii`); a REVEAL of zero bits costs 1 and of thirty-two bits costs 33. Revealing more costs more, one mark per bit, and the cert operand that comes with it is never read, so what's priced is the disclosure, not anything it proves.

17.6 Group 6: witness and certification (8)

Instruction	Effect	Cost	Fault	Positive
CHSH_TRIAL x, y, a, b, d	Bumps one of the eight witness counters.	d	LATCH if any input is not a bit	no
CERTIFY d	Sets the certified flag, steps on. One of the two flippers.	S d	none	yes
REVEAL module, bits, cert, d	Adds bits to one entry of the global mu tensor.	bits plus S d	none	yes
CHSH_LASSERT d	Runs the integer check of Section 25 on the eight counters.	S d	TRAP on a failed check	yes

Instruction	Effect	Cost	Fault	Positive
CHSH_LASSERT_1AB d	The level 1+AB check (Section 25).	S d	TRAP	yes
CHSH_LASSERT_1AB_G5 d, one pair	Same, with one supplied moment, gamma five.	S d	TRAP	yes
CHSH_LASSERT_1AB_G345 d, three pairs	Same, with gamma three, four and five.	S d	TRAP	yes
CHSH_LASSERT_1AB_G12345 d, five pairs	Same, with gamma one to five.	S d	TRAP	yes

What the cells leave out. CHSH_TRIAL, given four bits, bumps the “same” or the “different” counter for the setting pair $x y$, and that’s all it records: the witness counters keep only same and different per setting pair, so no marginals are recorded, and the inequality is not checked here. CERTIFY run again on a certified state still costs. REVEAL adds its bits to entry (operand mod 16) of the global μ tensor in the record; the operand is called module but acts as a direction index, not a module slot, and the cert operand is not checked. Each supplied moment of the last three checks comes in the instruction as a same/different pair of counts.

On success every CHSH_LASSERT variant only steps on: neither certification reading changes. Success is visible only as “no trap.” The check happened, and nothing was committed; Section 22 puts that next to Section 9’s test and commit. (The source defines a marker word for a passed check, and no step writes it.) CHSH_TRIAL, in the same group, never moves a certification reading from any state and costs its declared d , which may be zero; Section 22 sorts it as raw observation in Section 9’s sense (Definition 9.1), with the theorem. Because no marginals are kept, the checks are zero-marginal conditions (the bridge to the physics, `zero_marginal_column_contractive`, is Section 25’s).

What the G345 variant checks: the base column check on the eight counters; each supplied moment $\gamma_3, \gamma_4, \gamma_5$, given as a same/different pair, strictly between -1 and 1 ; and a list of strict positivity tests on integer polynomials in the counts and the supplied pairs, three for a two-by-two block and four for the leading minors of a four-by-four matrix (Sylvester’s test for positive definiteness). All of it is integer arithmetic.

Remark 17.1 (What a passing G345 step gives). Let s have $\text{err}(s)$ false, let i be CHSH_LASSERT_1AB_G345 with declared cost d and pairs $(a_3, b_3), (a_4, b_4), (a_5, b_5)$, and let $s' = \text{apply}(s, i)$. If $\text{pc}(s') = \text{pc}(s) + 1$ and $\text{err}(s') = \text{err}(s)$, then the nine-by-nine level 1+AB moment matrix built from the four correlators of s' ’s counters, with $\gamma_1 = \gamma_2 = 0$

and $\gamma_j = (a_j - b_j)/(a_j + b_j)$ for $j = 3, 4, 5$, is symmetric and positive semidefinite. Section 25 and Appendix B carry the matrix and the argument; the check itself is `qlab_g345_full_integer_check_kernel` (VMStep.v) and the implication is `chsh_1_assert_1ab_g345_no_trap_implies_npa_psd_qlab` (QuantumPartitionPSD_1AB.v).

17.7 Group 7: arrows (7)

Every one either succeeds or latches an error; none traps.

Instruction	Effect	Cost	Fault	Positive
MORPH dst, src_mod, dst_mod, coupling_idx, d	Adds an arrow between two modules, with a coupling read from memory; identifier to dst.	d	LATCH if either module is missing	no
COMPOSE dst, m1, m2, d	Adds the composite arrow when the first's target is the second's source; identifier to dst.	d	LATCH if undefined	no
MORPH_ID dst, module, d	Adds a fresh identity arrow on a module.	d	LATCH if the module is missing	no
MORPH_DELETE id, d	Removes that one arrow. No cascade.	d	LATCH if missing	no
MORPH_ASSERT id, property, cert, d	Writes the property's checksum into the assertion channel. The other flipper.	S d	LATCH if the arrow is missing	yes
MORPH_TENSOR dst, f, g, d	Adds the tensor product of two arrows (Section 18).	d	LATCH if undefined	no

Instruction	Effect	Cost	Fault	Positive
MORPH_GET dst, id, selector, d	dst := source, target, pair count, or identity flag (selectors 0 to 3; else 0).	d	LATCH if missing	no

And the leftovers from this table. MORPH reads a coupling block from memory (a count, then pairs, then a label) and keeps only the pairs whose ends lie in the two modules' regions. MORPH_ID's arrow has an empty coupling and the identity flag set, and every call makes a fresh one. MORPH_ASSERT's checksum is the sum of the property string's character codes, and it overwrites whatever the channel held; the cert operand is never read. MORPH_TENSOR adds its arrow only when both arrows and all four of their end modules exist, both source regions are disjoint, both target regions are disjoint, and modules owning exactly each union exist.

MORPH_GET reads the graph, and so does TENSOR_GET in group 5 (it reads a module's tensor). The source still files both under the classical instructions, because neither writes the graph, a channel, the certified flag or the witness counters. Keep that in mind in Section 20.

Definition 17.3 (Classical instructions). The *classical* instructions are twenty-nine of the fifty-one: the fourteen of Group 3; the eight of Group 4; and MDLACC, READ_PORT, WRITE_PORT, HEAP_LOAD, HEAP_STORE, TENSOR_GET and MORPH_GET. The other twenty-two are not classical: PNEW, PSPLIT, PMERGE, PDISCOVER, LASSERT, LJOIN, EMIT, REVEAL, TENSOR_SET, CHSH_TRIAL, CERTIFY, the five CHSH_LASSERT variants, and MORPH, COMPOSE, MORPH_ID, MORPH_DELETE, MORPH_ASSERT and MORPH_TENSOR. For a list t of instructions, $\text{run}(t, s)$ applies them to s in order with the step function, with no fetching by program counter.

Theorem 17.3 (Classical runs move nothing that certifies). *Let t be a list of classical instructions and s any state. Then $\text{graph}(\text{run}(t, s)) = \text{graph}(s)$, $\text{certaddr}(\text{run}(t, s)) = \text{certaddr}(s)$, and $\text{cert}(\text{run}(t, s)) = \text{cert}(s)$.*

Proof. First one step: for each classical instruction, every branch of its rule copies the graph, the control and status words' address field, and the certified flag from the old state. The three with two branches (JNEZ, TENSOR_GET, MORPH_GET) copy them in both. Coq checks this by a case split that closes by computation. Then induction on t . $\square$

Checked as `is_classical_opcode`, `classical_trace_preserves_graph`, `cl`

`classical_trace_preserves_cert_addr` and `classical_trace_preserves_certified` (`ClassicalConservativity.v`). The class is a choice of the file, not a closure property: `PDISCOVER` is left out although its step changes nothing, and `READ_PORT` is in although it costs at least one. The same three equalities hold when the program runs by program counter, jumps and all (`classical_opcodes_preserve_structure_run_vm`); Section 23 states that form.

The classical view can't recover the flag either. Take the four fields a classical observer sees, the program counter, the counter, the registers and the memory, and map a list of states to the list of those four-field snapshots. No function on the snapshot lists gives back the certified flag of the last state, for every list: `no_classical_certification_decider` (`WitnessPreservationImpossibility.v`). The proof is two hand-built states that agree on all four fields and differ in the flag; they're records the type allows, not states shown reachable from the start. Section 20 does the same with the graph.

Four plus three plus fourteen plus eight plus seven plus eight plus seven: fifty-one.

17.8 The schedule, read as a whole

Four cost shapes. Thirty-nine instructions cost exactly their declared d , which may be zero. Eight cost d plus one, the floor: `LJOIN`, `CERTIFY`, `MORPH_ASSERT`, and the five `CHSH_LASSERT` variants. Three cost a payload plus the floor: `EMIT` (eight per character), `READ_PORT` and `REVEAL` (their bit count). And `LASSERT` costs the formula length times eight plus the floor. The twelve with a floor are exactly the positive class; that much is read off the definitions, and the theorems are one line each: `cert_setter_cost_pos`, if the predicate holds, the cost is at least one; `cert_setter_cost_above_declared`, it's at least one more than declared; and `non_cert_setter_cost_is_declared`, if the predicate fails, the cost is exactly what was declared.

Definition 17.4 (The schedule). For an instruction i with declared cost d , the cost $\text{cost}(i)$ is: $8\ell + d + 1$ for `LASSERT` with declared length ℓ ; $8|p| + d + 1$ for `EMIT` with payload p of $|p|$ characters; $b + d + 1$ for `READ_PORT` and `REVEAL` with bit count b ; $d + 1$ for `LJOIN`, `CERTIFY`, `MORPH_ASSERT` and the five `CHSH_LASSERT` variants; and d for each of the other thirty-nine.

Definition 17.5 (The positive class and the setter class). The *positive class* is the twelve instructions in the first four cases of Definition 17.4: `LASSERT`, `EMIT`, `READ_PORT`, `REVEAL`, `LJOIN`, `CERTIFY`, `MORPH_ASSERT` and the five `CHSH_LASSERT` variants. The *setter class* is five of them: `LASSERT`, `LJOIN`, `EMIT`, `REVEAL` and `MORPH_ASSERT`.

Theorem 17.4 (The floor). If i is in the positive class, then $\text{cost}(i) \geq d + 1 \geq 1$. If i is outside it, then $\text{cost}(i) = d$, and with $d = 0$ the instruction is free.

Proof. Case split on i . Each of the twelve has a +1 in its cost, so the cost is at least one whatever d is. The other thirty-nine cost d by definition. $\square$

Checked as `instruction_cost`, `is_cert_setterb`, `cert_setter_cost_pos` and `cert_setter_cost_above_declared` for the first sentence and `non_cert_setter_cost_is_declared` for the second, with `declared_mu_delta` reading the declared cost d off any instruction (all `VMStep.v`), the setter class `cert_addr_setterb` (`AbstractNoFI.v`) and the payload length `payload_bit_length_ascii` (`VMStep.v`). Examples of the second sentence at work: `morph_can_be_free`, `morph_compose_cost_zero`, and the PNEW, PSPLIT and PMERGE cases inside `partition_free_but_certification_nonfree`. One example runs a whole program at zero: `blind_program` (`StructuralAdvantage.v`) is a ten-instruction search loop of `LOAD_IMM`, `ADD`, `SUB`, `JNEZ` and `JUMP`, every one at declared cost zero, and `blind_program_total_cost_is_zero` says its costs sum to zero.

Now the design choice. Only two of the twelve flip a reading: `CERTIFY` sets the flag and `MORPH_ASSERT` sets the assertion channel. The other ten are in the class because they're *checks or emissions*: `LASSERT` and the `CHSH` family check and charge for the checking (Section 9's Popper point: charge the attempt, pass or fail); `EMIT`, `REVEAL` and `READ_PORT` move information across the machine's boundary and charge per bit; `LJOIN` is a join of certificates that does nothing yet and charges the floor anyway. The toll of Section 3 (Definition 3.1) needs only the two flippers to be priced. The schedule prices more. That's allowed, Section 10's billed machine says so (the toll is a floor, and a surcharge only raises the price), and it's a choice I'd defend this way: the things that cost at least one are the things that claim to have *looked at or sent* something, and I'd rather the machine never let a look or a send be free than argue about which looks matter. You can disagree. The flippers are two; the floor is on twelve; both numbers are in the table.

And more things to watch doesn't mean a bigger floor. If an instruction triggers several of the events you want priced, one mark covers all of them; the floors don't add unless you write a rule that makes them add.

Theorem 17.5 (The joint floor is the least). *Let $E_1, \dots, E_n$ be yes/no tests on instructions, and let $J(i) = 1$ if some $E_j(i)$ holds and $J(i) = 0$ otherwise. For any cost function c from instructions to $\mathbb{N}$: $c(i) \geq 1$ for every j and every i with $E_j(i)$ if and only if $J(i) \leq c(i)$ for every i . So J is the least cost meeting all the unit floors at once.*

Proof. If c meets every floor and some $E_j(i)$ holds, then $c(i) \geq 1 = J(i)$; if none holds, $J(i) = 0 \leq c(i)$. Conversely, if $J \leq c$ and $E_j(i)$ holds, then $J(i) = 1$, so $c(i) \geq 1$. $\square$

Checked as `joint_floor_is_least` (`ObservationPolicy.v`). It's stated for any instruction type, not only this machine's.

A second thing you'll notice: the arrow-building instructions are free at d equals zero. MORPH, COMPOSE, MORPH_ID, MORPH_DELETE, MORPH_TENSOR, MORPH_GET all cost d (`morph_compose_cost_zero` and its siblings), and so does PNEW. Building structure is free. *Asserting* it is not. In Section 9's terms (Definition 9.1), building never turns a reading on, so it sorts with raw observation, and asserting is certified insight; Section 22 lays the tiers out on the machine. Notice what that does to a tempting sentence: a zero-cost PNEW changes the graph, so "every change to the structure costs something" is false for this machine. What costs something is vouching.

And the theorems the table makes checkable, all on this machine, all over the fifty-one cases: one step that takes the assertion channel from zero to nonzero costs at least one (`no_free_certification`), same for the flag (`no_free_certification_certified`), either one raises the counter by at least one (`certification_requires_positive_mu`), and over a run, a start with the channel clear and an end with it set contains a positive-cost instruction and a counter that rose by at least one (`no_free_certified_insight`). One detail in that last one: the instruction it finds comes from a narrower list than the twelve, a five-member predicate called `cert_addr_setter` (`b`) (LASSERT, LJOIN, EMIT, REVEAL and MORPH_ASSERT), of which only MORPH_ASSERT writes the channel. So the theorem says which kind of instruction was in the run, and `morph_assert_step_sets_supra_cert_iff` is the finer fact about which one moved the channel. Those are Section 3's universal floor (Theorem 3.1), landed on fifty-one constructors; Section 23 lines up the whole ladder of its instances.

Here are those facts exactly. Write `certaddr(s)` for the assertion channel, `cert(s)` for the certified flag, and $\Sigma(p)$ for the sum of the character codes of a string p .

Theorem 17.6 (Only CERTIFY touches the flag, and it stays set). *For every state s and instruction i : $\text{cert}(\text{apply}(s, i))$ is true if i is CERTIFY, and equals $\text{cert}(s)$ otherwise. Hence (1) if $\text{cert}(s)$ is true, so is $\text{cert}(\text{apply}(s, i))$ for every i ; and (2) if $\text{cert}(s)$ is false and $\text{cert}(\text{apply}(s, i))$ is true, then i is CERTIFY, $\text{cost}(i) \geq 1$, and $\mu(\text{apply}(s, i)) \geq \mu(s) + 1$.*

Proof. Case split over the fifty-one instructions and their branches: CERTIFY's rule writes true, and every other rule copies the flag. (1) follows because both cases give true when the old value is true. (2): the old value is false and the new one true, so the second case is impossible and i is CERTIFY with cost $d + 1$; the counter claim is conservation. $\square$

Checked as `step_certify` (VMStep.v), `vm_apply_certified` (PrimeAxiom.v), `certification_reading_permanent` (EventSwapTheorem.v) for (1), and `no_free_certification_certified` with `no_free_certification_certified_mu` (AbstractNoFI.v) for (2). The run form is `kernel_certified_implies_positive_mu` (PrimeAxiom.v): if a run fetched by program counter starts with the flag down

and the counter at zero and ends with the flag up, the counter at the end is positive.

Theorem 17.7 (Only MORPH_ASSERT touches the channel). *For every state s : (1) if i is not a MORPH_ASSERT, then $\text{certaddr}(\text{apply}(s, i)) = \text{certaddr}(s)$. (2) For $i = \text{MORPH_ASSERT}$ with arrow k , property p , certificate c and declared cost d : if arrow k is in the graph, the step sets certaddr to $\Sigma(p)$ and changes nothing else but pc and μ ; if it is not, the step leaves certaddr alone, sets the error code to 1 and the error flag to true, and changes nothing else but pc and μ . In both cases c is not read and $\mu(\text{apply}(s, i)) = \mu(s) + d + 1$. (3) If $\text{certaddr}(s) = 0$, then $\text{certaddr}(\text{apply}(s, i)) \neq 0$ if and only if i is a MORPH_ASSERT whose arrow is in the graph of s and whose property has $\Sigma(p) \neq 0$.*

Proof. (1) is a case split over the other fifty instructions and their branches; each copies the address field, including the faulting branches, whose error writer changes only the error code. (2) is the two rules read off. (3): if i is not a MORPH_ASSERT, (1) keeps the channel at 0; if it is and the arrow is missing, (2) keeps it at 0; if the arrow is present, the new value is $\Sigma(p)$, which is nonzero exactly when the right side says so. $\square$

Checked as `non_morph_assert_preserves_cert_addr` and `morph_assert_step_sets_supra_cert_iff` (RevelationRequirement.v), `step_morph_assert_ok` and `step_morph_assert_bad` (VMStep.v), and `ascii_checksum` (VMState.v). The channel is overwritten, not added to: a later MORPH_ASSERT with $\Sigma(p) = 0$ on an existing arrow puts it back to zero. No step rule reads the channel or the flag.

There's no gap between paying and flipping. The rule that writes the flag or the channel is the same rule that adds the cost; it's one step, and there is no state in between where the reading has moved and the charge hasn't landed. A simulator that did these as two separate operations could get the order wrong; this machine has no order to get wrong.

Theorem 17.8 (No free certification on this machine). *(1) If $\text{certaddr}(s) = 0$ and $\text{certaddr}(\text{apply}(s, i)) \neq 0$, then $\text{cost}(i) \geq 1$. (2) If $\text{cert}(s)$ is false and $\text{cert}(\text{apply}(s, i))$ is true, then $\text{cost}(i) \geq 1$. (3) In either case, $\mu(\text{apply}(s, i)) \geq \mu(s) + 1$. (4) If $\text{certaddr}(s) = 0$ and $\text{certaddr}(\text{run}(t, s)) \neq 0$, then some i in t is in the setter class with $\text{cost}(i) \geq 1$, and $\mu(\text{run}(t, s)) \geq \mu(s) + 1$.*

Proof. (1): every instruction outside the setter class leaves the channel alone, so i is in the setter class, and each of its five members has a +1 in its cost. (2) is part (2) of Theorem 17.6. (3) is conservation with (1) or (2). (4): the channel is 0 at the start and nonzero at the end, so some step of the run takes it from 0 to nonzero; by (1) that step's instruction is a setter with cost at least one, and since the counter never falls, the end counter is at least one above the start. $\square$

Checked as `no_free_certification`, `no_free_certification_certifie`

d and `certification_requires_positive_mu` (`AbstractNoFI.v`), and `no_free_certified_insight` (`InsightTaxonomy.v`). Here `run` applies the list in order, as in Definition 17.3. Three neighbours say the same thing in other shapes. `thiele_nfi_pc_indexed` (`AbstractNoFI.v`) is the setter half of (4) for a run fetched by program counter with fuel: if such a run ends with the channel set from a clear start, the program contains a setter. `partition_refinement_nonfree` (`PartitionRefinementNoFI.v`) is (4) under a name that says what it is for: going from no certified claim to one is a strict refinement, and it isn't free. And `partition_free_but_certification_nonfree` (same file) packages the contrast of this section in one statement: PNEW, PSPLIT and PMERGE each cost zero at declared cost zero, none of the three is in the setter class, and any run that takes the channel from clear to set raises the counter by at least one.

One more fact the theorems lean on, about the per-arrow cost field of Section 16: there is a function that writes a value into that field of one arrow, and `graph_certify_morphism_lookup` (`VMState.v`) says that after it the arrow reads back with the new value and everything else the same. No step rule calls it, and every arrow a step creates starts with zero there, so on every state reachable from the initial state the field is zero. That last sentence is read from the rules; no theorem states it.

17.9 What the chip has

The hardware encodes forty-seven of the fifty-one. The four it doesn't are the four CHSH_LASSERT_1AB variants: they exist in the kernel and in the hardware-side abstraction function, but the hardware assigns them no opcode byte and the CPU has no reference to them. (The Python side does number them, 47 to 50, which is 0x2F to 0x32 in hexadecimal: the generated opcode table `thielecpu/generated/generated_core.py` and the assembler script `scripts/forgery.py`. Neither Coq nor the hardware gives them a byte.) The test that checks kernel-against-hardware parity, which compares the OCaml extraction's opcode names with the ones defined in the hardware's type file, allows *at most* four missing, with a comment naming these four; it does not check that it's exactly these four. I'd rather you know the test's shape than assume it's tighter than it is. Everything else in the table has a byte, and the bytes are Section 27's.

One rule for reading everything above: where a comment in the step file and the step rule next to it disagree, the book follows the rule, because the rule is what the proofs are about.

That's what you can do. Next: the arrows, which the table has treated as seven opcodes and which are, underneath, a piece of mathematics I didn't know existed when I built them.

18 What a morphism is

Category theory, from scratch. I didn't know what it was when I started this project; Section 2 and Section 1 have the story of building it by feel, as "digital DNA," and

finding out after the thing already ran that it had a name and a founding paper. So this section is me explaining the mathematics I accidentally used, with the two laws I made you wait for in Section 2, and then holding my own machine up against those laws on the arrows it actually stores, including the one case where the obvious rule for identities breaks them. That case is the most useful paragraph in here, and it's near the end.

18.1 Objects, arrows, and a quiet promotion

A category is the part of mathematics that takes the relationships between things as seriously as the things, and then quietly decides the relationships matter more. Two ingredients. Objects: the things. Arrows: the relationships between them. That's the entire inventory, and it looks thin until you notice what got promoted. In most of mathematics the objects are the citizens and the maps between them are second-class traffic. A category turns that around. The arrows are first-class, and nearly everything you can say inside a category you say by pointing at arrows, not by reaching inside objects. That's the Eilenberg and Mac Lane move from Section 2 (1945, "a continuation of Klein"): stop asking what a thing is, ask how it's carried.

The objects can be anything, which is both why the theory reaches so far and why it feels slippery the first time through. Sets. Vector spaces. Shapes. The theory doesn't care what the objects are made of; it cares how they connect. Here, the objects are *modules*, the named pieces a partition cut the state into (Section 16), and the arrows are claims about how two pieces relate. Why modules and not cells or registers? Because a claim about a relationship has to attach to something the machine can name and keep, and a module is the one thing in the state an arrow's two ends can be: you can read an arrow's source and target back (MORPH_GET), and with selector two, the number of pairs in its coupling.

Arrows also go by the name *morphisms*, and the words are interchangeable; I'll use both. An arrow runs from one object to another, written f from A to B . The thing everyone imports from school and has to set down is the assumption that an arrow is a *function*. It needn't be. An arrow is whatever the category accepts as a structure-respecting way to get from A to B , and here it's something more pointed than a function: a morphism is a *claim*, "these two modules are related, and here is the kind of relation," with the evidence stapled on.

18.2 The two laws, and why each

Section 2 gave you the definition in one breath. Here it is slowed down: two ingredients first, then the two rules Section 2 said you'd watch me have to prove. Together they're the whole definition, and the rules are the part people skip.

Composition. If there's an arrow f from A to B and an arrow g from B to C , there must be a composite arrow from A to C , written g after f . Why? Because relationships have to chain. A to B , B to C , therefore A to C . Picture three dots in a row, A , B and C : f

hops from the first to the second, g from the second to the third, and the composite takes the whole stride from A to C in one. Without that, every multi-hop path has to be named individually, and the structure collapses into a pile of unconnected pairs. A category without composition is a phone book without a search function. In the machine, COMPOSE builds the long arrow.

Identity. Every object has a do-nothing arrow to itself. Why bother? Because every object needs a self-loop, even the boring ones: a module with no connections still has to exist somewhere in the framework, and “the arrow that does nothing” is how you say “this thing, unchanged” in a language that only speaks in arrows. On the drawing it’s a small loop on A , leaving A and arriving back at A . MORPH_ID builds the loop.

And the two rules that make following arrows dependable, the ones Section 2 named in a sentence and left for here. *Associativity:* following three arrows in a row gives the same result whether you pair up the first two or the last two. Why? Because parenthesisation shouldn’t matter. Both spell the same three-hop path. If they came out different, two programs that built the same chain in different orders would produce two different morphisms, and that’s a bug: the chain is the thing, how you assembled it is not. *Identity laws:* tacking the do-nothing arrow onto either end of an arrow gives back the arrow. If the self-loop *changed* things, it wouldn’t be the self-loop.

That’s a category. Objects, arrows, composition, identities, associativity, identity laws. Six words and you’ve got the whole of Eilenberg and Mac Lane’s definition.

Definition 18.1 (Category). A *category* has a collection of objects; for each pair of objects A, B a collection of arrows $f : A \rightarrow B$; for each $f : A \rightarrow B$ and $g : B \rightarrow C$ a composite $g \circ f : A \rightarrow C$; and for each object A an identity $\text{id}_A : A \rightarrow A$. These satisfy associativity, $h \circ (g \circ f) = (h \circ g) \circ f$ whenever the composites are defined, and the identity laws, $f \circ \text{id}_A = f = \text{id}_B \circ f$ for every $f : A \rightarrow B$.

This is the textbook definition. No Coq file in the tree states it as a structure and proves the machine is an instance; what Coq checks is the list of laws below, each on its own.

18.3 What a coupling is

Now the evidence stapled to the claim. In the machine, a morphism carries *coupling data*: a list of pairs of cells, one from the source module’s region and one from the target’s, plus a label. A pair (a, c) says: cell a in the source is related to cell c in the target. That’s the *relational content* of the arrow, the actual record of which parts of one module correspond to which parts of another. What the pairs mean is up to the programmer: “ B refines A , and here’s which cells map to which”; “ A is correlated with B at these pairs”; “ A simulates B , these transitions correspond.” The kernel enforces the composition rule on the pairs. The interpretation is yours.

Composition on couplings is *relational composition*, the thing you learned without a

name when you learned that “a is a parent of b” and “b is a parent of c” gives “a is a grandparent of c”: a pair (a, c) is in the composite exactly when there’s some middle cell b with (a, b) in the first coupling and (b, c) in the second. That’s `relational_compose_spec`, and the machine’s kernel function is proved to be that function (`kernel_relational_compose_same`).

And here’s a thing I’d never have seen without the laws. The identity arrow in this machine doesn’t carry its identity in its coupling. The cheap guess for an identity coupling is the *empty* list, and the empty relation *annihilates* under composition: compose anything with “no pairs” and you get “no pairs” back, not the thing you started with (`relational_compose_empty_l`, `relational_compose_empty_r`). The coupling that *would* act as an identity is the diagonal, every cell of the region paired with itself, and that’s proved to work on both sides (`morph_id_left_coupling`, `morph_id_right_coupling`); but storing a diagonal means listing every cell of the region in every identity arrow, so the machine stores a *flag* on the record instead, and COMPOSE honours the flag: if both arrows are flagged, the composite is flagged too, and stored exactly the way MORPH_ID stores an identity; if only the first is flagged, the composite’s pairs are the second’s; if only the second, the first’s; else the relational composition (`graph_compose_morphisms_coupling`, four cases). The flag is load-bearing. That’s why MORPH_ID stores an empty coupling and sets the flag, and why MORPH_GET has a selector for the flag. (The lemma that says the identity arrow is an identity, `graph_add_identity_coupling`, proves only the flag. It doesn’t prove the coupling is diagonal, because it isn’t.)

Here are the objects exactly.

Definition 18.2 (Coupling, relational composition, diagonal). A *coupling* is a finite list of pairs of natural numbers. Its *relational composition* with a coupling *S* is

$$R; S = [(a, c) \mid (a, b) \in R, (b', c) \in S, b = b'],$$

listed in the order the pairs of *R* and then *S* are visited. The *diagonal* of a list *X* of cells is $[(x, x) \mid x \in X]$. Two couplings are *equivalent*, $R \equiv S$, when they contain the same pairs: $(a, c) \in R \iff (a, c) \in S$ for all *a, c*.

Definition 18.3 (Stored arrow and the composition rule). A stored arrow has a source module, a target module, a coupling with a label, an identity flag, and a cost field. Storing an arrow removes duplicate pairs from its coupling and sets its cost field to 0. MORPH_ID on an existing module *A* stores the arrow from *A* to *A* with the empty coupling (no pairs, label “empty”) and the flag on. COMPOSE of stored arrows *f* and *h*, when the target of *f* equals the source of *h*, stores a new arrow from the source of *f* to the target of *h*. If both are flagged, the new arrow is flagged and has the empty coupling, the same record MORPH_ID stores. Otherwise its flag is off, its label is the

label of f , a semicolon, and the label of h , and its pairs are

$$\begin{cases} \text{pairs of } h & \text{if only } f \text{ is flagged,} \\ \text{pairs of } f & \text{if only } h \text{ is flagged,} \\ \text{pairs of } f ; \text{ pairs of } h & \text{if neither is.} \end{cases}$$

If either arrow is missing or the target and source differ, COMPOSE builds nothing.

Checked as the definitions `relational_compose`, `diagonal`, `coupling_equiv` (`CategoryLaws.v`), and `empty_coupling_data`, `graph_add_morphism`, `graph_add_identity`, `graph_compose_morphisms` (`VMState.v`); `kernel_relational_compose_same` says the kernel’s copy of relational composition is the same function, and `graph_compose_morphisms_coupling` (`CategoryBridge.v`) says the stored composite’s flag is the two flags anded together and its pairs are equivalent to the four-case rule (equivalent rather than equal because storing removes duplicates).

18.4 What Coq checks, exactly

I promised in Section 2 you’d watch me prove the two rules. Here’s what’s proved, with the hypotheses, and you can decide whether it’s enough.

On *coupling lists*, with “the same” meaning “hold the same pairs, order and repeats ignored” (`coupling_equiv`): composition is associative (`relational_compose_assoc`, with nothing assumed; the kernel-record forms `coupling_data_compose_assoc`, `morph_compose_assoc_coupling`, `morph_graph_compose_assoc`, which are stated on the pair lists stored in the records, and none of which mentions the identity flag or the composite a COMPOSE actually builds). The diagonal is a left identity when every pair’s source lies in the region, and a right identity when every target does. Composition respects the equivalence on both sides. The empty list absorbs. And on the *tensor* side (two arrows side by side, pairs appended), there’s an interchange law when the cross terms vanish, and append is associative with the empty list as unit (`tensor_bifunctor`, `monoidal_coherence`).

On *the stored records*: what COMPOSE builds is the four-case rule above; what MORPH_ID builds is flagged identity; a new arrow gets the next identifier and old ones are untouched; a composite’s certification-cost field is zero. Associativity and both identity laws hold on the stored arrows themselves, flag and all, which is the next part’s whole job. And the hardware’s COMPOSE step produces exactly the state the kernel’s does, whole state, coupling and label included (`driven_step_compose`, under the hardware well-formedness invariant). That last one is closed, with no axiom behind it; the whole-machine version, every opcode under its side conditions (`driven_step_wf`), uses one standard extensionality axiom at the hardware layer, which Section 28 will account for.

The laws on couplings, exactly.

Theorem 18.1 (Laws of relational composition). *For all couplings R, S, T, R', S' and every list of cells X : (1) $(a, c) \in R; S$ if and only if there is b with $(a, b) \in R$ and $(b, c) \in S$. (2) $(R; S); T \equiv R; (S; T)$. (3) If every pair (a, b) of R has $a \in X$, then $\text{diag}(X); R \equiv R$; if every pair has $b \in X$, then $R; \text{diag}(X) \equiv R$. (4) If $R \equiv R'$ and $S \equiv S'$, then $R; S \equiv R'; S'$. (5) $[]; R \equiv []$ and $R; [] \equiv []$.*

Proof. (1) Unfold the definition: a pair of the output comes from some (a, b) in R and some (b', c) in S that passed the test $b = b'$, and every such match produces one. (2) By (1) on both sides, each says there are b and b' with $(a, b) \in R$, $(b, b') \in S$ and $(b', c) \in T$. (3) By (1), $(a, c) \in \text{diag}(X); R$ means $a \in X$ and $(a, c) \in R$, and the hypothesis makes the first part automatic; the right case is the mirror. (4) and (5) are (1) again. $\square$

Checked as `relational_compose_spec` (item (1), an if and only if), `relational_compose_assoc`, `relational_compose_diagonal_left`, `relational_compose_diagonal_right`, `relational_compose_compat`, `relational_compose_empty_l` and `relational_compose_empty_r` (`CategoryLaws.v`). The record-facing forms are the same statements about the pair lists stored in records: `coupling_data_compose_assoc` (`CategoryLaws.v`), and `morph_compose_assoc_coupling`, `morph_graph_compose_assoc`, `morph_id_left_coupling`, `morph_id_right_coupling` (`CategoryBridge.v`). `morph_graph_compose_assoc` takes three stored arrows whose targets and sources line up and concludes (2) for their stored pairs. None of these mentions the identity flag or the arrow `COMPOSE` actually stores; the laws on stored arrows are Theorem 18.3 below.

Theorem 18.2 (Tensor on couplings). *Let $R \otimes S$ be the list R followed by S . Then $(R \otimes S) \otimes T = R \otimes (S \otimes T)$, $[] \otimes R = R$ and $R \otimes [] = R$, as lists. And if $F; G' \equiv []$ and $G; F' \equiv []$, then $(F \otimes G); (F' \otimes G') \equiv (F; F') \otimes (G; G')$.*

Proof. The first three are facts about appending lists. For the last, by (1) of Theorem 18.1 a pair of the left side comes from one of four combinations, and the two cross combinations are empty by hypothesis. $\square$

Checked as `monoidal_coherence` and `tensor_bifunctor` (`Category-Monoidal.v`).

18.5 Where the laws meet the stored arrows

Now the paragraph I said was the most useful. The associativity theorem above is about *coupling lists*. The question that matters is whether the laws hold for the arrows `COMPOSE` actually stores, flag and all, and the flag hides a trap you can walk into with three arrows. Take two identity arrows on module A (two calls to `MORPH_ID` give two arrows, each flagged), call them $i1$ and $i2$, and an arrow f from A to B with a nonempty coupling P .

Here's the wrong neighbour first, because the border is drawn by both sides. The obvious rule for the flag is "a composite is never an identity; if the first arrow is flagged, take the second's pairs." Under that rule, i_1 composed with i_2 takes i_2 's pairs, the empty list, and gets built with the flag *off*. Compose that with f : neither is flagged now, so it's the relational composition of the empty list with P , which is empty. Compose the other way, i_1 with (i_2 with f): i_2 with f gives P , then i_1 with that gives P again. Empty on one side, P on the other. Under the obvious rule associativity breaks the moment two identities meet, the mirror case breaks too (an arrow g from B to A with pairs Q , followed by the composite of i_1 and i_2 , gives the empty list instead of Q), and the composite of two identities is an arrow from A to A that is an identity for nothing. A book that proved the laws on pair lists and let that stand on the records would be proving the laws somewhere the machine doesn't live.

The machine's rule passes the flag along: when both inputs are flagged, the composite is flagged, with the empty coupling, stored exactly the way MORPH_ID stores an identity (Definition 18.3). Run the same three arrows through it. i_1 composed with i_2 is a flagged identity on A . That composed with f : only the first is flagged, so take f 's pairs, P . i_1 with (i_2 with f): i_2 with f has pairs P and the flag off, i_1 is flagged, so P . Same both ways. And g followed by the composite of i_1 and i_2 gives Q , and so does g followed by i_1 , then i_2 . The flag is the whole difference.

"Same" needs pinning down, further than feels necessary, because a stored arrow carries more than the laws care about. COMPOSE always stores a *new* arrow under a fresh identifier, so " i_1 followed by f equals f " can't mean the same identifier; it means a new arrow that is f in every way the machine can act on. Its labels don't match either: the two bracketings above store "empty;" in front of f 's label once and twice. Labels are names for a person reading the graph, and MORPH_GET has no selector for one. And a flagged arrow's pairs are never read by COMPOSE, so there's nothing to compare there. So two stored arrows are *the same arrow* when they have the same source, the same target, the same flag, and, when the flag is off, the same pairs. With that, the laws hold on the stored arrows, with two side conditions that carry weight: the graph has to be well formed (so the arrows being composed are still where they were after each COMPOSE adds one), and an identity arrow has to go from an object to itself. MORPH_ID only builds those, and a separate invariant says nothing else ever gets the flag.

Definition 18.4 (The same stored arrow). Two stored arrows x and y are *the same arrow*, $x \cong y$, when they have the same source, the same target and the same flag, and, if the flag of x is off, their pairs are equivalent (Definition 18.2). Labels, identifiers and cost fields are not compared.

Checked as the definition `stored_arrow_equiv` (CategoryBridge.v).

Theorem 18.3 (The category laws on stored arrows). *Write xy for the arrow COMPOSE stores from stored arrows x followed by y , each COMPOSE run in the graph the previous one left. (1) If i and j are flagged and the target of i is the source of j , then COMPOSE of i and j succeeds and stores the arrow from the source of i to the target of j with the flag on, the empty coupling and cost field 0. (2) If the graph is well formed and f, k, h are stored arrows with the target of f the source of k and the target of k the source of h , then the four COMPOSEs that build $(fk)h$ and $f(kh)$ all succeed, and $(fk)h \cong f(kh)$. (3) If i is flagged, its source equals its target, and its target is the source of a stored arrow f , then COMPOSE of i and f succeeds and $if \cong f$. (4) If i is flagged, its source equals its target, and the target of a stored arrow f is the source of i , then COMPOSE of f and i succeeds and $fi \cong f$.*

Proof. COMPOSE of two stored arrows whose ends match always succeeds, stores the composite under the next arrow identifier, and leaves every other identifier where it was. (1) is the first case of the rule. (2) Well-formedness says every stored identifier is below the next one, so f and h are still found after the first COMPOSE on each side. Both composites go from the source of f to the target of h , and both carry the flag exactly when all three of f, k, h do. If the flag is off, at least one of the three is unflagged, and there are seven cases, one per pattern of flags; in each, item (1) of Theorem 18.1 turns both sides into the same membership condition. With none flagged it's associativity on pairs; with only k flagged both sides are the pairs of f composed with the pairs of h ; with two flagged both sides are the third arrow's pairs. (3) If f is flagged, so is if , and both go from the source of f to its target, so there is nothing more to compare. If not, if has the pairs of f with repeats removed, which the equivalence ignores. (4) is the mirror. $\square$

Checked as `graph_compose_identities_is_identity` for (1), `graph_compose_assoc_stored` for (2), `graph_compose_left_identity_stored` for (3) and `graph_compose_right_identity_stored` for (4) (`CategoryBridge.v`), with `graph_compose_stored` and `composite_record` underneath.

Theorem 18.4 (Every stored identity is the identity of its object). *Call a graph's identity arrows canonical when every flagged arrow in it has its source equal to its target and no pairs. (1) If a state's identity arrows are canonical and it steps on any instruction, the new state's are canonical. (2) If $\text{graph}(s)$ has no arrows and s' is reachable from s , then the identity arrows of $\text{graph}(s')$ are canonical.*

Proof. (1) A case split over the step relation. MORPH_ID stores an arrow from a module to itself with no pairs. COMPOSE stores a flagged arrow only when both inputs are flagged, and then from the source of the first to the target of the second, with no pairs; when both inputs are canonical, those four ends are one object. MORPH and MORPH_TENSOR store the flag off. Deleting an arrow, or the cascade when PSPLIT or PMERGE removes a module, only removes arrows. Every other step leaves

the arrow list alone. (2) A graph with no arrows has nothing to check, and (1) carries it along each step. $\square$

Checked as `identity_arrows_canonical` (the definition), `vm_step_preserve_s_identity_arrows_canonical` for (1), and `vm_reachable_preserves_identity_arrows_canonical` with `vm_reachable_identity_arrows_canonical` for (2) (`CategoryBridge.v`). The initial state has no arrows, so on every reachable state every flagged arrow meets the side condition of (3) and (4) in Theorem 18.3, and the identity laws hold for every identity arrow a program can build. The chip’s COMPOSE follows the same rule: two flagged arrows give a flagged arrow whose coupling descriptor holds no pairs and the label “empty” (Section 27 has the chip).

So the stored arrows obey associativity and both identity laws, up to the sameness of Definition 18.4. Calling the whole thing a category in the full sense would still take more: a Coq structure that packages the objects, the arrows and that sameness and proves the laws once as an instance, and an account of a category whose objects and arrows come and go as modules are split and merged. Neither is in the tree, and nothing above claims either. The laws are each a theorem on their own.

Three more things Coq doesn’t say, smaller. Nothing proves composition is defined on exactly the right pairs beyond “the first’s target equals the second’s source” by number; a mismatch gives the error branch. Nothing in the law files mentions that splitting or merging a module deletes every arrow touching it, so an arrow’s lifetime is bounded by its modules’; that’s in the step rules (Section 16) and not in the laws. And the per-arrow field the comment says records what was paid to certify it is never written (Section 16 again), so “a composite’s cost field is zero” is a theorem about a field that’s always zero.

18.6 Why this isn’t just extra state

Here’s the question I chewed on longest. If you can bolt extra registers onto an ordinary machine and stash arrow identifiers in them, why does the arrow layer matter?

Because extra registers have no provenance. You can write 42 into register 5 and call it an arrow. The machine checks: does arrow 42 exist in the graph, put there by a MORPH or a COMPOSE that actually ran? MORPH_ASSERT looks the arrow up and fails, setting the error flag, if it isn’t there. Existence is all it checks; it doesn’t compare the source and target against anything (you can read those back with MORPH_GET), it stores a checksum of the property string, overwriting whatever the channel held, and doesn’t check what the property says, and it never reads the certificate operand you hand it. And it costs at least one even when it fails. Building the chain, by MORPH and COMPOSE and MORPH_ID, is free at zero declared cost. Asserting it into the assertion channel never is. You can build for nothing; you can’t vouch for nothing. The graph is structure the machine holds outright, and Section 20 shows two states

that differ in nothing *but* that graph.

18.7 Two small facts that matter later

Failing arrow instructions still charge; the cost is on the attempt, like everything else. And MORPH reads its coupling *from memory*: a count word (capped at sixty-four, half the memory size, so a garbage count can't send the reader past the end of memory), then pairs, then a label, and it drops every pair whose ends don't lie in the two regions. With zeroed memory the count is zero and the coupling is empty, which matters for Section 21.

Definition 18.5 (Loading a coupling from memory). For a MORPH whose coupling block starts at word b , between modules with regions X and Y : let $k = \min(\text{mem}[b], 64)$, read the pairs $(\text{mem}[b+1], \text{mem}[b+2]), \dots, (\text{mem}[b+2k-1], \text{mem}[b+2k])$, read a length-prefixed label starting at word $b+1+2k$, and keep only the pairs (a, c) with $a \in X$ and $c \in Y$.

Checked as the definition `load_coupling_from_mem` (`VMState.v`), with the cap `serialized_coupling_pair_count` (64 is half the memory size). Two Python tests outside Coq exercise it with real data. One is the Python test `test_morph_loads_nonempty_coupling_from_memory` (`tests/test_categorical_opcodes.py`), which drives the extracted runner of Section 16. Its program writes the count 1 and the pair $(0, 1)$ into words 80 to 82, runs PNEW twice, once on cell 0 and once on cell 1, and then a MORPH from module 1 to module 2 with its coupling block at word 80; the test asserts that the error flag is down, that the first arrow's identifier is 1, and that its stored coupling is exactly $[(0, 1)]$. The other, `test_morph_reads_upper_half_memory` (`tests/test_rtl_morph_opcodes.py`), runs the hardware in co-simulation, puts the pair $(1, 2)$ at word 80, in the upper half of memory, makes modules on cells 1 and 2, and expects $[(1, 2)]$. These are tests, not theorems. The laws of Theorem 18.1 hold for every list, so they cover nonempty couplings like these; that the hardware's MORPH produces the same state as the kernel's `is_driven_step_morph` (`GraphReconstructionBridge.v`), under the hardware invariant and with both modules present and nonempty.

So the machine has modules, arrows between them, and pairs of cells stapled to the arrows, and Coq has checked the laws those pairs obey, on the lists and on the stored arrows, with the identity flag carried through the one case where the obvious rule would break them. What none of it has yet is anything a module can be said to *claim*. A coupling says which cells correspond. It doesn't say anything is true about them. That's next.

19 Claims a module can carry

Here's the gap Section 18 left open. Back in Part III, Section 7 ended by asking what the claim behind a paid flag even is, and Section 9 answered that insight, at its best,

is a relation committed to in one priced step (Definition 9.1 and the remark after it), talking about the claim as if it were a thing. On a sheet of paper it can't be; the paper has a NO box and a YES box and nothing to hang a claim on. The machine has plenty. Section 16 gave it a state record with a graph of modules in it, each owning a range of cells; Section 17 gave it fifty-one instructions, two of which turn a certification reading on; Section 18 gave it the arrows. Time to make the claim a thing. The buckets in Section 2 were a sorting of a pile into groups, and I promised the sorting would cost something, and then the whole middle of this book turned out to be about raising a flag, and you'd be within your rights to ask what the flag has to do with the buckets. Here's the link, and it's the oldest link in this book: a group is a thing you can *say something about*. "Everything in this bucket is red." "This region of memory holds a sorted list." A group with nothing said about it is a label, and labels are cheap because labels promise nothing. Calling a group "module A" commits you to exactly zero about what's inside it. Structure that earns the name starts one step later, when you hang a claim on a group and make the group answer for it.

19.1 A question with a definite answer

Two words, so I can say that cleanly, and both are in every logic textbook; I'm giving them to you because the book promised no unexplained jargon, not because they're hard.

A *predicate* states a precise condition on a thing. That gives the claim a meaning; it does not, by itself, give an algorithm for deciding it. "Is this number bigger than five" is a predicate about a number. "Does this region hold a sorted list" is a predicate about a region of memory. A predicate is the shape a claim takes once its truth conditions are precise. The finite formulas used here can be checked by the procedures below; that is an additional fact about these formulas. Notice what that rules out: "this code is well written" isn't a predicate until you say what you'd check.

A *formula* is a predicate built out of smaller yes-or-no pieces wired together with "and," "or," and "not." Take three switches, call them x_1 , x_2 , x_3 , each up (true) or down (false), and the formula " $(x_1 \text{ or } x_2) \text{ and } (\text{not } x_1 \text{ or } x_3)$." It comes out true exactly when at least one of the first pair is up and, at the same time, at least one of the second pair is. George Boole worked out in 1847 that you can do arithmetic on claims this way, treating "and" like multiplication and "or" like a kind of addition, and that's the whole reason a machine can check a claim at all: a formula is something you *evaluate*, by plugging in switch positions and turning the crank.

Do it. Set x_1 up, x_2 down, x_3 up. First clause: x_1 is up, so true. Second clause: not x_1 is false, but x_3 is up, so true. Both true, so the formula is true: that choice of switch positions is called a *satisfying assignment*, and you just found one. Now set x_3 down instead. Second clause: not x_1 is false, x_3 is false, so the clause is false, so the formula is false. That's a *falsifying assignment*. You've now done everything a checker does. The checker has no cleverness in it; it has a formula, a row of switch positions, and a

crank.

19.2 Hang it on a group

Now take a group of memory cells and say: “the values in this group make this formula true.” That’s a constraint hung on a group, and the formula can carry real content: every value here sits between zero and a hundred; this region encodes a satisfying assignment to the formula sitting over in that other region; this region is sorted. Tony Hoare, in 1969, built a whole way of reasoning about programs out of exactly this move: attach a claim to a *point* in a program (“whenever execution reaches here, this is true”), and reason forward from claim to claim. Robert Floyd had done it two years earlier on flowcharts. Their claims were checked on paper by the person writing the proof. The machine’s version is a formula and two rows in memory, checked by a step while it runs, and I’ll be straight about it: the check names no module and no region. The machine has the pieces. It doesn’t join them.

And here’s the thing I most want you to see, because it’s where “structure” and “cost” finally touch. A claim hung on a group *rules things out*. Before the constraint, the cells could hold anything. After it, they can only hold values that make the formula true. That’s the fifth item on Section 2’s list, “adding structure rules possibilities out,” made concrete: a formula on a region is a partition of all the ways the region could be filled into “allowed” and “not,” and the checked assignment is the proof that the allowed side isn’t empty.

19.3 A label is not a check

Now the part I’m going to belabour, because it’s the single most common way to fool yourself with a machine like this.

Section 16’s state record stores, next to each module, a list of strings it calls axioms. You’d think you could write “sorted” in a list like that and see what happens. Go looking for the step that reads that list and tests memory against it, and there isn’t one. Nor does any step write it: every module the steps create starts with an empty list and keeps it, and a merge or a split removes the modules it acts on, lists and all, and makes its new modules with empty lists. A function that appends to the list exists in the Coq model, and no step calls it. So the list is a field the machine carries and nothing more. It’s a label, and one a program can’t even write. Whatever you put in it would cost nothing and prove nothing, and you should hear that from the person who built the thing. Try it with a card: write 5, 2, 9 on it, and underneath write “sorted.” Hand it to someone and have them check each neighbouring pair. The second check fails. The label sat there the whole time.

Remark 19.1 (What the steps do with the label lists). Each module in the state carries a list of strings, its axiom list. Read off the definition of the step function: no rule reads any axiom list. A step that makes a new module gives it the empty list, and one asked for a range that is already a module’s returns that module unchanged. The steps that

split and merge modules delete the modules they act on and create their new modules with empty lists. Every other rule leaves the lists of existing modules as they were. The helper that appends to a list (`graph_add_axiom`, with `graph_record_discovery` built on it, both in `VMState.v`) is called by no rule. So along any run, every module's list is either the list it had at the start or empty. This is an inspection of the definitions (`vm_apply` in `SimulationProof.v`, `graph_pnew` in `VMState.v`, `graph_hw_psplit` and `graph_hw_pmerge` in `VMStep.v`), not a named theorem.

A *check* is different. The machine has one step whose whole job is to read a formula from memory, read two rows of switch positions from memory, and turn the crank. Section 17's table lists it as `LASSERT`, in one row; here's what the row leaves out. The formula is stored in a fixed layout: a word saying how long it is, a word for the number of switches, a word for the number of clauses, and then the clauses, each a list of signed numbers (plus for the switch, minus for "not the switch," zero to end the clause). The two rows sit back to back with each other, at a separate address that a second register holds: first a row that's supposed to satisfy the formula, then a row that's supposed to falsify it. (The step also carries a flag for which kind of certificate it's checking, and one setting of that flag always traps. I'm describing the setting that does the work.) The step checks both. If the first row doesn't satisfy, or the second row doesn't falsify, or the length written in the instruction doesn't match the length written in memory, the step *traps*: it jumps to a fixed error address, sets an error flag, and leaves memory, registers, and the modules exactly as they were. If both rows check, it goes on to the next instruction.

Two things about that step that I think are the right design and want to defend. First, it charges whether it passes or fails: the formula's length times eight, plus its declared cost, plus one, every time. You pay to check, not to be right; Popper again. (The eight is a pricing convention, not a byte count; the file says so.) Second, and this is why there are *two* rows: a formula that *every* assignment satisfies rules nothing out, so a satisfying row alone proves nothing about structure, and the falsifying row is the cheap proof that the formula isn't that kind of formula, that it's a real constraint with a real "not allowed" side. The source's name for what the second row stops is "tautology inflation." Try "x1 or not x1": write out both rows and you'll find no falsifying one exists, so the step would refuse it. And the empty formula, with no clauses at all, is the simplest tautology, and it's refused too.

Here is the check, exactly as the step runs it. In the symbols, $\vee$ is "or," $\wedge$ is "and," and $\neg$ is "not," the same three words the switches used.

Definition 19.1 (The formula check). Let m be data memory, f the value of the instruction's first register and b the value of its second. The formula words are $w_j = m(f + j)$ for $0 \leq j < 3 + m(f)$: w_0 is the length written in memory, $w_1 = v$ the number of switches, $w_2 = q$ the number of clauses, and from w_3 on come the literals, each read

as a signed 32-bit number, with 0 closing a clause. An assignment a maps switch numbers to words, and switch x is up when $a(x) \neq 0$. $\text{Sat}(w, a)$ scans the literals in order, keeping a count of closed clauses: it is true as soon as the count reaches q ; it is false if a clause closes with no literal made true by a (a positive literal x is true when x is up, a negative literal $-x$ when x is down), or if the words run out first. The satisfying row is $a_1(x) = m(b + x)$ and the falsifying row is $a_2(x) = m(b + v + x)$. For the checking step with kind flag k and declared length ℓ , the check *passes*, written *ok*, when $m(f) = \ell$, k is the satisfiable kind, $\text{Sat}(w, a_1)$ is true, and $\text{Sat}(w, a_2)$ is false.

Theorem 19.1 (What the check step does). *Let i be the checking step with kind k , declared length ℓ and declared cost c_0 , and let $s' = \text{apply}(s, i)$. Then s' has the same graph, registers, memory, certified flag and assertion channel as s , and $\mu(s') = \mu(s) + 8\ell + c_0 + 1$. If *ok*, the program counter advances by one and the error flag is unchanged. If not, the program counter becomes the fixed trap address 3840 and the error flag is set.*

Proof. This is the checking step's case of the step function, read field by field: *ok* is a pure function of s , and each field of s' is written as stated. The counter line is conservation for this instruction, whose price is $8\ell + c_0 + 1$. $\square$

Checked as the checking step's case of `vm_apply` (`SimulationProof.v`) and the rule `step_lassert` (`VMStep.v`), with the check as `lassert_exec_ok` and `lassert_check_ok` (`VMStep.v`) over `check_model_binary_fn` and `check_countermodel_binary_fn` (`CertCheck.v`). The pieces are named in `vm_apply_lassert_p` (`DagRestriction.v`) and `vm_apply_mu` (`MuLedgerConservation.v`); on a step that doesn't trap, the declared length equals the one in memory, `lassert_honest_cost` (`StateSpaceCounting.v`).

Remark 19.2 (Why a tautology cannot pass). Argued from Definition 19.1; there is no separate Coq theorem. If the check passes, the scan with a_1 reaches q closing zeros, so the words hold q complete clauses. The scan with a_2 sees the same words and the same closing zeros, so it can only fail at a clause that closes with no true literal: a_2 makes some clause false. Hence a formula that every assignment satisfies, such as $x_1 \vee \neg x_1$, never passes, and neither does the formula with $q = 0$, which the scan accepts for every row.

And third, which is subtle and easy to get wrong: *this step doesn't turn either reading on*. It checks, and it changes nothing it checked. Its evidence is in the run: the program counter kept going and the error flag stayed off. Turning a reading on is a *different* step, `MORPH_ASSERT`, which puts a property on an arrow (Section 9's "commit"), or `CERTIFY`, the bare step that raises the certified flag (Section 17 has both), and nothing forces either to be adjacent to the check. A flag raised by the bare step carries no check at all. Check and commit are two acts with two prices, and the machine, correctly I

think, refuses to pretend they're one. What that does to Section 9's test-then-commit protocol on this machine is Section 22's to show.

19.4 Who checks the checker

A fast checker that might be wrong just moves the trust somewhere else, and in 2005 Henk Barendregt and Freek Wiedijk gave this worry a name, the de Bruijn criterion, after the logician who insisted on it: a proof system is trustworthy when its proofs can be checked by a checker *small enough that a person can audit it*, independent of whatever clever tool produced the proof. Euclid had the instinct: five postulates you can hold in your head, and everything else meant to follow from them. The machine's checker is of that kind. In the chip I build at the end of the book it's a finite-state controller of about twenty states that walks the formula word by word. The proofs hold it up against the kernel (Section 15's name for the machine's reference definition in Coq, the step function every proof in this part is about), and they show its word-by-word scan returns the same answer as the kernel's two checks, as long as the formula declares at least one clause, its words hold an end-of-clause zero for every clause it declares, and every word fits in thirty-two bits. It also proves that the hardware step and the kernel step agree on the counter always, and on the program counter whenever the check passes; the same theorem pins the kernel's error flag to the check's outcome, unchanged on a pass and set on a fail. That's a fact about the hardware and it's Section 27's to show; I mention it here so you know the checker isn't a black box, and so you know where to find the trusted base, which is short and listed in Section 28.

The two hardware facts, stated so their premises are in view. Two words first: a *hardware snapshot* is the chip's whole state at one tick of its clock, and α reads off which kernel state that snapshot stands for. Section 27 builds both, and nothing else in this part of the book leans on them. (Yes, these are theorems about a chip I have never held in my hand. Section 27 is very upfront about that.)

Theorem 19.2 (The chip's scan is the kernel's check). *(1) Let the formula words be w_0, w_1, q followed by literal words u , and let g_1, g_2 be the two rows. If $1 \leq q$, u contains at least q zeros, and every word of u is below 2^{32} , then the hardware scan of u returns $\text{Sat}(w, g_1) \wedge \neg \text{Sat}(w, g_2)$. (2) For every hardware snapshot h and every checking instruction i , write $\alpha(h)$ for the kernel state the snapshot stands for. There is a kernel state s' that the kernel's step relation reaches from $\alpha(h)$ under i such that: $\mu(s')$ equals the hardware counter after the step; if ok holds at $\alpha(h)$, the hardware program counter after the step equals that of s' ; and the error flag of s' is unchanged when ok holds and set otherwise.*

Proof. (1) Induction on the literal words, with an invariant that ties the scanner's three bits (current clause met by the first row, current clause met by the second row, some earlier clause already missed by the second row) and its clause countdown to where the two kernel scans stand. (2) Composes three lemmas about the hardware model: the kernel step exists, the two counters agree, and the program counters agree on a

pass. Section 27 goes through both. □

Checked as `hw_scan_certcheck (LassertSpec.v)`, from `hw_scan_spec`, and `logic_engine_equivalent_lassert (LogicEngineEquivalence.v)`.

So the little checker in the chip and the check the proofs talk about give the same verdict, as long as the formula meets those three conditions, and the chip's counter moves the way the kernel's does.

19.5 Where, not just how much

Last piece, and it's the one that the counter in Section 6 can't give you. The counter is a total. Two different runs can end at the same total having checked different things in different places. "Did this run check the sorting constraint before it used the sorted list" is not a question the total can answer, any more than a grocery total of twenty-three dollars and forty cents can tell you whether you bought milk. The itemised receipt can. The run gives you the itemised receipt: which step ran, what it charged (the checked work reads a ledger off the run, one entry per step, and the entries add up to the counter's rise), and whether the error flag came on. A run somebody else hands you needs its own evidence before you trust that receipt.

And the rule I named No Free Insight back in Section 5 has a *where*. "Structural gain" is what I call it when some step of a run takes the assertion channel from clear to set, the machine-wide reading that says some label was asserted on some arrow. The shallow question is whether the channel was clear at the start and is set at the end, and you settle it by holding the first state next to the last. The machine isn't built for the shallow question. It's built for *which step*. Walk the run and find a step where the assertion channel went from clear to set; there's always at least one such step when the channel is set at the end and was clear at the start, and that step cost at least one, and the cost can sit nowhere else. In the checked work the located version is a predicate on runs, `structure_addition_in_run`, which says some step is such a step. The gain has a location. The location has a price. That's No Free Insight, said with the claim behind it finally in view: a formula in memory, checked by a small auditable checker, and a label asserted on an arrow by a step you can point at.

The located gain, exactly, with $\text{run}(n, p, s)$ the run of program p from s with fuel n (Definition 17.1) and $\text{addr}(s)$ the assertion channel, which Section 17 writes $\text{certaddr}(s)$.

Definition 19.2 (Structural gain, located). The run $\text{run}(n, p, s)$ has a *structural gain* when some step it executes goes from a state with $\text{addr} = 0$ to a state with $\text{addr} \neq 0$.

Theorem 19.3 (The gain has a place and a price). (1) If $\text{addr}(s) = 0$ and $\text{addr}(\text{run}(n, p, s)) \neq 0$, the run has a structural gain. (2) The run has a structural gain exactly when, at some step it executes from a state whose channel is 0, the instruction at the program counter is an assertion step that names an arrow in the current graph and carries a

label of nonzero checksum. (3) Each such step costs at least one. (4) If p contains no assertion step, no run of p has a structural gain.

Proof. (1) Induction on the fuel. At each executed step either the channel turns nonzero, which is a structural gain, or it is still zero and the rest of the run starts again from a clear channel; a run that stops with the channel still zero ends at zero, which contradicts the end. (2) From a state with channel 0, one step makes the channel nonzero exactly in the stated case, by part (3) of Theorem 17.7; then induction on the fuel. (3) The assertion step is priced at its declared cost plus one (Definition 17.4), and any step that takes the channel from 0 to nonzero costs at least one (Theorem 17.8). (4) follows from (2). $\square$

Checked as `supra_cert_implies_structure_addition_in_run`, `structure_addition_in_run_iff_morph_assert_bridge_pattern`, `morph_assert_step_sets_supra_cert_iff` and `non_morph_assert_trace_has_no_structure_addition` (RevelationRequirement.v), with the price from `no_free_certification` (AbstractNoFI.v).

Remark 19.3 (Clear at the start and set at the end is the stronger reading). The converse of (1) fails, argued from part (2) of Theorem 17.7 and not checked as a theorem: a run can set the channel and later run an assertion whose label has checksum 0, which writes 0 back. That run has a structural gain and ends with the channel clear. So the shallow question (first state against last) and the located one (which step) can come apart, and the files answer the located one.

One more boundary and then I'll stop. A satisfying row and a falsifying row prove that the formula can be met and isn't trivial. They don't prove the formula *means* what the label next to it says. "sorted" in the axiom list and a checked formula in memory are two things with no wire between them, and wiring them is a job for a proved encoding, which is a separate piece of work. The step checks the formula. The label is still a label.

So: what a flag could stand for is a formula, which the machine can check in one step and which a different step can mark with a label. Nothing in the machine joins the two, or either to a module. The check is separate from the commit, and the commit has a place in the run.

Notice where everything in this section lives. The formula and its two rows sit in memory, the labels sit on the modules, the arrows that get asserted sit in the graph. Memory an ordinary computer has. The graph it doesn't. Which raises a question I can't dodge: if somebody looks at this machine the way they'd look at an ordinary computer, reading registers, memory, the counter, the program counter, the error flag and the certified flag, and nothing else, can they even tell whether the graph is there?

Two states that look the same to that reader and differ in exactly the part that carries the claims: that's Section 20.

20 Two states that look the same

Section 7 said: a window that doesn't show a thing can't recover it, because two states that look the same through it have to get the same answer (Theorem 7.1, decoders need constant fibres). Section 7's own pairs were about the flag and its price, and they're Section 23's to show. This section's pair is about the graph. Here it is, and it's smaller than you'd think, and then a caveat that has to go with it, and then a second pair, built on a step, whose caveat is smaller.

20.1 The pair

Take two states. Both have an empty register file and empty memory, program counter zero, counter zero, error flag down, certified flag down, all eight witness counters zero, a sixteen-zero tensor, and *no modules at all*. They agree on every one of those. Both have the fresh-identifier counters at one. In the first state, the arrow list holds one arrow: source module zero, target module zero, empty coupling, identity flag on, cost field zero, and its own number is zero. In the second state, the arrow list is empty.

That's it. Six fields an ordinary observer reads (registers, memory, counter, program counter, error, certified): identical. The rest of the record: identical. One list with one entry versus one list with none. The theorem is `categorical_separation`: there exist two states that are *computationally equivalent* (agree on those six) and *categorically distinct* (different arrow lists). The proof is reflexivity six times and one `discriminate` on a list with an entry against a list without. It's as short as a Coq proof gets.

Definition 20.1 (Equivalent and distinct). Machine states s_1 and s_2 are *computationally equivalent* when they have equal registers, memory, counter μ , program counter, error flag and certified flag. They are *categorically distinct* when their arrow lists differ.

Definition 20.2 (The separation pair). Let s_A and s_B both have empty registers and memory, program counter 0, $\mu = 0$, error and certified flags false, all four control and status words 0, logic accumulator and mode word 0, all eight witness counters 0, the tensor of sixteen zeros, and a graph with no modules, next module number 1 and next arrow number 1. The arrow list of s_A is the single entry numbered 0: source 0, target 0, empty coupling with empty label, identity flag on, cost field 0. The arrow list of s_B is empty.

Theorem 20.1 (Categorical separation). *There are states s_1 and s_2 that are computationally equivalent and categorically distinct; s_A and s_B are such a pair.*

Proof. The six fields are equal field by field, by construction. The arrow lists are a list

with one entry and the empty list, which are different constructors. $\square$

Checked as `categorical_separation` (`PartitionSeparation.v`), with the definitions `computationally_equivalent` and `categorically_distinct` from the same file; the named copies of the two states are `separation_A` and `separation_B` (`ShadowProjection.v`).

From it, the corollaries you'd expect. Any observer whose verdict depends only on the six fields gives the same verdict on both (`classical_observer_cannot_separate`); any yes/no test of that kind can't tell them apart; any function that factors through the six-field projection is constant on the pair (`no_classical_separation`); and the difference survives a probe step, an ADD of nothing to nothing: after it the arrow lists still differ (`shadow_separation_theorem`, `shadow_strictly_lossy`). ADD is a classical instruction, one that writes a register and nothing in the graph, which is why it makes a boring probe. And the plain statement that two distinct states can have equal shadow is the last of the four parts of the degenerate-projection theorem; its parts two to four are the quotient theorem just below, and Section 23 says what its first part is.

Definition 20.3 (The shadow). The *shadow* of a machine state s is the six-field record $\text{sh}(s) = (\text{registers}, \text{memory}, \text{program counter}, \mu, \text{error flag}, \text{certified flag})$. It drops the graph, the control and status words, the witness counters, the mode word, the logic accumulator and the tensor.

Theorem 20.2 (The shadow separation theorem). *There are states s_1, s_2 and an instruction p with (1) $\text{sh}(s_1) = \text{sh}(s_2)$, (2) the arrow lists of s_1 and s_2 differ, and (3) the arrow lists of the states after one step by p from s_1 and from s_2 differ. Moreover, for every type T and every function f from states to T such that $\text{sh}(s) = \text{sh}(s')$ implies $f(s) = f(s')$, $f(s_A) = f(s_B)$.*

Proof. Take s_A, s_B and $p = \text{ADD}$ into register 0 from registers 0 and 0 at cost 0. (1) and (2) are Theorem 20.1. For (3), ADD changes registers, program counter and μ and leaves the graph as it was, so each graph after the step equals the graph before. The last part applies the hypothesis on f to (1). $\square$

Checked as `shadow_separation_theorem`, `shadow_strictly_lossy` (the same three facts with the shadow equality written out) and `no_classical_separation` (`ShadowProjection.v`); the yes/no version is `classical_bool_test_in_distinguishable` and the observer version `classical_observer_cannot_separate` (`PartitionSeparation.v`).

20.2 The window is a quotient

Here's another way to say what the pair shows. The shadow is a map that sorts every state into a bin, one bin per six-field record. Two states share a bin exactly when they

agree on the six fields; that's all a bin is. A window that reads only the shadow can only ever answer "which bin," so it's blind to everything that varies inside a bin. The pair says the graph varies inside a bin. That's what "the window forgets the graph" means, said precisely.

Theorem 20.3 (The shadow is a lossy quotient). (1) $\text{sh}(s_1) = \text{sh}(s_2)$ exactly when s_1 and s_2 agree on registers, memory, program counter, μ , error flag and certified flag. (2) Let s_1 and s_2 have equal shadows, equal graphs and equal control and status words. Then for every list of classical instructions run in order from each, the two final states have equal shadows. (3) There are distinct states with equal shadows.

Proof. (1) Two records are equal exactly when their fields are. (2) By induction on the list: each classical instruction computes the shadow, graph and control and status words of its result from the shadow, graph and control and status words of its input, so agreement on those three passes from step to step. (3) is s_A and s_B . $\square$

Checked as parts (2) to (4) of `degenerate_projection_theorem` (`TuringClassicalEmbedding.v`), which are `shadow_proj_kernel_is_eq_on_classical_shadow`, `classical_run_preserves_shadow_compat` and `shadow_strictly_lossy`. Part (2) needs the equal graphs and control and status words, and the file says why: `MORPH_GET` and `TENSOR_GET` are classical but read the graph, and the heap instructions read one of the control and status words, the heap base. So a classical run is a function of the bin together with the graph and those words, not of the bin alone.

20.3 The caveat

Now the caveat. The first state has an arrow whose source and target are module zero, and *there is no module zero*; the module list is empty. So the first state is not well-formed by Section 16's three clauses, and since every step preserves well-formedness, it's not reachable from the initial state. (There's a second reason, independent of the first: both states have an empty register file and an empty memory, and every reachable state has sixteen registers and a hundred and twenty-eight words.) The theorem is about the *type* of states, not about states a run can get to. The pair is as simple as a pair gets, as a type-level pair. As a claim about what a running machine can do, it says nothing.

Remark 20.1 (Why neither state of the pair is reachable). Let $Z(s)$ say that s has exactly 16 registers and exactly 128 memory words. No step turns Z from true to false, and the initial state has Z true, so every state reachable from it has Z true. Both s_A and s_B have empty registers and memory, so Z is false for both, and neither is reachable.

Checked as `eg_size_permanent` (`EventGeneralization.v`), with the initial state's sizes from its definition (`MuInitiality.v`). The other reason in the text, the arrow whose ends name a module that doesn't exist, rests on Section 16's well-formedness clauses

and is not restated here.

Does that gut it? No, and here's why not: the type-level pair is enough for the *shape* of the claim (the projection is many-to-one on the type, full stop), and a second pair exists below, built on a step from the initial state, with no caveat of that kind. But it's a real distinction and I'd rather you have it than find it.

20.4 A step the shadow can't see

Here's a pair built on a step, and it's the strictness theorem. Start where every run starts: the initial state of Definition 16.7, with no modules and the next module number at zero. Take one step: create a module on address zero (PNEW on the single cell zero, which is inside memory, owned by nobody, with module numbers to spare, so no trap). The fresh-module counter goes from zero to one. Now consider every *classical* run from the same start: any sequence of steps, each by one of the twenty-nine opcodes the source calls classical (Section 17 lists them in Definition 17.3: arithmetic, loads, stores, jumps, the heap, the ports, and MORPH_GET and TENSOR_GET, which read the graph and write nothing), in any order the program's jumps pick. None of them touches the graph, so after any such run the fresh-module counter is still zero. One structural step changed a thing no classical run can change.

And it isn't a trick of the starting point. Stop a classical run anywhere, at any state it reaches from the initial state, and the same PNEW is a step from there too, raising the counter by one, while every classical run from there keeps it put. The state PNEW lands in has a graph that no classical run from the initial state ever has. Nothing here is hand-built: every state in the theorem is one the machine reaches.

Definition 20.4 (Classical reachability). A program is *classical* when each of its instructions is one of the twenty-nine classical opcodes. A state s' is *classically reachable* from s when a chain of zero or more steps leads from s to s' and each step is by a classical instruction; the steps can come in any order, so a run whose jumps revisit instructions counts. Running a list means applying the step to each instruction in order; the program-counter runner with fuel F fetches the instruction at the program counter F times, following jumps.

Theorem 20.4 (Strictness, and with it conservativity). *Let p be PNEW on the region $[0]$ at cost 0. (1) For every state s classically reachable from the initial state s_0 (with $s = s_0$ allowed): s is reachable from s_0 ; p is a step from s ; that step raises the next module number by one and leaves the error flag as it was; every state classically reachable from s has the next module number of s , and so does the state the program-counter runner reaches from s on every classical program with any fuel; and the graph after p differs from the graph of every state classically reachable from s_0 . (2) At s_0 itself: after p the next module number is 1, and every classical program run by the program-counter runner from s_0 , with any fuel, ends with next module number 0. (3) For every state s and every classical program, running it from s as a list, or*

under the program-counter runner with any fuel, leaves the graph, the assertion channel and the certified flag as they were in s ; so does every classically reachable state.

Proof. (3) By induction on the list, on the fuel, or on the chain of steps: the program-counter runner either stops or fetches an instruction of the program, which is classical, and no classical instruction writes the graph, the assertion channel or the certified flag. (1) A state classically reachable from s_0 therefore has s_0 's graph: no modules and next module number 0. Memory has a hundred and twenty-eight words in every reachable state, so cell 0 is inside it, no module overlaps it, and a module number is free, so PNEW does not trap and adds a module, raising the next module number by one. Reachability holds because each classical step is a step. Every state classically reachable from s_0 has next module number 0, and the graph after p has 1, so the graphs differ. (2) is (1) at $s = s_0$ with the program-counter half of (3). $\square$

Checked as `pnew_step_separates_thiele_from_classical_reachable` (TuringStrictness.v; the step is separating_pnew_step), `pnew_step_separates_thiele_from_classical_at_init` for (2), and `pnew_step_separates_thiele_from_classical`, the existential form with the initial state as witness. For (3): `classical_opcodes_preserve_structure` (lists), `classical_opcodes_preserve_structure_run_vm` (the program-counter runner) and `classical_reachable_preserves_structure` (chains of steps), all in `ClassicalConservativity.v`, with `classical_reachable_vm_reachable` for reachability. `thiele_strictly_extends_classical` states (3) for both runners together with the separation of (1).

The standalone copy of the kernel has its own witness for the strictness shape, and that one is on a hand-built base: the step is MORPH_ID on module zero and the probe is MORPH_DELETE of arrow zero; after the identity arrow is built the probe succeeds, and after any classical run it fails (`morph_delete_probe_separates_thiele_from_classical_tc`, `ThieleMachineComplete.v`, `base_probe_base_state_tc`). Same shape, different instruction, and the reachable version above is the one I lean on.

20.5 Three things the windows drop, each on its own

The graph isn't the only thing riding along beside the ordinary fields. There are three: the counter, the certified flag, and the graph. Each one can differ between two states while everything else you'd look at holds still, including the other two. Five pairs of states show it, one per claim. Four of them are one instruction away from a clean start, with CERTIFY on at least one side. The fifth is the separation pair from the start of this section.

Write Z_0 for the clean start: everything empty or zero, no modules, no arrows. It is a hand-built state, not the machine's initial state.

Theorem 20.5 (Each of the three is independent of the others). *None of the following*

functions exists. (1) A function from (memory, registers, program counter) to μ , correct on every state. (2) The same, to the certified flag. (3) A function from (memory, registers, program counter, μ) to the certified flag. (4) A function from (memory, registers, program counter, certified flag) to μ . (5) A function from (memory, registers, program counter, μ , certified flag) to the graph.

Proof. Each is the same argument: two states with equal inputs and different outputs would force the function to give two answers on one input. The pairs, all but the last one step from Z_0 : (1) and (2) CERTIFY at declared cost 0 ($\mu = 1$, flag on) against PNEW of the empty region at cost 0 ($\mu = 0$, flag off), both at program counter 1 with memory and registers untouched; (3) CERTIFY at declared cost 2 against PNEW of the empty region at cost 3, both with $\mu = 3$, flag on against flag off; (4) CERTIFY at declared cost 0 against CERTIFY at declared cost 1, both flagged, $\mu = 1$ against $\mu = 2$; (5) s_A and s_B . $\square$

Checked as `thiele_state_three_component_independence` (NecessityAbstract.v), the first four parts being `mu_ledger_mutual_independence` and the fifth `graph_not_recoverable_from_P_full`. In this kernel version the PNEW witnesses add a module, so the graph differs inside pairs (1) to (3), and the graph is not among the inputs held fixed there. The standalone copy of the machine has the stronger form, with the graph added to the inputs of (1) to (4): the PNEW witnesses are replaced by JUMP to 1 at cost 0 and at cost 3, which move only the program counter and μ (`mu_ledger_full_pairwise_independence`, with the records `FullStrictShadow`, `FullCostShadow` and `FullCertShadow`, `ThieleMachineComplete.v`). The agreement of that copy with the kernel is tested, not proved.

Two things to keep beside this. First, it's a fact about the certified flag, not about any flag you might put in its place. Replace the flag with an arbitrary reading that, once true, stays true and that some step turns on, and the independence can fail: "the registers and memory have their full sizes" is such a reading, and it's a function of (memory, registers, program counter). That's checked as `mutual_independence_not_event_generic` and `three_component_indep_not_event_generic` (EventGeneralization.v). Second, the step that sets the flag is itself a place where states run together: CERTIFY sends a state with the flag off and the same state with the flag on to one and the same result, so the step by CERTIFY is not one-to-one (`vm_certify_merges`, `FiniteCertMachine.v`). That's Section 4's room of chairs on the real machine (Theorem 4.1, a permanent flip merges).

The same shape answers a narrower question: which of these fields could a description that sees only the ordinary part of the state recover? None of the three that matter here. Write $fg(s)$ for (program counter, μ , registers, memory) and $bare(s)$ for (program counter, registers, memory).

Theorem 20.6 (Three fields no bare window recovers). (1) No function φ of $\text{fg}(s)$ equals the certified flag of s for every s . (2) No function of $\text{bare}(s)$ equals μ for every s . (3) No function of $\text{fg}(s)$ equals the assertion channel of s for every s . Equivalently, for each given φ the corresponding equation fails at some state.

Proof. Each uses two states that agree on the window and differ on the field. All six have program counter 10, registers $[42, 0, 0, 0]$ and memory $[0, 0, 0, 0]$. (1) $\mu = 5$, certified against uncertified. (2) $\mu = 0$ against $\mu = 1$. (3) $\mu = 5$, assertion channel 0 against 1. A φ would have to give both values on one input. $\square$

Checked as `cert_not_function_of_forget`, `mu_not_function_of_bare_observable` and `cert_addr_not_function_of_forget`, with the per- φ forms `no_classical_a2_cert_predicate`, `no_classical_mu_separation_predicate` and `no_classical_cert_addr_predicate` (`ProjectionNonExistence.v`); the witnesses are `trace_witness_A` and `forget_witness_uncert`, `mu_separation_witness_zero` and `mu_separation_witness_one`, and `cert_addr_witness_zero` and `cert_addr_witness_one`. The names are bigger than the statements. The first says the toll's flag can't be read off the bare window; it doesn't say the toll can't be stated on a Turing machine (it can: Section 3's certification system, Definition 3.1, is exactly that). The third is about the assertion channel; the property the diagonal uses on this machine (Section 24) is a different one, bounded-run equality with a reference program, and the file's comment linking the two overstates it.

20.6 What the pair doesn't prove

The obvious move is to reach for more room: bolt on a register, a tape region, any extra slot, and surely an ordinary machine wriggles out. By hand, with no Coq statement behind it, it doesn't: the same collision rebuilds one size up, with the added slot dropped from what the observer reads. That cuts both ways. The pair also says nothing about whether an ordinary machine can hold the graph. It can, encoded on tape. A projection that sends different graphs to the same place can't give the graph back on every state, and that's all it says. It doesn't rule out an encoding that keeps the whole state, and it doesn't make these particular opcodes necessary in every model. What a faithful simulation owes the counter is a question about the simulator's own step rule, and the next subsection is about that.

20.7 The machine versus the program it runs

One more theorem belongs here, because it says what the pair *means* for the rest of the book. Section 3's universal floor was stated for any certification system. Now take any such system that *simulates* my machine: it has a decoding of its instructions into mine, an embedding of its states into mine, its step commutes with mine through the embedding, and its certification bit reads my certified flag through the embedding. Then two things hold for any run of the base system from uncertified to certified: it paid at least one (that's just the universal floor), and the *embedded* run on my machine

ends certified (`thiele_represents_simulating_cert_system`). My machine itself is such a system, with the identity as the embedding (`thiele_self_simulating`), and that's the only such construction in the tree apart from some generalised variants.

Why does that matter? Because it separates two things people run together: *the machine doing the stepping* (what the machine is) and *the program it runs* (what runs on it). The floor is a property of the machine doing the stepping, and it transfers to anything that faithfully runs on that machine. The pair in this section is a fact about the machine's state space, not about any program.

A fixed machine can enforce an invariant in its own transition table, and that includes a Turing machine. Give its tape a ledger track and a certification cell, and write a table that never sets the cell without bumping the ledger: that machine obeys the toll in its step relation. It's an instance of the abstract certification system, not a counterexample to it. What it doesn't do is enforce the toll on the programs it runs. A universal machine simulating another carries the simulated certification bit as data on its tape, and a buggy guest can flip that bit without paying, because the host's step has no opinion about it. My machine is in exactly that position when it simulates something else. The kernel's theorems are about the designated reading of the machine doing the stepping. That's a statement about which reading a step rule prices. It isn't a claim that Turing machines can't price one. (The floor for a certification system with no embedding into mine at all is `universal_nfi_any_substrate`, and the assumption receipt for `thiele_represents_simulating_cert_system` is empty.)

The smallest checked version of “a ledger beside an ordinary machine” is a Turing machine configuration with a counter next to it, where each step adds one, plus one more if it writes a nonblank symbol. The counter doesn't disturb the machine.

Theorem 20.7 (A counter beside a Turing machine changes nothing). *Let a Turing machine with transition table δ step a configuration c to $\text{tm}_n(\delta, c)$ after at most n steps (stopping when δ has no entry). Pair each configuration with a counter, and step the pair by stepping the configuration and adding the step's charge. Then for every n , δ and pair (c, m) , the configuration part after at most n steps of the pair is $\text{tm}_n(\delta, c)$. In particular this holds from $(c, 0)$.*

Proof. By induction on n . For $n = 0$ both sides are c . For $n + 1$, if δ has no entry at c , both runs stop and both sides are c . Otherwise the pair steps to (c', m') where c' is exactly the Turing machine's next configuration, whatever m' is, and the induction hypothesis at (c', m') finishes it. $\square$

Checked as `thiele_simulates_turing_gen` and `thiele_simulates_turing` (ProperSubsumption.v). The “Thiele” in those names is a local record, a Turing

configuration and a number; it is not the machine of this book, and the file says not to cite it as the machine simulating Turing machines.

And one honesty: the simulating-system record has a cost-preservation field (the base charges at least what my machine would), and the theorem never uses it. It's there for a stronger statement that hasn't been made.

Here it is exactly. A certification system is Section 3's, Definition 3.1: states, moves, a step, a cost per move, a certified bit, and the toll as a field.

Definition 20.5 (A system that simulates the machine). A *simulating system* is a certification system $\mathcal{B}$ together with a decoding d from its moves to machine instructions and an embedding e from its states to machine states, satisfying three premises: the step commutes, $e(\text{step}_{\mathcal{B}}(s, i)) = \text{apply}(e(s), d(i))$ for all s, i ; cost is preserved upward, $\text{cost}_{\mathcal{B}}(i) \geq \text{cost}(d(i))$ for all i ; and the certified bit is reflected, $\text{cert}_{\mathcal{B}}(s) = \text{the certified flag of } e(s)$ for all s .

Theorem 20.8 (The machine represents any system that simulates it). Let $(\mathcal{B}, d, e)$ be a simulating system, s_0 a state of $\mathcal{B}$ and t a list of its moves. If s_0 is uncertified and the run of t from s_0 ends certified, then the total cost of t in $\mathcal{B}$ is at least 1, and the machine run of the decoded list $d(t)$ from $e(s_0)$ ends with the certified flag on.

Proof. The first part is the universal floor (Theorem 3.1) applied to $\mathcal{B}$ by itself. For the second, by induction on t using step commutation, the embedding of $\mathcal{B}$'s final state is the machine's final state on $d(t)$ from $e(s_0)$; reflection turns " $\mathcal{B}$'s final state is certified" into "that machine state has the flag on". Cost preservation is not used. $\square$

Checked as `thiele_represents_simulating_cert_system (UniversalCertificationCost.v)`, with the record `SimulatingCertificationSystem` (its fields `scs_step_commutes`, `scs_cost_preserved`, `scs_cert_reflects`), the run lemma `scs_run_embed`, and the identity instance `thiele_self_simulating`. The floor for a system with no embedding at all is `universal_nfi_any_substrate`.

One limit on all of this, so it isn't read as more. Certification is one event. Other kinds of observable a machine might carry would each need their own event, their own evidence and their own cost law, and nothing here supplies those. And stacking several such predicates side by side gives a list of bits and nothing more: no topology, no metric, and no independence between them beyond what Theorem 20.5 proves for its three named fields.

So: two states, same shadow, different graph, at the type level in two lines and from the initial state in one step. The graph is the thing the ordinary windows drop. Next, what a run that uses the graph looks like on a receipt, in four acts.

21 The receipt in four acts

Four short acts with one receipt. They show the difference between an arrow that lives in the graph and a register that holds its number, and they show the trace recording what an assertion attempt cost, even when it fails. None of this vouches for a receipt somebody else hands you, and none of it makes a property label true. I'll say what each act costs, and under each act I'll name what's checked, because the four acts *together* are a script that runs (a small Python demonstration that asserts its own numbers as it goes, and a test that runs it end to end), and the individual facts are the theorems.

21.1 Act 1: the attempt costs, even when it fails

Run the assertion step on an arrow number that doesn't exist. The demonstration does it on a machine that has built no arrows at all, asserting arrow ninety-nine with the property "two-hop" and declared cost zero. Any arrow number at or above the arrow counter is absent, because well-formedness (Section 16) puts every arrow's number below the counter. What happens: the error flag latches on, because the lookup failed; the assertion channel stays at zero, because failure doesn't activate it; and the counter goes up by one anyway, because the declared cost was zero and the floor is one.

What's checked: the failing branch of the assertion rule sets the error code, latches the flag, steps the program counter and charges the cost, and changes nothing else (`step_morph_assert_bad`); the counter rises by at least one on any assertion, any state, any argument (`morph_assert_mu_pos`); the exact cost is declared cost plus one (`morph_cost_morph_assert`); and from a clear channel, the channel becomes nonzero *if and only if* the instruction is an assertion, the arrow exists, and the checksum is nonzero (`morph_assert_step_sets_supra_cert_iff`), so a missing arrow leaves it clear. The machine charges for the failed existence check. That's what the assertion guarantees and all it guarantees: it checks the arrow exists, stores a checksum of the property string, and leaves the certificate operand unread.

Exactly, with $i = \text{MORPH_ASSERT}$ on arrow k with property p , certificate c and declared cost d :

Theorem 21.1 (A failed assertion still pays). (1) *If the graph of s has no arrow numbered k , then $\text{apply}(s, i)$ agrees with s on every field except four: pc becomes $\text{pc}(s) + 1$, μ becomes $\mu(s) + d + 1$, the error code becomes 1, and the error flag becomes true. In particular certaddr and the graph are unchanged.* (2) *For every state s , $\text{cost}(i) = d + 1$ and $\mu(\text{apply}(s, i)) \geq \mu(s) + 1$.*

Proof. (1) is the failure rule of MORPH_ASSERT, read through the bridge (Lemma 17.1): it builds the next state from s with the error writer, which changes only the error code, and the latch, which sets the flag. (2): the cost is the definition (Definition 17.4), and the counter claim is conservation (Theorem 16.1) with that cost. $\square$

Checked as `step_morph_assert_bad` (VMStep.v), `morph_cost_morph_assert` (CategoryBridge.v) and `morph_assert_mu_pos` (InsightTaxonomy.v). The if-and-only-if is part (3) of Theorem 17.7, `morph_assert_step_sets_supra_cert_iff`. That an arrow number at or above the counter is absent is read from the second clause of well-formedness (Definition 16.5); no lemma states it on its own, and the demonstration's machine has no arrows at all, so its case needs nothing.

21.2 Act 2: build the chain

Run the demonstration's program. Three PNEWs, each claiming a cell of its own (cells one, two and three, declared cost one each), which make modules A, B and C. Under Section 16's rules overlapping ranges trap, and so do ranges past memory, so the three claims have to be different ranges inside the hundred and twenty-eight words, and they are. Then MORPH *f* from A to B and MORPH *g* from B to C, declared cost two each. Then COMPOSE *h* from *f* and *g*, declared cost one. Then MORPH_GET into register zero the source of *h* and into register one its target, declared cost zero each. The modules are numbered one, two and three because the runner the demonstration drives starts its module counter at one; in the kernel's own initial state they would be zero, one and two (Section 16). The arrows are numbered one, two and three either way, since both start the arrow counter at one.

When it finishes, register zero holds 1, which is A, and register one holds 3, which is C. Registers ten, eleven and twelve hold the three arrow numbers, and *h*, arrow three, from A to C, sits in the graph. The counter reads eight: 1 + 1 + 1 + 2 + 2 + 1, and nothing for the two MORPH_GETs. The demonstration asserts all of that. Had every declared cost been zero, the counter would read zero; each of these instructions charges what it declares.

One detail from Section 18 that matters here: MORPH reads its coupling from memory, a count word then pairs. With zeroed memory the count is zero, so *f* and *g* have *empty* couplings, and if you MORPH_GET the pair count of *h* you get zero. The chain is built; the relational content is empty, because nobody wrote any. That's the honest state of Act 2 on a blank machine. If you want pairs, write a coupling block first.

What's checked: the success branches of MORPH, COMPOSE, MORPH_GET and their selector function; each costs its declared cost (the six cost lemmas in the bridge file); arrow construction can be free at zero (`morph_can_be_free`, `morph_compose_cost_zero`); and a run that contains none of the five instructions in the setter class (LASSERT, LJOIN, EMIT, REVEAL, MORPH_ASSERT) leaves the assertion channel exactly where it was (`structural_trace_preserves_cert_addr`). No named theorem gives the register values or the total for this exact program: the total is the sum of per-step costs the lemmas fix, and the register values come from running it.

Here is the channel fact Act 2 rests on, exactly. Write $\text{run}(t, s)$ for applying a list *t* of instructions to *s* in order (Definition 17.3).

Theorem 21.2 (Building leaves the channel alone). *Let t be a list of instructions none of which is in the setter class (`LASSERT`, `LJOIN`, `EMIT`, `REVEAL`, `MORPH_ASSERT`), and s any state. Then $\text{certaddr}(\text{run}(t, s)) = \text{certaddr}(s)$.*

Proof. For one step: every instruction outside the setter class copies the address field on every branch, by a case split that closes by computation. Then induction on t . $\square$

Checked as `structural_trace_preserves_cert_addr` (`InsightTaxonomy.v`), on top of `thiele_non_cert_addr_setter_preserves` (`AbstractNoFI.v`). Act 2's program uses `PNEW`, `MORPH`, `COMPOSE` and `MORPH_GET`, none of them setters. The theorem is about applying a list in order; the demonstration's runner fetches by program counter, and for a straight-line program with no jump and no trap the two orders are the same. That step is read from the program, not checked. The costs are fixed one instruction at a time: `MORPH`, `COMPOSE`, `MORPH_ID`, `MORPH_DELETE`, `MORPH_TENSOR` and `MORPH_GET` each cost exactly their declared d (`morph_cost_morph` and its five siblings, `CategoryBridge.v`), and so does `PNEW`, by the definition of the schedule.

21.3 Act 3: assert

Run the same program with one more instruction after the `COMPOSE`: the assertion on h , with the property “A-to-C-two-hop”, a certificate string the step never reads, and declared cost four. It succeeds, because h exists. The channel moves to the checksum of the property, which is the sum of its character codes, nonzero. The counter goes up by five, four declared plus the floor, and reads thirteen. The demonstration also runs a smaller probe to show the floor on its own: one `PNEW` at cost one, a `MORPH_ID` at cost zero, and an assertion at declared cost zero. The assertion still charges one, and the counter reads two.

Why does the property string need a non-NUL byte? (NUL is the character whose code is zero.) Because the channel stores a plain byte sum. The empty string sums to zero, and so would an all-NUL string, leaving the channel at zero: the step succeeds and nothing was activated. Any string with a non-NUL byte works. And because the step writes the checksum unconditionally, an assertion whose property sums to zero, made after a good one, writes the channel back to zero. The channel holds the fingerprint of the latest assertion, not a running record of all of them. “A-to-C-two-hop” is just a label, and I'd ask you to hear “just a label” with Section 19 in your ear: the step doesn't check that h is a two-hop path. It checks that h exists.

What's checked: the success branch (`step_morph_assert_ok`); the if-and-only-if above; and the machine's form of Section 9's point that certified insight isn't free (Remark 9.1): any flip of either reading (the channel or the flag) from off to on costs at least one and raises the counter by at least one (`certified_insight_nonfree`; parts (1) to (3) of Section 17's Theorem 17.8, which Section 22 files under Section 9's tiers). And one thing it does *not* do, which the demonstration prints out so nobody

misses it: the assertion sets the *assertion channel*, not the certified *flag*. Those are two different readings set by two different instructions (Section 17), and this act sets one of them.

Theorem 21.3 (A successful assertion). (1) If the graph of s has an arrow numbered k , then $\text{apply}(s, i)$ agrees with s on every field except three: pc becomes $\text{pc}(s) + 1$, μ becomes $\mu(s) + d + 1$, and certaddr becomes $\Sigma(p)$, the sum of the character codes of p . The certified flag is not touched. (2) For any state s and any instruction j : if certaddr goes from 0 to nonzero, or the certified flag goes from false to true, across the step $\text{apply}(s, j)$, then $\text{cost}(j) \geq 1$ and $\mu(\text{apply}(s, j)) \geq \mu(s) + 1$.

Proof. (1) is the success rule read through the bridge. (2) is parts (1) to (3) of Theorem 17.8, put together as one statement about either reading. $\square$

Checked as `step_morph_assert_ok` (VMStep.v) and `certified_insight_nonfree` (InsightTaxonomy.v). With Act 3's numbers: $8 + 4 + 1 = 13$, and $\Sigma(\text{A-to-C-two-hop})$ is nonzero because the string has a non-NUL character.

The trace now records a successful existence check and what it cost. The counter can't go down. The total by itself doesn't say which assertion happened or where; for that you need the trace (Section 19's grocery receipt). And keeping honest books is a different job from catching an adversary who hands you a different history (Section 7's escapes).

21.4 Act 4: equal observations can hide different graphs

Two programs and one probe. Program A is Act 2's program: the three PNEWs, the two MORPHs, the COMPOSE and the two MORPH_GETs (the demonstration appends a HALT, which the runner stops at). When it ends, register zero holds 1, register one holds 3, the counter reads eight, the error flag is down, and the graph holds three arrows.

Program B never runs MORPH or COMPOSE. It makes three modules the same way, claiming the same three cells, but declares their costs as two each, six in all. Then it loads the two values straight into registers with `LOAD_IMM`, register zero gets 1 and register one gets 3, at declared cost one each. Six and one and one make eight, so the counter matches. B's graph has three modules and no arrows, because PNEW and `LOAD_IMM` never add an arrow. Its registers hold the numbers one and three, a claimed source and a claimed target, and the numbers are tokens, not graph entries.

The demonstration compares four values: register zero, register one, the counter and the error flag. For A they are 1, 3, 8, down. For B they are 1, 3, 8, down. Same. That's a fingerprint, and a weaker one than Section 20's pair, which agrees on all six shadow fields. B is a different program from A in other ways, and the demonstration doesn't

look at them: registers ten, eleven and twelve hold the arrow numbers in A and zero in B, and B is shorter, five instructions against A's eight.

Now the probe. Append MORPH_DELETE of arrow one, declared cost zero, to each program. In A, arrow one is f, the first MORPH built, so the delete succeeds. In B there is no arrow one, and the delete fails and latches the error. The demonstration asserts both outcomes. MORPH_DELETE reads the graph, and the graph is the one thing a LOAD_IMM can't forge.

What's checked, and here I have to be careful, because it's tempting to point at Section 20's separation theorem as if it covered these two programs, and it doesn't: that theorem is one abstract pair of states, not Program A and Program B, and the equal fingerprint for these programs is pinned by the script, not by Coq. The conservativity theorem doesn't cover B either, because B uses PNEW, and PNEW isn't a classical instruction: it adds modules. What Coq proves is the general shape, and the probe's half of it lives in the standalone copy of the kernel (Section 16 says what that copy is). From any state whose arrow list is empty, MORPH_DELETE finds nothing (`graph_empty_morphisms_delete_fails_tc`), and when the delete finds nothing the error flag goes up (`morph_delete_no_morphism_err_tc`). B ends with an empty arrow list. For the classical case: in the standalone kernel's base state, any classical run followed by MORPH_DELETE of arrow zero ends with the error flag up (`classical_trace_fails_morph_delete_probe_tc`), and after a MORPH_ID the same probe ends with it down (`morph_id_step_passes_morph_delete_probe_tc`); the pair is packaged as the strictness witness (`morph_delete_probe_separates_thiele_from_classical_tc`). In the kernel proper, the classical half is `classical_trace_preserves_graph`, which says a run of classical instructions leaves the graph exactly as it was. So Act 4's *point* is a theorem. Act 4's *numbers* are a script.

The probe's half, exactly. Write $\text{del}(G, k)$ for removing arrow k from graph G ; it fails when no arrow in G has number k .

Lemma 21.4 (The probe fails on an empty arrow list). *(1) If the arrow list of G is empty, $\text{del}(G, k)$ fails for every k . (2) If $\text{del}(\text{graph}(s), k)$ fails, then for every declared cost d , the step from s on MORPH_DELETE of arrow k at cost d ends with the error flag true.*

Proof. (1): removal first asks whether some arrow has number k ; on the empty list the answer is no. (2): the step function's failure branch sets the flag to its old value or true, which is true. $\square$

Checked as `graph_empty_morphisms_delete_fails_tc` and `morph_delete_no_morphism_err_tc` (ThieleMachineComplete.v, the standalone copy of the kernel). In the kernel proper the same fact is the failure rule `step_morph_delete_bad` (VMStep.v).

Theorem 21.5 (One step the classical instructions can't match). *There are a state s_0 and instructions u and q such that the error flag of $\text{apply}(\text{apply}(s_0, u), q)$ is false, while for every list t of classical instructions the error flag of $\text{apply}(\text{run}(t, s_0), q)$ is true.*

Proof. Take s_0 with one module, numbered 0, no arrows, the arrow counter at 0, the error flag down; $u = \text{MORPH_ID}$ on module 0 at declared cost 0; and $q = \text{MORPH_DELETE}$ of arrow 0 at declared cost 0. After u there is an arrow 0, so q succeeds; this part Coq checks by computing. A classical run keeps the graph (Theorem 17.3, in the standalone copy's form), so the arrow list is still empty, and Lemma 21.4 gives the flag. $\square$

Checked as `morph_delete_probe_separates_thiele_from_classical_tc`, with its halves `morph_id_step_passes_morph_delete_probe_tc` and `classical_trace_fails_morph_delete_probe_tc` (ThieleMachineComplete.v). The classical class there is the standalone copy's own predicate, with the same twenty-two exclusions as the kernel's. Program B uses PNEW, which is not classical, so B is outside this theorem; what covers B is Lemma 21.4, because B ends with an empty arrow list.

21.5 What the receipt is

The counter is what the schedule added up to: with the demonstration's declared costs (one per PNEW, two per MORPH, one for COMPOSE, four plus the floor for the assertion), thirteen, of which the assertion's five is the only part that touched the assertion channel, and the floor's one is the only part the toll forced. The graph carries the structure the shadow leaves out. The trace tells you where things came from, as long as whoever reads it has a history they can trust, which is Section 7's question, the one its three escapes answer, and not this act's to close.

And one rule the acts have to respect: MORPH_DELETE removes one arrow and nothing else, but PSPLIT and PMERGE delete every arrow touching the modules they remove. An arrow's life is bounded by its modules'. In the four acts no module is split or merged, so nothing cascades; in a longer program, it would.

That's one receipt, read act by act, with the price of every line on it. What the receipt doesn't do on its own is tell you what anyone is *entitled* to conclude from it, or sort what happened in it into the kinds of learning Section 9 separated. Those were Part III's questions, asked about any system with the pieces. The next section asks them of this one.

22 Insight and entitlement on the machine

Section 21 left a receipt on the table and two questions beside it: what does a receipt like that entitle anyone to conclude, and which of the things on it were learning and which were commitment? Those are Section 8's and Section 9's questions, and in Part III they were about any system with the right pieces: a reading, a toll, a counter, a price list. They got by with cups, three playing cards and a sheet of paper with two boxes

on it, because that's all the claims needed. The machine of Section 16 to Section 21 has every one of those pieces at once, and more of each than the paper ever did: two readings instead of one, fifty-one moves instead of three, a declared price on every move, and a counter that lives inside the state. So this is where the two chapters meet something that runs. Some of their claims land cleanly. Some land with conditions the paper never had to mention, because the paper had nothing to hide them in, and I'll say which as they come. And the machine has one thing the paper couldn't: a theorem with a long name and a longer hypothesis list, which is what entitlement looks like when somebody actually has to check it.

22.1 The two readings, and who writes them

Section 3's paper had one reading. The machine keeps two that look alike, and I'm going to use them by name. (I'll call them readings and not marks: since Section 3 a mark has been one tally on the bill, and I'm keeping it that way.) One is the *certified flag*. Only CERTIFY sets it, nothing ever takes it down (that part is proved), and it carries no object and no check; it's a bit. The other is the *assertion channel*. Only MORPH_ASSERT writes it, and what it writes is a number worked out from the property's label, the sum of its character codes. A later assertion writes over it, an assertion with an empty label writes a zero, which is the same as clearing it, and nothing proves the channel stays set. Each sentence from here on says which of the two it's about.

And here's the part to hold on to: no step of the machine reads either one. Every rule that mentions them sets them or leaves them alone. Section 8 warned that nothing in the five parts makes a later move read the reading; this machine is a case where nothing does. So when I say a run is *entitled*, I mean what someone auditing the run may conclude from the reading and the receipt. The machine doesn't enforce it.

Section 8's note on the paper (the one beside Definition 8.1) has a machine version, and Section 18 already showed it: 42 written into a register and called an arrow is a note, raw data forged in one write, and the machine's assertion step looks the arrow up and refuses one that isn't there. For entitlement, what matters is what the assertion leaves behind: a paid event, charged at least one, and one reading on the whole machine, a number worked out from a label. (It doesn't record which arrow it named.) An asserted arrow has a receipt, and the receipt is the event. One limit on that word "arrow": the assertion step checks only that the arrow exists. Whether the run built it or started with it depends on where the run started.

Here are the two readings as symbols, and the one more bit that shows up below: the *error flag*, which a step raises when it can't do what it was asked, and which stays up once raised (Remark 16.1).

Definition 22.1 (The two readings). A state s of the machine carries a Boolean $\text{cert}(s)$, the *certified flag*, and a whole number $\text{addr}(s)$, the *assertion channel* (Section 16's field, which Section 17 writes $\text{certaddr}(s)$), and a Boolean $\text{err}(s)$, the error flag. Write

$\text{apply}(s, i)$ for the state after instruction i (Definition 17.1). The *checksum* of a label ℓ is $\Sigma(\ell) = \sum_{x \in \ell} \text{code}(x)$, the sum of its character codes; it is zero exactly when every character has code zero, and in particular for the empty label.

Who writes them, Section 17 already proved, over every instruction and every branch, and I won't state it twice: Theorem 17.6 for the flag and Theorem 17.7 for the channel are the home of every sentence above about who sets what. The one thing to add is that the flag's permanence also holds for the machine's unbounded step, the one that keeps every number exactly instead of cutting it to sixty-four bits (`vm_apply_u_certified_permanent`, `VMUnboundedLedger.v`).

22.2 Insight on the machine

Section 9 sorted what happens when a computation learns into raw observation, which can be free; certified insight, which the toll prices; and the observation that the price is flat whatever is certified (Definition 9.1 and Remark 9.1, argued there for any certification system). On the machine each of those gets a theorem, and the flat price gets a tier of its own, because the machine has a thing to certify that's striking enough to tempt anyone into thinking it should cost more.

The first thing, *raw observation*, on the machine. Something comes in and gets written down. A counter ticks. Neither certification reading moves because of it. Later steps can read what it wrote, and one of the machine's own checks does, but that reading is a separate, paid step. In the checked work the free case is the trial step, `CHSH_TRIAL` (one round of a two-player game I build in Section 25), and it's charged whatever its instruction declares, which can be zero. An input read isn't free: `READ_PORT` is on the cost floor. And "free" here is a fact about this machine's price list, not a claim that looking costs nothing in the physical world.

The second thing, *certified structural insight*. Some step turns a certification reading from off to on: the certified flag raised, or the assertion channel set by a property asserted on an arrow. On this machine that's a theorem over every one of its instructions (stated exactly below: any step that takes either reading from off to on costs at least one and raises the counter by at least one). Section 9's Gestalt reading of what's being charged for, a relation committed to, fits the assertion channel, which stores a number worked out from a property on an arrow. The certified flag by itself commits to no relation at all, which is Section 9's bare crossing, and I'll come back to that.

The third thing, *certified game-witness insight*. The run is certified *and* what it certifies is a result from that game, the one I haven't built yet, a result that no single fixed strategy can produce: a score above two, when I get there. (That's a statement about the tallies the machine kept: with those counts, no one fixed strategy gives them. Ruling out every model where the answers were settled in advance by some shared hidden plan, which physicists call hidden-variable models, would need a different argument and enough trials, and that isn't what's checked.) It gets its own tier because

the thing certified is so striking, and I'll tell you what the proof says about it, because it's humbling. The theorem for this tier takes the hypothesis that the end state shows that score above two, and then its proof *discards that hypothesis* and uses only the second thing's fact: that the certified flag went from off to on somewhere in a run, which costs at least one. (The run is taken in list order, from a state where the flag starts off.) The price of a certified game witness is the price of a certified anything: at least one. The game adds meaning. It doesn't add cost. That's Section 9's flat price, landed. The three things are collected as one theorem (its name is in the trail below), and I'd rather you know that its third part is its second part lifted from one step to a whole run, for the certified flag only, with the game hypothesis dropped.

Here are the three, exactly. The two readings are the certified flag cert and the assertion channel addr of Definition 22.1; $\text{apply}(s, i)$ is one step, $\text{cost}(i)$ is the price list's charge for i (Definition 17.4), and μ is the counter. The step that raises the flag is CERTIFY and the one that writes the channel is MORPH_ASSERT. For lists, $[]$ is the empty list and $i :: t$ is the list that starts with i and carries on with the list t , as in Section 9.

Definition 22.2 (Certified insight event, and the three tiers). A step from state s by instruction i is a *certified insight event* when it turns a reading on: either $\text{addr}(s) = 0$ and $\text{addr}(\text{apply}(s, i)) \neq 0$, or $\text{cert}(s) = \text{false}$ and $\text{cert}(\text{apply}(s, i)) = \text{true}$. An instruction is *raw observation* when no step by it, from any state, is a certified insight event. *Certified structural insight* is a certified insight event. For the third tier, runs are taken in list order: $\text{run}_{\text{list}}([], s) = s$ and $\text{run}_{\text{list}}(i :: t, s) = \text{run}_{\text{list}}(t, \text{apply}(s, i))$, with no program counter involved. *Certified game-witness insight* is such a run that starts with $\text{cert} = \text{false}$ and ends with $\text{cert} = \text{true}$ and with $S > 2$, where S is the game statistic computed from the trial tallies the machine keeps (Section 25).

Lemma 22.1 (A trial is raw observation). *Every trial step of the game is raw observation. Its charge is the cost written in the instruction, which may be zero.*

Proof. A trial step is neither CERTIFY nor MORPH_ASSERT, so by Theorem 17.6 and part (1) of Theorem 17.7 it leaves both readings as they were, from every state. The charge is read off the price list. $\square$

Checked as `chsh_trial_is_tier0` (WitnessInsightGeneral.v); the charge is the trial step's case of `instruction_cost` (VMStep.v).

Theorem 22.2 (Certified insight is not free). *If the step from s by i is a certified insight event, then $\text{cost}(i) \geq 1$ and $\mu(\text{apply}(s, i)) \geq \mu(s) + 1$.*

Proof. Two cases. If the channel turned on, i is one of the five steps that reveal, emit, join, check a formula or assert, because every other instruction leaves the channel alone (a case split over all instructions); each of those five is priced at its declared cost

plus at least one. (By Theorem 17.7 it is in fact MORPH_ASSERT, but the proof only needs the wider class.) If the flag turned on, i is CERTIFY, priced at its declared cost plus one. Either way $\text{cost}(i) \geq 1$, and conservation, $\mu(\text{apply}(s, i)) = \mu(s) + \text{cost}(i)$, gives the second half. $\square$

Checked as `certified_insight_nonfree` (`InsightTaxonomy.v`), from `no_free_certification`, `no_free_certification_certified` and their counter forms (`AbstractNoFI.v`), packaged there as `certification_requires_positive_mu`; conservation is `vm_apply_mu` (`MuLedgerConservation.v`). It's parts (1) to (3) of Section 17's Theorem 17.8 said in one breath, and part (2) of Theorem 21.3 is the same statement met in Act 3; it gets its own block here because the third tier leans on it.

Theorem 22.3 (The price of a game witness is flat). *Let t be a list of instructions and s a state with $\text{cert}(s) = \text{false}$. If $s' = \text{run}_{\text{list}}(t, s)$ has $\text{cert}(s') = \text{true}$ and $S(s') > 2$, then $\mu(s') \geq \mu(s) + 1$.*

Proof. The hypothesis $S(s') > 2$ is not used. Induction on t . The empty list leaves the flag false, which contradicts the end. For $i :: t$: if the flag is on after i , that step is a certified insight event, so it raises μ by at least one (Theorem 22.2), and the rest of the run never lowers μ . If the flag is still off after i , the induction hypothesis covers the rest, and the step by i didn't lower μ . $\square$

Checked as `witness_insight_nonfree_general`, through `no_free_certification_certified_trace_mu` (`WitnessInsightGeneral.v`). The same lift for the assertion channel, with the extra fact that the list contains an instruction of the five-member class, is `no_free_certified_insight` (`InsightTaxonomy.v`). The three tiers are collected as `witness_insight_complete_taxonomy` (`WitnessInsightGeneral.v`): Lemma 22.1, Theorem 22.2 and Theorem 22.3, joined by "and."

So Section 9's two facts and a warning hold here word for word. Looking can be free. Turning a reading on can't, and putting a game result behind it doesn't make it cost any more.

Section 9 also asked who decided the split, and said any concrete machine owes you its whole table. This one's table is Section 17's. The floor sits under the whole *cost-floor class*, Section 17's positive class of twelve (Definition 17.5), which is wider than the two steps that turn a reading on, and every step outside that class costs whatever its instruction declares, which can be zero. So the sorting into three things is by which reading moves, as it was in Section 9. The part of the schedule that's *forced* is the floor under the flag-raisers, and Section 4 is the argument for it (Theorem 4.2); the rest is a choice, and Section 17 defends it under the table.

22.3 Test, then commit, on the machine

Section 9 put a CHECK on the paper and found nothing in the five parts that makes VOUCH wait for it (Remark 9.2), and said a particular system could close that gap by writing it into its step. This machine doesn't. Here's its version, the same gap with numbers on it.

The test is LASSERT, the formula check Section 19 took apart: it charges on the attempt, pass or fail, and it raises neither certification reading. The commit is one of two other steps: MORPH_ASSERT, which puts a property on an arrow and sets the assertion channel, or CERTIFY, which raises the certified flag with no arrow and no property behind it at all, the weakest commit there is. So the honest protocol is two steps, with two separate charges, one for the checking work and one for the flip.

And nothing in the machine makes the second step follow the first. The assertion step checks only that the arrow it names exists. It has no way to read whether a test ran earlier in the run on the formula the property names. A program can assert “validated constraint” on an arrow with no test anywhere in its history and get the same stored value (the number worked out from the label) for the same charge on that step. The two whole states can still differ by whatever else the two programs did; pad the shorter program with a step that does nothing and costs the same, and they agree. So Section 9's sentence holds here exactly: the receipt says a commitment was paid for, not that it was earned. On this machine the toll prices the commit, and whether the commit was preceded by a test is a fact about the program, not a guarantee of the machine.

The two facts under that, exactly.

Lemma 22.4 (The check commits nothing). *For every state s and every formula-checking instruction i , $\text{addr}(\text{apply}(s, i)) = \text{addr}(s)$ and $\text{cert}(\text{apply}(s, i)) = \text{cert}(s)$, whether the check passes or fails.*

Proof. The check is neither MORPH_ASSERT nor CERTIFY. Apply part (1) of Theorem 17.7 and Theorem 17.6. $\square$

Checked as instances of `non_morph_assert_preserves_cert_addr` (Revelation-Requirement.v) and `vm_apply_certified` (PrimeAxiom.v); no theorem names the formula check's case by itself. The same two equalities sit inside Theorem 19.1, among everything else the check leaves alone; here they stand by themselves because they're the test half of the protocol.

Remark 22.1 (The commit reads no test). The assertion step's rule has one condition, that the arrow it names is in the current graph. When it is, the new channel value is $\Sigma(\ell)$ for the instruction's label ℓ , and the charge is the declared cost plus one.

Nothing in the rule depends on which instructions ran before. This is read off the rule (`step_morph_assert_ok` in `VMStep.v` and the matching case of `vm_apply` in `SimulationProof.v`); the files state no separate theorem for it.

A worked chain, so the two prices have numbers. Say a register holds the address of a formula whose declared length is four words, and another holds the address of the two rows of values it's checked against (one meant to make it true, one meant to make it false; Section 19 laid them out). The check, with declared cost 0, is charged $4 \cdot 8 + (0 + 1) = 33$, pass or fail, and moves neither reading. Then, if it passed, an assertion with declared cost 0 and the label "validated_constraint" on an arrow that exists is charged $0 + 1 = 1$ and sets the channel to the label's checksum, which isn't zero, since the label has characters with nonzero codes. The total is 34, and the channel crossed from zero on the second step, not the first. Skip the check and the assertion alone still costs 1 and writes the same number. These numbers are the price function evaluated on the two instructions (`instruction_cost`, `VMStep.v`), not a separate theorem.

One edge the paper didn't have, because the paper's VOUCH, played from NO, always crossed: an assertion with an empty label pays and raises nothing, because the number it stores is zero. Paid, and no reading moved. Section 9's "a crossing costs at least one" is untouched by it; it just isn't a crossing.

22.4 Entitlement on the machine

Now Section 8's count. Section 8 said a run is entitled to a claim when it has an object, a checked relation and a paid event (Definition 8.1), and that the leaning costs at least the index bits of the narrowing it commits to, given lists, a tree, a covering, and a run that paid for the tree's depth (Theorem 8.2). On this machine that is a theorem with a long name, `structural_entitlement_representation`, and I want to give you its hypotheses, all of them, because the theorem is only as good as what it's handed, and a list of hypotheses is the kind of thing that gets skimmed while the conclusion gets quoted.

First, what "paid step" means here. Section 8 counted every step the price list charges at least one for, and on this machine those are the cost-floor steps from a few paragraphs back: the steps that reveal, emit, join, check a formula, read a port (take in a value from outside the machine), certify or assert, and the checks for the two-player game of Section 25. Only two of them, CERTIFY and MORPH_ASSERT, turn a reading on. The tree is a bare shape, as in Section 8, and "no deeper than the number of cost-floor steps" is a count along the run that happened. It isn't a walk down the tree. In the example below, two of the counted steps emit, and emitting raises nothing.

The theorem is handed: a decoder that turns the run's receipts into an observation (in the files the receipts are the list of instructions the run was handed, not a log of the steps it took); an exact equality test on observations; a posterior that's a strict subset

of the prior; one eliminated state and an observation that gives it a value no surviving state gives (the *distinguishing test*, which is Plato's third account from Section 8, made into a function); a start with the assertion channel at zero, and an end that's certified, a word the files use here for something that never looks at the certified flag. It means three things at once: no error latched, the posterior's membership test accepts the decoded receipts, and the assertion channel is nonzero; a decision tree no deeper than the number of cost-floor steps the run executed; a non-empty posterior; and a second observation function, the representative one, with a family of fibres, one per surviving state, such that every prior state sits in the fibre of some survivor that gives it the same representative observation. The fibre sizes together have to add up to at least the prior's length, and no fibre may be larger than the tree has leaves.

That last clause is Section 8's covering condition, carried by fibres, and it's the nearest thing the record has to an anti-stopped-clock clause. I have to be exact about how near. It says every prior state is covered by some survivor, and no survivor stands for more states than the tree has leaves. That limits how much can be deleted per leaf you paid for. It doesn't say why anything was deleted. The representative observation is a function somebody supplies, and one that returns the same thing for every state makes "same representative observation" true of every pair; the small two-state example below does exactly that. So the clause rules out deleting too much for the tree you've paid for. It doesn't rule out deleting for no reason.

Given all that, the theorem concludes three things: the posterior's predicate is strictly stronger than the prior's, an assertion event happened, and the counter rose by at least the index bits lost. Section 8's three small prints hold here unchanged: both logarithms round up separately and the subtraction stops at zero, the lists are lists with repeats counted, and the difference isn't the number of questions anyone asked.

Picture that chain as three links you can hold one at a time, like three cups in a row: narrow the list, find the step that paid, count the bill. Two of the links are their own theorems and I'll name them so you can find them. Narrowing a list, with a test that distinguishes, makes the membership predicate strictly stronger: `feasible_strict_subset_implies_strict_predicates`, Section 8's card argument checked on lists of this machine's states. And on this machine, reaching a certified end from a clear start means an assertion event fired somewhere in the run: `strengthening_requires_structure_addition`. On that second one, one more thing about its proof: its "strictly stronger" hypothesis is taken in and split. The strict half is thrown away, and the other half only builds a premise that the last step ignores. What does the work is "the channel was clear at the start and set at the end," and the flip follows from zero-then-nonzero. The theorem is true. Its name promises a tighter link than its proof uses.

Here is the whole chain exactly, one link at a time. First the words.

Definition 22.3 (Runs, state lists, observations, predicates). A *program* p is a list of instructions. The run $\text{run}(n, p, s)$ with fuel n from state s repeats, at most n times, “fetch the instruction of p at the program counter and apply it,” and stops early if the counter points past the end of p . A *state list* is a finite list Ω of states; $|\Omega|$ is its length, repeats counted. Ω' is a *strict sublist* of Ω when every member of Ω' is in Ω and some member of Ω is not in Ω' . An *observation* is a function obs from states to lists of instructions, and an *equality test* is a Boolean function eq on such lists with $\text{eq}(o_1, o_2) = \text{true}$ exactly when $o_1 = o_2$. A *receipt predicate* is a Boolean function on lists. The *membership predicate* of Ω under obs is

$$P_\Omega(o) = \text{true} \iff \text{eq}(\text{obs}(s), o) = \text{true} \text{ for some } s \in \Omega.$$

P is *stronger* than Q when $P(o) = \text{true}$ implies $Q(o) = \text{true}$ for every o , and *strictly stronger* when, in addition, some o has $P(o) = \text{false}$ and $Q(o) = \text{true}$. A state w is *distinguished* from Ω' by obs when $\text{obs}(w) \neq \text{obs}(t)$ for every $t \in \Omega'$.

The run with fuel is Section 17’s (Definition 17.1), written out again here because every block below is stated on it.

Theorem 22.5 (Narrowing becomes strengthening). *Let eq be an equality test, let Ω' be a strict sublist of Ω , and let $w \in \Omega$ with $w \notin \Omega'$ be distinguished from Ω' by obs . Then $P_{\Omega'}$ is strictly stronger than P_Ω .*

Proof. “Strictly stronger” has two parts. (a) If $P_{\Omega'}(o)$ is true, some $t \in \Omega'$ has $\text{obs}(t) = o$; that t is in Ω , so $P_\Omega(o)$ is true. This part is only the inclusion. (b) Take $o = \text{obs}(w)$. Since $w \in \Omega$, $P_\Omega(o)$ is true. No member of Ω' has observation o , so $P_{\Omega'}(o)$ is false. The gap between the lists would not give (b) by itself: a removed state can share its observation with a surviving one, and then the two predicates accept the same observations. That is why the distinguished state is a hypothesis of its own. In the Coq proof the gap witness that comes with “strict sublist” is not used at all; the supplied w does the work. $\square$

Checked as `feasible_strict_subset_implies_strict_predicates` (InformationGainToStrengthening.v), with the predicate defined as `omega_predicate`; the form that only says some strictly ordered pair of predicates exists is `b4_information_reduction_derives_strict_predicates` (HonestNoFI_TheoremsWithoutAssumptions.v).

That’s the first link: narrow the list with a test that separates, and the predicate gets stronger. The second link needs the word “certified” pinned down, along with the one step it points at.

Definition 22.4 (Certified end, structure addition). Fix a *decoder* d , a function from

instruction lists to lists. The files call the program p of a run its *receipts*: what the decoder reads is the list of instructions the run was handed. The end state of a run of p is *certified* for a predicate P when $\text{err} = \text{false}$ there (the error flag is down), $P(d(p)) = \text{true}$, and $\text{addr} \neq 0$ there. The run $\text{run}(n, p, s)$ has a *structure addition* when one of the steps it executes goes from a state with $\text{addr} = 0$ to a state with $\text{addr} \neq 0$. (This is Section 19’s structural gain, Definition 19.2, under the name the files use here.)

Theorem 22.6 (A certified end from a clear start has an assertion event). *Let P_{strong} be strictly stronger than P_{weak} , let $\text{addr}(s) = 0$, and let the end of $\text{run}(n, p, s)$ be certified for P_{strong} . Then the run has a structure addition.*

Proof. Only the third clause of “certified” is used: the channel is zero at the start and nonzero at the end. Walk the run (induction on the fuel). At each executed step either the channel turns nonzero, which is a structure addition, or it is still zero and the rest of the run starts again from a clear channel. A run that stops with the channel still zero ends at zero, which contradicts the end. The strictness hypothesis is taken apart and its pieces feed nothing the conclusion needs. By part (3) of Theorem 17.7, the step found is a MORPH_ASSERT naming an arrow that exists, with a label of nonzero checksum. $\square$

Checked as `strengthening_requires_structure_addition` (NoFreeInsight.v), through `supra_cert_implies_structure_addition_in_run`; the step is identified by `structure_addition_in_run_iff_morph_assert_bridge_pattern` (both RevelationRequirement.v).

So a certified end from a clear start always points at a step you can name. Now the count. It needs a tree, the cost-floor steps, and the fibres.

Definition 22.5 (Decision tree, cost-floor count, fibres). Decision trees, their depth and their leaf count $L(T)$ are as in Definition 8.2. The *cost-floor* instructions are the ones that reveal, emit, join, check a formula, read a port, certify or assert, and the five game checks; the price list charges each of them its declared cost plus at least one. $c(n, p, s)$ is the number of cost-floor instructions among the steps $\text{run}(n, p, s)$ executes. The tree T is *realized* by the run when $\text{depth}(T) \leq c(n, p, s)$. A *posterior-representative reduction* of Ω to Ω' for an observation r and a tree T is a function F from states to state lists (the *fibres*) such that: every $s \in \Omega$ lies in $F(t)$ for some $t \in \Omega'$ with $r(s) = r(t)$; $|\Omega| \leq \sum_{t \in \Omega'} |F(t)|$; and $|F(t)| \leq L(T)$ for every $t \in \Omega'$.

Lemma 22.7 (Paid steps cover the counter). *For every run, $c(n, p, s) \leq \mu(\text{run}(n, p, s)) - \mu(s)$.*

Proof. The rise of the counter over the run is exactly the sum of the charges of the executed steps (Theorem 16.1). Every cost-floor step contributes at least one to that

sum, and no step contributes less than zero. $\square$

Checked as `info_priced_cert_executions_bound` (`MuShannonBridge.v`), resting on `cert_setter_cost_pos` (`VMStep.v`) and `delta_mu_equals_ledger_sum`.

Lemma 22.8 (Index bits are paid for). *If T is realized by $\text{run}(n, p, s)$, $|\Omega'| > 0$, and there is a posterior-representative reduction of Ω to Ω' for some r and T , then*

$$\lceil \log_2 |\Omega| \rceil \dot{-} \lceil \log_2 |\Omega'| \rceil \leq \mu(\text{run}(n, p, s)) - \mu(s),$$

where $\lceil \log_2 \cdot \rceil$ is the base-two logarithm rounded up and $\dot{-}$ is subtraction on whole numbers that stops at zero.

Proof. The fibres give $|\Omega| \leq \sum_{t \in \Omega'} |F(t)| \leq L(T) \cdot |\Omega'|$: sort the prior states by the survivor whose fibre holds them, and each of the $|\Omega'|$ groups has at most $L(T)$ members. The rounded-up logarithm respects order and is subadditive on products, $\lceil \log_2 ab \rceil \leq \lceil \log_2 a \rceil + \lceil \log_2 b \rceil$, so

$$\lceil \log_2 |\Omega| \rceil \leq \lceil \log_2 L(T) \rceil + \lceil \log_2 |\Omega'| \rceil.$$

A tree of depth d has at most 2^d leaves (induction on the tree), so $\lceil \log_2 L(T) \rceil \leq \text{depth}(T)$. Chain the rest:

$$\lceil \log_2 |\Omega| \rceil \dot{-} \lceil \log_2 |\Omega'| \rceil \leq \text{depth}(T) \leq c(n, p, s) \leq \mu(\text{run}(n, p, s)) - \mu(s),$$

the middle step being realization and the last Lemma 22.7. The first clause of the reduction, the one about the representative observation, is discarded by the Coq proof; only the two counting clauses are used. $\square$

Checked as `info_priced_posterior_representative_reduction_bound` (`MuShannonBridge.v`), which goes through `posterior_representative_reduction_implies_fibered_reduction`, `fibered_reduction_implies_tree_cover` (the bound $|\Omega| \leq L(T) \cdot |\Omega'|$, named `tree_covers_feasible_reduction`), `tree_cover_implies_log2_up_reduction_bound`, `decision_tree_log2_up_leaf_bound` and `info_priced_arbitrary_feasible_reduction_bound`. The arithmetic is Section 8's Theorem 8.2, here with the fibres supplying the covering and the cost-floor steps supplying the payment.

That's the third link: given the tree and the fibres, the counter covers the index bits the list lost. Put the three links on one hypothesis list and you have the theorem.

Theorem 22.9 (Structural entitlement, represented). *Fix fuel n , a program p , a start state s , a decoder d , two observations obs and r , an equality test eq , a prior Ω , a posterior Ω' , and*

a decision tree T . Assume: (H1) eq decides equality of observations; (H2) Ω' is a strict sublist of Ω ; (H3) some $w \in \Omega$ with $w \notin \Omega'$ is distinguished from Ω' by obs ; (H4) $\text{addr}(s) = 0$; (H5) the end of $\text{run}(n, p, s)$ is certified for $P_{\Omega'}$, the membership predicate of the posterior under obs ; (H6) T is realized by the run; (H7) $|\Omega'| > 0$; (H8) there is a posterior-representative reduction of Ω to Ω' for r and T . Then (C1) $P_{\Omega'}$ is strictly stronger than P_{Ω} ; (C2) the run has a structure addition; and (C3) $\lceil \log_2 |\Omega| \rceil - \lceil \log_2 |\Omega'| \rceil \leq \mu(\text{run}(n, p, s)) - \mu(s)$.

Proof. (C1) is Theorem 22.5 from (H1) to (H3). (C2) is Theorem 22.6 from (C1), (H4) and (H5). (C3) is Lemma 22.8 from (H6) to (H8). The three conclusions share a hypothesis list and nothing else: the proof of (C3) never looks at a membership predicate or at the channel. $\square$

Checked as `structural_entitlement_representation` (`HonestNoFI_TheoremsWithoutAssumptions.v`). The version with the end only required to be error-free and accepted, with no channel clause and so no (C2), is `observational_structural_entitlement_representation` (same file).

Hold it up against Section 8's three parts of entitlement (Definition 8.1) and they're all in the hypothesis list. The object is a pair of state lists and a decision tree. The checked relation is the membership test with its distinguishing observation. The paid event is a step that sets the assertion channel. That's as close as the checked work comes to the word; the word itself stays the book's.

Remark 22.2 (Why two observations). The distinguishing observation and the representative one can't be the same function. If $w \in \Omega$ is distinguished from Ω' by obs , the reduction's first clause puts w in the fibre of some $t \in \Omega'$ with $\text{obs}(w) = \text{obs}(t)$, which the distinction forbids. Checked as `distinguishing_observation_not_posterior_representable` (`HonestNoFI_TheoremsWithoutAssumptions.v`).

Remark 22.3 (The complete-tree bound). For any two lists with $|\Omega'| > 0$, a complete tree of depth $d = \lceil \log_2(|\Omega|/|\Omega'| + 1) \rceil$ satisfies the covering inequality (`exists_covering_tree`). So if a bounded run $\text{run}(n, p, s)$ realizes the complete tree of depth d (executes at least d cost-floor steps), then $\lceil \log_2 |\Omega| \rceil - \lceil \log_2 |\Omega'| \rceil \leq \mu(\text{run}(n, p, s)) - \mu(s)$ (`complete_tree_realization_bound`, `MuShannonBridge.v`). The tree is still a hypothesis, the complete one of the right depth; what the form drops is any tie between the two lists and the run. The lists only pick which depth the run has to realize. It shows how much of (C3) is arithmetic on lengths once the paid steps are counted. The same bound holds for lists whose states carry whole-number weights, with total weight in place of length, given a realized tree whose leaf count times the posterior's weight covers the prior's weight (`info_priced_weighted_feasible_reduction_bound`), and the natural fibres of an observation give a reduction whenever they meet the counting clauses (`observation_partition_reduction_implies_posterior_representative_reduction`).

22.5 Every sound shortcut lands here, and what that sentence is worth

The book’s big-sounding sentence in this neighbourhood is: every sound structural shortcut lands in that bound. I want to tell you exactly what it’s worth, which is less than it sounds and more than nothing.

What’s checked is a *record* with fifteen fields, and here “record” is Coq’s word for a bundle of named slots, not the kind of record Section 3 is about: seven pieces of data (the decoder, the observation function, a representative observation, the equality test, the prior, the posterior, the tree) and eight proof obligations over them. “Every sound shortcut lands here” is the theorem that unpacks that record and applies the bound, and its proof is four short steps (introduce, take the record apart, simplify, apply the bound), because the record was *defined* to carry exactly the bound’s hypotheses. It’s true by packaging. The right word for it is “a checklist”: a checklist for being a sound shortcut, not a discovery about shortcuts.

The checklist, exactly:

Definition 22.6 (Sound structural shortcut). For fuel n , a program p and a start state s , a *sound structural shortcut* is a record of seven pieces of data, d , obs , r , eq , Ω , Ω' and T , together with proofs of the eight hypotheses (H1) to (H8) of Theorem 22.9 for that data: fifteen fields in all.

Corollary 22.10 (Every sound shortcut lands in the bound). *For every sound structural shortcut, conclusions (C1), (C2) and (C3) of Theorem 22.9 hold for its data.*

Proof. Take the record apart into its fifteen fields and apply the theorem. Nothing else happens. $\square$

Checked as `every_sound_structural_shortcut_lands_here` (HonestNoFI_TheoremsWithoutAssumptions.v); a record is assembled from its fifteen parts by `sound_shortcut_from_components` (same file).

22.6 A search whose surviving candidates can be counted

Put three coins under three cups. Each coin is heads or tails, so there are eight possible rows under the cups. Someone hands you two answers: heads under the first cup, tails under the second. Lift the first cup and compare. A matching answer leaves four possible rows. Lift the second and compare. A match leaves two. The last coin can still be either way. The list really shrank: eight, four, two. Nothing in that count needed a claim about which way the last coin faces.

Here is that search on the machine. A candidate is a start state with an n -bit value in memory, one bit per word, and the prior contains all 2^n such states. The program carries k supplied answers. For each one it loads the corresponding memory bit, records the supplied answer through `READ_PORT`, compares the two, and jumps to a

trap if they disagree. If all the answers match, it makes a module, gives it an identity arrow and asserts that arrow. The answers are supplied, not discovered by an oracle inside the instruction: the question is which possible memory states this particular answer sequence accepts.

After j matching questions, the surviving candidates are exactly the values whose first j bits match those answers. There are 2^{n-j} of them. So the posterior is tied to what the program actually checks: run the program on any candidate, and it ends without an error exactly when that candidate is in the posterior.

The two observations have different jobs here. One looks at the asked bits and can tell a rejected candidate from every survivor. The other looks at the unasked bits. For any candidate, keep its unasked bits and replace its asked bits with the supplied answers. That gives a survivor with the same unasked bits. Each survivor represents 2^k candidates, one for every possible assignment of the asked bits. These are the fibres the bound needs. For the three cups and two questions, the survivor with the last coin heads represents four rows, and the survivor with the last coin tails represents the other four.

Each READ_PORT step costs two marks with these operands. The final assertion costs one. Everything else in this program costs zero on this ledger, so the total is $2k + 1$. The complete binary tree has depth k , and the run executes $k + 1$ cost-floor steps. That satisfies the representation theorem's realization condition, which counts paid steps; it is not a separate claim that the program stores or traverses that tree.

Theorem 22.11 (A checked bit search supplies the entitlement data). *Let $1 \leq k \leq n \leq 64$. Let the hidden memory value have length n , and let the k supplied answers agree with its first k bits. With the candidate start states, asked-bit observation, unasked-bit representative observation, complete tree of depth k , and program just described, all the hypotheses of Theorem 22.9 hold. The posterior predicate is strictly stronger, the run contains a structure addition, and*

$$\lceil \log_2 2^n \rceil - \lceil \log_2 2^{n-k} \rceil = k \leq \Delta\mu = 2k + 1.$$

For every candidate of length n , membership in the posterior is equivalent to the run ending without an error. The execution budget is $5k + 5$ steps.

Proof. Each successful question compares one more memory bit with its supplied answer; a mismatch reaches the trap. Induction over the questions gives the acceptance condition and the paid steps. Enumerating all assignments to the unasked bits gives the posterior's size. Keeping those bits and replacing the asked ones supplies a representative for every prior candidate, with 2^k candidates in each fibre. A candidate with its first bit opposite to the first answer witnesses strictness. The honest hidden value survives and the assertion suffix raises the channel. These facts fill the representation theorem's hypotheses; the instruction-cost sum is $2k + 1$. $\square$

Checked as `bit_search_entitlement`, `bit_search_bound_reads`, `posterior_is_what_the_run_certifies` (posterior membership is equivalent to an error-free run) and `bit_search_shortcut` (`BitSearchEntitlement.v`).

With the three cups, two answers cost five marks and remove two index bits. This is an example of the accounting theorem with a real candidate filter. It is not a speedup theorem, and its prices are the chosen instruction schedule. The relation between the answers and the candidates is proved for this program. The assertion instruction itself still accepts a label without requiring that proof; Section 14’s small machine is where the check-to-commitment rule is enforced by the step.

22.7 A small two-state witness

There is also a smaller witness to inhabiting the record. There’s a concrete run: a twenty-three instruction program, given a budget of thirty-three steps and ending with the counter at nineteen. It’s a factored search (find one cell in an N -by- N grid by searching the two coordinates separately instead of walking every cell), followed by three instructions, the last of which asserts an arrow. It has a prior list of two states and a posterior of one, a two-leaf tree (which I’d read as one leaf per factor), and that run is a closed term of the record type, which is Coq for a finished value with every one of the fifteen slots filled and nothing left open. The bound reads off it: stronger predicate, assertion event, counter up by at least one index bit. So the class isn’t empty. One thing is written down and type-checks.

Now the honest picture of that member, because it isn’t a picture of a search narrowing a candidate set. The prior is the start state and the end state of the same run, the observation asks whether the counter equals nineteen, and the representative observation is the constant empty one, so the covering clause asks for nothing. The member shows the record can be inhabited. It doesn’t show a real search shrinking a real list of candidates. And there’s a boundary on the other side of it: the same factored search with only its two emitting steps and no assertion at the end certifies the observation (no error, the posterior’s test passes, the bound’s arithmetic holds) but never raises the assertion channel, so the assertion-event conclusion doesn’t apply to it. The three-instruction suffix is what closes that gap.

Here’s the member laid open, all of it, because it’s small enough to count. Fifteen fields: seven data, eight proofs. The decoder is the identity on instruction lists. The program p is the twenty-instruction factored search for $N = 1$ followed by three steps that make a new group, give it its identity arrow (Section 18’s do-nothing arrow from a module to itself, `MORPH_ID`), and assert that arrow with the label “factorized”; the start s is the machine’s initial state; the fuel is 33; the end state, call it s_{19} , has counter 19. The other ingredients are these.

1. The prior is the two-element list $[s, s_{19}]$.
2. The posterior is the one-element list $[s_{19}]$.

3. Strict narrowing: s_{19} is in both lists, and s is in the prior and not the posterior.
4. The distinguishing observation returns p when the counter is 19 and the empty list otherwise. It returns p at s_{19} and the empty list at s , whose counter is 0.
5. The tree is one branch with two leaves, depth 1.
6. The tree is realized: the run executes three cost-floor steps (the two that emit and the assertion), and $1 \leq 3$. This is a count, not a walk down the tree.
7. The representative observation is written as “if the counter equals itself then the empty list,” which is always the empty list, and every fibre is the whole prior.
8. The certified posterior: no error at the end, the membership predicate of $[s_{19}]$ accepts p , and the channel holds the checksum of “factorized,” which is not zero.
9. The start is clear: the initial state’s channel is 0.

The equality test is decided by `receipt_list_eqb_spec`, the posterior is non-empty by `sighted_n1_supra_posterior_nonempty`, and the fibres are `sighted_n1_supra_representatives`.

Remark 22.4 (What the member computes). Feeding these into the corollary gives a strictly stronger posterior predicate, a structure addition, and $\lceil \log_2 2 \rceil \dot{-} \lceil \log_2 1 \rceil = 1 \leq 19 - 0$. Every obligation about the run is discharged by running it inside the proof checker (Coq’s `vm_compute` tactic), so the member is a computation, not an argument.

Checked as `factored_n1_shortcut` and `factored_n1_shortcut_lands_in_representation` (`StructuralAdvantageCertifiedShortcut.v`); the version without the suffix is in `StructuralAdvantageObservedShortcut.v`.

What it doesn’t do, said flatly. It doesn’t check that the factorisation is *right*. The assertion step certifies that an arrow was asserted with a property label; nothing ties that label to a factorisation checker. The same goes for the formula check of Section 19: when it passes, it shows a formula can be made true and can also be made false, and nothing makes the way of making it false one of the states in the prior list. Tying a check to a list takes a premise of its own, and here that premise is the lists and observation functions somebody supplies. It’s a receipt for what the run paid, not a proof of what the run claimed. The speedup in that example (searching the two coordinates separately, $2N$ steps, instead of the whole N -by- N grid, N squared steps) is divide-and-conquer, which is as old as computing and which nobody is selling here. And here’s what’s checked about it: the iteration counts are results about the runs, for every N . The blind search over the N -by- N grid runs N squared iterations and ends with the counter at 0; the factored one runs $2N$ and ends with the counter at 18. At $N = 1$ the factored program is the slower one, two iterations against one. The point of the example is only that the shortcut’s *admissibility* is on the record, with a cost, separate from its *dividend*. And it is not a result about P versus NP, and anyone who reads it

as one is reading their own ambition into the bytes. One more boundary: there is no unconditional bound on a single run's narrowing. A single execution path doesn't carry enough information for that; the bound needs the tree, or a per-branch account of what was ruled out, and the code refuses to claim more.

The numbers, so you can check them. The blind program, the one that walks every cell, has every declared cost set to zero and ends with the counter at 0. The factored one pays for two emitting steps with a one-character payload, each charged its payload's eight bits plus one, so 18; the three-instruction suffix adds one more for the assertion, so 19. Put a price on the counter: say one mark is worth λ iterations, so a run costs its iterations plus λ times its counter. The factored run is the cheaper one exactly when $2N + 18\lambda < N^2$, and once N is at least $18\lambda + 3$ it always is. That's the time tax in one line: the blind search pays in time, the factored one pays a fixed eighteen marks for knowing the grid splits, and past a size you can compute, knowing wins. The pure arithmetic around it is lopsided too: for $N \geq 6$, $N^2 > 2N + 18$, and for every $k \geq 1$ there is an $N \geq 2$ with $N^2 \geq k \cdot 2N$, so the ratio of the two iteration counts has no ceiling.

Remark 22.5 (What the speedup theorems state). Write B_t for the ten-instruction blind search with target t and $S_{l,r}$ for the twenty-instruction factored search with targets l and r , s_0 for the initial state, $\text{run}(F, p, s_0)$ for the program-counter runner with fuel F , and $r_{15}(s)$ for register 15, which counts loop iterations. (1) If $t + 1 < 2^{64}$ and $F \geq 5t + 8$, then $\text{run}(F, B_t, s_0)$ has $r_{15} = t + 1$, counter 0 and program counter 10. (2) If $l + r + 2 < 2^{64}$ and $F \geq 5(l + r) + 16$, then $\text{run}(F, S_{l,r}, s_0)$ has $r_{15} = (l + 1) + (r + 1)$, counter 18 and program counter 20. (3) For every $N \geq 1$ with $N^2 < 2^{64}$, every whole number λ , and fuel $F_b \geq 5N^2 + 3$ and $F_s \geq 10N + 6$, let $b = \text{run}(F_b, B_{N^2-1}, s_0)$ and $c = \text{run}(F_s, S_{N-1, N-1}, s_0)$. Then $r_{15}(b) = N^2$, $\mu(b) = 0$, $r_{15}(c) = 2N$, $\mu(c) = 18$, and

$$r_{15}(c) + \lambda\mu(c) < r_{15}(b) + \lambda\mu(b) \iff 2N + 18\lambda < N^2.$$

(4) With the hypotheses of (3) and $N \geq 18\lambda + 3$ in place of $N \geq 1$, $r_{15}(c) + \lambda\mu(c) < r_{15}(b) + \lambda\mu(b)$.

Proof. (1) and (2) by a loop invariant: each pass through a loop body is five steps, adds one to register 15 and one to the loop index, and charges nothing, because every arithmetic, load and jump in both programs declares cost zero; the index reaches its target because the subtraction on sixty-four-bit words is zero exactly when the two words are equal, and both are below 2^{64} . The exits are a few more steps each, and in the factored program each exit runs one EMIT of a one-character payload at nine marks. Past the end of the program the runner stops, so extra fuel changes nothing. (3) is (1) at $t = N^2 - 1$ and (2) at $l = r = N - 1$, then arithmetic. (4) is (3), since $N \geq 18\lambda + 3$ gives $N^2 \geq 18\lambda N + 3N > 2N + 18\lambda$. $\square$

Checked as `blind_program_run`, `sighted_program_run`, `time_tax_theorem` and `time_tax_sighted_wins` (`StructuralAdvantage.v`), all closed, with the loop lemmas `loop_iteration` and `loop_iterations` and the word fact `word64_sub_zero_iff_iteration_savings_dwarfs_mu_cost` and `advantage_factor_unbounded` (same file) are the two inequalities above, proved by arithmetic alone. The case $N = 1$ is also computed directly: `blind_halts_in_n_squared` (fuel 8, register 15 holds 1, counter 0) and `sighted_halts_in_two_n` (fuel 20, register 15 holds 2, counter 18).

22.8 What landed

So here's Section 8 and Section 9 on something that runs. Looking can be free: `CHSH_TRIAL` moves no reading and can cost zero. Turning a reading on can't: every one of the fifty-one instructions was checked, and any step that does it pays at least one. A game result behind the flag adds meaning and not a mark. The check and the commit are two steps, `LASSERT` and `MORPH_ASSERT`, priced separately, and nothing in the machine joins them or makes the second wait for the first. A certified end from a clear start points at an assertion you can name, and a run that supplies a narrowing, a tree, fibres and the cost-floor steps to cover the tree pays at least the index bits it commits to; the record that packages all that is a checklist with one member, and the member is honest about being small. A paid flag is still not a true claim, and the machine still never reads either reading. Those are the parts of Section 8 and Section 9 this machine witnesses. The rest of Parts II and III, the subsumption of an ordinary machine, the windows and their collisions, the floor on all fifty-one, the finite fragment that pays for its merges, gets the same treatment next, in Section 23.

23 The machine as witness

Parts II and III made claims about any machine and kept saying "when the machine exists on the page, I'll show you." The machine exists, and some of the showing has happened already: Section 16 put Section 6's counter rule on one field, Section 17 put Section 3's floor on fifty-one instructions, Section 20 showed Section 7's window losing the graph, and Section 22 just did Section 8 and Section 9 whole. This section is the rest: for each remaining claim, the theorem about *this* machine, with its hypotheses and its witness, so that the abstract parts can be checked against something that runs. I've grouped them by the section they pay off, and where an earlier chapter of this part already stated a fact, I point there instead of saying it again.

23.1 What this machine is, relative to an ordinary one (Section 2, Section 11)

An ordinary machine has arithmetic, memory, jumps, calls, ports and a halt. Of the fifty-one instructions, twenty-nine pass the machine's own test for that (`is_classical_opcode`), and I'd rather you see the test's answer than my description of it. It's the same line Section 17 drew, and the twenty-two that fail are listed by name in Section 17's Definition 17.3, so I won't make you read them twice. Everything else passes: the arithmetic, the memory and heap moves, the jumps, calls and returns,

the ports, HALT and CHECKPOINT, and three that aren't plain arithmetic but pass anyway, because they leave the structure alone: MDLACC, which only charges the ledger, and the two readers MORPH_GET and TENSOR_GET, which look at the structure without changing it. Call a run *classical* when every instruction in it is one of the twenty-nine.

Two theorems say the machine contains an ordinary one without disturbing the structure. *Faithfulness*: the shadow is, by definition, six fields (registers, memory, counter, program counter, error, certified), and a classical run leaves the graph, the assertion channel, and the flag where they were (`classical_run_shadow_faithful`; its first half is true by unfolding the definition, its second half is the next theorem). *Conservativity*: a classical run preserves the graph, the channel, and the flag (`classical_opcodes_preserve_structure`, from single-step lemmas per field). Neither says anything about the other fields (the witness counters, the tensor, the logic accumulator, the mode word), and I won't claim more than that. Conservativity holds both ways a program can run: as an in-order fold of the step function over a list, and under the program-counter runner, which follows the jumps. So a classical program that loops is inside what's proved, and Section 20's strictness theorem starts from the initial state for the same reason.

Here are the test and the theorem exactly. The theorem is Section 17's Theorem 17.3 under the name the files give it; it's restated here because this is where its partner, faithfulness, lives. For a state s write $\text{graph}(s)$ for its graph, $\text{chan}(s)$ for the assertion channel (Section 17 wrote it $\text{certaddr}(s)$), $\text{flag}(s)$ for the certified flag (Section 17's $\text{cert}(s)$), and $\mu(s)$ for the counter. The shorter names are only for this section.

Definition 23.1 (Classical opcode, classical run). `is_classical_opcode` returns false on exactly the twenty-two non-classical instructions of Definition 17.3 and true on the other twenty-nine. A list of instructions t is *classical* when every member passes. For a state s_0 , $\text{run}(t, s_0)$ applies the step function `apply` to s_0 and the members of t in list order, first to last; the program counter is not consulted. For fuel F , $\text{run}_{pc}(F, t, s_0)$ is the program-counter runner: F times, fetch the member of t at the program counter and apply it, stopping early when the program counter is past the end of t ; jumps move the counter.

Theorem 23.1 (Conservativity). *For every classical list t , every state s_0 and every fuel F , with s either $\text{run}(t, s_0)$ or $\text{run}_{pc}(F, t, s_0)$: $\text{graph}(s) = \text{graph}(s_0)$, $\text{chan}(s) = \text{chan}(s_0)$, and $\text{flag}(s) = \text{flag}(s_0)$.*

Proof. Induction on t . The empty list changes nothing. For a list i followed by t' , three single-step lemmas, each a case split over the instruction type, say that a classical instruction leaves the graph, the channel and the flag as they were, and the induction hypothesis covers t' . For the program-counter runner, induction on F : each round

either stops or fetches a member of t , which is classical, so the same single-step lemmas apply. $\square$

Checked as `classical_opcodes_preserve_structure` (`ClassicalConservativity.v`), from `classical_trace_preserves_graph`, `classical_trace_preserves_cert_addr` and `classical_trace_preserves_certified`, and `classical_opcodes_preserve_structure_run_vm` for the program-counter runner. `classical_run_shadow_faithful` (`TuringClassicalEmbedding.v`) repeats the three equalities and adds that the shadow of the final state is the record of its registers, memory, program counter, counter, error bit and flag, which holds by unfolding the definition.

A third, which goes by the name “Thiele simulates Turing” in the files and which I’ll state for exactly what it is: there’s a file-local pairing of a Turing machine’s configuration with a counter, where the lifted machine’s step tracks the Turing machine’s step and adds one to the counter (two if it wrote a non-blank), and a theorem that running the lifted machine on n units of fuel projects to running the Turing machine on n units (`thiele_simulates_turing_gen`, and `thiele_simulates_turing` for a start with counter zero). The file’s own scope note says not to cite it as *the* VM simulating Turing machines. It’s a small lifted model, not the twelve-field state. The honest sentence is: the pattern “an ordinary machine plus a counter that only climbs” is instantiated once in miniature and proved to track the ordinary machine; the twelve-field machine’s relation to ordinary computation is the faithfulness and conservativity pair.

The degenerate-projection theorem (`degenerate_projection_theorem`) packages four parts, and this small simulation is the first. The other three are Section 20’s quotient theorem, Theorem 20.3: two states have equal shadow exactly when they agree on the six fields, a classical run from states that agree on the shadow, the graph and the control and status words ends at equal shadows, and two distinct states can have equal shadow.

23.2 The counter on this machine (Section 6)

Section 6 argued its facts about the counter for any machine whose state carries a counter kept by a price list (Definition 6.1), and said the checked versions are about this machine. Here they are. The one-step rule, counter after equals counter before plus the instruction’s cost, is Section 16’s Theorem 16.1, checked by going through all fifty-one instructions, the failure branches included; everything in this subsection stands on it. Write $\text{cost}(t)$ for the sum of the costs of a list t , s_{init} for the initial state (counter 0), and call a state *reachable* when it is s_{init} or $\text{apply}(s, i)$ for a reachable s .

One thing about the checked name of the uniqueness half, because it misleads: `mu_is_initial_monotone` doesn’t have “monotone” among its hypotheses. Its two hypotheses are that the other books start at zero and follow the schedule; climbing

follows from the schedule, since the schedule never charges a negative amount. And its conclusion holds on reachable states only, which is what Section 6 said a receipt should promise.

Theorem 23.2 (The counter on this machine). (1) For every state s and list t , $\mu(\text{run}(t, s)) = \mu(s) + \text{cost}(t)$. (2) For every state s and instruction i , $\mu(s) \leq \mu(\text{apply}(s, i))$. (3) Let B be any function from states to $\mathbb{N}$ with $B(\text{apply}(s, i)) = B(s) + \text{cost}(i)$ for all s and i . If $B(s_{\text{init}}) = 0$, then $B(s) = \mu(s)$ for every reachable s . Without the start condition, there is a nondecreasing f with $B(s) = f(\mu(s))$ for every reachable s .

Proof. (1) Induction on t , one application of the one-step rule per instruction. (2) The one-step rule, with $\text{cost}(i) \geq 0$. (3) Induction on why s is reachable, exactly as in Theorem 6.2; for the last part take $f(n) = B(s_{\text{init}}) + n$. $\square$

Checked as `mu_accumulates_trace_cost` and `mu_is_initial_monotone` (`MuInitiality.v`), `vm_mu_monotonic_single_step` (`MuLedgerConservation.v`) and `monotone_factors_through_mu` (`MuInitiality.v`), all resting on `vm_apply_mu`.

Section 6 also pointed at a universal property nearby and said exactly how far the borrowed word “initial” reaches. Here’s the trace fold, because the word is going to get leaned on and you should see how little is under it. A *target* is any machine M that takes this machine’s instructions: a set of states, a step step_M , a cost per instruction, and a reading that obeys the toll. Pick a state b of M to start from. Write $t \cdot i$ for the list t with one more instruction i on the end, and $F_b(t)$ for the state M ends in when you walk the list t through it from b .

Theorem 23.3 (Folding a run is the only way). For every target M and starting state b : $F_b([]) = b$, and $F_b(t \cdot i) = \text{step}_M(F_b(t), i)$ for every list t and instruction i . And any function g from instruction lists to states of M with $g([]) = b$ and $g(t \cdot i) = \text{step}_M(g(t), i)$ for all t and i equals F_b on every list.

Proof. The first two facts are what walking a list means. For uniqueness, induct on the list from its end. On the empty list both give b . If $g(t) = F_b(t)$, then $g(t \cdot i) = \text{step}_M(g(t), i) = \text{step}_M(F_b(t), i) = F_b(t \cdot i)$. The proof never looks at the target’s reading or its cost. $\square$

Checked as `thiele_trace_fold_initial` (`ThieleInitiality.v`), which the file also exports under the older name `thiele_is_initial_a2_substrate`; the file’s own comment says that name does not assert the machine’s states are initial. The statement is over lists of this machine’s instructions, which is why it lives here and not in Section 6.

Coming down from lists to states is where the borrowed word stops. Write $\text{ev}(t)$ for the state this machine is in after running t from its start, and call v a *descended value* at a reachable state s when some list t has $\text{ev}(t) = s$ and $F_b(t) = v$.

Theorem 23.4 (When the fold comes down to states). *Fix a target M and a starting state b .*

- (1) *Every reachable state has exactly one descended value if and only if, for all lists t and u , $\text{ev}(t) = \text{ev}(u)$ implies $F_b(t) = F_b(u)$.*
- (2) *Suppose some function picks, for every reachable state, a list that reaches it. Then a map h from the machine's states to M 's states with $h(\text{ev}([])) = b$, $h(\text{apply}(\text{ev}(t), i)) = \text{step}_M(h(\text{ev}(t)), i)$, and the machine's reading at $\text{ev}(t)$ equal to M 's reading at $h(\text{ev}(t))$, for all t and i , exists if and only if two things hold: the condition in (1), and the machine's reading at $\text{ev}(t)$ equals M 's reading at $F_b(t)$ for every list t . Any two such maps agree on every reachable state.*
- (3) *A picking function as in (2) exists, so the equivalence in (2) holds with no premise about picking.*

Proof. (1) If equal machine states give equal target states, then $F_b(t)$ is a descended value at $\text{ev}(t)$, and any other one, $F_b(u)$ with $\text{ev}(u) = \text{ev}(t)$, equals it. Conversely, if $\text{ev}(t) = \text{ev}(u)$, then $F_b(t)$ and $F_b(u)$ are both descended values at that state, so they're the same one. (2) Any such map sends $\text{ev}(t)$ to $F_b(t)$, by induction on t from its end, exactly as in Theorem 23.3; so it gives both conditions, and any two such maps agree on reachable states. For the other direction, send each state s to $F_b(t)$ of the list picked for s . The first condition makes that equal to $F_b(t)$ for every t reaching s , which gives the start and the step; the second gives the reading. (3) Number every list of instructions so that every list gets a number. For a state s , pick the list with the first number whose run reaches s , or the empty list if none does. Equality of machine states is decidable, so each number can be checked; asking whether some number works at all is an unbounded search, settled by the choice principle that comes with Coq's real numbers. If s is $\text{ev}(t)$, the number of t works, so the search finds a list reaching s . $\square$

Checked as `trace_descent_unique_value_iff`, `reachable_simulation_exists_iff`, `reachable_simulation_requires_compatibility`, and `reachable_simulation_unique` (`TraceStateDescent.v`); the uniqueness for maps someone supplies is also `thiele_morphism_unique_on_reachable` (`ThieleInitiality.v`). For (3): `reachable_trace_representative`, `reachable_trace_representative_correct` and `reachable_representative_exists` (`ReachableTraceRepresentative.v`), and `generalized_reachable_simulation_holds`, the equivalence of (2) for every reading, target and starting state with the picking premise gone. Its one assumption is that choice principle, `sig_forall_dec`, already in the

standard-library set the rest of the tree uses. What it is not: a program that rebuilds a list from a state. The picking is a search that exists, not one anybody would run.

Three targets show where the edges are. One target has a single state and a reading that never says yes. It obeys the toll, because nothing ever crosses, and it gets no reading-preserving map from the machine's states at all, because the machine has a state that reads yes (`no_vm_state_morphism_to_never_certifies`, `ClaimBoundaryRegression.v`). Another keeps only one bit, whether the certified flag is up, raises it on `CERTIFY`, and leaves it alone on everything else; that one comes down fine, and it reaches both no and yes (`certification_quotient_satisfies_descent`, `TraceStateDescent.v`). The third is the interesting failure. It keeps the whole list of instructions it was handed, charges the machine's own prices, and reads the flag with the machine's own evaluator, so its reading agrees exactly. It still doesn't come down, because the empty list and a single zero-cost `JUMP` back to where the machine already is leave the machine in the same state and leave two different histories behind (`certification_agreement_does_not_imply_descent`, `TraceStateDescent.v`; it comes back as part (2) of Theorem 23.15 below). The toll plus a reading that agrees doesn't force a target to forget its history. So the fold is unique on lists, and that's what the word "initial" earns here. It doesn't make this machine's states initial among everything that obeys the toll, and I won't say it does.

Last, Section 6's short answer about windows, on this machine. Its *full window* shows the memory, the registers, the program counter, the counter and the certified flag. A window is *counter-complete* when some rule applied to what it shows returns the counter on every state, and *flag-complete* likewise; it *forgets the counter* when changing the counter and nothing else never changes what it shows, and likewise for the flag.

Theorem 23.5 (Neither the counter nor the flag can be dropped). *The full window is both counter-complete and flag-complete. Every window on this machine's states that forgets the counter, whatever else it shows, is not counter-complete. Every window that forgets the flag is not flag-complete.*

Proof. The full window shows both, so the rule just reads them off. For a window that forgets the counter, take one state and set its counter to 1 and, separately, to 2. The window shows the same thing for both, so any rule gives the same answer for both, and one of the two answers is wrong. The flag is the same argument with yes and no. □

Checked as `P_full_complete_neither_mu_nor_cert_droppable`, with `forgets_mu_not_mu_complete` and `forgets_cert_not_cert_complete` (`NecessityAbstract.v`; despite the file name, every statement is over this machine's states). It says neither part can be dropped. It doesn't say the full window is the smallest window that works, because no ordering of windows is defined in the file, so there's

nothing for it to be smallest in.

23.3 What no window recovers (Section 7)

Two windows, named. `forget` keeps the program counter, the counter, the registers, and the memory. `bare_observable` keeps the program counter, the registers, and the memory, and drops the counter. Each is a function from the twelve-field state to a smaller record.

This is where Section 7's fibre fact (Theorem 7.1: a decoder has to give the same answer on states the window can't tell apart) lands on the machine's own windows. Section 20 already stated the three witness pairs, as Theorem 20.6; here they are again under Section 7's name, with the two windows named. Each is listed as closed, with no axiom of any kind behind it, in the last assumption audit (the report the Linux build writes of every axiom each theorem leans on; Section 28 says what it covers), and together they are packaged as `thiele_content_irrecoverable_from_bare_projection`. And one fact Section 20 didn't state: *every* four-field snapshot has two distinct preimages, one certified and one not (`fiber_has_two_preimages`), so the many-to-one isn't a special pair; it's everywhere.

Definition 23.2 (Two windows; a function of a window). For a state s , $\text{forget}(s) = (\text{pc}(s), \mu(s), \text{regs}(s), \text{mem}(s))$ and $\text{bare}(s) = (\text{pc}(s), \text{regs}(s), \text{mem}(s))$. A field f of the state is a *function of* a window w when some φ has $\varphi(w(s)) = f(s)$ for every state s .

Theorem 23.6 (Three fields no window recovers). (1) *flag is not a function of forget.* (2) *μ is not a function of bare.* (3) *chan is not a function of forget.*

Proof. Each part is one pair of states written out as records. For (2) the two states agree on every field except the counter, which is 0 in one and 1 in the other (both have program counter 10, registers (42, 0, 0, 0) and four zero memory words). Their bare images are equal, so a φ would have to return both 0 and 1 on the same input. Parts (1) and (3) are the same argument with a pair whose forget images are equal and whose flags, or channels, differ. Nothing in these three proofs runs the machine; the pairs are records. $\square$

Checked as `cert_not_function_of_forget`, `mu_not_function_of_bare_observable` and `cert_addr_not_function_of_forget` (ProjectionNonExistence.v, 190, 209), packaged as `thiele_content_irrecoverable_from_bare_projection`.

A fourth, from the diagonal section (Section 11's second wall): no decoder on the four-field window agrees, on every state, with "the assertion channel is nonzero" (`structural_shortcut_not_function_of_classical`). The two states that break every decoder aren't hand-built records this time; they're reached from the initial state by

running programs. Both start with the same two instructions. Then one runs MORPH_ASSERT, which overwrites the channel with the checksum of its property string, and the other runs an MDLACC whose charge of one matches MORPH_ASSERT's, so the four fields agree and the channel doesn't. Because the witnesses are reachable, the impossibility isn't about records nobody could ever produce.

Three small theorems sit beside that one, and they're worth a sentence each because they close off the reading a skeptic reaches for. Hand the decoder an oracle of any kind, a halting oracle included, and it's still a function of the four-field window, so the same two states break it (`structural_axis_survives_any_classical_oracle`): information that isn't in the input can't be supplied by an oracle. The two axes are independent in that same sense, each not a function of the other's window (`axes_mutually_independent`). And the predicate itself is decidable from the full state (`structural_membership_decidable`), so this is not a gap in computability degree. It's a gap in what the window holds. The file says as much itself, and I'd rather you didn't read the independence of the two axes as a Turing-degree result.

One thing I have to say about how these are proved, because Section 7 stated the general fact (`decoding_requires_fiber_constancy`), and it's natural to say the pricing results "are" that fact with a price in place of a decoder. As mathematics, fair. As a proof dependency, no: outside its own file the general fact is used once, in the attestation fragment from Section 12, not in any of these. These are proved directly from their witness pairs. The abstract theorem and the concrete ones agree; neither cites the other.

23.4 The floor, on fifty-one instructions (Section 3, Section 9)

Section 3's universal floor (Theorem 3.1) says: any system with the toll, any run from uncertified to certified, paid at least one. Section 17 put the one-step form on this machine (Theorem 17.8). Here's the whole ladder of its instances, from the narrowest to the one I'd cite if I could cite only one.

Two readings of "certified" run through all of this, the flag and the channel, and who writes each is Section 17's Theorems 17.6 and 17.7. One thing about the channel's floor to keep in mind: its proof doesn't use the fact that only MORPH_ASSERT writes the channel; it uses a wider class of five instructions that may touch the channel, and shows every one of them pays.

One step, the assertion channel: a step that takes the channel from zero to nonzero costs at least one (`no_free_certification`; proof: only the five instructions in that class can change it, and each has the floor). One step, the flag: same (`no_free_certification_certified`; only CERTIFY sets it). One step, either reading: the counter rises by at least one (`certification_requires_positive_mu`, `certified_insight_nonfree`). Over a run, the channel: the counter ends at least one above where it started (`no_free_certification_trace_mu`), and the run contains a floor-cost instruction (`no_free_certified_insight`). Over a run, the

flag: the counter ends at least one above where it started (a lemma, `no_free_certification_certified_trace_mu`; it doesn't also hand you the instruction). And the machine packaged as a certification system twice, once per reading, so that Section 3's theorem applies to it directly (`thiele_universal_nfi_cert_addr`, `thiele_universal_nfi_certified`).

Which is strongest? `universal_nfi_any_substrate`, the abstract one, and the two packagings are its instances. The per-step and counter forms are *sharper* (they name the instruction, or the counter rather than the summed cost) but narrower. No single theorem says "either reading, over a run" for this machine; what exists is the one-step either-reading theorem plus the two run theorems separately. Together they say it. There's no one name for it.

Here is the ladder as one statement, with cost the machine's schedule (an instruction's declared cost, plus one for CERTIFY, MORPH_ASSERT, LJOIN and the game checks, plus more for LASSERT, EMIT, REVEAL and READ_PORT; Definition 17.4) and run the in-order runner of Definition 23.1. Parts (1) to (3) and the channel half of (4) are Section 17's Theorem 17.8 again; the new piece is the flag half of (4).

Theorem 23.7 (The floor on this machine). *Let s be a state, i an instruction and t a list of instructions. (1) If $\text{chan}(s) = 0$ and $\text{chan}(\text{apply}(s, i)) \neq 0$, then $\text{cost}(i) \geq 1$. (2) If $\text{flag}(s)$ is false and $\text{flag}(\text{apply}(s, i))$ is true, then $\text{cost}(i) \geq 1$. (3) In either case $\mu(\text{apply}(s, i)) \geq \mu(s) + 1$. (4) If $\text{chan}(s) = 0$ and $\text{chan}(\text{run}(t, s)) \neq 0$, then $\mu(\text{run}(t, s)) \geq \mu(s) + 1$; the same holds when $\text{flag}(s)$ is false and $\text{flag}(\text{run}(t, s))$ is true.*

Proof. (2) The step leaves the flag alone unless i is CERTIFY, and CERTIFY with declared cost d costs $d + 1$. (1) An instruction outside the class REVEAL, EMIT, LJOIN, LASSERT, MORPH_ASSERT leaves the channel unchanged, by a case split over the instruction type, and every member of the class costs at least one. (3) is (1) or (2) together with conservation, $\mu(\text{apply}(s, i)) = \mu(s) + \text{cost}(i)$. (4) For the channel: if no member of t were in the class, the channel would stay 0, so some member is, and the total charge of t is at least that member's charge; conservation over the run turns the total into the rise of μ . For the flag: induction on t , splitting on whether the flag is already up after the first step. $\square$

Checked as `no_free_certification` (AbstractNoFI.v), `no_free_certification_certified`, `certification_requires_positive_mu`, `no_free_certification_trace_mu` and the lemma `no_free_certification_certified_trace_mu` (WitnessInsightGeneral.v).

23.5 The finite fragment that pays for its merges (Section 4)

Section 4's room argument (Theorem 4.2: finiteness, permanence and merge pricing force the toll) needs a finite state space, and Section 16 told you the counter has no

ceiling, so the twelve-field state space is infinite. Here's how that's faced rather than hidden.

Build a finite machine: four slots times a flag, eight states. Three moves: stamp (set the flag, go to the next slot, cost one), jump to slot a (cost two), next slot (cost zero). On it, every one of Section 4's premises is a theorem, not a hypothesis: finite (fin_finite), permanent (fin_permanent), the next-slot move forgets nothing (fnext_injective), the stamp and the jump merge (fcertify_merges , fjump_merges), merges are priced ($\text{fin_merging_priced}$), and per halving ($\text{fin_compression_priced}$). So Section 4's two conclusions hold on it outright ($\text{fin_a2_from_merging_price}$, $\text{fin_a2_from_compression_price}$). The counting lemma underneath, that a duplicate-free list with at most K preimages per image has at most K times as many members as images, is proved generically ($\text{fiber_bound_compression}$).

Then the bridge. Take a window on the twelve-field machine: the program counter modulo four as a slot, and the certified flag. Map the three moves to CERTIFY with declared cost zero (the floor makes the charge one, which is what the finite machine charges the stamp), JUMP with declared cost two, and a CHECKPOINT with declared cost zero. Then for every state and every move, the window of the stepped state is the finite machine's step of the window ($\text{vm_runs_finite_machine}$), the counter rises by exactly the finite machine's cost ($\text{vm_pays_finite_price}$), both over runs ($\text{vm_runs_finite_trace}$), and a run of that fragment from unflagged to flagged raised the counter by at least one ($\text{vm_fragment_certification_paid}$). So Section 4's argument applies to this machine *through a window onto a finite fragment*. That's the honest form. One limit goes with it: the jump's charge of two is a choice the program makes when it writes the instruction. The machine's schedule allows it and doesn't force it.

Theorem 23.8 (The machine runs the finite fragment). *Let $\text{win}(s) = (\text{pc}(s) \bmod 4, \text{flag}(s))$, and send the finite moves stamp, jump to a , and next to CERTIFY with declared cost 0, JUMP to a with declared cost 2, and CHECKPOINT with the empty label and declared cost 0. Write $\text{run}(t, s)$ for the machine run of the images of a list t of finite moves, and frun , fcost for the finite machine's run and total cost (stamp 1, jump 2, next 0). For every t and every state s ,*

$$\text{win}(\text{run}(t, s)) = \text{frun}(t, \text{win}(s)), \quad \mu(\text{run}(t, s)) = \mu(s) + \text{fcost}(t).$$

If moreover $\text{flag}(s)$ is false and $\text{flag}(\text{run}(t, s))$ is true, then $\mu(\text{run}(t, s)) \geq \mu(s) + 1$.

Proof. One step at a time, by computation: CERTIFY 0 moves the program counter on by one and sets the flag, JUMP sets the program counter to a and keeps the flag, CHECKPOINT moves on by one and keeps the flag, which are the three finite moves read through the window; conservation gives the rises 1, 2 and 0. Runs follow by induction on t . For the last part, the window run goes from an unflagged to a flagged

finite state, and on the finite machine every move that raises the flag costs at least one, which is Section 4’s toll proved there from finiteness, permanence and merge pricing (all three theorems on eight states). $\square$

Checked as `vm_runs_finite_machine`, `vm_pays_finite_price`, `vm_runs_finite_trace` and `vm_fragment_certification_paid` (`FiniteCertMachine.v`, 265, 286, 302); the finite toll is `fin_a2_from_merging_price`.

And on the full machine, Section 4’s merge-or-revoke (Theorem 4.3), directly: any step that turns the flag on is non-injective and costs at least one (`vm_certifying_step_is_priced_merge`), and JUMP 0 0 is non-injective and costs zero (`vm_jump_is_free_merge`). So the full machine prices the certifying merge and leaves other merges free; it does *not* satisfy “every merge is priced” (`vm_prices_certifying_merge_leaves_others_free` states all three). That’s the sharpest sentence in this subsection and it’s the one to carry: the machine’s schedule puts the floor on the flag, not on forgetting in general, and Section 4’s physics premise, if you grant it, would charge the jump too. The machine is honest about certification and silent about everything else it forgets.

In the words of Section 4 (Definition 4.1 and Premise 4.2), on the full machine with step apply and schedule cost:

Theorem 23.9 (The certifying merge is priced; a free merge exists). (1) For every declared cost d , CERTIFY d merges. (2) For every state s and instruction i , if $\text{flag}(s)$ is false and $\text{flag}(\text{apply}(s, i))$ is true, then i merges and $\text{cost}(i) \geq 1$. (3) JUMP to 0 with declared cost 0 merges and cost of it is 0. (4) Hence the full machine does not satisfy merge pricing.

Proof. (1) Take the initial state with the flag down and the same state with the flag up. CERTIFY sends both to the same state: its result does not read the old flag (Coq closes this by computation). The two starts differ, so CERTIFY is not injective. (2) Only CERTIFY turns the flag on, so i is CERTIFY d for some d ; it merges by (1) and costs $d + 1$. (3) The initial state with program counter 0 and with program counter 1 both land at program counter 0, with nothing charged. (4) Merge pricing applied to the jump of (3) would demand a cost of at least one. $\square$

Checked as `vm_certify_merges` (`FiniteCertMachine.v`), `vm_certifying_step_is_priced_merge`, `vm_jump_is_free_merge` and `vm_prices_certifying_merge_leaves_others_free`, which states (2), the existence of a free merge, and (4) as one conjunction.

23.6 Learning for free (Section 5)

Section 5’s demon (Theorem 5.2: an observer can learn for free) on this machine: load a hidden value x into register one, with the observer watching register two. For x zero and x one the observer sees the same thing (register two is zero either way). Now run

XOR_ADD of register one into register two at declared cost zero: the observer sees x . The instruction's cost is zero, the counter is unchanged, both runs started at zero (`vm_observer_narrowing_at_zero_cost`, proved by computing). One bit learned, nothing paid. Unconditional, in the sense that no physics premise is involved. I'll also say what the theorem doesn't say: the file's small abstract demon never merges two states, but I haven't stated that for this machine's step.

23.7 Exact-price collisions (Section 7)

Section 7's clerk (Theorem 7.2: no exact window price across a collision), on this machine. From the clean state (empty registers and memory, counter zero, uncertified), CERTIFY with declared cost zero and JUMP to one with declared cost zero show the same `bare_observable` before and the same after, and the first raises the flag (`vm_bare_observable_collision`). The counters differ afterwards, one against zero, because CERTIFY pays the floor, and `bare_observable` doesn't look at the counter. For `forget`, which keeps the counter, give the JUMP a declared cost of *one* so the charges match: same before, same after, one flips (`vm_forget_collision`). So no price computed from either window both meets the floor and never overcharges (`vm_bare_shadow_cannot_price_exactly`, `vm_forget_shadow_cannot_price_exactly`). The exact price of certification on this machine lives in the step, or in a window that shows the flag. And the characterisation: the floor and no-overcharge hold together exactly when the charged predicate is the flip predicate at unit price (`exact_commitment_pricing_characterization`).

The words, and then the result. Write $\text{flips}(s, i)$ for “flag(s) is false and flag($\text{apply}(s, i)$) is true”.

Definition 23.3 (Shadow price, floor, no overcharge, collision). Fix a window w . A *shadow price* is a cost of the form $c(s, i) = p(w(s), w(\text{apply}(s, i)))$ for some function p of the observed pair. A cost c *meets the floor* when $\text{flips}(s, i)$ implies $c(s, i) \geq 1$, and *never overcharges* when $c(s, i) = 0$ whenever $\text{flips}(s, i)$ fails. A *collision* for w is two steps (s_1, i_1) and (s_2, i_2) with $w(s_1) = w(s_2)$, $w(\text{apply}(s_1, i_1)) = w(\text{apply}(s_2, i_2))$, $\text{flips}(s_1, i_1)$, and not $\text{flips}(s_2, i_2)$.

Theorem 23.10 (Neither window prices certification exactly). (1) *bare has a collision: from the clean state, CERTIFY with declared cost 0 against JUMP to 1 with declared cost 0.* (2) *forget has a collision: from the clean state, CERTIFY with declared cost 0 against JUMP to 1 with declared cost 1.* (3) *For any window with a collision and any p , the shadow price from p does not both meet the floor and never overcharge. So no shadow price from bare, and none from forget, is exact.*

Proof. (1) and (2) are computed. The clean state has program counter 0, empty registers and memory, counter 0, and the flag down. Both steps end at program counter 1 with registers and memory untouched; CERTIFY raises the flag and charges $0 + 1$, the jump

raises nothing. In (2) the jump charges 1, so the counters agree as well. (3) Meeting the floor charges the certifying step at least one. The other step shows the same observed pair, so the shadow price charges it the same amount, and not overcharging demands zero for it. $\square$

Checked as `vm_bare_observable_collision`, `vm_forget_collision` (`ShadowPricing.v`, 154), the general `shadow_cannot_price_exactly`, and its two instances `vm_bare_shadow_cannot_price_exactly` and `vm_forget_shadow_cannot_price_exactly`. The other direction is Theorem 7.2: a window that shows the flag has a shadow price that is exact (`window_showing_reading_prices_exactly`) and no collision (`window_showing_reading_has_no_collision`).

23.8 The verifier and its three escapes (Section 7)

Section 7 told the verifier argument with a toy: two runs that leave the same transcript, one that paid and one that didn't, and a verifier that has to give them one verdict. Here is the pair on this machine, and the two theorems with it. Start the machine at its clean state (empty registers and memory, counter zero, uncertified). State *A* is where it lands after one CERTIFY with declared cost zero, which charges one; its counter reads 1. State *B* is where it lands after one PNEW with an empty region (a new module with no memory cells in it), declared at cost zero; its counter reads 0. The bare window of Definition 23.2 shows the same thing on both runs: the same start, then the same memory, the same registers, the same program counter. The *bare transcript* τ is that pair of photographs. A *verifier* V takes a transcript and says accept or reject. It is *sound* when every transcript it accepts comes only from states where the claim holds, and *complete* when every transcript from a state where the claim holds is accepted. The claim is “the counter is 1,” and τ is explained by exactly two states, *A* and *B*.

Theorem 23.11 (No sound and complete verifier on the bare window). *For the claim “the counter is 1,” with τ explained by *A* and by *B* and by nothing else, no verifier on bare transcripts is both sound and complete.*

Proof. Suppose V were both. The counter at *A* is 1 and *A* explains τ , so completeness says V accepts τ . Now soundness, on that same accepted τ : every state that explains it satisfies the claim, and *B* explains it. So the counter at *B* is 1. It is 0. $\square$

Checked as `bare_setting_no_sound_complete_verifier` (`VerifierImpossibility.v`), with the two states `pol_state_A` and `pol_state_B` from `NecessityOfMuLedger.v`; no axioms.

Now dress the transcript up. Let T be any richer kind of transcript, with a way π to squash one down to its bare photographs, and a relation saying which states explain which transcripts. A verifier V on T *factors through* π when it only looks at the squashed part: $V = V' \circ \pi$ for some rule V' on bare transcripts (the small circle means

“squash first with π , then apply V' ”), or equivalently, two transcripts with the same π always get the same answer.

Theorem 23.12 (A richer verifier that only looks at the bare part does no better). *Suppose there are two transcripts t_A, t_B in T with $\pi(t_A) = \pi(t_B)$, t_A explained by A , and t_B explained by B . Then no verifier on T that is sound and complete for “the counter is 1” factors through π .*

Proof. If $V = V' \circ \pi$, then $V(t_A) = V'(\pi(t_A)) = V'(\pi(t_B)) = V(t_B)$. Completeness at t_A , which A explains with counter 1, gives $V(t_A) = \text{accept}$. So $V(t_B) = \text{accept}$, and soundness at t_B , which B explains, says the counter at B is 1. It is 0. $\square$

Checked as `V_does_not_factor_through_classical` (VerifierExhaustiveness.v), with every premise an explicit hypothesis and the two states fixed to A and B . The pair t_A, t_B is a premise, and it’s the whole condition: the collision has to survive into the richer transcript type. A transcript type with no such pair doesn’t meet the premise, and the theorem says nothing about it. So it doesn’t classify every transcript design.

Here are the three constructions, with the relations written down. The relation matters: it says which state a transcript is allowed to describe. If it quietly assumes a truthful answer, the verifier has not earned the right to trust a liar.

Definition 23.4 (Three richer transcripts and their explanations). Keep the states A, B and their shared bare transcript τ from above, with $\mu(A) = 1$ and $\mu(B) = 0$. Write $E_0(s, t)$ for $t = \tau \wedge (s = A \vee s = B)$. The claim remains $\mu(s) = 1$.

A *full-state transcript* is a finite list t of machine states. Define $E_S(s, t)$ to mean that t is nonempty and its last entry is s . The verifier rejects the empty list; otherwise it accepts exactly when the last entry’s counter is 1.

A *commitment transcript* is a pair (t, b) of a bare transcript and a Boolean. For any chosen explanation relation E_C , the *commitment contract* is the pair of implications, for all s, t, b :

$$E_C(s, (t, b)) \wedge b = \text{true} \implies \mu(s) = 1,$$

$$E_C(s, (t, b)) \wedge \mu(s) = 1 \implies b = \text{true}.$$

These are binding and honesty, respectively. The verifier accepts exactly when b is true. One relation meeting the contract is $E_H(s, (t, b)) \iff E_0(s, t) \wedge (b = \text{true} \iff \mu(s) = 1)$.

A *response transcript* is a pair (t, r) of a bare transcript and a natural number. Define $E_I(s, (t, r)) \iff \mu(s) = r$. The verifier accepts exactly when $r = 1$. This relation

imposes no condition on the bare component t ; the response alone carries its entire truth condition.

Each construction assigns verification the constant abstract cost 1.

Theorem 23.13 (Three conditional escapes). *The full-state verifier is sound and complete relative to E_S . For every E_C satisfying the commitment contract, the commitment verifier is sound and complete relative to E_C . The response verifier is sound and complete relative to E_I . Each has cost 1 under the stipulated cost assignment. The honest relation E_H satisfies the commitment contract; the unchecked relation $E_U(s, (t, b)) \iff E_0(s, t)$ does not.*

Proof. For the full-state verifier, an explaining state is precisely the last entry. Reading that entry's counter therefore accepts exactly when the explaining state's counter is 1. An empty transcript is rejected and has no explaining state.

For the commitment verifier, acceptance says b is true. Binding then gives the claim for every explaining state, which is soundness. Conversely, honesty says that any explaining state satisfying the claim has a true bit, which the verifier accepts. The equivalence in E_H gives both implications directly. Under E_U , however, B explains (τ, true) even though $\mu(B) = 0$. That violates binding.

For the response verifier, explanation gives $\mu(s) = r$. Acceptance adds $r = 1$, so $\mu(s) = 1$. Conversely, $\mu(s) = 1$ and explanation give $r = 1$, hence acceptance. The three cost claims follow from the assigned constant. $\square$

Checked as `substrate_escape_succeeds (VerifierEscape_Substrate.v)`, `commitment_contract_verifier`, `honest_commitments_satisfy_contract`, `unchecked_bit_violates_contract (VerifierEscape_Hardness.v)`, and `interactive_escape_succeeds (VerifierEscape_Interaction.v)`.

The collision has disappeared because the extra information separates its two states. The full-state transcript shows the counter. The honest commitment for A has a true bit and the one for B a false bit. Their honest responses are 1 and 0. None asks a verifier to distinguish identical inputs.

That is also the limit. A list of states has no authentication built into it. The commitment contract is assumed, and the unchecked bit shows why the assumption matters. The response relation allows only a truthful counter value, so asking a question here does not establish resistance to a cheating prover. And cost 1 is an abstract price, not a measurement of the work needed to authenticate a state, check a cryptographic proof or communicate a response. These constructions exhibit the extra distinctions a verifier can use. They supply no adversary model, security parameter or hardness reduction.

23.9 Records on this machine (Section 10)

Section 10 showed, over any base, that an honest record is a latch, and argued that two things stay free: the price list and the event. Every checked statement of those two freedoms is about this machine, so here they are. First the latch itself: Section 10's Theorem 10.1 holds with this machine's unbounded step as the base, for each program, halting when the program counter is past the end (`record_axis_is_latch_on_vm_holds`, `CrossBaseGranularity.v`). So this machine with an honest record is this machine plus a cup, the same as the three outside bases.

Then the strong reading of uniqueness: is every honest extension of this machine *the same machine* as this one? In the form the words suggest it's *false*, and the counterexample is Section 10's surcharged cup, made real. Take this machine and add a step counter to the ledger, so every step costs one more than it did. It's honest: the toll is a *floor*, and a surcharge only raises the price. It extends the base. And it is not the same machine, because this machine with an empty program sits still and pays nothing per step, and the billed one pays at least one per step for sitting still (`adequate_core_uniqueness_refuted`, `honest_vm_extension_uniqueness_refuted`; the counterexample is `BilledCore`). A machine that charges for CPU time is honest and isn't this one. One thing about how those two theorems are stated, so the five rules don't quietly change on you: neither is written with the five rules of Section 10 (Definition 10.2). The first uses a looser test of "honest" (a ledger that never falls, the toll, a run that reaches the record being on, and a clause about halting: for every two-counter machine, some start state has to halt exactly when that machine does). That's Minsky's two-counter machine, the one Section 14's small machine is built on, with its halting problem borrowed there from the Coq undecidability library; here it only has to be matched, and Section 24 runs one as an interpreter on this machine. The second is written for machines that run this machine underneath, with a map onto its states standing in for the "driven by the computation" rule. Slightly different lists, the same spirit.

Here are both lists, and the sameness they're tested against. One word in the first definition needs saying: the machine has two step functions, and the *unbounded* one keeps every number in its registers and memory exactly, instead of cutting it to a fixed width the way a chip has to. The files use that one here. And "halts" gets a convention: the program counter has run off the end of the program with register nine reading one, which is the "guest halted" status the two-counter interpreter of Section 24 writes there, so this machine's halting lines up with the two-counter problems in the adequacy clause. One letter, too: in this section M and N are record-carrying machines in Section 10's sense, not the trace fold's target from earlier in the chapter.

Definition 23.5 (This machine as a record-carrying machine). The *Thiele core* has states (p, s) , a program and a machine state. Its step runs one instruction of p from s on the unbounded machine, and leaves (p, s) alone when the program counter points

past the end of p . Every state is a starting state. Its record is the certified flag $\text{cert}(s)$, its ledger is $\mu(s)$, and it halts when the program counter equals the length of p and register 9 holds 1.

Definition 23.6 (Adequate; same core). A record-carrying machine M is *adequate* when (R2) and (R3) hold, some run from a starting state reaches a state with the record on, and for every two-counter machine problem P some starting state m_P has: P halts exactly when some $\text{next}_M^k(m_P)$ halts. That last clause is an existential match, one problem at a time, not a simulation in either direction. M and N have the *same core* when some relation R between their states relates every starting state of each to some starting state of the other, and any related m, n have the same record, the same next-step cost, and related next states. R need not be a function, so state one machine keeps and the other doesn't can be ignored.

Definition 23.7 (Honest extension of this machine; observed sameness). M is an *honest extension of this machine* when there is a cover of the Thiele core by M (Definition 10.1, with the Thiele core as the base) and (R2) to (R5) hold. M and N are *observed the same* when some R relates starting states both ways as above, and any related m, n have the same record, the same ledger value, the same halting, the same next-step cost, and related next states.

Theorem 23.14 (Exact uniqueness is false). *Let the billed core have states (p, s, k) , step $(p, s, k) \mapsto (p, s', k + 1)$ where (p, s') is the Thiele core's step from (p, s) , every state starting, record $\text{cert}(s)$, ledger $\mu(s) + k$, and the Thiele core's halting. It is adequate and it is an honest extension of this machine, and it has neither the same core as the Thiele core nor is it observed the same.*

Proof. Every step of the billed core costs at least one. The Thiele core's starting state $([], s_0)$, empty program and initial machine state, never moves and costs 0 per step. Any relation of either kind must relate it to some billed state, and the two must have the same next-step cost: 0 against at least 1. Adequacy and honesty are checked clause by clause, the halting clause by the two-counter interpreter that already runs on the Thiele core. $\square$

Checked as `billed_core_adequate`, `billed_core_honest_extension`, `billed_core_not_equiv`, `billed_core_not_observed_equiv`, `adequate_core_uniqueness_refuted` and `honest_vm_extension_uniqueness_refuted` (`StructuralUniqueness.v`), with the definitions `Adequate`, `core_equiv` (`StructuralCore.v`), `ComputationalCover`, `HonestVMExtension` and `observed_core_equiv` (`StructuralCoreCover.v`), and the Thiele core's own adequacy `thiele_core_adequate` (`StructuralCore.v`).

Keeping more history doesn't make a machine different under this sameness, and it's

worth seeing why the relation has to be allowed to forget.

Theorem 23.15 (History is forgotten by the relation, and only by it). (1) *The machine that runs the Thiele core and keeps a list of every state it has passed through has the same core as the Thiele core.* (2) *Consider instead a target that is fed instruction lists one instruction at a time, keeps the whole list it was fed, charges the price list, and reads certification by running its list on this machine from the initial state. Its reading agrees with this machine's on every list, and the price list obeys the toll on it, yet it does not factor through this machine's state: the empty list and the list holding one free jump to address 0 leave this machine in the same state and leave the target in different ones.*

Proof. (1) Relate (p, s, H) to (p, s) ; records, costs and steps match because the history is never read. (2) Call the free jump to address 0 j . From the initial state it costs 0 and sets the program counter to the 0 it already had, so the two lists give the same machine state, and the target's states are the lists themselves, $[]$ and $[j]$, which differ. $\square$

Checked as `history_core_equiv_thiele` (StructuralCore.v) and `certification_agreement_does_not_imply_descent` (TraceStateDescent.v), the third target of Theorem 23.4.

So agreeing with this machine on every certification reading doesn't make a machine a function of this machine's state. When a history-keeper counts as the same as this machine, it's because the relation was allowed to drop the history, and for no other reason.

But the billed machine isn't a different *structure*. It's the same machine with a different price list. So the claim worth keeping is uniqueness *up to the price schedule*: compare two machines by their record light, their halting, and their steps, through their maps down to this machine, and don't compare charges. Here I have to be plain about how much that claim says. Every machine it covers runs this one underneath, and "the same machine" means each state is related only to the state of this machine that its own map assigns it. So the result says the extra bookkeeping and the price list don't matter. It doesn't say unrelated machines coincide. With the record fixed to *this machine's own certification reading* (the certified flag: the bit only CERTIFY sets, which carries no object and no check), uniqueness up to schedule holds (`cert_record_schedule_uniqueness_holds`): the billed machine is an instance, and so is any machine that adds to the bill any amount that's a function of the program and the current state. (The flat one-per-step bill is the separate `BilledCore`.) One structure, many price lists.

Definition 23.8 (The same up to the schedule). Let M cover the Thiele core through π . M is *priced* when (R2) and (R3) hold. M is *the same as the Thiele core up to the schedule* when both are priced and some relation R satisfies: $R(m, t)$ only when $t = \pi(m)$; every

starting state of each side is related to some starting state of the other; and related states have the same record, the same halting, and related next states. Charges are not compared. M is an *honest certification extension* when $\text{rec}(m) = \text{cert of } \pi(m)$ for every m , M is priced, and (R4) and (R5) hold. It is an *honest tied extension* when instead $\text{rec}(m) = g(\pi(m))$ for some one function g , with the other clauses the same.

Theorem 23.16 (Unique up to the schedule, for the certified flag). *Every honest certification extension is the same as the Thiele core up to the schedule. In particular, so is the billed core, and so is the machine that runs the Thiele core and adds $e(p, s)$ to a side counter at every step, with ledger $\mu(s)$ plus the side counter, for any function e from programs and machine states to whole numbers.*

Proof. Take $R(m, t)$ to mean $t = \pi(m)$. The cover gives the two start conditions, halting, and the step clause; the record clause is the hypothesis $\text{rec}(m) = \text{cert of } \pi(m)$. Pricedness is assumed for M and proved for the Thiele core (conservation, and the toll for the certifying step). The proof is short because the definition pins R to the cover: the content is in the hypotheses, which is the point made in the text above. $\square$

Checked as `cert_record_schedule_uniqueness_holds`, `billed_core_equiv_mod_schedule`, `surcharged_core_honest_cert` and `surcharged_core_equiv_mod_schedule` (`StructuralScheduleUniqueness.v`), with the definitions `record_is_certification`, `priced`, `HonestCertExtension`, `HonestTiedExtension` and `equiv_mod_schedule_via` (`StructuralCoreSchedule.v`), and `thiele_core_priced`.

And then the second free choice, Section 10's two cups on one die, on this machine. Let the record be *any* permanent reading that a paid step writes and that obeys the toll, instead of the certification reading in particular. Now uniqueness *fails* (`tied_record_schedule_uniqueness_refuted`). The counterexample: let the record be "the counter has reached one." That's permanent, a paid step switches it on, the toll holds for it. And at a state where the counter is one and nothing is certified, that record says on and the certification record says off. Two honest records, same machine, disagreeing: Corollary 10.7 with a real machine underneath.

Theorem 23.17 (Not unique for a tied record). *Let the meter core be the Thiele core with record " $\mu(s) \geq 1$ " and the identity as cover. It is an honest tied extension and it is not the same as the Thiele core up to the schedule. So uniqueness up to the schedule fails once the record is only required to be some reading of the computation.*

Proof. The record is $g(\pi(m))$ with $g(p, s) = [\mu(s) \geq 1]$. It is priced because the ledger is the Thiele ledger and a step that turns it on raises μ from 0 to at least 1; permanent because μ never falls; and written by the certifying step from the initial state. Now take the starting state $([], s_1)$ with s_1 the initial machine state except $\mu = 1$, flag off.

Any R must relate it to some starting state of the meter core, and only to itself, since the cover is the identity. There the meter core's record is on and the Thiele core's is off. $\square$

Checked as `meter_core_honest_tied`, `meter_core_not_equiv_mod_schedule` and `tied_record_schedule_uniqueness_refuted` (`StructuralScheduleUniqueness.v`).

23.10 Pricing and verifying on this machine (Section 12)

Section 12 read five real systems at three levels (specification, transcription, wrapper) and said some of the wrappers are written for this machine. Here they are, together with the one sentence about pricing this machine's own flag.

First the gas wrapper's class, on this machine. Price the certified flag at one per raise and nothing otherwise, and both halves of Section 12's gas-schedule exactness hold, the floor and no overcharge (`thiele_vm_commit_pricing_is_exact`, `GasMetering.v`, through Theorem 12.4). That exact schedule isn't this machine's own counter, which also bills computation that commits nothing; on every raise the unit price is at most what the counter charges (`thiele_unit_price_lower_bounds_mu`). It's the same split the exact-price subsection above drew with collisions: the exact price of certification lives in the step, and the counter is a bigger bill that contains it.

The attestation wrapper is a separate file, built on the verifier pair A and B of the previous subsection. A report there is a bare transcript plus one measurement register, and an honest report of a state carries a transcript that state explains and a register equal to its μ .

Theorem 23.18 (Attestation cannot factor through the bare transcript). *Let V be a verifier on reports that is sound (it accepts only reports every honest explanation of which has $\mu = 1$) and complete (it accepts every honest report of a state with $\mu = 1$). Then V is not a function of the bare transcript: there are reports with equal transcripts on which V differs.*

Proof. The pair A, B gives two reports with the same transcript, one honestly explaining a state that paid ($\mu = 1$) and one a state that did not ($\mu = 0$). Completeness makes V accept the first; soundness makes it reject the second. $\square$

Checked as `attestation_cannot_factor_through_bare_transcript` (`TEEAttestation.v`), an instance of `V_does_not_factor_through_classical` (Theorem 23.12); the same file has the pair itself (`replay_is_a_two_preimage_witness`) and the positive side, a sound and complete verifier at unit cost once the register is readable (`measurement_enriched_attestation_succeeds`). The register's faithfulness is a premise of the model, not a fact about hardware. Section 12's quote fragment has two more statements that lean on this machine's transcript type:

`quote_projection_faithful` says the quote, written into the machine's bare-transcript format, loses nothing, and `quote_runtime_verifier_separation` says no verifier correct for the platform's runtime flag depends only on that encoding (`TPMQuoteGap.v`; the separation is `V_does_not_factor_through_classical` again, with the pair A, B as its fixed witnesses).

The transparency wrapper is Section 7's commitment escape in log vocabulary. Under a *disclosure contract* (a premise: the inclusion bit is set on every honest transcript of a run with $\mu = 1$, and a set bit binds every explaining run to $\mu = 1$), there is a verifier that reads the log's inclusion bit, is sound and complete for the claim $\mu = 1$, and costs one unit (`abstract_log_bit_verifier`, `TransparencyLog.v`). Delete the log and no verifier on the bare transcript is sound and complete for that claim at any cost (a `bstrack_bare_verifier_impossible`, which is Theorem 23.11 under another name), with the collision pair behind it exhibited as `abstract_mu_collision_witness`. The pair has the shape of a split view, one transcript for two runs; it is not an RFC split view, which is two inconsistent log histories.

The proof-carrying wrapper is Section 7's interaction escape. A transcript that carries the certificate admits a checker, sound and complete for $\mu = 1$ at unit cost (`proof_rounds_escape`, `ProofCarryingVerifier.v`); strip the certificate and no checker on the bare transcript is sound and complete at any cost (`bare_pcc_impossible`). And the prover's side has a floor. A *script* is a program run from the initial state with a fixed fuel, and it's *level k* when it ends with the certified flag up and one of the CERTIFY steps it executed was charged at least k . Then its counter rises by at least k , and for every $k \geq 1$ a single CERTIFY charged k does it at exactly k (`level_k_verification_floor`, `level_k_verification_floor_tight`). The first is `level_k_certification_cost_floor` (`MuHierarchyTheorem.v`) by exact application, which is the lower half of Section 26's mu-hierarchy, Theorem 26.8, so the formal statement and proof live there. The floor prices the run's certification events. It says nothing about proof size, prover time, or the checker's work.

In each of the three, the unit cost is stipulated, and none of them prices or verifies the real system it borrows a name from. How to knock them down: for attestation, a sound and complete verifier on the bare transcript; for transparency, a sound and complete verifier that needs no log, for the same claim; for proof-carrying code, a script of level k whose counter rises by less than k , or a sound and complete checker with no certificate.

23.11 The certified flag as a durable consensus (Section 13)

Section 13 showed that agreement among observers doesn't buy permanence and durability does. Here's the instance it pointed at: this machine's certified flag is the durable case.

Theorem 23.19 (The certified flag is a permanent consensus). *For any $N > 0$ and any*

instruction i , the game of Definition 13.2 whose states are machine states, whose event and every view are the certified flag, and whose step is the unbounded machine step by i , has a permanent event.

Proof. Authenticity holds by definition, and durability is the kernel fact that no unbounded step turns the certified flag off; apply part (2) of Theorem 13.5. $\square$

Checked as `vm_certification_is_permanent_consensus` (EcosystemGame.v); the durability fact is `vm_apply_u_certified_permanent`. That one is for a run that repeats a single fixed instruction, with every observer reading the same bit. It says nothing about the assertion channel: no theorem says the channel stays set, and an assertion with an empty property writes a zero to it.

23.12 What landed

Every pointer from Parts II and III that said “on the machine” now has a name. What’s *not* here, and stays not here: a single either-reading run theorem; “every merge priced” on the full machine (false, by the free jump); a statement of Section 6’s counter facts or Section 10’s two free choices over an arbitrary base (those are argued in their chapters, checked only here); and any claim that the small lifted model is the twelve-field machine. The machine is a witness. It’s not a bigger claim than the theorems on it.

24 Interpreters and the guest machine

Section 11 left the diagonal standing on any substrate, on a little family built to fit it, and on L, and said the machine is one more substrate of that kind, conditionally. The machine is on the table now, so here it is, in three pieces. First the wall pointed at the full machine, where it stands under a premise nobody has filled in, and where a decider standing outside exists and is checked. Then two interpreters, programs of the machine that run a smaller machine handed to them as data. Then the guest, where the recursion theorem is proved rather than assumed. I’ll say which interpreter the recursion theorem lives on, because nobody can check a claim about “the interpreter” if there are two.

24.1 The diagonal, pointed at the machine

Here’s Section 11’s question, asked of this machine. The machine carries a record; the record is a latch on an event (Section 10); for my machine the event is a certification, and Section 22 said a certification reading, with its receipt, is what lets an auditor treat a shortcut as *admissible*. So: can the machine have, as one of its own programs, a procedure that correctly decides, for every program, “does this program admit a sound shortcut”? That would be autopilot. Hand it any program and it says yes or no, and it’s always right.

That’s the question in plain words, and the checked statement is narrower, so here it is before the proof. In the checked work “admits a shortcut” is a concrete property of

programs: its bounded run from the start state ends in the same state as the bounded run of one fixed reference program, a three-instruction one that allocates a group, makes its identity arrow (Section 2's do-nothing arrow from a thing to itself), and asserts it. Two programs that behave alike agree on the property, and there's one program that has it and one that doesn't. The diagonal doesn't see soundness. For that one reference program the checked work does show the property lining up with a sound-shortcut record from Section 22. For other programs it says nothing of the kind, and a procedure that returns soundness records for some programs, or only for restricted inputs, or with outside evidence, is not what the wall rules out.

Here is that property exactly. Write s_{init} for the machine's initial state and $\text{run}_{1000}(p, s)$ for the state the machine reaches from s running p , cut off after a thousand steps.

Definition 24.1 (The machine's shortcut property). Let r be the three-instruction program that opens a group (a module) on cell 0, builds that module's identity arrow, and asserts the arrow just built; in Section 17's table these are PNEW, MORPH_ID and MORPH_ASSERT. A program p *admits the shortcut* when $\text{run}_{1000}(p, s_{\text{init}}) = \text{run}_{1000}(r, s_{\text{init}})$. The program r admits it; the empty program does not.

Checked as `vm_admits_shortcut_extensional`, with `vm_admits_shortcut_yes` and `vm_admits_shortcut_no` for the two witnesses (StructuralUndecidability.v; r is `simple_morph_trace` in `SimpleMorphShortcut.v`). The no-witness is a computation: the empty program leaves s_{init} where it is, and r does not. The link to Section 22's record is `vm_instantiation_target_witnessed`: for r alone, a sound-shortcut record exists.

Section 11's wall (Theorem 11.1) is about any substrate: programs, a run, a counter, an encoding of programs as data, a class of representable transformations, and a recursion theorem for that class. The machine supplies most of that on its own. It doesn't supply the last piece. Here's the honest form.

The *concrete machine* (`vm_structural_shortcut_undecidable`): the generic theorem applied to my machine's bounded runs (no decider says yes exactly when a program admits the shortcut), carrying as explicit hypotheses an encoding of programs as data, a class of representable transformations, and a recursion theorem for that class. (A companion statement, `vm_structural_shortcut_undecidable_encoded`, discharges the encoding with a proved round trip between programs and numbers, leaving only the representable class and its recursion theorem as premises.) Asked for every map on programs, that recursion premise is false, and the reason is the cutoff, not the size of memory: equality after a thousand steps is something you can check by running both programs, so the flip of that check has no fixed point. So the corollary needs a class that leaves the flip of every correct decider out, and with such a class it says exactly that: the flip of a correct decider has no program whose

thousand-step runs match its image's from every state. The proved recursion theorem further down is for the machine's guest, a smaller machine it runs as data, and it answers a different question.

Premise 24.2 (A recursion theorem for the full machine). A class Rep of maps on machine programs such that every f in Rep has a program q with $\text{run}_{1000}(q, s) = \text{run}_{1000}(f(q), s)$ for every state s . The class with only the identity map meets it (take q empty), and so does any class of constant maps. The class of all maps doesn't: Theorem 24.3 names a map with no such q .

Theorem 24.1 (The wall on the machine, conditional). *For every class Rep satisfying Premise 24.2, there is no Boolean function d on machine programs such that the flip of d (the empty program when $d(p)$ is true, r when it is false) is in Rep and, for every p , $d(p) = \text{true} \iff p$ admits the shortcut.*

Proof. Build the substrate from the machine's bounded run, with $\text{enc}(p)$ the initial state with the number of p stored in one of its number fields (the logic accumulator of Section 16) and dec reading it back (the round trip between programs and numbers is proved), the given Rep , and the premise as its recursion theorem. The predicate of Definition 24.1 is a shortcut predicate with witnesses r and the empty program, and it is extensional because it reads only the run from one state. Then Theorem 11.1 applies. $\square$

Checked as `vm_structural_shortcut_undecidable` (StructuralUndecidability.v, encoding as hypotheses) and `vm_structural_shortcut_undecidable_encoded` (VMSubstrateEncoded.v, encoding discharged by `vm_encode_decode_concrete`). In both, the decider ruled out is one that says true exactly when a program admits the shortcut. That the empty class meets the premise is logic, not a stated theorem.

Now the honest part Section 11 did with a pencil on its little family: *who counts as a decider*. The theorem rules out a decider whose flip the machine can itself represent. It says nothing about a decider standing *outside*. And here's a thing that surprised me about my own machine: its bounded runs (a thousand steps, then stop) make the shortcut predicate decidable from *outside*, by just running both programs and comparing. So an outside decider exists. What the theorem says is that such a decider's flip can't be one of the machine's own representable transformations, under the recursion premise. Same boundary as on Section 11's family, Richard's lesson again: the language you talk about the machine in is not the language it talks in.

Theorem 24.2 (Bounded runs are decidable from outside). *Let $d_{\text{out}}(p)$ be true exactly when the two states $\text{run}_{1000}(p, s_{\text{init}})$ and $\text{run}_{1000}(r, s_{\text{init}})$ are equal. Then d_{out} is a total Boolean function, and $d_{\text{out}}(p)$ is true exactly when p admits the shortcut. Moreover, let Rep*

be any class of maps on machine programs such that every f in Rep has a program q with $\text{run}_{1000}(q, s) = \text{run}_{1000}(f(q), s)$ for every state s . Then the flip of d_{out} (the empty program when $d_{\text{out}}(p)$ is true, r when it is false) is not in Rep .

Proof. Equality of machine states is decidable, since a state is a finite record of numbers, lists of numbers, and flags; comparing the two bounded runs is therefore a total function, and it is correct by its definition. If its flip were in Rep , then d_{out} would be a decider of exactly the kind Theorem 11.1 rules out, applied to the machine's bounded runs with that Rep (the instance is Theorem 24.1). $\square$

Checked as `vm_bounded_shortcut_decide_correct` and `vm_bounded_decider_flip_not_representable` (VMBoundedDecidability.v).

The same file says the two sentences I keep leaning on as theorems of their own, so you don't have to take my word for how they follow.

Theorem 24.3 (The full-machine recursion premise fails, and every correct flip has no fixed point). (1) *It is not the case that every map f on machine programs has a program q with $\text{run}_{1000}(q, s) = \text{run}_{1000}(f(q), s)$ for every state s .* (2) *Let d be any Boolean function on machine programs with $d(p) = \text{true} \iff p$ admits the shortcut, for every p , and let $f_d(q)$ be the empty program when $d(q)$ is true and r when it is false. Then there is no program q with $\text{run}_{1000}(q, s) = \text{run}_{1000}(f_d(q), s)$ for every state s .*

Proof. (2) If such a q existed, the class whose only member is f_d would satisfy Premise 24.2 and contain the flip of a correct decider, which Theorem 24.1 rules out. (1) If every map had such a q , the class of all maps would satisfy the premise and contain the flip of d_{out} , which is correct by Theorem 24.2; that theorem rules this out. $\square$

Checked as `vm_full_recursion_premise_refuted` and `vm_correct_flip_has_no_fixed_point` (VMBoundedDecidability.v), both closed under the global context. In Coq, (1) reads $\sim (\text{forall } f, \text{exists } q, \text{forall } s, \text{vm_run } q \text{ } s = \text{vm_run } (f \text{ } q) \text{ } s)$, and (2) reads $\text{forall } d, (\text{forall } p, d \text{ } p = \text{true} \iff \text{vm_admits_shortcut_extensional } p) \rightarrow \sim \text{exists } q, \text{forall } s, \text{vm_run } q \text{ } s = \text{vm_run } (\text{if } d \text{ } q \text{ then } [] \text{ else simple_morph_trace}) \text{ } s$, with `vm_run` the thousand-step run and `simple_morph_trace` the program r .

So the machine's property is decidable from outside, and the wall on the machine separates nothing from classical halting: it's about bounded runs, and the premise can't be granted for every map (part (1) of Theorem 24.3), and no class can both meet it and contain a correct decider's flip (part (2)). So the machine's version can't be knocked over that way, and it doesn't say much either: it's the diagonal applied to a property I can decide.

Take the thousand-step cutoff away and the question changes shape, so here's the run with no stopwatch on it. Unbounded halting is its own relation, and it has its own facts.

Definition 24.3 (Halting without a cutoff). Program p *halts at* r from s when there is some number k with $\text{run}_k(p, s) = r$ and the program counter of r is at or past the end of p .

Theorem 24.4 (A program halts at most once). If p *halts at* r_1 from s and *halts at* r_2 from s , then $r_1 = r_2$.

Proof. Let the witnesses be $k_1 \leq k_2$. Once the program counter is past the end, a step changes nothing, so $\text{run}_{k_2}(p, s) = \text{run}_{k_1}(p, s)$, and the two results are equal. The other order is symmetric. $\square$

Checked as `vm_halts_at` and `vm_halts_at_deterministic` (VMUnboundedExec.v). This relation is not the bounded run of Definition 24.1: the bounded run always returns a state, halted or not, and nothing identifies the two. The interpreters in the rest of this chapter work with unbounded runs too (under a sibling step I'll flag below): programs of my machine that run a smaller machine handed to them as data. I'll call the smaller machine the *guest*.

24.2 Why an interpreter at all

An interpreter is a program that reads another program as *data* and does what it says. It's one of the oldest tricks in computing, and it's the trick that makes the diagonal bite: the Contrarian card in Section 11 needs “hand me to the Checker,” which means a program has to be able to be *handed*, as a number, to another program. On this machine, a program is a list of instructions, and there's a proved round-trip encoding of programs as natural numbers (`program_to_nat` and `back`; the wall above used it). So programs can be data. What's left is to show that a program on the machine can *run* a program given as data, and to say exactly which programs.

A warning about what the round trip does and doesn't give you. The decoder is a Coq function, outside the machine. The encoding the wall above uses parks the program's number in the logic accumulator, a field no instruction ever reads, and there's a theorem that no fixed program can then report even the encoded program's certification bit through any observation that ignores that field (`no_logic_acc_encoded_interpreter`). It doesn't rule out a different encoding. The two interpreters below are different encodings: the code arrives as an ordinary number in an ordinary register, which is something a program can read.

Here it is exactly. Write $s[\text{acc} := n]$ for the state s with its logic accumulator replaced by n , apply for the sixty-four-bit step, and $\text{run}(F, q, s)$ for the program-counter runner:

F times, fetch the instruction of q at the program counter and apply it, stopping early when the program counter is past the end of q .

Lemma 24.5 (The accumulator commutes with every step and every run). *For every state s , instruction i , number n , program q and fuel F : $\text{apply}(s[\text{acc} := n], i) = \text{apply}(s, i)[\text{acc} := n]$ and $\text{run}(F, q, s[\text{acc} := n]) = \text{run}(F, q, s)[\text{acc} := n]$.*

Proof. The step form is a case split over every instruction: none reads the accumulator and none writes it. The run form is induction on F , using the step form at each fetch; the fetch reads only the program counter. $\square$

Checked as `vm_apply_logic_acc_commut`es and `run_vm_logic_acc_commut`es (VMEncodedInputAccess.v).

Theorem 24.6 (No interpreter for the accumulator encoding). *Say q halts from s showing b under o , for a test o from states to booleans and a bit b , when for some F the state $\text{run}(F, q, s)$ has its program counter at or past the end of q and o of it is b . Let $\text{enc}(p)$ be the initial state with the accumulator set to the number of p . For every program I and every test o with $o(t[\text{acc} := n]) = o(t)$ for all t and n , it is not the case that for every program p and bit b , I halts from $\text{enc}(p)$ showing b under o exactly when p halts from the initial state showing b under the certified flag.*

Proof. Suppose I did. The program consisting of CERTIFY 0 halts from the initial state with the flag up, so I halts from its encoding showing true. By Lemma 24.5 the run of I from the encoding of the empty program is the same run with a different accumulator, and o ignores the accumulator, so I also halts from the encoding of the empty program showing true. Then the empty program would halt from the initial state with the flag up; it halts at once with the flag down. $\square$

Checked as `vm_apply_logic_acc_commut`es, `run_vm_logic_acc_commut`es, `halts_observ`es_logic and `no_logic_acc_encoded_interpreter` (VMEncodedInputAccess.v, 33, 47, 56).

Both interpreters below run under the machine's *natural-number* sibling (`vm_apply_u`, `run_vm_u`). That's a second execution model and I'll flag it: same instruction type, same twelve-field record, but writes and arithmetic keep the exact value instead of cutting it to sixty-four bits, and subtraction stops at zero instead of wrapping around. The guest registers are unbounded because of it. The two steps are separate definitions, and no theorem here ties the sixty-four-bit machine to its unbounded sibling.

24.3 The first interpreter: a two-counter machine as guest

A two-counter machine (Minsky's, from the 1960s) has a program counter and two counters that hold naturals; its instructions are: halt, increment one or the other, and

“decrement-or-jump” on one or the other (if the counter is zero, step on; otherwise decrement and jump to a target). That’s all, and once the input is encoded as a number it’s enough to compute anything a Turing machine can, which is why it’s the classic small target for undecidability proofs.

The machine hosts one. Each guest instruction is packed into a word: opcode in the low three bits, jump target in the rest, and the words are packed into one number at a width chosen from the program. The host program is a fixed list of sixty of the machine’s instructions, all at declared cost zero, with the terminal program counter at sixty and a status register that reads one for “guest halted” and two for “malformed guest word.” What’s proved, with none of it carrying an axiom in the last assumption audit (Section 23): each guest step is simulated by a positive number of host steps landing in a state that represents the new guest configuration (`cm2_uniform_interpreter_simulation_total`); the guest halts with a given final configuration if and only if some fuel brings the host to the halted status with that result (`cm2_uniform_interpreter_raw_correct_total`); the canonical encoding never reaches the malformed status (`cm2_uniform_interpreter_raw_never_malformed`); and the guest diverges if and only if the host program counter never reaches sixty at any fuel (`cm2_uniform_interpreter_raw_divergence_total`). Through that, the two-counter halting problem maps onto “does this host run ever reach the halted status” (`mm2_halting_host_iff`), and so, in the synthetic sense of the Saarland library of undecidable problems that Section 11 met beside L (what “synthetic” buys is spelled out in the Rice part below), “does this host run ever halt” is undecidable (`actual_unbounded_host_synthetic_undecidability`). It’s a reduction, not a self-interpreter: it shows the machine can host a universal-enough guest, and it connects the machine’s halting to a known undecidable problem.

One detail here changes what counts as a guest. The two-counter machine this interpreter runs branches when a decrement succeeds and falls through when the counter is zero. The convention is not cosmetic: the two-instruction program “increment counter 0, then decrement counter 0 and jump back” runs forever under that convention (`cm2_loop_diverges`, and the interpreter running it never halts, `cm2_loop_host_never_halts`), while the same pair under the zero-branching convention halts after two steps (`zero_branch_pair_terminates`, `VMUnboundedCM2Applicability.v`). That a zero-branching guest is a weaker machine overall is argued, not checked, and no result depends on it; the zero-branching guest is simply not used as a witness of universality.

24.4 The second interpreter: the machine running a piece of itself

Now the one the recursion theorem lives on. Take a twelve-instruction fragment of the machine’s own instruction set: halt, load-immediate, transfer, add, subtract, multiply, and, or, shift left, shift right, jump, jump-if-nonzero. Four guest registers (numbers zero to three), unbounded values, a guest program counter, a guest ledger, and “terminated” meaning the guest program counter has run off the end of the guest

program. A guest program is *well-formed* when every register name is below four and every declared cost is below two hundred and fifty-six.

The literal host listings, complete encodings and block-by-block verification are in Section H. In this guest, HALT advances the program counter; leaving the program is what ends execution.

The host is a fixed list of a hundred and twenty-two of the machine's instructions, at declared cost zero. It keeps the guest's four registers in its registers zero to three, a status in register nine (running, terminated, malformed), the guest ledger in register eleven, the guest program counter in register twelve, a width in thirteen, and the packed guest code in fourteen. The host's own counter never moves, since every host instruction is at cost zero; the guest's ledger is just a number sitting in register eleven. Under the hypotheses that the guest program is well-formed and fits the width: a terminating guest run gives a host run that reaches program counter one hundred and twenty-two in the corresponding done state (`self_interpreter_complete`); any host run that reaches one twenty-two from an encoded configuration decodes to a terminal guest run (`self_interpreter_sound`); together, a biconditional on the final guest configuration (`self_interpreter_correct`); the guest never terminates if and only if no fuel reaches one twenty-two (`self_interpreter_divergence`); and for *arbitrary* packed data, a fetched word whose low four bits are thirteen, fourteen or fifteen sends the host to the end with the malformed status and the guest registers, ledger and program counter as they were (`self_interpreter_malformed`; opcode zero is the sentinel for "no instruction here", which is how a guest ends). The guest step is proved to be the unbounded sibling's own step on the guest fields (`g_step_is_vm_apply_u`), the code round-trips (`g_decode_guest_code_roundtrip`), and running the guest is actual machine execution (`g_eval_is_actual_vm_execution`).

Here's the same thing in the notation that makes it checkable and unreadable in equal measure:

Definition 24.4 (Guest configurations and runs). A guest configuration c is a program counter, a ledger and four registers, all natural numbers. A guest program p is a list of the twelve guest instructions. $\text{gstep}_p(c)$ executes the instruction of p at c 's program counter on unbounded values and leaves c alone when there is none; $\text{grun}_p(n, c)$ is n such steps. c is *terminal* for p when its program counter is at least the length of p . p is *well-formed* when every register field is below 4 and every declared cost below 256, and p *fits* the width w when every packed instruction word of p is below 2^w .

Theorem 24.7 (The self-interpreter is correct). Let p be well-formed and fit w , and let U be the fixed host program of 122 instructions, run by the program-counter runner on the unbounded step apply_u ; write $\text{urun}(F, s)$ for F steps. For a guest configuration c , let $H(c)$ be

a host state at host program counter 0 whose registers 0 to 3 hold c 's registers, register 9 holds 0, registers 11 and 12 hold c 's ledger and program counter, register 13 holds w and register 14 holds the packed code of p ; the seven scratch registers (4 to 8, 10 and 15), the host counter and every other field are arbitrary. Say a host state is done for c' when it has host program counter 122, registers 0 to 3, 11 and 12 holding c' , register 9 holding 1, registers 13 and 14 as in $H(c)$, and the host counter and every field outside the registers as in $H(c)$. Then for all c and c' ,

$$(\exists n. \text{grun}_p(n, c) = c' \text{ and } c' \text{ is terminal}) \iff (\exists F. \text{urun}(F, H(c)) \text{ is done for } c'),$$

and the guest run from c never reaches a terminal configuration exactly when no F brings $\text{urun}(F, H(c))$ to host program counter 122.

Proof. The core is a block lemma: from $H(c)$ with c not terminal, a positive number of host steps reaches $H(\text{gstep}_p(c))$, with new scratch values. The block fetches the packed word at the guest program counter (well-formedness and the width make the fetch return exactly that instruction's word), splits it into fields with shifts and masks, dispatches on the opcode, and performs the guest instruction on registers 0 to 3, 11 and 12; each opcode's block is checked by computing with the host's own step. Induction on n gives the forward direction, followed by the fixed terminal sequence that sees the end sentinel and writes status 1. The backward direction is strong induction on F : 122 is past the end of U , so once reached the host stays there, and a block boundary has host program counter 0, so a run that reaches 122 must pass through whole blocks. Divergence is the contrapositive of the two directions. $\square$

Checked as `self_interpreter_complete`, `self_interpreter_sound`, `self_interpreter_correct` and `self_interpreter_divergence` (VMSelfRun.v, 208, 244, 264), with `h_block` and `h_simulation` (VMSelfRun.v) and the done-state definition `h_done`.

So the machine runs a fragment of itself, as data, with a proof that what it computes is what the fragment computes. Twelve of fifty-one; four registers of sixteen; and the "itself" is the unbounded sibling, with its saturating subtraction, not the sixty-four-bit step. A fragment. "Self-interpreter" can sound as if it covered the whole instruction set. It doesn't, and I'd rather the numbers be in front of you.

24.5 Rice, on the guest

With the interpreter in hand, Section 11's wall lands on the guest without any premise about recursion. Say two guest programs *behave the same* when, for every input placed in register zero, they terminate with the same final registers and the same guest ledger, or both don't (`g_equiv`; note it sees returned registers and ledger, not the final program counter or the step count). A property of guest programs is *extensional* when well-formed programs that behave the same agree on it. Then: for any extensional property that holds of some well-formed program and fails of the program that jumps to itself forever, the predicate "well-formed and has the property" is undecidable, in

the synthetic sense the Saarland library uses (the complement of two-counter halting reduces to its complement) (`self_rice`). “Decided” has a stated meaning here (`g_decides`): a well-formed guest program that, given any fixed way of numbering programs, terminates on the number of every well-formed program and reports the property by whether register zero ends at one. A guest program that decided such a property would make the complement of halting for single-tape Turing machines over a two-letter alphabet enumerable (the library calls these SBTM, single-tape binary Turing machines; the book just says single-tape) (`self_rice_representable`). “Halts on input zero” is a named instance. That “undecidable” is the library’s synthetic notion, a reduction and an enumerability statement, and not a bare negation of every decision procedure one could write in Coq.

One more word on that, because “undecidable” here is the Saarland library’s word and it means something exact. Inside Coq, the library doesn’t say “no algorithm decides this” outright. It says: if you could decide this, you could *enumerate* (list, one after another, by a program) every Turing machine that never halts. Nobody can do that: the machines that do halt can already be listed (run them all a little at a time and write each one down when it stops), so with both lists you could decide halting by waiting to see which list a machine turns up on, and that’s Turing’s wall. So it’s a reduction, a promise that a decider here would be a decider there.

The words first, then the theorem.

Definition 24.5 (Behaviour, extensional, decides, undecidable). For a guest program p and input x , $\text{beh}_p(x, g, \mu)$ holds when some $\text{grun}_p(n, c_x)$ is terminal with registers g and ledger μ , where c_x has program counter 0, ledger 0 and registers $(x, 0, 0, 0)$. Programs p and q *behave the same* when $\text{beh}_p(x, g, \mu) \iff \text{beh}_q(x, g, \mu)$ for all x, g, μ . A property P of guest programs is *extensional* when $P(p)$ implies $P(q)$ for well-formed p and q that behave the same. A guest program D *decides* P under a numbering e of programs when D is well-formed and for every well-formed p there are g and μ with $\text{beh}_D(e(p), g, \mu)$ and $(g_0 = 1 \iff P(p))$. A predicate Q is *undecidable*, in the library’s sense, when a Coq function deciding Q would make the complement of single-tape Turing-machine halting enumerable.

Theorem 24.8 (Rice on the guest). *Let P be extensional, let w be a well-formed program with $P(w)$, and let P fail for the one-instruction program that jumps to itself. Then “well-formed and P ” is undecidable. If moreover some guest program decides P under some numbering, the complement of single-tape Turing-machine halting is enumerable.*

Proof. For the first part, send a two-counter machine m to a guest program that saves its input in register 2, loads m ’s two starting counters, runs a guest copy of m , puts the input back in register 0 and clears the other three, and then runs w , with every instruction before w at declared cost 0. If m halts, the program behaves like w , so it

has P ; if m does not halt, it never terminates, behaves like the self-jump, and lacks P . That reduces the complement of two-counter halting, undecidable in the library, to the complement of “well-formed and P ”, and a predicate whose complement is undecidable is undecidable. For the second part, a guest decider gives a Coq decider: for a well-formed p , search for the least fuel at which D on $e(p)$ is terminal (it exists, so the constructive search over the naturals finds it) and read register 0. $\square$

Checked as `self_rice` (VMSelfRiceUndec.v), `g_decides_decidable` and `self_rice_representable` (VMSelfRiceUndec.v), with the definitions `g_beh`, `g_equiv` (VMSelfRice.v, 41), `g_extensional` (VMSelfRiceUndec.v) and `g_decides`; `g_halts_on_zero_undecidable` is the instance for “halts on input zero”. The first part is also stated as the proposition `vm_guest_rice` (VMRecursionTarget.v), with “fails for the self-jump” written as failing for `g_bottom`, and proved as `vm_guest_rice_holds` (VMRecursionAudit.v) by `self_rice`.

24.6 The guest recursion theorem

Here’s the theorem the fragment exists for. Statement, in full, because which transformations the guest can compute is the condition that matters. For every transformation F from guest programs to guest programs that maps well-formed programs to well-formed programs, and every guest program D that *represents* F (D is well-formed, and run on the code of any well-formed program p , D terminates with register zero holding the code of F applied to p), there is a well-formed guest program p that behaves the same as F applied to p (`vm_guest_recursion_theorem_closed`). Kleene’s second recursion theorem, for the twelve-instruction guest, with “the machine can carry out the transformation” meaning “some guest program computes it on codes.”

The proof is a construction, which is Section 11’s point about Kleene’s little gem (Moschovakis’s phrase): the content is the building, and the contradiction in Section 11 was just the last line. The shape: from the representing program D , build an *evaluator* that reads a program’s code, specialises that program to its own code, runs D on the specialised code, and runs the program D returns. The finite algorithm and its output encoding are given below. Realise that evaluator as a guest program e ; take the code of e ; and the witness is e *specialised* to its own code by a ten-instruction guest prefix that pairs a fixed number with the input (`g_pair_specialize`). The prefix is the specialising step made as guest instructions (logicians call this step *s-m-n*: fix one input of a program and get a new program that needs one input fewer), and the theorem that it specialises correctly (`g_pair_smn`, which says the specialised program on an input and the original on the paired input finish the same way, in both directions) is what makes the witness a *guest* program rather than a Coq-level construction. So: p is e with its own code baked in; run on any input, it evaluates D on the code of p , gets the code of F p , and runs that. It’s tempting to say the prefix pairs “ p ’s own code”; more exactly it pairs the code of the evaluator e , and e builds p ’s code from it. So L is a tool on this route, not a source: the recursion theorem is proved on the guest

directly, and it isn't pulled back from L 's own recursion theorem in Section 11. The mathematical dependency is an effective translation from the computable evaluator to a counter program and then to the guest. The checked implementation uses L and counter-machine compilers for that translation; their role is to preserve the input-output relation, not to supply the fixed-point argument.

Here is the statement with every word unfolded, using Definitions 24.4 and 24.5. Write $\ulcorner p \urcorner$ for the code of a guest program: translate p into the machine's own instructions, then take the machine's program number of that list.

Definition 24.6 (Represents, pairing prefix). A guest program D represents a map F from guest programs to guest programs when D is well-formed and for every well-formed p there are g and μ with $\text{beh}_D(\ulcorner p \urcorner, g, \mu)$ and $g_0 = \ulcorner F(p) \urcorner$. The pairing is $\langle x, y \rangle = (2y + 1) \cdot 2^x$. For a number x and a guest program e , $\text{spec}(e, x)$ is a fixed ten-instruction prefix, all at declared cost 0, followed by e with its jump targets moved up by ten.

Here are the ten instructions themselves. Lines are numbered from zero, every instruction has declared cost zero, and the relocated program starts at line 10. The input is $(r_0, r_1, r_2, r_3) = (y, 0, 0, 0)$.

Line	Operation
0	Set $r_2 := 1$.
1	Set $r_0 := r_0 + r_0$.
2	Set $r_0 := r_0 + r_2$.
3	Set $r_1 := x$.
4	If r_1 is nonzero, jump to line 7; otherwise advance.
5	Set $r_2 := 0$.
6	Jump to line 10, the start of the relocated program.
7	Set $r_0 := r_0 + r_0$.
8	Set $r_1 := r_1 - r_2$, with natural subtraction.
9	Jump to line 4.

At line 4 after j turns through the loop, the registers satisfy $r_0 = (2y + 1)2^j$, $r_1 = x - j$, $r_2 = 1$, $r_3 = 0$, with $0 \leq j \leq x$. A nonzero r_1 doubles the first number and reduces the second by one. At $j = x$ the loop exits and clears r_2 , leaving exactly the paired input and three zero registers. Relocation adds 10 to every instruction address and jump target of e ; ordinary next-line steps and jumps therefore correspond in both directions.

Lemma 24.9 (The prefix pairs). For all e, x, y, g, μ : $\text{beh}_{\text{spec}(e, x)}(y, g, \mu) \iff \text{beh}_e(\langle x, y \rangle, g, \mu)$.

Proof. The prefix doubles register 0 and adds one, then doubles it x more times by a counted loop on register 1, then clears its scratch register and jumps to the start of the moved copy of e . That takes $4x + 7$ steps, charges nothing, and leaves registers $(\langle x, y \rangle, 0, 0, 0)$, which is where a run of e on $\langle x, y \rangle$ starts. A moved program runs exactly as the original with every program counter shifted. $\square$

Checked as `g_pair_smn` (VMGuestRecursion.v), both directions of the equivalence.

24.7 The evaluator, without a black box

There are two program encodings in this chapter. The interpreter appendix packs words at a chosen fixed width for fast field selection. The recursion theorem instead uses a self-delimiting syntax code. Mixing them would change the program being constructed.

For the syntax code, encode a natural number n as n true bits followed by one false bit. Encode an instruction by its numeric tag followed by its operands and cost, each in that unary form. The tags are:

Instruction	Tag	Following fields, in order
HALT	24	cost
LOAD_IMM	8	destination, immediate, cost
XFER	7	destination, source, cost
ADD, SUB, MUL	11, 12, 35	destination, first source, second source, cost
AND, OR, SHL, SHR	31, 32, 33, 34	destination, first source, second source, cost
JUMP	13	target, cost
JNEZ	14	source, target, cost

Prefix the instruction sequence with its length, also in unary. Convert the resulting bit list to a number by $B([]) = 1$ and $B(b :: r) = b + 2B(r)$, treating false and true as 0 and 1. The final leading one distinguishes the end of the bit list. Decoding repeatedly takes parity and divides by two until that leading one is reached, then parses the unary fields and the specified number of instructions. Reject an unknown tag, an unfinished field or leftover bits. Encoding then decoding a guest program returns that program: induction on each unary field gives its boundary, followed by induction on the instruction list. This is the number $\lceil p \rceil$ used in the recursion theorem.

Fix the representing program D . The following finite calculation $E_D(n, z)$ either returns a number or reports that this trial has no result:

1. Unpair z as $\langle c, y \rangle$. For a positive z , repeatedly divide by two until it is odd: the number of divisions is c , and the odd remainder is $2y + 1$. Decode c as a guest program e' ; a failed decode gives no result.

2. Form the actual instruction list $\text{spec}(e', c)$ using the ten-instruction prefix above and the relocated copy of e' . Encode that list.
3. Run D on that code for n guest steps. If it has not terminated, give no result. Otherwise decode register zero as a guest program q ; reject a failed decode.
4. Run q on y for n guest steps. If it terminates, return the packing of its four registers and ledger given below. Otherwise give no result.

Only positive paired inputs are needed for the fixed point; the implementation's unpairing convention sends zero to $(0, 0)$, whose program code fails to decode. Every trial terminates. It uses finite list operations, arithmetic and two bounded runs. Now try $n = 0, 1, 2, \dots$ until a trial returns a result. This search need not terminate, which is exactly what is needed to represent a partial computation. Once a trial returns a result, every larger trial returns the same result: a terminated guest run stays at its terminal configuration. If the two required runs terminate after n_1 and n_2 steps, any $n \geq \max(n_1, n_2)$ succeeds. Conversely, a successful trial contains those two terminating runs.

The output packing also matters because behaviour includes the ledger. For final registers (a, b, c, d) and ledger μ , put

$$s = a + b + c + d, \quad v = b + 2^s(d + 2^s c), \quad P(a, b, c, d, \mu) = (2((2v + 1)2^s) + 1)2^\mu.$$

To decode a valid packing, first count its trailing zero bits to recover μ , remove the following odd factor's low one bit, and divide the remainder by two. The result is $(2v + 1)2^s$. Repeat the trailing-zero operation to recover s and v . Since each register is at most $s < 2^s$, successive base-2^s digits of v recover b, d, c , and $a = s - b - c - d$. This works also at $s = 0$: all four registers are zero.

The evaluator's search is a partial recursive calculation. The compilation fact used here is that such a calculation can be realised by a finite counter program with its input and output represented as natural numbers. The traditional universality constructions and the precise counter conventions are identified in Section I. The implementation checks the translation through the weak call-by-value calculus and an increment/decrement counter machine. This is a computability result; it supplies no useful bound on the resulting program's size or running time.

The counter program is then simulated in the four-register guest. Its administration has declared cost zero. Its output epilogue unpacks P : the first trailing-zero loop charges one per division, producing exactly μ ledger units; the second loop and the register reconstruction charge zero. Thus the guest evaluator ends with the actual four registers and ledger, rather than merely a code of them. The paired input's construction and all work before that final charging loop cost zero. This is why including the ledger in behavioural equality does not break the fixed-point construction.

Theorem 24.10 (Guest recursion theorem). *For every map F from guest programs to guest programs that sends well-formed programs to well-formed programs, and every guest program D that represents F , there is a well-formed guest program p that behaves the same as $F(p)$.*

Proof. Let R be the relation that holds of a number z and an output m when $z = \langle c, y \rangle$ with c the code of some program e' , the run of D on $\ulcorner \text{spec}(e', c) \urcorner$ terminates with the code of a program q in register 0, and the run of q on y terminates with registers and ledger that pack to m . The finite trials $E_D(n, z)$ above return exactly those outputs. Their unbounded search, realised by the counter translation and the cost-restoring epilogue, gives a well-formed guest evaluator e with exactly this input-output behaviour. Put $c = \ulcorner e \urcorner$ and $p = \text{spec}(e, c)$, well-formed because e is. By Lemma 24.9, $\text{beh}_p(y, g, \mu) \iff \text{beh}_e(\langle c, y \rangle, g, \mu)$, and by the construction of e the right side holds exactly when $F(\text{spec}(e, c)) = F(p)$ terminates on y with registers g and ledger μ . That is p behaving the same as $F(p)$. The self-reference is the prefix handing e its own code; the recursion theorem for L in Theorem 11.3 is not used. $\square$

Checked as `vm_guest_recursion_theorem_closed` (`VMGuestRecursion.v`), a proof of the proposition `vm_guest_recursion_theorem` (`VMRecursionTarget.v`), with `g_represents_transformer` (`VMRecursionTarget.v`), `g_pair_sm` (`VMGuestRecursion.v`), `hfun_sem` and `RD_MMA` (`VMGuestEvalL.v`), and `g_pipeline_beh_pack` (`VMGuestMMAPipeline.v`) for the compiled Minsky machine and its epilogue (given that every output of the Minsky machine is a packed pair, the guest run ends with g and μ exactly when the Minsky machine outputs their packing). A guest run is the unbounded sibling's own step on the denoted machine instructions (`vm_guest_execution_is_actual`, `VMRecursionAudit.v`). The axiom report for the theorem is the `Print Assumptions` line at the end of `VMGuestRecursion.v`, read from the build.

Two things it isn't. It isn't a recursion theorem for the full fifty-one-instruction machine, and nothing can be one in the sense Premise 24.2 asks for every map: that premise is false (Theorem 24.2). The guest theorem answers a different question. It gives fixed points for the guest's representable transformations, under the unbounded sibling's runs, up to equal registers and ledger, and no correspondence theorem turns that into equal thousand-step runs of the full machine from every state. And the ledger: the pairing prefix, the evaluator and the compiled Minsky machine all run at declared cost zero, and a last loop charges, one cost-one instruction per unit, exactly the ledger of the program being imitated. So the witness ends with the same registers *and the same ledger* as F applied to it, because the ledger is part of what "behaves the same" means. What the theorem doesn't speak to is efficiency, how many instructions the witness executes to get there.

24.8 What this closes

The natural objection to a guest recursion theorem is that the evaluator and the specialiser aren't things a guest program can call. Here they are: the specialiser is a guest prefix and the evaluator is a compiled guest program. So the two ingredients of Section 11's diagonal hold on this machine's guest *by proof*: Rice from the interpreter, recursion from the construction. What I haven't proved is the shortcut predicate itself on the guest; the wall at the start of this chapter is about bounded-run equality with one reference program on the full machine (Definition 24.1), and the guest results are about behaviour on registers and ledger. On the full machine the diagonal holds under a premise (Theorem 24.1). Both are true; they're different sentences; and the fragment is where the unconditional ingredients live.

Next: the one instruction family I've been pointing at since Section 16 without building, the two-player game and the integer check the machine runs on it.

25 The CHSH check

I've pointed at a game since Section 16 without building it. Here it is, built from zero, and then the one thing my machine actually does with it, which is smaller and more exact than "it checks for quantum behaviour," a sentence that's tempting and wrong. The short version, so you know where this is going: the machine never computes the famous number and never writes down the famous bound. It tests seven facts about integers in eight counters. If they pass, a certain five-by-five table has a property called positive semidefinite, and that property forces the famous number under the famous bound. The wall is a consequence, not an ingredient. Everything else is explaining those sentences.

25.1 The game

Two players, Alice and Bob, in separate rooms. A referee flips two coins and hands Alice a question x , zero or one, and Bob a question y , zero or one. Alice answers a , zero or one; Bob answers b . No talking during the game. They may agree on a plan beforehand, as detailed as they like. They win a round when their answers *differ exactly when both questions were one*. In words: if either question is zero, match; if both are one, differ.

Try to win every time. A plan is a list: what Alice says to x equals zero, what she says to x equals one, and the same two for Bob. Four bits. Now write the four requirements the rounds impose: a_0 equals b_0 (both asked zero); a_0 equals b_1 ; a_1 equals b_0 ; a_1 differs from b_1 . Chain the first three: a_0 equals b_0 equals a_1 , and a_0 equals b_1 , so b_0 equals b_1 . But the fourth says a_1 differs from b_1 , and a_1 equals b_0 equals b_1 . Contradiction. One of the four must break. So any plan loses at least one of the four question pairs, and if the referee asks all four equally often, the best a plan can do is three out of four. Seventy-five percent. Do it the long way too: there are sixteen plans (four bits), and if you write them all out and count wins you'll find eight win three rounds and

eight win one, and none wins four (the broken requirements always come in odd numbers, which you can see by adding up the four equations: the left sides cancel to zero and the right sides to one). Sharing a coin before the game to pick a plan at random doesn't help; a mixture of numbers no larger than three-quarters is no larger than three-quarters. Arthur Fine proved in 1982 that the whole "pre-agreed answers, no influence between rooms" story is *exactly* the mixtures of those sixteen lists, one scoreboard for all four questions at once; the sixteen are the whole classical story.

The number. Recode answers zero and one as plus one and minus one. For each question pair, let E be the average, over many rounds, of Alice's answer times Bob's. The CHSH quantity is S equals E_{00} plus E_{01} plus E_{10} minus E_{11} . For one fixed plan each win adds one to S and each loss subtracts one, so S is wins minus losses: a three-win plan has S equal to two, a one-win plan has minus two. With equal question frequencies, the win rate is one half plus S over eight. So "seventy-five percent" and " S at most two" are the same fact, and the second is the one that got the name.

The machine can play the three-out-of-four ceiling for free. One short program sets up two modules and then runs four trials, one per question pair, with both players answering zero every time. After the recoding every answer is plus one, so all four correlators are one: the all-ones plan, which matches on every pair and so wins the three pairs that ask for a match. Every instruction in it is declared at cost zero.

Theorem 25.1 (The classical ceiling, reached at zero cost). *Let t be the list PNEW, PSPLIT, and four CHSH_TRIAL instructions, one for each question pair (x, y) , each with both answers 0 and declared cost 0. The cost model's total charge for t is 0, and running t in order from the clean state (all registers zero, counter zero, no trials recorded) leaves trial counters whose score $S = E_{00} + E_{01} + E_{10} - E_{11}$, computed in rationals from the counters, equals 2.*

Proof. Computation: the charge is a sum of six zeros, and running the six steps leaves one "same" round in each of the four pairs, so each correlator is 1 and $S = 1 + 1 + 1 - 1 = 2$. $\square$

Checked as `classical_bound_achieved` (ClassicalBound.v). Nothing in it says the rounds were played; a program can tally anything at zero cost, and the theorem only shows that the ceiling is reachable. The other side, that no finite mixture of plans beats two, is Theorem E.2 in Appendix E, one direction of Fine's result.

25.2 Where it came from, and why anyone cared

In 1935 Einstein, Podolsky and Rosen asked whether quantum mechanics describes everything that's real about a pair of particles. Their argument: if measuring one particle lets you predict the other's result with certainty, the result must already have been fixed, and the theory leaves that out. The wall they had in mind was a pre-agreed list of answers, the plan above. For thirty years that was philosophy. In 1964 John Bell turned it into arithmetic: if each particle carries pre-agreed answers and nothing

travels between the rooms, certain averages can't exceed a fixed number, and quantum theory predicts they do. What his inequality rules out is the *pair* of assumptions together, called local realism (answers exist before you ask, and no influence across the gap), not either one alone. In 1969 Clauser, Horne, Shimony and Holt rewrote it as the four-average form above, S at most two, which could be tried in a lab with imperfect equipment. Freedman and Clauser tried it in 1972; Aspect's group did it in 1982 with the measurement settings switched while the photons were in flight; and in 2015 three groups (Hensen's in Delft with electrons a kilometre apart; Giustina's; Shalm's) closed the two main escape routes at once. Clauser, Aspect and Zeilinger got the 2022 Nobel for it. The lab number sits above two. A remaining escape, that the referee's coins were secretly correlated with the particles, is a philosophical one and I'm not going to pretend it's closed.

What quantum play does: if the players share an entangled pair and choose what to measure from their question, the best they can do is S equals two times the square root of two, about 2.83, a win rate of about eighty-five percent. Boris Tsirelson showed in 1980 that that's a *ceiling*: quantum correlations violate Bell, but only up to there.

And the wall above that, which is the one I find most instructive. In 1994 Popescu and Rohrlich asked: what if you only demand that nothing can be *signalled*, that Alice's answers on their own have the same odds whatever Bob is asked, and the other way round? Then you can write down a "box" (the PR box, after their initials) that wins *every* round (its answers differ exactly when both questions are one, by fiat, each answer alone a fair coin), S equals four, and nothing about it lets either room send a message. So relativity alone doesn't explain the quantum ceiling; the PR box obeys relativity and beats 2.83. The order is: two for plans, 2.83 for quantum, four for no-signalling. Which principle fixes 2.83 is still an open question, and this unit won't answer it. My machine's check is a fence at the middle step.

25.3 Wrong turns to take off the table

"Faster than light." Nothing travels. Each room alone sees fair coins; the correlation shows up only when you lay the two columns side by side afterward. "Quantum means random." The average S is fixed by the state; what's random is each single answer; the plans are the less random side. "A Bell violation proves nonlocality." It rules out local realism, both assumptions together, and every real test has loopholes with names. "A tally above two proves nonlocality." No: S is a *population* average, and one finite tally can land above two by luck. Four rounds, one per question pair, with coin-flip answers, reach S equals four one time in sixteen. That gap between a population bound and a finite sample is the problem Zhang, Glancy and Knill work through for real Bell tests (Physical Review A, 2013), and the book doesn't solve it. The kernel proves only the narrow thing, that a tally above two is not explained by *one fixed plan replayed every round* (`chsh_violation_rules_out_locally_factorizable_coupling`), and that narrow thing is what the sentence has to stay. And "the PSD test proves the data is quantum." (PSD is short for positive semidefinite, the

property of the five-by-five table from the short version; the next part builds both.) It doesn't, and the direction matters. Passing implies the five-by-five table, pinned in a way the next part explains, is PSD, and that implies S squared is at most eight; both are proved. Whether the correlators of a PSD table like that are the correlators of some quantum strategy is the cited Tsirelson, Landau and Masanes result, and I haven't proved it here. Failing says nothing: the check is sound and not complete, and quantum behaviour can fail it. Every one of the sixteen plans fails it, and I'll show you why. What it guarantees is in the last part of this section.

25.4 The five-by-five, and what positive semidefinite means

Here's the mathematics the check rests on, and it's one idea: a table of numbers *could be a table of dot products*. Take a two-by-two symmetric table with a and c on the diagonal and b off it. It's positive semidefinite (PSD) when a and c are at least zero and a times c minus b squared is at least zero. Check two by hand: ones on the diagonal and one half off, the determinant is three quarters, passes; ones on the diagonal and 1.2 off, the determinant is negative, fails. The reading: PSD means the table could be the dot products of some arrows, and 1.2 is too big a dot product for two arrows of length one. That's the whole idea, generalised to bigger tables.

Now the table. Rows and columns labelled $I, A0, A1, B0, B1$ (the identity, Alice's two questions, Bob's two). Diagonal all ones. The I row and column all zero (that's "zero marginals": the table takes each answer alone to average to zero, a fair coin; the machine records no marginals and doesn't test this). $A0$ with $A1$ zero, $B0$ with $B1$ zero (no round asks Alice both questions, so no tally holds that average, and the table sets it to zero). Those zeros are choices, they pin the table to one slice, and I'll come back to what they cost. And the four Alice-Bob cells hold the four correlators E . That's a five-by-five with four free numbers in it. It splits into the one in the corner and a four-by-four block made of an identity and the two-by-two grid E of correlators, and PSD of that block is the same as saying *E shrinks every vector* (E is a contraction), which is three inequalities: one minus E_{00} squared minus E_{10} squared at least zero; one minus E_{01} squared minus E_{11} squared at least zero; and their product at least the square of ($E_{00} E_{01}$ plus $E_{10} E_{11}$).

Run it. All four correlators 0.6 except E_{11} at minus 0.6, S equals 2.4: the two diagonal terms are 0.28 and 0.28, the cross term zero, passes; above two, below 2.83. All four at one over root two (E_{11} negative): diagonals zero, cross term zero, passes with no slack, S equals two root two exactly, the 2.83; nudge any entry's size up and a diagonal goes negative. The all-ones plan (one, one, one, one): first diagonal is one minus one minus one, negative, *fails*, though S equals two and it's a perfectly legal classical play. The PR box (one, one, one, minus one): fails the same way. A fence at the middle step, and a fence that keeps out every one of the sixteen plans: each has all four correlators at plus or minus one, so the first diagonal is always one minus one minus one. Only diluted mixtures of plans get through.

In 2007 Navascués, Pironio and Acín showed this is the first rung of a ladder: bigger tables of expected products, each PSD, each a necessary condition for quantum behaviour when its entries are left free, each rung tighter, and (their 2008 paper) converging to the quantum set (strictly, the set from commuting operators, which can differ from the tensor-product set in general scenarios). The machine's table is the first rung with the marginals and the two within-party entries pinned to zero. Pinning is a stronger demand than the rung makes, so passing puts you inside the first rung, and a quantum point can still fail. Landau in 1988 and Masanes in 2003 found the exact boundary for this two-questions-each case, and it's not a straight line at 2.83 but a curved surface (the arc-sine form) touching it at one point. And a 2026 paper by Chaturvedi shows no *finite* rung of the ladder captures the whole quantum set even in this simplest case, which is the caution this book keeps about the ladder, confirmed from outside.

25.5 What the machine actually does

Eight counters (Section 16): for each of the four question pairs, one counting rounds where the answers matched and one where they differed. One instruction, the trial, bumps one counter per round, after checking all four inputs are bits; it checks no inequality and records no marginals. Then one instruction, the check, reads the eight and tests seven integer facts. For each pair, let N be same plus different and D be same minus different (so E is D over N). The seven: the four N are strictly positive (every pair was sampled; a pair with no rounds is rejected, not read as a correlator of zero); A , which is N_{00} squared N_{10} squared minus D_{00} squared N_{10} squared minus D_{10} squared N_{00} squared, is at least zero; B , the same with the other column, is at least zero; and with C equal to $D_{00} D_{01} N_{10} N_{11}$ plus $D_{10} D_{11} N_{00} N_{01}$, C squared is at most A times B . Those are the three contraction inequalities with the denominators cleared. No division, no reals, no square root, no S . Try it: counters (same, different) of (4, 1), (4, 1), (4, 1), (1, 4) give E of 0.6, 0.6, 0.6, minus 0.6; N is five and D is three (minus three for the last); A is 175, B is 175, C is zero; passes. That's the first example above, the S equals 2.4 one, in integers.

If the seven pass, the instruction steps on: the program counter advances and the ledger rises by the charge, and nothing else changes, not the flag, not the channel; success is visible only as "no trap." If any fails, it traps to the fixed address with the error code and flag. Either way it costs the declared cost plus one, the floor. The theorem chain, in words: a passed check gives PSD of the five-by-five built from the counters; contraction and PSD are equivalent, both directions; and PSD forces S squared at most eight. The names are in the statements below. The underlying algebra is one identity: four times the sum of the four correlators squared, minus S squared, is a sum of six squares (*chsh_gap_is_sum_of_squares*), so if the row sums of squares are at most one, S squared is at most eight (*tsirelson_squared*), and the point one over root two reaches it (*tsirelson_tight*). That's Tsirelson's bound as *algebra*, with no Hilbert space (the mathematical home of quantum states) in it.

Here is the chain as three statements. The derivations are in Appendix E; this is what they prove.

Definition 25.1 (Correlators from the counters; the pinned matrix). For a state s and a question pair xy , let $E_{xy}(s) = (\text{same}_{xy} - \text{diff}_{xy}) / (\text{same}_{xy} + \text{diff}_{xy})$ from its trial counters, read as 0 when both are 0. For reals E_{xy} , $M(E)$ is the symmetric five-by-five matrix on I, A_0, A_1, B_0, B_1 with ones on the diagonal, E_{xy} in the cells (A_x, B_y) and (B_y, A_x) , and zero everywhere else (Definition E.5). M is PSD when $v^T M v \geq 0$ for every real vector v , and $S(E) = E_{00} + E_{01} + E_{10} - E_{11}$.

Theorem 25.2 (The gate implies the pinned matrix is PSD). *Let s be a state whose error flag is false, let d be a declared cost, and let s' be the result of CHSH_LASSERT d on s . If the error flag of s' equals that of s , then $M(E(s))$ is symmetric and PSD.*

Proof. When the integer check fails the step sets the error flag, which the hypotheses rule out, so the check passed. Its four positivity facts make every denominator positive, so each $E_{xy}(s)$ is D_{xy}/N_{xy} , and dividing the other three facts by positive squares turns them into the three contraction inequalities (Lemma E.15). Contraction is equivalent to PSD of $M(E)$ by a Schur complement on the four-by-four block (Corollary E.13). Nothing about the program counter is needed: the error flag alone says which branch ran. $\square$

Checked as `chsh_lassert_no_trap_implies_npa_psd` (QuantumPartitionPSD.v), through `chsh_lassert_check_implies_npa_psd` and `npa_psd_iff_column_contractive`.

Theorem 25.3 (PSD forces the Tsirelson bound). *For all real $E_{00}, E_{01}, E_{10}, E_{11}$: if $M(E)$ is symmetric and PSD, then $S(E)^2 \leq 8$.*

Proof. A PSD matrix with unit diagonal has every three-by-three principal minor at least zero. On rows A_0, B_0, B_1 that minor is $1 - E_{00}^2 - E_{01}^2$, and on rows A_1, B_0, B_1 it is $1 - E_{10}^2 - E_{11}^2$, so the four squares sum to at most 2. The identity $4 \sum_{xy} E_{xy}^2 - S^2 =$ a sum of six squares then gives $S^2 \leq 4 \cdot 2 = 8$. $\square$

Checked as `npa_psd_implies_tsirelson_bound` (MuLedgerQuantumBridge.v), with the row bounds `npa_psd_zero_marginal_implies_row_bounds` and the identity `chsh_gap_is_sum_of_squares` (TsirelsonFromAlgebra.v); this is Theorem E.14 in the appendix.

Corollary 25.4 (A passed gate means the bound). *Under the hypotheses of Theorem 25.2, $S(E(s))^2 \leq 8$.*

Proof. Theorem 25.2 supplies positive semidefiniteness of the pinned moment matrix for these four correlators. Theorem E.14 in the derivation appendix applies to every such matrix and gives $S^2 \leq 8$. Its quadratic form calculation is printed there; no additional property of the machine is used. $\square$

Checked as the composition of `chsh_lassert_no_trap_implies_npa_psd` and `npa_psd_implies_tsirelson_bound`; no single Coq theorem states the two together. The same conclusion is also proved along a parallel route, `chsh_lassert_no_trap_implies_state_column_contractive` followed by `state_column_contractive_implies_tsirelson` (`MuLedgerQuantumBridge.v`).

Remark 25.1 (Sound, not complete). The pinned matrix is stricter than the quantum question it is usually read against. PSD of $M(E)$ gives the correlators a PSD completion, since the zeros are one choice of the two free entries (`zero_marginal_implies_elliptope`, `ElliptopeCompletion.v`). The converse fails. The all-ones plan has a PSD completion (`deterministic_strategy_elliptope`), and every deterministic plan is a quantum strategy, a standard fact cited and not proved here; yet its minor on rows A_0, B_0, B_1 is $1 - 1 - 1 = -1$, so $M(E)$ is not PSD there, and a tally of one matching round per pair gives $A = 1 - 1 - 1 = -1$ in the integer check, so the gate traps. Those two failures are my arithmetic, not named theorems. So a pass proves the bound and a trap proves nothing about the tally. The zero-cost trace of Theorem 25.1 is exactly such a tally: it reaches $S = 2$ and the gate would throw it out.

One thing about that trap is worth saying on its own. The check traps to the same fixed address that LASSERT uses when a formula’s certificate fails, with the same error flag and the same rule for the charge: the attempt pays, declared cost plus one, and LASSERT more for the formula’s length. One wall comes from the moment-matrix mathematics (the table of expected products above), which I didn’t write. The other is a logical constraint a program hands to LASSERT. The step treats them the same way: the counter charges the attempt to narrow, and it never asks whether the wall was found or built. And that’s a claim about the step, not about the chip. Four of the check instructions below have no hardware encoding at all, and that doesn’t change a word of the step theorems. The law lives in the transition, not in the gate count.

None of that algebra is new, and I don’t want it to read as if it were. Tsirelson’s bound has been formalised before, in Isabelle/HOL by Echenim and Mhalla and in Lean’s Mathlib, and Navascués, Pironio and Acín already analyse this very matrix by Schur complement and discuss the level above it. What I think is mine here is smaller: an integer decision procedure on the counters, living inside an instruction; a checked link from “the instruction didn’t trap” to PSD to the bound; and a circuit proved equal to that check for thirty-two-bit counters.

So here, in one paragraph, is what the gate guarantees. If the machine steps past the

check without trapping, then every question pair was sampled, the four empirical ratios satisfy three polynomial inequalities, hence the fixed five-by-five is PSD, hence S squared is at most eight. It does *not* guarantee that the tally came from honest play (a program chooses its own trial inputs and can tally whatever it likes at zero cost; the kernel has a zero-cost trace with S equals four, `TsirelsonUpperBound.v`, to make sure nobody forgets). It does not guarantee any marginal is zero (the machine records no marginals and can't test them). It does not guarantee any quantum state exists, or that the correlators lie in the quantum set rather than a larger fence. And it traps the tally of every single plan, honest classical play included. The check certifies a *shape* of the tally. What the shape means is a question for whoever supplied the rounds.

And the tally isn't a claim. Trials pile up for free; the counters are a record of what a program said happened, and the ledger has no opinion about a record. The priced step is the one that stands up and commits to something. On this machine the only instruction that can move the assertion channel off zero is MORPH_ASSERT: a run of a program with no MORPH_ASSERT in it, from channel zero, never reaches a nonzero channel (`non_morph_assert_trace_cannot_gain_supra_cert`, `RevelationRequirement.v`). A wider theorem with a misleading name, `nonlocal_correlation_requires_revelation`, says that if a run from channel zero ends with it nonzero, the program contains a REVEAL, EMIT, LJOIN, LASSERT or MORPH_ASSERT. Its "supra" predicate is just "the channel is nonzero"; it knows nothing about CHSH, and asking for REVEAL in front of a super-quantum claim is runtime policy, which the source says. What MORPH_ASSERT commits to is a label on an existing arrow. Whether the label is true of the pile of tallies is a separate question, and CHSH_LASSERT is the priced check that answers part of it. Statistics are free; the claim about them is what costs.

In the chip, the check is a circuit, and the word-level circuit is proved to compute the same seven facts as the kernel for every set of eight thirty-two-bit counters (the intermediate products need 134 and 268 bits, which is why the circuit takes twenty-nine firings, one hardware rule run twenty-nine times over (Section 27 says what a rule is); `chsh_check_word_correct`, `chsh_run_result`, and, for the whole step under its usual side conditions, `chsh_lassert_refines`). The kernel's counters are unbounded naturals and the chip's are thirty-two bits, so the equivalence is for counters that fit. That's Section 27's.

25.6 A rational stand-in for an irrational number

One number you'll see in the files: 5657 over 2000, which is 2.8285, and two root two is 2.82842... It's the four-decimal round-up, chosen so that rational code can test "above the Tsirelson bound" without square roots, and it's safe because 5657 squared is 32,001,649 and eight times 2000 squared is 32,000,000, so the fraction sits above root eight by about seven parts in a hundred thousand (`rational_tsirelson_bound`). It's used in a receipt-level predicate and a few rational theorems. It is *not* used in the integer gate, and the real-number theorems use root eight. Nothing in the hardware

files mentions it either: the chip checks CHSH in integers.

25.7 Two more rungs, built and not bound in

The second rung: a nine-by-nine table on I , the four questions, and the four Alice-Bob *products*, with the correlators and five extra free numbers for three- and four-body entries. Its PSD is again equivalent to a contraction-like condition (`qlab_psd_iff_column_contractive`), and there are four more check instructions in the kernel for it, taking the extra numbers as caller-supplied counters, with a cascade of two Schur steps down to a four-by-four that Sylvester’s criterion closes (`chsh_lassert_lab_g12345_no_trap_implies_npa_psd_qlab`). Soundness only; the interior predicates demand strict positivity, so boundary points need another route. And a fact that’s easy to list without noticing what it means: the variant with all five extra numbers at zero certifies a strict *subset of the classical set* (the ball where the four correlators’ squares sum to at most one), forcing S at most two (`qlab_check_at_gzero_implies_classical_bound`); it can’t certify anything above the classical line. To get a point above two you have to supply the extras (`qlab_certifies_a_superclassical_chsh_point`, S equals 2.4 with one extra at minus one half). Those four instructions are the four with no hardware encoding (Section 17). In order: the one with all five extra numbers at zero, then the one with only the four-body entry free, then three free entries (the two three-body entries and the four-body one, closed by a four-by-four Sylvester test), then all five. Each is proved sound on its own (`chsh_lassert_lab_no_trap_implies_npa_psd_qlab`, `chsh_lassert_lab_g5_no_trap_implies_npa_psd_qlab`, `chsh_lassert_lab_g345_no_trap_implies_npa_psd_qlab`, `chsh_lassert_lab_g12345_no_trap_implies_npa_psd_qlab`, QuantumPartitionPSD_1AB.v, 5076, 5104, 5156). Their parameter domains grow in that order. That the sets they *accept* grow too is not proved, and I won’t say it. The Python side (the assembler script and the generated opcode table, Section 17) gives them the numbers 0x2F to 0x32; no Coq file and no hardware file uses those bytes.

The two sizes are one test, not two. A single positive-semidefinite predicate takes the size as a parameter, and the five-by-five and the nine-by-nine are it at two sizes.

Definition 25.2 (The size-generic PSD test). A *claim* is a top index n and a matrix M indexed by naturals. It is *realizable* when $M_{ij} = M_{ji}$ for all $i, j \leq n$ and $\sum_{i \leq n} \sum_{j \leq n} v_i M_{ij} v_j \geq 0$ for every real vector v . The CHSH claim of E is top index 4 with the matrix $M(E)$ of Definition 25.1; the level-1+AB claim uses top index 8 and the nine-by-nine matrix.

Corollary 25.5 (The level-1 test is the size-five case). *For all real $E_{00}, E_{01}, E_{10}, E_{11}$: the three contraction inequalities hold if and only if the CHSH claim of E is realizable. The same holds at size nine for the level-1+AB contraction condition and its claim.*

Proof. Realizability at top index 4 unfolds to symmetry and PSD of the five-by-five, by computing the finite sums, and that is equivalent to contraction by Corollary E.13. At size nine the same unfolding meets Theorem E.20. Nothing new is proved about the matrices; the point is that one predicate covers both. $\square$

Checked as `column_contractive_iff_general_realizable` (GenRealizability.v) and `column_contractive_qlab_iff_general_realizable`, with the machine-step form `vm_chsh_lassert_step_is_general_realizable`, which has the hypotheses of Theorem 25.2.

The third thing: the *elliptope*, the set of correlators for which *some* choice of the two missing entries (A0 with A1, B0 with B1) makes the five-by-five PSD. Every one of the sixteen plans is inside it (`deterministic_strategy_elliptope`), mixtures stay inside (`elliptope_convex`), Tsirelson’s point is inside (`elliptope_tsirelson`), the PR box is out (`pr_box_not_elliptope`), and it’s strictly bigger than the classical set (`beyond_classical_elliptope`). There’s a decider for it with a soundness proof, in two branches: a strict Sylvester check with the completion supplied, and a rational factorisation certificate of the completed matrix, checked in integers. What’s proved is that a pass puts the correlators inside the set and that no choice of inputs gets the PR box through (`elliptope_check_full_sound`, `elliptope_full_gate_never_accepts_pr_box`), and, by computation, that some named points pass, the all-ones plan and a point that the cited arc-sine boundary puts exactly on the quantum edge among them (`gate_accepts_all_ones`, `gate_accepts_pythagorean_boundary`). The file’s header also argues that every point with a rational PSD completion has such a certificate; that is a comment, not a theorem. It is *not* an instruction. No step of the machine mentions it, and its file says the opcode and cost schedule are not done. I may call it a decider with a soundness proof. I may not call it part of the machine.

25.8 What isn’t proved

The arc-sine curve. That any of these sets equals what quantum mechanics can reach. That any state or measurement achieves 2.83. Those are the cited external results, and the files say so in their scope notes. Also: the finite-mixture theorem (`fine_theorem` in the source) proves only one direction, that a finite local model satisfies the bound; Fine’s actual theorem is an equivalence, so “Fine’s theorem is proved” would be wrong and I haven’t said it. And one more thing the kernel says about itself, which the guarantee paragraph above already used: a zero-cost trace is not thereby classical (the S equals four trace). The correlators (one, one, one, minus one) are the PR box, not a classical play; the classical vertex that fails the gate is (one, one, one, one).

That’s the game, the number, the two walls, and the check. Last of the machine sections: what all this costs against physics, and the hierarchy of what costs more.

26 Irreversibility accounting and the mu-hierarchy

Section 4 did the physics argument once: on a finite machine a permanent flag merges states, Landauer’s principle (a named premise) prices merges, and the toll follows. I’m not going to do it again. This section is about what the *machine’s* counter has to do with any of that, and the answer is: less than the word “Landauer” in a file name suggests, and I’d rather you hear the exact amount from me.

26.1 A bookkeeping indicator, bounded by what it indicates

The machine has a function that, for each instruction, returns one if the instruction’s cost is positive and zero otherwise. The source calls it `irreversible_bits`, and the name is the problem, so let me say what it is before saying what it’s called. It’s an *indicator*: it marks cost-positive instructions. Sum it over a run and you get the count of cost-positive instructions executed. The theorem is that this count is at most the run’s total cost (`total_irreversible_bits_le_cost`), and the proof is two cases per instruction: if the cost is zero the indicator is zero; if the cost is positive the indicator is one and one is at most the cost. A zero-one indicator bounded by what it indicates. That’s all.

What it shares with Landauer: the shape, a lower-bound quantity attached to an event. What it doesn’t share: everything else. It’s not an irreversibility theorem. There’s no physics in the proof, no microscopic erasure model, no physical entropy, no Boltzmann constant. The label “irreversible” on a cost-positive instruction is bookkeeping, not a proof that the instruction erased anything or was physically irreversible. Section 23 showed you that the machine’s zero-cost JUMP *does* merge states and is charged nothing. And several of the cost-positive instructions (the check, the join, the emit) change only the program counter and the ledger when they succeed; by the step rule they’re one-to-one on the state, which I get from reading the definition and not from a stated theorem. The indicator counts charges. It doesn’t count erasures.

Definition 26.1 (The indicator). For an instruction i with schedule cost $\text{cost}(i)$, let $\text{ind}(i) = 0$ if $\text{cost}(i) = 0$ and 1 otherwise. For a list t of instructions, $\text{ind}(t)$ and $\text{cost}(t)$ are the sums over its members.

Theorem 26.1 (The indicator is bounded by the cost). *For every list of instructions t , $\text{ind}(t) \leq \text{cost}(t)$.*

Proof. Per instruction, two cases: cost zero gives $0 \leq 0$, positive cost gives $1 \leq \text{cost}(i)$. Induction on t adds the cases up. $\square$

Checked as `irreversible_bits_le_cost` and `total_irreversible_bits_le_cost` (LandauerDerivation.v, 240). The source names the indicator `irreversible_bits`.

26.2 The calibration is a unit conversion

There's a function in the source that multiplies the counter by a chosen energy per mark, and a theorem that a state with one mark, at the scale $kT \ln 2$ per mark, carries $kT \ln 2$ of energy (`calibrated_mu_landauer_energy`, proving the predicate `mu_landauer_calibration` for every k and T). Read that for exactly what it is: a unit conversion, true by definition, with the scale as a free input. Nothing in it picks the scale. The theorem would be just as true with the scale set to a dollar a mark, and I'd rather say that than let the name "Landauer" in it do any work. What *would* connect marks to joules is the premise in Section 4, `landauer_heat`, and that one is an assumption with a name that nothing proves.

Definition 26.2 (Energy at a scale). For a real number c (joules per mark) and a state s , the energy of s at scale c is $\mu(s) \cdot c$. The *Landauer calibration* at k_B and T is the statement that every state with $\mu(s) = 1$ has energy $k_B T \ln 2$ at scale $k_B T \ln 2$.

Theorem 26.2 (The calibration is a unit conversion). (1) For all real k_B and T , the Landauer calibration at k_B and T holds. (2) There are two energy readings of the state, at scales 1 and 2, that disagree on every state with one mark.

Proof. (1) Substitute $\mu(s) = 1$: $1 \cdot k_B T \ln 2 = k_B T \ln 2$, by ring arithmetic, for any k_B and T , positive or not. (2) At $\mu(s) = 1$ the two readings are 1 and 2. $\square$

Checked as `calibrated_mu_landauer_energy` (`PricingPhysicsAudit.v`), which proves `mu_landauer_calibration` (`PricingPhysicsTarget.v`) for all k_B and T , and `mu_has_no_intrinsic_joule_value` (`PricingPhysicsAudit.v`). The physics is the premise of Section 4, restated here once in the form the next theorems test. In words first: a *dissipation* hands each instruction a whole number of heat units, each unit worth $kT \ln 2$. Landauer's principle, in that currency, says an instruction that takes at least one state where some yes-or-no property is false to a state where it's true, and never sends a true one back to false, dissipates at least one unit. The calibration says no instruction dissipates more units than the counter charges it. Two premises, nothing proved about either, and the only question is whether they can sit in the same room:

Premise 26.3 (Landauer, in units of the counter). A *dissipation* is a function δ from instructions to naturals, counted in units of $k_B T \ln 2$. Landauer's principle, in this vocabulary (Premise 4.5 of Section 4): every instruction that collapses a yes-or-no property has $\delta(i) \geq 1$, where i *collapses* P when some state with P false is sent to a state with P true and every state with P true is sent to one with P true. The *calibration*: $\delta(i) \leq \text{cost}(i)$ for every instruction. Neither is proved anywhere; the theorems below ask whether they can hold together.

26.3 The premises have to be able to hold together

Here's the one piece of this section that I think is good physics discipline, and it's a consistency test. Suppose you want to claim the counter tracks dissipation. You need two premises: that a positively charged step dissipates something positive, and an upper calibration that bounds dissipation by the charge. Can those two hold at once? There's a theorem saying exactly when (`calibrated_model_exists_iff_positive_cost`): a model satisfying both exists precisely when every operation you've chosen to count (the statement below calls those *eligible*) costs at least one. I should say how cheap that is. The model the proof builds takes the dissipation to be the cost itself, so the theorem shows the two premises don't contradict each other, not that the counter tracks any dissipation. Still, it does bite on a schedule: if you count a zero-cost step, no such model can exist. The one place the tree shows the premises holding is a one-bit reset, where resetting costs one and idling costs nothing (`bit_model_has_satisfiable_calibration`); for the machine's certifying steps it follows from that equivalence and the floor theorems of Section 23, and I haven't stated it as a separate theorem. The full machine fails the test, and there's a theorem that says so flatly: no dissipation function can both charge at least one for every step that collapses two classes of states, for any yes-or-no property of states you care to pick, and stay bounded by the machine's cost, because JUMP to one at cost zero collapses the classes of the property "the program counter is one" and costs nothing (`landauer_dissipation_premises_inconsistent`). The premise pair fails for the whole instruction set. That's the honest shape, and it's the same shape as Section 23's "prices the certifying merge, leaves the jump free."

Theorem 26.3 (When the premises can hold together). (1) For any set of operations, any predicate "eligible" on them, and any cost, a dissipation with $\delta(i) \geq 1$ on every eligible i and $\delta(i) \leq \text{cost}(i)$ on every i exists if and only if every eligible operation costs at least one. (2) On a one-bit machine with a reset of cost 1 and an idle of cost 0, taking "eligible" to mean "sends two different states to one", such a dissipation exists. (3) On the full machine, no dissipation satisfies Premise 26.3 for every yes-or-no property.

Proof. (1) If such a δ exists, $1 \leq \delta(i) \leq \text{cost}(i)$. Conversely, take $\delta = \text{cost}$. (2) The reset is the only operation that sends two states to one, and it costs 1; apply (1). (3) Let P be "the program counter is 1". JUMP to 1 with declared cost 0 collapses P : it sends the initial state, where P is false, to one where it is true, and every state lands at program counter 1. So δ of that jump is at least 1 and at most its cost, which is 0. $\square$

Checked as `calibrated_model_exists_iff_positive_cost` (`ObservationPolicy.v`, with `calibrated_positive_model` at :83), `bit_model_has_satisfiable_calibration` and `landauer_dissipation_premises_inconsistent` (`LandauerDissipationStrongForm.v`), with the definitions `step_collapses_bool_classes` (`LogicalErasureCertFlip.v`) and `cost_dissipation_calibrated` (`Land-`

dauerDissipationStrongForm.v).

But now the history escape from Section 4, pointed at the machine. Keep the previous state in a history list (push it onto the front each step; below, $s :: h$ is the list h with s put on the front) and every fixed-instruction step becomes injective, while the step you can see stays the same; the program counter can collapse to one place while the bigger state keeps its past. That move applied to the zero-cost jump is a theorem (`zero_cost_vm_jump_has_injective_history_lift`): the free merge, lifted to a machine with history, isn't a merge. So "this step merged, therefore it dissipated" is not a sentence about the visible step alone; it's a sentence about the whole state including whatever history is retained, and a physical model has to say what's retained. The permanent-certificate theorem in Section 4 supplies that for the certifying step on finite memory. For the rest of the machine it's unsaid.

Theorem 26.4 (Keeping history makes the free merge injective). *Let the history step send a pair (s, h) , a state and a list of past states, to $(\text{apply}(s, i), s :: h)$. For any step function, the history step with a fixed instruction is injective, and its first component is the original step. In particular JUMP to 1 with declared cost 0 costs 0, takes the initial state with empty history to program counter 1, and its history step is injective.*

Proof. Undoing the history step, by popping the list, returns the old pair, so two pairs with the same image are equal. The rest is computation. $\square$

Checked as `retained_history_step_injective` (ObservationPolicy.v) and `zero_cost_vm_jump_has_injective_history_lift`.

26.4 The calorimeter protocol: a specification, not a measurement

Here is how one *would* measure this, and it's here because it makes the boundary exact. Take a two-state register with energies zero and a gap, a population p in the excited state, and the simplest possible rate equation for how p changes in one time step (a rate for climbing into the excited state, a rate for falling out of it, and the length of the step). For the canonical reset (population one half to zero, with the rates chosen so the equation closes), the heat delivered to the bath is exactly half the gap (`canonical_reset_heat_exact`). Choose the gap to be twice $kT \ln 2$ and that heat is exactly $kT \ln 2$ (`selected_gap_gives_landauer_heat`).

Read that twice, because the temptation is to read it as a derivation of Landauer's number and it's the opposite. The gap was *chosen* to make the heat come out as Landauer's number. The same populations and rates with half that gap give half the Landauer value, and there's a theorem exhibiting identical population dynamics with different heat (`master_equation_does_not_fix_heat_scale`). The rate equation fixes the *shape*; the energy scale is a selected Hamiltonian (the physicist's name for the rule that says what energy each state has), and selecting it is not deriving it.

Definition 26.4 (The two-state protocol). For a gap Δ , the register's mean energy at excited population p is $\Delta \cdot p$. One step of the rate equation is

$$p' = p + \delta t (k_{01}(1 - p) - k_{10} p),$$

and the heat to the bath over a step from p to p' is defined as $\Delta p - \Delta p'$. The canonical reset takes $p = \frac{1}{2}$, $\delta t = 1$, $k_{01} = 0$, $k_{10} = 1$, ending at $p' = 0$, and counts one logical event.

Theorem 26.5 (The heat is fixed by the chosen gap). (1) The canonical numbers satisfy the rate equation. (2) For every Δ , the canonical reset's heat is $\Delta/2$. (3) For all k_B , T , the gap $\Delta = 2k_B T \ln 2$ gives heat $k_B T \ln 2$. (4) If $k_B > 0$ and $T > 0$, the gap $k_B T \ln 2$ gives heat strictly below $k_B T \ln 2$. (5) There are two different gaps for which the same canonical populations satisfy the rate equation and the heats differ.

Proof. All five are field arithmetic on the definitions: $\frac{1}{2} + 1 \cdot (0 - \frac{1}{2}) = 0$; $\Delta \cdot \frac{1}{2} - \Delta \cdot 0 = \Delta/2$; and $\ln 2 > 0$ for (4). For (5) take gaps 1 and 2. $\square$

Checked as `canonical_reset_satisfies_master_equation`, `canonical_reset_heat_exact`, `selected_gap_gives_landauer_heat`, `smaller_gap_refutes_unconditional_landauer_floor` (given $k_B > 0$ and $T > 0$) and `master_equation_does_not_fix_heat_scale` (`CalorimeterProtocol.v`, 19, 29, 41, 57), with the definitions in `CalorimeterProtocolTarget.v`.

What ties it to the machine: the protocol counts one logical event (the file defines that count as the constant one), and that equals what CERTIFY from the clean state charges, and no step of the machine switches certification on for less (`vm_minimal_certification_charges_canonical_reset_mu`, `vm_certification_charges_at_least_canonical_reset_mu`, given a step that turns certification from off to on). Both are stated for the unbounded sibling of the step from Section 24, not for the sixty-four-bit one. There is no theorem mapping the protocol's two register states to any machine state. The file's "heat to the bath" is the register's mean energy before minus after, so it also assumes the bath receives what the register loses, and no Coq says that. So a device test would have to measure the gap, the populations, the rates, and the bath heat independently; separately establish thermal admissibility and detailed balance (that the up and down rates stand in the ratio the temperature dictates); and separately establish that the device's physical reset is the machine's certify event. The protocol is an executable experimental specification. It is not a measured device, and it is not an unconditional conversion of marks to joules, and anyone who tells you otherwise is reading a file name.

Theorem 26.6 (The protocol's one event against the machine's floor). Write `apply_u` for the unbounded step. (1) From the clean state, whose flag is down, CERTIFY with declared

cost 0 under apply_u raises the flag and raises μ by exactly 1, the protocol's count. (2) For every state s with the flag down and every instruction i , if $\text{apply}_u(s, i)$ has the flag up then $\mu(\text{apply}_u(s, i)) \geq \mu(s) + 1$.

Proof. (1) is computation. (2) is the floor for the unbounded step, only CERTIFY sets the flag and it costs its declared cost plus one, with conservation for apply_u . $\square$

Checked as `vm_minimal_certification_charges_canonical_reset_mu` and `vm_certification_charges_at_least_canonical_reset_mu` (CalorimeterProtocol.v, 97). No theorem connects the protocol's two register states to any machine state.

There's one more calibration in the files, and it gets the same treatment. It feeds a chain of files (under `coq/kernel/curvature`) that this book doesn't otherwise visit, so here is the chain in plain words. In 1995 Ted Jacobson got Einstein's equation of gravity out of thermodynamics, by asking that heat equal temperature times the change in entropy (Clausius's old relation) across every tiny horizon, with the temperature being the one an accelerating observer sees in empty space (Unruh's, from 1976: accelerate hard enough and the vacuum looks warm). The files try a discrete version of that walk, starting from the counter. The calibration is the step that ties counter marks to that Unruh heat, and the end of the chain is a statement about curvature on graphs, a discrete form of the Gauss-Bonnet identity: a closed surface's total curvature is fixed by its shape up to stretching, through one whole number, the Euler characteristic, which for a closed surface is fixed by how many holes it has. The files say plainly that this is an analogy to Einstein, not his equation recovered. The symbols below are the ones the file uses; $\hbar$, c and k_B are the usual physical constants (Planck's, the speed of light, Boltzmann's), and $a(P)$ is an acceleration. The rest are borrowed physics words wearing very small costumes, so here's what each one is in the files. A *split* P is a row of cells cut into a left list and a right list, with some pairs of cells coupled across the cut. A *support* is a list of pairs of values, the joint states the two sides can be in together; there's one for before and one for after. The *entanglement entropy* of a support, in bits, is the base-two logarithm, rounded up, of how many different left-side values show up in it: a counting stand-in for the physicist's quantity, not the quantity itself. The *entropy increment* is that number of bits times the entropy per bit. A *congruence*, in relativity, is a bundle of light rays; in the files it's a record of two numbers for each split, and the *focusing factor* is the first minus the second. For the one congruence the calibration uses, the two numbers are the split's acceleration and its boundary size, and the focusing factor comes out to exactly 1 (`boundary_geodesic_focusing_unit`, `RaychaudhuriFluxBridge.v`). Small costumes, like I said.

Premise 26.5 (The Unruh-Clausius calibration). For constants $\hbar$, c , k_B , an entropy per bit, two states s, s' , a split P and two supports, `mu_landauer_unruh_calibrated`

is the equation

$$(\mu(s') - \mu(s)) \cdot A(P) \cdot \theta(P) = T_U(P) \cdot \Delta S,$$

where $A(P)$ is one plus the boundary size of the split, $\theta(P)$ is a focusing factor that is 1 for the congruence used, $T_U(P) = \hbar a(P) / (2\pi c k_B)$ is an Unruh temperature for an acceleration $a(P)$ read off the split, and ΔS is the change in entropy increment between the supports. It is a hypothesis of the theorems that use it, not a theorem.

Remark 26.1 (What the Unruh-Clausius calibration does and doesn't do). With $c > 0$, $k_B > 0$ and the entropy per bit set to $k_B \ln 2$, it can be derived from two further premises: that $\hbar \ln 2 = 2\pi c$ as numbers, which is a choice of units, and that the change in entanglement entropy in bits between the supports equals the change in μ , which is a claim about the run (`mu_landauer_unruh_calibrated_from_constant_and_bit_calibration`, `BekensteinCalibration.v`). So it is a unit choice plus a substantive premise, not a unit conversion alone. What it feeds is small: when the counter goes up and the calibration holds, the calibrated flux across the split isn't zero (`nfi_cost_nonzero_implies_nontrivial_calibration`, `NoFIToEinstein.v`). The curvature end of the chain doesn't use it at all. `discrete_gauss_bonnet_delta` (`NoFIToEinstein.v`) says that for two well-formed triangulated graphs the change in total curvature equals a constant times the change in Euler characteristic, and its only premises are the two well-formedness hypotheses: it's the discrete Gauss-Bonnet identity `einstein_emerges` (`EinsteinEmergence.v`) for a pair of states. No theorem derives curvature from the counter, and none joins the cost side to the curvature side. (`discrete_einstein_emergence_component`, `ThermoEinsteinBridge.v`, takes the Clausius data as arguments and ignores them.)

There's one more limit on that chain, and it's about where the geometry lives. The curvature files prove things about partition graphs in general, any graph you can write down. The graphs the machine actually reaches from its initial state are much plainer, and that's proved too.

Theorem 26.7 (What the machine reaches is flat). *Let s be any state reachable from the initial state. (1) The regions of distinct modules of s share no address and the module numbers are distinct, so no two distinct modules are adjacent, every module has no neighbours and no triangles, and the face-triangle count is 0. (2) Every module m reads flat: its Laplacian is 0, its angle-defect curvature is 2π and its calibration residual is 2π . (3) So s is calibrated exactly when it has no modules. (4) If the graph of s passes the triangulation test, it has no interior edges, and with F triangles it has $3F$ boundary edges, $3F$ edges, $3F$ vertices and Euler characteristic F . (5) That case happens: one PNEW of the cells 0, 1, 2 from the initial state reaches a triangulated state with one triangle and Euler characteristic 1.*

Proof. (1) Start from the initial state, whose graph is well-formed with no modules:

every step keeps regions disjoint and module numbers distinct, by induction on the run, and modules adjacent by region would have to share an address. (2) With no neighbours, the Laplacian's sum is empty, and with no triangles no angle is taken off 2π . (3) Calibrated means residual 0 at every module, and (2) makes every residual 2π . (4) Triangles that share no vertex share no edge, so each of the F triangles brings three boundary edges and three vertices of its own, and $3F - 3F + F = F$. (5) is computed. $\square$

Checked as `vm_reachable_regions_separate`, `reachable_no_adjacent_modules`, `reachable_flat_reading`, `reachable_calibrated_iff_no_modules`, `reachable_triangulated_isolated` and `reachable_triangulate_d_exists` (`ReachableGeometry.v`). The region and adjacency facts are closed; the flat reading, the calibration and the Euler count rest on the standard real-number assumptions. So the Gauss-Bonnet identity is a true statement about graphs, and on a running machine the surfaces it talks about are separate triangles: every edge is a boundary edge, so no run reaches a closed surface, let alone one with holes. The chain's theorems stay theorems; what they're about is a graph you can write down, not one this machine has been seen to build.

26.5 What permanence adds to Landauer: nothing new, said plainly

Here's a fair question: is there a prediction the permanent-certificate theorems make that Landauer's principle alone doesn't? No, and the reason is short. The smallest case, one certified state and one flipping in, is "set a bit to one whatever it held," which is the reset Landauer described and Bérut measured; Landauer's own name for it is "restore to one." The general bound, $kT \ln((m \text{ plus } k) \text{ over } m)$ for squeezing m plus k equally likely states into m , is Landauer's formula for that squeeze. What the permanence theorems add is the *derivation*: which premises about a machine (finite, permanent) make the certifying step a reset, so that Landauer applies to it. That's a result about the machine, not about physics, and it's the right size.

26.6 The mu-hierarchy, and how small it is

Computer science has a famous result called the time-hierarchy theorem: more time buys you problems you provably couldn't solve with less. The silhouette is "more of the resource buys something you couldn't have had." Here's the machine's version, and then how much smaller the thing inside the silhouette is.

Call a run *level- k certified* when it ends with the flag up and its executed instructions include a CERTIFY whose charge (declared cost plus one) is at least k . Then for every k at least one: there's a level- k certified run with total cost exactly k (the one-instruction run CERTIFY with declared cost k minus one; its charge is k , it certifies, and that's the whole bill), and any level- k certified run costs at least k (the executed CERTIFY at level k charged at least k , everything else at least zero, and Section 6's conservation sums them) (`mu_hierarchy_theorem`). Corollary: no fixed budget caps all certifications,

because for any budget B the level B plus one run exceeds it (`mu_hierarchy_no_upper_bound`).

Definition 26.6 (Level- k certified). For fuel F and a program t , let $\text{exec}(F, t)$ be the instructions the program-counter runner actually executes from the initial state in F steps, in order. The pair is level- k certified when the final state has the flag up and $\text{exec}(F, t)$ contains a CERTIFY with declared cost d and $d + 1 \geq k$. The cost of the pair is the final μ minus the initial μ , and the initial one is 0.

Theorem 26.8 (The μ -hierarchy). *For every $k \geq 1$: some pair has cost exactly k and is level- k certified, and every level- k certified pair has cost at least k . Consequently, for every budget B there is a $k \geq 1$, namely $B + 1$, such that no pair with cost at most B is level- k certified.*

Proof. Witness: the one-instruction program CERTIFY $k - 1$, run for one step, charges k and sets the flag. Lower bound: the executed CERTIFY charged at least k , its charge appears in the run's ledger entries, every other entry is at least zero, and conservation makes the final μ the sum of the entries. The budget statement is the lower bound at $k = B + 1$. $\square$

Checked as `mu_hierarchy_theorem` (`MuHierarchyTheorem.v`), with `level_k_certified`, `mu_hierarchy_strict`, `mu_hierarchy_no_upper_bound` and `mu_dimension_unbounded`.

How small is that? Very. The time-hierarchy theorem is deep because it separates *what can be computed* with more time. This separates *what was charged*: a run that declared a bigger charge was charged more. It's conservation plus the floor, and the "level" is defined by the charge itself. I keep it because it's true and because the silhouette is instructive: the counter is a resource in the formal sense that more of it is strictly more, and no budget is enough for everything. I don't keep it because it's surprising. It isn't.

26.7 What this section is

A ledger that counts charges, bounded by cost; a calibration that's a premise; a consistency check the premises pass; a measurement protocol that's a specification with a selected scale; and a hierarchy that's arithmetic. The physics in this book is in Section 4, as a named premise with its critics named. Here there's accounting, and I've tried to call every line by its right name. That's the end of the machine. The chip is next, and after that, what it took to check any of this and where the checking stops.

Part V

The hardware

27 From the proof to the chip

Aristotle’s bronze, from Section 2: the same statue in three materials, and a promise I made in Section 15 that I’d keep here. The materials are a step relation in Coq, a hardware model in a language called Kami, and Verilog that a tool chain turns into a configuration file for a programmable chip. This section walks the chain from the top down and, at every link, says one of three words: *proved*, *tested*, or *trusted*. The discipline is the seL4 paper’s, where a team that had just proved an operating-system kernel correct spent a paragraph listing what they *hadn’t* proved (the compiler, the assembly, the hardware), and I think that paragraph is the most important one in the paper. So here’s mine, at length.

27.1 Hardware as rules

First the idea that makes any of this possible, because it’s not obvious that a circuit can be a thing you prove theorems about. In the early 2000s Arvind’s group at MIT described hardware as a set of *rules*: each rule is a condition (when may I fire) and a batch of register updates that happen all at once or not at all. The compiler’s job is to build the arbitration logic so that rules which don’t conflict can fire in the same clock cycle while the result still looks as if they fired one at a time. Bluespec is the commercial language built on that idea; its compiler, *bsc*, went open source in 2020. What the idea buys: a circuit whose behaviour is “as if one rule at a time,” which is a thing you can reason about the way you reason about a program.

Then Kami, from Chlipala’s and Arvind’s groups in 2017: a Coq library for writing Bluespec-style hardware *inside Coq* and proving things about it as transition systems; their case study verified a family of multicore RISC-V processors (RISC-V is an open, freely usable instruction set). What it buys: one Kami design refines another, checked by Coq. What it leaves trusted is the path *out* of Coq: if the Bluespec compiler preserves meaning, the circuits inherit the proofs. If.

27.2 The chain, top to bottom

The CPU as a Kami module. The processor is one Kami module, with registers (sixteen general registers of thirty-two bits, a hundred and twenty-eight words of data memory, a hundred and twenty-eight instruction words of a hundred and twenty-eight bits each, the partition table, the arrow tables, the tensor store, the eight counters) and rules: a main `step` rule, and multi-cycle state machines for the checking step, for the two arrow-building instructions that read memory, and for the game check (twenty-nine firings, Section 25). Two methods (a method is an entry point the module offers to whatever sits outside it) matter for what follows: `loadInstr`, which writes

one instruction word, and `start`, which clears the halted flag and sets the program counter to zero and does nothing else. The CPU *resets halted*: it comes up stopped, so a program can be loaded before anything runs. The arrow tables are six, of sixteen slots each: arrows, coupling descriptors, formula descriptors, certificate descriptors, descriptor metadata, and coupling pairs. The checking step’s machine keeps two buffers of sixty-four words for the formula and its certificate. And the ledger is written by the same `step` rule that executes the instruction, not by a separate datapath. (ThieleTypes.v and ThieleCPUCore.v give these numbers.)

Start from reset: proved. From the reset state, calling `start` produces exactly the state every execution theorem begins from, calling no other method (`start_from_reset`), and that call is a legal step in Kami’s semantics (`start_substep`).

Each instruction refines the specification: proved, under premises. Three layers. A firing of the real `step` rule, read through Kami’s semantics, updates the registers exactly as a Gallina function `step_next` says (Gallina is the plain programming language inside Coq, the one the kernel’s step is written in) (`rule_next_correct`, `step_next_correct`). Then one theorem per opcode, thirty-eight of them generated by a script plus ten more for the arrow instructions at both encodings: if the fetched word is that opcode, the machine is live (not halted, no error, every multi-cycle machine idle), the alarm is down (the alarm is a flag the chip raises when the ledger falls below the tensor total), and the stated counters don’t overflow thirty-two bits, then the hardware snapshot after the step equals the abstract `kami_step` of the decoded instruction (`step_add_refines` and its siblings). The overflow premises are where the two machines meet: the kernel’s registers are sixty-four-bit words and the chip’s are thirty-two. Some of these theorems carry more premises (an address inside the active module’s range, module ranges that don’t overlap, and others), so I’ll stay with “under its stated side conditions.” The full admission catalogue, including the different conditions on the two arrow encodings and on fault paths, is printed in Section F. Then the multi-cycle ones: the game check retires through its twenty-nine firings to the same place (`chsh_lassert_refines`, with `chsh_only_rule_enabled` saying nothing else can fire meanwhile), the checking step through its header and scan firings (`lassert_sat_refines`, `lassert_unsat_refines`; the scan’s premises include that the formula’s clause count is at least one and at most the number of terminator words it holds, and that the formula’s addresses and the charge fit in thirty-two bits; the abstraction function agrees with the kernel on the ledger always, and on the program counter when the check passes, `logic_engine_equivalent_lassert`), and the memory-reading arrow instructions under their admission premises (table room, modules present, pairs fit; `morph_ext_retire`, `compose_ext_retire`). A master theorem gathers them: any instruction in an “admitted” list of fifty-five cases (some opcodes at two encodings, plus fault paths) retires, meaning from a live state the scheduler picks `step`, every later firing runs to an idle state, and that state matches `kami_step` (`admitted_retires`). The word “admitted” there means

“admitted under premises,” not Coq’s `Admitted`, and no proof in the hardware tree ends in Coq’s `Admitted` (the pun would have been unfortunate). Finally `kami_step` meets the kernel’s own `vm_apply` through an abstraction function (`embed_step_compute`), whose file sorts the opcodes into unconditional, preconditioned, and covered elsewhere.

The refinement theorems are equations between snapshots, field by field, which is the stronger of the two contracts Appendix A tells apart, not a looser relation picked to make the proofs go through. (The checking step against the kernel is weaker than that, and says exactly where.) Here are the words and five of those theorems exactly: one per-opcode refinement, two multi-cycle ones, the checking step against the kernel, and the master theorem.

Definition 27.1 (Board state, snapshot, step firing). A board state b is the value of every register of the Kami module. $\text{snap}(b)$ reads off the fields the specification talks about, as natural numbers, and kstep is the Gallina function `kami_step` that the specification gives for one instruction on a snapshot. $\text{next}(b)$ is the board state after one firing of the `step` rule, the instruction word $\text{fetched}(b)$ being the one at the fetch address. b is *live* when it is not halted, has no error, and every multi-cycle machine is idle; the *alarm* is up when the ledger is below the tensor total. The hardware word width is 32 bits.

Theorem 27.1 (ADD refines). *Let b be a board state whose fetched word is an ADD word with destination d , sources r_1, r_2 and declared cost c . Suppose the alarm is down, the error flag is false, the halted flag is false, $\text{pc}(b) + 1 < 2^{32}$, $\mu(b) + c < 2^{32}$, and the sum of the two source registers is below 2^{32} . Then $\text{snap}(\text{next}(b)) = \text{kstep}(\text{snap}(b), \text{ADD } d \ r_1 \ r_2 \ c)$.*

Proof. The step rule’s writes, read through Kami’s semantics, are given field by field by generated lemmas, one per field of the snapshot: the program counter goes up by one, the ledger by c , register d gets the 32-bit sum, everything else is unchanged. With the three bounds, each 32-bit addition equals the natural-number addition the specification uses, and the two snapshots agree field by field. $\square$

Checked as `step_add_refines` (`StepRefine.v`), on top of `rule_next_correct` and `step_next_correct` (`RuleStep.v`, 66). The thirty-seven siblings have the same shape with their own bounds (`SUB`, for instance, asks that the subtrahend not exceed the minuend).

Theorem 27.2 (The game check refines). *Let the fetched word of b be `CHSH_LASSERT` with declared cost c , the alarm down, no error, not halted, $\text{pc}(b) + 1 < 2^{32}$, $\mu(b) + c + 1 < 2^{32}$, and the trap vector equal to the kernel’s trap address. Then after the step firing and twenty-nine firings of the check rule, the snapshot equals $\text{kstep}(\text{snap}(b), \text{CHSH_LASSERT } c)$.*

Proof. The step firing charges the ledger $c + 1$ and starts the check machine on the eight counters. The twenty-nine firings compute the seven integer facts at the widths of Section 25, and `chsh_run_result` gives the final program counter, error flag and error code as functions of the word-level check, which `chsh_check_word_correct` equates with the kernel's check for every eight 32-bit counters. Every other field is untouched by the check firings. $\square$

Checked as `chsh_lassert_refines (ChshRetire.v)`, with `chsh_run_result (ChshRun.v)` and `chsh_only_rule_enabled (ChshRetire.v)`.

Theorem 27.3 (The satisfiable LASSERT refines). *Let the fetched word of b be LASSERT of the satisfiable kind with formula register f , certificate register, length ℓ and declared cost c , and let a be the value of register f . Suppose the alarm is down, no error, not halted, $\text{pc}(b) + 1 < 2^{32}$, the trap vector is the kernel's trap address, the memory word at a is ℓ , $a + 3 + \ell < 2^{32}$, the clause count n stored at $a + 2$ satisfies $1 \leq n \leq z$, where z is the number of zero (terminator) words among the formula's literal words, and $\mu(b) + 8\ell + c + 1 < 2^{32}$. Then there is an m with $1 \leq m \leq \ell$ such that after the step firing, the header firing and m scan firings the check machine is idle for the first time, and the snapshot there equals $\text{kstep}(\text{snap}(b), \text{LASSERT})$.*

Checked as `lassert_sat_refines (LassertRetire.v)`, through the scan lemma that follows.

Lemma 27.4 (The hardware scan computes the kernel's checks). *For any two assignments g_m, g_c , numbers a, v, n and a list of words w : if $1 \leq n$, n is at most the number of zero words in w , and every word of w is below 2^{32} , then the hardware scan of w for n clauses returns the conjunction of the kernel's model check of g_m and countermodel check of g_c on the formula whose words are a, v, n followed by w .*

Proof. Induction on w . A zero word closes a clause: if it was satisfied, the scan moves to the next clause, and the last clause ends the scan; a nonzero word is a literal, read as a signed 32-bit number, whose truth under the assignment is the kernel's. The bound $n \leq$ the number of terminators keeps the list from running out before the last clause. For the theorem, memory words are 32-bit by construction, and the bounds on a and ℓ keep every address in range. $\square$

Checked as `lassert_sat_refines (LassertRetire.v)`, `lassert_unsat_refines` for the other kind, and `hw_scan_spec, hw_scan_certcheck (LassertSpec.v, 120)`.

The partition-only reading `abs0` and the full reading `abs` are defined field by field in Section F. The first comparison uses the partition-only reading; the later full-state theorem uses the full reading.

Theorem 27.5 (The checking step against the kernel). *For every snapshot h and every LASSERT instruction i , there is a kernel state v' with $\text{vm_step}(\text{abs}_0(h), i, v')$ such that $\mu(v') = \mu(\text{kstep}(h, i))$; if the kernel's check of the formula in $\text{abs}_0(h)$ passes, the program counters agree too; and the error flag of v' is that of $\text{abs}_0(h)$ when the check passes and true when it fails.*

Proof. Choose v' to be the kernel successor of $\text{abs}_0(h)$ for this LASSERT. Both definitions add $8\ell + c + 1$ to the ledger, independently of success, so their ledgers agree. On a passing check each advances the program counter by one. The kernel's success branch preserves the old error flag and its failure branch sets it to true. These substitutions give the three conclusions; equality of every state field is not asserted by this theorem. $\square$

So the ledger always agrees, the program counter on a pass, and the error flag in both cases. Checked as `logic_engine_equivalent_lassert` (`LogicEngineEquivalence.v`).

Definition 27.2 (Retires; admitted). An instruction i *retires* from b to d when b is live, the firings that follow the step firing (each chosen by the module's own scheduler while some machine is busy) run from $\text{next}(b)$ to d , d is idle, and $\text{snap}(d) = \text{kstep}(\text{snap}(b), i)$. *Admitted* means that the fetched instruction and initial board state satisfy one of the fifty-five cases in Section F. That catalogue gives the common conditions, each case's ledger increment and every additional premise.

Theorem 27.6 (Every admitted instruction retires). *For every b and i , if (b, i) is admitted then i retires from b to some d .*

Proof. A case split over the fifty-five constructors, each closed by its own retirement lemma, which packages a refinement theorem above with the fact that the scheduler picks `step` at a live state and that the multi-cycle firings run to idle. $\square$

Checked as `admitted_retires` (`RetireMaster.v`), with `Retire` and `admitted`.

And the whole-machine version, every opcode under its side conditions (`driven_step_wf`), which in the last assumption audit (Section 23) uses one standard axiom, functional extensionality on dependent functions. Most of the per-rule theorems above use that one and a second, `eq_rect_eq`, the axiom form of uniqueness of identity proofs; the audit lists which. Both are standard, and Section 28 accounts for them, along with how to rerun the audit on the sources in front of you.

The whole-machine statement compares `kami_step` with the kernel's step through an abstraction function, under a per-opcode condition, and lifts to runs.

Theorem 27.7 (The specification step is the kernel step). *Let abs be the full snapshot reading and WF the per-instruction conditions printed in Section F. (1) For every snapshot h and instruction i with $\text{WF}(h,i)$: $\text{abs}(\text{kstep}(h,i)) = \text{apply}(\text{abs}(h),i)$. (2) For every fuel F , program t and snapshot h such that WF holds at every state the run visits, for the instruction it fetches there: abs of the specification’s run of t for F steps from h equals the kernel’s program-counter run of t for F steps from $\text{abs}(h)$.*

Proof. (1) A case split on the instruction: the opcodes with no condition go through one shared lemma for the plain compute instructions, and each conditioned opcode has its own. (2) Induction on F , using (1) at each fetched instruction; the condition is asked only of states the run reaches. $\square$

Checked as `driven_step_wf` (GraphReconstructionBridge.v) and `driven_trace_commutes`, with `WFDDrivenRun` checking `WFDDrivenPrecondition` along the run. The condition is True for every opcode not named here. CALL and RET need a well-formed snapshot, with a program-counter bound for CALL. CHSH_TRIAL needs its four operands to be bits. LASSERT needs its length to agree with the memory header. TENSOR_SET and TENSOR_GET need valid indices, and TENSOR_GET an existing module. MORPH, COMPOSE, MORPH_TENSOR and MORPH_GET need the extended table invariant; MORPH_ID needs the empty-descriptor invariant and an existing nonempty module; MORPH_ASSERT and MORPH_DELETE need a well-formed arrow table. PNEW needs a table counter of at most sixty-four, a nonempty normalized region and a clear tensor slot; PSPLIT and PMERGE need a well-formed partition table, live operands with nonempty ranges (at least two cells for PSPLIT, two different modules for PMERGE) and clear fresh slots. None of the three needs room in the table. When the module numbers run out, or a PNEW range runs past memory, the chip traps the way the kernel does (Section 16), charges the counter and keeps running, and the theorem covers that trap like any other branch. The chip’s error word tells the partition faults apart, 0xBADF001D for a missing module number or a range past memory and 0xBADF001E for an overlap or a merge whose ranges don’t touch (`ERR_PARTITION_VAL` and `ERR_PARTITION_OVERLAP_VAL`, ThieleTypes.v); the kernel writes 1 for every fault. Reset doesn’t make these true forever; a run has to keep establishing them. This is a theorem about the Gallina specification, one level above the clocked rules.

Underneath all of that sit four small files that check the floorboards: that every one of the twelve CPU rules is something Kami’s evaluator can actually run; that a run picked rule by rule (always the first rule that’s ready) really is a Kami run, with a bound on firings, an early stop exactly when no rule is ready, and a ready rule at reset; that no register ever changes its declared kind on any run from reset; and thirteen facts about what the registers hold at reset. (In order, those are `CoreRules`, `CoreExecution`, `CoreTyping` and `DispatchReset`.) What none of them pins down: fairness, the allocation invariants along a reachable run, or meaning through the tools

below Kami. One more file is worth naming so nobody leans on it: `rtl_inventory_arithmetic` (`RTLGapRegistry.v`) is the equation $37 + 10 + 0 = 47$, an inventory count of opcodes (not of the refinement theorems above), and it would still hold if every hardware proof were deleted.

The arrow-building instructions run through a coupling state machine, and the picture is simple: it builds a list of cell pairs, then tidies it. Descriptor zero is reserved for the empty coupling, with base 0 and count 0 (`CouplingComposeKami.v`); the pair table holds sixteen entries; and a normalization loop runs over the newly built pair range (`CouplingFsmNorm.v`). Their fault branches retire in the step firing alone with the error set (`step_morph_ext_fault_refines` and its siblings, `CouplingFaults.v`), and their success paths retire under the admission premises (`morph_ext_retire`, `compose_ext_retire`). From `kami_step` down to the kernel they also need the extended table invariant above.

The loader: proved for the receive, the hand-off and the report, tested for the rest. Here's the part that answers a question worth asking in plain words: what's the point of a chip you can't load? The answer is a second Kami module, also extracted, not hand-written: a serial receiver. It samples one input pin through two registers (the standard way to avoid reading a wire mid-change), reads each bit at the middle of its period, and reassembles bytes in one of the oldest serial formats still in use: one low start bit, eight data bits least-significant first, one high stop bit, at 115,200 bits per second. A twenty-megahertz clock gives 174 cycles per bit, which is 0.22 percent slow, within tolerance. On top of that a small assembler takes a two-byte count N , then N instructions of sixteen bytes each, writes each through `loadInstr`, and then calls `start`. And a reporter sends fifteen bytes back: a header byte, a status byte, then the program counter, the counter, and the error code as four bytes each, and a trailer byte. That's the whole interface: one wire in, one wire out, four LEDs.

What's proved: take any program of one to a hundred and twenty-eight instructions, start the receiver from reset, and play on the simulated pin the serial frames of the count and of instructions zero through k . Then the loader holds instruction k addressed to slot k , its request flag has flipped once per instruction, and it asks for `start` exactly when k is the last one (`serial_program_load`); every cycle of that run is a call of the real sample method; the load rule passes exactly that address and word to `loadInstr` (`load_rule_hands_off`); and `loadInstr` puts the word at that address (`load_instr_writes`). A program of zero instructions is a separate, smaller fact: fed straight to the assembler, with no serial pin in between, the count bytes alone raise the start request (`empty_program_starts`).

And the way back out is proved too. Once the CPU is halted or in error, run the loader's report rule once per clock cycle. The pin sits high for one cycle, then carries the serial frames of fifteen bytes: the header `0xDE`, a status byte whose three low bits are halted, error and certified, then the program counter, the counter and the CPU's

error code as four bytes each, least significant first, and the trailer 0xAD. The values are the kernel state's, read through the same abstraction every execution theorem uses, so the bytes on the wire are the machine's state and not a separate story. A receiver built the loader's way, started idle, reads back exactly those fifteen bytes, and the pin stays high from then on (`serial_status_report`). Before a halt or an error the reporter does nothing (`report_waits`). Its premise is worth saying out loud: Kami's semantics don't say when a rule fires, so the theorem takes one firing of the report rule per clock cycle as given.

What's *tested only*: the rule that actually fires `start` and the LED output. No theorem names them. And one thing the loader file says about itself that I'll repeat: Kami's semantics let a rule fire whenever its guard holds, and the file does not prove that the load rule takes each word before the next replaces it. That's a real gap between "the receiver assembles the right word" and "the CPU got every word," and it's closed by the simulation, not by proof.

Here's the start, in the theorem's own words:

Theorem 27.8 (Start from reset). *From the reset register map, any execution of the start method calls no other method, and its updates applied over the reset map give exactly the state every execution theorem starts from: halted false and program counter 0, everything else at reset.*

Proof. The start operation makes two assignments: clear the halted bit and set the program counter to zero. It performs no call and preserves all other registers. Applying those assignments to the reset map gives exactly the stated map, field by field. $\square$

Checked as `start_from_reset` (CPUStart.v), with `start_substep` for the step of the module.

Theorem 27.9 (Serial load). *Let $w_0, \dots, w_{N-1}$ be instruction words with $1 \leq N \leq 128$, and $k < N$. Run the receiver model from its reset state over the serial frames of the bytes $N, 0$, and then the sixteen bytes of each of $w_0, \dots, w_k$. At the end, the loader holds address k and word w_k for the CPU, its request flag equals "k is even", and its start request is set exactly when $k + 1 = N$.*

Proof. One frame, a start bit, eight data bits and a stop bit, takes the receiver from idle back to idle having delivered exactly that byte once (`one_frame`, by computation). Induction over the bytes then follows the assembler: two count bytes, then sixteen bytes per word with the address counting up, the request flag toggling per word and the start request raised after the last. $\square$

Checked as `serial_program_load` (LoaderSerial.v), with `one_frame` and `step_`

`is_rxSample`, which says each cycle of the model is a call of the real Kami sample method. The zero-instruction case is `empty_program_starts`, in the same file.

Theorem 27.10 (Hand-off and write). *(1) If the load rule (`load_rule`) runs from a register map holding request r , acknowledge a , address x and word w , then $r \neq a$, its only method call is `loadInstr` with x and w , and its only update sets acknowledge to r . (2) If `loadInstr` runs with x and w , it calls no method and its only update writes w at address x of instruction memory.*

Proof. The load rule's guard is $r \neq a$. Once that guard holds, its action sends the stored address and word to the instruction-memory write method and copies r into the acknowledgement register. It has no other action. The called method updates the instruction-memory array at the supplied address to the supplied word, preserving every other array entry and every other register; it calls nothing. These are the two local contracts. They do not by themselves prove that every serial request eventually receives a matching call. $\square$

Checked as `load_rule_hands_off` and `load_instr_writes` (`LoaderSerial.v`, 695).

Theorem 27.11 (Status report). *Let b be the CPU's registers with the halted bit h or the error bit set, and let s be the kernel state they abstract to. Let R be the fifteen bytes: `0xDE`; the status byte $h + 2e + 4c$, where e and c are the error and certified flags of s ; the four bytes, least significant first, of each of the program counter of s , the counter $\mu(s)$ and the CPU's error code, each taken modulo 2^{32} ; and `0xAD`. Start the transmitter neither active nor done, and run the report rule once per cycle for $1 + 15 \cdot 10 \cdot 174$ cycles, the status getters returning b in the first cycle and anything afterwards. Then the pin reads high in the first cycle and then the serial frames of R in order; the loader's receiver, started idle, decodes that sequence to exactly R ; and for any further cycles the pin stays high and the transmitter does not change.*

Proof. In the first cycle the guard sees halted or error, each status getter returns its register of b , and the rule latches the fifteen bytes and starts sending. Each byte then goes out as a low start bit, eight data bits least significant first and a high stop bit, 174 cycles per bit, by induction over bytes, bits and cycles. The fifteen latched bytes equal R by computation on words. The loader's receiver turns each frame back into its byte (`one_frame`, above), so it reads R . After the last stop bit the transmitter is done, and a done transmitter keeps the pin high whatever the rule sees. $\square$

Checked as `serial_status_report` (`ReportSerial.v`), from `report_transmits`, `report_bytes_kernel`, `report_bytes_fields`, `status_byte_value` and `report_done_stays`, with `report_step_is_rule` (each modelled cycle is a firing of the real Kami rule, its calls the CPU's six getters), `get_tx_is_tx_out` (the pin is

the transmitter’s output) and `report_waits` (nothing happens before a halt or an error). Assumptions: functional extensionality and the uniqueness of equality proofs, the two Kami itself uses.

Composition: proved by reflexivity. The system is the CPU module followed by the loader module. The loader’s calls to `loadInstr`, `start`, and the status getters are the CPU’s own methods, so they close inside the design; what’s left over (sample, transmit, LEDs) becomes the pins. A one-line theorem says the printer (the program, below, that writes the design out as Bluespec text) receives the CPU’s syntax followed by the loader’s.

A second extraction, to software: tested. Separately from the chip, Coq’s extraction also turns the kernel’s step into OCaml, and a runner built on it executes programs. What Coq proves about it is about the Coq side. For every state and instruction, the observable record taken after one kernel step satisfies the floor (a flag that goes from down to up comes with the counter at least one higher) and never shows the counter lower than before (`ocaml_observable_nofi_and_monotone`, `OCamlExtraction-Bridge.v`). The lemma named `ocaml_runner_observable_defined` says only that the observable exists, which is true of any Coq function; it is not a cross-language equivalence. That the built binary returns the same observable is checked by a parity test suite, not proved. And the checked-in runner file (`build/extracted_vm_runner.ml`) takes each step by calling the extracted Coq step, so the steps themselves follow the current rules, but the code wrapped around them is its own and differs from the kernel: it starts both graph counters at one, stops on an error, stops at HALT and writes a status word, and advances CHECKPOINT without charging. Nothing in this book is a claim about that runner.

Extraction: trusted. Coq’s extraction turns the Kami syntax tree for the system into OCaml. The stock extraction command is an unverified translator (Letouzey’s 2003 design; later work proves erasure correct, but that’s not the command used here). And the step just before it, turning Kami’s module syntax into Bluespec’s, is a Kami library function in a file that contains no theorem at all. Two trusted links in a row, and I’m naming them.

Printing, patching, compiling: trusted, tested. A vendored printer (about a thousand lines of OCaml) prints Bluespec. A script then does things to the text that I’ll list because “nothing hand-written but clock primitives” is a tempting sentence and it’s not exact: it strips the printer’s wrapper module; rewrites one vector construction into the form the 2024 bsc accepts; turns large vector registers into register-file cells; runs bsc (2024.07) and deletes the date line bsc stamps, so the output depends only on the input; and turns flattened registers into Verilog arrays. For the system it also wraps the top module with an interface made of the outermost methods, a wrapper that keeps the printed instantiation lines verbatim and, by its own docstring, writes no expression, register, or rule. Each text transform is audited by a script that says

of itself that it is “not a compiler correctness proof.” And one Bluespec library cell, a hundred-line register file in Verilog, is read by synthesis and comes from nobody’s proof. These change storage shape, not logic, by their own account, and they’re tested, not proved.

The gate that catches hand edits: tested. The nightly full run rebuilds the extraction, reruns the whole pipeline, and fails unless the two Verilog files it produces are byte-for-byte identical to the tracked ones. A hand edit to the generated Verilog fails the build. And the CPU printed inside the system must compile byte-for-byte to the CPU printed alone, else the script exits; that’s how I know the system wrapper didn’t touch the CPU. Be clear about what this gate protects: that the tracked Verilog is exactly what the extractor prints from the current sources. It doesn’t say those sources are the ones the proofs were checked against; that’s the job of the proof build on the same commit.

Simulation: tested. Two programs (an arithmetic-then-halt and a jump loop) go in as serial bytes to the extracted system under a Verilog simulator, and again to a gate-level netlist that yosys builds from it, and both must match the CPU-alone reference on halted, error, program counter, counter, and error code. Two programs. Generic yosys gates, not the chip-specific mapped netlist. I’d want more of both and I’m telling you the number.

Synthesis, place and route, bitstream: trusted, tested. yosys maps the Verilog to the chip’s lookup tables and flip-flops; nextpnr-xilinx places and routes it; and Project X-Ray’s tools turn the result into a bitstream, the file that fills a programmable chip’s configuration memory and makes the blank grid into this circuit. None of those three carries a proof; yosys (2013) is a free synthesis suite covering most of Verilog, nextpnr followed, and X-Ray documented the chip’s bitstream format by building many designs with the vendor’s tool and cross-correlating the bits. Two tests sit on this link. A timing gate reads the clock-synthesiser settings out of the board wrapper, computes the clock it produces (twenty megahertz), and fails if nextpnr’s achieved frequency is lower; the last recorded run achieved 42.78 megahertz against the 20 required. And a round trip decodes the bitstream two ways: the raw bits must equal the frames that were encoded, and the bits decoded back to named features and re-encoded must match too, so a bit the database can’t name shows up as a mismatch.

The board wrapper: the one hand-written logic file. Seventy-two lines. It takes the board’s two-hundred-megahertz clock, which arrives as a pair of wires with opposite voltages, and turns it into one signal; feeds that to the chip’s clock synthesiser, which multiplies by five and divides by fifty to get twenty megahertz and raises a LOCKED flag when it’s stable; drives the clock onto the chip’s clock network only while locked, so the CPU never sees a half-formed clock; and then instantiates the extracted system, ties three enables high, and wires four LEDs. A pin file maps the clock pair, the reset button, the LEDs, and the two serial pins to the connector names of a specific board, a Digilent Genesys 2 (its chip is the Kintex-7 xc7k325tffg900-2, the part the synthesis

script targets), per the file’s note. The pin names match Digilent’s published master constraints file for that board.

27.3 Trusted, tested, proved: the table

Link	Status	What does the checking
Coq’s kernel (8.18)	trusted	the small checker every proof runs through; the base
Kami’s semantics	trusted as the definition of meaning	nothing; every hardware proof is stated in its terms
rule firing to <code>kami_step</code> to <code>vm_apply</code>	proved, under stated premises	RuleStep, StepRefine, the Retire files, <code>admitted_retires</code> , <code>EmbedStep</code> , <code>driven_step_wf</code>
loader receive and hand-off	proved	<code>serial_program_load</code> , <code>load_rule_hands_off</code> , <code>load_instr_writes</code> , <code>start_from_reset</code>
status report on the serial pin	proved, under one firing of the report rule per cycle	<code>serial_status_report</code> , <code>report_waits</code>
loader start rule, word-by-word timing of the hand-off, LED output	tested	the serial simulation; no theorem
CPU and loader composed into the system	proved	a one-line theorem, by reflexivity
kernel step extracted to an OCaml runner	tested	the parity test suite; nothing in the book rests on the runner
Kami syntax to Bluespec syntax	trusted	a library function in a file with no theorems
OCaml extraction of the system’s Kami syntax	trusted	an unverified translator
printer, text rewrites, bsc	trusted, tested	byte-identical regeneration; co-simulation
RegFile.v library cell	trusted	read by synthesis; from no proof
yosys	trusted, tested	gate-level simulation on generic gates
nextpnr	trusted, tested	the timing gate
X-Ray tools and database	trusted, tested	the bitstream round trip
board wrapper (the one hand-written logic file) and its clock primitives	trusted	vendor documentation and Digilent’s pin file; the timing gate reads the clock settings
the board and the chip	trusted, unchecked	no physical run is recorded anywhere

27.4 Why generated, and why it's the stronger artifact

Someone will say “it’s just generated code.” Here’s why that’s backwards. The generated Verilog is the artifact the proofs are *about*, one translation removed; a hand-written CPU would carry no proof at all. The CI gate makes any hand edit fail. So generated code is the stronger artifact, with its translators named as trusted, which is a better position than hand-written code whose translator is a person. And “it synthesized, so it’s correct” is the wrong turn in the other direction: synthesis checks that the text describes *some* circuit; a wrong circuit synthesises as happily as a right one. The evidence of function is the co-simulation and the serial runs, not that yosys finished.

27.5 Thompson’s knife, again

Every link marked “trusted” is a place where Ken Thompson’s compiler from Section 11 could sit unseen: a translator that plants something and re-plants it when it translates itself. Extraction, the printer, bsc, yosys, nextpnr, the X-Ray database. I don’t believe any of them does. I can’t prove it, and the discipline of this section is to say so rather than to let “formally verified” cover a chain whose bottom half is trust and tests.

27.6 The numbers, true of the last recorded run

Sixteen registers of thirty-two bits; a hundred and twenty-eight words of data and of instructions; instructions a hundred and twenty-eight bits wide; sixty-four partition slots, the same sixty-four module numbers the kernel hands out per run (Section 16), each module in its own slot; forty-seven of fifty-one opcodes encoded (Section 17); fifty-five retirement cases. From the last full run on record: 37,057 of 407,600 lookup-table sites (nine percent; nextpnr counts each half of a six-input lookup table as a site, and the datasheet lists 203,800 six-input tables), 8,336 flip-flops, no DSP slices, two block RAMs, a bitstream of 11.4 megabytes whose round trip matched 2,045,121 bits by encoding and 2,045,509 by meaning, and both serial programs agreed in RTL and in gates. Those numbers belong to one run, and the gate above is what ties any run’s numbers to the sources it was built from; when the sources change, the numbers do, and the gate says so.

27.7 What the chip is, and isn’t

The chip is the third material. The proofs reach from the kernel’s step relation to the firing of Kami rules. Everything below that is translated by tools I trust and checked by tests I’ve described in numbers. The bitstream is a wiring list, not evidence that the wiring computes anything; the evidence is the simulations. And no board was ever powered on: I don’t own one, the repository records no run on one, and I’d rather end this section on that sentence than on a flourish. What I can say is this. Any claim of the form “the proof says the chip does X” has to pass through this table, and the table says exactly where the proof stops.

Part VI

Checking it, and its edge

28 What Coq is, and why

Coq is a proof assistant, which is a phrase I had to go and look up. Underneath the phrase it's a programming language, but one whose type system is strict enough to do a thing ordinary languages can't: you write a proof down as a program, and the compiler refuses it unless the program genuinely matches the statement you claimed. You can't talk it into a maybe.

28.1 Types as claims

Here's the idea, and it's one idea. In a normal language a type says what kind of value a variable holds: in Python, "x is an int" means x is a whole number. In Coq, types are *claims*. The type "n is greater than zero" is the type of *proofs* that n is positive. A function from "n is greater than zero" to "n plus one is greater than zero" is a proof that if n is positive, n plus one is too, and the compiler checks that the function really has that type, the way it would check that a function from integers to strings really returns strings. This correspondence between proofs and programs has a name, Curry-Howard, after two people who noticed it separately decades apart, and Coq's foundation, the Calculus of Inductive Constructions, from Coquand, Huet and Paulin-Mohring in the late 1980s, is a type system built to exploit it. The "inductive" part means you define your own recursive types (numbers, lists, trees, machine states) inside the system instead of having them handed down as primitives, so the foundation is explicit; nothing is smuggled.

The compiler checks whether the types line up. If your proof has a hole and you write "trust me," it rejects you. But here's the part people skip, and it's the part that matters: the compiler checks that the proof matches the *statement*. It does not check that the statement says what you *meant*. Hand it a statement that means nothing and it'll prove it for you, three characters, no complaint. So a proof being accepted is not the same as the thing being true. It means this proof matches this statement, and whether that's the right statement is on me, not the machine. Section 3 said a short version of this and I'm saying it again because every section since has leaned on it: the checked names in this book are a promise that the *proof* is there, and the sentences around them are my claim about what the statement *says*, and the second can be wrong while the first is fine, which is why so many sections of this book stop to say exactly what a statement's hypotheses are.

28.2 Why I used it

Not to make anything true. I used it because I don't trust my own eye to catch a gap in an argument I want to believe. I spent months convinced by my own reasoning,

pushed the proof through, and watched a key step come apart in my hands. The machine caught where I'd fooled myself. That's what it's for. And it's the same source the running machine and the chip are extracted from (Section 27), so the thing that checks the argument is the thing that becomes the artifact. Gap-catcher and extractor. Not the proof. The proof is the argument, and an argument has to close in a way a person can follow, or it hasn't closed at all, which is why this book exists in prose and not as a list of theorem names.

Two precedents I leaned on when deciding whether a proof assistant was a serious tool or a toy. CompCert, Xavier Leroy's C compiler, written and proved correct in Coq: the assembly it emits behaves as the source does, for programs with defined behaviour. And Georges Gonthier's 2005 proof of the four-colour theorem, a century-old problem whose first proof (in 1976) needed a computer to check thousands of cases and was distrusted for exactly that reason, redone in Coq so that the *checker* was the small, auditable thing and the cases were the program. That's de Bruijn's criterion from Section 19 applied to mathematics itself: the thing you have to trust is the small kernel, not the big proof.

28.3 What “zero Admitted” means

“Zero Admitted” is literal. Nowhere in the proof tree did I write “trust me” and walk on. Every step the checker could see, it saw, end to end, with no “and the rest is obvious.” A hygiene fact about the gap-catcher, not a verdict on the argument: it says I didn't let myself skip a step where the machine was watching. It does not say the statements are the right statements.

What it doesn't mean, listed. It doesn't mean the theorems prove physical reality; they prove things about the machine as formalised, and whether that describes the world is answered by experiments (Section 4, Section 26). It doesn't mean Coq's own foundation is proved consistent; it's believed consistent, and that can't be proved from inside, by Gödel (Section 11), so the foundation is trusted, and “trusted” has substance: it's the same kernel CompCert and the four-colour proof stand on, and a bug in it would take all of them down together, which is a kind of safety in numbers. It doesn't mean the hardware correspondence is unconditional; Section 27's table says where the proof stops. And it doesn't mean no axioms: the proofs use five standard-library assumptions, and the assumption audit names them: functional extensionality on dependent functions (two functions that agree everywhere are equal); uniqueness of equality proofs, in the form the standard library calls `eq_rect_eq` (only in the hardware proofs, through the Kami library); two decidability principles that come with the standard library's real numbers; and, in a little over a hundred theorems, the law of excluded middle (every proposition is true or false). The count, from the full audit: the full-corpus probe covers 14,571 named theorems across 509 files: 6,991 close under the global Coq context outright, and 7,580 use only Coq standard-library assumptions. Zero project-local axioms appear in any of the 14,571 dependency trees. In plain words: about fourteen thousand theorems, a little under half of them needing

nothing at all, and the rest leaning only on those standard-library assumptions, none of mine. Those numbers belong to the audit run that produced them; the shape doesn't move between runs, the counts do.

28.4 What you're trusting, in one place

If you accept every checked theorem in this book, here is what you're still taking on trust, all of it, so you don't have to collect it from five chapters.

1. The statements. The checker confirms a proof proves its statement. It doesn't confirm the statement says what my sentence about it says. That's why the book prints the important ones in full, premises and all. A machine check supplements that reading.
2. Coq's kernel and its foundation, and the five standard assumptions above.
3. The path out of Coq. Programs pulled out of Coq as runnable code by its extraction mechanism, and the circuit description handed to the Bluespec compiler, are trusted from that point on. The synthesis and layout tools, the chip, and the board are trusted too, and Section 27 lays the chain out link by link. This is the list Ken Thompson's lecture from Section 11 was about. I didn't write Coq, or OCaml, or the Bluespec compiler, or yosys, so by his moral I can't fully trust any of them, and neither can you. What I can do is keep the list short and print it, so you know exactly which tools you're being asked to believe have no spare key cut into them.
4. The tested parts. The Python demonstrations and the test suite are evidence that the code does what's claimed on the runs tried, not proof that it always does.
5. The physics. Anywhere a theorem has Landauer's principle or a calorimeter in it, the physical premise is named and assumed (Section 4, Section 26), never derived.

28.5 The auditor on top

Beyond the type checker, the project has a custom proof auditor I called the Inquisitor, because a thing that reads every proof looking for ways you've cheated deserves a name you won't forget. It runs on every commit, and its source names about a hundred distinct rules. The main families: no holes (no Admitted, no admit, no give-up, no unproved axiom or parameter in project code); vacuity heuristics (no theorem whose conclusion is literally True or a tautology; no theorem of the shape "P implies P"; existence claims backed by a constant witness get flagged, and a flag is a reason to look, not a verdict, since some constructions like a fixed interpreter program are *supposed* to be fixed and get judged on what they compute); physics-definition heuristics (no physical quantity defined as a placeholder constant, no physics claim without an invariance condition); scope enforcement (the proof tree is split into tiers, the kernel tier may not import from research-side files, and every active file must

reach the cost or semantics foundation through its imports, or it doesn't get to claim the kernel's pedigree); and comment hygiene (no unfinished or historical markers in maintained source). Its rules are heuristics. They point at places to look. They don't prove a theorem means what it should, and they can't promise every empty premise got caught.

28.6 Checking the companion formalization

The mathematical claims are the statements printed in this book, with their stated hypotheses and the arguments given here. Running software is an additional way to check their formal counterparts. It isn't a prerequisite for reading the argument.

A reproducible check needs a fixed copy of the proof sources, dependencies and probes. Changing that copy changes the object being checked; a later software result cannot silently revise a theorem in this edition. The procedure below describes the companion proof build. With Python 3, GNU make, Coq 8.18 and its standard library, OCaml, `ocamlfind` and CSDP installed, run this from the source snapshot's root:

```
python3 scripts/reproduce_coq.py
```

That copies the sources into a fresh directory and works only from the copy: it builds the two vendored libraries the hardware proofs use (bbv and Kami), regenerates the build file, rebuilds every proof from source, runs the contract probes that re-check the named theorems and their assumptions, and runs Coq's independent checker over the result. It reuses no compiled proof from the checkout, and it writes its pass file only after every stage passes. A failed run resumes from the snapshot it captured, and a change to the sources needs a fresh run:

```
python3 scripts/reproduce_coq.py --resume \
  --output artifacts/reproduction/RUN_DIRECTORY
```

The run has to pass. Don't stop at the pass: read the theorem types and their global dependencies, because a closed report doesn't make a theorem's own premises go away. The hardware generation is a separate command needing the Bluespec compiler (Section 27), and its outputs have to name the source they were built from; a passing proof build doesn't give you a fresh synthesis.

That's the checker. What's left is to tell you how to break what it checked.

29 How to break this

Every main claim in this book comes with the exact shape of the thing that would kill it. I went to the trouble on purpose. A claim you can't break is a claim that wasn't saying anything, and I'd rather hand you a loaded gun pointed at my own work than a sentence too vague to aim at. So these are written as instructions: here is the trace to find, the object to build, the theorem to exhibit. Build one and the proof tree falls, and I mean that as an invitation.

29.1 The abstract claims (Parts II and III)

The universal floor (Section 3). Build a certification system that satisfies the one-step toll and give a run from uncertified to certified with total cost zero. `universal_nfi_any_substrate` falls. If you instead drop the toll, you’ve built a free-certification system, not a counterexample.

The toll from premises (Section 4). Exhibit a finite, duplicate-free list of every state, a certificate no step revokes, and an *injective* step that switches it on. `permanent_flip_is_not_injective` falls, and the derivation of the toll with it. An infinite state space, a revocable certificate, or an unpriced merge is not a counterexample; the file already builds one of each. For the squeeze bound, a finite permanent machine with a cost meeting the halving premise where one step switches k states on beside m certified ones and two to the cost times m is less than m plus k ; `permanent_flips_compression_bound` falls. For merge-or-revoke, a finite machine and an injective instruction that switches a reading on without switching it off anywhere; `flip_merges_or_revokes` falls. A move that merges and also revokes isn’t a counterexample to anything; it’s the three cards.

The bill in bits (Section 4). Exhibit a finite machine with a permanent certificate, a move that switches k states on beside m certified ones (k at least one), and a spread of probability over those m plus k states whose image under the move has entropy above log base two of m . `permanent_step_entropy_ceiling` falls. For the general form, a step and a spread under which two different states, each with some probability, land on the same state while the entropy doesn’t drop; `entropy_drop_pos_of_support_merge` falls. A heat measurement on a real device below what the bound asks doesn’t touch either theorem. It refutes the premise `landauer_heat` for that device, which is Landauer’s principle, and that’s a fight with physics, not with the counting. And a certifying step from a state already known that gives off no heat isn’t a counterexample: `known_state_flip_forces_no_heat` says that case owes nothing.

Learning is free (Section 5). Show that the four-state demon’s measure step merges two states, or that its observer’s candidate list doesn’t shrink. `observer_narrowing_can_be_free` falls. A different price schedule that charges the measure step is allowed and isn’t a counterexample; the claim is “not forced,” not “always free.”

The counter (Section 6; on this machine, Section 16 and Section 23). Find a state and an instruction such that the counter after is anything other than the counter before plus that instruction’s cost; the loud version is a counter that goes down. `vm_apply_mu` falls. Or find two bookkeepings that both start at zero, both follow the schedule step by step, and disagree on a reachable state; `mu_is_initial_monotone` falls.

Windows (Section 7). Exhibit a window and a price computed from the observed before and after that meets the floor and never overcharges, on a machine where two steps share their before and after pictures and only one certifies. `shadow_cannot_pri`

`ce_exactly` falls. A window that shows certification is not a counterexample; the theorem says exactly that such a window prices exactly.

Entitlement (Section 8; on this machine, Section 22). Produce a strict narrowing of a candidate list, with a distinguishing observation, that does not yield a strictly stronger membership predicate; `feasible_strict_subset_implies_strict_predicates` falls. Or exhibit a run meeting every hypothesis of the representation theorem whose counter rose by less than the index bits lost; `structural_entitlement_representation` falls.

The latch (Section 10). Exhibit an extension of a base whose record is driven by the computation and never switches off, and which is not a latch on any event. `record_axis_is_latch_holds` falls. A record that switches off, or one driven by a hidden clock, is the file's own pair of counterexamples, not yours.

The diagonal (Section 11, Section 24). Two routes. A decider for some substrate with a shortcut predicate whose flip the substrate can represent and which answers correctly on every program; the generic proof fails at a named step. For the nat family, a `d` that correctly decides the predicate of the substrate built from `d`. (The concrete machine's version offers no third route: its recursion premise is false for the class of all maps, and given a decider correct on every program, its flip has no fixed point with equal thousand-step runs from every state; Section 24 proves both, as `vm_full_recursion_premise_refuted` and `vm_correct_flip_has_no_fixed_point`.) The guest has no premise to supply, so it gets two targets of its own: a map on well-formed guest programs that some guest program represents, with no well-formed program equivalent to its own image, and `vm_guest_recursion_theorem_closed` falls; or a decider for a property of guest programs that respects equivalence, holds of some well-formed program and fails for JUMP 0, and `vm_guest_rice_holds` falls.

The pointer conjecture (Section 13). It's a conjecture, so the three routes are its own: a deployed discipline pricing a non-proliferating event; a rival event proliferating as well as certification in a real ecosystem; an honest formalisation with independence defined where certification fails the test.

One concession goes with it, and I make it in full. The window results, the separation of the two witness states, and the counter's uniqueness don't single out certification. Their proofs use only that the step carries something the classical picture doesn't determine, so the same shape of result holds for any such field. Nothing in Parts II to V proves that certification is the one event a model is *forced* to price; Section 4 forces a price on every merge, certifying or not. That's the conjecture, and Section 13 is the case I make for it.

29.2 The machine claims (Parts IV and V)

The partition (Section 16). From the initial state, run any program and land on a state where two modules share a cell, a module owns a cell numbered 128 or more, the

module counter is past sixty-four, or two modules carry one number. `vm_reachable_partition_in_bounds` falls (and with the first, `vm_reachable_regions_disjoint`). A PNEW past memory or on a spent budget that traps isn't a counterexample; trapping is the rule doing its job.

The arrows (Section 18). Three stored arrows with matching ends in a well-formed graph whose two bracketings, built by COMPOSE, differ in source, target, flag, or (flag off) a pair; `graph_compose_assoc_stored` falls. An identity arrow from an object to itself and an arrow out of that object whose composite isn't the same arrow, in that sense; `graph_compose_left_identity_stored` falls, and the mirror for `graph_compose_right_identity_stored`. Or a run from the initial state that stores a flagged arrow between two different modules, or one with pairs; `vm_reachable_identity_arrows_canonical` falls. Two composites with different labels, or different identifiers, aren't counterexamples; the sameness ignores both on purpose, and Section 18 says why.

Separation (Section 20). Show that the two witness states differ in one of the six shadow fields, or that their arrow lists are the same. `categorical_separation` falls. Error behaviour is not the target; the arrow list is.

Strictness (Section 20, Section 23). Exhibit a classical program (any list of the twenty-nine classical opcodes, run as a list or by program counter with its jumps, for any fuel) whose execution changes anything in the graph (the fresh-module counter, the arrow list, any of it), the assertion channel or the certified flag. Or find a state some classical run reaches from the initial state where PNEW on cell zero traps or leaves the fresh-module counter fixed, or where the graph it produces is one a classical run from the initial state also produces. `thiele_strictly_extends_classical` falls, and with it `pnew_step_separates_thiele_from_classical_reachable`. Both start from the initial state, so there's no hand-built base to argue with.

The floor on fifty-one (Section 23). Find a run starting with the flag down and the counter at zero, ending with the flag up and the counter at zero. `certification_requires_positive_mu` breaks at the step that prices the flip. Add the counterexample as a theorem and the build refuses to compile on top of it.

The finite fragment (Section 23). Exhibit a state and one of CERTIFY at zero, JUMP at two, CHECKPOINT at zero whose result, read as program counter modulo four and the flag, differs from the finite machine's step, or whose counter rise differs from one, two, zero. `vm_runs_finite_machine` or `vm_pays_finite_price` falls. A program using JUMP at cost zero is outside the fragment; the full machine leaves that merge free on purpose.

The classical CHSH bound (Section 25). Exhibit four fixed bits and a tally consistent with that one plan, every pair sampled, scoring above two. `chsh_violation_rules_out_locally_factorizable_coupling` falls, and so does known mathematics. A

randomised plan landing past two by luck doesn't count; the theorem is about one fixed plan, and Section 25's coin-flip example shows the luck.

The integer gate (Section 25). The game check has two walls, and here's how to knock each one over. A correlator that passes the three contraction inequalities whose five-by-five is not PSD (forward), or a PSD five-by-five of that shape whose correlators aren't contractive (reverse); `npa_psd_iff_column_contractive` falls. A correlator whose pinned five-by-five is PSD (contractive, by the equivalence) with S squared above eight; `npa_psd_implies_tsirelson_bound` falls. The PR box and the classical mixture $(1, 0, 1, 0)$ are *not* counterexamples to any of these. Both fail contraction by plain arithmetic (the table in Appendix E.4), which is what the theorems need them to do, not what would break them.

The four 1+AB instructions (Section 25). Four bigger checks, four targets, one per instruction. Each of the four instructions that checks the nine-by-nine matrix has its own theorem: from a state with no error, if the instruction advances the program counter by one and leaves the error flag alone, the nine-by-nine matrix at the state's correlators and the instruction's moments (all zero for the plain one; for the other three, the free ones read from the supplied counts and the rest zero) is positive semidefinite. Exhibit such a run where the matrix isn't, and the matching theorem falls: `chsh_lassert_lab_no_trap_implies_npa_psd_q1lab`, `chsh_lassert_lab_g5_no_trap_implies_npa_psd_q1lab`, `chsh_lassert_lab_g345_no_trap_implies_npa_psd_q1lab`, or `chsh_lassert_lab_g12345_no_trap_implies_npa_psd_q1lab`. A trap on a point inside the set isn't a counterexample. The checks are sound, not complete, and the full one traps wherever its matrix is semidefinite without being definite, because each of its tests is strict (Appendix E.6).

The elliptope completion (Section 25). This one's a shape, not an instruction, so you're trying to push a point in or out of it. Three targets, one per theorem, plus the decider beside it. Signs a_0, a_1, b_0, b_1 , each squaring to one, and a vector on which the completed matrix of that plan goes negative; `deterministic_strategy_elliptope` falls. A correlator with some positive semidefinite completion and S squared above eight; `elliptope_tsirelson` falls, and with it the claim that one completion can't stretch both Cauchy-Schwarz budgets at once. Real numbers x and y that make the PR box's completed matrix positive semidefinite; `pr_box_not_elliptope` falls. And for the decider that sits beside it: inputs on which it says yes while the correlators have no PSD completion at all (S squared above eight is one way to show that); `elliptope_check_full_sound` falls. The all-ones plan and $(1, 0, 1, 0)$ aren't counterexamples here either, from the other side: the completion admits them (`turing_point_elliptope` for the second), which is what the theorems require.

The hardware (Section 27). A counterexample to the whole-machine refinement (`driven_step_wf`) has to satisfy its precondition and break its stated equality. A co-simulation mismatch on a valid input challenges the *tested* transfer for that revision;

then pull it apart link by link (encoding, observation timing, extraction, printing, bsc, the harness). An input outside the theorem’s domain, or a toolchain mismatch, doesn’t refute the conditional theorem. It refutes a trusted link, which is worse for me and more useful for you.

The five models (Section 12; the wrappers stated on this machine, Section 23). Each fragment has a theorem with a named falsifier: a finality fork with nobody slashable; a gas schedule meeting floor and no-overcharge whose charging predicate differs from the commit predicate on some step; a sound and complete attestation verifier that reads only the bare transcript, or a function of the quote that recovers the unmeasured runtime flag; one of the RFC’s own example proofs that the transcribed inclusion check rejects, or an input with leaf index equal to tree size that it accepts; a program the bounds checker accepts whose verification condition fails. Each kills one fragment. None of them, and I want this clear, says anything about the real system beyond the fragment.

The audit (Section 28). Run the assumption probe on any kernel-tier theorem and find a project-local axiom or parameter in its dependencies. The audit has degraded. Run the Inquisitor and read what it says; it fails on high and medium findings. A new finding means somebody looks. It doesn’t automatically mean a theorem is false.

29.3 What’s mine and what isn’t

This is not a claim to have been first. Appendix C does the sorting item by item, with sources: what was known, what’s added, and what kind of addition it is. Here’s the short form, and I’d welcome correction on any boundary.

Borrowed, with names. Everything in Part I (every source is cited where it’s used). The category theory (Eilenberg, Mac Lane). Turing, Church, Kleene, Rice, Cantor, Gödel, the diagonal pattern, Lawvere’s unification, and the mechanized versions of them in HOL4, Lean, and Coq (Norrish, Carneiro, Forster, Kunze, Smolka). Bell, CHSH, Tsirelson, Wehner, NPA, Landau, Masanes, Fine, the elliptope, and the machine-checked proofs of Tsirelson’s bound that came before mine (Echenim and Mhalla in Isabelle, and Lean’s mathlib). Landauer, Bennett, Szilard, Maxwell, and their critics. The potential method (Tarjan) and its mechanizations (Nipkow, Charguéraud and Pottier), Lamport’s clocks, Floyd and Hoare. Gettier, Necula, Cook, Hartley, Shannon, Rissanen. Zurek. The seL4 and CompCert discipline. Kami, Bluespec, yosys, nextpnr, X-Ray. The facts under the permanence results are Landauer’s own example and Shannon’s bounds, and “set a bit to one whatever it held” is my way of saying his RESTORE TO ONE.

Formal reconstructions. The Coq development defines the state types, transition systems, real-matrix predicates, encodings, and proof terms this book uses. When I say a theorem is checked, I mean the stated proposition has a kernel-checked proof under its printed premises. That doesn’t make the underlying idea mine, and it doesn’t make it new.

What this project proved, inside its stated interfaces. The abstract floor, which is the potential method with one particular potential. The derivation of the toll from finiteness, permanence, and a named price, in counted states and in bits, with each premise's counterexample built. The fixed-schedule ledger identity and uniqueness. The named projection witnesses. The conditional representation theorem for entitlement. The latch form over any base, with its two counterexamples and converse, and the two uniqueness results (held up to schedule, refuted for tied records). The substrate diagonal, the L results, and the guest recursion theorem by construction. The integer gate implying the pinned matrix is PSD (a sufficient test, not a necessary one), the equivalence with contraction, and Tsirelson's bound from PSD, which is known mathematics checked again here and tied to an instruction. The per-opcode hardware refinement and the loader theorems. All of it with premises printed. Of that list, the toll derivation, the pricing form of the window fact, and the latch form are the only pieces I'd call new statements, and only in the sense that I didn't find them stated where I looked.

Interpretation, which is mine and not proved. That structure is what a computation establishes and the toll is the right thing to price. That certification is the pointer event. That the record axis "belongs in the step." Those are proposals, with the arguments I've given, and the proposals are where I'd aim first if I were you.

30 The claim ledger

Here's everything this book claims, one line each: what kind of claim it is, and where it lives. "Proved" means proved under the premises printed in that section, and nothing more. If a line here ever says more than its section does, the section is right and the line is wrong.

30.1 Claims about the model

First the model. Every line in this table is about any system that has the pieces the line names: a state and a step, plus whatever the result asks for, a reading and the toll, a counter and a price list, a window, programs. None of them is about my machine. The table runs over several pages, in reading order.

Claim	Kind	Home
The early engine derived physics	not claimed; it ran a simulation	Section 1
Structure: relations, kept under chosen changes, carried by any stuff, relative to a frame	choice (my overlay of the sources, not something they proved together)	Section 2
Turing's four parts have no slot for "these belong together"	not claimed; I looked and didn't find one, which is weaker	Section 2

Claim	Kind	Home
Equivalence relation, partition	definition	Section 2, Definition 2.1
Sameness and sorting are one object	argued, not checked in Coq (textbook, Halmos)	Section 2, Theorem 2.1
A partition is the start, smallest by the count of names	choice	Section 2
Nobody prices vouching for a sorting	not claimed; only that I found no one who does	Section 2
Claim	Kind	Home
Certification system: states, moves, step, cost, reading	definition	Section 3, Definition 3.1
The toll	choice	Section 3, Definition 3.1
The universal floor: no to yes costs at least one	proved (premise: the toll)	Section 3, Theorem 3.1
The floor speaks about the world, or computing power	not claimed	Section 3
A computation is in scope just by being one	not claimed; someone has to supply the five parts and prove the toll	Section 3
On a finite machine, a permanent flip merges	definition; proved (no price premise)	Section 4, Definition 4.1, Theorem 4.1
Merge pricing	assumed (premise)	Section 4, Premise 4.2
Finite, permanent and merge pricing give the toll	proved (premise: merge pricing)	Section 4, Theorem 4.2
Each premise has a counterexample	proved (the strip, the coin, the stamp); “free” is read off the pictures	Section 4
A flip merges or revokes	proved	Section 4, Theorem 4.3
Forced to be priced exactly when it merges	definition; proved (premise: move names decidable)	Section 4, Definition 4.3, Theorem 4.4
Permanent records are forced; the converse fails	proved (the three cards)	Section 4, Corollary 4.5
Halving price	assumed (premise)	Section 4, Premise 4.4
The squeeze bound	proved (premise: halving price)	Section 4, Theorem 4.6

Claim	Kind	Home
Forgetting nothing removes no bits	proved	Section 4, Theorem 4.7
A permanent flip removes at least $\log_2 \frac{m+k}{m}$ bits	proved	Section 4, Theorem 4.8
Landauer's principle	assumed (premise)	Section 4, Premise 4.5
Heat floor of a permanent flip	proved (premises: Landauer, $k_B T \geq 0$)	Section 4, Corollary 4.9
Positive heat for any full spread	proved (premises: Landauer, $k_B T > 0$)	Section 4, Corollary 4.10
Entropy pricing gives the toll	definition; proved (premise: entropy pricing)	Section 4, Definition 4.6, Theorem 4.11
A known state owes nothing	proved	Section 4, Theorem 4.12
The bits removed, one term per state	proved	Section 4, Theorem 4.13
Landauer's principle is true of the world	not claimed; a named premise, with its critics named	Section 4
Any machine worth building is of this kind	not claimed	Section 4
Permanence is the job to pick	choice	Section 4
The machine's own narrowing is priced	proved (premise: halving price)	Section 5, Theorem 5.1
An observer can learn for free	definition; proved (one schedule that allows it)	Section 5, Definition 5.1, Theorem 5.2
Learning is free on every schedule	not claimed	Section 5
Claim	Kind	Home
Three states are the minimum for free teaching	proved	Section 5, Theorem 5.3
"No Free Insight" names the committed kind	choice (the name kept, the scope stated)	Section 5
Watching narrows like a quantum measurement	not claimed	Section 5
A counter kept by a price list	definition	Section 6, Definition 6.1
Conservation	argued, not checked in Coq	Section 6, Theorem 6.1
Only one honest count	argued, not checked in Coq	Section 6, Theorem 6.2

Claim	Kind	Home
The fold on lists is unique; it reaches states only under a condition	argued, not checked in Coq	Section 6
Initiality picks the prices, or makes marks physical	not claimed	Section 6
The toll is the potential method	definition; proved	Section 6, Definition 6.2, Theorem 6.3
The potential method is new here	not claimed (it's Tarjan's)	Section 6
The counter embeds the cost-semantics frameworks	not claimed; left open	Section 6
A window that forgets the counter can't recover it	argued, not checked in Coq	Section 6
A decoder needs constant fibres	proved (any types)	Section 7, Theorem 7.1
No exact window price across a collision	proved (premise: a collision)	Section 7, Theorem 7.2

Claim	Kind	Home
On the toy, no verifier on bare transcripts is sound and complete	argued, not checked in Coq	Section 7
The one-round "ask" models a cheating prover	not claimed	Section 7
The window fact is mysticism, or Rice	not claimed	Section 7
Entitlement: object, checked relation, paid event	definition (the book's word, not in Coq)	Section 8, Definition 8.1
A decision tree has at most two to its depth leaves	definition; proved	Section 8, Definition 8.2, Lemma 8.1
Paid narrowing, counted	argued, not checked in Coq (premises: conservation, a covering tree, paid steps)	Section 8, Theorem 8.2
Entitlement is truth, or permission	not claimed	Section 8
The count prices why candidates were dropped	not claimed; it prices the size of the drop	Section 8

Claim	Kind	Home
Certified insight event, raw observation	definition	Section 9, Definition 9.1
The toll prices only the crossing, flat in depth	fact read off the definitions	Section 9, Remark 9.1
Sorting moves by whether the reading moves	choice	Section 9
Nothing makes the crossing wait for a check	fact read off the definitions	Section 9, Remark 9.2
A paid flag proves its claim true	not claimed	Section 9

Claim	Kind	Home
Base, cover, honest extension, latch	definition	Section 10, Definition 10.1, Definition 10.2
Over any base, an honest record is a latch	proved	Section 10, Theorem 10.1
Every reachable latch is an honest record	proved	Section 10, Theorem 10.2
A finite reversible base can't write a record	proved; an endless history can	Section 10, Theorem 10.3
Two records are two latches	proved	Section 10, Lemma 10.4
The toggle: driven, not permanent, no latch	proved	Section 10, Lemma 10.5
The clock: permanent, not driven	proved	Section 10, Lemma 10.6
Honest records agree up to the schedule	argued, not checked in Coq	Section 10
The event is a free choice	argued, not checked in Coq (the converse applied twice)	Section 10, Corollary 10.7
The schedule doesn't fix the probabilities	proved, with the branching latch refuted	Section 10, Theorem 10.8
One state space up to renaming	not claimed	Section 10
The latch on a Turing machine, a register machine, L	definition; proved (and the relation is an equivalence)	Section 10, Definition 10.4, Theorem 10.9

Claim	Kind	Home
Weak base equivalence connects those bases	not claimed	Section 10
A growing record is a family of latches	definition; proved, and one latch isn't enough	Section 10, Definition 10.5, Theorem 10.10
Claim	Kind	Home
Substrate; shortcut predicate	definition	Section 11, Definition 11.1, Definition 11.2
No inside decider with a representable flip	proved (premise: the substrate's recursion theorem)	Section 11, Theorem 11.1
The nat family: every decider misses program two	proved	Section 11, Theorem 11.2
An outside decider gets the nat family right	argued, not checked in Coq (the pencil argument)	Section 11
Recursion, Rice and halting for L	definition; proved	Section 11, Definition 11.3, Theorem 11.3
Undecidable means unknowable; the wall separates Turing degrees	not claimed	Section 11
Casper setting; one storage slot; a gas schedule	definition (transcribed; the gas schedule's charge premise stipulated)	Section 12, Definition 12.1, Definition 12.2, Definition 12.3
Accountable safety	proved (premises: quorum intersection, one parent)	Section 12, Theorem 12.1
The quorum premise can hold; one vote finalizes free	proved	Section 12, Theorem 12.2
A persistent write nets at least 20000	proved, on the transcription	Section 12, Theorem 12.3
Gas is exact exactly when commits pay one	proved (premise: the stipulated charge predicate)	Section 12, Theorem 12.4
A permanent write on a finite machine merges	proved	Section 12, Theorem 12.5
Platform and quote; inclusion check; bounds fragment	definition	Section 12, Definition 12.4, Definition 12.5, Definition 12.6

Claim	Kind	Home
The quote can't attest what it never measured	proved	Section 12, Theorem 12.6
The RFC's inclusion examples pass	proved (symbolic hash)	Section 12, Theorem 12.7
The proof-carrying checker is the verification condition	proved	Section 12, Theorem 12.8
A transcription is the real system	not claimed	Section 12
A toll is a better design than a wall	not claimed	Section 12
Ecosystem, proliferation, pointer; observation game	definition	Section 13, Definition 13.1, Definition 13.2
Certification is the event whose records proliferate	conjecture	Section 13, Conjecture 13.1
In the toy, "certified" is the unique pointer	proved	Section 13, Theorem 13.2
The survey; the swap shows the verdict follows the observer maps	proved	Section 13, Theorem 13.3
One blind observer stops proliferation	proved	Section 13, Theorem 13.4
Agreement doesn't buy permanence; durable views do	proved	Section 13, Theorem 13.5
The formal objects prove the conjecture	not claimed; "independent" and "deployed" are defined nowhere	Section 13
The floor raised to a threshold	proved (premises: the witness grows by at most the cost and meets the threshold)	Appendix B, Theorem B.1

30.2 Claims about the machine

Claim	Kind	Home
Checker and arithmetic meaning agree	proved on the minimal machine, under the stated premises	Section 14, 14.1

Claim	Kind	Home
Two-counter simulation and halting correspondence	proved on the minimal machine, under the stated premises	Section 14, 14.2
Halting is undecidable by the vendored two-counter reduction	proved on the minimal machine, under the stated premises	Section 14, 14.4
Exact ledger, permanent certification and toll	proved on the minimal machine, under the stated premises	Section 14, 14.5
Instance of the abstract certification system	proved on the minimal machine, under the stated premises	Section 14, 14.6
Certification factors as a latch over the core base	proved on the minimal machine, under the stated premises	Section 14, 14.7
A passing commitment has a prior matching check at the same version	proved on the minimal machine, under the stated premises	Section 14, 14.8
Certification has a check, commitment and certification in that order	proved on the minimal machine, under the stated premises	Section 14, 14.9
Unearned commitment and uncommitted certification trap	proved on the minimal machine, under the stated premises	Section 14, 14.10
Current-version checked claims are true	proved on the minimal machine, under the stated premises	Section 14, 14.11
No stored fact is forged on a clean-start trace	proved on the minimal machine, under the stated premises	Section 14, 14.12
Bounded table keeps facts and traps when full	proved on the minimal machine, under the stated premises	Section 14, 14.13
Equal computational windows can hide different records	proved on the minimal machine, under the stated premises	Section 14, 14.14
The window cannot recover the ledger, flag or commitment permission	proved on the minimal machine, under the stated premises	Section 14, 14.15
Clean-start certification costs at least three	proved on the minimal machine, under the stated premises	Section 14, 14.16

Claim	Kind	Home
A three-charge certification run attains the bound	proved on the minimal machine, under the stated premises	Section 14 , 14.17

Now the machine, the one I built. A lot of these lines are a claim from Parts II and III landing on something real, and where one is, the Kind column says which claim it instantiates. “Proved on the machine” means under the premises printed in the home section, and those are often more than you’d guess from the line. This table runs over several pages too, in reading order.

Claim	Kind	Home
Sixteen registers, 128 words, fifty-one instructions	choice (each number)	Section 15
The state record; registers, memory, the word	definition	Section 16 , Definition 16.1 , Definition 16.2
Conservation, failing steps included	proved on the machine; instance of Section 6 ’s conservation	Section 16 , Theorem 16.1
The error flag latches	argued, not checked in Coq	Section 16 , Remark 16.1
Sixteen registers and 128 words are kept	proved on the machine	Section 16 , Theorem 16.2
Regions stay disjoint, contiguous, inside memory; at most sixty-four modules	definition; proved on the machine (reachable states)	Section 16 , Definition 16.3 , Definition 16.4 , Theorem 16.3 , Definition 16.6 , Theorem 16.5
A well-formed graph stays well formed	definition; proved on the machine	Section 16 , Definition 16.5 , Theorem 16.4
What every run from the start keeps	definition; proved on the machine	Section 16 , Definition 16.7 , Corollary 16.6
The sizes are literals the proofs compute with	fact read off the definitions	Section 16 , Remark 16.2
No program, output or separate heap in the state	fact read off the definitions	Section 16
The step relation computes the step function	definition; proved on the machine	Section 17 , Definition 17.1 , Theorem 17.1
PNEW, PSPLIT, PMERGE	definition	Section 17 , Definition 17.2
LASSERT pays for the length it read	proved on the machine	Section 17 , Theorem 17.2

Claim	Kind	Home
A passing G345 step gives a PSD matrix	proved on the machine	Section 17, Remark 17.1
Classical runs move nothing that certifies The schedule; the positive and setter classes	definition; proved on the machine definition	Section 17, Definition 17.3, Theorem 17.3 Section 17, Definition 17.4, Definition 17.5
Twelve instructions have a floor	proved on the machine	Section 17, Theorem 17.4
The joint floor is the least	proved on the machine	Section 17, Theorem 17.5
Only CERTIFY touches the flag; it stays set	proved on the machine	Section 17, Theorem 17.6
Only MORPH_ASSERT touches the channel	proved on the machine	Section 17, Theorem 17.7
No free certification	proved on the machine; instance of Section 3's floor	Section 17, Theorem 17.8
The chip encodes forty-seven of the fifty-one	fact read off the definitions	Section 17
Category; coupling; stored arrow and composition	definition	Section 18, Definition 18.1, Definition 18.2, Definition 18.3
Relational composition is a category on couplings	proved	Section 18, Theorem 18.1
Tensor on couplings	proved	Section 18, Theorem 18.2
Stored arrows: associativity, both identity laws; identities stay canonical	definition; proved on the machine	Section 18, Definition 18.4, Theorem 18.3, Theorem 18.4
The stored records form a category in the full sense	not claimed	Section 18
Loading a coupling from memory	definition	Section 18, Definition 18.5

Claim	Kind	Home
No step reads an axiom label	fact read off the definitions	Section 19, Remark 19.1
What the formula check step does	definition; proved on the machine	Section 19, Definition 19.1, Theorem 19.1
A tautology can't pass	argued, not checked in Coq	Section 19, Remark 19.2
The chip's scan is the kernel's check	proved on the machine	Section 19, Theorem 19.2
Structural gain has a place and a price	definition; proved on the machine	Section 19, Definition 19.2, Theorem 19.3
A run can gain and end with the channel clear	argued, not checked in Coq	Section 19, Remark 19.3
A checked formula means what its label says	not claimed	Section 19
Equivalent states, different arrows	definition; proved on the machine (at the type level)	Section 20, Definition 20.1, Definition 20.2, Theorem 20.1
Equal shadows, kept apart by one step	definition; proved on the machine	Section 20, Definition 20.3, Theorem 20.2
The shadow is a lossy quotient	proved on the machine	Section 20, Theorem 20.3
Neither state of the pair is reachable	proved on the machine	Section 20, Remark 20.1
One PNEW does what no classical run can, from every classically reachable state	definition; proved on the machine (from the initial state; list and program-counter runs)	Section 20, Definition 20.4, Theorem 20.4
Counter, flag and graph are independent	proved on the machine	Section 20, Theorem 20.5
Three fields no bare window recovers	proved on the machine	Section 20, Theorem 20.6

Claim	Kind	Home
No ordinary machine holds the graph; a class separation	not claimed	Section 20
A counter beside a Turing machine changes nothing	proved	Section 20, Theorem 20.7
The machine represents a system that simulates it	definition; proved (premises: the three simulation clauses); instance of Section 3's floor	Section 20, Definition 20.5, Theorem 20.8
A Turing machine can't price a reading	not claimed	Section 20

Claim	Kind	Home
A failed assertion still pays	proved on the machine	Section 21, Theorem 21.1
Building leaves the channel alone	proved on the machine	Section 21, Theorem 21.2
A successful assertion; every raise pays	proved on the machine	Section 21, Theorem 21.3
The probe fails on an empty arrow list	proved on the machine	Section 21, Theorem 21.4
One step no classical run can match	proved on the machine	Section 21, Theorem 21.5
The receipt reads thirteen; Act 4's shadows match	tested, not proved (a script asserts them)	Section 21
The two readings; the label checksum	definition	Section 22, Definition 22.1
Certified insight event; the three tiers	definition	Section 22, Definition 22.2
A game trial is raw observation	proved on the machine; instance of Section 9's raw observation	Section 22, Lemma 22.1
Certified insight isn't free	proved on the machine; instance of Section 9's floor	Section 22, Theorem 22.2

Claim	Kind	Home
A game witness pays a flat price	proved on the machine; instance of Section 9's flat price	Section 22, Theorem 22.3
The check commits nothing	proved on the machine	Section 22, Lemma 22.4
The commit reads no test	fact read off the definitions; instance of Section 9's crossing that doesn't wait	Section 22, Remark 22.1
Narrowing becomes strengthening	definition; proved on the machine	Section 22, Definition 22.3, Theorem 22.5
A certified end from a clear start had an assertion	definition; proved on the machine	Section 22, Definition 22.4, Theorem 22.6
Paid steps cover the counter	definition; proved on the machine	Section 22, Definition 22.5, Lemma 22.7
Index bits are paid for	proved on the machine (premises: a realized tree, a reduction)	Section 22, Lemma 22.8
Structural entitlement, represented	proved on the machine (premises H1 to H8); instance of Section 8's count	Section 22, Theorem 22.9

Claim	Kind	Home
The two observations can't be one	proved on the machine	Section 22, Remark 22.2
A run that realizes the complete tree pays the index bits	proved on the machine (premise: the run realizes the complete tree of the right depth)	Section 22, Remark 22.3
Every sound shortcut lands in the bound	definition; proved on the machine, by packaging	Section 22, Definition 22.6, Corollary 22.10
The worked member: $1 \leq 19$	proved on the machine, by computation	Section 22, Remark 22.4
The speedup theorems: iteration counts, counters, and the price comparison	proved on the machine (runs from the initial state, every N with $N^2 < 2^{64}$; the comparison is an if and only if)	Section 22, Remark 22.5
A new speedup, or anything about P versus NP	not claimed	Section 22

Claim	Kind	Home
A bound on one run's narrowing, no tree given	not claimed	Section 22
Classical runs keep graph, channel and flag	definition; proved on the machine (list runs and program-counter runs with jumps)	Section 23, Definition 23.1, Theorem 23.1
The counter sums, climbs, is unique	proved on the machine; instance of Section 6's conservation and initiality	Section 23, Theorem 23.2
Folding a run is the only way	proved on the machine; instance of Section 6's fold	Section 23, Theorem 23.3
When the fold comes down to states	proved on the machine (the picking function is built by search; assumption: the choice principle of Coq's real numbers)	Section 23, Theorem 23.4
Counter and flag can't be dropped	proved on the machine; instance of Section 6's window argument	Section 23, Theorem 23.5
Three fields no window recovers	definition; proved on the machine; instance of Section 7's fibre fact	Section 23, Definition 23.2, Theorem 23.6
The floor on all fifty-one	proved on the machine; instance of Section 3's floor	Section 23, Theorem 23.7
The machine runs the finite fragment	proved on the machine; instance of Section 4's room	Section 23, Theorem 23.8

Claim	Kind	Home
CERTIFY's merge is priced; JUMP's is free, so merge pricing fails	proved on the machine; instance of Section 4's derivation	Section 23, Theorem 23.9
An observer learns a bit for free	proved on the machine; instance of Section 5's free learning	Section 23
No exact window price	definition; proved on the machine; instance of Section 7's collision theorem	Section 23, Definition 23.3, Theorem 23.10
No sound, complete bare verifier	proved on the machine; instance of Section 7's verifier argument	Section 23, Theorem 23.11
A richer verifier on the bare part does no better	proved on the machine	Section 23, Theorem 23.12
Full-state, commitment and response verifiers are sound and complete for their explicit explanation relations; commitment requires binding and honesty	proved under the printed relations and contract; verification cost is stipulated	Section 23, Theorem 23.13

Claim	Kind	Home
Record carrier; adequate; honest	definition	Section 23, Definition 23.5, Definition 23.6, Definition 23.7
A record over this machine is a latch	proved on the machine; instance of Section 10's latch	Section 23
Exact uniqueness is false	proved on the machine (the billed core); instance of Section 10's free price list	Section 23, Theorem 23.14
History is forgotten by the relation, and only by it	proved on the machine	Section 23, Theorem 23.15
Unique up to the schedule	definition; proved on the machine; instance of Section 10's argument	Section 23, Definition 23.8, Theorem 23.16
Not unique for a tied record	proved on the machine; instance of Section 10's free event	Section 23, Theorem 23.17
One per raise is exact; the counter charges at least that	proved on the machine; instance of Section 12's gas exactness	Section 23

Claim	Kind	Home
Attestation can't use the bare transcript	proved on the machine; instance of Section 12's attestation	Section 23, Theorem 23.18
The flag is a permanent consensus	proved on the machine; instance of Section 13's durable views	Section 23, Theorem 23.19
The certification channel stays set	not claimed; an empty label writes zero	Section 23
Every merge priced; the lifted model is the machine	not claimed	Section 23
The machine's shortcut property	definition	Section 24, Definition 24.1
A recursion theorem for the full machine, for every map	false, proved: the cutoff property is decidable, and no correct decider's flip has a fixed point	Section 24, Premise 24.2, Theorem 24.3
No inside decider	proved on the machine (premise: the full machine's recursion); instance of Section 11's diagonal	Section 24, Theorem 24.1

Claim	Kind	Home
Bounded runs are decidable from outside	proved on the machine	Section 24, Theorem 24.2
A program halts at most once	definition; proved on the machine	Section 24, Definition 24.3, Theorem 24.4
The accumulator commutes with every run	proved on the machine	Section 24, Lemma 24.5
No interpreter reads input from the accumulator	proved on the machine	Section 24, Theorem 24.6
The self-interpreter is correct	definition; proved on the machine (the unbounded step)	Section 24, Definition 24.4, Theorem 24.7
Rice on the guest	definition; proved on the machine	Section 24, Definition 24.5, Theorem 24.8
The pairing prefix pairs	definition; proved on the machine	Section 24, Definition 24.6, Lemma 24.9
Guest recursion theorem, by construction	proved on the machine (closed); instance of Section 11's recursion field	Section 24, Theorem 24.10

Claim	Kind	Home
The guest discharges the full machine's premise	not claimed; no theorem ties the 64-bit step to the unbounded one	Section 24
The classical ceiling, $S = 2$, at zero cost	proved on the machine	Section 25, Theorem 25.1
A passed gate makes the pinned matrix PSD	definition; proved on the machine	Section 25, Definition 25.1, Theorem 25.2
PSD forces $S^2 \leq 8$	proved (algebra only)	Section 25, Theorem 25.3
A passed gate means $S^2 \leq 8$	proved on the machine	Section 25, Corollary 25.4
A PSD pinned matrix has a PSD completion	proved	Section 25, Remark 25.1

Claim	Kind	Home
Every deterministic plan fails the gate	argued, not checked in Coq	Section 25, Remark 25.1
The level-1 test is the size-five case	definition; proved	Section 25, Definition 25.2, Corollary 25.5
A tally above two isn't one plan replayed	proved	Appendix E, Theorem E.3
A tally above two proves nonlocality; a pass proves the data quantum	not claimed	Section 25
A finite local model meets the bound	proved; the converse, and so Fine's full theorem, isn't claimed	Appendix E, Theorem E.2
The four 1+AB checks are each sound	proved on the machine	Appendix E, Theorem E.23
Their accepted sets are nested	not claimed	Section 25
The elliptope gate is sound; the PR box fails it	proved	Appendix E, Theorem E.30
The elliptope gate is an instruction	not claimed	Section 25
The arc-sine boundary; the quantum set; reaching $2\sqrt{2}$	not claimed (cited, not proved)	Appendix E
The integer route to the bound is new mathematics	not claimed; the route is NPA's	Section 25
The indicator is bounded by the cost	definition; proved on the machine	Section 26, Definition 26.1, Theorem 26.1

Claim	Kind	Home
The calibration is a unit conversion	definition; proved	Section 26, Definition 26.2, Theorem 26.2
Landauer in counter units; dissipation at most cost	assumed (premise)	Section 26, Premise 26.3
Claim	Kind	Home
The premises fit a one-bit reset, not the full machine	proved on the machine; instance of Section 4's Landauer premise	Section 26, Theorem 26.3
Keeping history makes the free merge injective	proved on the machine	Section 26, Theorem 26.4
The heat is fixed by the chosen gap	definition; proved	Section 26, Definition 26.4, Theorem 26.5
The protocol's one event against the floor	proved on the machine	Section 26, Theorem 26.6
The Unruh-Clausius calibration	assumed (premise)	Section 26, Premise 26.5
It comes from a unit choice plus a premise	proved	Section 26, Remark 26.1
The curvature identity depends on the counter or the calibration	not claimed; the Gauss-Bonnet delta has only the two triangulation premises	Section 26, Remark 26.1
Reachable states are flat: separate regions, no neighbours, triangulations of separate triangles	proved on the machine	Section 26, Theorem 26.7
The machine reaches a closed curved surface	not claimed; every reachable triangulation is separate triangles	Section 26, Theorem 26.7
The mu-hierarchy	definition; proved on the machine	Section 26, Definition 26.6, Theorem 26.8
The counter is physical dissipation, in joules	not claimed	Section 26
Permanence predicts more than Landauer	not claimed	Section 26
ADD refines	definition; proved on the machine	Section 27, Definition 27.1, Theorem 27.1
The game check refines	proved on the machine	Section 27, Theorem 27.2

Claim	Kind	Home
The satisfiable LASSERT refines	proved on the machine	Section 27, Theorem 27.3
The hardware scan computes the kernel's checks	proved on the machine	Section 27, Lemma 27.4
Claim	Kind	Home
The checking step against the kernel	proved on the machine	Section 27, Theorem 27.5
Every admitted instruction retires	definition; proved on the machine	Section 27, Definition 27.2, Theorem 27.6
The specification step is the kernel step	proved on the machine (premise: well formed at every step)	Section 27, Theorem 27.7
Start from reset; serial load; hand-off	proved on the machine	Section 27, Theorem 27.8, Theorem 27.9, Theorem 27.10
The status report on the serial pin carries the kernel state's fields	proved on the machine (premise: one firing of the report rule per cycle)	Section 27, Theorem 27.11
Everything below Kami works	assumed (trusted; some links tested, as the table there says)	Section 27
A board ran it	not claimed; no run is recorded	Section 27
Zero Admitted; five standard assumptions	fact from the assumption audit, true of the run that printed it	Section 28
The theorems prove physical reality, or Coq's consistency	not claimed	Section 28
Claim	Kind	Home
Checked bit answers filter the candidate list and instantiate entitlement	proved for supplied honest answers, with one to n questions and n at most 64	Section 22, Theorem 22.11

30.3 What I propose and haven't proved

And then there's what I'm proposing. These are mine. They reach further than any theorem in here, and nothing in the tables puts an unavoidable physical price on every observable change.

- That the axis belongs in the step (Section 31, Section 32).
- That the usual account of a computation is a shadow of the step (Section 32).

- That certification is the event worth recording, which is the pointer conjecture (Section 13).
- That this is a structural account of computation and not just a cost model (Section 31).
- That the toll is the right thing to price, against a wall or a penalty for contradiction (Section 12, Section 32).

And three things left open: whether Landauer’s critics are right, which would leave every theorem standing and one premise unowned by the world (Section 4); whether certification really is the pointer event (Section 13); and whether there’s a smaller start than a partition, by a ruler someone can state (Section 2).

That’s the whole ledger. If you find a claim in this book that isn’t on it, or a line that says more than its home does, that’s my mistake, and I’d want to hear about it.

31 Scope

This is the line around what I’ve proved. Inside it: an accounting law for one event, a derivation of that law from premises each with its own counterexample, a set of windows that provably lose it, a record axis that’s a latch over any base, a diagonal that holds by proof on a fragment of the machine, a six-instruction machine on which commitments are earned and which computes what two counters can, an integer gate that implies a bound by algebra, and a chain from the step relation to a wiring list with each link labelled. Outside it: everything that follows. This is the fence. I built it myself, so lean on it before you trust it.

Physical interpretation of the counter. The ledger is unique once the start and schedule are chosen. That doesn’t make it entropy, and it doesn’t make it energy. A physical bridge needs a model of which states carry information, premises that can all hold at once, and a real argument tying the formal charge to dissipation. The permanent-certificate theorems supply the first for an abstract finite machine with a permanent reading. The two premises are shown able to hold together only for a one-bit reset; for the machine’s certifying steps that isn’t stated as a theorem. The full machine’s jump merges program counters and is charged nothing, which breaks the bridge outright (Section 23, Section 26), and the calorimeter protocol shows that population dynamics don’t fix the energy scale. No joule value attaches to one mark without thermal and device premises (Section 4, Section 26).

Class separation. The witnesses show specific windows losing specific fields. They don’t stop a conventional simulator from keeping those fields in its encoding. No separation of standard complexity classes follows from the collisions, or from the charge levels climbing without bound (Section 7, Section 20, Section 23, Section 26).

Trace initiality, and what it doesn’t settle. The universal property of the trace fold is about evaluating lists of instructions: pick where a target starts, and there’s exactly one way

to fold a list into it (`thiele_trace_fold_initial`, Section 23). The counter’s uniqueness assumes a schedule. Exact pricing assumes an event, and a reference count that says what charging too much means (Section 7). Those results make the construction precise. None of them shows that the category I chose, or the event I chose, is physically necessary.

Other observables. Certification is one worked example. A family of other observations might turn up other structural laws, but each would need its own events, its own evidence, and its own cost rules, justified on their own terms. The window and oracle results are about what specific maps keep. They don’t show infinitely many independent dimensions, zero mutual information, or a cost for changing coordinates, and piling predicates on top of each other doesn’t give a topology, a metric, or independence (Section 20).

Program predicates versus state observations. The state-level collision (Section 7’s wall) and the program-level diagonal (Section 11’s wall) live in different domains. Linking them would take a theorem of its own. The nat family (Section 11) and the bounded-machine theorem (Section 24) keep their explicit quantifiers and representability premises.

Entitlement bounds. The representation theorem is handed its tree and its representative reduction, along with the covering and payment conditions. It doesn’t dig the tree out of an arbitrary run, and it isn’t a Shannon bound on every trace (Section 22).

Quantum behaviour. The fixed-matrix equivalence is about one completion with zero marginals and zero within-party moments. The ellipsope lets the within-party moments move, at the level of correlators. Full behaviours with marginals, and operator realisations, need identifications this book doesn’t make. A sound runtime check isn’t complete for those bigger sets, and the ellipsope decider is not an instruction (Section 25).

The record axis. The proved structural result is: a permanent record driven by a base computation is a latch on an event; on finite hardware its write merges states; and the machine’s named projections don’t recover it. Certification is one instance, not a theorem selecting the unique event every system must record (Section 10, Section 13). A finite monotone multi-valued record decomposes into threshold latches, not into one Boolean latch. Revocation, including revocation priced from the other side as in Casper, lies outside the permanent-record axis; a price on revocation doesn’t transfer to a price on the write (Section 12). Probabilistic records don’t inherit the uniqueness result either. Two honest random records can have the same branches and the same charges and still give those branches different weights, one choosing evenly and one two to one, so the schedule doesn’t fix the probabilities (`schedule_determines_probabilities_refuted`). And no single deterministic latch explains a record that can branch both ways from the same observation (`deterministic_latch_handles_branching_refuted`). Weak base equivalence is defined and is an equivalence;

no two of the four bases are related by it in the checked work, and the latch form holds on each separately, so it doesn't tell them apart (Section 10, Section 23).

The guest. The twelve-instruction guest has a direct Rice reduction, numeric decoding, verified dispatch, a guest-callable specialiser, and a recursion theorem proved by construction. For the full fifty-one-instruction machine the recursion premise is false when asked for every map, because equality after the thousand-step cutoff can be decided by running both programs; the machine's wall is the diagonal applied to that decidable property, and nothing more (`vm_full_recursion_premise_refuted`, `vm_correct_flip_has_no_fixed_point`, Section 24).

The five models. Each is a fragment transcribed from a specification, with a named theorem and a named limit: the Casper transcription has no deposits or fork choice; the gas fragment omits the refund cap; the attestation fragment has no signatures; the transparency verifier covers control flow, not collision resistance; the proof-carrying checker is a three-instruction role model. None is a claim about the real system beyond its fragment (Section 12).

The two machines. Part IV has a small machine and a big build, and they prove different things. The small machine of Section 14 (two counters, six instructions) is where commitments are earned: a raised flag went through a passing check, a commitment and a certification of the same claim, nothing is forged, and the cheapest certificate costs exactly three. It's also where universality is proved, by matching Minsky's two-counter machines step for step and borrowing their undecidable halting from the Coq library. Its check price is flat, its fact table holds sixteen, and its certificate is historical (Section 14 lists the rest). The big build of fifty-one instructions prices every commitment and doesn't enforce that one was earned: its check and its commit are separate steps, and nothing makes the second wait for the first (Section 22). So "commitments are earned" is a theorem about the small machine, not the big one.

The comparison machines. The small machines I built to compare against are fragments too. The random-access memory result covers an addressed list memory with load, increment and decrement on cells (`concrete_ram_write_reads_back` and its neighbours), and it's about that machine. The register machine Section 10 puts the latch on (`record_axis_is_latch_on_ram_holds`) is a different small machine, the one with indirect addressing, and that theorem is about it. The reversible-machine result covers arithmetic update cores: add and subtract, each undone by the other, with or without wrapping at a modulus (`janus_like_unbounded_inverse`, `janus_like_bounded_inverse`). Neither is a theorem about real memories or full reversible languages. The cost-semantics results are local correspondences, not full cost-framework embeddings: the trace's cost is a writer cost and a grade, and the toll is the potential method with one potential (Section 6). Nobody's whole type system gets embedded either.

Outward consequences. I looked for results about real systems that would need the

record axis. Five narrow ones are proved, each as a two-state countermodel: a certificate-transparency client's local tree head can't decide whether the world's log is consistent (`ct_local_view_insufficient`); a quote check that matches only the composite digest accepts a mismatched selection (`tpm_selection_binding_is_necessary`); a local suffix can't decide a trusted anchor (`weak_subjective_suffix_insufficient`); an acknowledgement without a durable commit record can be lost (`wal_ack_requires_durability`); and a current local log can't decide what happened before it (`audit_local_snapshot_insufficient`). Each reduces to a known indistinguishability or durability argument. Five stronger candidates would need protocol or hardware semantics I haven't formalised, and I've proved nothing about them. So none of this yields a new outside result that both needs the record axis and is ready to use.

Geometry. The curvature files prove things about partition graphs in general, and the chain from the counter to a discrete Gauss-Bonnet identity carries its physical premises without using them. On states the machine reaches from its initial state, modules never share an address, every module reads flat, and every triangulation is separate triangles with Euler characteristic equal to their count, so no run reaches a closed surface (Section 26, Theorem 26.7).

The hardware. Proof stops at Kami's semantics. Below that: extraction, printing, bsc, yosys, nextpnr, X-Ray, and the board primitives are trusted, with the tests named. Two programs are simulated. Generic gates, not the mapped netlist. The loader's start rule has no theorem; the status report has one, under the premise that the report rule fires once per clock cycle. No board was powered on (Section 27).

Where each result lives. Every system that follows the toll, whatever its states: the floor, at most as many certifications as cost, and the potential-method reading. Every finite machine whose price covers merges: the toll for permanent records, and which instructions every such price must charge. Every finite machine paying per halving: the machine's own narrowing bound. Every machine extending a base with a driven, permanent record: the latch. Every window through which a certifying and a non-certifying step look alike: no exact price from the window. This build only: the schedule and its numbers, the collisions of its own windows, the integer gate and its PSD bridge, the hardware correspondence, the fifty-one instructions, the finite fragment.

What I'm proposing beyond that. That the axis belongs in the step; that certification is the event worth recording; that this is a structural account of computation and not just a cost model. Those reach further than any theorem here, and no theorem here puts an unavoidable physical price on every observable change. They're the argument, and the argument is what the rest of the book was.

32 Conclusion

I started with one word and a machine I couldn't explain. Here's where the word ended up, and where the machine did.

Structure turned out to be relations among things, kept under chosen changes, carried by whatever stuff, always relative to a frame. The start I chose, the smallest one by the count of names, was a sorting of a pile, and the thing nobody I read had priced was the moment you stand behind the sorting. That was Part I, and it ended with a question.

The question got an exact form in Part II: five parts, one rule, one theorem you proved at the table. Then the rule stopped being only my preference. On a finite machine where the flag stays up, raising it forgets something, and if forgetting is charged for, the way Landauer's principle charges for it in its worst case, the toll follows. That's a derivation under three premises you can refuse one at a time, each with its counterexample built, not a proof that every computation has to adopt the rule; and the charge on forgetting is a named premise with its critics named. Learning can be free; committing to a record that stays put can't be, on those premises. That's as far as physics goes in this book, and I've tried to keep it exactly that far.

Part III was the logic, and here's what it earned, in the order it was earned. The counter is arithmetic once the schedule is chosen. A window in which a raising step and a quiet one look the same can't price the raise exactly, by a fact about functions far older than the machine. Entitlement is defined as an object, a checked relation, and a paid event; on the machine, under the hypotheses of the representation theorem, it costs at least the index bits it commits to; and it is a license to lean on a claim, not a proof that the claim is true. Insight, in the sense the toll prices, is a committed relation, not a feeling, and its floor is flat: at least one, however deep. The claim behind the flag is a formula on a group, checked by a small checker, and the check is a different step from the commit. Over any base, an honest record is a latch, with two free choices, the event and the price list; "only one substrate" is false and "one structure, many price lists" is true once the event is fixed. No procedure the substrate itself can represent decides the shortcut predicate, given a recursion theorem, by the same diagonal that broke Frege's floor, and the machine's answer is to record the commitment rather than automate the judgement. Of five record disciplines I modelled in fragments, one prices the write. And which event to record is a conjecture, with its refutations listed.

Part IV built twice. The small machine came first, two counters and six instructions, and on it the model stands whole: a raised flag was checked, then committed, then certified, on the same edition of the same counter, nothing gets forged, the cheapest certificate costs exactly three, and it computes everything Minsky's two-counter machines can, so its halting is undecidable. Earned commitments and universality are proved there. Then the big build, one try, in three materials, whose job was to be the witness: every "when the machine exists" from the earlier parts landed on a

theorem with a name. It prices every commitment on its fifty-one instructions and doesn't enforce that one was earned; the check and the commit stay two separate steps there, on purpose and on the page. The places where the machine came up short of the idea stayed on the page too: the free jump, which has its theorem, and the four instructions with no hardware encoding. So did the near miss: the obvious rule for composing identity arrows breaks associativity the moment two of them meet, and the machine's rule passes the identity flag along, so the laws hold on the arrows it stores, each one a theorem. Part V was the chain to the chip, with every link labelled proved, tested, or trusted, and the sentence that no board was ever powered on. Part VI was how to check it and how to break it. Every result carries the shape of the thing that would kill it, gathered in one place (Section 29): a run from uncertified to certified with the counter never rising; a contractive correlator whose pinned matrix isn't positive semidefinite; a certification system that follows the toll and certifies at total cost zero; a function on numbers that correctly decides the shortcut predicate of the substrate built from it. Any one of them brings down the theorem it matches. I hunted. I came up empty.

Two things I'd point at if you asked what's sharpest.

In the build: an integer check on eight counters, run by one instruction, forces a five-by-five matrix to be positive semidefinite, and from there the Tsirelson bound follows, and the chain never once reaches for physics. No Hilbert space, no operators, no observables. The route through the matrix isn't mine; it's the first rung of a ladder Navascués, Pironio and Acín built in 2007, and the semidefinite road to Tsirelson's bound is older than that. What's here is the check that runs in whole numbers inside an instruction, and the checked link from "this instruction didn't trap" to the bound. It's algebra you can follow line by line, and the assumption audit on that link finds exactly three standard axioms from Coq's own library (two that come with its real numbers, and functional extensionality) and nothing of mine, which is how anyone can see that no physics was smuggled in.

In the model: the diagonal rules out a correct decider whenever its flip can be expressed inside the substrate and the substrate supplies the fixed point; in L the fixed point is proved from the reduction rules and the diagonal is Rice's theorem; and on my machine's own guest the fixed point is proved by construction, so the wall stands there by proof and not by premise. For the full machine the recursion premise is false when asked for every map, because its cutoff property is decidable from outside, and the flip of every correct decider has no fixed point; both are theorems. Those are different sentences, and sharing a vocabulary doesn't make them one.

What's open, by my count. Whether the toll is the *right* thing to price, as against a wall or a penalty for contradiction, which is what the five real systems mostly built. Whether certification is the pointer event. Whether Landauer's critics are right, which would leave every theorem standing and one premise unowned by the world. And

the question I'd most like someone to take from me: is there a smaller start than a partition, by a ruler they can state? I said in Section 2 it would change this book. It would.

So that's the Thiele Machine. An abstract model with five parts and one rule, an argument made with it that the usual account of a computation is a shadow and the axis it leaves out belongs in the step, and one build of it in three materials to check the argument against. The theorems are what they are, with their premises printed. The interpretation is mine, and I've tried to say at every turn which is which, because the first time through I was the only one checking, and I've met me, and that's not enough people. Now there's you.

A Things I had to learn first

I had been around computers my whole life and paid attention to people like John Carmack on how graphics actually get made, but I came in with none of the formal fields this work turned out to need. By the time the proofs compiled I'd needed enough of half a dozen of them that I had to put down somewhere what those fields had words for. This appendix is that place. None of it is the argument. It's what I needed to be able to make it. Everything below is what I understood after I went and read about it; I'm not claiming to teach you these subjects, and if you already know a part, skip it.

A.1 Bits, bytes, and what a CPU sees

A bit is one digit that's either zero or one, the smallest fact a machine can hold. Everything else is bits stacked up. Eight of them make a byte, which lands you any number from 0 to 255, and the first thing that genuinely surprised me is that the same byte is also a letter: a convention called ASCII simply agrees to call 65 the letter A, 48 the digit 0, 32 a space. Nothing in the byte knows which it is. The meaning is a convention laid on top.

Everything in this subsection is about real CPUs and about my machine, the one build of Part IV. None of it belongs to the abstract model, which has a kind of state and a step rule and nothing about widths or slots.

A word is a chunk of bits the CPU treats as one unit. My machine's reference kernel (its definition in Coq) uses 64-bit words (any number from 0 to about 18 quintillion); the synthesised hardware uses 32-bit words (0 to about 4 billion). The abstract model doesn't care which width.

A register is a named storage slot inside the CPU that holds one word; my machine has sixteen, r0 through r15, and they're the fastest place to read or write. Memory is a longer list of word-sized slots accessed by index, the address; my machine has 128 words of it, and that's the entire address space. The program counter is one specific register that tracks where in the program the machine is; after every instruction it

moves forward by one unless the instruction says jump. A stack pointer is a register pointing at the top of a region used for nested calls: CALL pushes a return address there, RET pops it, and r15 is the stack pointer by my convention. A port is a named channel for talking to the outside world; the kernel doesn't model the outside, it charges for the communication and accepts the value the instruction declares.

Modular arithmetic means counting wraps at a maximum: with a maximum of 16, 17 wraps to 1. The machine wraps register indices modulo 16 and memory indices modulo 128, so r17 is r1. Bitwise operations work on the bits of a value one position at a time: AND is 1 only when both bits are, OR when either is, XOR (exclusive or) exactly when they differ. A shift moves every bit left or right, which multiplies or divides by a power of two. Popcount is the count of one-bits in a value. Hexadecimal is base 16, digits 0 to 9 then A to F, written with a 0x prefix: 0xFF is 255.

A trap means the machine routes execution to a fixed address instead of advancing, the hardware's "this went wrong, go to the handler." An ISA, instruction set architecture, is the list of every instruction a CPU recognises, each identified by an opcode, a numeric tag the hardware decodes. My machine has fifty-one. The abstract model has none; it has a step rule.

A.2 The notation the formal parts use

I don't think in these symbols, and I had to learn to read them. The natural numbers are 0, 1, 2, and so on, written with a double-struck $\mathbb{N}$; the integers add the negatives; the reals are the whole number line. A function from a set A to a set B is written with an arrow. For a finite set, vertical bars around it mean its size; for a number, its absolute value. A big sigma is a sum over every member of a set. A delta before a quantity means the change in it. An upside-down $\forall$ means "for every"; a backwards $\exists$ means "there exists." A double arrow is implication; a double-headed one is "if and only if." A small circle between two functions is composition, apply the right one first. Angle brackets around a random quantity mean its long-run average, which I picked up from the Bell-test papers. In this book a circled plus is bitwise XOR and a circled times has one meaning, the tensor instruction's concatenation of two coupling lists; the quantum sense of that symbol never appears, because the game section does its work in plain integer arithmetic. A superscript T on a matrix is its transpose, rows and columns swapped, and I with a subscript n is the identity matrix.

A.3 Coq, in just enough detail

Coq is a proof assistant: a programming language whose type system is expressive enough that proofs are programs and the compiler checks them (Part VI has the long version). A Theorem is a named statement followed by a proof; a Lemma is the same thing used for an intermediate result; a Definition introduces a named value; an Inductive type is one with a finite list of constructors, the only ways to build a value of it (Coq's natural numbers have two: zero, and successor of another natural). A tactic is a step inside a proof: induction (base case, then step), destruct (split into

cases), reflexivity (prove a thing equals itself), simpl (compute), lia and lra and nra (arithmetic solvers for integers, linear reals, and polynomial reals), psatz (a certificate search for polynomial inequalities). Qed ends a proof and asks Coq to check the whole chain. Admitted is the keyword for “I claim this without proof,” and it appears nowhere in the active tree. A Hypothesis inside a Section is a named premise that becomes an argument of every theorem that uses it when the Section closes; “closed under the global context” doesn’t make it go away. Print Assumptions lists the global axioms a theorem depends on; on the kernel tier it shows only standard-library ones. A total function always returns a value, and Coq requires every function to be total; the machine’s step function is, error cases included. “Parametric in K” means the proof works for any K; the machine’s sizes are literal constants, and the proofs are about those literals.

A.4 Category theory beyond Part IV

Part IV builds objects, arrows, composition, identities, associativity, and the identity laws from scratch. Beyond that: a functor is a structure-preserving map from one category to another, sending objects to objects and arrows to arrows while respecting composition and identities; think of it as a translation rule between two worlds of structure. An embedding is a functor that throws nothing away, and faithful is the precise version for arrows: two distinct arrows never land on one. Conservative is the version for inverses: if the image of an arrow has an inverse, the arrow already had one, so the functor can’t manufacture an isomorphism (an arrow with an inverse) that wasn’t there. The two properties are different, and neither gives the other. Read the natural numbers under addition as a category with one object and an arrow for each number, and the same for the integers. The inclusion of the first into the second is faithful, since distinct naturals stay distinct integers, but not conservative: 1 has no inverse among the naturals, and its image has one, minus 1. The other way round, the functor from the integers onto the trivial one-arrow group is conservative, since every arrow of the integers already has an inverse, but not faithful: it sends every integer to the same arrow. An embedding needs faithfulness specifically. A monoidal product combines two objects or two arrows into one, with a unit and laws that make the combining behave; the tensor instruction is tensor-shaped, and the checked laws for it are the interchange law under a side condition and associativity with the empty list as unit. An initial object has exactly one arrow into every object; the counter’s uniqueness in this book is a uniqueness-on-reachable-states fact, not a category-theoretic initial object. Yoneda’s lemma is where the categorical engine first clicked for me, the night I stopped borrowing other people’s runtimes, and nothing in the formal development uses it.

A.5 Bell-test vocabulary

A correlation between two outcomes is the average of their product; with outcomes plus or minus one it lies between minus one and one, plus one meaning always match, minus one always differ. A correlator is one such average in a labelled setting, E00 for

Alice's setting 0 and Bob's 0. The marginal of a joint distribution is the distribution of one side alone with the other averaged out; zero-marginal means each side's outcome is plus or minus one equally often on average. A local hidden-variable model is one where both outcomes are fixed by a shared random variable agreed before the game plus each side's own setting; Bell's theorem says no such model produces certain correlations quantum mechanics does. No-signalling means neither side's marginal depends on the other's setting. Quantum-realizable means compatible with some quantum state and measurements; the NPA hierarchy gives a ladder of necessary tests, each a positive-semidefiniteness condition on a moment matrix, and no single rung is sufficient in general. The PR box is the no-signalling correlation that wins the game every time, stronger than quantum mechanics allows; nature doesn't provide it. Cauchy-Schwarz is the inequality that a dot product is bounded by the product of the lengths. The Tsirelson bound is two root two, the largest game score quantum mechanics allows; the classical bound is two and the PR bound is four. The elliptope is the set of correlators that can be filled out into a positive-semidefinite matrix.

A.6 Hardware and synthesis vocabulary

A hardware description language describes circuits as text; Bluespec is high-level (rules, and the compiler builds the arbitration) and Verilog is low-level (registers and wires). Register-transfer-level Verilog describes how data moves between registers each clock cycle and is what synthesis tools consume. Synthesis compiles a description down to logic gates; yosys is an open-source synthesis tool. Place-and-route assigns gates to positions on a chip and wires them; nextpnr-xilinx does it for the chips this design targets. A bitstream is the binary file an FPGA reads to configure itself, and the X-Ray tools produce one from the routed design in two moves: `fasm2frames` packs the routed design into configuration frames, then `xc7frames2bit` writes the frames out as the bitstream. An FPGA is a chip you program after manufacturing: physical gates, configurable connections. A LUT, look-up table, is the FPGA's basic logic primitive, a small memory programmed to implement any small Boolean function; a flip-flop is a one-bit memory cell; a carry chain is wiring for fast addition; distributed RAM is memory built out of LUTs; block RAM is dedicated memory; DSP slices are dedicated arithmetic. This design doesn't use the DSP slices. Synthesis runs with yosys's `-nodsp` option, so the multiplier inside the game check (one wide multiplier, shared across the phases of a multi-cycle check) is built out of LUTs. With DSP slices the design didn't finish placing and routing inside the build's time limit; the open-source placer stalled and the routing came out congested. It computes the same thing either way. The choice is about what the tools can place and route, not about correctness. A netlist is the gate-level circuit written out; a VCD waveform is a record of every signal at every time step in a simulation; iverilog and Verilator are open-source Verilog simulators. A bisimulation is a precise notion of two systems behaving the same under a chosen relation on their states; field-by-field equality of states is one particular, stronger contract, and Part V says which the hardware proofs use. A parity test is the empirical cousin: run two implementations on the same set of inputs and

check that they agree, a correspondence earned by testing rather than by proof.

A.7 Other terms I use without rebuilding

Provenance is the recorded chain of where an object came from; an arrow's provenance is the sequence of instructions that built it. Simulation, in computer science, is one system imitating another's input-output behaviour; a Turing machine simulates any system of the model, my machine included, by writing the whole state out on its tape, and the difference this book is about is what a window on that tape shows. That's real, and it's exactly what people reach for when they want to wave the whole question away: if a tape can stand in for the state, what was the difference? The tape carries a picture of the state, not the state, the way a photograph of a fire keeps you exactly as warm as any other photograph. The counter and the toll live in the step rule, not in the bytes, and an encoding copies the bytes. A Turing machine built with the toll in its own transition table is a different case: it's an instance of the model, not a simulation of it. What no machine can do is make a simulated rule binding just by writing it down on tape.

Projection is a function that drops information on purpose. The bare window on my machine keeps three things, the registers, the memory and the program counter, which are what a Turing machine already has, and that's why projecting lands you back in ordinary computation. It drops everything else, including the three that were doing the new work: the counter, the certification readings, and the arrow graph. Other windows keep more: `forget` also keeps the counter, and `shadow_proj` keeps six fields and drops the graph. Nothing is wrong with any of them. They're perfectly good functions. They just cost you what made the state more than a tape. A quotient is the operation of declaring two things the same when they agree on what you chose to look at and working with the classes instead. A projection does exactly that: it doesn't just drop fields one at a time, it glues together states that the dropped fields kept apart. The fibre of a function over a value is everything the function sends to that value. An injection sends no two inputs to the same output; a surjection covers every output. Decidable means an algorithm always halts with a yes-or-no answer; keep the claim and the algorithm apart, since the algorithm decides the claim only once there's a proof that its answer matches. Polynomial time means running time bounded by a power of the input size; P is the problems solvable that way, NP the problems where a candidate solution can be *checked* that way, and whether the two are the same is open. The two budget predicates in the checked work, `error_free_preservation_budget` and `positive_input_certification_budget`, are not complexity classes. Each lets the trace be chosen separately for every input, and the bounds they take are plain functions with no requirement to be polynomial.

"Kernel" wears three hats in this book, and I'd rather point that out than let it ambush you. First, mathematics: the kernel of a map is what it sends to the identity, and for a projection the useful version is the relation "these two states project to the same thing." Second, the proofs, where the hat comes in two sizes: the kernel tier is the

part of the Coq development the main theorems live in (Section 28), holding both the model's theorems and my machine's definition, and the files are where the model is written down, not the model itself; and Coq's own kernel is the small checker that accepts or rejects every proof. Third, the operating-system kernel, which I never use and mention only so you know it isn't meant. The sentence around the word always says which one is.

A.8 Matrices

A matrix is positive semidefinite (PSD) when $v^T M v \geq 0$ for every vector v ; the game check is all about one matrix being PSD. The determinant of a square matrix is a single number that is zero exactly when the matrix squashes some direction flat. For a two-by-two matrix with rows (a, b) and (c, d) it's $ad - bc$. For a bigger one, expand along a row: for each entry, delete its row and column, take the determinant of what's left, multiply by the entry, alternate the signs, and add up; the recursion bottoms out at two by two. A principal submatrix is what you get by choosing some of the indices and keeping only those rows and the same columns; its determinant is a principal minor. A symmetric matrix is PSD exactly when every principal submatrix is, every one of them, and the game check uses the three-by-three ones (Appendix E.4). The Schur complement turns a question about a block matrix into a question about a smaller one. For a symmetric block matrix with corner block P , off-diagonal block Q and other corner R , it's $R - Q^T P^{-1} Q$, and when P is positive definite the whole matrix is PSD exactly when that complement is (Lemma E.8). Positive definite matters; invertible isn't enough. Take $P = -1$ and every other entry zero: the complement is zero, which is PSD, and the whole matrix isn't.

A.9 Borrowed ideas the book pushes on

These get their full story in the chapter that uses them; here's the one-line version for when you've forgotten which one Landauer was.

The pigeonhole principle: put more pigeons than holes and some hole holds two. Section 4's room with too few chairs is that, and so is the proof that a permanent flag can only be raised by a step that forgets something. Landauer's principle (Rolf Landauer, 1961): a step that wipes information, lowering the entropy of what a machine holds by some number of bits, gives off at least Boltzmann's constant times the temperature times the natural log of two in heat per bit. In this book it's always a premise with its name on it, never derived (Section 4, Section 26). Entropy, in Claude Shannon's sense: for a spread of chances over states, roughly the average number of yes-or-no questions you'd need to pin the state down, so an even spread over n states has $\log_2 n$ bits. A spread, in this book, is just a list of chances that add up to one. Maxwell's demon (Section 5): a little creature that watches molecules and sorts fast from slow, seemingly for free; Charles Bennett's answer was that the measuring can be free and the wiping of the demon's notes is what costs. The potential method (Robert Tarjan, 1985): pay for a run with a quantity stored in the state, so a step's bill plus the change

in the stored quantity is what you count; Section 6 reads the toll as that method with one particular potential.

The halting problem (Alan Turing, 1936): no program decides, for every program and input, whether it stops. Kleene’s recursion theorem (Stephen Kleene, 1938): for any transformation of programs that can itself be computed, some program behaves exactly like its own transform. Rice’s theorem (Henry Rice, 1953): any property that depends only on what a program does, and isn’t true of all programs or of none, can’t be decided by a program. The diagonal is the move under all three: build the thing that does the opposite of what the table says about it (Section 11). L is a small lambda calculus, a language made of nothing but functions applied to functions, which Yannick Forster and Gert Smolka built in Coq and which Section 11 and Section 24 use as a tool. A hash is a short fingerprint of a longer piece of data, made so that finding two inputs with the same fingerprint is supposed to be hard; a hash chain folds each new record into the fingerprint of the last, and a Merkle tree folds them pairwise into one root (Section 3, Section 12).

Extraction is Coq’s mechanism for turning definitions into runnable code in another language; nothing it produces is checked by the proofs after it leaves Coq (Section 28). Kami is a Coq library for writing hardware as rules and proving things about them, from Choi and coauthors (Section 27). A refinement proof says a lower-level step does what a higher-level one says, under stated side conditions; Section 27’s are per instruction.

A.10 Words this book builds

These are mine, or mine in the sense that I pinned them down for this book. Each is built properly in the chapter named; this is a reminder, not the build.

A few words first, because everything else hangs off them and each has exactly one meaning. The *model* (or the abstract model) is a kind of state and a rule for stepping, plus exactly the extra pieces a given theorem asks for: a certification reading and the toll, or a counter and a price list, or a window, or programs written as states and a recursion theorem. The name Thiele Machine belongs to the model. Parts I to III live there, and every claim in them is about any system with the pieces it names. A *substrate* is the model’s shape with whatever extension a theorem names, and nothing else; the word never means my machine, and never the side of a simulation that does the stepping ([Let me be straight with you](#)). *My machine* (also the machine I build, the build, the big build) is the concrete machine that fills in many of those pieces at once: its state record, its fifty-one instructions and their prices, the reference kernel in Coq, the guest, the extracted runner, and the hardware. The *small machine* (the smallest honest machine, Section 14) is the other piece of scaffolding, and it comes first in Part IV. The *scaffolding* is both of them together. Parts IV and V live there. When a word below belongs to only one of the two layers, the entry says which.

A *partition* is a sorting of a pile into groups, nothing in two groups and nothing left

over (Section 2). A *certification system* is a state type, moves, a step, a cost per move, and a yes-or-no reading, plus one rule, the *toll*: a move that turns the reading from no to yes costs at least one (Section 3). The files call that rule A2 in one of their two numbered lists (Appendix B). A move *merges* when two different starting states land on the same end; one that never does is *injective*, it forgets nothing. A reading is *permanent* when no move turns it from yes back to no (Section 4).

The *counter*, which I also call the *ledger*: in the model, a number carried in the state that starts at zero and that every step raises by exactly that step's charge (Section 6). My machine's counter is one; the source calls it mu (μ) (Section 16). A *price list* (on my machine, the *cost schedule*) says what each move costs; the model takes it as given, and my machine's table of what each instruction costs is in Section 17. A *receipt* is a total that lives in the state, only climbs, equals the sum of the charges on every path including the failures, and is the only such total for its price list (Section 6 gives the argument for any system with a counter and a price list; the checked theorems are about my machine, Section 23).

These belong to my machine only, and none of them appears before Part IV. A *module* is one group in my machine's partition of its memory; its *region* is the run of consecutive data-memory addresses it owns, a base and a length, inside the hundred and twenty-eight words; no two modules' regions overlap, and a run hands out at most sixty-four module numbers (Section 16). An *arrow* (a *morphism*) goes from one module to another and carries a *coupling*, a list of pairs of cells, one from each side, plus a label (Section 16, Section 18); in Section 2, "arrow" means only the category theorist's general word. The *certified flag* is set by CERTIFY and nothing ever clears it; the *assertion channel* is a separate number written by MORPH_ASSERT (Section 16, Section 17, Section 23). In the model there is just a *certification reading*, yes or no (Section 3). The *guest* is a twelve-instruction fragment of my machine, run as data by an interpreter written in my machine's own instructions (Section 24).

These belong to the small machine only (Section 14). A *two-counter machine* is Minsky's: two counters holding whole numbers with no upper limit, and two instructions: add one, and subtract one and jump if the counter is above zero, or else go on to the next line; already enough that nobody can decide in general whether one halts. The small machine is a two-counter machine plus HALT and three instructions for the record, six in all: INC, DEC, HALT, CHECK, COMMIT and CERTIFY. A counter's *version* is its edition number: it starts at 0 and goes up by one every time the counter is written, so an edition never comes back. A *fact* is a property, a counter and a version together, such as "A is even, as of edition 3"; the properties come from a fixed language of three, *is zero*, *is even* and **is at least n*. The **fact table* holds up to sixteen facts, and the only way in is a CHECK that passed; nothing ever leaves it. The small machine's *channel* holds one committed fact or nothing, and COMMIT puts a fact there only if the table has it about the counter's current edition. The *trap* is a bit that goes up when an instruction refuses, and then nothing in the core moves again. A commitment is *earned*

when a passing check put its fact in the table first, about the same edition, with no write to that counter in between; on the small machine every flag raised from a clean start is earned, and that's a theorem there, not on my machine. CHECK, COMMIT and CERTIFY cost one each and everything else costs nothing, so the cheapest certificate costs exactly three.

A *window* is any function that shows you part of a state (Section 7); a *shadow*, said loosely, is what a window leaves you with, and *the shadow*, as a named window, is my machine's six-field one of Section 20 (registers, memory, program counter, counter, error flag, certified flag). A *collision* is two steps that look the same through a window, before and after, where one raises the flag and the other doesn't (Section 7). *Entitlement* is what a run has when it carries an object, a checked relation between that object and a claim, and a paid event that put the object in place: a license to lean on the claim, not a proof that it's true (Section 8). A *latch* is a record that switches on the first time some event holds and never switches off (Section 10). The diagonal's *substrate* (Section 11) is the model's shape extended with programs, a run that may give nothing back, a counter that never falls, a way to write programs as states, the transformations the substrate counts as its own, and its own recursion theorem for those; the proof of the diagonal uses neither the counter nor the toll.

B Every assumption in one place

This book leans on a small number of rules that nothing proves, plus a handful of record types that carry rules as fields. I've named each one where it's used. Here they are side by side, because a reader who wants to break this book should be able to see every place where it says "suppose," in one list, with what each "suppose" buys.

There's one tangle to undo first, and it's my fault. The checked work has two different records with two different numbered lists of "axioms," and both lists use the label A2 for different things. The book uses "the toll" for the rule that matters and avoids the bare labels; this appendix maps the labels for anyone reading the files.

B.1 The records that carry a rule

Definition B.1 (Certification system). The record `CertificationSystem` (`UniversalCertificationCost.v`) has five data fields, a state type, a move type, a step function, a cost per move in $\mathbb{N}$, and a yes/no reading, and one proof field, `cs_cert_costs`: a move that turns the reading from no to yes costs at least one. This proof field is the toll of Section 3, and in this record's numbering it is called A2. Every result stated "for any certification system" assumes it, because the record cannot be built without it.

Definition B.2 (Abstract certification machine). The record `AbstractCertMachine` (`AbstractNoFI.v`) is narrower: its moves are the fifty-one instructions of the machine in Part IV, and it has a step, a reading, and one proof field, `acm_preserve`, saying that an instruction outside a fixed list of five certification-address setters never turns the

reading from no to yes. In this record’s numbering, A1 is the reading itself, the field `acm_cert`, a yes/no certification indicator on every state: the state has to include one. Coq enforces A1 through the record type. A record with that field left out isn’t a record of this type, so A1 is discharged when the record is built. The field `acm_preserve` is A3, the starting condition “the reading starts at no” is a theorem hypothesis called A2, and “each of the five setters costs at least one” is a separate lemma called A4 (`cert_addr_setter_cost_pos`). Its theorem `abstract_nfi` concludes that a run from no to yes contains one of the five setters; it does not mention cost.

So the same idea, “the crossing costs,” is A2 in one list and A4 in the other. The book means the first: the field `cs_cert_costs`.

Definition B.3 (Quantitative certification system). The record `QuantitativeCertificationSystem` (`QuantitativeNoFI.v`) extends a certification system with a *witness*, a function W from states to $\mathbb{N}$, and a threshold $N \in \mathbb{N}$, and adds two proof fields: (A3, `qcs_cost_bounds_witness`) $W(s) + \text{cost}(i) \geq W(\text{step}(s, i))$, the witness grows by at most what the move cost; and (A5, `qcs_cert_threshold_witness`) a state that reads yes has $W(s) \geq N$. An optional extension, `QuantitativeCertificationSystem_full`, adds (A4, `qcsf_witness_nondecreasing`) $W(s) \leq W(\text{step}(s, i))$.

Theorem B.1 (The floor raised to a threshold). *In a quantitative certification system, every run from a state with witness 0 (this starting condition is called A6) to a state that reads yes has total cost at least N .*

Proof. Adding A3 along the run telescopes: the witness at the end is at most the witness at the start plus the total cost, so at most the total cost. A5 says the end witness is at least N . Neither A4 nor the toll is used. $\square$

Checked as `universal_nfi_quantitative` (`QuantitativeNoFI.v`). A3 and A4 bound opposite directions, and neither implies the other: witness 1, cost 0, witness 0 satisfies A3 and breaks A4, and witness 0, cost 0, witness 1 does the reverse. Two instances are built: the machine’s assertion channel with threshold 1 (`thiele_quantitative_nfi_cert_addr`), and a family for the game trials of Section 25, whose cost counts valid trial instructions and whose threshold is any N : reaching N recorded trials from none takes at least N valid trial instructions (`chsh_trial_count_lower_bound`).

Here are the six labels side by side, with both numberings, so none of them is left without a meaning.

Label	In AbstractCertMachine	In the certification records
A1	the reading exists: field <code>acm_certificate</code> , enforced by the record type	(not used)
A2	the reading starts at no: a hypothesis of <code>abstract_nfi</code>	the toll: field <code>cs_cert_costs</code> of <code>CertificationSystem</code>
A3	only the five setters turn the reading on: field <code>acm_preserve</code>	the witness grows by at most the cost: field <code>qcs_cost_bounds_witness</code>
A4	each setter costs at least one: lemma <code>cert_addr_setter_cost_pos</code>	the witness never drops: field <code>qcsf_witness_nondecreasing</code> of the full record
A5	(not used)	yes needs a witness at the threshold: field <code>qcs_cert_threshold_witness</code>
A6	(not used)	the run starts with witness zero: a hypothesis of <code>universal_nfi_quantitative</code>

B.2 The premises the derivations take

These are definitions used as hypotheses. Nothing proves them in general; each theorem that uses one says so in its statement.

Premise	What it says	Where it's used	Checked name
Finite	a list with no repeats contains every state	the room argument, Section 4	<code>finite_states</code>
Permanent	no move turns the reading from yes to no	the room argument	<code>permanent</code> (and <code>permanent_at</code> for one move)
Merge pricing	every move that merges costs at least one	the toll derived, Section 4	<code>merging_steps_priced</code>
Halving price	each unit of cost pays for at most one halving of distinct states	the squeeze bound; narrowing along a run, Section 5	<code>compression_priced</code>
Entropy pricing	each move costs at least the bits it removes from any spread	the third road to the toll	<code>entropy_priced</code>
Landauer's principle	removing ΔH bits gives off heat $\geq k_B T \ln 2 \cdot \Delta H$	the heat floor; with $k_B T \geq 0$ (or > 0) as a separate hypothesis on each theorem	<code>landauer_heat</code>

Premise	What it says	Where it's used	Checked name
Recursion (generic)	every transformation the substrate can represent has a fixed point	the diagonal, Section 11	field recursion_theorem of the class Substrate
Recursion (full machine)	a fixed point for a chosen class of program transformations of the bounded machine	the machine's diagonal corollary (no decider says yes exactly when a program admits the shortcut), Section 24	section hypotheses of vm_structural_shortcut_undecidable
Commitment contract	a commitment bit is set only when the claim holds, and whenever it holds	the commitment escape, Section 7; on the machine, Section 23	CommitmentBitContract
Hardware side conditions	per-instruction preconditions (arrow-table room, modules present, no 32-bit overflow, machine live)	the hardware refinement, Section 27	WFDivenPrecondition and the admission premises of admitted_retires

Three remarks on the table. First, the recursion premise for the full machine is false when asked for every map (its thousand-step cutoff property can be decided by running both programs), and holds only for trivial classes such as the identity map; for the twelve-instruction guest, the corresponding statement is a theorem with no premise (`vm_guest_recursion_theorem_closed`, Section 24). Second, the class `Substrate` (`Substrate.v`) carries a different rule from the toll, `mu_monotone`: the counter never goes down on a step. Neither implies the other, the diagonal uses neither, and on the machine both hold. Third, merge pricing is Landauer's principle taken in its worst case over what the machine might be holding: it charges every merging move, while the principle in its general form charges only for uncertainty actually removed (Theorem 4.12).

B.3 The axioms of the checker itself

Beyond the premises above, the checked proofs rest on Coq's own foundation and on five standard-library axioms, which the assumption audit lists theorem by theorem: functional extensionality on dependent functions (the most used, across the kernel and the hardware); uniqueness of equality proofs, `eq_rect_eq` (only in the hardware proofs); two decidability principles for the real numbers (used wherever real analysis is, for example in the game check of Section 25); and the law of excluded middle (in about a hundred theorems). No axiom or parameter of the project's own appears in the audited theorems. Section 28 says what that does and doesn't mean.

C What's mine, and what isn't

Every chapter of this book leans on somebody, and I've named them where I lean. This appendix does a different job, and it's the one where I'd most like to be caught out early rather than late. For each thing the book could be accused of claiming as its own, it says what was already known, what this work actually defines, builds, proves, or connects, and what kind of contribution that is. Six kinds, and they aren't the same claim: a *new result* (a statement nobody had); a *new proof* of something known; a *mechanization* (a known result checked by a machine, here Coq); an *implementation* (something built that runs); a *synthesis* (known pieces put together under one question); and a *conjecture*. Every kind can be worth doing. None of them is the others. And if you read down the list and notice that "new result" hardly shows up, good. You're reading it right.

Two warnings about my own evidence before the list. First, "I didn't find it" is a fact about my searches, not about the literature. I searched with every tool I had, and some tools will cheerfully invent a citation if you let them, so where it matters I say where I looked, and you should assume it may well be sitting in a paper I'd need a few more lifetimes of homework to read. Second, nothing below is inferred from how many theorems there are, from the audit's clean bill, or from a good story about who came before. Those tell you the work was checked. They don't tell you it's new. Fourteen thousand checked theorems could, in principle, be fourteen thousand careful ways of agreeing with Tarjan.

C.1 1. The universal floor and the potential method

Known: the potential method of amortized analysis, a function on states whose drop bounds the cost of a run, is Tarjan's (1985), and it has been mechanized more than once (Nipkow in Isabelle/HOL, 2015 and 2019; Charguéraud and Pottier's time credits in Coq, 2019), built into type systems for automatic resource analysis (Hoffmann and coauthors, 2011 onward), and recast in cost-aware type theories (Niu, Sterling, Grodin and Harper, 2022). Mine: choosing the potential that is one when a designated reading says no and zero when it says yes, so that "the crossing costs" (the toll) and "every run that crosses paid" (the floor) are the potential method's two halves (Section 6). Kind: a mechanization of a known technique with a particular choice of potential. The choice is the point: it makes "was certified" something the account must carry. The theorem is not new mathematics. If it looks clever, that's Tarjan's cleverness; my part was picking which number to plug in.

C.2 2. The toll from finiteness, permanence, and a price on merges

Known: that steps which merge states (no single-valued inverse) are what Landauer's principle charges for is Landauer's (1961), sharpened by Bennett (1973, 1982, 2003: "the erasure of a bit or the merging of two computation paths"); generalizations and the dependence on the input distribution are Maroney's (2009), Sagawa's (2014), Wolpert's; the critique is Earman and Norton's and Norton's. Write-once memory is an old topic

in coding theory (Rivest and Shamir, 1982), about reuse rather than price. Mine: the short pigeonhole argument that on a finite machine a reading nothing can turn off can only be turned on by a merging step (Theorem 4.1), the merge-or-revoke dichotomy, the characterisation of the moves every merge-pricing schedule must charge, the squeeze and entropy forms for a permanent flip, and a built counterexample for each premise (Section 4). Kind: small new statements, I think, with mechanized proofs; the physics is a named premise and is Landauer's. Searches: I didn't find this exact statement anywhere I looked, and nobody's search covers a whole field, mine least of all; I found no mechanization of Landauer's principle in Coq, Isabelle, or Lean.

C.3 3. Learning free, committing priced

Known: that measurement can be done at no fundamental cost and the cost sits in erasing the record is Bennett's resolution of Maxwell's demon (1982), with the quantitative trade-off by Sagawa and Ueda (2009). Mine: the four-state demon and the three-state ring as checked models, and the two-state impossibility (Section 5). Kind: mechanization and illustration of a known physical point inside my ledger. Not a new physical claim. Bennett did the hard part in 1982; I built a toy with four dots on it and got Coq to agree with him.

C.4 4. Windows that can't price what they hide

Known: a function of an observation is constant on the observation's fibres; that's elementary, and the systems version of it is Moore's distinguishing experiments (1956), Kalman's observability (1960), and the completeness question of abstract interpretation (Cousot and Cousot, 1977; Giacobazzi, Ranzato and Scozzari, 2000). Mine: stating the pricing version (no window price is exact across a collision) and exhibiting colliding pairs for the machine's two named windows (Section 23). Kind: mechanization of an elementary lemma, plus specific facts about this machine.

C.5 5. Structural entitlement and its bound

Known: decision-tree counting bounds (a tree that tells N cases apart has depth at least $\log_2 N$); certifying algorithms that hand over checkable certificates (McConnell, Mehlhorn, Näher and Schweitzer, 2011); proof-carrying code (Necula, 1997); the gap between justified true belief and knowledge (Gettier, 1963). Mine: the definition of entitlement as an object, a checked relation, and a paid event on the record, and the assembled theorem that, under its listed hypotheses, a certified strengthening on the machine pays at least the index bits it commits to (Section 8, Section 22). Kind: synthesis plus mechanization; the counting idea is standard, the assembly and its vocabulary are this work's, and the pricing policy is assumed, not derived.

C.6 6. The record axis is a latch

Known: monotone, write-once state is old: the latch (Eccles and Jordan, 1918), sticky bits (Plotkin, 1989), grow-only state in replicated data types (Shapiro and coauthors), history variables and stuttering in refinement (Abadi and Lamport, 1991). Mine: the

exact statement that a record driven by the computation that never switches off is, over any base, a latch on one event, with its converse and a counterexample for each of the two conditions, and the uniqueness-up-to-price-list result with the refutation of the stronger uniqueness (Section 10, Section 23). Kind: a small structural lemma, I think not stated before in this form (I found no prior statement in the searches I ran), built from old notions, and mechanized.

C.7 7. The diagonal, L, and the guest’s recursion theorem

Known: Kleene’s recursion theorem and Rice’s theorem; Lawvere’s fixed-point form (1969) and Yanofsky’s retelling (2003); mechanized recursion theorems and Rice (Norris in HOL4, 2011; Carneiro in Lean; Forster, Kunze, Smolka and coauthors in Coq, including L itself, 2017, and the Coq Library of Undecidability). Mine: the diagonal stated once over any substrate that supplies a recursion theorem; Rice and halting proved again for L from its reduction rules; and a recursion theorem constructed for the machine’s twelve-instruction guest, ledger included, with the evaluator compiled into guest instructions through L and the library (Section 11 and Section 24). Kind: checked instances, not new theorems of computability. The guest result is an instance of a theorem that holds for any model with universality and s-m-n; the work is in building that for this machine.

C.8 8. The game check

Known: that positive semidefiniteness of a correlation matrix gives the Tsirelson bound is Tsirelson’s (1980) and Wehner’s semidefinite proof (2006); the moment-matrix hierarchy, the Schur-complement analysis of this very level-one matrix, the arcsine characterization, and the level $1 + AB$ are Navascués, Pironio and Acín’s (2007, 2008), with Landau (1988) and Masanes (2003); and Tsirelson’s bound has already been checked by machine, in Isabelle/HOL (Echenim and Mhalla, 2023) and in Lean’s mathlib. Rational sum-of-squares certificates are Peyrl and Parrilo’s (2008). Mine: a decision procedure in integer arithmetic on outcome counters that runs inside one instruction; a checked chain from “that instruction didn’t trap” to “the pinned level-one matrix is positive semidefinite” to “ $S^2 \leq 8$ ”; proofs of the contraction equivalence by explicit test vectors; the level-1 + AB cascade and an elliptope gate in whole numbers, with a soundness proof and no step of the machine attached to it; and a circuit proved to compute the same check on 32-bit counters (Section 25 and its derivation appendix). Kind: implementation and mechanization, with a new proof of a known equivalence in one slice. I found no prior formalization taking counters as input or linking a machine instruction to the bound, and that’s a report on where I looked, not on everything that exists.

C.9 9. The verified path to a chip

Known: Kami (Choi, Vijayaraghavan, Sherman, Chlipala and Arvind, 2017), Kôika (Bourgeat, Pit-Claudel, Chlipala and Arvind, 2020), the end-to-end verified stack of Erbsen and coauthors (2021), CakeML on the verified Silver processor (Löow and

coauthors, 2019), and the discipline of naming what isn't proved (CompCert, seL4). Mine: an instruction-by-instruction Kami refinement for this machine, a proved serial loader, and a labelled chain to a bitstream (Section 27). Kind: implementation of an established method. No new verification method; extraction, synthesis, and the chip are trusted, and no board was run.

C.10 10. Five record disciplines, one question

Known: each system and its guarantee belongs to its designers: Casper (Buterin and Griffith, 2017), Ethereum's storage gas and refunds (EIP-2200, EIP-3529), extend-only TPM registers, Certificate Transparency (RFC 9162), proof-carrying code. Mine: asking all five whether the write is priced, with a small checked fragment of each (Section 12). Kind: synthesis and an organizing reading; the fragments are not verifications of those systems.

C.11 11. The pointer conjecture

Known: Zurek's quantum Darwinism (2009), with Ollivier, Poulin and Zurek (2004) and Blume-Kohout and Zurek (2006): preferred states are the ones whose records proliferate redundantly in the environment. Mine: borrowing that shape for the question of which event deserves a record, with definitions, a toy, and refutations of two stronger forms (Section 13). Kind: conjecture. No claim about quantum physics.

C.12 What that adds up to

Read straight down the list and the honest summary is this. The mathematics underneath this book is mostly known, and some of it is very old. What's mine is the question (should an account of computation carry what was certified and what certifying cost, as part of how it steps), a handful of small exact statements that the question needed and that I didn't find stated elsewhere (items 2, 4's pricing form, and 6), a large body of mechanized proof that ties those statements to one machine that runs, and my best attempt at saying which is which, which you should check as hard as anything else in here. If someone shows me item 2, the pricing form in item 4, or item 6 stated earlier, I'll move it up the list and be glad to, and a little embarrassed, in that order.

D Every result and where it is checked

Every numbered result in this book is in the table below, in the order it appears, with the Coq names that carry it, the file each lives in, and what the last full assumption audit printed for it. "none" means the theorem is closed: its proof uses nothing beyond Coq's own rules. The other entries name the standard-library assumptions it uses: FE is functional extensionality on dependent functions, UIP is uniqueness of equality proofs (`eq_rect_eq`), RD is the pair of decidability principles that come with the real numbers, and EM is excluded middle. "def" marks a definition, which has nothing to assume. "not probed" marks a name the audit doesn't list, usually a local lemma; its file is still given. A premise or a conjecture has no Coq name on purpose: it's assumed

or open, and the section that states it says so.

This table is an index of companion formalizations. The statements, hypotheses and arguments are in the numbered sections it points to. An audit status reports the recorded check; running a later version does not change a claim in this book.

Result	Sec.	Coq name and file	Axioms
Definition 2.1: Equivalence relation, partition	§2	stated in the text	
Theorem 2.1: Sameness and sorting are one object	§2	argued in the text, not checked in Coq	
Definition 3.1: Certification system	§3	CertificationSystem (UniversalCert ificationCost.v)	def
Theorem 3.1: The universal floor	§3	universal_nfi_any_substrate (Univ ersalCertificationCost.v) cs_state (UniversalCertificationCos t.v) cs_instr (UniversalCertificationCos t.v) cs_step (UniversalCertificationCost. v) and 3 more	none def def def
Definition 4.1: Finite, permanent, injective	§4	permanent (PermanentCertification.v) step_injective (PermanentCertificat ion.v)	def def
Theorem 4.1: A permanent flip merges	§4	permanent_flip_is_not_injective (PermanentCertification.v)	none
Premise 4.2: Merge pricing	§4	merging_steps_priced (PermanentCer tification.v)	def
Theorem 4.2: The toll from finiteness, permanence, and merge pricing	§4	a2_from_merging_price_and_perman ence (PermanentCertification.v) permanent_certification_trace_fl oor (PermanentCertification.v) certification_system_from_mergin g_price (PermanentCertification.v)	none none def
Theorem 4.3: Merge or revoke	§4	flip_merges_or_revokes (PermanentR ecordPricing.v) permanent_at_flip_is_not_injecti ve (PermanentRecordPricing.v)	none none
Definition 4.3: Forced to be priced	§4	forced_priced (PermanentRecordPrici ng.v)	def
Theorem 4.4: Forced price is merging	§4	forced_priced_iff_merges (Permane ntRecordPricing.v)	none

Result	Sec.	Coq name and file	Axioms
Corollary 4.5: Permanent records are forced	§4	forced_priced_iff_merges (PermanentRecordPricing.v) permanent_record_write_is_forced_priced (PermanentRecordPricing.v) forced_price_without_permanent_record (PermanentRecordPricing.v)	none none none
Premise 4.4: Halving price	§4	compression_priced (PermanentRecordPricing.v)	def
Theorem 4.6: The squeeze bound	§4	a2_from_compression_price_and_permanence (PermanentRecordPricing.v) permanent_flips_compression_bound (PermanentRecordPricing.v) permanent_flips_log_bound (PermanentRecordPricing.v) permanent_flips_collapse_at_least (PermanentCertification.v)	none none none none
Theorem 4.7: A move that forgets nothing removes no bits	§4	step_entropy_invariant_if_injective (PermanentCertificationEntropy.v)	FE, RD, EM
Theorem 4.8: Entropy after a permanent flip	§4	permanent_step_entropy_ceiling (PermanentCertificationEntropy.v) permanent_flip_uniform_entropy_drop (PermanentCertificationEntropy.v)	FE, RD, EM FE, RD, EM
Premise 4.5: Landauer's principle, as a premise	§4	landauer_heat (PermanentCertificationEntropy.v)	def
Corollary 4.9: Heat floor of a permanent flip	§4	permanent_step_entropy_ceiling (PermanentCertificationEntropy.v) permanent_flip_uniform_entropy_drop (PermanentCertificationEntropy.v) entropy_le_log_support (PermanentCertificationEntropy.v) permanent_flip_heat_floor (PermanentCertificationEntropy.v) and 1 more	FE, RD, EM FE, RD, EM FE, RD, EM FE, RD, EM FE, RD, EM
Corollary 4.10: Positive heat for any full spread	§4	permanent_flip_full_support_entropy_drop_positive (PermanentCertificationEntropy.v) permanent_flip_full_support_heat_positive (PermanentCertificationEntropy.v)	FE, RD, EM FE, RD, EM
Definition 4.6: Entropy-priced	§4	stated in the text	

Result	Sec.	Coq name and file	Axioms
Theorem 4.11: The toll from entropy pricing	§4	a2_from_entropy_price_and_permanence (PermanentCertificationEntropy.v) entropy_priced_trace_floor (PermanentCertificationEntropy.v) certification_system_from_entropy_price (PermanentCertificationEntropy.v)	FE, RD, EM FE, RD, EM def
Theorem 4.12: A known state owes nothing	§4	known_state_step_removes_no_entropy (PermanentCertificationEntropy.v) known_state_flip_forces_no_heat (if and only if) (PermanentCertificationEntropy.v)	FE, RD, EM FE, RD, EM
Theorem 4.13: The bits removed, one term per state	§4	entropy_drop_as_point_sum (PermanentCertificationEntropy.v) entropy_drop_nonneg (PermanentCertificationEntropy.v) entropy_drop_pos_of_support_merge (PermanentCertificationEntropy.v)	FE, RD, EM FE, RD, EM EM
Theorem 5.1: The machine's own narrowing is priced	§5	run_narrowing_priced_log (KnowledgeNarrowing.v)	none
Definition 5.1: What an observer knows	§5	knowledge (KnowledgeNarrowing.v)	def
Theorem 5.2: An observer can learn for free	§5	observer_narrowing_can_be_free (KnowledgeNarrowing.v) demon_observer_learns (KnowledgeNarrowing.v) demon_machine_spread_kept (KnowledgeNarrowing.v) measure_forgets_nothing (KnowledgeNarrowing.v) and 2 more	none none none none
Theorem 5.3: Three states are the minimum for free teaching	§5	free_incremental_narrowing_with_three (KnowledgeNarrowingMinimal.v) no_free_incremental_narrowing_below_three (KnowledgeNarrowingMinimal.v) demon_refutes_incremental (KnowledgeNarrowingMinimal.v)	none none none
Definition 6.1: A counter kept by a price list	§6	stated in the text	
Theorem 6.1: Conservation	§6	argued in the text, not checked in Coq	

Result	Sec.	Coq name and file	Axioms
Theorem 6.2: Only one honest count	§6	argued in the text, not checked in Coq	
Definition 6.2: Potential, amortized cost	§6	amortized_nonneg (CostSemanticsComparison.v) cert_potential (CostSemanticsComparison.v)	def def
Theorem 6.3: The toll is the potential method	§6	a2_iff_nonnegative_amortized_cost (CostSemanticsComparison.v) potential_telescoping (CostSemanticsComparison.v) nfi_by_potential (CostSemanticsComparison.v) certification_system_is_potential_method (CostSemanticsComparison.v) and 2 more	none none none none
Theorem 7.1: Decoders need constant fibres	§7	decoding_requires_fiber_constancy (ObservationPolicy.v) selected_representatives_give_decoder (ObservationPolicy.v)	none none
Theorem 7.2: No exact window price across a collision	§7	shadow_floor_overcharges (ShadowPricing.v) shadow_cannot_price_exactly (ShadowPricing.v) window_showing_reading_prices_exactly (ShadowPricing.v) window_showing_reading_has_no_collision (ShadowPricing.v)	none none none none
Definition 8.1: Entitlement	§8	stated in the text	
Definition 8.2: Decision tree	§8	stated in the text	
Lemma 8.1: A tree is no wider than two to its depth	§8	decision_tree_leaves_le_pow2_depth (MuShannonBridge.v) decision_tree_log2_up_leaf_bound (MuShannonBridge.v)	none none
Theorem 8.2: Paid narrowing, counted	§8	argued in the text, not checked in Coq	
Definition 9.1: Certified insight event, raw observation	§9	stated in the text	
Remark 9.1: What the toll prices, and what it doesn't	§9	argued in the text	
Remark 9.2: Nothing makes the crossing wait	§9	argued in the text	

Result	Sec.	Coq name and file	Axioms
Definition 10.1: Base, record-carrying machine, cover	§10	stated in the text	
Definition 10.2: Honest extension, latch	§10	stated in the text	
Theorem 10.1: The record is a latch	§10	record_axis_is_latch_holds (StructuralRecordAxis.v) BaseMachine (StructuralCoreAnyBase.v) BaseCover (StructuralCoreAnyBase.v) computation_driven (StructuralCoreAnyBase.v) and 6 more	none def def def
Theorem 10.2: Every reachable latch is an honest record	§10	latch_core_honest (RecordAxisDiscrimination.v)	none
Theorem 10.3: Records on reversible machines	§10	history_latch_injective (RecordAxisDiscrimination.v) history_latch_honest (RecordAxisDiscrimination.v) finite_reversible_cannot_write (RecordAxisDiscrimination.v) permanent_flip_is_not_injective (PermanentCertification.v)	none none none none
Lemma 10.4: Two records are two latches	§10	record_pair_is_two_latches_holds (StructuralRecordAxis.v)	none
Lemma 10.5: The toggle	§10	toggle_not_permanent (StructuralRecordAxis.v) toggle_not_latch (StructuralRecordAxis.v)	none none
Lemma 10.6: The clock	§10	toggle_computation_driven (StructuralRecordAxis.v) toggle_not_permanent (StructuralRecordAxis.v) toggle_not_latch (StructuralRecordAxis.v) clock_record_permanent (StructuralRecordAxis.v) and 1 more	none none none none
Corollary 10.7: The event is a free choice	§10	argued in the text, not checked in Coq	
Theorem 10.8: The schedule doesn't fix the probabilities	§10	schedule_determines_probabilities_refuted (ProbabilisticRecord.v) deterministic_latch_handles_branching_refuted (ProbabilisticRecord.v)	none none

Result	Sec.	Coq name and file	Axioms
Definition 10.4: Weak base equivalence	§10	stated in the text	
Theorem 10.9: Outside bases, and the relation	§10	weak_base_equiv_refl_holds (CrossBaseGranularity.v) weak_base_equiv_sym_holds (CrossBaseGranularity.v) weak_base_equiv_trans_holds (CrossBaseGranularity.v) weak_match_left_runs (CrossBaseGranularity.v) and 5 more	none none none none
Definition 10.5: A growing record	§10	stated in the text	
Theorem 10.10: Growing records are families of latches	§10	growing_record_decomposes_holds (GrowingRecord.v) thresholds_determine_record_holds (GrowingRecord.v) record_price_iff_threshold_price_holds (GrowingRecord.v) one_latch_refuted (GrowingRecord.v) and 1 more	none none none none
Definition 11.1: Substrate	§11	stated in the text	
Definition 11.2: Shortcut predicate	§11	stated in the text	
Theorem 11.1: No inside decider	§11	structural_shortcut_undecidable (if and only if) (StructuralUndecidability.v) Substrate (Substrate.v) WithShortcutPredicate (StructuralUndecidability.v)	none def def
Theorem 11.2: The nat family	§11	nat_self_undecidable (NatSubstrateInstance.v) nat_recursion_theorem (NatSubstrateInstance.v) nat_structural_shortcut_undecidable (if and only if) (NatSubstrateInstance.v)	none none none
Definition 11.3: L, as built here	§11	stated in the text	
Theorem 11.3: Recursion, Rice and halting for L	§11	second_recursion (LRecursion.v) L_recursion_theorem (LRecursion.v) L_rice (LRecursion.v) L_halting_undecidable (LRecursion.v) and 2 more	none none none none

Result	Sec.	Coq name and file	Axioms
Definition 12.1: Casper setting and votes	§12	stated in the text	
Theorem 12.1: Accountable safety	§12	accountable_safety (CasperFFG.v)	none
		conflicting_records_are_priced (CasperRecordReading.v)	none
Theorem 12.2: The premise can hold, and the write is free	§12	casper_fork_exists (CasperForkWitness.v)	none
		casper_fork_slashable (CasperForkWitness.v)	none
		finalization_without_slashing (CasperRecordReading.v)	none
Definition 12.2: One slot in one transaction	§12	stated in the text	
Theorem 12.3: The persistent write is priced; the revoked one nearly is not	§12	persistent_write_priced (EVMStorageGas.v)	none
		revoked_write_nearly_free (EVMStorageGas.v)	none
Definition 12.3: A gas schedule	§12	stated in the text	
Theorem 12.4: Gas schedule exactness	§12	gas_schedule_exactness (if and only if) (GasMetering.v)	none
		undercharged_opcode_admits_free_commitment (GasMetering.v)	none
		undercharged_opcode_breaks_certification_floor (GasMetering.v)	none
		overcharge_breaks_exactness (GasMetering.v)	none
		and 1 more	
Theorem 12.5: The logical level always pays	§12	permanent_write_has_logical_payment (PricingPhysicsAudit.v)	none
		permanent_at_flip_is_not_injective (PermanentRecordPricing.v)	none
Definition 12.4: Platform and quote	§12	stated in the text	
Theorem 12.6: The quote cannot attest unmeasured state	§12	quote_cannot_attest_unmeasured_state (TPMQuoteGap.v)	none
		quote_decides_measured_claims (TPMQuoteGap.v)	none
		tpm_interface_authenticity_refuted (TPMQuoteAuthenticity.v)	none
Definition 12.5: Inclusion check, RFC 9162 section 2.1.3.2	§12	stated in the text	

Result	Sec.	Coq name and file	Axioms
Theorem 12.7: The RFC's own examples pass	§12	rfc9162_example_inclusion_d3 (RFC9162Merkle.v) rfc9162_example_inclusion_d0 (RFC9162Merkle.v) rfc9162_example_inclusion_d4 (RFC9162Merkle.v) rfc9162_example_inclusion_d6 (RFC9162Merkle.v) and 3 more	none none none none
Definition 12.6: The bounds fragment	§12	stated in the text	
Theorem 12.8: The checker is the verification condition	§12	pcc_checker_accepts_iff_vc (NeculaPCC.v) pcc_certificate_implies_vc (NeculaPCC.v) pcc_unsafe_program_rejected (NeculaPCC.v)	none none none
Definition 13.1: Ecosystem, records, proliferates, pointer	§13	stated in the text	
Conjecture 13.1: The pointer conjecture	§13	open	
Theorem 13.2: The toy	§13	toy_cert_unique_pointer (PointerObservable.v) toy_work_not_proliferating (PointerObservable.v)	none none
Theorem 13.3: The survey, and the swap	§13	twelve_candidate_measurements_checked (RecordProliferationSurvey.v) swapped_event_is_pointer_checked (RecordProliferationSurvey.v) five_labeled_models_have_selected_pointer (PointerObservableReductions.v) pointer_criterion_holds_indexed (PointerObservableReductions.v)	none none none def
Theorem 13.4: One blind observer is enough	§13	blind_observer_blocks_proliferation (PointerObservableCounterexamples.v) labeled_model_verdicts (PointerObservableCounterexamples.v)	none none
Definition 13.2: Observation game	§13	stated in the text	
Theorem 13.5: Agreement does not buy permanence; durability does	§13	toggle_game_refutes_strong_pointer_necessity (EcosystemGame.v) durable_consensus_implies_permanence (EcosystemGame.v)	none none

Result	Sec.	Coq name and file	Axioms
Definition 14.1: Properties, checker, meaning	§14	stated in the text	
Lemma 14.1: The checker says what it means	§14	<code>eval_iff (EarnedCore.v)</code>	not probed
Definition 14.2: Core, state, claim, start	§14	stated in the text	
Definition 14.3: Instructions, costs, the step	§14	<code>cexec (EarnedCore.v)</code> <code>exec (EarnedCore.v)</code> <code>run (EarnedCore.v, +10)</code> <code>total_cost (EarnedCore.v, +1)</code> and 2 more	def def not probed def
Definition 14.4: Two-counter program, compile, window	§14	stated in the text	
Theorem 14.2: Step for step	§14	<code>simulation_step (EarnedCore.v)</code> <code>simulation_run (EarnedCore.v)</code>	not probed not probed
Theorem 14.3: Halting matches	§14	<code>halting_correspondence (EarnedCore.v)</code>	not probed
Theorem 14.4: Halting is undecidable for this machine	§14	<code>mm2_step_iff (EarnedCoreLinks.v)</code> <code>mm2_stop_iff (EarnedCoreLinks.v)</code> <code>mm2_terminates_iff (EarnedCoreLinks.v)</code> <code>mm2_halting_iff (if and only if) (EarnedCoreLinks.v)</code> and 1 more	none none none none
Theorem 14.5: Ledger, toll, permanence	§14	<code>mu_conservation (EarnedCore.v, +2)</code> <code>mu_conservation_trace (EarnedCore.v, +1)</code> <code>only_certify_certifies (EarnedCore.v, +1)</code> <code>a2 (EarnedCore.v, +3)</code> and 1 more	none not probed not probed not probed
Theorem 14.6: An instance of the certification system	§14	<code>earned_core_floor (EarnedCoreLinks.v)</code> <code>universal_nfi_any_substrate (UniversalCertificationCost.v)</code> <code>nfi_floor (EarnedCore.v, +1)</code> <code>earned_cs (EarnedCoreLinks.v)</code>	none none not probed def

Result	Sec.	Coq name and file	Axioms
Theorem 14.7: The flag is a latch of the base	§14	earned_core_honest (EarnedCoreLinks.v) earned_core_is_latch (EarnedCoreLinks.v) record_axis_is_latch_holds (StructuralRecordAxis.v) cert_latch (EarnedCore.v) and 1 more	none none none not probed
Definition 14.5: Untouched	§14	untouched (EarnedCore.v)	def
Theorem 14.8: A commitment was checked first	§14	earned_commitment_provenance (EarnedCore.v)	not probed
Theorem 14.9: A raised flag was checked, then committed, then certified	§14	earned_certification_provenance (EarnedCore.v)	not probed
Theorem 14.10: Unearned steps trap and change no claim	§14	unearned_commit_traps (EarnedCore.v) uncommitted_certify_traps (EarnedCore.v)	not probed not probed
Theorem 14.11: Checker soundness	§14	checker_soundness (EarnedCore.v) committed_claim_holds (EarnedCore.v) sound_step (EarnedCore.v) sound (EarnedCore.v)	not probed not probed not probed def
Definition 14.6: Earned fact; no forgery	§14	stated in the text	
Theorem 14.12: No forging	§14	facts_step (EarnedCore.v) no_forging_step (EarnedCore.v) no_forging (EarnedCore.v)	not probed not probed not probed
Theorem 14.13: Bounded table, no eviction	§14	facts_keep (EarnedCore.v) full_table_traps (EarnedCore.v) facts_bounded_step (EarnedCore.v)	not probed not probed not probed
Theorem 14.14: Same window, different record	§14	receipt_separation (EarnedCore.v, +1)	not probed

Result	Sec.	Coq name and file	Axioms
Theorem 14.15: No window oracle	§14	no_mu_oracle (EarnedCore.v, +2) no_cert_oracle (EarnedCore.v, +2) no_commit_oracle (EarnedCore.v)	not probed not probed not probed
Theorem 14.16: The minimum price of a certificate	§14	certified_run_min_cost (EarnedCore.v) program_certified_min_cost (EarnedCore.v)	not probed not probed
Theorem 14.17: Three is tight	§14	min_cost_tight (EarnedCore.v) witness (EarnedCore.v)	not probed def
Definition 16.1: The state record	§16	stated in the text	
Definition 16.2: Registers, memory, and the word	§16	REG_COUNT (ThieleMachineComplete.v, +1) MEM_SIZE (ThieleMachineComplete.v, +1) word64 (ThieleMachineComplete.v, +1) read_reg (ThieleMachineComplete.v, +1) and 4 more	def def def def
Theorem 16.1: Conservation on this machine	§16	vm_apply_mu (ThieleMachineComplete.v, +1)	none
Remark 16.1: The error flag latches, argued from the rules	§16	latch_err (ThieleMachineComplete.v, +1)	def
Lemma 16.2: The two lengths are kept	§16	eg_size_permanent (EventGeneralization.v)	none
Definition 16.3: Partition graph, module, arrow	§16	PartitionGraph (ThieleMachineComplete.v, +1) ModuleState (ThieleMachineComplete.v, +1) MorphismState (ThieleMachineComplete.v, +1) CouplingData (ThieleMachineComplete.v, +1) and 1 more	def def def def
Definition 16.4: Disjoint and contiguous regions	§16	stated in the text	

Result	Sec.	Coq name and file	Axioms
Theorem 16.3: The regions stay a partition of addresses	§16	vm_step_preserves_regions_disjoint (VMStep.v) vm_step_preserves_regions_contiguous (VMStep.v) vm_reachable_regions_disjoint (VMStep.v) graph_pnew_preserves_regions_disjoint (VMStep.v) and 4 more	none none none none
Definition 16.5: Well-formed graph	§16	stated in the text	
Theorem 16.4: Well-formedness is kept	§16	vm_step_preserves_well_formed_graph (VMStep.v) vm_apply_preserves_well_formed_graph (SimulationProof.v) run_vm_preserves_well_formed_graph (SimulationProof.v)	none none none
Definition 16.6: A partition in bounds	§16	partition_in_bounds (VMStep.v) regions_in_memory (VMStep.v) module_ids_distinct (VMStep.v)	def def def
Theorem 16.5: The partition stays in bounds	§16	vm_step_preserves_partition_in_bounds (VMStep.v) vm_step_preserves_regions_in_memory (VMStep.v) vm_step_preserves_modules_bounded (VMStep.v) vm_step_preserves_module_ids_distinct (VMStep.v) and 7 more	none none none none
Definition 16.7: The initial state	§16	stated in the text	
Corollary 16.6: What every run from the start keeps	§16	empty_graph_well_formed (VMState.v) init_state (ThieleMachineComplete.v, +1) empty_graph (ThieleMachineComplete.v, +1)	none def def
Remark 16.2: The four constants	§16	REG_COUNT (ThieleMachineComplete.v, +1) MEM_SIZE (ThieleMachineComplete.v, +1) NUM_MODULES (ThieleMachineComplete.v, +1) REGION_SIZE (VMState.v)	def def def def
Definition 17.1: Instructions, step relation, step function	§17	stated in the text	

Result	Sec.	Coq name and file	Axioms
Lemma 17.1: The relation computes the function	§17	vm_step_vm_apply (SimulationProof.v) vm_step_deterministic (SimulationProof.v) vm_step (VMStep.v, +1) vm_apply (ThieleMachineComplete.v, +2) and 1 more	none none def def
Definition 17.2: PNEW, PSPLIT, PMERGE	§17	step_pnew (VMStep.v) step_psplitt (VMStep.v) step_pmerge (VMStep.v) pnew_region (ThieleMachineComplete.v, +1) and 14 more	def def def def
Theorem 17.2: A successful LASSERT pays for what it read	§17	lassert_honest_cost (StateSpaceCounting.v) lassert_honest_mu_cost (StateSpaceCounting.v) lassert_cost_from_component_floors (MuCostDerivation.v) lassert_cost_is_its_formula (MuCostDerivation.v)	none none none none
Remark 17.1: What a passing G345 step gives	§17	chsh_lassert_lab_g345_no_trap_implies_npa_psd_qlab (QuantumPartitionPSD_1AB.v) qlab_g345_full_integer_check_kernel (ThieleMachineComplete.v, +1)	FE, RD def
Definition 17.3: Classical instructions	§17	stated in the text	
Theorem 17.3: Classical runs move nothing that certifies	§17	classical_trace_preserves_graph (ClassicalConservativity.v) classical_trace_preserves_cert_addr (ClassicalConservativity.v) classical_trace_preserves_certified (ClassicalConservativity.v) classical_opcodes_preserve_structure_run_vm (ClassicalConservativity.v) and 1 more	none none none none
Definition 17.4: The schedule	§17	stated in the text	
Definition 17.5: The positive class and the setter class	§17	stated in the text	

Result	Sec.	Coq name and file	Axioms
Theorem 17.4: The floor	§17	cert_setter_cost_pos (VMStep.v) cert_setter_cost_above_declared (VMStep.v) non_cert_setter_cost_is_declared (VMStep.v) payload_bit_length_ascii (ThieleMachineComplete.v, +1) and 9 more	none none none none
Theorem 17.5: The joint floor is the least	§17	joint_floor_is_least (ObservationPolicy.v)	none
Theorem 17.6: Only CERTIFY touches the flag, and it stays set	§17	vm_apply_certified (ThieleMachineComplete.v, +1) certification_reading_permanent (EventSwapTheorem.v) no_free_certification_certified (ThieleMachineComplete.v, +1) no_free_certification_certified_mu (AbstractNoFI.v) and 2 more	none none none none
Theorem 17.7: Only MORPH_ASSERT touches the channel	§17	non_morph_assert_preserves_cert_addr (RevelationRequirement.v) morph_assert_step_sets_supra_cert_iff (if and only if) (RevelationRequirement.v) step_morph_assert_ok (VMStep.v) step_morph_assert_bad (VMStep.v) and 1 more	none none def def
Theorem 17.8: No free certification on this machine	§17	no_free_certification (AbstractNoFI.v) no_free_certification_certified (ThieleMachineComplete.v, +1) certification_requires_positive_mu (ThieleMachineComplete.v, +1) no_free_certified_insight (InsightTaxonomy.v) and 3 more	none none none none
Definition 18.1: Category	§18	stated in the text	
Definition 18.2: Coupling, relational composition, diagonal	§18	stated in the text	

Result	Sec.	Coq name and file	Axioms
Definition 18.3: Stored arrow and the composition rule	§18	kernel_relational_compose_same (CategoryLaws.v) graph_compose_morphisms_coupling (CategoryBridge.v) relational_compose (ThieleMachineComplete.v, +2) diagonal (CategoryLaws.v) and 5 more	none none def def
Theorem 18.1: Laws of relational composition	§18	relational_compose_spec (if and only if) (ThieleMachineComplete.v, +1) relational_compose_assoc (ThieleMachineComplete.v, +1) relational_compose_diagonal_left (ThieleMachineComplete.v, +1) relational_compose_diagonal_right (ThieleMachineComplete.v, +1) and 8 more	none none none none
Theorem 18.2: Tensor on couplings	§18	monoidal_coherence (ThieleMachineComplete.v, +1) tensor_bifunctor (ThieleMachineComplete.v, +1)	none none
Definition 18.4: The same stored arrow	§18	stored_arrow_equiv (CategoryBridge.v)	def
Theorem 18.3: The category laws on stored arrows	§18	graph_compose_identities_is_identity (CategoryBridge.v) graph_compose_assoc_stored (CategoryBridge.v) graph_compose_left_identity_stored (CategoryBridge.v) graph_compose_right_identity_stored (CategoryBridge.v) and 2 more	none none none none
Theorem 18.4: Every stored identity is the identity of its object	§18	vm_step_preserves_identity_arrows_canonical (CategoryBridge.v) vm_reachable_preserves_identity_arrows_canonical (CategoryBridge.v) vm_reachable_identity_arrows_canonical (CategoryBridge.v) identity_arrows_canonical (CategoryBridge.v)	none none def
Definition 18.5: Loading a coupling from memory	§18	driven_step_morph (GraphReconstructionBridge.v) load_coupling_from_mem (ThieleMachineComplete.v, +1) serialized_coupling_pair_count (ThieleMachineComplete.v, +1)	none def def

Result	Sec.	Coq name and file	Axioms
Remark 19.1: What the steps do with the label lists	§19	graph_add_axiom (ThieleMachineComplete.v, +1) graph_record_discovery (ThieleMachineComplete.v, +1) vm_apply (ThieleMachineComplete.v, +2) graph_pnew (ThieleMachineComplete.v, +1) and 2 more	def def def def
Definition 19.1: The formula check	§19	stated in the text	
Theorem 19.1: What the check step does	§19	vm_apply_lassert_pc (DagRestriction.v) vm_apply_mu (ThieleMachineComplete.v, +1) lassert_honest_cost (StateSpaceCounting.v) vm_apply (ThieleMachineComplete.v, +2) and 5 more	none none none def
Remark 19.2: Why a tautology cannot pass	§19	argued in the text	
Theorem 19.2: The chip's scan is the kernel's check	§19	hw_scan_certcheck (LassertSpec.v) hw_scan_spec (LassertSpec.v) logic_engine_equivalent_lassert (LogicEngineEquivalence.v)	none none none
Definition 19.2: Structural gain, located	§19	stated in the text	
Theorem 19.3: The gain has a place and a price	§19	supra_cert_implies_structure_addition_in_run (RevelationRequirement.v) structure_addition_in_run_iff_morph_assert_bridge_pattern (RevelationRequirement.v) morph_assert_step_sets_supra_cert_iff (if and only if) (RevelationRequirement.v) non_morph_assert_trace_has_no_structure_addition (RevelationRequirement.v) and 1 more	none none none
Remark 19.3: Clear at the start and set at the end is the stronger reading	§19	argued in the text	
Definition 20.1: Equivalent and distinct	§20	stated in the text	

Result	Sec.	Coq name and file	Axioms
Definition 20.2: The separation pair	§20	stated in the text	
Theorem 20.1: Categorical separation	§20	categorical_separation (ThieleMachineComplete.v, +1) computationally_equivalent (ThieleMachineComplete.v, +1) categorically_distinct (ThieleMachineComplete.v, +1) separation_A (ShadowProjection.v) and 1 more	none def def def
Definition 20.3: The shadow	§20	stated in the text	
Theorem 20.2: The shadow separation theorem	§20	shadow_separation_theorem (ShadowProjection.v) shadow_strictly_lossy (ShadowProjection.v) no_classical_separation (ShadowProjection.v) classical_bool_test_indistinguishable (PartitionSeparation.v) and 1 more	none none none none
Theorem 20.3: The shadow is a lossy quotient	§20	degenerate_projection_theorem (if and only if) (TuringClassicalEmbedding.v) shadow_proj_kernel_is_eq_on_classical_shadow (TuringClassicalEmbedding.v) classical_run_preserves_shadow_compat (TuringClassicalEmbedding.v) shadow_strictly_lossy (ShadowProjection.v)	none none none none
Remark 20.1: Why neither state of the pair is reachable	§20	eg_size_permanent (EventGeneralization.v)	none
Definition 20.4: Classical reachability	§20	stated in the text	
Theorem 20.4: Strictness, and with it conservativity	§20	pnew_step_separates_thiele_from_classical_reachable (TuringStrictness.v) pnew_step_separates_thiele_from_classical_at_init (TuringStrictness.v) pnew_step_separates_thiele_from_classical (TuringStrictness.v) classical_opcodes_preserve_structure (ClassicalConservativity.v) and 5 more	none none none none

Result	Sec.	Coq name and file	Axioms
Theorem 20.5: Each of the three is independent of the others	§20	thiele_state_three_component_independence (NecessityAbstract.v) mu_ledger_mutual_independence (ThieleMachineComplete.v, +1) graph_not_recoverable_from_P_full (NecessityAbstract.v) mu_ledger_full_pairwise_independence (ThieleMachineComplete.v) and 3 more	none none none none
Theorem 20.6: Three fields no bare window recovers	§20	cert_not_function_of_forget (ProjectionNonExistence.v) mu_not_function_of_bare_observable (ProjectionNonExistence.v) cert_addr_not_function_of_forget (ProjectionNonExistence.v) no_classical_a2_cert_predicate (ProjectionNonExistence.v) and 8 more	none none none none
Theorem 20.7: A counter beside a Turing machine changes nothing	§20	thiele_simulates_turing_gen (ProperSubsumption.v) thiele_simulates_turing (ProperSubsumption.v)	none none
Definition 20.5: A system that simulates the machine	§20	stated in the text	
Theorem 20.8: The machine represents any system that simulates it	§20	thiele_represents_simulating_cert_system (UniversalCertificationCost.v) scs_run_embed (UniversalCertificationCost.v) universal_nfi_any_substrate (UniversalCertificationCost.v) SimulatingCertificationSystem (UniversalCertificationCost.v) and 4 more	none none none def
Theorem 21.1: A failed assertion still pays	§21	morph_cost_morph_assert (CategoryBridge.v) morph_assert_mu_pos (InsightTaxonomy.v) morph_assert_step_sets_supra_cert_iff (if and only if) (RevelationRequirement.v) step_morph_assert_bad (VMStep.v)	none none none def

Result	Sec.	Coq name and file	Axioms
Theorem 21.2: Building leaves the channel alone	§21	structural_trace_preserves_cert_addr (InsightTaxonomy.v) thiele_non_cert_addr_setter_preserves (ThieleMachineComplete.v, +1) morph_cost_morph (CategoryBridge.v)	none none none
Theorem 21.3: A successful assertion	§21	certified_insight_nonfree (InsightTaxonomy.v) step_morph_assert_ok (VMStep.v)	none def
Lemma 21.4: The probe fails on an empty arrow list	§21	graph_empty_morphisms_delete_fails_tc (ThieleMachineComplete.v) morph_delete_no_morphism_err_tc (ThieleMachineComplete.v) step_morph_delete_bad (VMStep.v)	none none def
Theorem 21.5: One step the classical instructions can't match	§21	morph_delete_probe_separates_thiele_from_classical_tc (ThieleMachineComplete.v) morph_id_step_passes_morph_delete_probe_tc (ThieleMachineComplete.v) classical_trace_fails_morph_delete_probe_tc (ThieleMachineComplete.v)	none none none
Definition 22.1: The two readings	§22	stated in the text	
Definition 22.2: Certified insight event, and the three tiers	§22	stated in the text	
Lemma 22.1: A trial is raw observation	§22	chsh_trial_is_tier0 (WitnessInsightGeneral.v) instruction_cost (ThieleMachineComplete.v, +1)	none def
Theorem 22.2: Certified insight is not free	§22	certified_insight_nonfree (InsightTaxonomy.v) no_free_certification (AbstractNoFI.v) no_free_certification_certified (ThieleMachineComplete.v, +1) certification_requires_positive_mu (ThieleMachineComplete.v, +1) and 1 more	none none none none
Theorem 22.3: The price of a game witness is flat	§22	witness_insight_nonfree_general (WitnessInsightGeneral.v) no_free_certification_certified_trace_mu (WitnessInsightGeneral.v) no_free_certified_insight (InsightTaxonomy.v) witness_insight_complete_taxonomy (WitnessInsightGeneral.v)	none none none none

Result	Sec.	Coq name and file	Axioms
Lemma 22.4: The check commits nothing	§22	non_morph_assert_preserves_cert_addr (RevelationRequirement.v) vm_apply_certified (ThieleMachineComplete.v, +1)	none none
Remark 22.1: The commit reads no test	§22	step_morph_assert_ok (VMStep.v) vm_apply (ThieleMachineComplete.v, +2)	def def
Definition 22.3: Runs, state lists, observations, predicates	§22	stated in the text	
Theorem 22.5: Narrowing becomes strengthening	§22	feasible_strict_subset_implies_strict_predicates (InformationGainToStrengthening.v) b4_information_reduction_derives_strict_predicates (HonestNoFI_TheoremsWithoutAssumptions.v) omega_predicate (InformationGainToStrengthening.v)	none none def
Definition 22.4: Certified end, structure addition	§22	stated in the text	
Theorem 22.6: A certified end from a clear start has an assertion event	§22	strengthening_requires_structure_addition (ThieleMachineComplete.v, +1) supra_cert_implies_structure_addition_in_run (RevelationRequirement.v) structure_addition_in_run_iff_morph_assert_bridge_pattern (RevelationRequirement.v)	none none none
Definition 22.5: Decision tree, cost-floor count, fibres	§22	stated in the text	
Lemma 22.7: Paid steps cover the counter	§22	info_priced_cert_executions_bounded (MuShannonBridge.v) cert_setter_cost_pos (VMStep.v) delta_mu_equals_ledger_sum (MuShannonBridge.v)	none none none

Result	Sec.	Coq name and file	Axioms
Lemma 22.8: Index bits are paid for	§22	info_priced_posterior_representative_reduction_bound (MuShannonBridge.v) posterior_representative_reduction_implies_fibered_reduction (MuShannonBridge.v) fibered_reduction_implies_tree_cover (MuShannonBridge.v) tree_cover_implies_log2_up_reduction_bound (MuShannonBridge.v) and 3 more	none none none none
Theorem 22.9: Structural entitlement, represented	§22	structural_entitlement_representation (HonestNoFI_TheoremsWithoutAssumptions.v) observational_structural_entitlement_representation (HonestNoFI_TheoremsWithoutAssumptions.v)	none none
Remark 22.2: Why two observations	§22	distinguishing_observation_not_posterior_representable (HonestNoFI_TheoremsWithoutAssumptions.v)	none
Remark 22.3: The complete-tree bound	§22	exists_covering_tree (MuShannonBridge.v) complete_tree_realization_bound (MuShannonBridge.v) info_priced_weighted_feasible_reduction_bound (MuShannonBridge.v) observation_partition_reduction_implies_posterior_representative_reduction (MuShannonBridge.v)	none none none none
Definition 22.6: Sound structural shortcut	§22	stated in the text	
Corollary 22.10: Every sound shortcut lands in the bound	§22	every_sound_structural_shortcut_lands_here (HonestNoFI_TheoremsWithoutAssumptions.v) sound_shortcut_from_components (HonestNoFI_TheoremsWithoutAssumptions.v)	none none
Theorem 22.11: A checked bit search supplies the entitlement data	§22	bit_search_entitlement (BitSearchEntitlement.v) bit_search_bound_reads (BitSearchEntitlement.v) posterior_is_what_the_run_certifies (if and only if) (BitSearchEntitlement.v) bit_search_shortcut (BitSearchEntitlement.v)	not probed not probed not probed def

Result	Sec.	Coq name and file	Axioms
Remark 22.4: What the member computes	§22	factored_n1_shortcut_lands_in_representation (StructuralAdvantageCertifiedShortcut.v) factored_n1_shortcut (StructuralAdvantageCertifiedShortcut.v)	none def
Remark 22.5: What the speedup theorems state	§22	blind_program_run (StructuralAdvantage.v) sighted_program_run (StructuralAdvantage.v) time_tax_theorem (if and only if) (StructuralAdvantage.v) time_tax_sighted_wins (StructuralAdvantage.v) and 7 more	none none none none
Definition 23.1: Classical opcode, classical run	§23	is_classical_opcode (ClassicalConservativity.v)	def
Theorem 23.1: Conservativity	§23	classical_opcodes_preserve_structure (ClassicalConservativity.v) classical_trace_preserves_graph (ClassicalConservativity.v) classical_trace_preserves_cert_addr (ClassicalConservativity.v) classical_trace_preserves_certified (ClassicalConservativity.v) and 2 more	none none none none
Theorem 23.2: The counter on this machine	§23	mu_accumulates_trace_cost (ThieleMachineComplete.v, +1) mu_is_initial_monotone (ThieleMachineComplete.v, +1) vm_mu_monotonic_single_step (ThieleMachineComplete.v, +1) monotone_factors_through_mu (MuInitiality.v) and 1 more	none none none none
Theorem 23.3: Folding a run is the only way	§23	thiele_trace_fold_initial (ThieleInitiality.v) thiele_is_initial_a2_substrate (ThieleInitiality.v)	none def

Result	Sec.	Coq name and file	Axioms
Theorem 23.4: When the fold comes down to states	§23	trace_descent_unique_value_iff (TraceStateDescent.v) raceStateDescent.v) reachable_simulation_exists_iff (TraceStateDescent.v) reachable_simulation_requires_compatibility (TraceStateDescent.v) reachable_simulation_unique (TraceStateDescent.v) and 5 more	none none none none
Theorem 23.5: Neither the counter nor the flag can be dropped	§23	P_full_complete_neither_mu_nor_cert_droppable (ThieleMachineComplete.v, +1) forgets_mu_not_mu_complete (ThieleMachineComplete.v, +1) forgets_cert_not_cert_complete (ThieleMachineComplete.v, +1)	none none none
Definition 23.2: Two windows; a function of a window	§23	stated in the text	
Theorem 23.6: Three fields no window recovers	§23	cert_not_function_of_forget (ProjectionNonExistence.v) mu_not_function_of_bare_observable (ProjectionNonExistence.v) cert_addr_not_function_of_forget (ProjectionNonExistence.v) thiele_content_irrecoverable_from_bare_projection (ProjectionNonExistence.v)	none none none none
Theorem 23.7: The floor on this machine	§23	no_free_certification (AbstractNoFI.v) no_free_certification_certified (ThieleMachineComplete.v, +1) certification_requires_positive_mu (ThieleMachineComplete.v, +1) no_free_certification_trace_mu (ThieleMachineComplete.v, +1) and 1 more	none none none none
Theorem 23.8: The machine runs the finite fragment	§23	vm_runs_finite_machine (FiniteCertMachine.v) vm_pays_finite_price (FiniteCertMachine.v) vm_runs_finite_trace (FiniteCertMachine.v) vm_fragment_certification_paid (FiniteCertMachine.v) and 1 more	none none none none

Result	Sec.	Coq name and file	Axioms
Theorem 23.9: The certifying merge is priced; a free merge exists	§23	vm_certify_merges (FiniteCertMachine.v) vm_certifying_step_is_priced_merge (FiniteCertMachine.v) vm_jump_is_free_merge (FiniteCertMachine.v) vm_prices_certifying_merge_leaves_others_free (FiniteCertMachine.v)	none none none none
Definition 23.3: Shadow price, floor, no overcharge, collision	§23	stated in the text	
Theorem 23.10: Neither window prices certification exactly	§23	vm_bare_observable_collision (ShadowPricing.v) vm_forget_collision (ShadowPricing.v) shadow_cannot_price_exactly (ShadowPricing.v) vm_bare_shadow_cannot_price_exactly (ShadowPricing.v) and 3 more	none none none none
Theorem 23.11: No sound and complete verifier on the bare window	§23	bare_setting_no_sound_complete_verifier (VerifierImpossibility.v) pol_state_A (NecessityOfMuLedger.v, +1) pol_state_B (NecessityOfMuLedger.v, +1)	none def def
Theorem 23.12: A richer verifier that only looks at the bare part does no better	§23	V_does_not_factor_through_classical (VerifierExhaustiveness.v)	none
Definition 23.4: Three richer transcripts and their explanations	§23	stated in the text	
Theorem 23.13: Three conditional escapes	§23	substrate_escape_succeeds (VerifierEscape_Substrate.v) commitment_contract_verifier (VerifierEscape_Hardness.v) honest_commitments_satisfy_contract (VerifierEscape_Hardness.v) unchecked_bit_violates_contract (VerifierEscape_Hardness.v) and 1 more	none none none none
Definition 23.5: This machine as a record-carrying machine	§23	stated in the text	

Result	Sec.	Coq name and file	Axioms
Definition 23.6:	§23	stated in the text	
Adequate; same core			
Definition 23.7: Honest extension of this machine; observed sameness	§23	stated in the text	
Theorem 23.14: Exact uniqueness is false	§23	billed_core_adequate (StructuralUniqueness.v) billed_core_honest_extension (StructuralUniqueness.v) billed_core_not_equiv (StructuralUniqueness.v) billed_core_not_observed_equiv (StructuralUniqueness.v) and 8 more	none none none none
Theorem 23.15: History is forgotten by the relation, and only by it	§23	history_core_equiv_thiele (StructuralCore.v) certification_agreement_does_not_imply_descent (TraceStateDescent.v)	none none
Definition 23.8: The same up to the schedule	§23	stated in the text	
Theorem 23.16: Unique up to the schedule, for the certified flag	§23	cert_record_schedule_uniqueness_holds (StructuralScheduleUniqueness.v) billed_core_equiv_mod_schedule (StructuralScheduleUniqueness.v) surcharged_core_honest_cert (StructuralScheduleUniqueness.v) surcharged_core_equiv_mod_schedule (StructuralScheduleUniqueness.v) and 6 more	none none none none
Theorem 23.17: Not unique for a tied record	§23	meter_core_honest_tied (StructuralScheduleUniqueness.v) meter_core_not_equiv_mod_schedule (StructuralScheduleUniqueness.v) tied_record_schedule_uniqueness_refuted (StructuralScheduleUniqueness.v)	none none none

Result	Sec.	Coq name and file	Axioms
Theorem 23.18: Attestation cannot factor through the bare transcript	§23	attestation_cannot_factor_throug h_bare_transcript (TEEAttestation. v) V_does_not_factor_through_classi cal (VerifierExhaustiveness.v) replay_is_a_two_preimage_witness (TEEAttestation.v) measurement_enriched_attestation _succeeds (TEEAttestation.v) and 2 more	none none none none
Theorem 23.19: The certified flag is a permanent consensus	§23	vm_certification_is_permanent_co nsensus (EcosystemGame.v) vm_apply_u_certified_permanent (V MUnboundedLedger.v)	none none
Definition 24.1: The machine's shortcut property	§24	vm_admits_shortcut_yes (Structural Undecidability.v) vm_admits_shortcut_no (StructuralU ndecidability.v) vm_instantiation_target_witnesse d (StructuralUndecidability.v) vm_admits_shortcut_extensional (S tructuralUndecidability.v) and 1 more	none none none def
Premise 24.2: A recursion theorem for the full machine	§24	assumed, not proved	
Theorem 24.1: The wall on the machine, conditional	§24	vm_structural_shortcut_undecidab le (if and only if) (StructuralUndecidabil ity.v) vm_structural_shortcut_undecidab le_encoded (if and only if) (VMSubstrateE ncoded.v) vm_encode_decode_concrete (VMSubs trateEncoded.v)	none none none
Theorem 24.2: Bounded runs are decidable from outside	§24	vm_bounded_shortcut_decide_corre ct (VMBoundedDecidability.v) vm_bounded_decider_flip_not_repr esentable (VMBoundedDecidability.v)	none none
Theorem 24.3: The full-machine recursion premise fails, and every correct flip has no fixed point	§24	vm_full_recursion_premise_refute d (VMBoundedDecidability.v) vm_correct_flip_has_no_fixed_poi nt (VMBoundedDecidability.v) vm_run (VMSubstrateInstance.v) simple_morph_trace (SimpleMorphSho rtcut.v)	none none def def
Definition 24.3: Halting without a cutoff	§24	stated in the text	

Result	Sec.	Coq name and file	Axioms
Theorem 24.4: A program halts at most once	§24	vm_halts_at_deterministic (VMUnboundedExec.v) vm_halts_at (VMUnboundedExec.v)	none def
Lemma 24.5: The accumulator commutes with every step and every run	§24	vm_apply_logic_acc_commutates (VMEncodedInputAccess.v) run_vm_logic_acc_commutates (VMEncodedInputAccess.v)	none none
Theorem 24.6: No interpreter for the accumulator encoding	§24	vm_apply_logic_acc_commutates (VMEncodedInputAccess.v) run_vm_logic_acc_commutates (VMEncodedInputAccess.v) halts_observes_logic (if and only if) (VMEncodedInputAccess.v) no_logic_acc_encoded_interpreter (if and only if) (VMEncodedInputAccess.v)	none none none none
Definition 24.4: Guest configurations and runs	§24	stated in the text	
Theorem 24.7: The self-interpreter is correct	§24	self_interpreter_complete (VMSelfRun.v) self_interpreter_sound (VMSelfRun.v) self_interpreter_correct (if and only if) (VMSelfRun.v) self_interpreter_divergence (if and only if) (VMSelfRun.v) and 3 more	none none none none
Definition 24.5: Behaviour, extensional, decides, undecidable	§24	stated in the text	
Theorem 24.8: Rice on the guest	§24	self_rice (VMSelfRiceUndec.v) g_decides_decidable (VMSelfRiceUndec.v) self_rice_representable (VMSelfRiceUndec.v) g_halts_on_zero_undecidable (VMSelfRiceUndec.v) and 7 more	none none none
Definition 24.6: Represents, pairing prefix	§24	stated in the text	
Lemma 24.9: The prefix pairs	§24	g_pair_smn (if and only if) (VMGuestRecursion.v)	none

Result	Sec.	Coq name and file	Axioms
Theorem 24.10: Guest recursion theorem	§24	vm_guest_recursion_theorem_close d (VMGuestRecursion.v) g_pair_smn (if and only if) (VMGuestRecursion.v) hfun_sem (if and only if) (VMGuestEvalNat.v) RD_MMA (VMGuestEvalL.v) and 4 more	none none none none
Theorem 25.1: The classical ceiling, reached at zero cost	§25	classical_bound_achieved (ThieleMachineComplete.v, +1)	none
Definition 25.1: Correlators from the counters; the pinned matrix	§25	stated in the text	
Theorem 25.2: The gate implies the pinned matrix is PSD	§25	chsh_lassert_no_trap_implies_npa_psd (QuantumPartitionPSD.v) chsh_lassert_check_implies_npa_psd (QuantumPartitionPSD.v) npa_psd_iff_column_contractive (if and only if) (QuantumPartitionPSD.v)	FE, RD FE, RD FE, RD
Theorem 25.3: PSD forces the Tsirelson bound	§25	npa_psd_implies_tsirelson_bound (MuLedgerQuantumBridge.v) npa_psd_zero_marginal_implies_row_bounds (MuLedgerQuantumBridge.v) chsh_gap_is_sum_of_squares (ThieleMachineComplete.v, +1)	FE, RD FE, RD FE, RD
Corollary 25.4: A passed gate means the bound	§25	chsh_lassert_no_trap_implies_npa_psd (QuantumPartitionPSD.v) npa_psd_implies_tsirelson_bound (MuLedgerQuantumBridge.v) chsh_lassert_no_trap_implies_state_column_contractive (MuLedgerQuantumBridge.v) state_column_contractive_implies_tsirelson (MuLedgerQuantumBridge.v)	FE, RD FE, RD FE, RD FE, RD
Remark 25.1: Sound, not complete	§25	zero_marginal_implies_elliptope (ElliptopeCompletion.v) deterministic_strategy_elliptope (ElliptopeCompletion.v)	RD FE, RD
Definition 25.2: The size-generic PSD test	§25	stated in the text	

Result	Sec.	Coq name and file	Axioms
Corollary 25.5: The level-1 test is the size-five case	§25	column_contractive_iff_general_realizable (GenRealizability.v) column_contractive_qlab_iff_general_realizable (GenRealizability.v) vm_chsh_lassert_step_is_general_realizable (GenRealizability.v)	FE, RD FE, RD FE, RD
Definition 26.1: The indicator	§26	stated in the text	
Theorem 26.1: The indicator is bounded by the cost	§26	irreversible_bits_le_cost (MuLedgerConservation.v, +1) total_irreversible_bits_le_cost (LandauerDerivation.v) irreversible_bits (MuLedgerConservation.v, +1)	none none def
Definition 26.2: Energy at a scale	§26	stated in the text	
Theorem 26.2: The calibration is a unit conversion	§26	calibrated_mu_landauer_energy (PricingPhysicsAudit.v) mu_has_no_intrinsic_joule_value (PricingPhysicsAudit.v) mu_landauer_calibration (PricingPhysicsTarget.v)	FE, RD, EM FE, RD def
Premise 26.3: Landauer, in units of the counter	§26	assumed, not proved	
Theorem 26.3: When the premises can hold together	§26	calibrated_model_exists_iff_positive_cost (ObservationPolicy.v) bit_model_has_satisfiable_calibration (ObservationPolicy.v) landauer_dissipation_premises_inconsistent (LandauerDissipationStrongForm.v) calibrated_positive_model (ObservationPolicy.v) and 2 more	none none none def
Theorem 26.4: Keeping history makes the free merge injective	§26	retained_history_step_injective (ObservationPolicy.v) zero_cost_vm_jump_has_injective_history_lift (ObservationPolicy.v, +1)	none none
Definition 26.4: The two-state protocol	§26	stated in the text	

Result	Sec.	Coq name and file	Axioms
Theorem 26.5: The heat is fixed by the chosen gap	§26	canonical_reset_satisfies_master_equation (CalorimeterProtocol.v) canonical_reset_heat_exact (CalorimeterProtocol.v) selected_gap_gives_landauer_heat (CalorimeterProtocol.v) smaller_gap_refutes_unconditional_landauer_floor (given $0 < k_B$ and $0 < T$) (CalorimeterProtocol.v) and 1 more	FE, RD FE, RD FE, RD, EM FE, RD, EM
Theorem 26.6: The protocol's one event against the machine's floor	§26	vm_minimal_certification_charges_canonical_reset_mu (CalorimeterProtocol.v) vm_certification_charges_at_least_canonical_reset_mu (given a step that turns certification on) (CalorimeterProtocol.v)	none none
Premise 26.5: The Unruh-Clausius calibration	§26	mu_landauer_unruh_calibrated (ThieleMachineComplete.v, +1)	def
Remark 26.1: What the Unruh-Clausius calibration does and doesn't do	§26	mu_landauer_unruh_calibrated_from_constant_and_bit_calibration (BekensteinCalibration.v) nfi_cost_nonzero_implies_nontrivial_calibration (ThieleMachineComplete.v, +1) discrete_gauss_bonnet_delta (NoFIToEinstein.v) einstein_emerges (EinsteinEmergence.v) and 1 more	FE, RD, EM FE, RD FE, RD FE, RD
Theorem 26.7: What the machine reaches is flat	§26	vm_reachable_regions_separate (ReachableGeometry.v) reachable_no_adjacent_modules (ReachableGeometry.v) reachable_flat_reading (ReachableGeometry.v) reachable_calibrated_iff_no_modules (ReachableGeometry.v) and 2 more	none none FE, RD FE, RD
Definition 26.6: Level-k certified	§26	stated in the text	

Result	Sec.	Coq name and file	Axioms
Theorem 26.8: The mu-hierarchy	§26	mu_hierarchy_theorem (MuHierarchyTheorem.v) mu_hierarchy_strict (MuHierarchyTheorem.v) mu_hierarchy_no_upper_bound (MuHierarchyTheorem.v) mu_dimension_unbounded (MuHierarchyTheorem.v) and 1 more	none none none none
Definition 27.1: Board state, snapshot, step firing	§27	kami_step (Abstraction.v) step (EarnedCore.v, +14)	def not probed
Theorem 27.1: ADD refines	§27	step_add_refines (StepRefine.v) rule_next_correct (RuleStep.v) step_next_correct (RuleStep.v)	FE, UIP FE, UIP FE, UIP
Theorem 27.2: The game check refines	§27	chsh_run_result (ChshRun.v) chsh_check_word_correct (ChshArith.v) chsh_lassert_refines (ChshRetire.v) chsh_only_rule_enabled (ChshRetire.v)	FE, UIP UIP FE, UIP FE, UIP
Theorem 27.3: The satisfiable LASSERT refines	§27	lassert_sat_refines (LassertRetire.v)	FE, UIP
Lemma 27.4: The hardware scan computes the kernel's checks	§27	lassert_sat_refines (LassertRetire.v) lassert_unsat_refines (LassertRetire.v) hw_scan_spec (LassertSpec.v) hw_scan_certcheck (LassertSpec.v)	FE, UIP FE, UIP none none
Theorem 27.5: The checking step against the kernel	§27	logic_engine_equivalent_lassert (LogicEngineEquivalence.v)	none
Definition 27.2: Retires; admitted	§27	stated in the text	
Theorem 27.6: Every admitted instruction retires	§27	admitted_retires (RetireMaster.v) step (EarnedCore.v, +14) Retire (RetireMaster.v) admitted (RetireMaster.v)	FE, UIP not probed def def

Result	Sec.	Coq name and file	Axioms
Theorem 27.7: The specification step is the kernel step	§27	driven_step_wf (GraphReconstructionBridge.v) driven_trace_commutates (GraphReconstructionBridge.v) WFDrivenRun (GraphReconstructionBridge.v) WFDrivenPrecondition (GraphReconstructionBridge.v) and 2 more	FE FE def def
Theorem 27.8: Start from reset	§27	start_from_reset (CPUStart.v) start_substep (CPUStart.v) start (EarnedCore.v)	FE, UIP FE def
Theorem 27.9: Serial load	§27	one_frame (LoaderSerial.v) serial_program_load (LoaderSerial.v) step_is_rxSample (LoaderSerial.v) empty_program_starts (LoaderSerial.v)	FE, UIP FE, UIP FE, UIP none
Theorem 27.10: Hand-off and write	§27	load_rule_hands_off (LoaderSerial.v) load_instr_writes (LoaderSerial.v) load_rule (LoaderSerial.v)	FE, UIP FE, UIP def
Theorem 27.11: Status report	§27	one_frame (LoaderSerial.v) serial_status_report (ReportSerial.v) report_transmits (ReportSerial.v) report_bytes_kernel (ReportSerial.v) and 6 more	FE, UIP FE, UIP FE, UIP none
Definition B.1: Certification system	§B	CertificationSystem (UniversalCertificationCost.v) cs_cert_costs (UniversalCertificationCost.v)	def def
Definition B.2: Abstract certification machine	§B	cert_addr_setter_cost_pos (ThieleMachineComplete.v, +1) abstract_nf_i (ThieleMachineComplete.v, +1) AbstractCertMachine (ThieleMachineComplete.v, +1) acm_preserve (AbstractNoFI.v) and 1 more	none none def def

Result	Sec.	Coq name and file	Axioms
Definition B.3: Quantitative certification system	§B	QuantitativeCertificationSystem (QuantitativeNoFI.v) qcs_cost_bounds_witness (QuantitativeNoFI.v) qcs_cert_threshold_witness (QuantitativeNoFI.v) QuantitativeCertificationSystem_full (ThieleMachineComplete.v, +1) and 1 more	def def def def
Theorem B.1: The floor raised to a threshold	§B	universal_nfi_quantitative (ThieleMachineComplete.v, +1) thiele_quantitative_nfi_cert_add_r (ThieleMachineComplete.v, +1) chsh_trial_count_lower_bound (ThieleMachineComplete.v, +1)	none none none
Definition E.1: Settings, outcomes, correlators	§E	stated in the text	
Definition E.2: Counters	§E	WitnessCounts (ThieleMachineComplete.v, +1) record_trial (ThieleMachineComplete.v, +1)	def def
Lemma E.1: Deterministic strategies	§E	argued in the text, not checked in Coq	
Definition E.3: Finite factorizable model, is_ factorizable	§E	is_factorizable (MinorConstraints.v)	def
Theorem E.2: fine_th eorem, one direction only	§E	fine_theorem (MinorConstraints.v) factorizable_CHSH_classical_bound (MinorConstraints.v)	FE, RD FE, RD
Remark E.1: What the name does and does not cover	§E	fine_theorem (MinorConstraints.v) local_box_CHSH_bound (MinorConstraints.v)	FE, RD FE, RD
Definition E.4: WCLocallyConsistent	§E	WCLocallyConsistent (CHSHStatisticalBridge.v)	def
Theorem E.3: A tally above 2 is not one replayed strategy	§E	chsh_violation_rules_out_locally_factorizable_coupling (CHSHCouplingBridge.v) locally_consistent_classical_bound (CHSHCouplingBridge.v) chsh_stat_from_wc (CHSHStatisticalBridge.v)	none none def
Lemma E.4: The six-square identity, chs h_gap_is_sum_of_s quares	§E	chsh_gap_is_sum_of_squares (ThieleMachineComplete.v, +1)	FE, RD

Result	Sec.	Coq name and file	Axioms
Theorem E.5: Row-bound form of Tsirelson's bound	§E	tsirelson_squared (TsirelsonFromAlgebra.v) tsirelson_from_row_bounds (ThieleMachineComplete.v, +1) tsirelson_bound_abs (ThieleMachineComplete.v, +1)	FE, RD FE, RD FE, RD
Theorem E.6: Tightness	§E	tsirelson_tight (TsirelsonFromAlgebra.v) tsirelson_achievable (ThieleMachineComplete.v, +1)	FE, RD FE, RD
Remark E.2: Where equality can happen	§E	argued in the text	
Lemma E.7: A rational constant above the bound, rational_tsirelson_bound	§E	rational_tsirelson_bound (ThieleMachineComplete.v, +1)	FE, RD
Definition E.5: The pinned level-1 matrix	§E	zero_marginal_npa (NPAMomentMatrix.v) NPAMomentMatrix (NPAMomentMatrix.v) npa_to_matrix (NPAMomentMatrix.v)	def def def
Lemma E.8: Schur complement, the standard form	§E	argued in the text, not checked in Coq	
Definition E.6: Column contractive, zero_marginal_column_contractive	§E	zero_marginal_column_contractive (MuLedgerQuantumBridge.v)	def
Lemma E.9: The 2x2 test, psd2_quadratic_form_nonneg	§E	psd2_quadratic_form_nonneg (MuLedgerQuantumBridge.v)	FE, RD
Theorem E.10: Contractive implies PSD	§E	zero_marginal_npa_column_contractive_implies_psd (MuLedgerQuantumBridge.v) PSD5 (ConstructivePSD.v)	FE, RD def
Lemma E.11: Discriminant, quadratic_nonneg_discriminant	§E	quadratic_nonneg_discriminant (ConstructivePSD.v)	FE, RD
Theorem E.12: PSD implies contractive	§E	npa_psd_implies_column_contractive (QuantumPartitionPSD.v) npa_quad5_test_col0 (QuantumPartitionPSD.v) npa_quad5_test_col1 (QuantumPartitionPSD.v) npa_quad5_test_schur (QuantumPartitionPSD.v) and 1 more	FE, RD FE, RD FE, RD FE, RD

Result	Sec.	Coq name and file	Axioms
Corollary E.13: The equivalence	§E	npa_psd_iff_column_contractive (if and only if) (QuantumPartitionPSD.v) column_contractive_iff_npa_psd (QuantumPartitionPSD.v) PSD5 (ConstructivePSD.v) npa_psd (NPAMomentMatrix.v) and 1 more	FE, RD FE, RD def def
Theorem E.14: PSD implies Tsirelson	§E	npa_psd_implies_tsirelson_bound (MuLedgerQuantumBridge.v) npa_psd_implies_tsirelson_bound_abs (MuLedgerQuantumBridge.v) psd_3x3_determinant_nonneg (ConstructivePSD.v) npa_psd_zero_marginal_implies_row_bounds (MuLedgerQuantumBridge.v) and 3 more	FE, RD FE, RD FE, RD FE, RD
Remark E.3: What PSD of the pinned matrix means	§E	zero_marginal_implies_elliptope (ElliptopeCompletion.v)	RD
Definition E.7: column_contractive_check_witness	§E	column_contractive_check_witness (ThieleMachineComplete.v, +1)	def
Lemma E.15: The seven facts are the three conditions with denominators cleared	§E	argued in the text, not checked in Coq	
Theorem E.16: Soundness of the integer check	§E	column_contractive_check_witness_sound (MuLedgerQuantumBridge.v) state_column_contractive_check_witness_sound (MuLedgerQuantumBridge.v) column_contractive_check_witness (ThieleMachineComplete.v, +1) true (ThieleMachineComplete.v, +6) and 1 more	FE, RD FE, RD def def
Theorem E.17: No trap implies PSD, chsh_lassert_no_trap_implies_npa_psd	§E	chsh_lassert_no_trap_implies_npa_psd (QuantumPartitionPSD.v) chsh_lassert_check_implies_npa_psd (QuantumPartitionPSD.v) instr_chsh_lassert (ThieleMachineComplete.v, +14) npa_psd (NPAMomentMatrix.v) and 1 more	FE, RD FE, RD def def

Result	Sec.	Coq name and file	Axioms
Corollary E.18: No trap implies the Tsirelson bound	§E	chsh_lassert_no_trap_implies_state_column_contractive (MuLedgeQuantumBridge.v) state_column_contractive_implies_tsirelson (MuLedgeQuantumBridge.v) state_zero_marginal_npa (MuLedgeQuantumBridge.v) zero_marginal_npa (NPAMomentMatrix.v)	FE, RD FE, RD def def
Remark E.4: The circuit	§E	chsh_check_nat_correct (ChshArith.v) chsh_check_word_correct (ChshArith.v) chsh_check_word_spec (ChshArith.v) chsh_run_result (ChshRun.v) and 2 more	none UIP UIP FE, UIP
Definition E.8: qlab_moment_matrix	§E	qlab_moment_matrix (QuantumPartitionPSD_1AB.v)	def
Definition E.9: The residual, qlab_residual	§E	qlab_residual (QuantumPartitionPSD_1AB.v) column_contractive_qlab (QuantumPartitionPSD_1AB.v)	def def
Lemma E.19: Three squares plus the residual, quad9_qlab_sos_decomposition	§E	quad9_qlab_sos_decomposition (QuantumPartitionPSD_1AB.v)	FE, RD
Theorem E.20: PSD at level 1+AB, qlab_psd_iff_column_contractive	§E	qlab_psd_iff_column_contractive (if and only if) (QuantumPartitionPSD_1AB.v) column_contractive_qlab_implies_psd9 (QuantumPartitionPSD_1AB.v) psd9_implies_column_contractive_qlab (QuantumPartitionPSD_1AB.v) PSD9 (QuantumPartitionPSD_1AB.v) and 1 more	FE, RD FE, RD FE, RD def
Lemma E.21: Fraction-free Schur step	§E	sym6_Schur_identity (QuantumPartitionPSD_1AB.v) sym5_Schur_identity (QuantumPartitionPSD_1AB.v) sym6_qf_nonneg_from_pd (QuantumPartitionPSD_1AB.v) sym5_qf_nonneg_from_pd (QuantumPartitionPSD_1AB.v)	FE, RD FE, RD FE, RD FE, RD

Result	Sec.	Coq name and file	Axioms
Lemma E.22: Sylvester for 4x4, sym4_qf_nonneg_from_pd	§E	sym4_qf_nonneg_from_pd (QuantumPartitionPSD_1AB.v) sym4_LDLT_identity (QuantumPartitionPSD_1AB.v) sym4_d1 (QuantumPartitionPSD_1AB.v) sym4_d4 (QuantumPartitionPSD_1AB.v) and 1 more	FE, RD FE, RD def def
Definition E.10: What the integer check computes	§E	g12345_COMMON_Z (ThieleMachineComplete.v, +1) cleared_g12345_H11_Z (ThieleMachineComplete.v, +1) cleared_g12345_H56_Z (ThieleMachineComplete.v, +1) schur_step_Z (ThieleMachineComplete.v, +1) and 3 more	def def def def
Theorem E.23: Full 1+AB check, no trap implies PSD	§E	chsh_lassert_lab_g12345_no_trap_implies_npa_psd_qlab (QuantumPartitionPSD_1AB.v) instr_chsh_lassert_lab_g12345 (ThieleMachineComplete.v, +14) PSD9 (QuantumPartitionPSD_1AB.v)	FE, RD def def
Remark E.5: Interior only	§E	argued in the text	
Theorem E.24: The zero-moment check sits inside the classical bound	§E	qlab_check_at_gzero_forces_unit_ball (QuantumPartitionPSD_1AB.v) qlab_check_at_gzero_implies_classical_bound (QuantumPartitionPSD_1AB.v) qlab_residual_g_zero_decomp (QuantumPartitionPSD_1AB.v) column_contractive_check_qlab_sound_at_g_zero (QuantumPartitionPSD_1AB.v) and 2 more	FE, RD FE, RD FE, RD FE, RD
Definition E.11: Completion and membership	§E	completed_npa (ElliptopeCompletion.v) elliptope_realizable (ElliptopeCompletion.v)	def def
Theorem E.25: Deterministic plans are inside, deterministic_strategy_elliptope	§E	deterministic_strategy_elliptope (ElliptopeCompletion.v)	FE, RD

Result	Sec.	Coq name and file	Axioms
Theorem E.26: Convexity and finite mixtures	§E	elliptope_convex (ElliptopeComple tion.v) elliptope_finite_mixture (Ellipto peCompletion.v) lhv_mixture_elliptope (ElliptopeCo mpletion.v) elliptope_zero (ElliptopeCompletion .v)	FE, RD FE, RD FE, RD FE, RD
Theorem E.27: Tsirelson inside the completion set, elliptope_tsir elson	§E	elliptope_tsirelson (ElliptopeComp letion.v) psd_cauchy_schwarz (ElliptopeCompl etion.v)	FE, RD FE, RD
Corollary E.28: The PR box is outside, pr_box_ not_elliptope	§E	pr_box_not_elliptope (ElliptopeCom pletion.v)	FE, RD
Theorem E.29: Strictly beyond the classical bound, beyond_class ical_elliptope	§E	beyond_classical_elliptope (Ellip topeCompletion.v)	FE, RD
Definition E.12: The two branches	§E	elliptope_pd_check (ElliptopeGate. v) elliptope_ldl_check (ElliptopeGate. v) ldl_equations (ElliptopeGate.v) elliptope_check_full (ElliptopeGat e.v)	def def def def
Theorem E.30: Soundness of the gate, e lliptope_check_fu ll_sound	§E	elliptope_check_full_sound (Ellip topeGate.v) elliptope_full_gate_never_accept s_pr_box (ElliptopeGate.v) elliptope_pd_check_sound (Ellipto peGate.v) elliptope_ldl_check_sound (Ellipt opeGate.v) and 3 more	FE, RD FE, RD FE, RD FE, RD

E The CHSH derivation, in full

This appendix supports Section 25. That section tells the story; this one does the algebra. Each item gives the exact statement with all its hypotheses, the steps a reader needs to check it, and the Coq name with file and line. Nothing here identifies any matrix with a quantum state. Where the reading “quantum” appears, it is a cited result from outside the kernel.

E.1 A. Setting and notation

Definition E.1 (Settings, outcomes, correlators). Alice receives a setting $x \in \{0, 1\}$ and Bob a setting $y \in \{0, 1\}$. Each answers a bit, recoded as an outcome $a, b \in \{+1, -1\}$ (bit 0 becomes $+1$, bit 1 becomes -1). The correlator E_{xy} is the average of the product ab over the rounds with settings (x, y) . The CHSH quantity is

$$S = E_{00} + E_{01} + E_{10} - E_{11}.$$

I collect the four correlators into the 2×2 matrix

$$E = \begin{pmatrix} E_{00} & E_{01} \\ E_{10} & E_{11} \end{pmatrix},$$

whose rows are indexed by Alice's setting and whose columns by Bob's. The rows of E are (E_{00}, E_{01}) and (E_{10}, E_{11}) ; the columns are (E_{00}, E_{10}) and (E_{01}, E_{11}) .

The same polynomial S appears under three names in the source: `CHSH` (kernel/quantum/TsirelsonGeneral.v), `CHSH_value` (kernel/quantum/TsirelsonFromAlgebra.v) and `chsh_S` (kernel/quantum/ElliptopeCompletion.v).

Definition E.2 (Counters). The machine keeps eight counters in the record `WitnessCounts` (kernel/foundation/VMState.v): for each setting pair, `samexy` counts rounds whose two answer bits matched and `diffxy` counts rounds whose bits differed. `record_trial` (kernel/foundation/VMStep.v) adds one to exactly one of the eight. Write

$$N_{xy} = \text{same}_{xy} + \text{diff}_{xy}, \quad D_{xy} = \text{same}_{xy} - \text{diff}_{xy}, \quad E_{xy} = \frac{D_{xy}}{N_{xy}}.$$

A matched round contributes $ab = +1$ and a differing round $ab = -1$, so D_{xy} is the sum of the products and D_{xy}/N_{xy} is their average. Since $|D_{xy}| \leq N_{xy}$, every correlator from counters lies in $[-1, 1]$. In Coq the integers are `chsh_n_z` and `chsh_d_z` (kernel/foundation/VMStep.v). When a bucket is empty, the real-valued correlator `state_bucket_correlation` (kernel/nfi/MuLedgerQuantumBridge.v) and the rational one `chsh_correlator_q` (kernel/quantum/CHSHStatisticalBridge.v) both return 0 by convention. The CHSH score of a tally, `chsh_stat_from_wc` (CHSH-StatisticalBridge.v), adds the first three correlators and obtains $-E_{11}$ by swapping the two counters of the last bucket.

E.2 B. The classical bound

Lemma E.1 (Deterministic strategies). *Let $a_0, a_1, b_0, b_1 \in \{+1, -1\}$. Then $-2 \leq a_0b_0 + a_0b_1 + a_1b_0 - a_1b_1 \leq 2$.*

Proof. Factor: $a_0b_0 + a_0b_1 + a_1b_0 - a_1b_1 = a_0(b_0 + b_1) + a_1(b_0 - b_1)$. If $b_0 = b_1$ then

$b_0 + b_1 = \pm 2$ and $b_0 - b_1 = 0$; if $b_0 \neq b_1$ the other way round. In both cases exactly one of the two brackets is ± 2 and the other is 0, so the expression is ± 2 times a sign, which is $+2$ or -2 . There are sixteen sign choices, and every one of them gives exactly ± 2 , never 0 or ± 4 . $\square$

The Coq proofs do not factor; they split into the sixteen sign cases and compute. The rational statement is `local_S_2_deterministic` (`kernel/quantum/BoxCHSH.v`): for $f_A, f_B : \mathbb{N} \rightarrow \mathbb{Q}$ with every value ± 1 , the absolute value of $f_A(0)f_B(0) + f_A(0)f_B(1) + f_A(1)f_B(0) - f_A(1)f_B(1)$ is at most 2. The real statement is `deterministic_strategy_chsh_bounded` (`kernel/quantum/MinorConstraints.v`), with conclusion $-2 \leq \dots \leq 2$.

Definition E.3 (Finite factorizable model, `is_factorizable`). A real-valued correlation function $E(a, b, x, y)$ is factorizable (`kernel/quantum/MinorConstraints.v`) when there exist response functions $A(x, \lambda)$, $B(y, \lambda)$, weights $p(\lambda)$ and a natural number $\lambda_{\max}$ such that: $0 \leq p(\lambda) \leq 1$ for $\lambda \leq \lambda_{\max}$; $\sum_{\lambda=0}^{\lambda_{\max}} p(\lambda) = 1$; every $A(x, \lambda)$ and every $B(y, \lambda)$ is -1 or $+1$; and for all a, b, x, y , $E(a, b, x, y) = \sum_{\lambda=0}^{\lambda_{\max}} p(\lambda) A(x, \lambda) B(y, \lambda)$. The outcome arguments a, b are dummies; only correlations are modelled.

Theorem E.2 (`fine_theorem`, one direction only). If E is factorizable, then $-2 \leq E(0,0,0,0) + E(0,0,0,1) + E(0,0,1,0) - E(0,0,1,1) \leq 2$ (`kernel/quantum/MinorConstraints.v`; restated as `factorizable_CHSH_classical_bound`).

Proof. Substitute the factorization: $S = \sum_{\lambda} p(\lambda) S_{\lambda}$, where S_{λ} is the deterministic expression of Lemma E.1 at the signs $A(\cdot, \lambda), B(\cdot, \lambda)$. Each $S_{\lambda} \in [-2, 2]$, the weights are nonnegative and sum to 1, so $-2 = \sum_{\lambda} p(\lambda)(-2) \leq S \leq \sum_{\lambda} p(\lambda) \cdot 2 = 2$. $\square$

Remark E.1 (What the name does and does not cover). Fine's 1982 theorem is an equivalence between several descriptions of local models and the Bell inequalities. `fine_theorem` proves one direction: a finite factorizable model obeys the bound. It proves no converse, and it covers only finitely many hidden states ($\lambda \leq \lambda_{\max}$), not an arbitrary probability space. The transfer to rational boxes is `local_box_CHSH_bound` (`kernel/quantum/MinorConstraints.v`): if `box_correlations B` is factorizable then the real image of $|S(B)|$ is at most 2.

The finite-tally statement concerns one fixed strategy replayed on every recorded round.

Definition E.4 (`WCLocallyConsistent`). For bits $a_0, a_1, b_0, b_1 \in \{0, 1\}$ and a counter record w (`kernel/quantum/CHSHStatisticalBridge.v`), w is locally consistent with

(a_0, a_1, b_0, b_1) when, for each setting pair (x, y) : if $a_x = b_y$ then $\text{diff}_{xy} = 0$, otherwise $\text{same}_{xy} = 0$; and every bucket is sampled, $N_{00}, N_{01}, N_{10}, N_{11} > 0$.

Theorem E.3 (A tally above 2 is not one replayed strategy). *Let w be a counter record whose rational score `chsh_stat_from_wc` w is greater than 2. Then for all bits a_0, a_1, b_0, b_1 , w is not locally consistent with (a_0, a_1, b_0, b_1) . This is `chsh_violation_rules_out_locally_factorizable_coupling` (`kernel/quantum/CHSHCouplingBridge.v`). Its contrapositive form, `locally_consistent_classical_bound`, states that a locally consistent record has score of absolute value at most 2.*

Proof. Under local consistency each sampled bucket is pure: if $a_x = b_y$ every round in the bucket matched, so $E_{xy} = N_{xy}/N_{xy} = +1$; otherwise every round differed and $E_{xy} = -1$. In signs, $E_{xy} = \alpha_x \beta_y$ with $\alpha_x = (-1)^{a_x}$ and $\beta_y = (-1)^{b_y}$. The score is then the deterministic expression of Lemma E.1, which is at most 2 in absolute value. A score above 2 contradicts this. $\square$

The theorem rules out one explanation, a single four-bit plan used on every round. It says nothing about a local model that redraws its hidden state every round; Section 25 gives a four-round coin-flip tally with score 4.

E.3 C. Tsirelson's bound from algebra

Lemma E.4 (The six-square identity, `chsh_gap_is_sum_of_squares`). *For all real a, b, c, d ,*

$$\begin{aligned} & 4(a^2 + b^2 + c^2 + d^2) - (a + b + c - d)^2 \\ &= (a - b)^2 + (a - c)^2 + (a + d)^2 + (b - c)^2 + (b + d)^2 + (c + d)^2. \end{aligned}$$

(`kernel/quantum/TsirelsonFromAlgebra.v`, closed by `ring`.)

Proof. Expand the square on the left: $(a + b + c - d)^2 = a^2 + b^2 + c^2 + d^2 + 2ab + 2ac - 2ad + 2bc - 2bd - 2cd$. So the left side is

$$3(a^2 + b^2 + c^2 + d^2) - 2ab - 2ac + 2ad - 2bc + 2bd + 2cd.$$

On the right, each variable appears in exactly three of the six brackets (a in $a - b, a - c, a + d$; b in $a - b, b - c, b + d$; c in $a - c, b - c, c + d$; d in $a + d, b + d, c + d$), so the squared terms add to $3(a^2 + b^2 + c^2 + d^2)$. The cross terms are $-2ab$ from $(a - b)^2$, $-2ac$ from $(a - c)^2$, $+2ad$ from $(a + d)^2$, $-2bc$ from $(b - c)^2$, $+2bd$ from $(b + d)^2$ and $+2cd$ from $(c + d)^2$. Both sides agree term by term. $\square$

Since the right side is a sum of squares, $S^2 \leq 4 \sum_{xy} E_{xy}^2$ for any four reals; this is `cauchy_schwarz_chsh` (`TsirelsonFromAlgebra.v`).

Theorem E.5 (Row-bound form of Tsirelson’s bound). *For all real $E_{00}, E_{01}, E_{10}, E_{11}$ with $E_{00}^2 + E_{01}^2 \leq 1$ and $E_{10}^2 + E_{11}^2 \leq 1$, $S^2 \leq 8$. This is `tsirelson_squared` (`kernel/quantum/TsirelsonFromAlgebra.v`) and `tsirelson_from_row_bounds` (`kernel/quantum/TsirelsonGeneral.v`). The absolute form $|S| \leq \sqrt{8} = 2\sqrt{2}$ is `tsirelson_bound_abs` (`TsirelsonGeneral.v`).*

Proof. Adding the two row bounds gives $\sum_{xy} E_{xy}^2 \leq 2$. By Lemma E.4, $S^2 \leq 4 \sum_{xy} E_{xy}^2 \leq 8$. $\square$

Theorem E.6 (Tightness). *The point $E_{00} = E_{01} = E_{10} = 1/\sqrt{2}$, $E_{11} = -1/\sqrt{2}$ satisfies both row bounds and has $S = \sqrt{8}$. This is `tsirelson_tight` (`TsirelsonFromAlgebra.v`) and `tsirelson_achievable` (`TsirelsonGeneral.v`).*

Proof. Each row has squared length $1/2 + 1/2 = 1$, and $S = 4/\sqrt{2} = 2\sqrt{2} = \sqrt{8}$. $\square$

Remark E.2 (Where equality can happen). This step is argued here, not checked in Coq. Equality $S^2 = 8$ under the row bounds forces equality in both steps of the proof above: the six squares must all vanish, which gives $a = b = c = -d$, and the rows must be tight, which gives $2a^2 = 1$. So the point of Theorem E.6 and its negative are the only places the bound is reached.

Lemma E.7 (A rational constant above the bound, `rational_tsirelson_bound`). $\sqrt{8} < 5657/2000$ (`kernel/quantum/TsirelsonFromAlgebra.v`).

Proof. Both numbers are positive, so it suffices to compare squares. $5657^2 = 32,001,649$ and $8 \cdot 2000^2 = 32,000,000$, so $(5657/2000)^2 = 32,001,649/4,000,000 > 8 = (\sqrt{8})^2$. If $\sqrt{8} \geq 5657/2000$ held, squaring both nonnegative sides would give $8 \geq (5657/2000)^2$, a contradiction. $\square$

The constant is the four-decimal round-up of $2\sqrt{2} = 2.8284271\dots$; the gap is about 7.3×10^{-5} . It lets rational code test “above the Tsirelson bound” without a square root. It plays no part in the integer check of item E.

E.4 D. The zero-marginal 5x5 moment matrix

Definition E.5 (The pinned level-1 matrix). `zero_marginal_npa` (`kernel/quantum/NPAMomentMatrix.v`) fills the general record `NPAMomentMatrix` with all four one-party entries and both within-party entries set to zero, and `npa_to_matrix` lays it out on rows and columns I, A_0, A_1, B_0, B_1 :

$$M(E) = \begin{array}{c} I \\ A_0 \\ A_1 \\ B_0 \\ B_1 \end{array} \begin{array}{ccccc} & I & A_0 & A_1 & B_0 & B_1 \\ \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & E_{00} & E_{01} \\ 0 & 0 & 1 & E_{10} & E_{11} \\ 0 & E_{00} & E_{10} & 1 & 0 \\ 0 & E_{01} & E_{11} & 0 & 1 \end{pmatrix} \end{array}$$

Positive semidefiniteness in the source is defined by quadratic forms, not eigenvalues: `PSD5` (`kernel/quantum/ConstructivePSD.v`) says $v^\top M v \geq 0$ for every $v \in \mathbb{R}^5$, and `n pa_psd` (`NPAMomentMatrix.v`) is “symmetric and `PSD5`”. In block form,

$$M(E) = \begin{pmatrix} 1 & 0 \\ 0 & G \end{pmatrix}, \quad G = \begin{pmatrix} I_2 & E \\ E^\top & I_2 \end{pmatrix}.$$

The corner 1 is decoupled, so $M(E)$ is PSD exactly when G is.

Lemma E.8 (Schur complement, the standard form). *Let P be symmetric positive definite. The symmetric block matrix $\begin{pmatrix} P & Q \\ Q^\top & R \end{pmatrix}$ is PSD if and only if $R - Q^\top P^{-1} Q$ is PSD.*

Proof. For u, w of the right sizes,

$$\begin{pmatrix} u \\ w \end{pmatrix}^\top \begin{pmatrix} P & Q \\ Q^\top & R \end{pmatrix} \begin{pmatrix} u \\ w \end{pmatrix} = (u + P^{-1} Q w)^\top P (u + P^{-1} Q w) + w^\top (R - Q^\top P^{-1} Q) w.$$

The first term is never negative because P is positive definite, and it is zero at $u = -P^{-1} Q w$. So the form is nonnegative for all (u, w) exactly when the second term is nonnegative for all w . $\square$

The Coq proofs do not invoke this lemma abstractly. They carry out its one instance with $P = I_2$, $Q = E$, $R = I_2$, where the complement is

$$I_2 - E^\top E = \begin{pmatrix} 1 - E_{00}^2 - E_{10}^2 & -(E_{00}E_{01} + E_{10}E_{11}) \\ -(E_{00}E_{01} + E_{10}E_{11}) & 1 - E_{01}^2 - E_{11}^2 \end{pmatrix}.$$

The diagonal of $E^\top E$ holds the squared lengths of the columns of E , which is why the conditions below are column conditions.

Definition E.6 (Column contractive, `zero_marginal_column_contractive`). (`kernel/nfi/MuLedgerQuantumBridge.v`) Write $p = 1 - E_{00}^2 - E_{10}^2$, $q = 1 - E_{01}^2 - E_{11}^2$

and $s = E_{00}E_{01} + E_{10}E_{11}$. The correlators are column contractive when

$$p \geq 0, \quad q \geq 0, \quad pq - s^2 \geq 0.$$

Lemma E.9 (The 2x2 test, `psd2_quadratic_form_nonneg`). (*kernel/nfi/MuLedgerQuantumBridge.v*) If $\alpha \geq 0$, $\delta \geq 0$ and $\alpha\delta - \beta^2 \geq 0$, then $\alpha u^2 + 2\beta uw + \delta w^2 \geq 0$ for all real u, w .

Proof. If $w = 0$ the form is $\alpha u^2 \geq 0$. Otherwise put $t = u/w$, so the form is $w^2(\alpha t^2 + 2\beta t + \delta)$. If $\alpha = 0$, then $-\beta^2 \geq 0$ forces $\beta = 0$ and the bracket is $\delta \geq 0$. If $\alpha > 0$, $\alpha t^2 + 2\beta t + \delta = \alpha(t + \beta/\alpha)^2 + (\alpha\delta - \beta^2)/\alpha$, a sum of two nonnegative terms. $\square$

Theorem E.10 (Contractive implies PSD). If $p \geq 0$, $q \geq 0$ and $pq - s^2 \geq 0$, then `PSD5` holds of $M(E)$. This is `zero_marginal_npa_column_contractive_implies_psd` (*kernel/nfi/MuLedgerQuantumBridge.v*).

Proof. Write $v = (v_0, x_0, x_1, y_0, y_1)$. Reading off the entries of $M(E)$,

$$v^\top M(E) v = v_0^2 + x_0^2 + x_1^2 + y_0^2 + y_1^2 + 2x_0(E_{00}y_0 + E_{01}y_1) + 2x_1(E_{10}y_0 + E_{11}y_1).$$

Complete the squares in x_0 and x_1 :

$$\begin{aligned} v^\top M(E) v &= v_0^2 + (x_0 + E_{00}y_0 + E_{01}y_1)^2 + (x_1 + E_{10}y_0 + E_{11}y_1)^2 \\ &\quad + p y_0^2 - 2s y_0 y_1 + q y_1^2. \end{aligned}$$

To check the last three terms, expand the two squares: they contribute $(E_{00}y_0 + E_{01}y_1)^2 + (E_{10}y_0 + E_{11}y_1)^2$ beyond the cross terms, which is $(E_{00}^2 + E_{10}^2)y_0^2 + 2s y_0 y_1 + (E_{01}^2 + E_{11}^2)y_1^2$; subtracting it from $y_0^2 + y_1^2$ leaves $p y_0^2 - 2s y_0 y_1 + q y_1^2$. The first three terms are squares. The last is the 2×2 form with $\alpha = p$, $\beta = -s$, $\delta = q$, nonnegative by Lemma E.9. $\square$

Lemma E.11 (Discriminant, `quadratic_nonneg_discriminant`). (*kernel/quantum/ConstructivePSD.v*) If $a + 2bt + ct^2 \geq 0$ for every real t , then $b^2 \leq ac$.

Proof. If $c < 0$ the quadratic tends to $-\infty$, so $c \geq 0$. If $c = 0$, the line $a + 2bt$ is nonnegative for every t only if $b = 0$, and then $b^2 = 0 = ac$. If $c > 0$, evaluate at $t = -b/c$: $a - b^2/c \geq 0$, so $b^2 \leq ac$. (The Coq proof is the same case analysis, with explicit witnesses t for the contradictions.) $\square$

Theorem E.12 (PSD implies contractive). If `PSD5` holds of $M(E)$, then $p \geq 0$, $q \geq 0$ and $pq - s^2 \geq 0$. This is `npa_psd_implies_column_contractive` (*kernel/quantum/QuantumPartitionPSD.v*).

Proof. Feed the quadratic form three test vectors, written in the order (I, A_0, A_1, B_0, B_1) ; their values are the lemmas `npa_quad5_test_col0`, `npa_quad5_test_col1` and `npa_quad5_test_schur` (`QuantumPartitionPSD.v`). Each is the choice $x = -Ey$ that zeroes both completed squares in the proof of Theorem E.10, so only $y^T(I_2 - E^T E)y$ is left.

First, $v = (0, -E_{00}, -E_{10}, 1, 0)$ gives $v^T M v = p$, so $p \geq 0$.

Second, $v = (0, -E_{01}, -E_{11}, 0, 1)$ gives $v^T M v = q$, so $q \geq 0$.

Third, for each real t , $v(t) = (0, -(E_{00}t + E_{01}), -(E_{10}t + E_{11}), t, 1)$ gives $v(t)^T M v(t) = p t^2 - 2s t + q \geq 0$. Apply Lemma E.11 with $a = q$, $b = -s$, $c = p$: $s^2 \leq pq$. $\square$

Corollary E.13 (The equivalence). *For all real $E_{00}, E_{01}, E_{10}, E_{11}$, PSD5 of $M(E)$ holds if and only if the correlators are column contractive: `npa_psd_iff_column_contractive` (`kernel/quantum/QuantumPartitionPSD.v`). With symmetry packaged in, `column_contractive_iff_npa_psd` states contractivity if and only if `npa_psd` holds of `zero_marginal_npa` at the four correlators.*

The bound itself goes through rows rather than columns.

Theorem E.14 (PSD implies Tsirelson). *If `npa_psd` holds of `zero_marginal_npa` at $E_{00}, E_{01}, E_{10}, E_{11}$, then $S^2 \leq 8$, and $|S| \leq \sqrt{8}$. These are `npa_psd_implies_tsirelson_bound` and `npa_psd_implies_tsirelson_bound_abs` (`kernel/nfi/MuLedgerQuantumBridge.v`).*

Proof. `psd_3x3_determinant_nonneg` (`kernel/quantum/ConstructivePSD.v`) says a symmetric PSD matrix with unit diagonal has every 3×3 principal minor $1 - x^2 - y^2 - z^2 + 2xyz \geq 0$. On rows A_0, B_0, B_1 the entries are $x = E_{00}$, $y = E_{01}$, $z = 0$, so $E_{00}^2 + E_{01}^2 \leq 1$. On rows A_1, B_0, B_1 the same gives $E_{10}^2 + E_{11}^2 \leq 1$. This is `npa_psd_zero_marginal_implies_row_bounds` (`MuLedgerQuantumBridge.v`). Then Theorem E.5 applies, through `tsirelson_from_minors` (`kernel/quantum/TsirelsonGeneral.v`). $\square$

Here are the three conditions and the score at a few points, all by direct arithmetic. In this table, $a = 3/5$, $b = 4/5$ and $t = 1/\sqrt{2}$.

point ($E_{00}, E_{01}, E_{10}, E_{11}$)	S	p	q	$pq - s^2$	contractive
(0,0,0,0)	0	1	1	1	yes
(0.6,0.6,0.6,-0.6)	2.4	0.28	0.28	0.0784	yes
($a, b, b, -a$)	2.8	0	0	0	yes, with no slack

point ($E_{00}, E_{01}, E_{10}, E_{11}$)	S	p	q	$pq - s^2$	contractive
$(t, t, t, -t)$	$2\sqrt{2}$	0	0	0	yes, with no slack
$(1, 1, 1, 1)$, all-ones plan	2	-1	-1		no
$(1, 0, 1, 0)$, a classical mixture	2	-1	1		no
$(1, 1, 1, -1)$, PR box	4	-1	-1		no

Every deterministic plan has all four correlators equal to ± 1 , so $p = 1 - 1 - 1 = -1$ and every one of the sixteen fails.

Remark E.3 (What PSD of the pinned matrix means). $M(E)$ is the level-1 moment matrix with the two within-party entries $\langle A_0 A_1 \rangle$ and $\langle B_0 B_1 \rangle$ pinned to zero. Its PSD is sufficient for the correlators to admit some PSD completion (item G, `zero_marginal_implies_elliptope`), and it is not necessary: the all-ones plan has a completion but fails here. So a check built on $M(E)$ is sound for the completion set and not complete for it, and it rejects some honest classical tallies. That correlators with a PSD completion are exactly those reachable by quantum measurements is Tsirelson's theorem, cited and not proved here (item H).

E.5 E. The integer check

Definition E.7 (`column_contractive_check_witness`). (`kernel/foundation/VMStep.v`) From the counters of Definition E.2, with all arithmetic in $\mathbb{Z}$, let

$$\begin{aligned} A &= N_{00}^2 N_{10}^2 - D_{00}^2 N_{10}^2 - D_{10}^2 N_{00}^2, \\ B &= N_{01}^2 N_{11}^2 - D_{01}^2 N_{11}^2 - D_{11}^2 N_{01}^2, \\ C &= D_{00} D_{01} N_{10} N_{11} + D_{10} D_{11} N_{00} N_{01}. \end{aligned}$$

The check is the conjunction of seven facts, in this order: $0 < N_{00}$, $0 < N_{01}$, $0 < N_{10}$, $0 < N_{11}$, $0 \leq A$, $0 \leq B$, and $C \cdot C \leq A \cdot B$.

There is no division, no real number, no square root and no S in it.

Lemma E.15 (The seven facts are the three conditions with denominators cleared). Assume $N_{00}, N_{01}, N_{10}, N_{11} > 0$ and put $E_{xy} = D_{xy} / N_{xy}$. Then, with p, q, s as in Definition E.6,

$$\begin{aligned} p &= \frac{A}{N_{00}^2 N_{10}^2}, & q &= \frac{B}{N_{01}^2 N_{11}^2}, & s &= \frac{C}{N_{00} N_{01} N_{10} N_{11}}, \\ pq - s^2 &= \frac{AB - C^2}{(N_{00} N_{01} N_{10} N_{11})^2}. \end{aligned}$$

Because every denominator is positive, $0 \leq A \iff p \geq 0$, $0 \leq B \iff q \geq 0$ and $C^2 \leq AB \iff s^2 \leq pq$.

Proof. For p : $1 - D_{00}^2/N_{00}^2 - D_{10}^2/N_{10}^2$; multiply by $N_{00}^2 N_{10}^2$ to get $N_{00}^2 N_{10}^2 - D_{00}^2 N_{10}^2 - D_{10}^2 N_{00}^2 = A$. For q the same with columns 01 and 11, multiplied by $N_{01}^2 N_{11}^2$. For s :

$$\frac{D_{00}}{N_{00}} \frac{D_{01}}{N_{01}} + \frac{D_{10}}{N_{10}} \frac{D_{11}}{N_{11}} = \frac{D_{00}D_{01}N_{10}N_{11} + D_{10}D_{11}N_{00}N_{01}}{N_{00}N_{01}N_{10}N_{11}} = \frac{C}{N_{00}N_{01}N_{10}N_{11}}.$$

Then $pq = AB/(N_{00}^2 N_{10}^2 N_{01}^2 N_{11}^2) = AB/(N_{00}N_{01}N_{10}N_{11})^2$ and $s^2 = C^2/(N_{00}N_{01}N_{10}N_{11})^2$; subtract. Multiplying an inequality by a positive number preserves it in both directions. $\square$

The Coq source states the direction it needs:

Theorem E.16 (Soundness of the integer check). *If `column_contractive_check_witness w = true`, then the four correlators `state_bucket_correlation samexy diffxy` of w are column contractive. This is `column_contractive_check_witness_sound (kernel/hfi/MuLedgerQuantumBridge.v)`, lifted to a machine state as `state_column_contractive_check_witness_sound`.*

Proof. The first four conjuncts make every N_{xy} positive, so the empty-bucket convention never applies and each correlator is D_{xy}/N_{xy} . The integer facts lift to the reals by the injection $\mathbb{Z} \rightarrow \mathbb{R}$, which preserves order. Lemma E.15 turns them into the three conditions. $\square$

The instruction is defined by two step rules. `step_chsh_lassert_ok` (`kernel/foundation/VMStep.v`): when the check is true, the program counter advances by one and every other field is unchanged except the cost counter. `step_chsh_lassert_bad`: when it is false, the program counter goes to `LASSERT_TRAP_PC = 3840`, the error flag is set, and the error register is set to 1. Both branches charge the declared cost plus one (`S mu_delta`).

Theorem E.17 (No trap implies PSD, `chsh_lassert_no_trap_implies_npa_psd`). (`kernel/quantum/QuantumPartitionPSD.v`) *Let s be a state with error flag false, and let s' be the result of applying `instr_chsh_lassert d` to s , where d is the declared cost. If the error flag of s' equals that of s , then `npa_psd` holds of `state_zero_marginal_npa s`: the pinned matrix $M(E)$ built from the correlators of s 's counters is symmetric and PSD.*

Proof. If the check had failed, the step would set the error flag to true, contradicting “error flag unchanged and false before”. So the check passed. Theorem E.16 gives column contractivity, and Corollary E.13 gives PSD. The intermediate form is `chsh_lassert_check_implies_npa_psd (QuantumPartitionPSD.v)`. $\square$

Corollary E.18 (No trap implies the Tsirelson bound). *Under the hypotheses of Theorem E.17, the correlators of s 's counters satisfy $S^2 \leq 8$. There is no single named theorem for this; it is the composition of Theorem E.17 with Theorem E.14 (the matrix `state_zero_marginal_npas` unfolds to `zero_marginal_npa` of the four state correlators). The same conclusion is also reached through `chsh_lassert_no_trap_implies_state_column_contractive` (`kernel/nfi/MuLedgerQuantumBridge.v`) and `state_column_contractive_implies_tsirelson`.*

A worked tally. Take $(\text{same}, \text{diff}) = (4, 1), (4, 1), (4, 1), (1, 4)$ for the pairs 00, 01, 10, 11. Then $N_{xy} = 5$ for all four, $D_{00} = D_{01} = D_{10} = 3$, $D_{11} = -3$, and the correlators are 0.6, 0.6, 0.6, -0.6 with $S = 2.4$. Compute:

$$\begin{aligned} A &= 25 \cdot 25 - 9 \cdot 25 - 9 \cdot 25 = 625 - 225 - 225 = 175, \\ B &= 25 \cdot 25 - 9 \cdot 25 - 9 \cdot 25 = 175, \\ C &= 3 \cdot 3 \cdot 5 \cdot 5 + 3 \cdot (-3) \cdot 5 \cdot 5 = 225 - 225 = 0. \end{aligned}$$

All seven facts hold: four positive N , $0 \leq 175$ twice, and $0 \leq 175 \cdot 175$. Dividing back, $p = 175/625 = 0.28$ and $s = 0$, the second row of the table in item D.

Two more, for contrast. One round per pair from the all-ones plan, $(1, 0)$ four times, gives $N = D = 1$ everywhere and $A = 1 - 1 - 1 = -1 < 0$: trap. The tally $(4, 1), (9, 1), (9, 1), (1, 4)$ gives correlators $3/5, 4/5, 4/5, -3/5$, with $N = 5, 10, 10, 5$ and $D = 3, 8, 8, -3$: $A = 25 \cdot 100 - 9 \cdot 100 - 64 \cdot 25 = 0$, $B = 100 \cdot 25 - 64 \cdot 25 - 9 \cdot 100 = 0$ and $C = 3 \cdot 8 \cdot 10 \cdot 5 + 8 \cdot (-3) \cdot 5 \cdot 10 = 0$. It passes with no slack, at $S = 2.8$.

Remark E.4 (The circuit). The hardware check is proved equal to this definition. `chsh_check_nat_correct` (`kami_hw/ChshArith.v`) equates a natural-number form with `column_contractive_check_witness`; `chsh_check_word_correct` (`ChshArith.v`) and `chsh_check_word_spec` equate the word-level circuit with it for every choice of eight 32-bit counters. The intermediate products are carried at 134 bits and the product $A \cdot B$ at 268 bits (`ChshArith.v`). `chsh_run_result` (`kami_hw/ChshRun.v`) composes the 29 firings of the check, and `chsh_lassert_refines` (`kami_hw/ChshRetire.v`) is the refinement statement. Section 27 tells that part.

E.6 F. Level 1+AB

The source indexes Alice's observables A_1, A_2 and Bob's B_1, B_2 ; I keep the book's A_0, A_1, B_0, B_1 , so the Coq e_{11} is E_{00} here, and so on. The five extra parameters are named in the record `Q1ABMoments` (`kernel/quantum/QuantumPartitionPSD_1AB.v`): γ_1 for $\langle A_0 A_1 B_0 \rangle$, γ_2 for $\langle A_0 A_1 B_1 \rangle$, γ_3 for $\langle A_0 B_0 B_1 \rangle$, γ_4 for $\langle A_1 B_0 B_1 \rangle$, γ_5 for $\langle A_0 A_1 B_0 B_1 \rangle$. These are labels only; the file builds no operators.

Definition E.8 (`qlab_moment_matrix`). (`kernel/quantum/QuantumPartitionPSD_`

1AB.v) The 9×9 real matrix on rows and columns $I, A_0, A_1, B_0, B_1, A_0B_0, A_0B_1, A_1B_0, A_1B_1$, with the entries exactly as the Coq definition places them:

	I	A_0	A_1	B_0	B_1	A_0B_0	A_0B_1	A_1B_0	A_1B_1
I	1	0	0	0	0	E_{00}	E_{01}	E_{10}	E_{11}
A_0	0	1	0	E_{00}	E_{01}	0	0	γ_1	γ_2
A_1	0	0	1	E_{10}	E_{11}	γ_1	γ_2	0	0
B_0	0	E_{00}	E_{10}	1	0	0	γ_3	0	γ_4
B_1	0	E_{01}	E_{11}	0	1	γ_3	0	γ_4	0
A_0B_0	E_{00}	0	γ_1	0	γ_3	1	0	0	γ_5
A_0B_1	E_{01}	0	γ_2	γ_3	0	0	1	$-\gamma_5$	0
A_1B_0	E_{10}	γ_1	0	0	γ_4	0	$-\gamma_5$	1	0
A_1B_1	E_{11}	γ_2	0	γ_4	0	γ_5	0	0	1

Two conventions are fixed by this layout. First, the four-body cells carry opposite signs: $+\gamma_5$ at (A_0B_0, A_1B_1) and $-\gamma_5$ at (A_0B_1, A_1B_0) . The file states this is a convention of the real matrix, not derived from operators, and guards it with `qlab_four_body_cells_are_conjugate` (`QuantumPartitionPSD_1AB.v`). Second, the matrix keeps the level-1 pinning: every cell whose label reduces to a marginal or to $\langle A_0A_1 \rangle$ or $\langle B_0B_1 \rangle$ is zero, for instance (A_0, A_0B_0) , (A_0B_0, A_0B_1) and (A_0B_0, A_1B_0) . Symmetry is `qlab_moment_matrix_symmetric`.

Write a vector as $(v_I, v_{A_0}, v_{A_1}, v_{B_0}, v_{B_1}, v_{00}, v_{01}, v_{10}, v_{11})$, where v_{xy} is the coordinate of A_xB_y . Put

$$L_1 = E_{00}v_{00} + E_{01}v_{01} + E_{10}v_{10} + E_{11}v_{11},$$

$$L_2 = E_{00}v_{B_0} + E_{01}v_{B_1} + \gamma_1v_{10} + \gamma_2v_{11},$$

$$L_3 = E_{10}v_{B_0} + E_{11}v_{B_1} + \gamma_1v_{00} + \gamma_2v_{01}.$$

Definition E.9 (The residual, `qlab_residual`). (`QuantumPartitionPSD_1AB.v`)

$$\begin{aligned} R = & v_{B_0}^2 + v_{B_1}^2 + v_{00}^2 + v_{01}^2 + v_{10}^2 + v_{11}^2 \\ & + 2v_{B_0}(\gamma_3v_{01} + \gamma_4v_{11}) + 2v_{B_1}(\gamma_3v_{00} + \gamma_4v_{10}) \\ & + 2\gamma_5(v_{00}v_{11} - v_{01}v_{10}) - L_1^2 - L_2^2 - L_3^2. \end{aligned}$$

The predicate `column_contractive_qlab` says $R \geq 0$ for all six real inputs.

Lemma E.19 (Three squares plus the residual, `quad9_qlab_sos_decomposition`). (`QuantumPartitionPSD_1AB.v`, closed by `ring`.) For every vector,

$$v^T M_9 v = (v_I + L_1)^2 + (v_{A_0} + L_2)^2 + (v_{A_1} + L_3)^2 + R.$$

Proof. The coordinates v_I, v_{A_0}, v_{A_1} appear in the form only through their diagonal squares and the cross terms $2v_I L_1, 2v_{A_0} L_2, 2v_{A_1} L_3$, read off rows I, A_0, A_1 of the matrix. Completing each square adds $L_1^2 + L_2^2 + L_3^2$, which the residual subtracts. What is left in R is the diagonal of the remaining six coordinates and the cross terms among them, read off the lower right 6×6 block. $\square$

Theorem E.20 (PSD at level 1+AB, `qlab_psd_iff_column_contractive`). (*QuantumPartitionPSD_1AB.v*) For all real E_{xy} and $\gamma_1, \dots, \gamma_5$, PSD9 of the matrix of Definition E.8 holds if and only if `column_contractive_qlab` holds. The directions are `column_contractive_qlab_implies_psd9` and `psd9_implies_column_contractive_qlab`.

Proof. If $R \geq 0$ everywhere, Lemma E.19 writes the form as three squares plus R . Conversely, given the six residual inputs, choose $v_I = -L_1, v_{A_0} = -L_2, v_{A_1} = -L_3$; the three squares vanish and PSD gives $R \geq 0$. This is the level-1 argument of item D one size up. $\square$

As a quadratic form in $(v_{B_0}, v_{B_1}, v_{00}, v_{01}, v_{10}, v_{11})$, numbered 1 to 6, the residual has the symmetric coefficient matrix H with these upper entries. They are the `q12345_H11` to `q12345_H66` definitions (*QuantumPartitionPSD_1AB.v* onward), and `q12345_sym6_qf_equals_residual` proves the form equals R . Each entry can be read back off R by collecting the coefficient of $v_i v_j$.

$$\begin{aligned}
H_{11} &= 1 - E_{00}^2 - E_{10}^2, & H_{22} &= 1 - E_{01}^2 - E_{11}^2, \\
H_{12} &= -(E_{00}E_{01} + E_{10}E_{11}), & H_{13} &= -E_{10}\gamma_1, \\
H_{14} &= \gamma_3 - E_{10}\gamma_2, & H_{15} &= -E_{00}\gamma_1, \\
H_{16} &= \gamma_4 - E_{00}\gamma_2, & H_{23} &= \gamma_3 - E_{11}\gamma_1, \\
H_{24} &= -E_{11}\gamma_2, & H_{25} &= \gamma_4 - E_{01}\gamma_1, \\
H_{26} &= -E_{01}\gamma_2, & H_{33} &= 1 - E_{00}^2 - \gamma_1^2, \\
H_{34} &= -(E_{00}E_{01} + \gamma_1\gamma_2), & H_{35} &= -E_{00}E_{10}, \\
H_{36} &= \gamma_5 - E_{00}E_{11}, & H_{44} &= 1 - E_{01}^2 - \gamma_2^2, \\
H_{45} &= -\gamma_5 - E_{01}E_{10}, & H_{46} &= -E_{01}E_{11}, \\
H_{55} &= 1 - E_{10}^2 - \gamma_1^2, & H_{56} &= -(E_{10}E_{11} + \gamma_1\gamma_2), \\
H_{66} &= 1 - E_{11}^2 - \gamma_2^2.
\end{aligned}$$

The top left 2×2 block is $I_2 - E^T E$ from item D.

The full check does not decide $R \geq 0$ directly. It proves the stronger fact that H is positive definite, by two Schur steps and a Sylvester test, all fraction-free.

Lemma E.21 (Fraction-free Schur step). *For a symmetric $n \times n$ matrix h and any vector v , with $S_{ij} = h_{11}h_{ij} - h_{1i}h_{1j}$ for $i, j \geq 2$ (here S is the Schur matrix, not the score),*

$$h_{11} v^T h v = \left(\sum_j h_{1j} v_j \right)^2 + \sum_{i,j \geq 2} S_{ij} v_i v_j.$$

So if $h_{11} > 0$ and the $(n-1) \times (n-1)$ matrix S gives a nonnegative form, then h does too. The instances used are `sym6_Schur_identity` (`QuantumPartitionPSD_1AB.v`) and `sym5_Schur_identity`, with the conclusions `sym6_qf_nonneg_from_pd` and `sym5_qf_nonneg_from_pd`.

Proof. Expand the square: $h_{11}^2 v_1^2 + 2h_{11}v_1 \sum_{j \geq 2} h_{1j}v_j + (\sum_{j \geq 2} h_{1j}v_j)^2$. The left side is $h_{11}^2 v_1^2 + 2h_{11}v_1 \sum_{j \geq 2} h_{1j}v_j + h_{11} \sum_{i,j \geq 2} h_{ij}v_i v_j$. The difference is $\sum_{i,j \geq 2} (h_{11}h_{ij} - h_{1i}h_{1j})v_i v_j$. Dividing by $h_{11} > 0$ keeps the sign. $\square$

Lemma E.22 (Sylvester for 4x4, `sym4_qf_nonneg_from_pd`). (`QuantumPartitionPSD_1AB.v`) *Let h be symmetric 4×4 with leading principal minors d_1, d_2, d_3, d_4 (`sym4_d1` to `sym4_d4`). If all four are strictly positive, then $v^T h v \geq 0$ for all v .*

Proof. `sym4_LDLT_identity` (closed by ring) is the fraction-free factorization

$$d_1^2 d_2 d_3 v^T h v = d_1 d_2 d_3 P_1^2 + d_1 d_3 Q_1^2 + Q_2^2 + d_1^2 d_2 d_4 v_4^2,$$

where $P_1 = \sum_j h_{1j}v_j$, $Q_1 = (h_{11}h_{22} - h_{12}^2)v_2 + (h_{11}h_{23} - h_{12}h_{13})v_3 + (h_{11}h_{24} - h_{12}h_{14})v_4$, and Q_2 is the third linear form (`sym4_Q2`). With every $d_k > 0$ the right side is nonnegative and the factor on the left is positive. $\square$

The cascade for the full check runs on H . Step one: $H_{11} > 0$ and the 5×5 matrix $S_{ij}^{(6)} = H_{11}H_{ij} - H_{1i}H_{1j}$, $2 \leq i, j \leq 6$, gives a nonnegative form. Step two: $S_{22}^{(6)} > 0$ and the 4×4 matrix $S_{ij}^{(5)} = S_{22}^{(6)}S_{ij}^{(6)} - S_{2i}^{(6)}S_{2j}^{(6)}$, $3 \leq i, j \leq 6$, gives a nonnegative form. Step three: the four leading minors of $S^{(5)}$ are positive. The real predicates are `sym5_pd_interior`, `sym6_pd_interior` and the witness `qlab_g12345_minors_witness`, which implies `column_contractive_qlab` and then `PSD9` (`qlab_g12345_minors_witness_implies_psd9`).

Definition E.10 (What the integer check computes). The supplied moments arrive as bucket pairs: $\gamma_k = N_{g_k}/D_{g_k}$ with $N_{g_k} = \text{same}_{g_k} - \text{diff}_{g_k}$ and $D_{g_k} = \text{same}_{g_k} + \text{diff}_{g_k}$. Watch the letters: for these five buckets the source swaps them, so here N is the difference and D the count, the reverse of item A. I keep the source's letters so the names below match the files. Let $K = (N_{00}N_{01}N_{10}N_{11}D_{g_1}D_{g_2}D_{g_3}D_{g_4}D_{g_5})^2$ (`g12345_COMMON_Z`, `kernel/foundation/VMStep.v`). The kernel computes the 21 integers KH_{ij} (`cleared_g12345_H11_Z` to `cleared_g12345_H56_Z`), the 15 integers of $S^{(6)}$ at scale K^2 and the 10 integers of $S^{(5)}$ at scale K^4 , each by `schur_step_Z`, $h_{11}h_{ij} - h_{1i}h_{1j}$

on integers. The decider `qlab_g12345_check_z_kernel` is the conjunction of: $N_{xy} > 0$ for the four pairs; $D_{g_k} > 0$ for $k = 1..5$; $-D_{g_k} < N_{g_k} < D_{g_k}$ for $k = 1..5$; and six strict positivities, namely $KH_{11} > 0$, $K^2S_{22}^{(6)} > 0$, and the four leading minors of the integer $S^{(5)}$. The instruction's check `qlab_g12345_full_integer_check_kernel` is `column_contractive_check_witness` of item E and this decider.

The bridge lemmas, for example `cleared_g12345_H11_Z_bridge` (QuantumPartitionPSD_1AB.v), prove that each cleared integer, read as a real, equals the corresponding real quantity times the stated positive power of K . A $k \times k$ leading minor of a matrix scaled by $c > 0$ scales by c^k . So every integer positivity holds exactly when the real one does.

Theorem E.23 (Full 1+AB check, no trap implies PSD). `chsh_lassert_lab_g12345_no_trap_implies_npa_psd_qlab` (`kernel/quantum/QuantumPartitionPSD_1AB.v`). Let s be a state with error flag false and let s' result from `instr_chsh_lassert_lab_g12345` with declared cost d and ten bucket counts for $\gamma_1, \dots, \gamma_5$. If the program counter of s' is that of s plus one and the error flag is unchanged, then the matrix of Definition E.8 at the four state correlators and $\gamma_k = N_{g_k}/D_{g_k}$ is symmetric and PSD9.

Proof. As in Theorem E.17, no trap means the check passed. The bucket conjuncts make every denominator positive, the bridges carry the six integer positivities to the real cascade, and Lemma E.21 twice plus Lemma E.22 give $R \geq 0$, hence PSD9 by Theorem E.20. $\square$

Remark E.5 (Interior only). Every test in the cascade is strict, so a passing check certifies that H is positive definite, not merely semidefinite. A point where H is PSD but singular cannot pass, and neither can a moment with $|\gamma_k| = 1$. For example, the tally of item E with correlators $(3/5, 4/5, 4/5, -3/5)$ has $H_{11} = 1 - 9/25 - 16/25 = 0$, so the full 1+AB check traps on it for every choice of γ , though the level-1 check of item E accepts it. The source proves soundness only; no converse is stated.

A worked example of the cascade, in exact rational arithmetic (argued here, not a Coq Example). Take the correlators $(0.6, 0.6, 0.6, -0.6)$ and $\gamma = (0, 0, 0, 0, -1/2)$; as buckets, γ_1 to γ_4 are $(1, 1)$, giving $0/2$, and γ_5 is $(1, 3)$, giving $-2/4$, both inside the strict bounds. Then $H_{11} = H_{22} = 7/25$, $H_{12} = 0$, and every entry H_{1j} , H_{2j} with $j \geq 3$ is zero. So $S_{22}^{(6)} = (7/25)^2 = 49/625$, and $S^{(5)}$ is a positive multiple of the lower right block of H :

$$S^{(5)} = \frac{343}{781250} \begin{pmatrix} 32 & -18 & -18 & -7 \\ -18 & 32 & 7 & 18 \\ -18 & 7 & 32 & 18 \\ -7 & 18 & 18 & 32 \end{pmatrix},$$

where $50H_{33} = 32$, $50H_{34} = 50H_{35} = -18$, $50H_{36} = 50(-1/2 + 0.36) = -7$, $50H_{45} = 50(1/2 - 0.36) = 7$, and $50H_{46} = 50H_{56} = 18$. The leading minors of the integer matrix are 32, 700, 15000 and 140625, all positive. With $H_{11} = 7/25 > 0$ and $S_{22}^{(6)} > 0$, all six tests pass, and with item E's $A = B = 175$, $C = 0$ the instruction would not trap. At $\gamma_5 = 0$ instead, the same block becomes $I_4 - \mathbf{e}\mathbf{e}^T$ with $\mathbf{e} = (0.6, 0.6, 0.6, -0.6)$, whose third leading minor is negative ($\frac{1}{25^3}$ times -1250), and the check traps.

That last fact is general. The instruction `instr_chsh_lassert_lab` (kernel/foundation/VMStep.v) takes no γ buckets; its check `column_contractive_check_qlab_kernel` is the level-1 check together with `sum_E_sq_check_witness`, the integer form of $E_{00}^2 + E_{01}^2 + E_{10}^2 + E_{11}^2 \leq 1$ with denominators cleared by $(N_{00}N_{01}N_{10}N_{11})^2$.

Theorem E.24 (The zero-moment check sits inside the classical bound). *If `column_contractive_check_qlab w = true` (QuantumPartitionPSD_1AB.v, the same function), then the four correlators of w satisfy $E_{00}^2 + E_{01}^2 + E_{10}^2 + E_{11}^2 \leq 1$ (`qlab_check_at_gzero_forces_unit_ball`) and $|S| \leq 2$ (`qlab_check_at_gzero_implies_classical_bound`).*

Proof. At $\gamma = 0$ the residual splits into the level-1 form in (v_{B_0}, v_{B_1}) and $|\mathbf{v}|^2 - \langle \mathbf{e}, \mathbf{v} \rangle^2$ in $\mathbf{v} = (v_{00}, v_{01}, v_{10}, v_{11})$, where $\mathbf{e} = (E_{00}, E_{01}, E_{10}, E_{11})$ (`qlab_residual_g_zero_decomp`). The check gives column contractivity at $\gamma = 0$ (`column_contractive_check_qlab_sound_at_gzero`). Test at $v_{B_0} = v_{B_1} = 0$ and $\mathbf{v} = \mathbf{e}$: with $r = |\mathbf{e}|^2 \geq 0$, the residual is $r - r^2 \geq 0$, so $r \leq 1$. Then Lemma E.4 gives $S^2 \leq 4r \leq 4$. $\square$

So the zero-moment instruction certifies a strict part of the classical region: no point above $|S| = 2$, and none of the sixteen deterministic points either, since each has $|\mathbf{e}|^2 = 4$. A point above 2 needs supplied moments. `qlab_certifies_a_superclassical_chsh_point` (QuantumPartitionPSD_1AB.v) proves PSD9 at correlators $(3/5, 3/5, 3/5, -3/5)$, $\gamma = (0, 0, 0, 0, -1/2)$, with $S = 12/5$, through the γ_5 slice: its real condition `qlab_g5_caller_witness` asks $-1 < \gamma_5 < 1$ and

$$(1 - \gamma_5)[(E_{00} + E_{11})^2 + (E_{01} - E_{10})^2] + (1 + \gamma_5)[(E_{00} - E_{11})^2 + (E_{01} + E_{10})^2] \leq 2(1 - \gamma_5^2).$$

At this point the left side is $\frac{3}{2} \cdot 0 + \frac{1}{2} \cdot (\frac{36}{25} + \frac{36}{25}) = \frac{36}{25} = 1.44$ and the right side is $2 \cdot \frac{3}{4} = 1.5$. The integer version, `qlab_g5_kernel_check_accepts_superclassical`, runs the check of `instr_chsh_lassert_lab_g5` on the counters $(4, 1), (4, 1), (4, 1), (1, 4)$ and the bucket $(1, 3)$ and returns `true` by computation.

Each of the four 1+AB checks (VMStep.v) contains `column_contractive_check_witness` as a conjunct, so a passing 1+AB check also passes the level-1 check and inherits $S^2 \leq 8$ from item E. That is a composition I did by reading the definitions; the

source has no single theorem stating it. The hardware theorems of item E cover CHSH_LASSERT only.

E.7 G. The elliptope completion

Definition E.11 (Completion and membership). `completed_npa` (kernel/quantum/ElliptopeCompletion.v) is the record of item D with the within-party entries set to free reals: x in the (A_0, A_1) cells and y in the (B_0, B_1) cells, marginals still zero. The correlators are elliptope realizable (`elliptope_realizable`) when some x, y make the completed matrix symmetric and PSD.

$$M(E; x, y) = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & x & E_{00} & E_{01} \\ 0 & x & 1 & E_{10} & E_{11} \\ 0 & E_{00} & E_{10} & 1 & y \\ 0 & E_{01} & E_{11} & y & 1 \end{pmatrix}$$

Its quadratic form at $v = (v_0, \dots, v_4)$ is $\sum_k v_k^2 + 2xv_1v_2 + 2yv_3v_4 + 2E_{00}v_1v_3 + 2E_{01}v_1v_4 + 2E_{10}v_2v_3 + 2E_{11}v_2v_4$ (`completed_quad_expand`). The pinned matrix is the case $x = y = 0$, so PSD of $M(E)$ implies membership: `zero_marginal_implies_elliptope`.

Theorem E.25 (Deterministic plans are inside, `deterministic_strategy_elliptope`). (*ElliptopeCompletion.v*) If $a_0^2 = a_1^2 = b_0^2 = b_1^2 = 1$, then $(a_0b_0, a_0b_1, a_1b_0, a_1b_1)$ is elliptope realizable, with $x = a_0a_1$ and $y = b_0b_1$.

Proof. With these entries the form equals $v_0^2 + (a_0v_1 + a_1v_2 + b_0v_3 + b_1v_4)^2$. Expanding the second square gives $a_0^2v_1^2 + \dots$ on the diagonal, which is $v_1^2 + \dots$ by the unit squares, and the cross terms $2a_0a_1v_1v_2 = 2xv_1v_2$, $2a_0b_0v_1v_3 = 2E_{00}v_1v_3$, and so on for all six pairs. A sum of two squares is never negative. $\square$

The all-ones plan is the instance `classical_tightness_witness_elliptope`, and the classical mixture $(1, 0, 1, 0)$ is `turing_point_elliptope`, at $x = 1, y = 0$, with form $v_0^2 + (v_1 + v_2 + v_3)^2 + v_4^2$.

Theorem E.26 (Convexity and finite mixtures). For $0 \leq \lambda \leq 1$, if E and F are elliptope realizable, so is $\lambda E + (1 - \lambda)F$ (`elliptope_convex`, *ElliptopeCompletion.v*). For finitely many members with nonnegative weights summing to at most 1, the weighted sum is realizable (`elliptope_finite_mixture`). Every finite mixture of deterministic plans is realizable (`lhv_mixture_elliptope`).

Proof. The form is affine in (E, x, y) for each fixed v . Take the completion $(\lambda x_1 + (1 - \lambda)x_2, \lambda y_1 + (1 - \lambda)y_2)$; the form at the mixture is the same mixture of two nonnegative

forms. The finite version is induction on the number of members, with the shortfall in weight absorbed by the zero point (`elliptope_zero`). The last statement combines this with Theorem E.25. $\square$

Theorem E.27 (Tsirelson inside the completion set, `elliptope_tsirelson`). (*ElliptopeCompletion.v*) If E is elliptope realizable then $S^2 \leq 8$.

Proof. For a symmetric PSD matrix the bilinear form obeys Cauchy-Schwarz, $\mathcal{B}(u, w)^2 \leq \mathcal{Q}(u) \mathcal{Q}(w)$ (`psd_cauchy_schwarz`), because $\mathcal{Q}(u + tw) = \mathcal{Q}(u) + 2t\mathcal{B}(u, w) + t^2\mathcal{Q}(w) \geq 0$ for all t and Lemma E.11 applies. Take $u_1 = e_{A_0}$ and $w_+ = e_{B_0} + e_{B_1}$: $\mathcal{B}(u_1, w_+) = E_{00} + E_{01}$, $\mathcal{Q}(u_1) = 1$, $\mathcal{Q}(w_+) = 2 + 2y$. Take $u_2 = e_{A_1}$ and $w_- = e_{B_0} - e_{B_1}$: $\mathcal{B}(u_2, w_-) = E_{10} - E_{11}$, $\mathcal{Q}(w_-) = 2 - 2y$. So $c_1^2 \leq 2 + 2y$ and $c_2^2 \leq 2 - 2y$ with $c_1 = E_{00} + E_{01}$, $c_2 = E_{10} - E_{11}$, and both budgets are nonnegative. Then $c_1^2 c_2^2 \leq (2 + 2y)(2 - 2y) = 4 - 4y^2 \leq 4$, so $c_1 c_2 \leq 2$, and $S^2 = (c_1 + c_2)^2 = c_1^2 + 2c_1 c_2 + c_2^2 \leq (2 + 2y) + 4 + (2 - 2y) = 8$. $\square$

The completion can make a row of E longer than 1 (the all-ones plan has rows of squared length 2), which is why this proof does not use the row bounds of item D.

Corollary E.28 (The PR box is outside, `pr_box_not_elliptope`). (*ElliptopeCompletion.v*) $(1, 1, 1, -1)$ is not elliptope realizable, since its $S^2 = 16 > 8$.

Theorem E.29 (Strictly beyond the classical bound, `beyond_classical_elliptope`). (*ElliptopeCompletion.v*) $(3/5, 3/5, 3/5, -3/5)$ is elliptope realizable and has $S = 12/5 > 2$.

Proof. At $x = y = 0$ the form equals $v_0^2 + (v_1 + \frac{3}{5}v_3 + \frac{3}{5}v_4)^2 + (v_2 + \frac{3}{5}v_3 - \frac{3}{5}v_4)^2 + \frac{7}{25}v_3^2 + \frac{7}{25}v_4^2$. Check: the two middle squares give v_3^2 coefficient $\frac{9}{25} + \frac{9}{25} = \frac{18}{25}$, plus $\frac{7}{25}$ is 1; the $v_3 v_4$ terms are $+\frac{18}{25}$ and $-\frac{18}{25}$ and cancel; the cross terms are $2 \cdot \frac{3}{5}v_1 v_3$, $2 \cdot \frac{3}{5}v_1 v_4$, $2 \cdot \frac{3}{5}v_2 v_3$ and $-2 \cdot \frac{3}{5}v_2 v_4$, as the entries require. $\square$

By Theorem E.2 no finite factorizable model has $S > 2$, so this point is not a finite mixture of plans, while Theorem E.26 puts every such mixture inside. The completion set therefore strictly contains the finite classical set, and the PR box lies outside it. The source does not state that the completion set lies inside the no-signalling set.

The gate (`kernel/quantum/ElliptopeGate.v`) is an integer decider with a soundness proof. Its completion entries arrive as bucket pairs, and an empty bucket is read as 0/1 (`bden`), so unlike item E it does not require every pair to be sampled.

Definition E.12 (The two branches). `elliptope_pd_check` (*ElliptopeGate.v*): with the completion supplied as buckets and all entries scaled to integers by a positive com-

mon factor K , the four leading minors of the 4×4 block on A_0, A_1, B_0, B_1 are strictly positive. `elliptope_ldl_check`: with the completion and a rational certificate supplied (six subdiagonal entries over a common positive denominator, four pivots $d_k \geq 0$ over another), the ten entry equations of $M = LDL^T$ hold in cross-multiplied integer arithmetic (`ldl_equations`). `elliptope_check_full` is their disjunction.

Theorem E.30 (Soundness of the gate, `elliptope_check_full_sound`). (*Elliptope-Gate.v*) If `elliptope_check_full` returns `true` on a counter record, any completion buckets and any certificate, then the four correlators of the record are elliptope realizable. And `elliptope_full_gate_never_accepts_pr_box`: on the PR box tally the gate returns `false` for every choice of the remaining arguments.

Proof. The first branch reduces to Lemma E.22 through the same fraction-free identity (`elliptope_pd_check_sound`). The second is a weighted sum of squares: given the ten equations, the form is $\sum_k d_k P_k^2$ with $d_k \geq 0$ (`elliptope_ldl_check_sound`). The PR statement is soundness followed by Corollary E.28. $\square$

Computed examples, each closed by reflexivity: `gate_accepts_all_ones`, the all-ones plan, through the second branch at $x = y = 1$; `gate_accepts_turing_point`; `gate_accepts_beyond_classical`, the 12/5 point, through the first branch at $x = y = 0$; `gate_accepts_pythagorean_boundary`, the point $(3/5, 4/5, 4/5, -3/5)$, through the second branch with pivots $(1, 1, 0, 0)$.

The gate is not an instruction. Neither `VMStep.v` nor `VMState.v` mentions it, and the file's own header says that binding it into the step relation, a cost rule and a hardware mirror is not done.

E.8 H. What is not proved

The following are cited results or open, and none of them is a Coq theorem in this development.

The arcsine boundary. For two settings and two outcomes per side, the correlators reachable by quantum measurements are characterized by $|\arcsin E_{00} + \arcsin E_{01} + \arcsin E_{10} - \arcsin E_{11}| \leq \pi$ and its sign variants (Tsirelson, Landau 1988, Masanes 2003). No arcsine appears in any Coq statement; the only mentions are comments in `ElliptopeGate.v`. The claim that $(3/5, 4/5, 4/5, -3/5)$ lies on that boundary is such a comment: $\arcsin \frac{3}{5} + \arcsin \frac{4}{5} = \frac{\pi}{2}$ by the 3-4-5 triangle, so the sum is π . It is correct arithmetic about a cited characterization.

Equality with the quantum set. That the completion set of item G is exactly the set of quantum correlators is Tsirelson's representation theorem, cited in the header of `ElliptopeCompletion.v` and not proved. That PSD of any of these matrices is necessary for quantum behaviour is the NPA reading, also cited.

Achievability. No state and no pair of measurements is constructed. Theorem E.6 is a point of $\mathbb{R}^4$, not an experiment.

Completeness of the checks. Every check here is proved sound, never complete. The 1+AB cascade is strict and misses boundary points. The comment in `ElliptopeGate.v` that the second branch accepts every point with a rational PSD completion is a mathematical remark about $LDL^\top$ factorizations, not a theorem in the file; nor is there a theorem that the four 1+AB checks accept nested sets.

Finite levels. For full behaviours, marginals included, the NPA levels are relaxations. Chaturvedi (arXiv 2607.14569, 2026) shows that no finite NPA level characterizes the complete quantum set even in this simplest Bell scenario.

Statistics and physics. Nothing here says how many rounds make a tally informative, and nothing says the counters came from honest play; a program chooses its own trial inputs.

E.9 I. Prior work

Tsirelson proved the bound $2\sqrt{2}$ for quantum correlations in 1980 and showed that quantum two-outcome correlators are inner products of real unit vectors. Landau (1988) and Masanes (2003, arXiv quant-ph/0309137) gave the exact characterization for two settings and two outcomes. Wehner (2006, arXiv quant-ph/0510076) proved Tsirelson bounds, CHSH among them, by a semidefinite program on the 4×4 Gram matrix of those vectors. Navascués, Pironio and Acín (2007, arXiv quant-ph/0607119; 2008, arXiv 0803.4290) built the hierarchy of moment-matrix tests; their 2008 paper applies a Schur complement to this level-1 matrix, recovers the arcsine condition, and discusses the 1+AB level. Formal proofs of Tsirelson’s bound already exist, by Echenim and Mhalla in Isabelle/HOL (arXiv 2306.12535) and in Lean’s mathlib (tsirelson_inequality in Mathlib.Algebra.Star.CHSH), and exact rational sum-of-squares certificates go back at least to Peyrl and Parrilo (2008). So the moment-matrix route to $2\sqrt{2}$ is standard. What is specific here is narrower: an integer certificate computed from outcome counters and checked inside one instruction, a mechanized chain from “the instruction did not trap” to PSD of the pinned matrix and on to $S^2 \leq 8$, and a hardware circuit proved equal to that integer check for every choice of 32-bit counters. I did not find that chain elsewhere, and I make no claim beyond it.

F Hardware retirement: the assumptions in full

The word *admitted* in Section 27 names a set of inputs on which the retirement theorem applies. It is not permission to hide the inputs. This appendix spells out that set. All the conditions here are conditions on the state before the instruction starts. A bound in one row is not automatically a hypothesis of another.

F.1 Common notation and conditions

Write $W = 2^{32}$. A hardware word denotes its unsigned value from 0 to $W - 1$; word addition and subtraction wrap modulo W . All inequalities below concern the unsigned values as natural numbers, before any wrap. Write r_j for register j , H for the heap base, q for the program counter, and c for the declared cost decoded from the instruction. Register indices are four bits, module indices six, and data-memory addresses seven. Thus taking an address means reducing it modulo 128. The stack pointer is register 15.

Every case requires the fetched word to encode the indicated instruction and its operands, the alarm to be down, the machine to be live, and $q + 1 < W$. Here *live* means not halted, error false, and the logic, game-check and coupling state machines all idle. The alarm tests whether the ledger is below the sum of the tensor entries. The ledger requirement is $\mu + \Delta < W$, with Δ given in each row. Even a jump or a halting instruction has the printed $q + 1 < W$ premise: these are the sufficient conditions of the theorem, not a claim that each one is necessary.

For a module m , write b_m and s_m for its stored base and size. An address x is *in the active range* when $b_m \leq x < b_m + s_m$ for the active module m . The *trap agreement* condition says that the board's trap-vector word equals the fixed LASSERT trap address in Section 17. The partition-operation and MDL-operation counters are diagnostic fields; call them P and D .

For the arrow tables, let N_M, N_D, N_P be the next allocation indices for arrows, coupling descriptors and coupling pairs. A descriptor records a base and count in the pair table, together with a label length and label word. Each table also marks which entries are valid. The following abbreviations state complete predicates:

Name	Condition
V	Every valid arrow index is less than N_M .
R	Every valid arrow refers either to descriptor 0, or to a valid descriptor whose index is less than N_D .
Z	Descriptor 0 is marked invalid.
E	Descriptor 0 has base 0 and count 0.
B	For every valid descriptor, its base plus its count is at most N_P .
Pairs	Every pair-table index below N_P is marked valid.
Labels	Every valid descriptor has label length L with $1 \leq L \leq 32$ and label word less than 2^L .
Disjoint	The regions of distinct modules in the snapshot graph are disjoint.

An invalid descriptor has effective label length 1 and effective label word 1. An arrow's

coupling list is the slice at its descriptor's base and count. For composition, the output list is empty if both arrows are identities, the other arrow's list if exactly one is an identity, and the relational join otherwise: enumerate a pair (x, z) for each (x, y) in the first list and (y, z) in the second. Repeated pairs count as repeated entries when measuring the room required.

F.2 Single-step cases

Operands below mean the decoded operands used in the instruction table of Section 17. Ordinary and extended arrow encodings are separate admission cases; where a row names both, its conditions apply to each encoding.

Instruction or family	Increment Δ and additional conditions
ADD	$\Delta = c; r_{s_1} + r_{s_2} < W.$
SUB	$\Delta = c; r_{s_2} \leq r_{s_1}.$
MUL	$\Delta = c; r_{s_1} r_{s_2} < W.$
AND, OR, XOR_ADD, XFER, LOAD_IMM, LUI, XOR_LOAD, JUMP, JNEZ, XOR_SWAP, TENSOR_SET, TENSOR_GET, XOR_RANK, HALT, CHECKPOINT, WRITE_PORT	$\Delta = c$; no additional condition.
SHL	$\Delta = c$; shift amount $r_{s_2} < 32$ and $r_{s_1} 2^{r_{s_2}} < W.$
SHR	$\Delta = c$; shift amount $r_{s_2} < 64.$
MDLACC	$\Delta = c; D + 1 < W.$
LJOIN, CERTIFY	$\Delta = c + 1$; no additional condition.
READ_PORT	$\Delta = n + c + 1$, with the encoded bit count n ; the compared abstract instruction supplies value 0. Its channel parameter is arbitrary.
EMIT	$\Delta = n + c + 1$; the abstract payload has bit length equal to the encoded n , and the information-gain counter plus n is less than $W.$
REVEAL	$\Delta = n + c + 1$; the selected tensor entry plus n is less than $W.$ The certificate string is arbitrary.
PDISCOVER	$\Delta = c$; the encoding's evidence byte is zero. The abstract evidence list is arbitrary.
LOAD	$\Delta = c; r_{\text{address}} \bmod 128$ is in the active range.
STORE	$\Delta = c; r_{\text{address}} \bmod 128$ is in the active range.
HEAP_LOAD, HEAP_STORE	$\Delta = c; H + r_{\text{address}} < W$, and $(H + r_{\text{address}}) \bmod 128$ is in the active range.
CALL	$\Delta = c$; the stack pointer p satisfies $p < 128$ and $p + 1 < W$; $p \bmod 128$ is in the active range.
RET	$\Delta = c; 1 \leq p \leq 128$ and $(p - 1) \bmod 128$ is in the active range.

Instruction or family	Increment Δ and additional conditions
PNEW	$\Delta = c$; the normalized region's length and first element equal the encoded size and base (first element 0 for an empty region); $P + 1 < W$; trap agreement.
PSPLIT	$\Delta = c$; the next module index is at most 64; $P + 1 < W$; the selected module's $b_m + s_m < W$; trap agreement. The abstract left and right lists are arbitrary.
PMERGE	$\Delta = c$; $P + 1 < W$; the two selected module sizes sum to less than W ; trap agreement.
CHSH_TRIAL	$\Delta = c$; each of the eight outcome counters, including those not selected on this round, is strictly below $W - 1$.
MORPH_DELETE, ordinary and extended	$\Delta = c$; V .
MORPH_ID, ordinary and extended	$\Delta = c$; $N_M < 16$.
MORPH_ASSERT, ordinary	$\Delta = c + 1$; V ; the property string's character-code sum is 0. The certificate string is arbitrary.
MORPH_ASSERT, extended	$\Delta = c + 1$; V ; the property string's character-code sum equals the unsigned extension word. The certificate string is arbitrary.
MORPH_GET, ordinary	$\Delta = c$; V .
MORPH_GET, extended	$\Delta = c$; V , R and Z .
MORPH_TENSOR, ordinary and extended	$\Delta = c$; Disjoint. The two abstract arrow operands are arbitrary.

The arbitrary operands are deliberate limitations of these correspondences. If a snapshot omits an abstract field, agreement of snapshots does not recover it. In particular, a result for an arbitrary evidence list does not show that the board stored or checked that list. Section 27 separates snapshot agreement from a statement about the entire kernel state.

F.3 Memory-reading and checking cases

The remaining seven cases are the two extended MORPH paths, the two extended COMPOSE paths, the two LASSERT kinds, and CHSH_LASSERT. They carry the common conditions above and the following additional ones.

MORPH, missing-module path. The increment is c . Require $N_M < 16$ and $N_D < 16$, and at least one of the selected source and target modules has size zero. This is a fault retirement, not a successful arrow construction.

COMPOSE, invalid-input path. The increment is c . Require V and $N_M < 16$, and either one of the two arrow slots is marked invalid or the first arrow's target differs from the second arrow's source. This too is a fault retirement.

MORPH, pair-loading path. The increment is c . Require $N_M < 16$, $N_D < 16$, and both selected modules to have nonzero size. Let a be the seven-bit address decoded for the coupling buffer and let n be its first memory word. Require $N_P < 16$, $n + N_P \leq 16$, $a + 2n \leq 127$, and $a + 1 + 2n < 128$. The word at $a + 1 + 2n$ must be zero. The n pairs stored in the next $2n$ words must each have their first coordinate in the source region and their second coordinate in the target region. Thus neither a count nor an endpoint is accepted merely because the buffer fits.

COMPOSE, pair-building path. The increment is c . Require $N_M < 16$, $N_D < 16$, $N_P < 16$, and V, R, Z, E, B, Pairs and Labels. Both input arrow slots must be valid and their middle module must match. Their effective label lengths must sum to at most 32. Finally, N_P plus the length of the output coupling list defined above must be at most 16.

LASSERT, unsatisfiable kind. The increment is $8\ell + c + 1$. Require trap agreement and the formula's memory header to equal the declared length ℓ . This case retires through the unsupported-unsatisfiability failure described in Section 17; it does not establish an unsatisfiability checker.

LASSERT, satisfiable kind. The increment is $8\ell + c + 1$. Require trap agreement, the memory header at the formula address a to equal ℓ , and $a + 3 + \ell < W$. The clause count stored at $a + 2$ must be at least one and at most the number of zero terminator words in the literal sequence. These are the conditions of Theorem 27.3 and Lemma 27.4.

CHSH_LASSERT. The increment is $c + 1$. Require trap agreement. The arithmetic check itself may pass or fail; retirement covers either result. Its complete integer predicate and its mathematical consequence are in Section E.

F.4 What the catalogue establishes

These are fifty-five admission cases, counting the separately named encodings and fault paths. They are sufficient hypotheses for the retirement theorem. They do not assert that a program can never leave this set, that a failed premise makes the hardware incorrect, or that all abstract instructions have an encoding.

The proof divides the cases by what can change. For an ordinary arithmetic or data-moving instruction, evaluate the register writes and the program-counter and ledger updates, then use the bounds to replace word arithmetic by the natural-number arithmetic of the specified instruction. For memory and partition instructions, the active-range and representation conditions make their guards and selected addresses agree. For arrow operations, the validity, descriptor and capacity conditions make the table lookup or update represent the stated graph operation. A fault path instead proves agreement on its specified refusal.

A multi-cycle instruction also needs an invariant through its internal firings: the scan

has processed exactly the prefix already visited, the partial arithmetic equals the corresponding expression on the saved operands, and unrelated state fields are unchanged. The scan and game-check arguments are given in Section 27. The coupling scan loads the next pair, checks its two region memberships and advances its count; composition enumerates the finite join defined above, with the identity cases separated. The room and address conditions keep each write within its intended table. At completion, the accumulated list is the specified one. This is how the per-instruction arguments enter the master case split; a list of theorem names alone would not supply that explanation.

F.5 Reading a snapshot as a kernel state

Two readings occur in Section 27. They copy the same scalar state: program counter, ledger, error flag, logic accumulator, status, certification flag, four control registers and eight witness counters. They turn register entries 0–15, memory entries 0–127 and tensor entries 0–15 into lists in increasing index order. The board’s halted flag and diagnostic counters have no separate kernel-state fields.

The partition-only reading, abs_0 , reconstructs modules by visiting indices $N - 1, N - 2, \dots, 0$, where N is the next module index. It omits a module whose size is zero. Each remaining module gets the interval of addresses from its base through base plus size minus one, an empty axiom list and a zero tensor. The graph’s next module index is N , its arrow list is empty and its next arrow index is one. This is the reading used in the checking-step comparison.

The full reading, abs , uses the same modules and intervals, but gives each module its sixteen stored tensor entries. It visits arrow indices in descending order below N_M , omitting invalid slots. A valid entry supplies source, target and identity flag. Its descriptor supplies the coupling label and the list of pairs in its stored range, read in increasing offset order and skipping absent pairs. An absent descriptor supplies an empty pair list and the default empty-coupling label. Each reconstructed arrow has certification cost zero. The graph’s next arrow index is N_M . This is the reading in Theorem 27.7. The theorem compares the reconstructed state; it does not recover evidence absent from the snapshot.

F.6 The full-state bridge conditions

The full-state bridge and clocked retirement have different hypotheses. The following predicates complete the definition of WF in Theorem 27.7. They concern the snapshot’s natural-number fields and optional table entries, before any extra bounds required by the clocked board.

A *well-formed partition table* has $1 \leq N \leq 64$. A *well-formed arrow table* has no valid arrow at an index at least N_M . The *empty-descriptor condition* says that descriptor zero is absent. The *extended invariant* consists of these three conditions together with $N_D > 0$ and all of the following: every valid arrow names a descriptor index below N_D ; every present descriptor’s base plus count is at most N_P ; and every slot in every present

descriptor's range holds a pair. It does not require every index below N_D to hold a descriptor.

The *stack condition* is $p < 128$ and $(p - 1) \bmod 2^{64} < 128$, where $p = r_{15}$ and the subtraction in that second expression is subtraction modulo 2^{64} . Since $p < 128$, this is equivalent to $1 \leq p < 128$. A *clear tensor slot* has every stored tensor entry zero, including entries beyond the sixteen that the reading retains.

Instruction	Additional condition for WF
CALL	Stack condition and program counter below 128.
RET	Stack condition.
CHSH_TRIAL	Each of its four supplied operands is 0 or 1.
LASSERT	Declared length equals the formula memory header.
TENSOR_SET	Both tensor indices are below 4.
TENSOR_GET	Both tensor indices are below 4, and the selected module exists in the reconstructed graph.
MORPH_DELETE, MORPH_ASSERT	Well-formed arrow table.
MORPH_GET, COMPOSE, MORPH_TENSOR	Extended invariant.
MORPH	Extended invariant; source and target module indices are below N and each has positive size.
MORPH_ID	Empty-descriptor condition; selected module index below N with positive size.
PNEW	$N \leq 64$, normalized input region nonempty, and tensor slot N clear.
PSPLIT	Well-formed partition table; selected index reduced modulo 64 is below N and has size at least 2; sizes at N and $N + 1$ are zero and both tensor slots are clear.
PMERGE	Well-formed partition table; both indices reduced modulo 64 are below N , distinct and of positive size; size at N is zero and its tensor slot is clear.
Every other instruction	True.

These are the conditions on one step. The run theorem asks for the appropriate row again at every fetched instruction; it does not infer their preservation from reset.

G Constructing the quoting operation

The proof in Section 11 uses a program that takes the code of a term and returns the code of that code. Here is its construction. Nothing in the operation lets a program inspect itself by magic: its input is an ordinary tree encoded as a closed lambda term.

G.1 Numerals and syntax as data

For readability, the formulas below give names to bound variables. Renaming a bound variable changes nothing; the underlying syntax uses the de Bruijn indices of Definition 11.3. Applications associate to the left. Write $X \Downarrow Y$ here for a finite sequence of the weak call-by-value steps ending in the value Y .

Scott numerals are the following closed values:

$$\bar{0} = \lambda z. \lambda s. z, \quad \overline{n+1} = \lambda z. \lambda s. s \bar{n}.$$

Thus a numeral selects either the supplied zero case or the supplied successor case, passing its predecessor to the latter. The cases below are values, so choosing one does not prematurely run the other.

The three syntax constructors, operating on already encoded children, are

$$\begin{aligned} V &= \lambda n. \lambda v. \lambda a. \lambda l. v \bar{n}, \\ A &= \lambda p. \lambda q. \lambda v. \lambda a. \lambda l. a \text{ } p \text{ } q, \\ L &= \lambda p. \lambda v. \lambda a. \lambda l. l \text{ } p. \end{aligned}$$

For a variable of index n , an application and an abstraction, respectively, define their codes by the following resulting values:

$$\begin{aligned} \ulcorner \#n \urcorner &= \lambda v. \lambda a. \lambda l. v \bar{n}, \\ \ulcorner p \text{ } q \urcorner &= \lambda v. \lambda a. \lambda l. a \text{ } \ulcorner p \urcorner \ulcorner q \urcorner, \\ \ulcorner \lambda p \urcorner &= \lambda v. \lambda a. \lambda l. l \text{ } \ulcorner p \urcorner. \end{aligned}$$

In particular, $V \bar{n}$, $A \text{ } \ulcorner p \urcorner \ulcorner q \urcorner$ and $L \text{ } \ulcorner p \urcorner$ reduce to these three codes. A code is closed even when the term it describes has a free variable: the index of that variable is stored as a numeral. It is data inside the code, not a free variable of the code.

G.2 Recursion without running under a lambda

The usual self-application trick has to delay its recursive call in a call-by-value language. Use

$$W = \lambda x. \lambda f. \lambda v. f (\lambda a. x x f a) v, \quad \text{rec}(F) = \lambda v. W W F v.$$

When F and u are closed values, beta reduction gives

$$\text{rec}(F) u \longrightarrow W W F u \longrightarrow^* F (\lambda a. W W F a) u = F (\text{rec}(F)) u.$$

The recursive argument is a lambda, hence already a value. It starts work only when F applies it to an argument. That is the whole reason this construction survives the call-by-value rule.

G.3 Quoting numerals

Let $E_j = \ulcorner \#j \urcorner$ for $j = 0, 1, 2$, and let $Z = \ulcorner \bar{0} \urcorner$, obtained directly from the finite syntax of $\bar{0} = \lambda\lambda.1$. Write $L^k(X)$ for k nested applications of the syntax builder L to X ; it is an abbreviation for constructing k lambda nodes, not a new operation of the language.

Define

$$F_N = \lambda r. \lambda n. n Z (\lambda k. L^2(A E_0 (r k))), \quad Q_N = \text{rec}(F_N).$$

Then $Q_N \bar{n} \Downarrow \ulcorner \bar{n} \urcorner$, by induction on n . At zero the Scott numeral selects Z . At a successor it passes $\bar{n}$ to the second case; the induction hypothesis supplies the code of $\bar{n}$, and the builders produce the code of $\lambda\lambda.(\#0 \bar{n})$, which is exactly the syntax of $\overline{n+1}$.

G.4 Quoting terms

For a proposed recursive call r , define three handlers:

$$\begin{aligned} H_V &= \lambda n. L^3(A E_2(Q_N n)), \\ H_A(r) &= \lambda p. \lambda q. L^3(A(A E_1(r p))(r q)), \\ H_L(r) &= \lambda p. L^3(A E_0(r p)). \end{aligned}$$

Now set

$$F_Q = \lambda r. \lambda t. t H_V(H_A(r))(H_L(r)), \quad Q = \text{rec}(F_Q).$$

The handlers are lambdas and so are values. The Scott code of the input chooses one of them and hands it the encoded children. Induction on the represented term proves $Q \ulcorner t \urcorner \Downarrow \ulcorner \ulcorner t \urcorner \urcorner$:

- For $t = \#n$, the first handler constructs three lambda nodes over an application of variable 2 to the quoted numeral. That is the syntax of $\lambda\lambda\lambda.(\#2 \bar{n})$, the code of $\#n$.
- For $t = p q$, the induction hypotheses quote both child codes. The second handler constructs three lambdas over variable 1 applied to those two codes. That is the syntax of the application code.
- For $t = \lambda p$, the last handler constructs three lambdas over variable 0 applied to the quoted child code. That is the syntax of the abstraction code.

Each recursive call works on a proper child of the finite input tree, or on a smaller numeral. Each builder takes finitely many beta steps. The induction therefore establishes termination as well as the value returned.

The `mkapp` used in Section 11 is the builder A here. With Q and that builder now explicit, its self-referential term has every piece on the page. The companion names are `mk_app_enc`, `rec_spec`, `Qn_spec` and `Q_spec` (`LRecursion.v`); the construction above is the argument those names identify.

H The two fixed interpreters, in full

Section 24 uses two fixed programs. Their code is part of the construction, so here it is. Each listing runs on the unbounded instruction semantics in Section 17. Sixteen registers hold natural numbers. Subtraction stops at zero. Left shift by b multiplies by 2^b ; right shift divides by 2^b and discards the remainder. AND and OR act on binary digits.

LOAD_IMM takes a destination and a constant; XFER takes a destination and a source. Arithmetic and bit instructions take a destination followed by two source-register indices. JUMP takes an absolute instruction address; JNEZ takes a source-register index and an absolute address. A taken jump replaces the program counter; otherwise it advances by one. All listed instructions have declared cost zero, omitted from the operands. Fetching beyond the listing ends the host run.

H.1 Packing the program

For words $a_0, \dots, a_{n-1}$ choose a positive width w with $a_j < 2^w$ for every j . Such a width exists for every finite list. Store

$$C = \sum_{j=0}^{n-1} a_j 2^{wj}, \quad \text{fetch}(C, w, p) = \lfloor C / 2^{wp} \rfloor \bmod 2^w.$$

Fetch is a right shift followed by an AND with $2^w - 1$. Repeated division by 2^w proves that it returns a_p for $p < n$ and zero for $p \geq n$. The width condition prevents a word spilling into its neighbour. Code and width are inputs, so the host listing stays fixed when the guest program grows.

H.2 The four-register guest

The twelve-instruction guest of Definition 24.4 encodes an instruction as

$$o + 16d + 64s_1 + 256s_2 + 1024c + 262144v,$$

where o is the opcode, $d, s_1, s_2 < 4$ are register indices, $c < 256$ is its declared cost and v is an immediate natural number. Unused fields are zero. These fields occupy bits 0–3, 4–5, 6–7, 8–9, 10–17 and 18 upward.

Opcode	Instruction	Fields used besides cost
1	HALT	None
2	LOAD_IMM	d, v
3	XFER	d, s_1
4, 5, 6	ADD, SUB, MUL	d, s_1, s_2
7, 8	AND, OR	d, s_1, s_2
9, 10	SHL, SHR	d, s_1, s_2
11	JUMP	v
12	JNEZ	s_1, v

Here HALT charges its cost and advances the guest program counter, as the kernel instruction does. The guest ends when its program counter leaves the program. Every genuine guest word has a nonzero opcode; zero is the end sentinel from an out-of-range fetch. Opcodes 13–15 are malformed input. The two-counter guest below has a different halt instruction.

Host registers 0–3 hold guest registers; 11 and 12 hold the guest ledger and program counter; 13 and 14 hold width and code. Register 9 holds status: zero running, one terminated, two malformed. Registers 4–8, 10 and 15 are scratch. Begin at host address zero with running status. Other host fields remain unchanged. The guest ledger in register 11 is separate from the host ledger.

Address	Instruction	Operands
0	XFER	7,12
1	MUL	7,7,13
2	SHR	8,14,7
3	LOAD_IMM	10,1
4	SHL	7,10,13
5	SUB	7,7,10
6	AND	8,8,7
7	LOAD_IMM	7,6
8	SHR	7,8,7
9	LOAD_IMM	15,3
10	AND	7,7,15
11	JNEZ	7,14
12	XFER	4,0
13	JUMP	23
14	SUB	7,7,10
15	JNEZ	7,18
16	XFER	4,1
17	JUMP	23
18	SUB	7,7,10
19	JNEZ	7,22
20	XFER	4,2
21	JUMP	23
22	XFER	4,3
23	LOAD_IMM	7,8
24	SHR	7,8,7
25	LOAD_IMM	15,3
26	AND	7,7,15
27	JNEZ	7,30
28	XFER	5,0
29	JUMP	39
30	SUB	7,7,10
31	JNEZ	7,34
32	XFER	5,1
33	JUMP	39

Address	Instruction	Operands
34	SUB	7,7,10
35	JNEZ	7,38
36	XFER	5,2
37	JUMP	39
38	XFER	5,3
39	LOAD_IMM	7,15
40	AND	7,8,7
41	JNEZ	7,44
42	LOAD_IMM	9,1
43	JUMP	122
44	SUB	7,7,10
45	JNEZ	7,47
46	JUMP	115
47	SUB	7,7,10
48	JNEZ	7,52
49	LOAD_IMM	7,18
50	SHR	6,8,7
51	JUMP	99
52	SUB	7,7,10
53	JNEZ	7,56
54	XFER	6,4
55	JUMP	99
56	SUB	7,7,10
57	JNEZ	7,60
58	ADD	6,4,5
59	JUMP	99
60	SUB	7,7,10
61	JNEZ	7,64
62	SUB	6,4,5
63	JUMP	99
64	SUB	7,7,10
65	JNEZ	7,68
66	MUL	6,4,5
67	JUMP	99
68	SUB	7,7,10
69	JNEZ	7,72
70	AND	6,4,5
71	JUMP	99
72	SUB	7,7,10
73	JNEZ	7,76
74	OR	6,4,5
75	JUMP	99
76	SUB	7,7,10
77	JNEZ	7,80
78	SHL	6,4,5
79	JUMP	99
80	SUB	7,7,10

Address	Instruction	Operands
81	JNEZ	7,84
82	SHR	6,4,5
83	JUMP	99
84	SUB	7,7,10
85	JNEZ	7,89
86	LOAD_IMM	7,18
87	SHR	12,8,7
88	JUMP	116
89	SUB	7,7,10
90	JNEZ	7,97
91	JNEZ	4,94
92	ADD	12,12,10
93	JUMP	116
94	LOAD_IMM	7,18
95	SHR	12,8,7
96	JUMP	116
97	LOAD_IMM	9,2
98	JUMP	122
99	LOAD_IMM	7,4
100	SHR	7,8,7
101	LOAD_IMM	15,3
102	AND	7,7,15
103	JNEZ	7,106
104	XFER	0,6
105	JUMP	115
106	SUB	7,7,10
107	JNEZ	7,110
108	XFER	1,6
109	JUMP	115
110	SUB	7,7,10
111	JNEZ	7,114
112	XFER	2,6
113	JUMP	115
114	XFER	3,6
115	ADD	12,12,10
116	LOAD_IMM	7,10
117	SHR	7,8,7
118	LOAD_IMM	15,255
119	AND	7,7,15
120	ADD	11,11,7
121	JUMP	0

To verify one iteration, split it at its branch destinations. Addresses 0–6 fetch the word; 7–22 select the first source register and 23–38 the second. Each two-bit index selects one of registers 0–3. Addresses 39–40 extract the opcode. Zero takes 41–43 to address

122 with status one. Successive subtractions at 44, 47, 52, 56, 60, 64, 68, 72, 76, 80, 84 and 89 dispatch the twelve genuine opcodes. Arithmetic results go to register 6; 99–114 writes that result to the destination. A jump writes register 12 directly and skips its ordinary increment at 115. Addresses 116–120 extract and add the guest cost; 121 returns to address zero.

This proves the block lemma in Theorem 24.7 by direct evaluation: a guest step has a finite, positive host block with the same new guest fields. No valid opcode takes the malformed exit at 97–98. The displayed branches also rule out an exit partway through an ordinary block. Induction on completed blocks supplies both directions of the correspondence, with a final sentinel block for termination.

H.3 The two-counter guest

Word codes are 0 for halt, 1 for increment counter zero, 2 for increment counter one, $3 + 8t$ for decrement counter zero and jump to t , and $4 + 8t$ for the same operation on counter one. A zero counter stays zero and falls through; a positive counter is decremented and takes the jump. Halt has no successor. The same packing formula applies. Fetching outside the program returns zero and therefore halts too.

Registers 11 and 10 hold counters zero and one, register 12 the guest program counter, and 13 and 14 width and code. Register 9 has the three status values above. Registers 0–8 and 15 are scratch. At the canonical initial boundary the scratch registers, status and host program counter are zero. The host ledger and all non-register fields are preserved.

Address	Instruction	Operands
0	XFER	0, 12
1	MUL	1, 0, 13
2	SHR	5, 14, 1
3	LOAD_IMM	2, 1
4	SHL	3, 2, 13
5	SUB	3, 3, 2
6	AND	5, 5, 3
7	LOAD_IMM	4, 7
8	AND	8, 5, 4
9	JNEZ	8, 12
10	LOAD_IMM	9, 1
11	JUMP	60
12	LOAD_IMM	6, 1
13	SUB	7, 8, 6
14	JNEZ	7, 18
15	ADD	11, 11, 6
16	ADD	12, 12, 6
17	JUMP	0
18	LOAD_IMM	6, 2
19	SUB	7, 8, 6

Address	Instruction	Operands
20	JNEZ	7,25
21	LOAD_IMM	6,1
22	ADD	10,10,6
23	ADD	12,12,6
24	JUMP	0
25	LOAD_IMM	6,3
26	SUB	7,8,6
27	JNEZ	7,36
28	SHR	7,5,6
29	JNEZ	11,32
30	ADD	12,12,2
31	JUMP	0
32	LOAD_IMM	6,1
33	SUB	11,11,6
34	XFER	12,7
35	JUMP	0
36	LOAD_IMM	6,4
37	SUB	7,8,6
38	JNEZ	7,58
39	LOAD_IMM	6,3
40	SHR	7,5,6
41	JNEZ	10,44
42	ADD	12,12,2
43	JUMP	0
44	LOAD_IMM	6,1
45	SUB	10,10,6
46	XFER	12,7
47	JUMP	0
48	LOAD_IMM	0,0
49	LOAD_IMM	1,0
50	LOAD_IMM	2,0
51	LOAD_IMM	3,0
52	LOAD_IMM	4,0
53	LOAD_IMM	5,0
54	LOAD_IMM	6,0
55	LOAD_IMM	7,0
56	LOAD_IMM	8,0
57	JUMP	0
58	LOAD_IMM	9,2
59	JUMP	60

Addresses 0–8 fetch the word and extract its low three bits. Opcode zero takes 9–11 to the end with halted status. Addresses 12–17 increment counter zero; 18–24 increment counter one. Addresses 25–35 perform the first decrement: 30–31 is the zero branch, 32–35 the positive branch. Addresses 36–47 do the same for the second counter. Each

successful block returns to address zero with the specified guest update. Width, code and non-register fields do not change.

Opcodes 5–7 reach the malformed exit at 58–59. The cleanup block 48–57 is part of the fixed listing, but no dispatch path from address zero enters it. Correctness at successive boundaries allows arbitrary scratch contents.

Every non-halting guest step therefore has a finite, positive host block. The halt block ends the host run. Induction over these blocks proves simulation, and reading a host run block by block gives the converse. The canonical encoding uses only opcodes 0–4, so an encoded guest cannot take the malformed exit. This supplies the construction behind the two-counter correspondence in Section 24.

I Sources and mathematical dependencies

These works identify the principal mathematical dependencies and the intellectual background named in the book. The definitions, statements and worked examples remain on its pages. A published result about thermodynamic cost, quantum correlations or universal computation is not an additional theorem proved by this manuscript.

The guest recursion construction uses effective computability and counter-machine simulation. Its diagonal step and cost restoration are given in Section 24; the counter and lambda references identify the established machinery that makes its evaluator executable. The thermodynamic references do not remove the explicit physical premise of the heat bound. The environmental-record papers motivate Section 13; they do not prove this book’s pointer conjecture.

I.1 Structure, information and knowledge

Euclid. Elements, Books I and X. Definitions, postulates, common notions and commensurability.

Plato. Timaeus; Theaetetus. The receptacle and the discussion of knowledge as true judgement with an account. Philosophical antecedents, not premises of the formal results.

Aristotle. Physics II.1; Metaphysics VII.3. Matter, form and substance.

Gottlob Frege (1884). Die Grundlagen der Arithmetik, section 64. Directions and parallel lines.

Emmy Noether (1918). Invariante Variationsprobleme. Nachrichten von der Gesellschaft der Wissenschaften zu Göttingen, Mathematisch-Physikalische Klasse, pp. 235–257. [Original paper](#), [EuDML 59024](#).

Samuel Eilenberg and Saunders Mac Lane (1945). General Theory of Natural Equivalences. *Transactions of the American Mathematical Society* 58(2), pp. 231–294. [Original paper](#).

Jorma Rissanen (1978). Modeling by shortest data description. *Automatica* 14(5), pp. 465–471. DOI: [10.1016/0005-1098\(78\)90005-5](#).

Ralph V. L. Hartley (1928). Transmission of Information. *Bell System Technical Journal* 7(3), pp. 535–563. DOI: [10.1002/j.1538-7305.1928.tb01236.x](#).

Claude E. Shannon (1948). A Mathematical Theory of Communication. *Bell System Technical Journal* 27, pp. 379–423 and 623–656. [Bell Labs publication record](#).

Edmund L. Gettier (1963). Is Justified True Belief Knowledge? *Analysis* 23(6), pp. 121–123. JSTOR: [3326923](#).

I.2 Cost, records and thermodynamics

Robert Endre Tarjan (1985). Amortized Computational Complexity. *SIAM Journal on Algebraic and Discrete Methods* 6(2), pp. 306–318. DOI: [10.1137/0606031](#).

Yue Niu, Jonathan Sterling, Harrison Grodin and Robert Harper (2022). A Cost-Aware Logical Framework. *Proceedings of the ACM on Programming Languages* 6(POPL), article 9. DOI: [10.1145/3498670](#).

Rolf Landauer (1961). Irreversibility and Heat Generation in the Computing Process. *IBM Journal of Research and Development* 5(3), pp. 183–191. DOI: [10.1147/rd.53.0183](#).

Charles H. Bennett (1973). Logical Reversibility of Computation. *IBM Journal of Research and Development* 17(6), pp. 525–532. [Original paper](#).

Charles H. Bennett (2003). Notes on Landauer’s principle, reversible computation, and Maxwell’s Demon. *Studies in History and Philosophy of Modern Physics* 34(3), pp. 501–510. [arXiv: physics/0210005v2](#); DOI: [10.1016/S1355-2198\(03\)00039-X](#).

Takahiro Sagawa (2014). Thermodynamic and Logical Reversibilities Revisited. *Journal of Statistical Mechanics: Theory and Experiment*, P03025. [arXiv: 1311.1886v2](#).

Leslie Lamport (1983). Specifying Concurrent Program Modules. *ACM Transactions on Programming Languages and Systems* 5(2), pp. 190–222. [Author publication record](#).

Vitalik Buterin and Virgil Griffith (2017). Casper the Friendly Finality Gadget. [arXiv: 1710.09437v1](#).

Ben Laurie, Eran Messeri and Rob Stradling (2021). Certificate Transparency Version 2.0. RFC 9162. [RFC 9162](#); DOI: [10.17487/RFC9162](#).

I.3 Computation, self-reference and checking

Alan M. Turing (1936–1937). On Computable Numbers, with an Application to the Entscheidungsproblem. Proceedings of the London Mathematical Society, series 2, volume 42, pp. 230–265. DOI: [10.1112/plms/s2-42.1.230](#).

Stephen Cole Kleene (1952). Introduction to Metamathematics. North-Holland and D. Van Nostrand. Partial recursive functions, parameterisation and recursion theorems.

Henry Gordon Rice (1953). Classes of Recursively Enumerable Sets and Their Decision Problems. Transactions of the American Mathematical Society 74(2), pp. 358–366. DOI: [10.1090/S0002-9947-1953-0053041-6](#).

Marvin L. Minsky (1961). Recursive Unsolvability of Post’s Problem of Tag and Other Topics in Theory of Turing Machines. Annals of Mathematics 74(3), pp. 437–455. [Original paper](#); DOI: [10.2307/1970290](#).

Stephen A. Cook and Robert A. Reckhow (1973). Time bounded random access machines. Journal of Computer and System Sciences 7(4), pp. 354–375. DOI: [10.1016/S0022-0000\(73\)80029-7](#).

Yiannis N. Moschovakis (2009). Kleene’s Amazing Second Recursion Theorem. Computer Science Logic, LNCS 5771, pp. 24–39. [Author lecture text](#).

Yannick Forster and Gert Smolka (2017). Weak Call-by-Value Lambda Calculus as a Model of Computation in Coq. Interactive Theorem Proving, ITP 2017. [Author paper](#).

Yannick Forster, Dominik Kirst and Gert Smolka (2019). On Synthetic Undecidability in Coq, with an Application to the Entscheidungsproblem. Certified Programs and Proofs, CPP 2019. [Conference publication record](#).

Andrej Dudenhefner (2022). Certified Decision Procedures for Two-Counter Machines. FSCD 2022, LIPIcs 228, article 16. The successful-decrement jump convention used in the counter guest. DOI: [10.4230/LIPIcs.FSCD.2022.16](#).

Gabriel C. Necula (1997). Proof-Carrying Code. Proceedings of POPL 1997, pp. 106–119. [Author publication list](#).

Joonwon Choi, Muralidaran Vijayaraghavan, Benjamin Sherman, Adam Chlipala and Arvind (2017). Kami: A Platform for High-Level Parametric Hardware Specification and Its Modular Verification. Proceedings of the ACM on Programming Languages 1(ICFP), article 24. DOI: [10.1145/3110268](#).

I.4 Correlations and environmental records

John F. Clauser, Michael A. Horne, Abner Shimony and Richard A. Holt (1969). Proposed Experiment to Test Local Hidden-Variable Theories. *Physical Review Letters* 23, pp. 880–884. DOI: [10.1103/PhysRevLett.23.880](https://doi.org/10.1103/PhysRevLett.23.880).

Boris S. Cirel'son (Tsirelson) (1980). Quantum generalizations of Bell's inequality. *Letters in Mathematical Physics* 4, pp. 93–100. DOI: [10.1007/BF00417500](https://doi.org/10.1007/BF00417500).

Arthur Fine (1982). Hidden Variables, Joint Probability, and the Bell Inequalities. *Physical Review Letters* 48, pp. 291–295. DOI: [10.1103/PhysRevLett.48.291](https://doi.org/10.1103/PhysRevLett.48.291).

Lluís Masanes (2003). Necessary and sufficient condition for quantum-generated correlations. *arXiv: quant-ph/0309137v1*.

Miguel Navascués, Stefano Pironio and Antonio Acín (2007). Bounding the Set of Quantum Correlations. *Physical Review Letters* 98, 010401. DOI: [10.1103/PhysRevLett.98.010401](https://doi.org/10.1103/PhysRevLett.98.010401); *arXiv: quant-ph/0607119v2*.

Miguel Navascués, Stefano Pironio and Antonio Acín (2008). A convergent hierarchy of semidefinite programs characterizing the set of quantum correlations. *New Journal of Physics* 10, 073013. *arXiv: 0803.4290v1*.

Harold Ollivier, David Poulin and Wojciech H. Zurek (2004). Objective Properties from Subjective Quantum States: Environment as a Witness. *Physical Review Letters* 93, 220401. *arXiv: quant-ph/0307229v2*.

Wojciech H. Zurek (2009). Quantum Darwinism. *Nature Physics* 5, pp. 181–188. DOI: [10.1038/nphys1202](https://doi.org/10.1038/nphys1202); *arXiv: 0903.5082v1*.